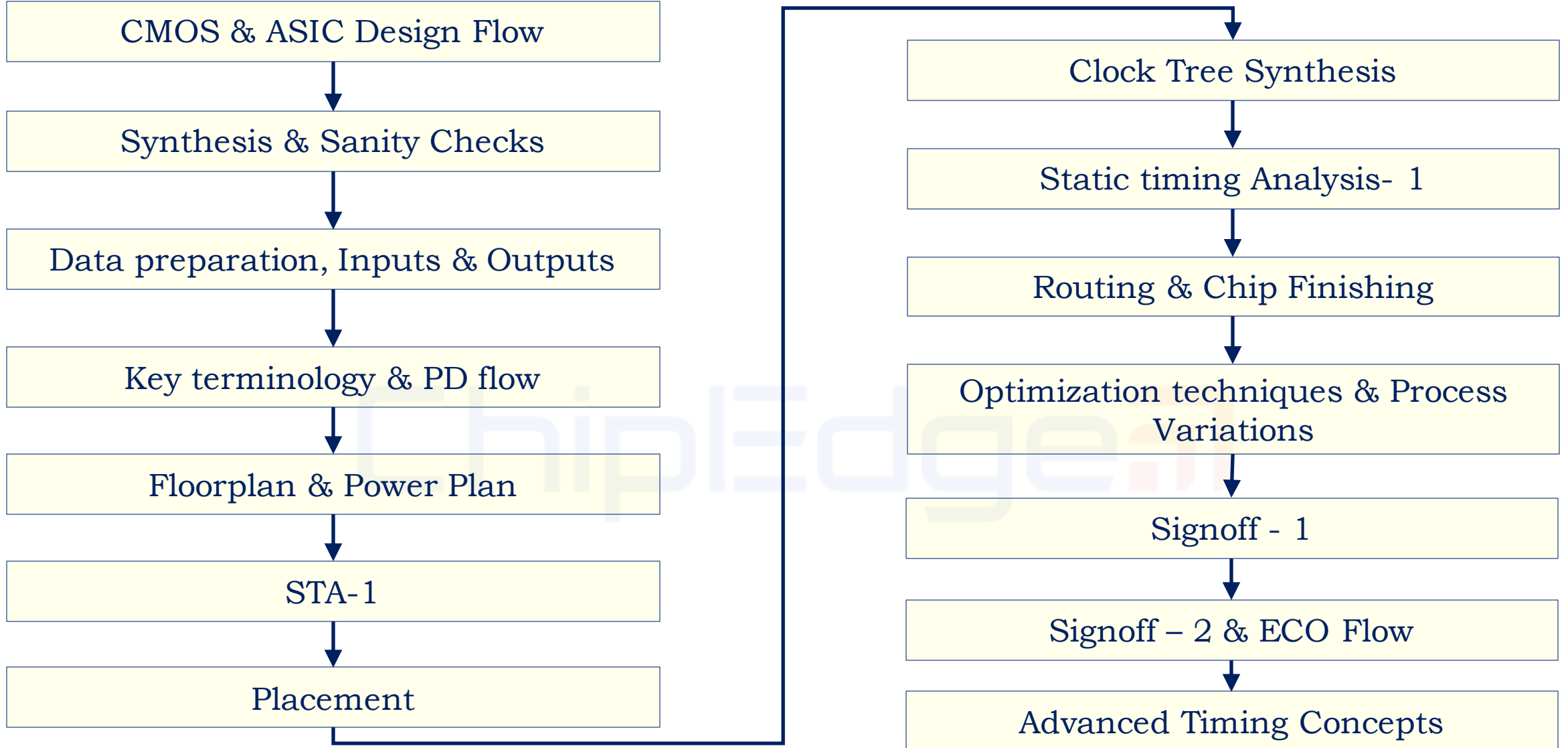
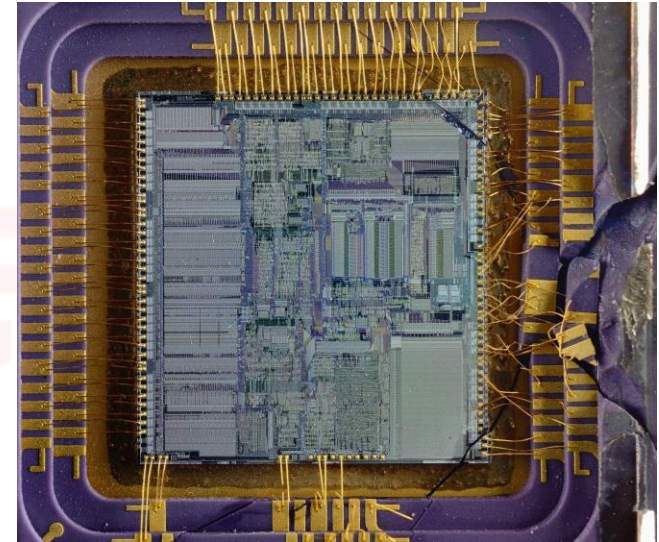
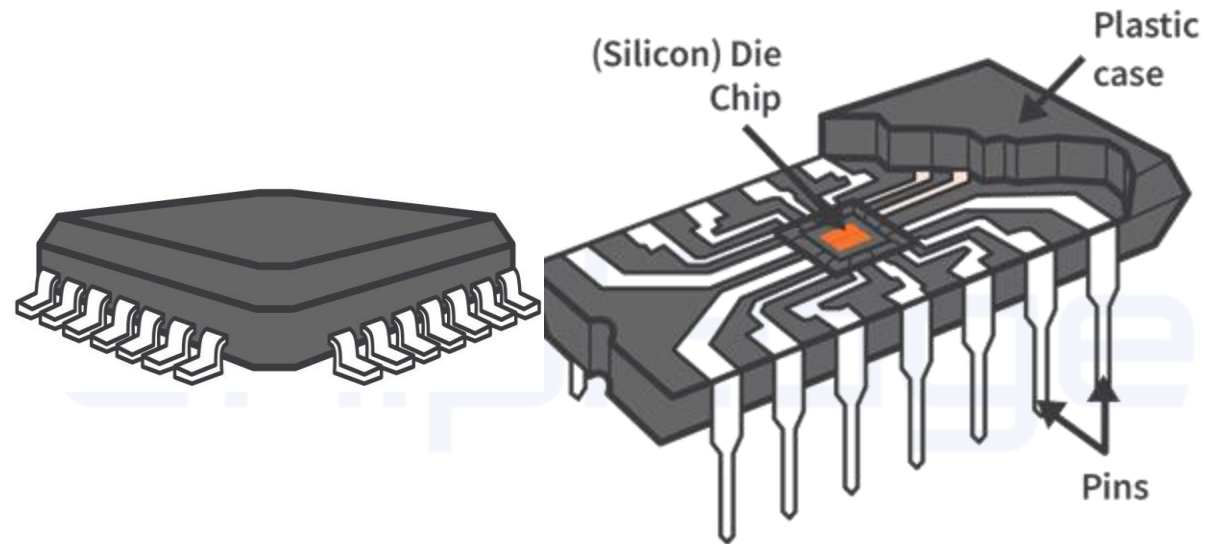
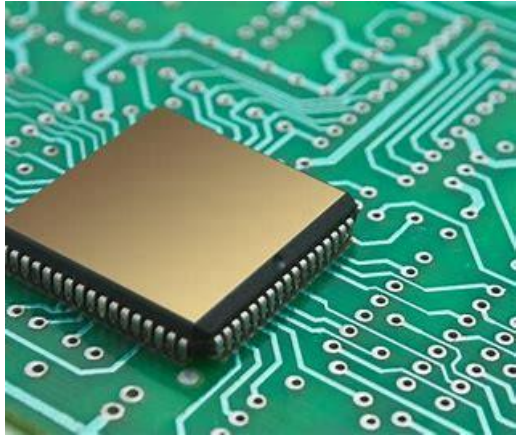


Module 1

CMOS & ASIC Design



What is PD



- In this module, you will learn about overview and operation of NMOS, PMOS and CMOS.
- Brief overview of Technology Nodes & Trends is explained in this section
- The overview of ASIC Design : Ecosystem and Design Specification are addressed.
- This module Covers overview of Physical design flow.

- Introduction to MOS
 - ✓ NMOS
 - ✓ PMOS
 - ✓ CMOS
- Technology Nodes & Trends
- ASIC Design : Ecosystem
- Design Specification
- Synthesis
- Physical Design Flow
- Post GDS

Introduction to MOS

This section explains the overview of MOSFET, NMOS, PMOS and CMOS.

Technology Nodes & Trends



ASIC Design : Ecosystem



Design Specification



Synthesis



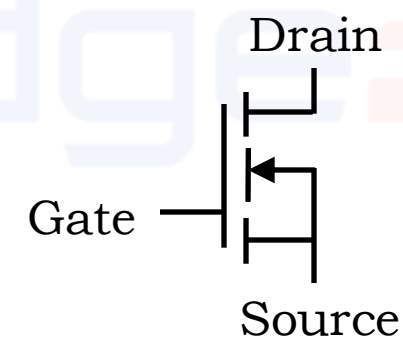
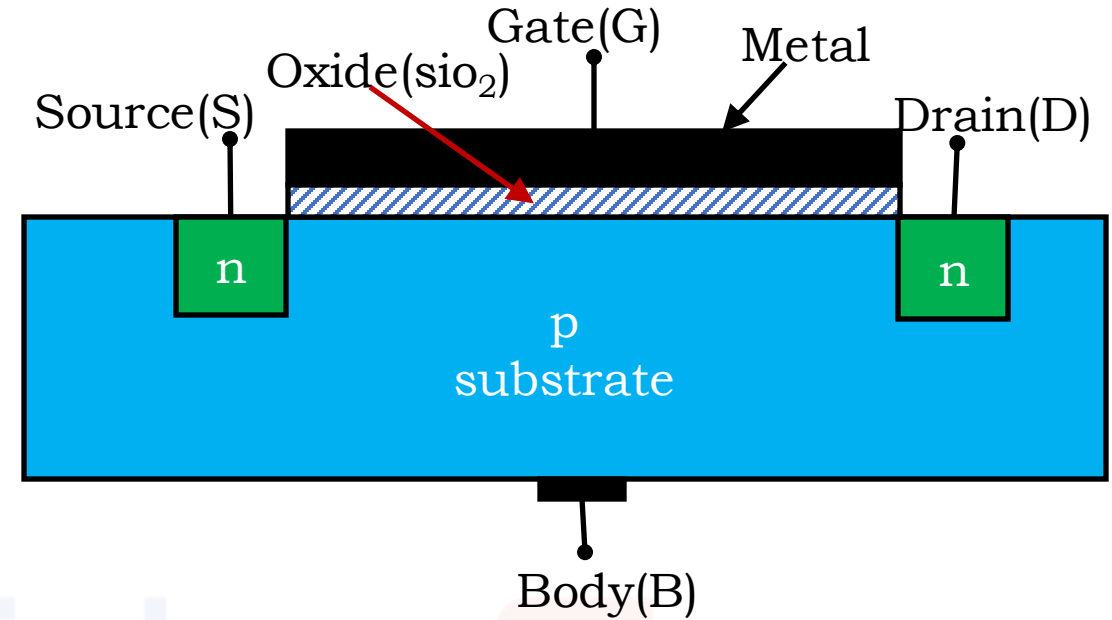
Physical Design Flow



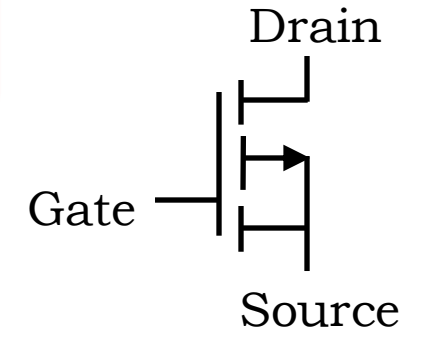
Post GDS



- Metal Oxide Semiconductor Field Effect Transistors.
- The transistor is switched on/off by the signal to gate.
- Switch on/off -> current flowing from source to drain.



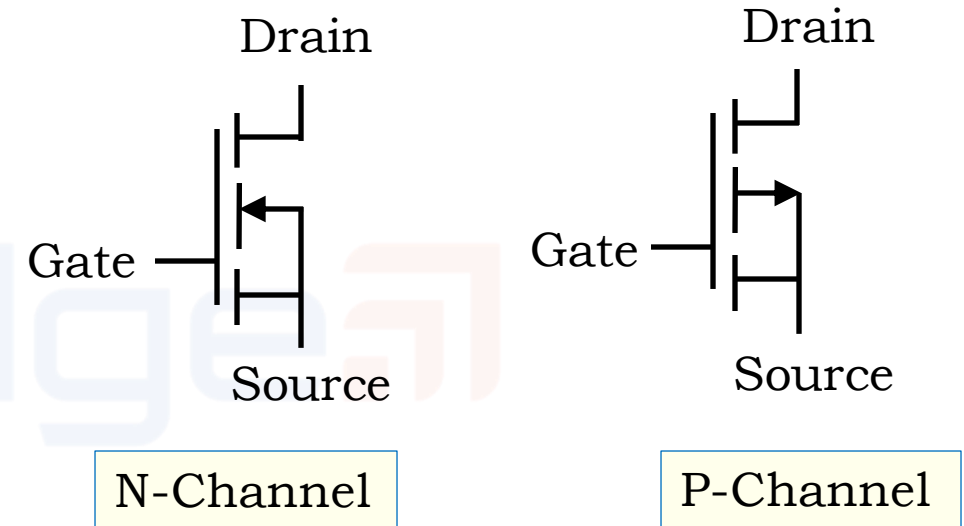
N-Channel



P-Channel

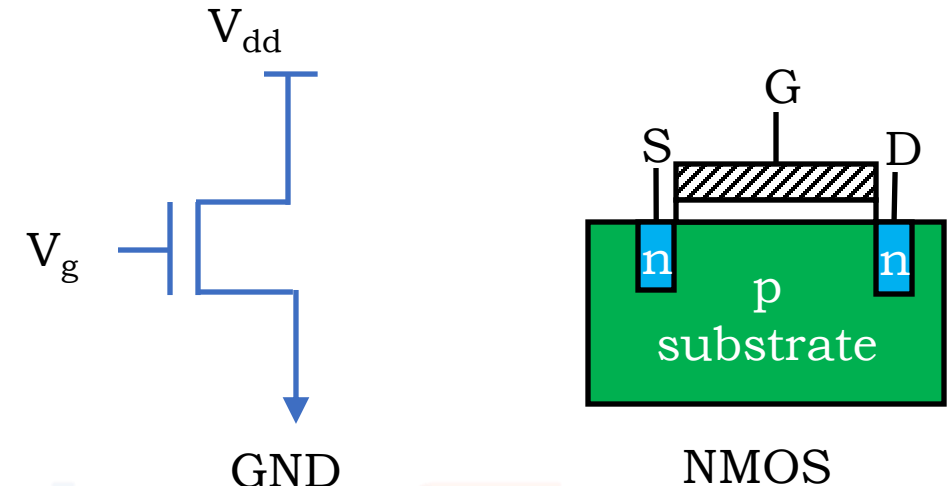
- The structure of the device - a sandwich of a metal conductor, an oxide insulator, and a semiconductor substrate.
- MOSFET is a four-terminal device.

- Source (S)
- Gate (G)
- Drain (D)
- Body (B)



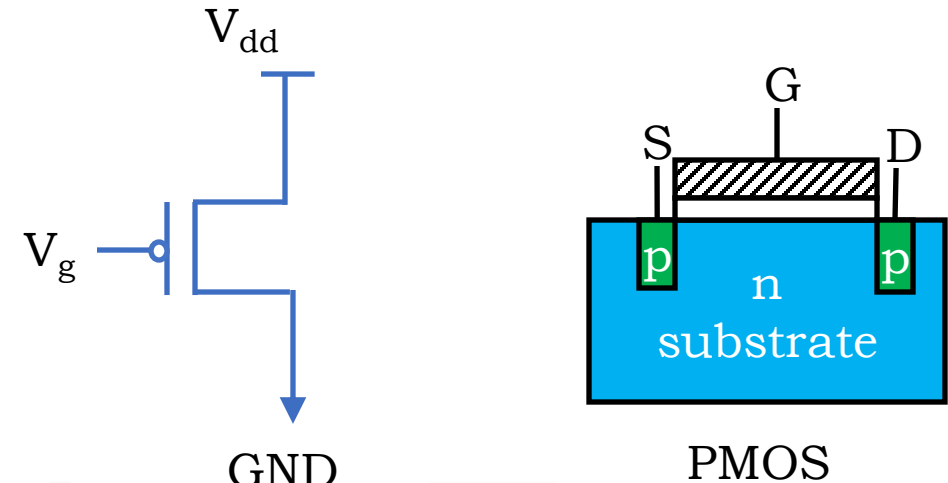
- $I_{ds} = \mu C_{ox} \frac{W}{L} * (V_{gs} - V_{th})^2 / 2$

- Drain is at a higher voltage than Source.
- Source is usually tied to ground.
- When the gate is at a low voltage No current flows, transistor is OFF.
- When the gate is at high voltage Current flows Transistor is ON.
- $V_{gs} > V_{th}$ (NMOS-ON).



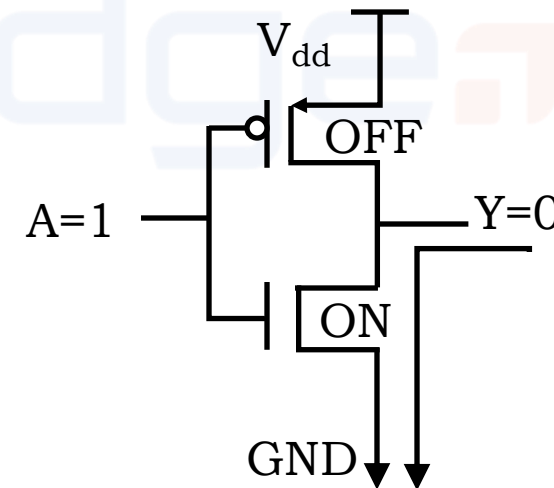
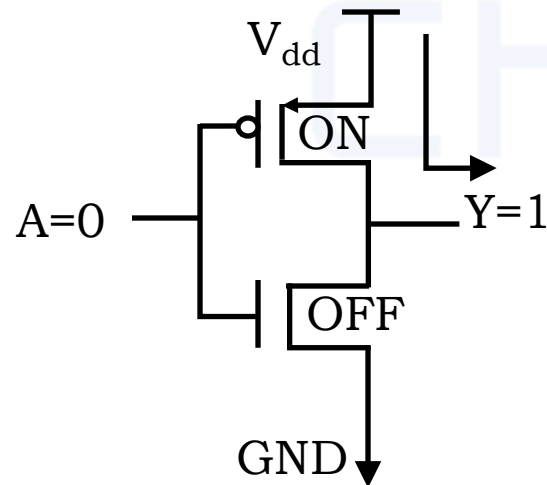
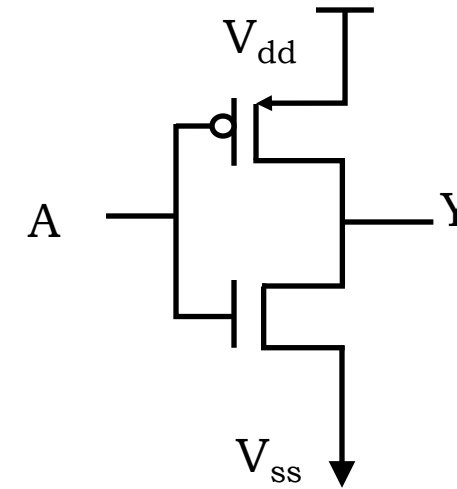
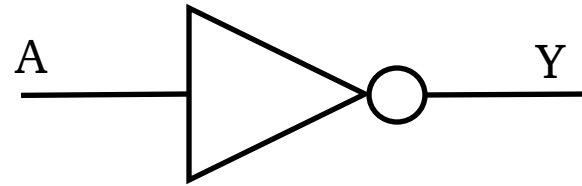
V_g	NMOS
0	OFF
1	ON

- Source is usually tied to V_{dd} .
- When the gate is at a High voltage No current flows, transistor is OFF.
- When the gate is at low voltage Current flows Transistor is ON.
- V_{th} is negative for PMOS.
- $V_{gs} < V_{th}$ (PMOS-ON).

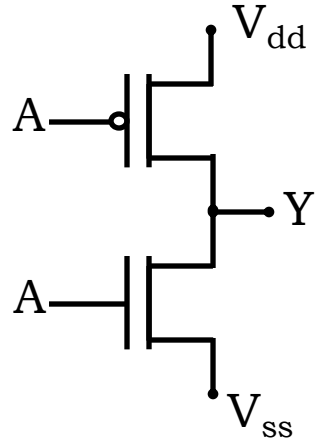


V_g	PMOS
1	OFF
0	ON

A	Y
0	1
1	0

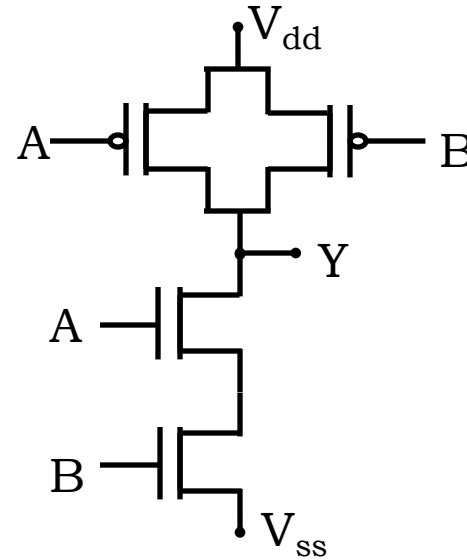


Series and Parallel MOSFET Networks



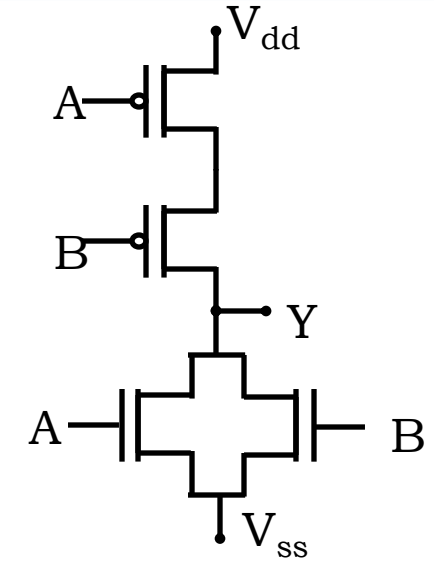
NOT

A	Y
0	1
1	0



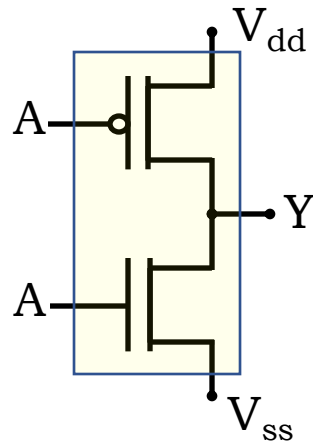
NAND

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

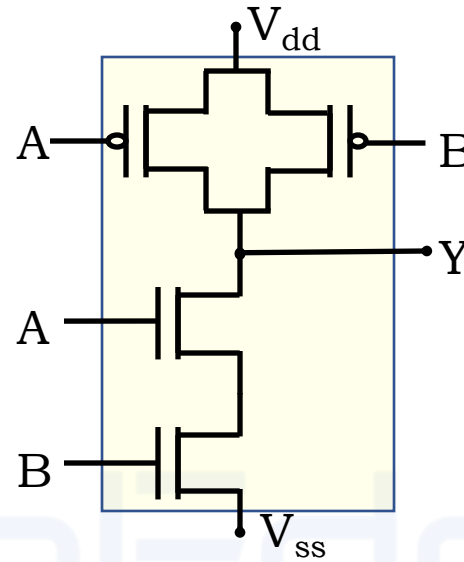


NOR

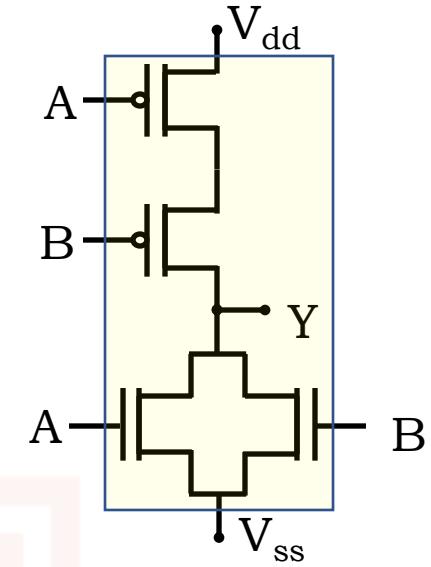
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0



NOT

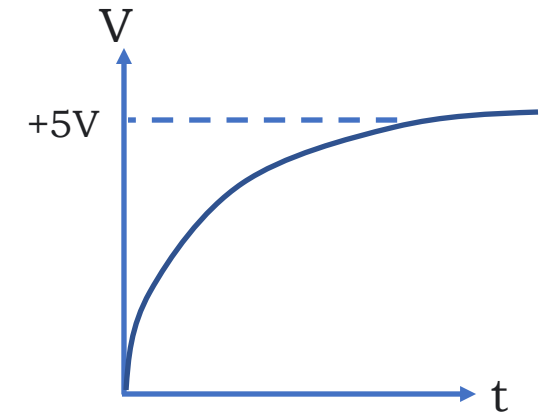
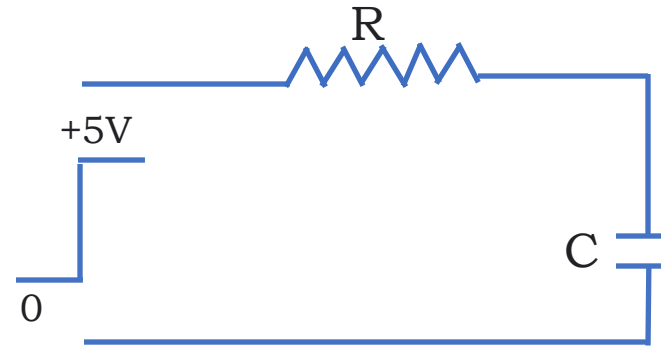


NAND

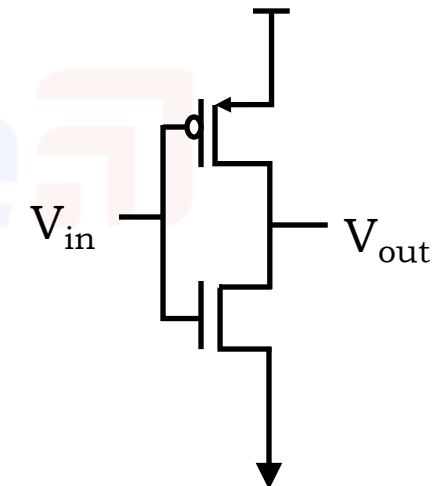


NOR

- Delay is $\propto$ to RC
- Delay = $K \cdot RC$



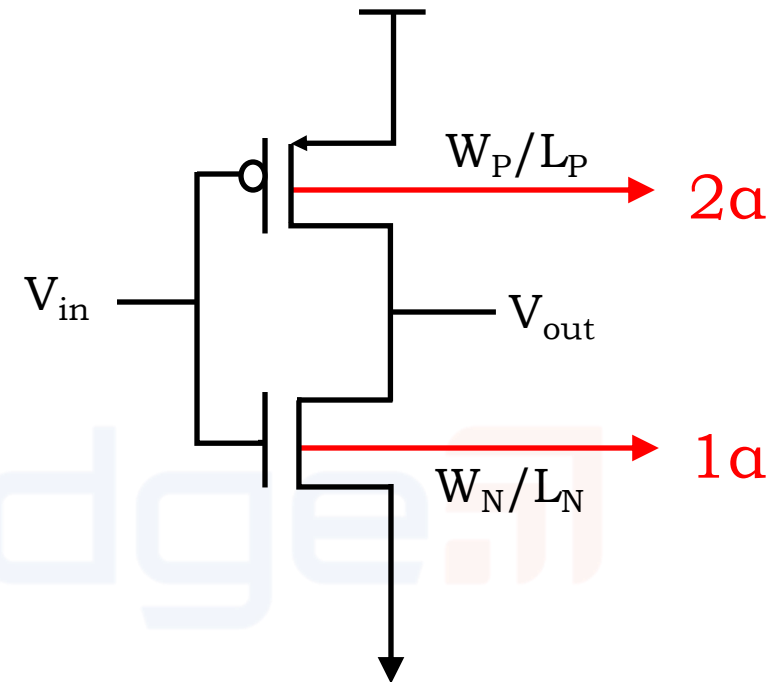
	IDS	Delay
M(μ)	↑	↓
W	↑	↓
L	↓	↑
V _{gs} (V _{dd})	↑	↓
V _{th}	↓	↑



$$I_{ds} = \mu C_{ox} \frac{W}{L} * (V_{gs} - V_{th})^2 / 2$$

Let's make the following assumptions

1. All transistors are minimum length.
2. All gates should have equal rise/fall times. Since PMOS are twice as slow as NMOS they must be twice as wide to have the same effective resistance.
3. Normalize all transistor widths to minimum width NMOS.



Introduction to MOS



Technology Nodes & Trends

This section explains about the Technology Nodes and Impact of Technology Shrinking

ASIC Design : Ecosystem



Design Specification



Synthesis

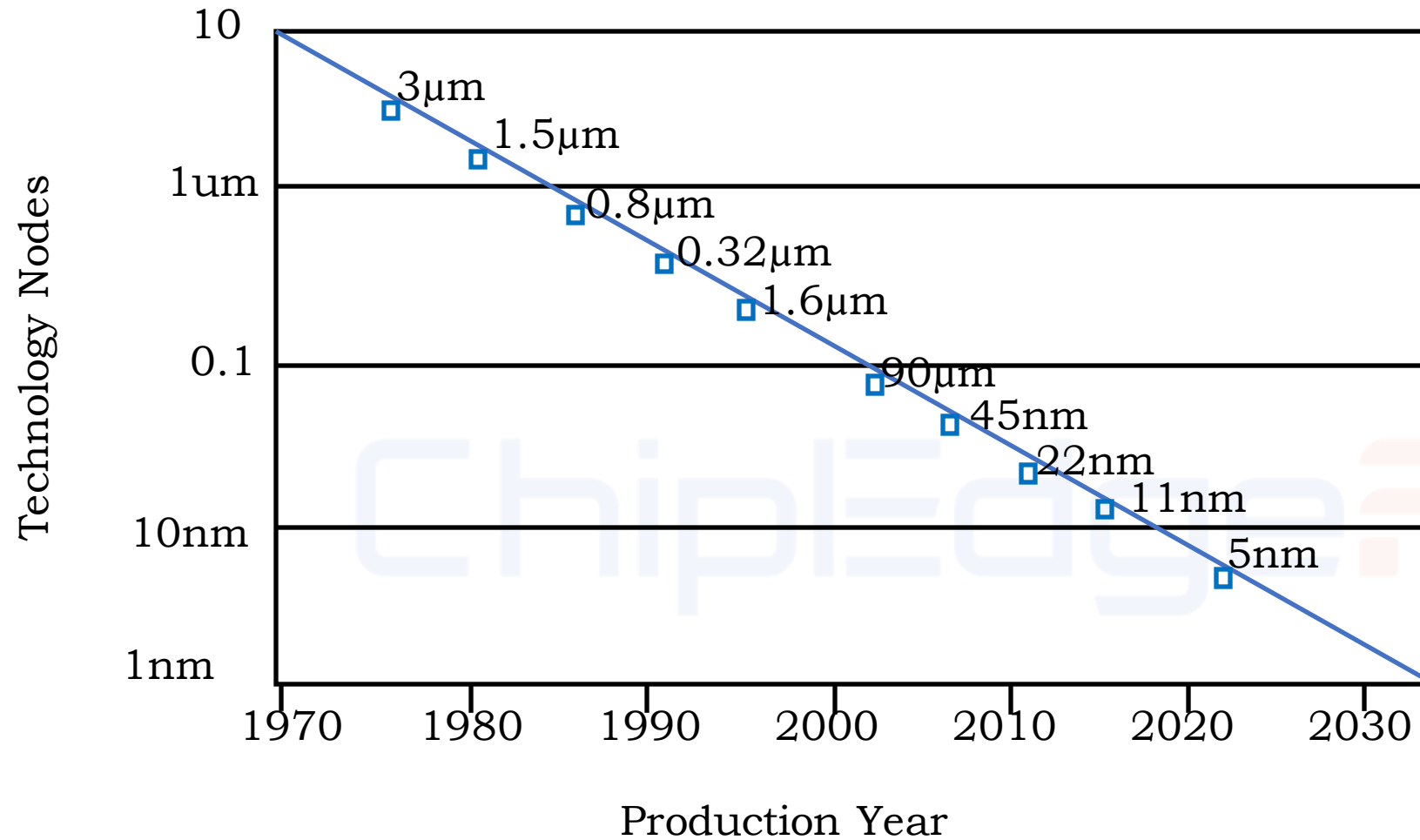


Physical Design Flow



Post GDS

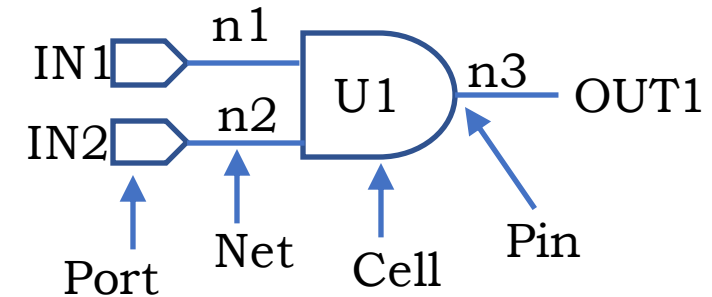


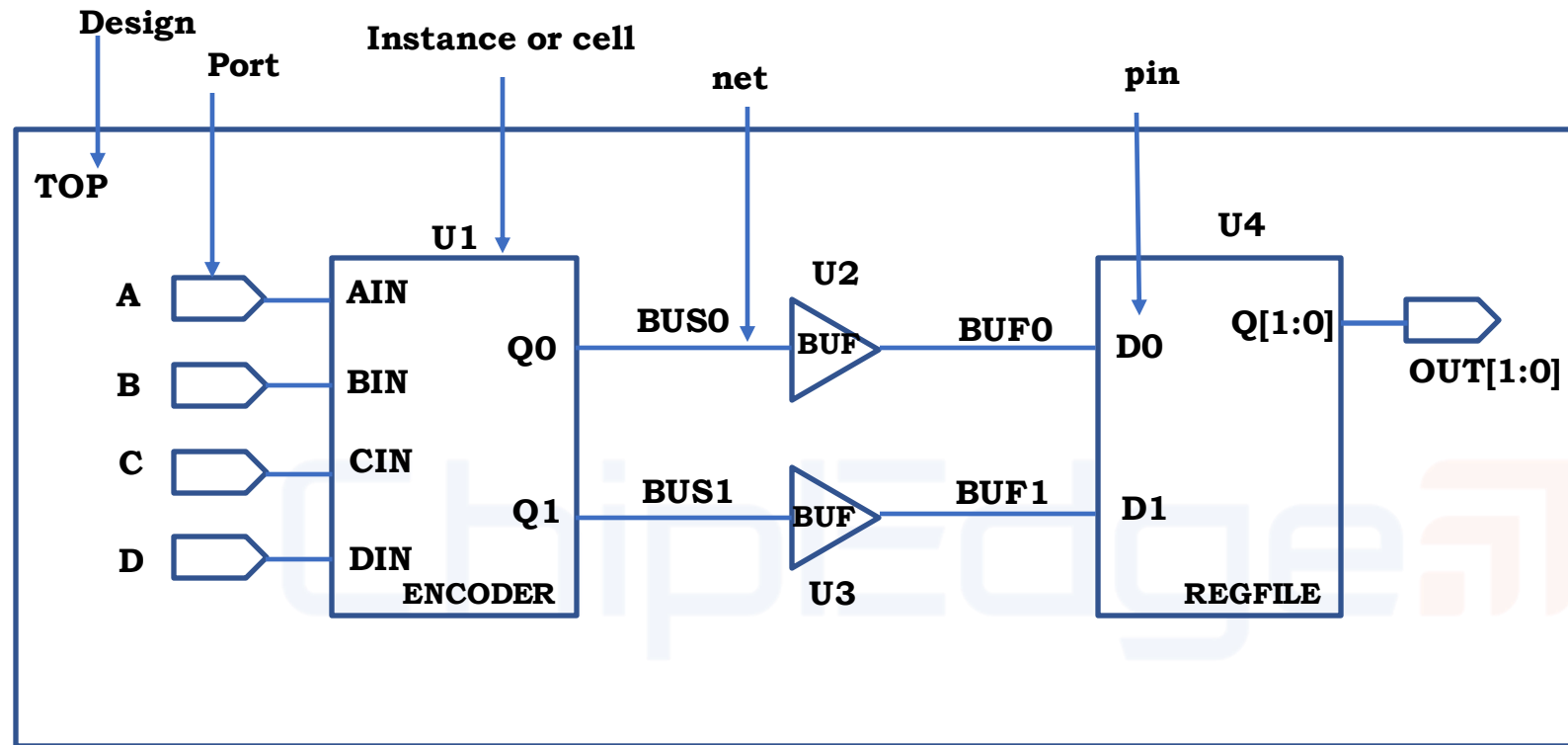


- Less Transistor area 😊
 - ✓ More logic can fit in same die area
 - ✓ More dies per wafer: cheaper per-die cost
- Faster transistors 😊
 - ✓ Shorter Channel Length
 - ✓ Reduced V_{dd} , Shorter “Voltage Swing”
- Increased Current Densities 😞
 - ✓ Require bigger metal to handle currents

- Increased Power Densities 😞
 - ✓ Require better cooling systems to dissipate heat
- Higher Leakage current 😞
 - ✓ Continuous current drain, not good for battery operated devices
 - ✓ Limiting factor for using advanced technologies

- **Design:** A circuit that performs one or more logical functions.
- **Cell:** An instance of a design.
- **Port:** The input or output of a design.
- **Pin:** The input or output of a cell.
- **Net:** A wire that connected between ports or between pins or between ports to pins.
- **Clock:** A timing reference object to describe a waveform for timing analysis.





Design {TOP ENCODER REGFILE}
Reference {ENCODER REGFILE BUF}
Instance {U1 U2 U3 U4}

Introduction to MOS



Technology Nodes & Trends



ASIC Design : Ecosystem

This section explains the ASIC Design Flow

Design Specification



Synthesis

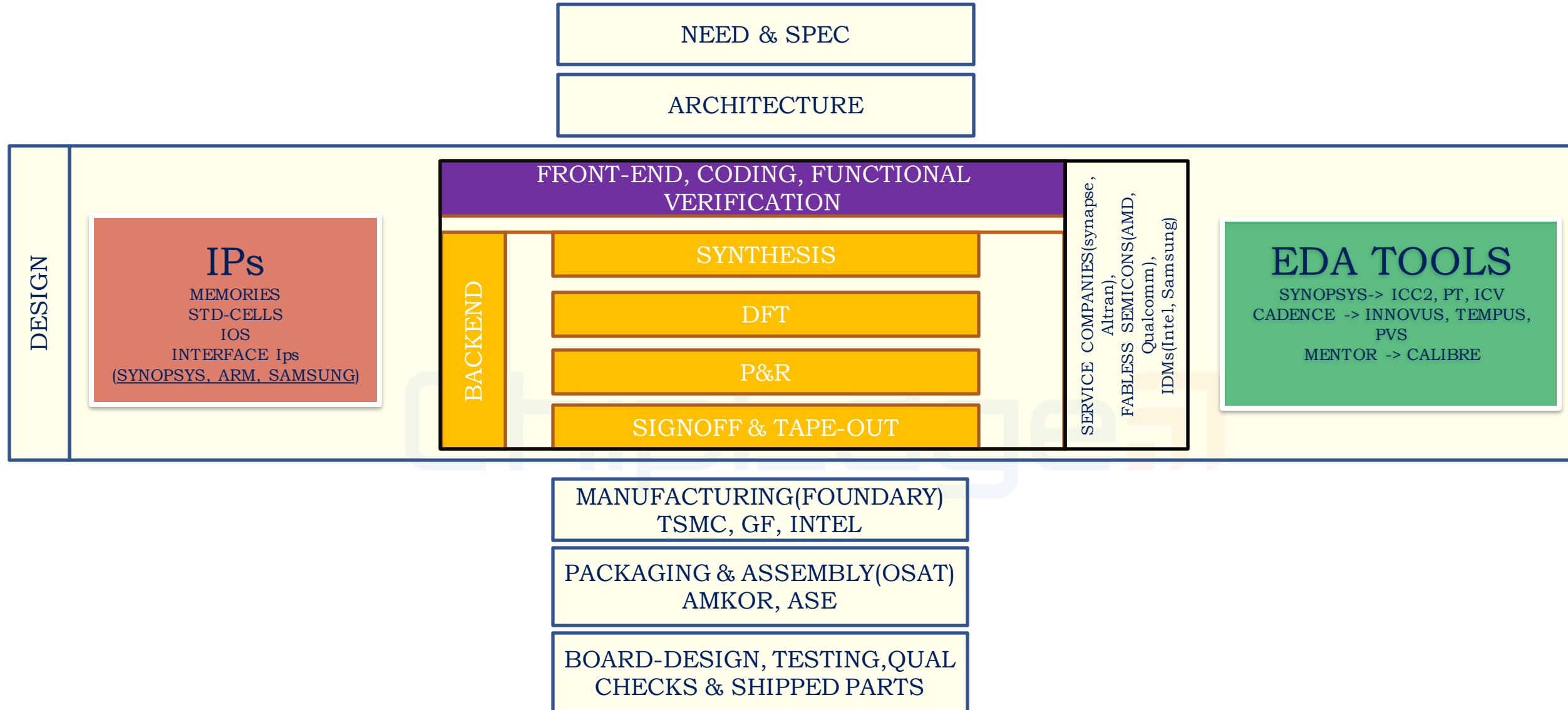


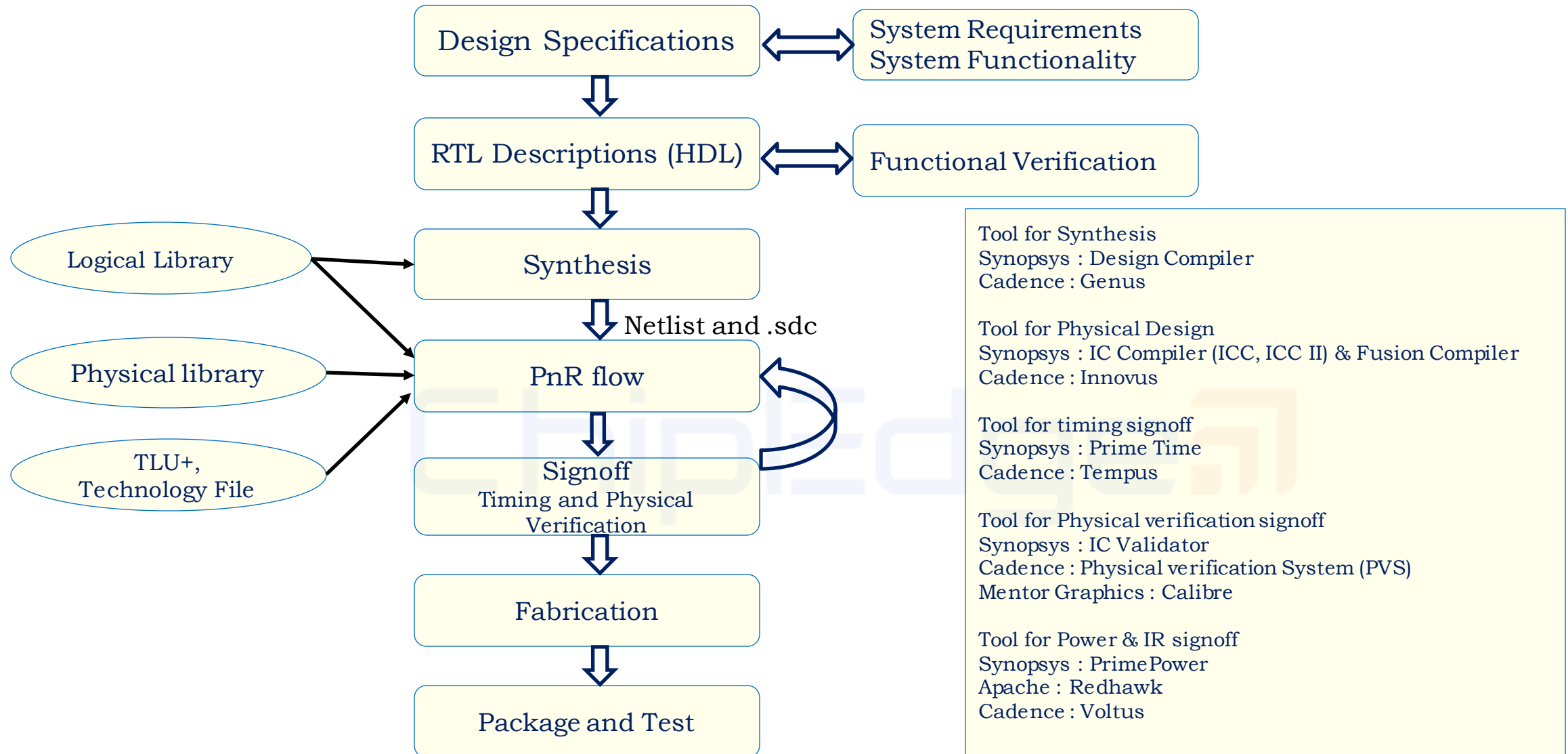
Physical Design Flow



Post GDS







Introduction to MOS



Technology Nodes & Trends



ASIC Design : Ecosystem



Design Specification

This section explains about the Design Specification and RTL Description.

Synthesis

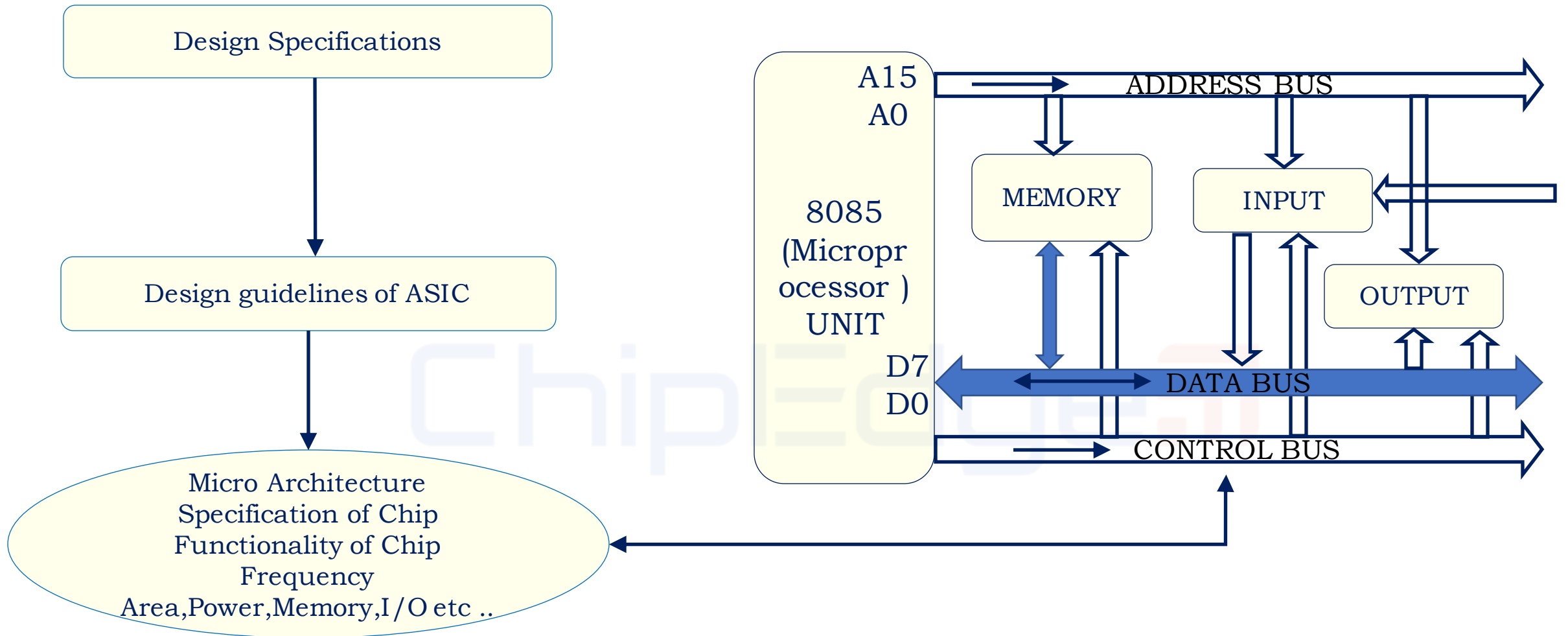


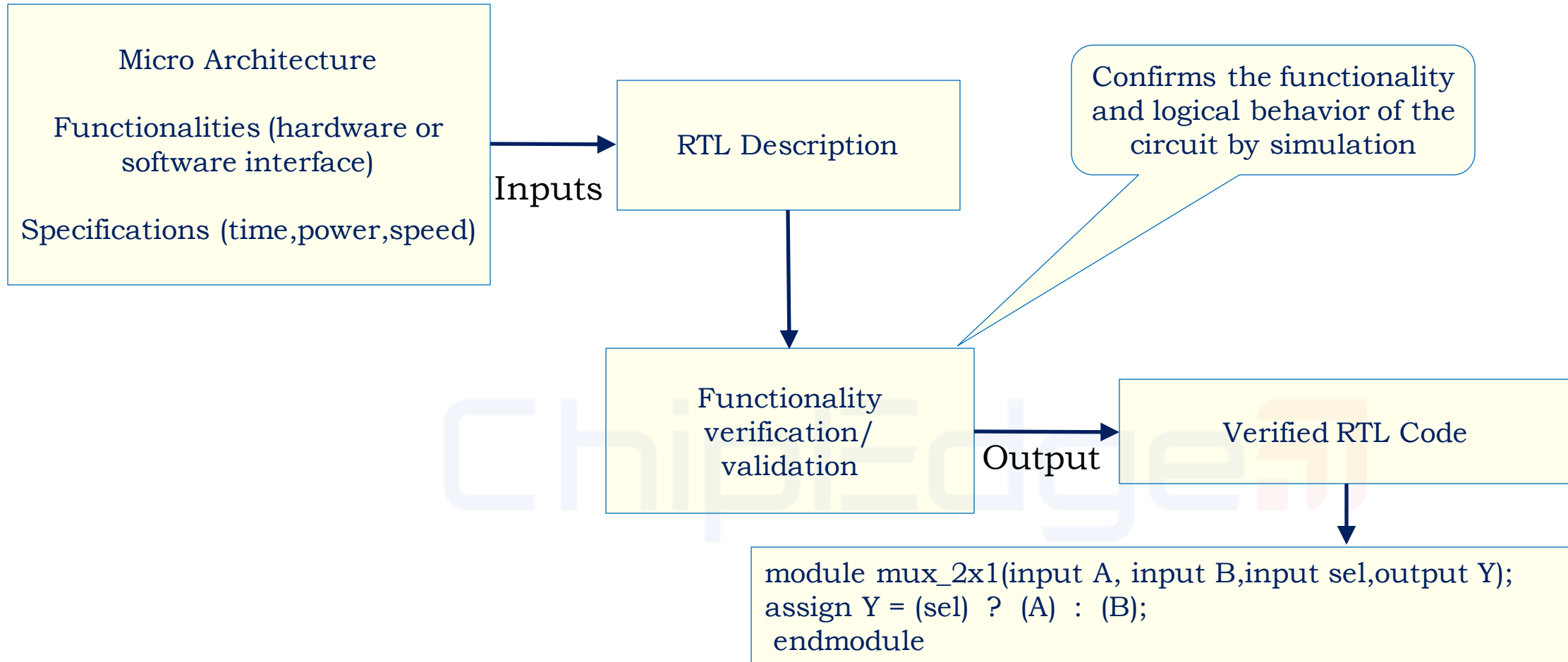
Physical Design Flow



Post GDS







Example RTL code

Introduction to MOS



Technology Nodes & Trends



ASIC Design : Ecosystem



Design Specification



Synthesis

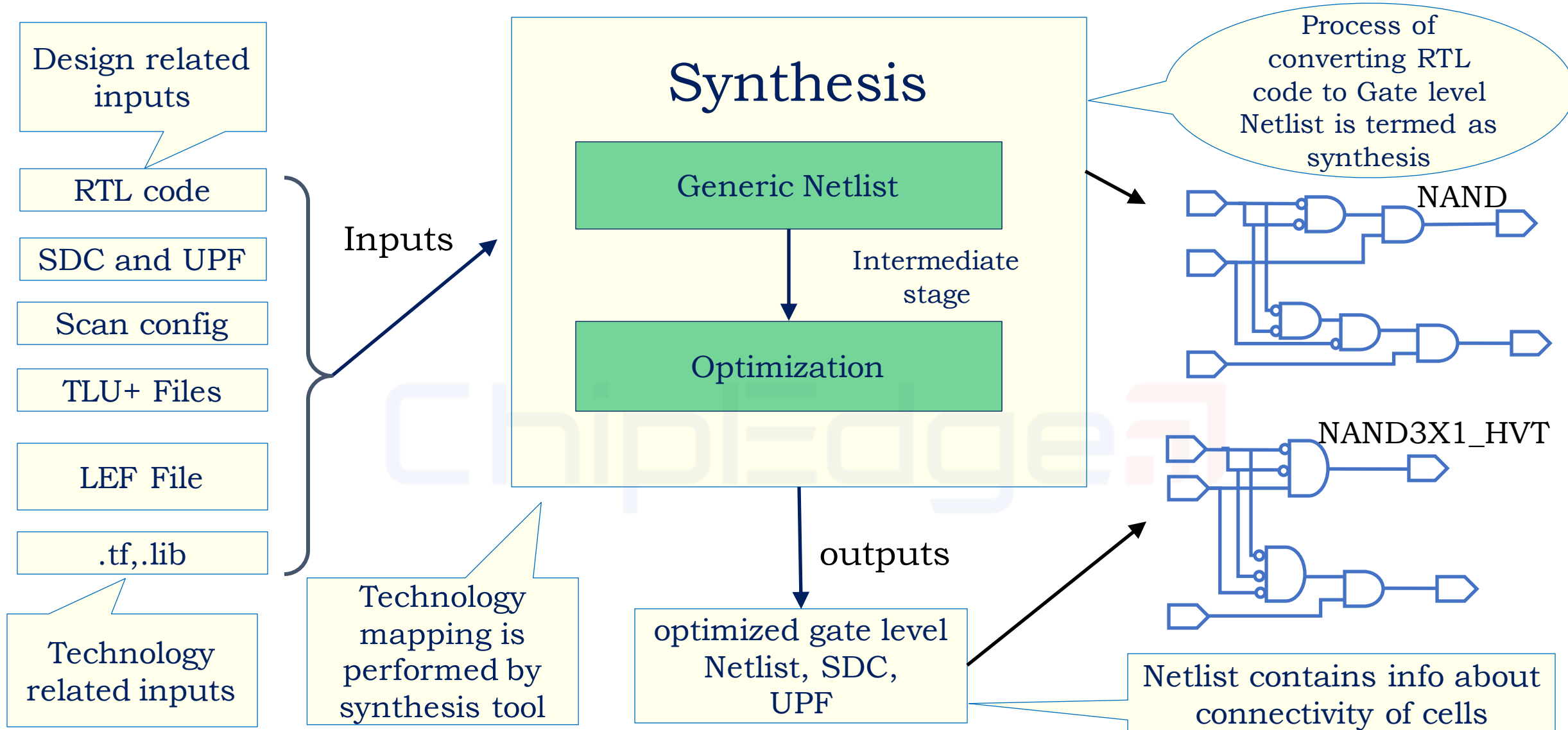
This section explains about the overview of Synthesis and the flow converting RTL to Optimized Netlist

Physical Design Flow



Post GDS





```
module mux2_1(in, sel, out);  
Input [1:0]in;  
Input sel;  
Output out;  
assign out=sel? In[1] :in[0];  
endmodule
```

RTL Code

RTL code
converted to
Intermediate
or Generic
Netlist

Generic Netlist

```
module mux2_1(in, sel, out);  
Input [1:0]in;  
Input sel;  
Output out;  
Wire N0,N1,N2;  
SELECT_OP C11(.DATA1(in[1]), .DATA2(in[0])),  
.Z(out)); .CONTROL1(N0), .CONTOL(N1),  
GTECH_BUF B_0(.A(sel), .Z(N0));  
GTECH_BUF B_1(.A(N2), .Z(N1));  
GTECH_NOT I_0(.A(sel), .Z(N2));  
endmodule
```

```
module mux2_1(in, sel, out);  
Input [1:0]in;  
Input sel;  
Output out;  
MX21X1_HVT U3(.A1(in[0]),  
.A2(in[1]), .S0(sel), .Y(out));  
endmodule
```

Final Netlist

Generic to
Optimized
Gate level
Netlist

Introduction to MOS



Technology Nodes & Trends



ASIC Design : Ecosystem



Design Specification



Synthesis

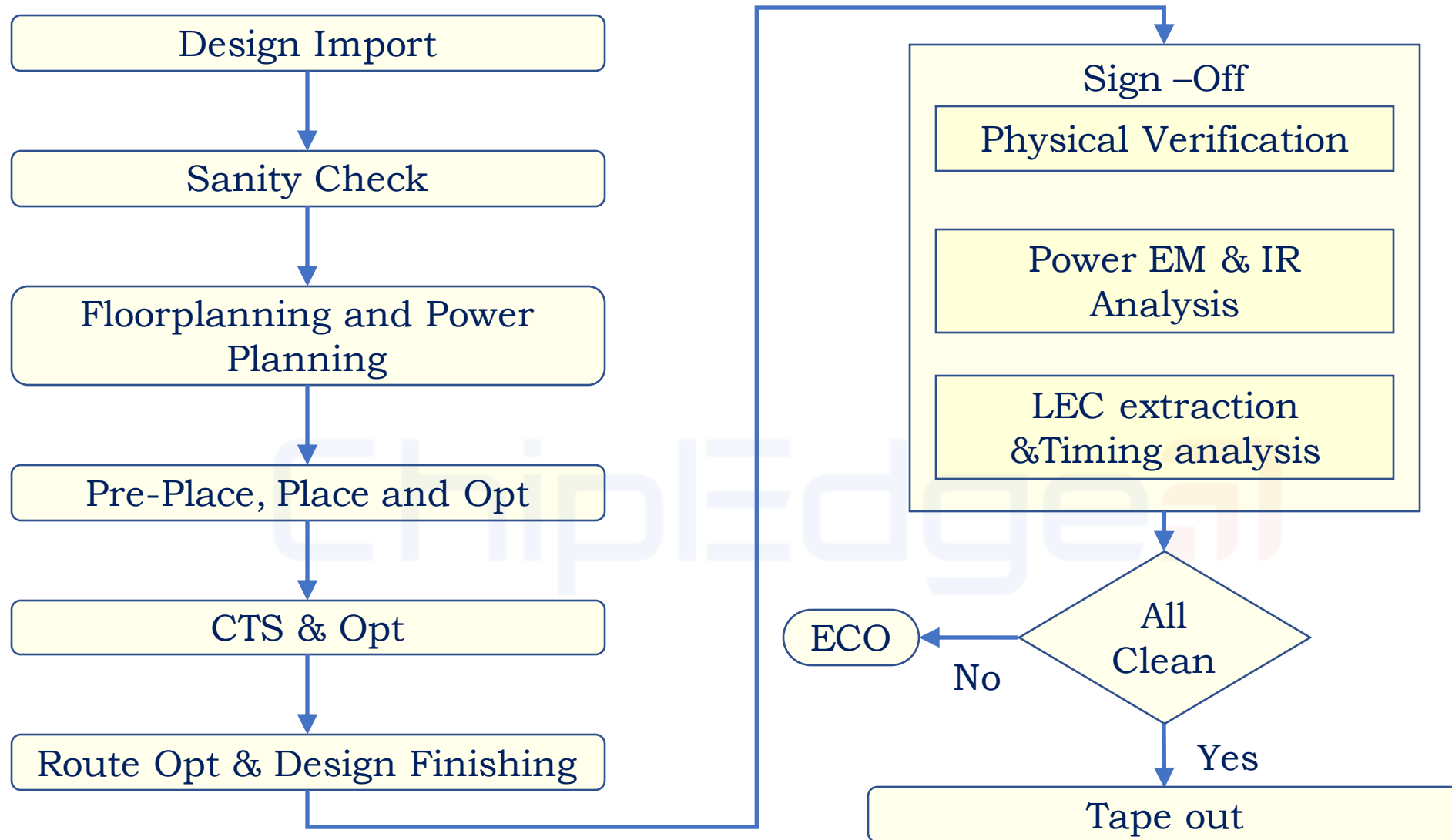


Physical Design Flow

This section explains about the PD design flow from Netlist to GDS.

Post GDS





Introduction to MOS



Technology Nodes & Trends



ASIC Design : Ecosystem



Design Specification



Synthesis

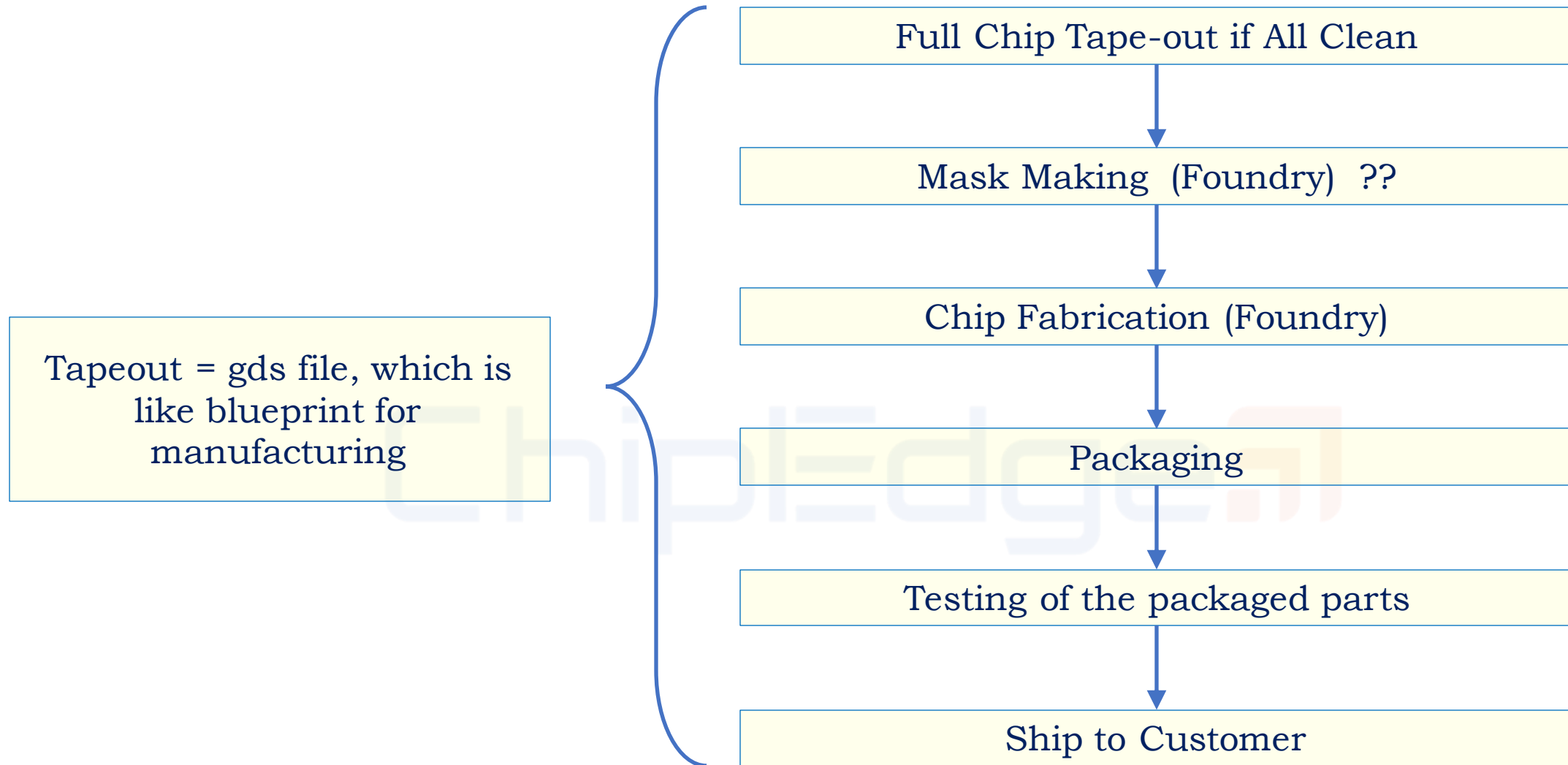


Physical Design Flow



Post GDS

This section explains the overview of Post GDS.



- Introduction to MOS
- NMOS
- PMOS
- CMOS
- Technology Nodes & Trends
- ASIC Design : Ecosystem
- Design Specification
- Synthesis
- Physical Design Flow
- Post GDS

Module 2

Synthesis

- In this module, you will learn about how RTL is Converted to Netlist, High Level Design Flow and Synthesis Flow using Design Compiler.
- Brief overview of various topics like input files for synthesis, Clock Attributes and Timing Constraints are addressed.
- The detailed explanation of Synthesis Flow is discussed in this section.
- Brief overview of QoR report and Functional Equivalence are addressed in this section.

- Introduction to Synthesis
- High Level Design Flow
- Synthesis Flow
- Timing Constraints
- QoR Report
- Functional Equivalence



Introduction to Synthesis

This section explains about Synthesis with examples.

High Level Design Flow



Synthesis Flow



Timing Constraints



QoR Report

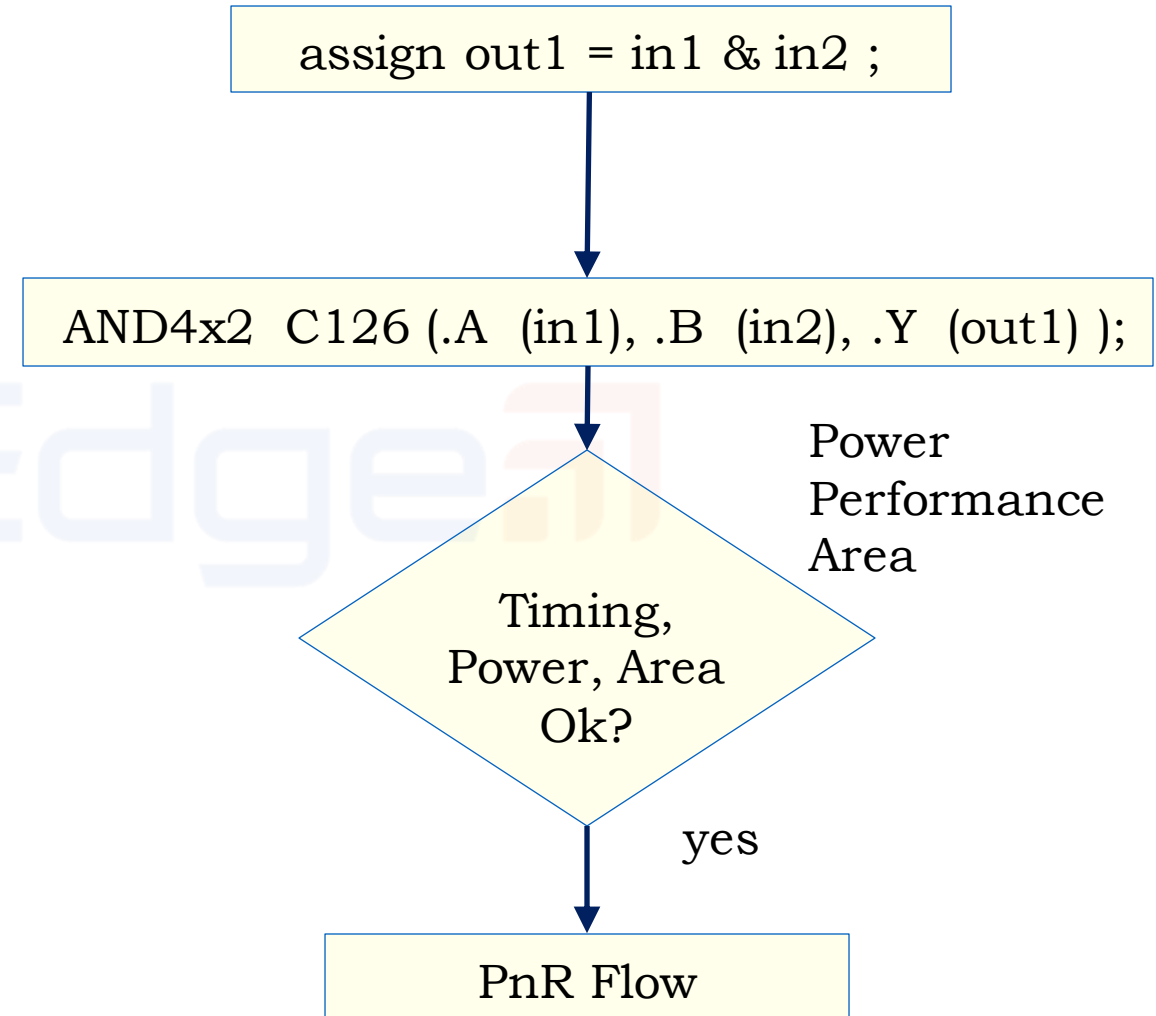


Functional Equivalence



- Synthesis converts RTL to gate-level-netlist, which is start of P&R flow.
- Synthesis goals : To generate a gate level netlist to meet the optimized parameters like
 - ✓ Performance (Timing)
 - ✓ Power
 - ✓ Area
 - ✓ Routability (*physical aware synthesis*)

- Synthesis is an automatic method of converting a higher level of abstraction to a lower level of abstraction.
- In other words, the synthesis process converts Register Transfer Level (RTL) description to gate-level net-list.
- The gate-level netlist can be optimized for area, speed, testability, etc.

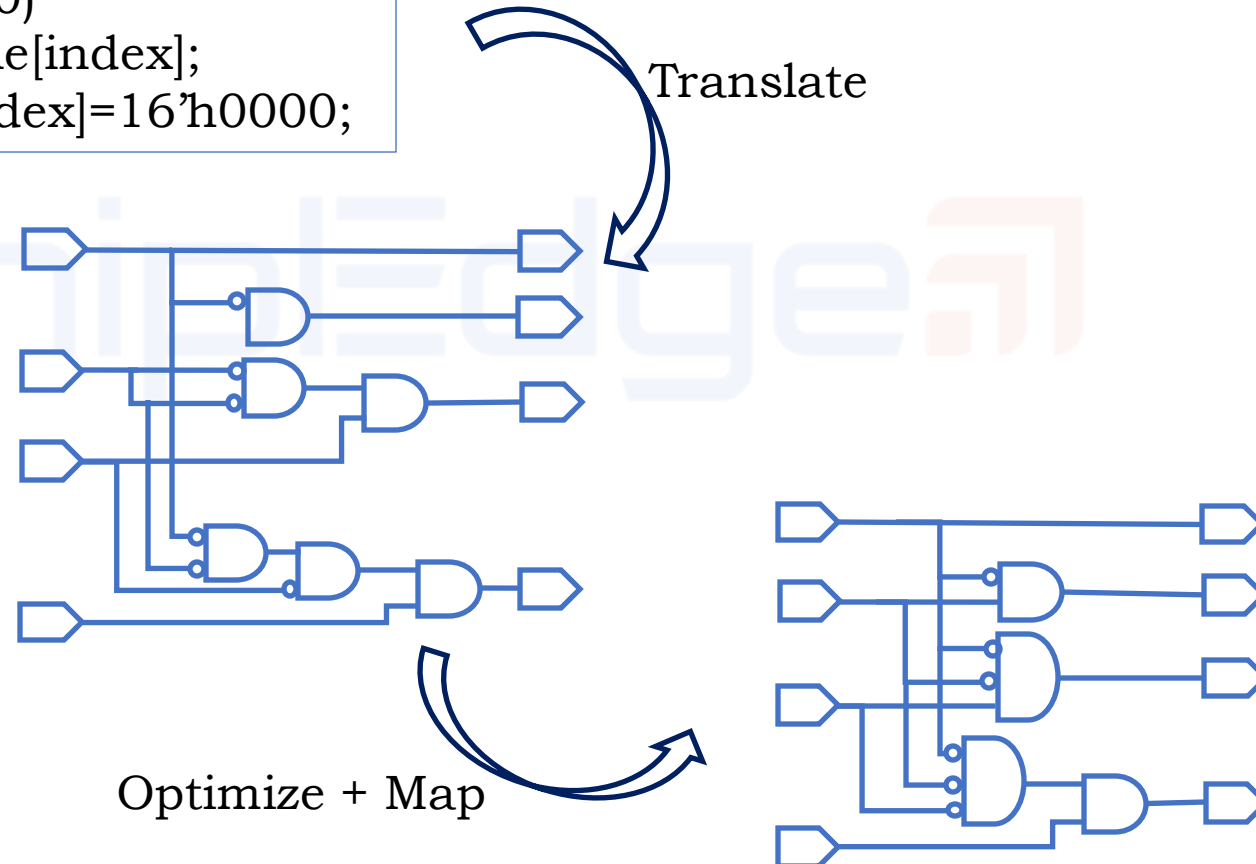


- Synthesis = Translation + Optimization + Mapping

```
residue=16'h0000;  
If(high_bits==2'b10)  
residue=state_table[index];  
else state_table[index]=16'h0000;
```

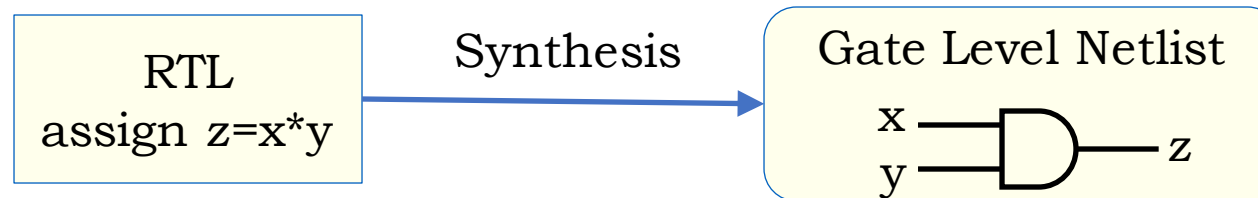
HDL Source

Generic Boolean
(GTECH)



Target
Technology

RTL	Generic Netlist	Synthesized Netlist
<code>assign z = x y;</code>	<code>or I2 (z, x, y);</code>	<code>OR2X1 I2(.Y(z), .A(x), .B(y));</code>
<code>assign z = x & y;</code>	<code>and I3 (z, x, y);</code>	<code>AND2X2 I3 (.Y(z), .A(x), .B(y));</code>



Introduction to Synthesis



High Level Design Flow

This section explains about the High Level Design Flow

Synthesis Flow



Timing Constraints

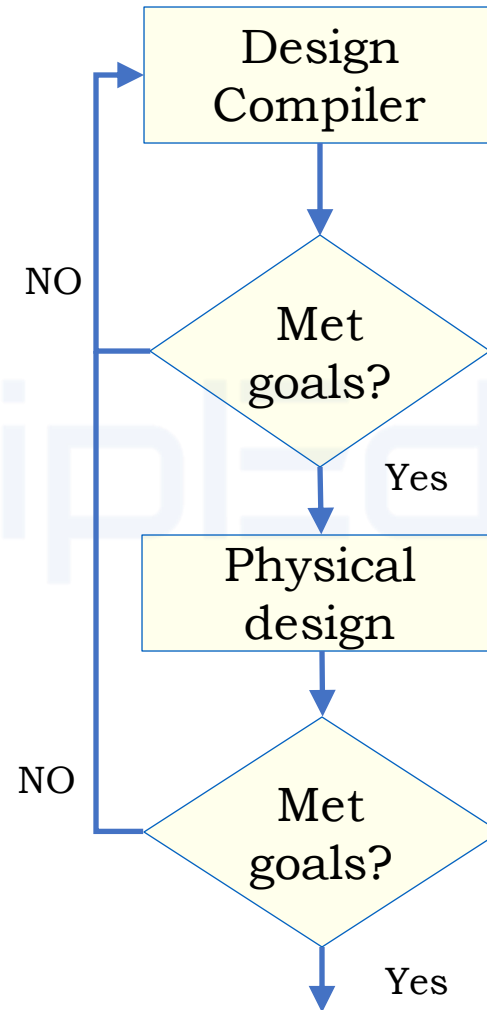


QoR Report



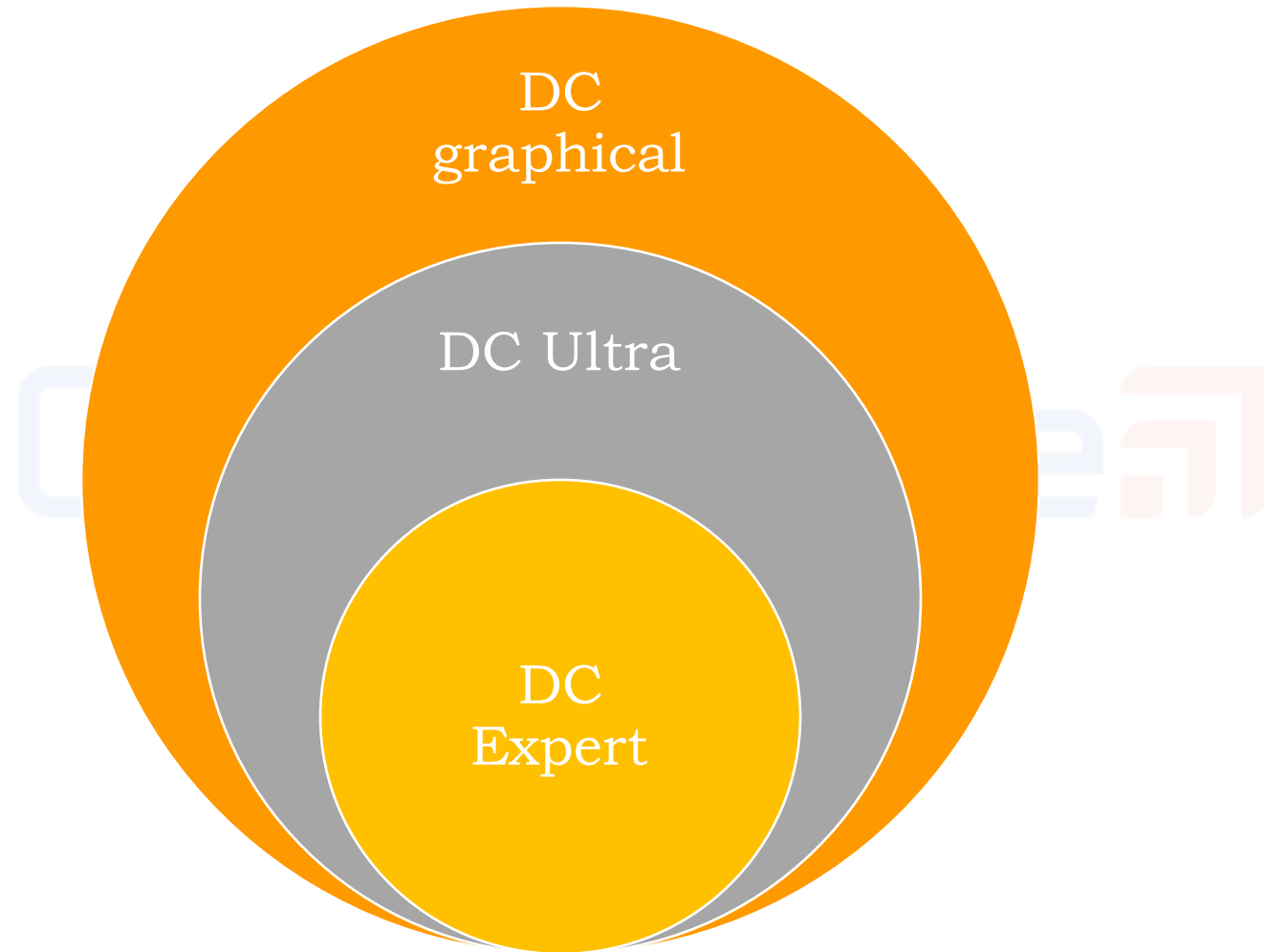
Functional Equivalence





Design Implementation
Stage

- Design Compiler – Synopsys tool ! It optimizes the design to provide smallest and fastest logical representation of a given function.
- Supports wide range of flat and hierarchical design styles.
- Optimizes both combinational and sequential designs for speed area & power.
- Three Variants of DC
 - ✓ DC Expert
 - ✓ DC ultra
 - ✓ Design Compiler graphical



Introduction to Synthesis



High Level Design Flow



Synthesis Flow

This section explains about synthesis flow.

Timing Constraints



QoR Report



Functional Equivalence

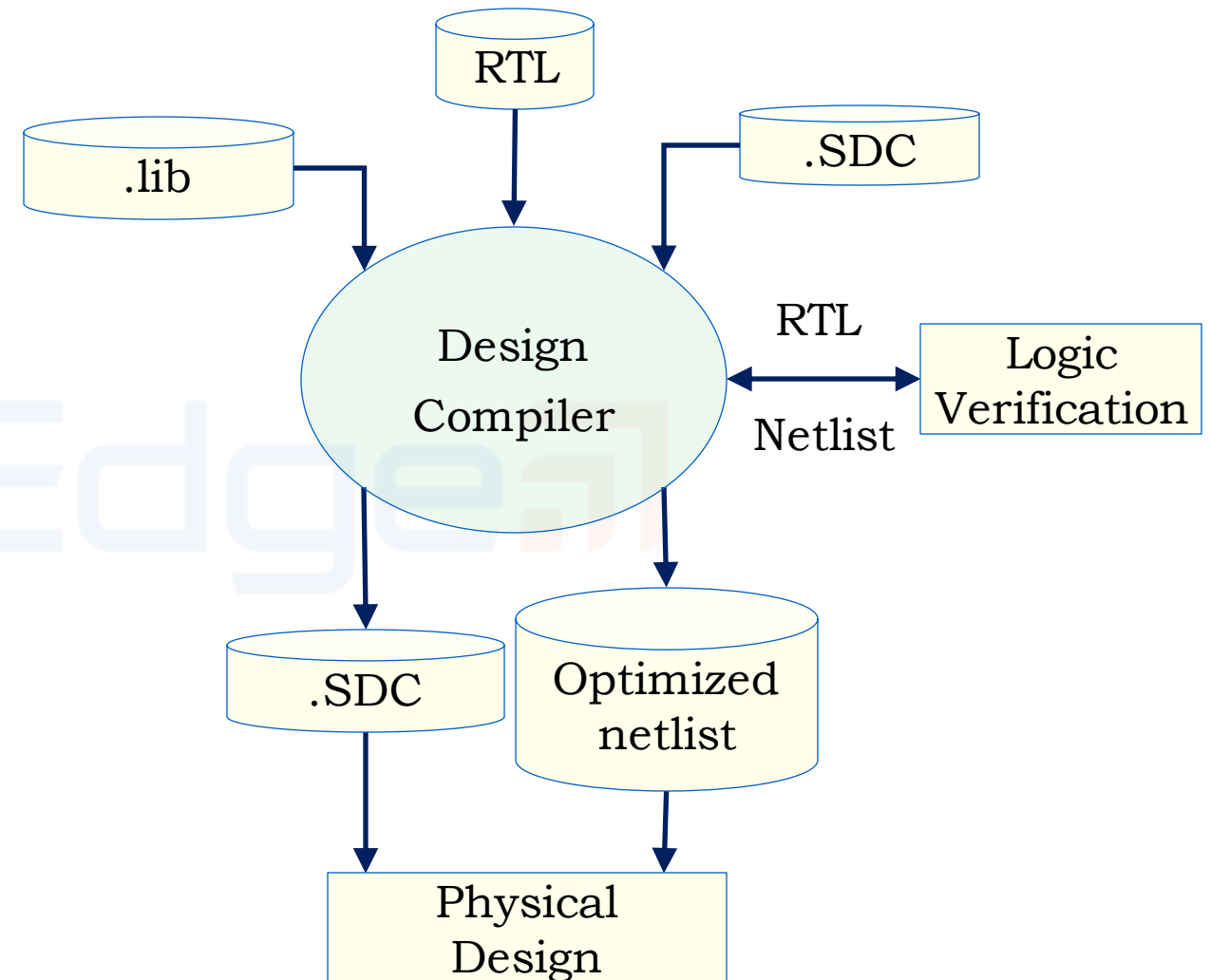


Inputs

- RTL: Verilog, VHDL
- Constraints: .sdc
- Library: .lib/.db files

Outputs

- Optimized gate level netlist
- Constraints: .sdc



- **RTL coding:** Subset of Verilog constructs are used. Typically used for expressing a functionality that is intended to translate into hardware
- **RTL Code for Full adder :-**

```
module full_adder(input a,b,c, output sum,cout);  
  
assign sum = (a ^ b ^ c );  
  
assign cout = (a & b ) | (b & c) | (a & c);  
  
endmodule
```

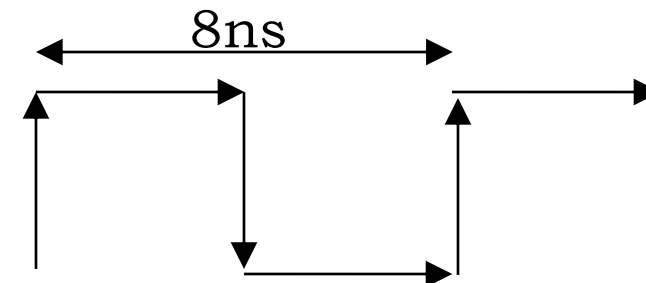
- Timing Constraints are defined in sdc (Synopsys/Standard Design Constraints) file format
- Clock Definitions
 - Create_clock
 - Create_generated_clock
- Timing Constraints
 - set_input_delay
 - set_output_delay

- `create_clock` (#command)

`create_clock` is the command to define master clock

- TCL Syntax for creating clock

```
create_clock -name scan_clock -period 8 -waveform {0 4} [get_ports scan_clk]
```



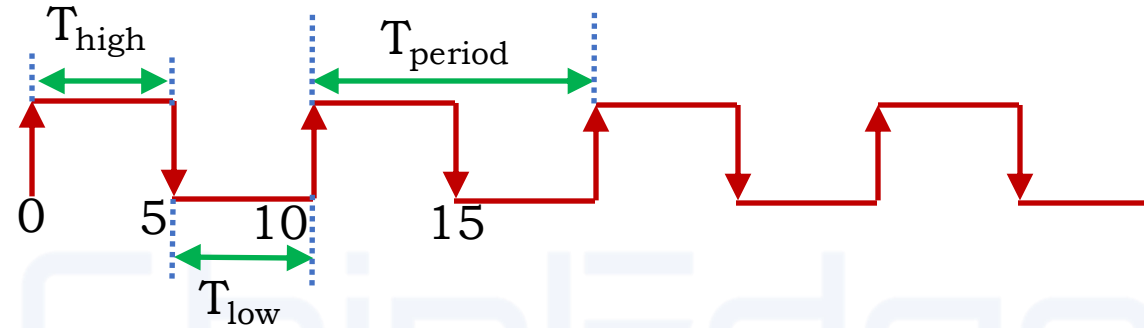
-name used to declare name of the clock . From above scan_clock is the name of the clock

-period used to declare clock period of the clock .
From below 8 is the clock period of scan clock

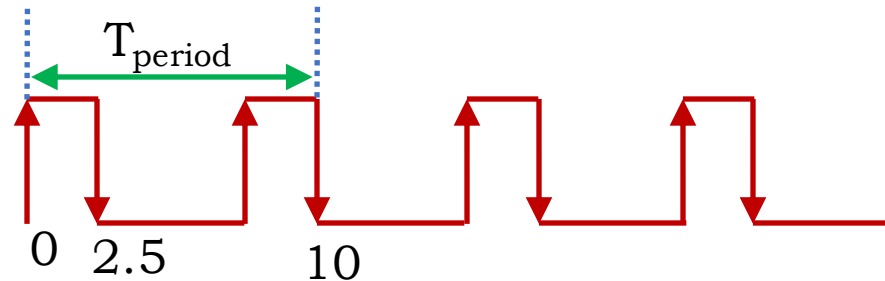
-waveform indicates rise time and fall time of the clock . From above , rise time starts at 0 and fall time starts at 4(i.e rise time =4 and fall time =4)

get_ports scan_clock defines the port named scan_clock .i.e(The clock named scan_clock is driven by the port named scan_clock)

$$\text{Duty Cycle} = \frac{T_{\text{high}}}{T_{\text{low}}} * 100 = 50\%$$



Creating clock with 25% duty cycle –create_clock PHI2 -period 10 -waveform {0 2.5}



Introduction to Synthesis



High Level Design Flow



Synthesis Flow



Timing Constraints

This section explains about Timing Constraints.

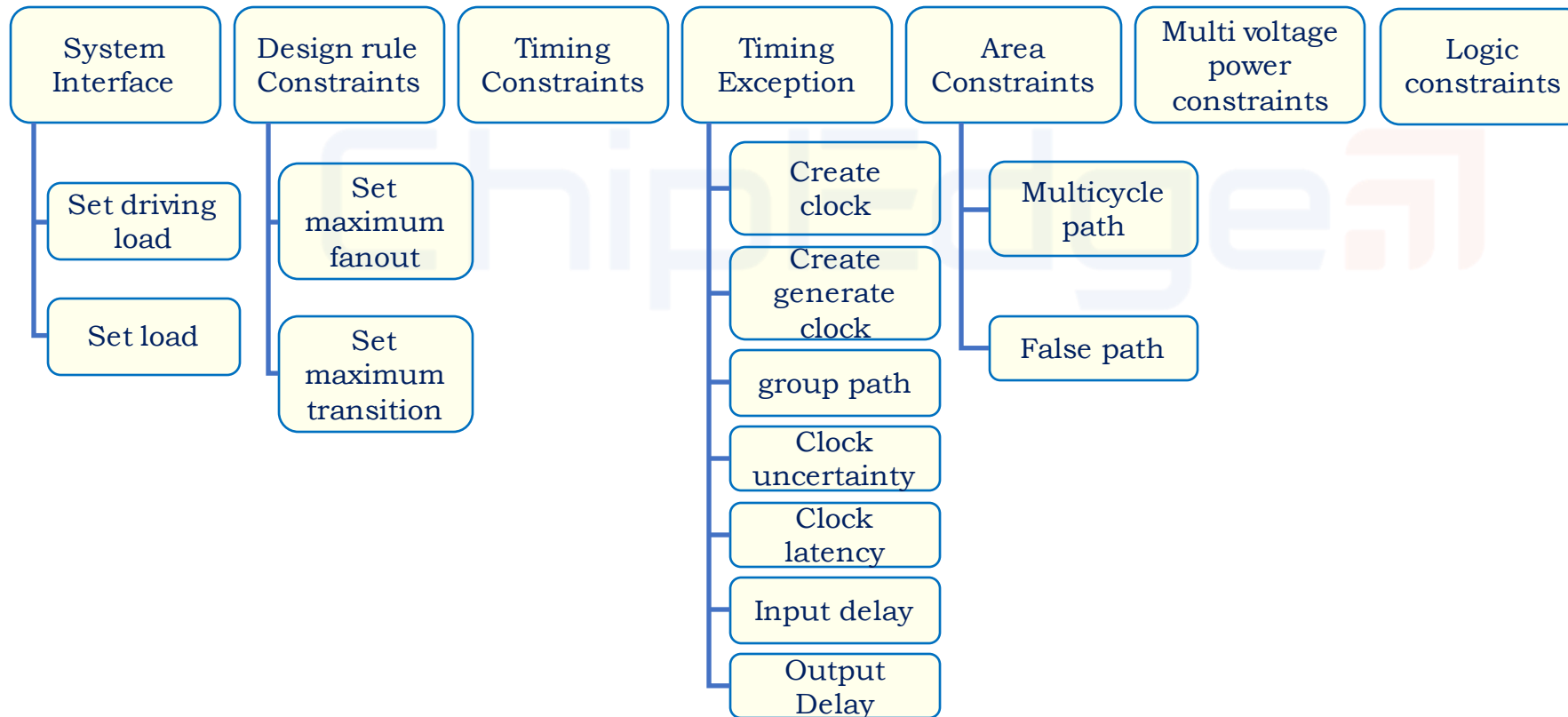
QoR Report

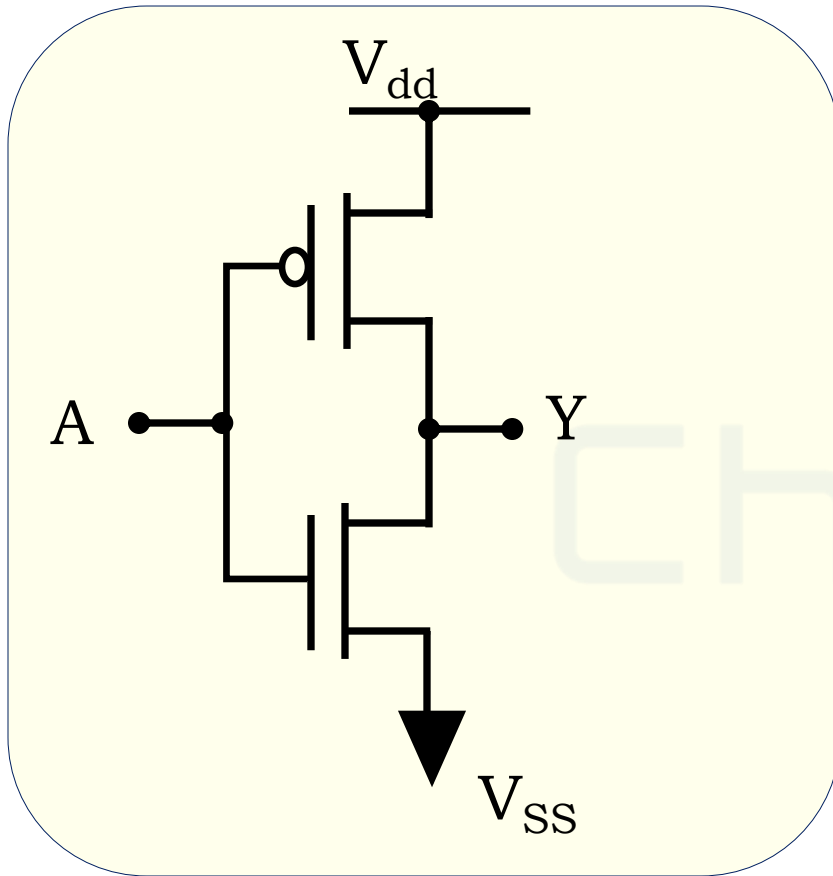


Functional Equivalence



- Timing Constraints are defined in sdc (Synopsys/Standard Design Constraints) file format.
- Few significant constraints are as follows





Example of Inverter Cell
In Logical Library

```
cell (INVX16_RVT) {  
    cell_footprint : "AV" ;  
    area : 5.08288 ;  
    cell_leakage_power : 1.635493e+04 ;  
    driver_waveform_rise : "preDrv" ;  
    driver_waveform_fall : "preDrv" ;  
    leakage_power () {  
        when : "A" ;  
        value : "1.6354926e+04" ; }  
}
```

Area Of The Inverter Cell

- Logic Library is .lib file and is also called as Liberty Timing File.
- Timing information like cell delay, output transition time for all cells.
- The parameters related to logical views like area, pin direction, max transition, rise and fall capacitances are mentioned in this library file.
- Setup and hold times for all the flops.
- Functionality information of Standard cells, IO's and Macros are part of this library.
- Power information like leakage power, internal power are present in .lib

```
timing () {  
  related_pin: "A";  
  timing type: "combinational";  
  timing sense: "negative_unate";  
  cell_rise ("del_1_7_7") {  
    index_1("0.016, 0.032, 0.064, 0.128, 0.256, 0.512, 1.024");  
    index_2("0.1, 4, 8, 16, 32, 64, 82");  
    values("0.0136840, 0.0157534, 0.0174946, 0.0206528, 0.0269745, 0.0397110, 0.0468436",\  
    "0.0198602, 0.0229519, 0.0255315, 0.0298314, 0.0365002, 0.0489652, 0.0561423",\  
    "0.0289894, 0.0335950, 0.0374534, 0.0440349, 0.0537839, 0.0683675, 0.0752175",\  
    "0.0425756, 0.0493727, 0.0553501, 0.0649100, 0.0796316, 0.1009481, 0.1107494",\  
    "0.0635862, 0.0733248, 0.0818341, 0.0958451, 0.1176826, 0.1502164, 0.1646117",\  
    "0.0998563, 0.1119520, 0.1231932, 0.1430849, 0.1755651, 0.2227145, 0.2446352",\  
    "0.1679215, 0.1811684, 0.1949991, 0.2192866, 0.2627818, 0.3323777, 0.3646340");  
  }
```

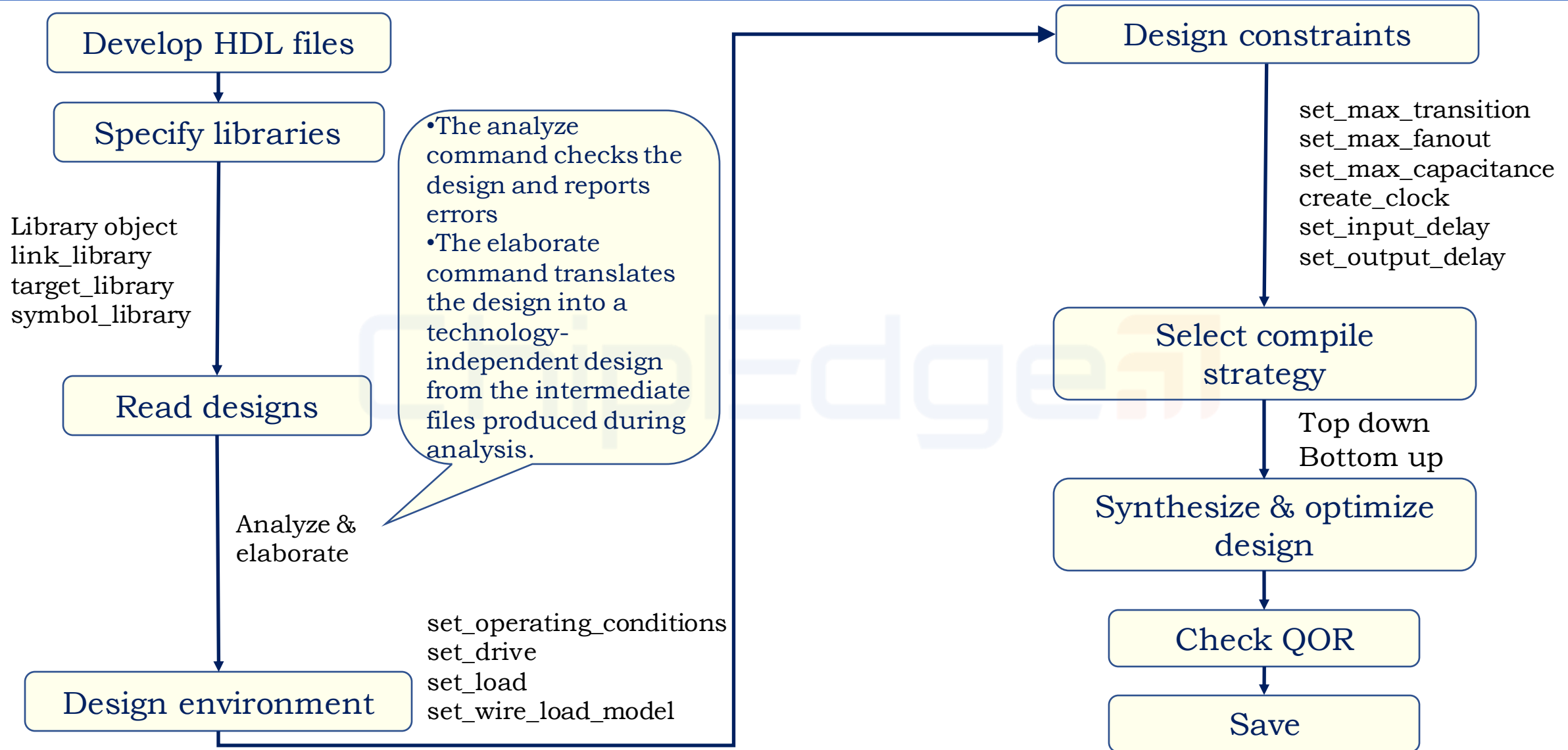
Cell Rise

Index_1 indicates input transition values

Index_2 indicates Output load values

Values indicates the cell delays

Synthesis Flow



- Compile is a process in which the Synthesis tool reads HDL source code and optimizes the design.
- Compile or compile_ultra – Compiles the design in Design Compiler (DC).
- Compile strategies –
 - ✓ Top down: top level design and all its sub-designs are compiled together.
 - ✓ Bottom up: bottom most hierarchy compiled first and then moved to higher hierarchies until top design.

- Compile flows
 - ✓ Full compile
 - ✓ Incremental compile

ChipEdge

- Optimization is the synthesis step that maps the design to an optimal combination of specific target logic library cells, based on the design's functional, speed, and area requirements.
- Optimization transforms the design into a technology-specific circuit based on the attributes and constraints you place on the design.

Design Compiler performs the levels of optimization in the following order:

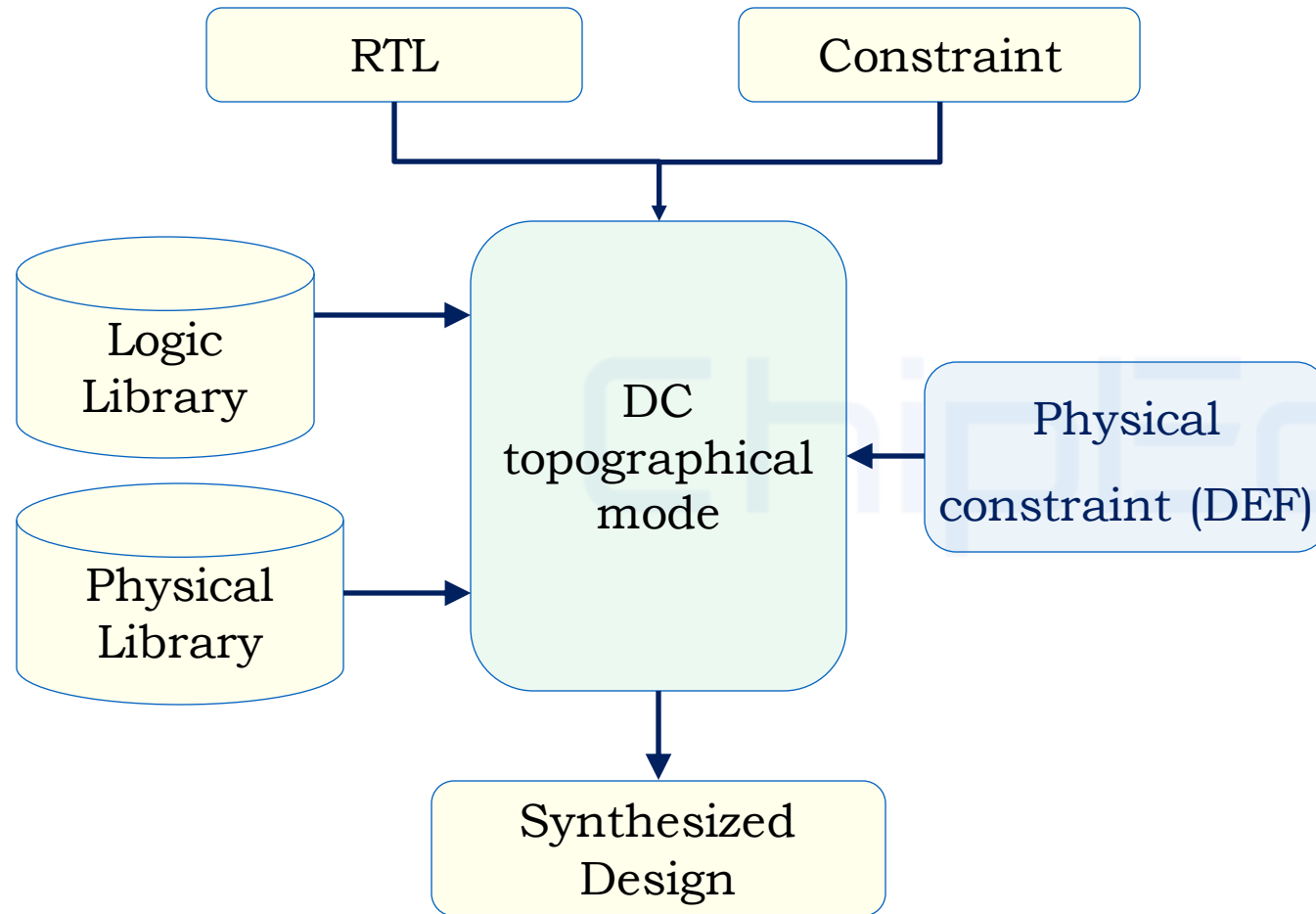
- Architectural Optimization (generates Generic netlist)
- Logic-Level Optimization (structuring & flattening of the generic n/l)
- Gate-Level Optimization (creates a tech specific n/l). Does mapping and optimization

Combinational optimization: transforms the logic level description of combinational logic to a gate level netlist and has following steps

- Technology independent – Algebraic and Boolean techniques to set of logic equations and implements it.
- Mapping – DC selects component from logic library to implement logic structure.
- Technology specific – synthesizes gate level design to meet PPA.

- Sequential optimization: Has 2 steps
 - ✓ Initial step : Map sequential cells to cells in the target library.
 - ✓ Final step : Improve design timing by choosing high performance sequential cells.
- Local optimizations
 - ✓ Final step in gate level optimization.
 - ✓ Incremental modifications to design to adjust for timing and area

- Normally, DC does synthesis assuming some wire load models.
- Better timing correlation as compared to wire load mode synthesis
- Design compiler uses topographical technology to accurately predict post layout timing, area and power during RTL synthesis
- Uses Synopsys placement & optimization technique to compute timings and ensures better correlation with physical design



DEF file Contains

- Die Area
- Core area & shape.
- Port locations.
- Keep-out margin
- Placement blockages.
- DC supports high level physical constraint

- Timing report (`report_timing`) command finds all the individual timing paths in the design for analysis
- Each path is analyzed for timing twice:
 - ✓ Once for a rising edge input
 - ✓ Once with a falling edge input
- The critical path (worst violator) for each clock group is found and reported (default)

Introduction to Synthesis



High Level Design Flow



Synthesis Flow



Timing Constraints



QoR Report

This section explains about QoR Report.

Functional Equivalence



Header

```
Startpoint: q_reg[48] (rising edge-triggered flip-flop clocked by CLK)
Endpoint: q_reg[63] (rising edge-triggered flip-flop clocked by CLK)
Path Group: CLK
Path Type: max
```

Data Arrival

Point	Incr	Path
clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.40	0.40
q_reg[48]/CLK (DFFARX2_LVT)	0.00	0.40 r
q_reg[48]/Q (DFFARX2_LVT)	0.19	0.59 r
U101/Y (NAND2X0_LVT)	0.04	0.64 f
U24/Y (INVX0_HVT)	0.04	0.68 r
U25/Y (AND3X1_HVT)	0.12	0.80 r
U55/Y (NAND4X0_HVT)	0.28	1.08 f
U102/Y (NOR2X1_LVT)	0.09	1.16 r
U115/Y (NAND2X0_LVT)	0.02	1.19 f
U116/Y (NOR2X0_LVT)	0.06	1.24 r
U117/Y (XOR2X1_LVT)	0.07	1.32 f
q_reg[63]/D (DFFARX2_LVT)	0.00	1.32 f
data arrival time		1.32

Data Required

clock CLK (rise edge)	0.50	0.50
clock network delay (ideal)	0.40	0.90
clock reconvergence pessimism	0.00	0.90
q_reg[63]/CLK (DFFARX2_LVT)		0.90 r
library setup time	-0.04	0.86
data required time		0.86

Slack

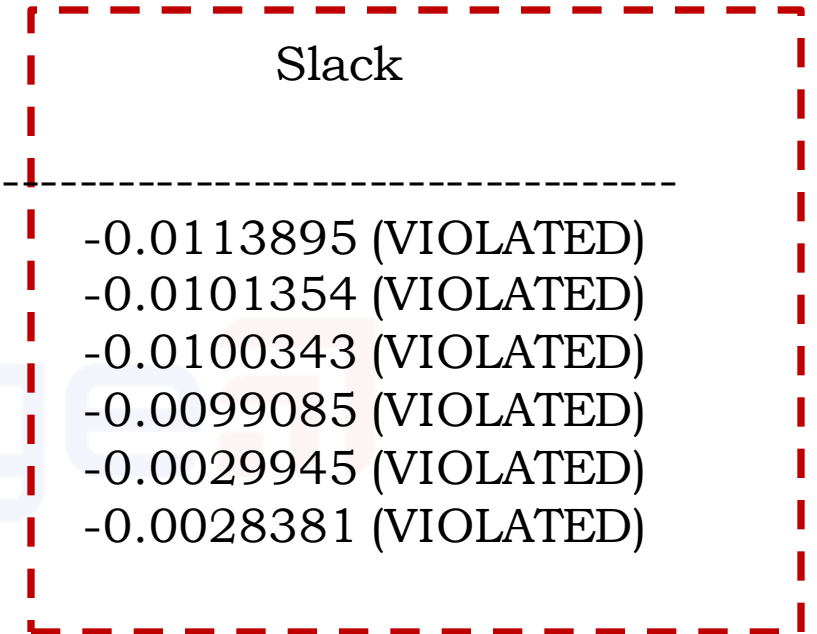
data required time	0.86
data arrival time	-1.32
slack (VIOLATED)	-0.46

Constraint Report

```
prompt> report_constraint -all_violators -max_capacitance
```

.....

Net	Required Capacitance	Actual Capacitance	Slack
n_6	0.0014226	0.0128121	-0.0113895 (VIOLATED)
n_16	0.0014226	0.0115580	-0.0101354 (VIOLATED)
n_121	0.0014226	0.0114569	-0.0100343 (VIOLATED)
n_21	0.0014226	0.0113311	-0.0099085 (VIOLATED)
div_18_17/n_2	0.0014226	0.0044171	-0.0029945 (VIOLATED)
div_18_17/n_430	0.0014226	0.0042607	-0.0028381 (VIOLATED)



DRC: Max Capacitance: Violators

```
prompt> report_area
```

```
*****
```

```
Report area
```

```
Design: TEST TOP
```

```
.....
```

```
*****
```

```
Library (s) Used:
```

```
test lib (File: test lib.db)
```

```
Number of ports:          565
```

```
Number of nets:           9654
```

```
Number of cells:          8120
```

```
Number of combinational cells: 6594
```

```
Number of sequential cells: 1526
```

```
Number of macros:         0
```

```
Number of buf/inv:        1439
```

```
Number of references:     163
```

```
Combinational area:       562404.432061
```

```
Noncombinational area:    6720840.351067
```

```
Net Interconnect area:    undefined
```

```
Total cell area:         7283244.783128
```

```
Total area:               undefined (Wire load has zero net area)
```

Area Summary:
Notice Interconnect
area = 0; Why ?

```
de shell>report_power -hier
*****
```

Report: power

-hier

-analysis effort low

Design: DESIGN_4

version: A-2007.12-SP2

Date : Fri Feb 22 01:51:42 2008

Library (s) Used:

slow (File: slow.db)

Operating Conditions: slow Library: slow

Wire Load Model Mode: Inactive.

Global operating Voltage = 1.62

Power-specific unit information :

Voltage Units=1V

Capacitance Units= 1.000000pf

Time Units=1ns

Dynamic Power Units=1mw (derived from V.C.T units)

Leakage Power Units=1nw

Information: Reporting correlated power. (PWR-620)

Hierarchy	Switch Power	IntPower	LeakPower	TotalPower	%
A7s_top	8.683	27.368	2.66e+03	36.054	100.0
CLOCK TREE EST	3.102	1.837	9.914	4.940	13.7
U_CORE (A7S_core)	4.318	21.712	2.23e+03	26.032	72.2

Power
categories

```
prompt> report_qor
```

```
*****
```

```
Report: qor
```

```
Design: TEST_TOP
```

```
....
```

```
*****
```

Timing Path Group 'clk'

Levels of Logic:	1.00
Critical Path Length:	0.02
Critical Path Slack:	-0.68
Critical Path Clk Period:	8.00
Total Negative Slack:	-393.61
No. of Violating Paths:	1066.00
Worst Hold Violation:	0.00
Total Hold Violation:	0.00
No. of Hold Violations:	0.00

Cell Count

Hierarchical Cell Count:	7
Hierarchical Port Count:	2360
Leaf Cell Count:	43978
Buf/Inv Cell Count:	6764
CT Buf/Inv Cell Count:	4
Combinational Cell Count:	37212
Sequential Cell Count:	67730
Macro Count:	0

Path group Timing
QoR

Cells utilized
Statistics

Area

Combinational Area:562404.432061Noncombinational Area:6720840.351067Net Area:0.000000Net XLength :2836623.25Net Ylength:2555007.75

Cell Area:7283244.783128Design Area:7283244.783128Net Length:5391631.00Design Rules:

Total Number of Nets:47207Nets With Violations:5Max Trans Violations:3

Hostname: machineCompile CPU Statistics

Resource Sharing:21.54Logic Optimization:182.63Mapping Optimization:230.79Overall Compile Time:631.32Overall Compile Wall Clock Time:288.11

Area statistics and QoR

DRC QoR

Compile Time QoR

Introduction to Synthesis



High Level Design Flow



Synthesis Flow



Timing Constraints



QoR Report



Functional Equivalence

This section explains about the Functional Equivalence.

- After optimization synthesis, you can use an equivalence checking tool to verify that your gate-level netlist is functionally equivalent to your RTL.
- This verification step ensures that the synthesis process or manual design changes did not introduce functional errors (design differences from original RTL).
- LEC is a 2-step process.
- Match : Prior to design verification, Formality tries to match each primary output, sequential element, black box input pin, and qualified net in the implementation design with comparable design object in the reference design.

- **Verify** : Verification is the primary function of equivalence checking. By default, Formality checks for design verification consistency when you verify a design. The term design equivalence refers to the verification test objective. LEC tool verifies a compare point by comparing the logic cone from a compare point in the implementation design against a logic cone for a matching compare point from the reference design

- `read_db -tech_lib.db`
- `set_svf files.svf`
- `read_Verilog -r reference_netlist.v` (RTL/Gate level Netlist)
- `read_Verilog -i revised_netlist.v` (RTL/Gate level Netlist)
- `match`
- `verify`

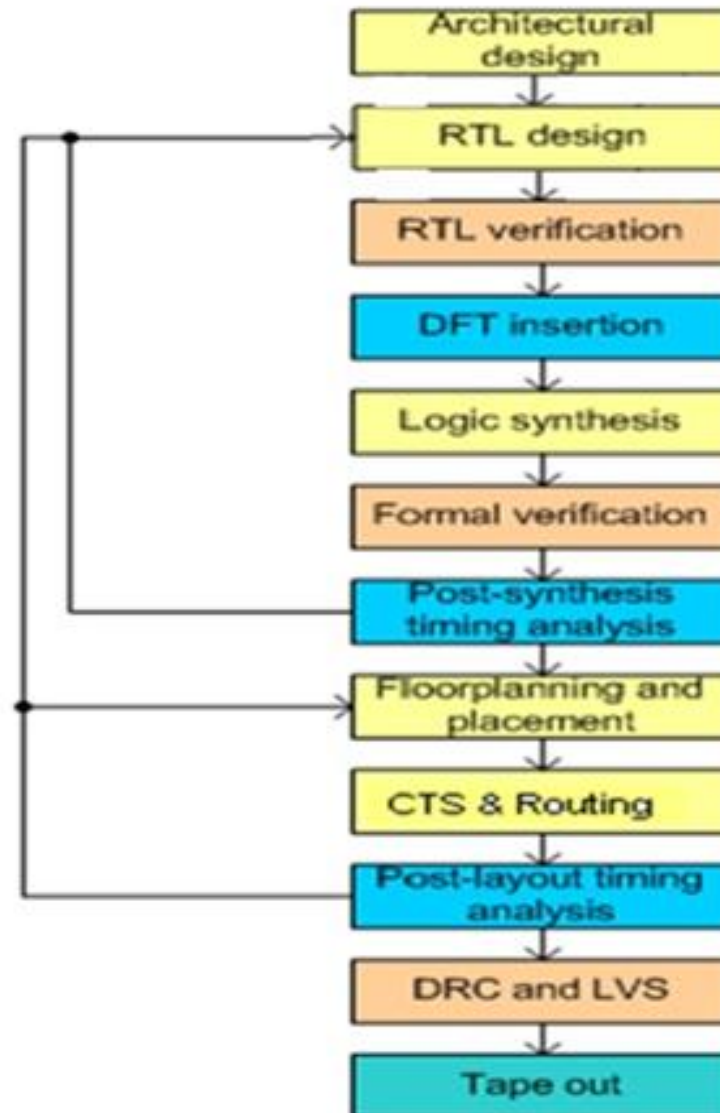
- Introduction to Synthesis
- High Level Design Flow
- Synthesis Flow
- Timing Constraints
- Compiling Design
- Optimizing Design
- Optimizing Phases
- QoR Report
- Functional Equivalence

Synthesis 1

- Synthesis
 - Design Compiler User Guide – Synopsys
 - Advanced ASIC chip synthesis – Himanshu Bhatnagar
- Equivalence Checking
 - Formality User Guide – Synopsys
- Static Timing Analysis
 - Static Timing Analysis for Nanometre Designs – J.Bhaskar, Rakesh Chadha.

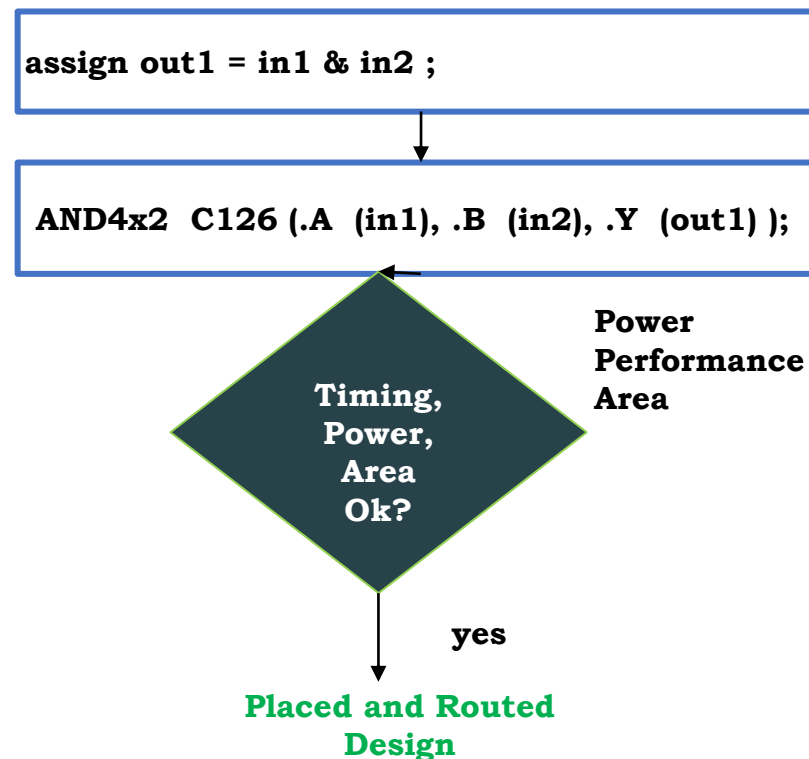
Synthesis Overview & Flow

Overview Of ASIC Design Flow

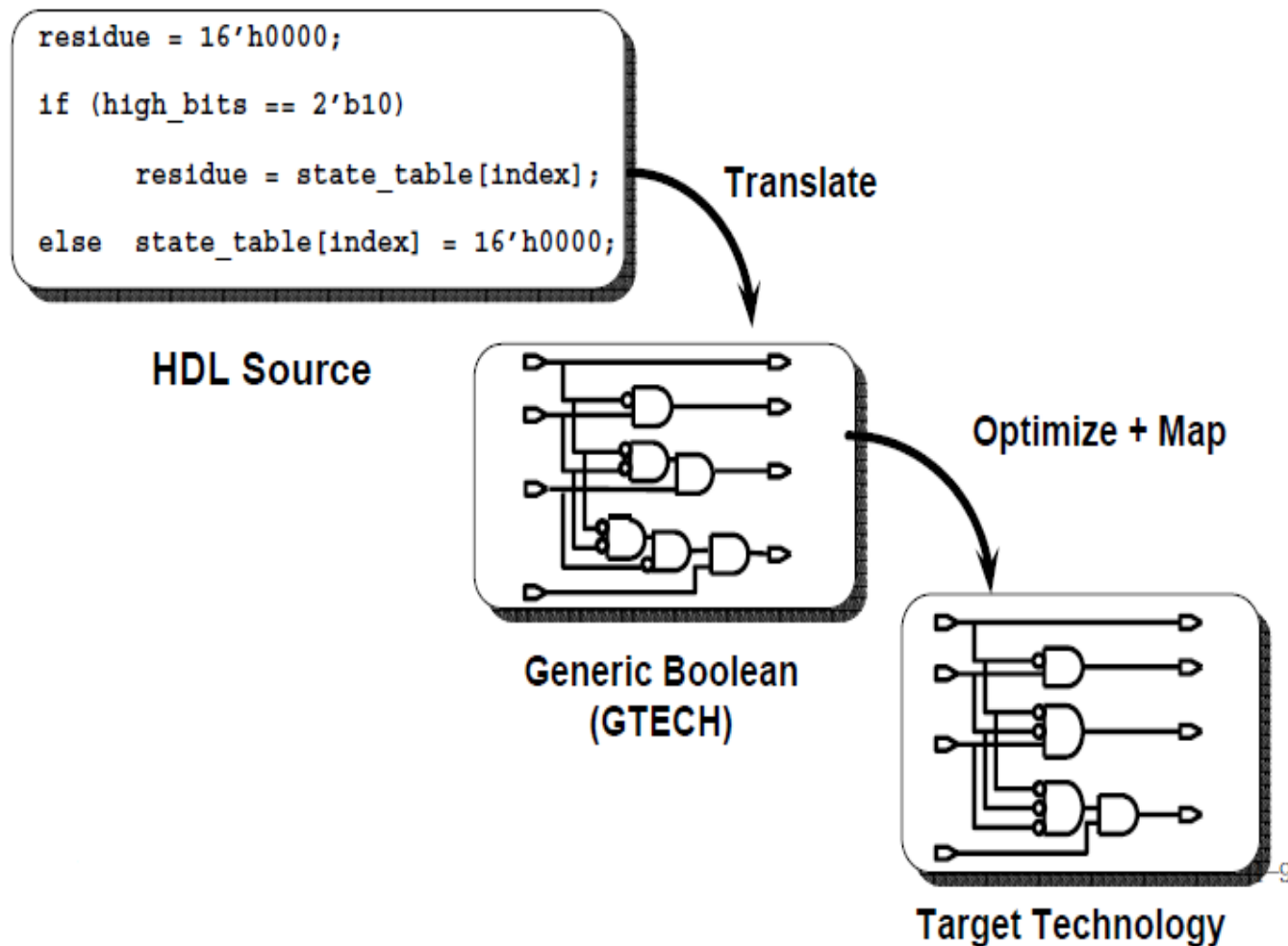


What Is Synthesis?

- **Synthesis** is an automatic method of converting a higher level of abstraction to a lower level of abstraction.
- In other words the synthesis process converts Register Transfer Level (RTL) description to gate-level net-list.
- The gate-level netlist can be optimized for area, speed, testability, etc.



Synthesis = Translation + Optimization + Mapping



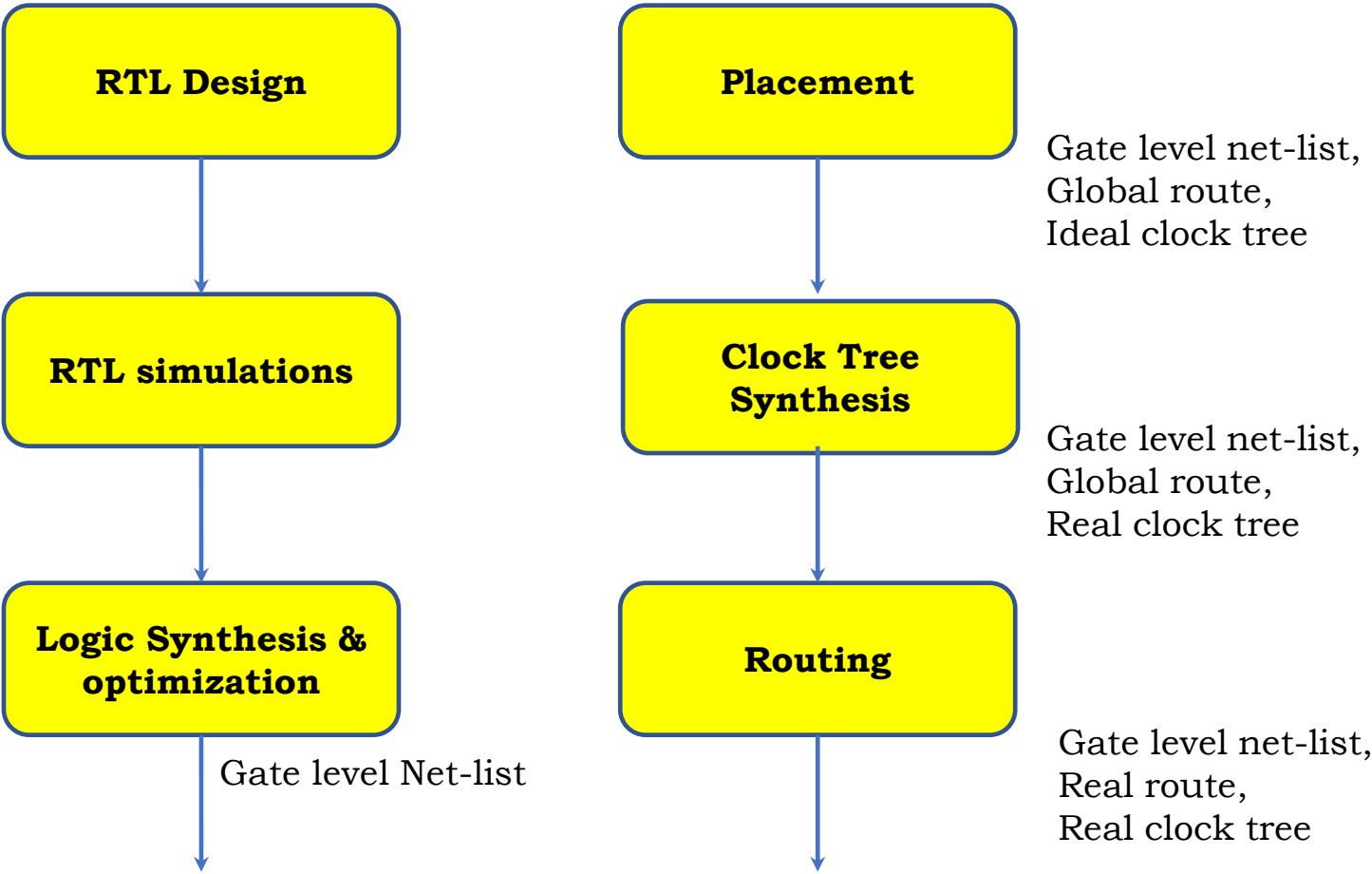
RTL	Generic Netlist	Synthesized Netlist
<code>assign z = x y;</code>	<code>or I2 (z, x, y);</code>	<code>OR2X1 I2(.Y(z), .A(x), .B(y));</code>
<code>assign z = x & y;</code>	<code>and I3 (z, x, y);</code>	<code>AND2X2 I3 (.Y(z), .A(x), .B(y));</code>

- Bad netlist – Poorly structured paths, higher area
- Backend Issues
 - Difficult to close timing
 - Too many iterations to converge
 - Congestion / Cross talk issues
- Project issues
 - Increased Power consumption
 - Larger die size
 - Increased Engineering / computing costs

To produce a Netlist which meets below goals

- Timing
- Area
- Power
- Routable

Synthesis Flow using Design Compiler



Design Compiler is the core of the Synopsys synthesis software products.

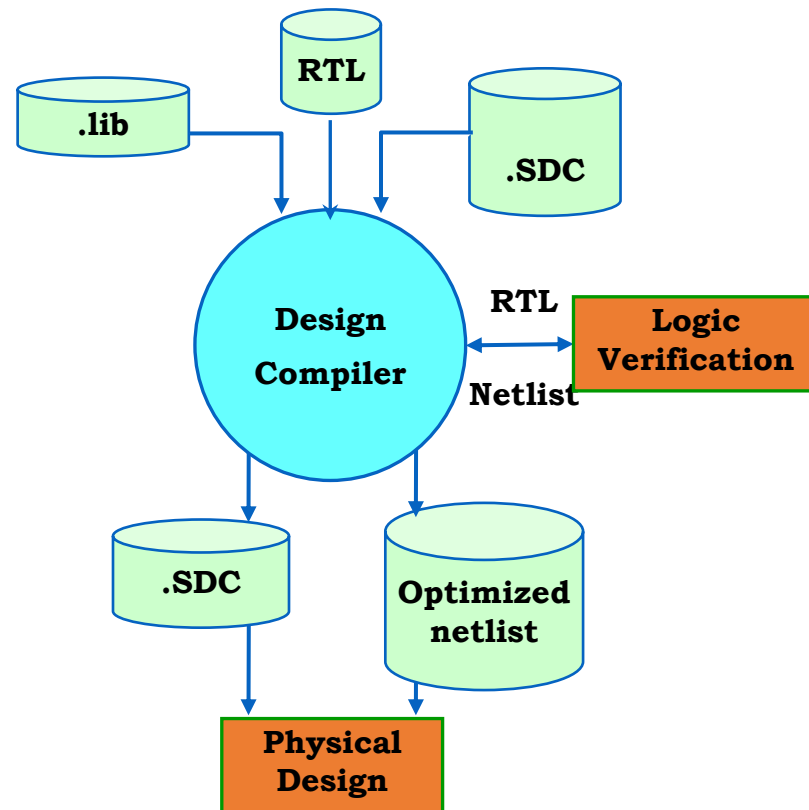
It includes tools that synthesize the HDL designs into optimized technology-dependent, gate level designs.

Inputs

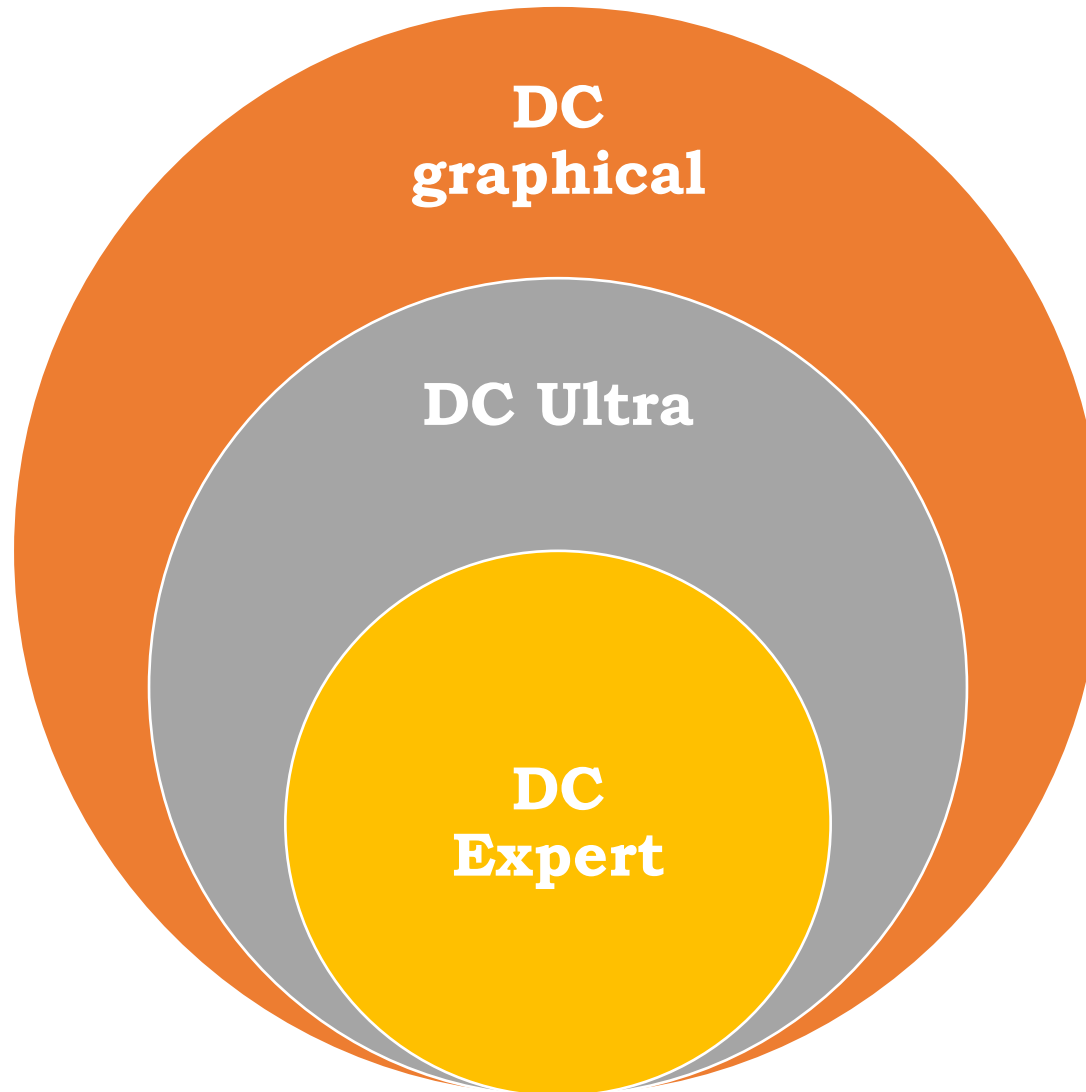
- RTL: Verilog, VHDL
- Constraints: .sdc
- Library: .lib

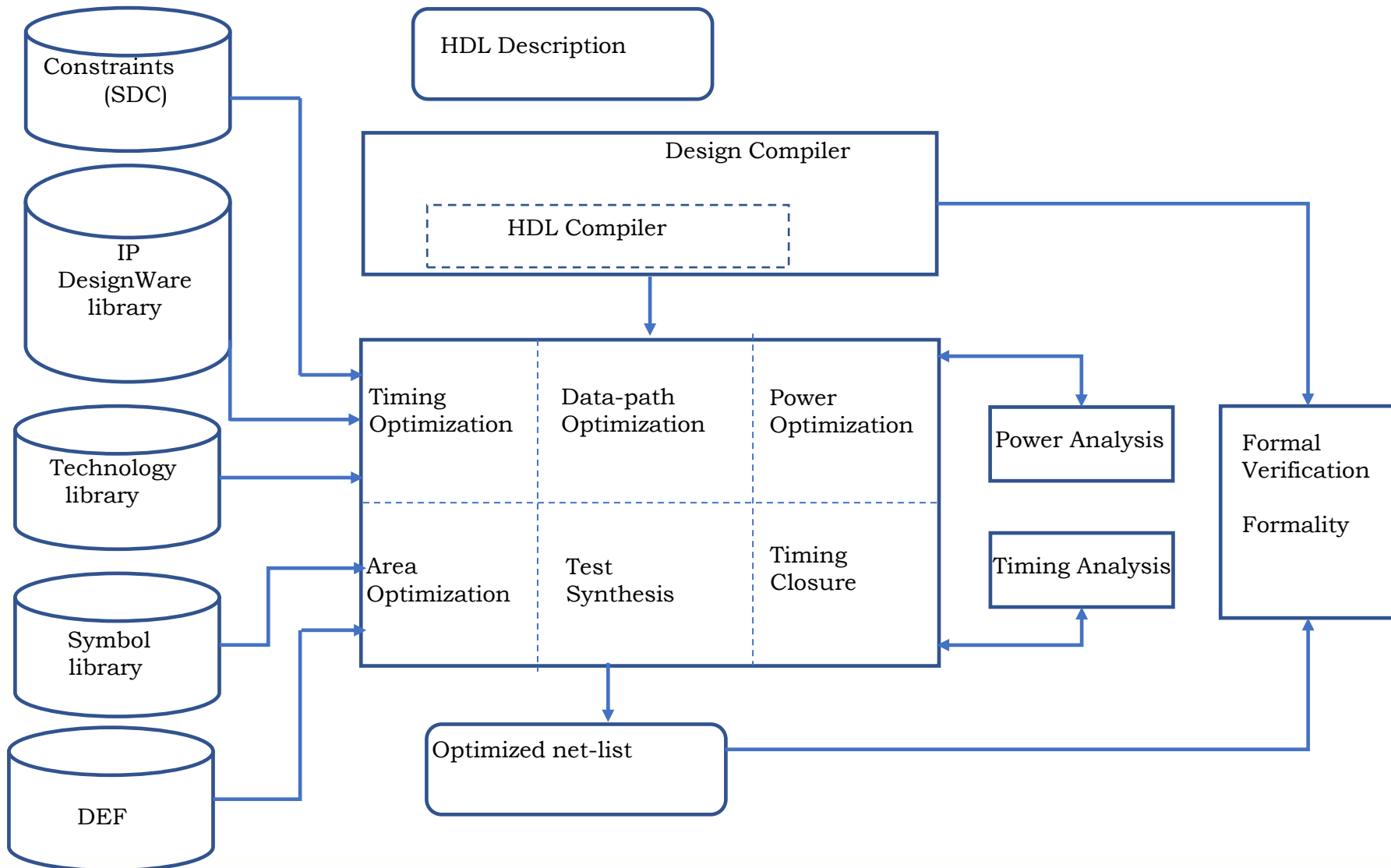
Outputs

- Optimized netlist
- Constraints: .sdc

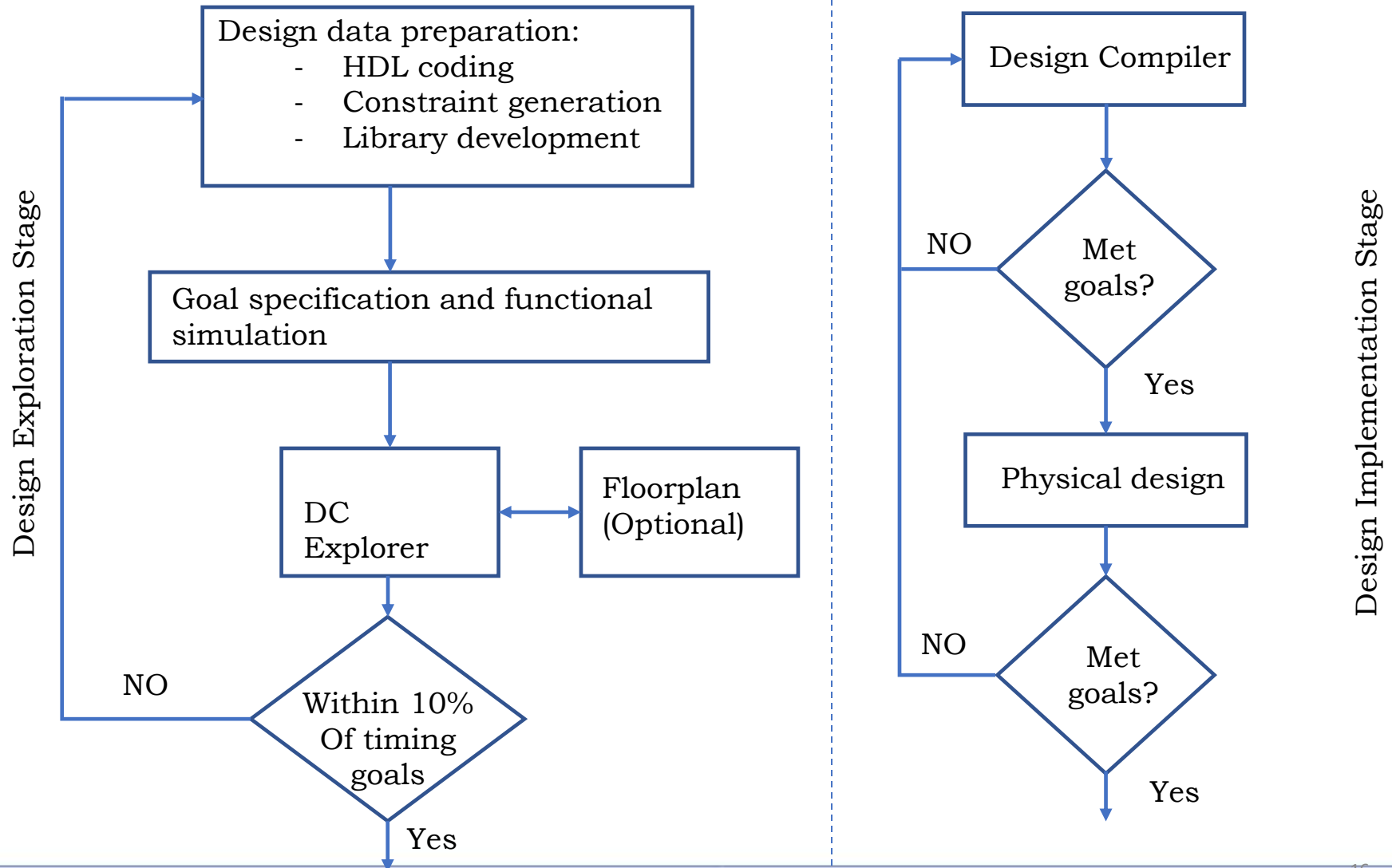


- Design Compiler – Synopsys tool ! It optimizes the design to provide smallest and fastest logical representation of a given function.
- Supports wide range of flat and hierarchical design styles.
- Optimizes both combinational and sequential designs for speed area & power.
- Three Variants of DC
 - DC Expert
 - DC ultra
 - Design Compiler graphical

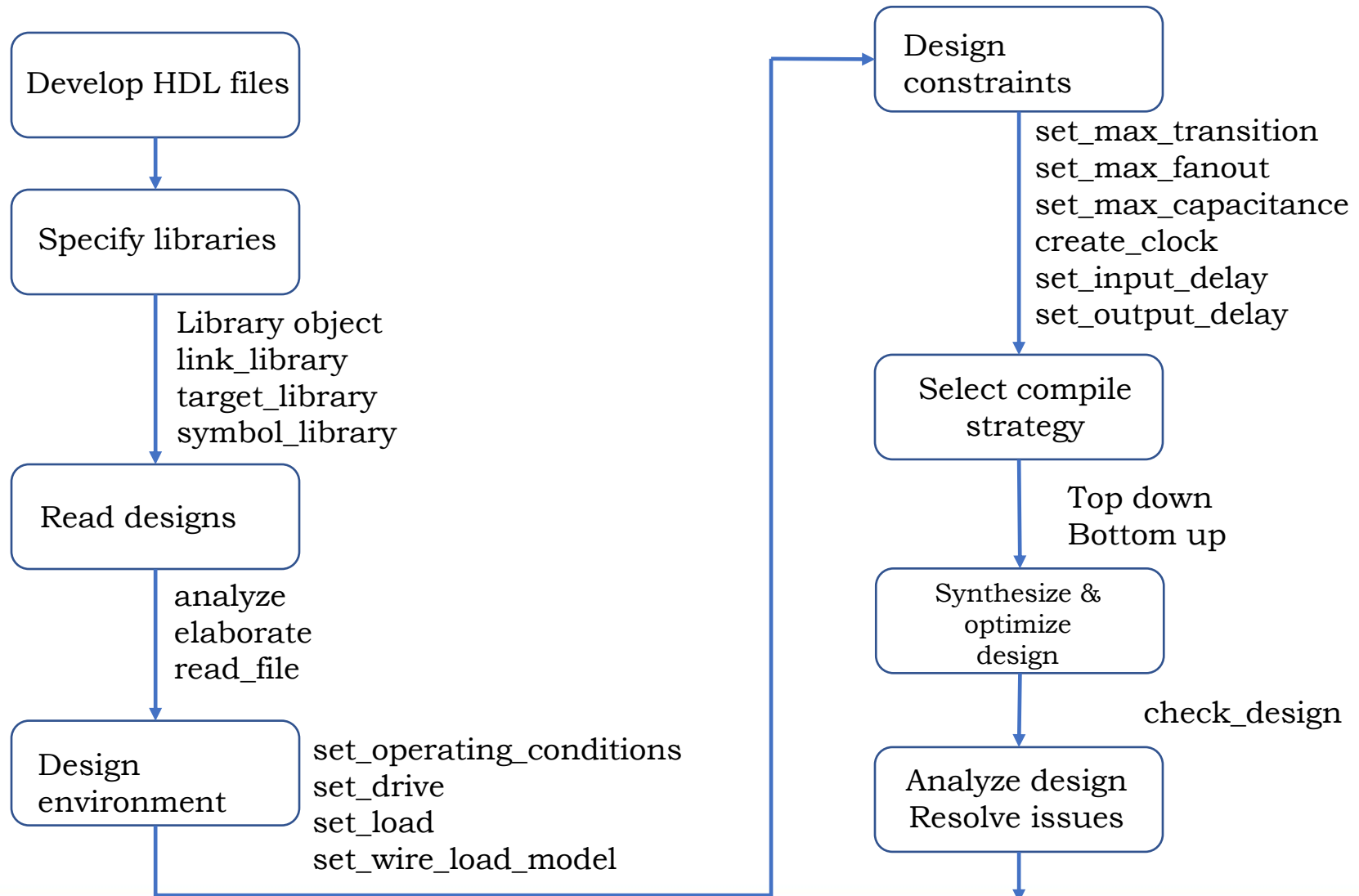




High Level Design Flow

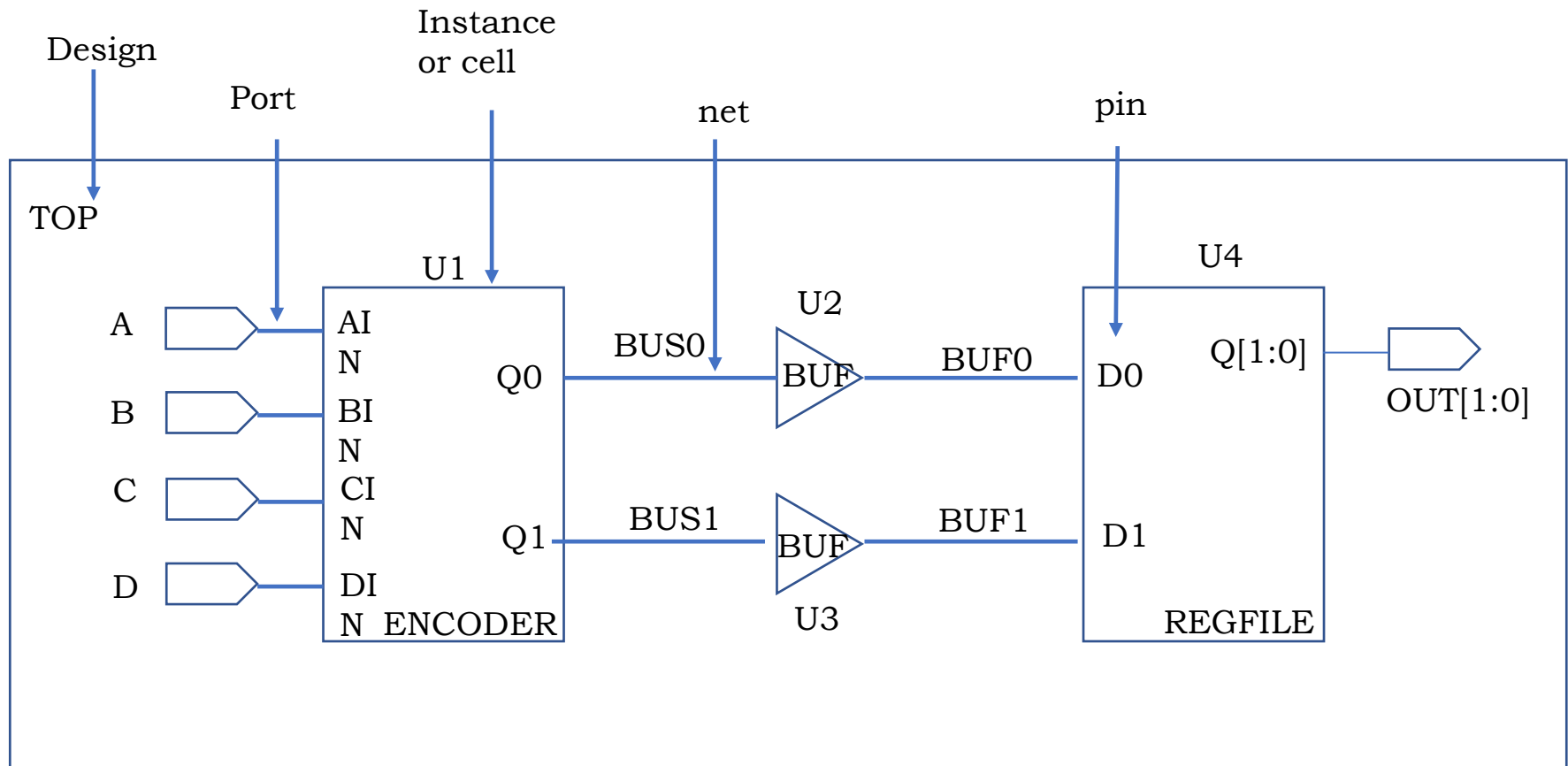


The synthesis flow



- Synthesis – process of generating gate level net-list from HDL
- Optimization – Implements a combination of library cell that best meet the functional, timing area & power requirement.
- Compile – step in the DC that executes the synthesis and optimization step.

- Flat Designs
 - No sub-design and only one structural level
- Hierarchical Designs
 - One or more sub-design which further can have sub-design.



Design {TOP ENCODER REGFILE}
 Reference {ENCODER REGFILE BUF}
 Instance {U1 U2 U3 U4}

- DC modes
 - Wire load mode
 - Topographical mode
 - Multimode
 - UPF mode
- Setup files - .synopsys_dc.setup
 - Synopsys root directory
 - Your home directory
 - Current working directory
- Launching DC
 - % dc_shell (wire load mode)
 - % dc_shell -topographical_mode

Different Options to start DC

```
Unix> dc_shell  
      [-f script_file]  
      [-x command_string]  
      [-checkout feature_list]  
      [-wait wait_time]  
      [-version]  
      [-output_log_file console_log]  
      [-no_log]
```

Example

```
Unix> dc_shell -f synth.tcl -output_log trail.log
```

quit or exit to close DC Session

```
dc_shell> quit
```

Read:

Loading into the memory.

To be used for entering pre-compiled designs or netlists in DC.

```
dc_shell> read_verilog MY_DESIGN.v
```

To see the designs loaded,

```
dc_shell> list_designs
```

Analyze:

Reads an HDL source file. Checks it for errors.

Translates into HDL library objects in an HDL intermediate format and stores the result of

the translation in the specified design library (UNIX directory) that may be used later.

```
dc_shell > analyze -format verilog {top.v adder.v}
```

Elaborate:

Creates a technology-independent design from the intermediate files produced during analyze.

Infers registers in the design.

Performs high level HDL optimization, such as dead code removal

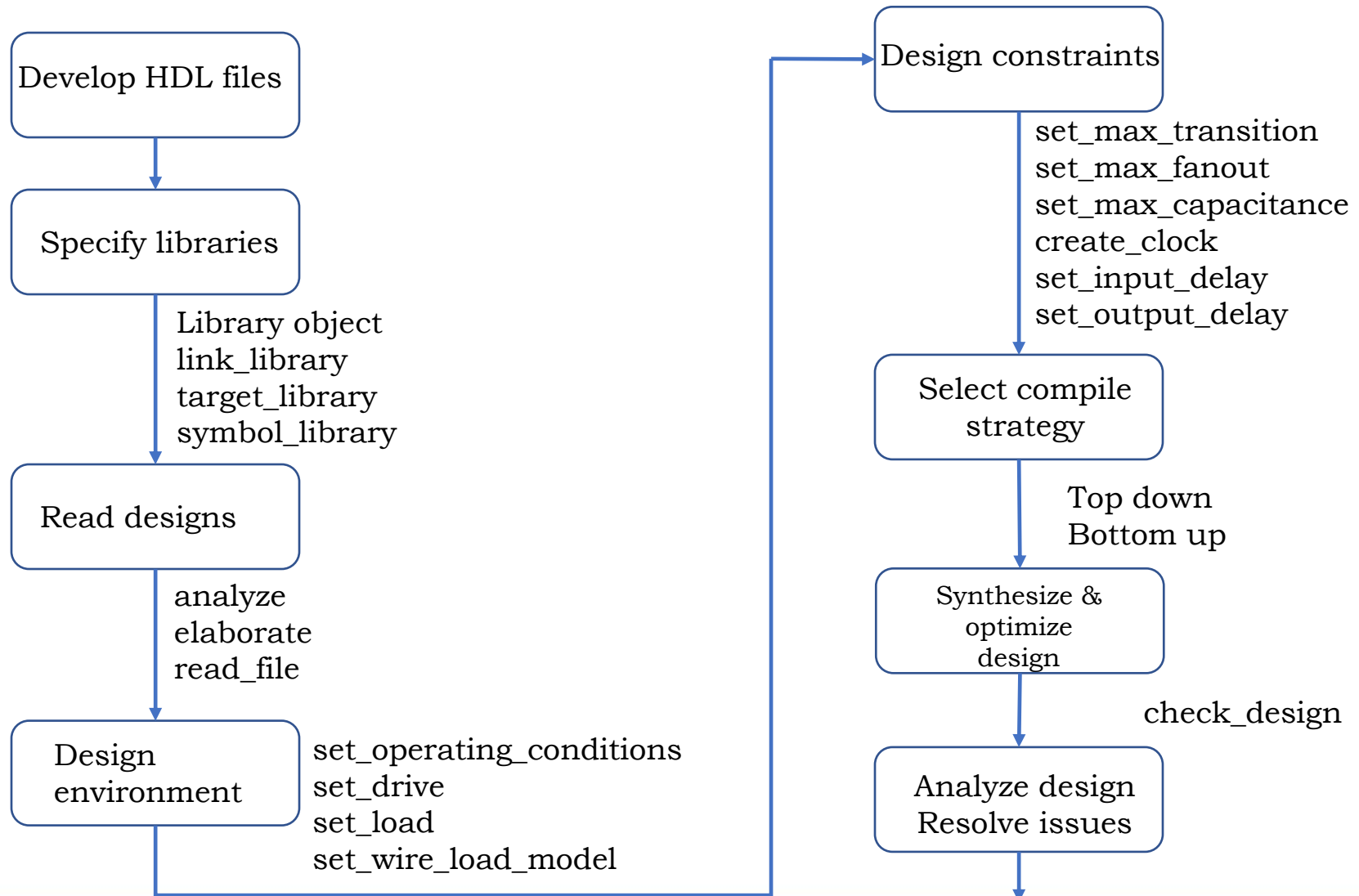
dc_shell > elaborate \${DESIGN_NAME}

Check the quality of the design, for any unresolved references, undriven or multi driven ports, pins and unloaded pins and constant connected ports and pins.

```
dc_shell> check_design  
      [-no_connection_class]  
      [-post_layout]  
      [-ports]  
      [-designs]  
      [-nets]
```

- Unresolved references need to be resolved before proceeding

The synthesis flow



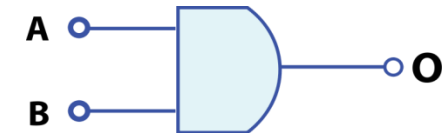
Technology Library

- Selecting semiconductor vendor
 - Maximum frequency of operation
 - Physical restrictions
 - Power restrictions
 - Clock tree implementation
- Library requirement
 - Logic libraries
 - Target library
 - Link library
 - Symbol libraries
 - Designware libraries

- DC uses logic library for the following purpose
 - Implementing the design function.
 - Resolving cell references.
 - Calculating timing values and path delays.
 - Calculating power consumption.

Excerpt from a Technology Library:

```
Cell (AND2_3 ) {  
  area : 8.000 ;  
  pin (O) {  
    direction : output;  
    timing ( ) {  
      related_pin : "A";  
      timing_sense : positive_unate ;  
      rise_propagation (drive_3_table_1) {  
        values ("0.2616, 0.2608, 0.2831,..")  
      }  
      rise_transition (drive_3_table_2) {  
        values ("0.0223, 0.0254,...")  
        . . . .  
      }  
    }  
    function : "(A & B)" ;  
    max_capacitance : 1.14810 ;  
    min_capacitance : 0.00220 ;  
  }  
  pin (A) {  
    direction : input;  
    capacitance : 0.012000;  
    . . . .  
  }  
}
```



Pin A → Pin O nominal delays (look-up table)

Pin O functionality

Design Rules for Output Pin

Electrical Characteristics of Input Pins

- Target Libraries
 - DC selects functionally correct gates from the target libraries.
 - Target libraries are specified by target_library.
 - Shouldn't specify any designware libraries or macro libraries.
- Link Libraries
 - Used to resolve references.
 - Specified by link_library

- Contains definition of graphics symbol that represent library cells in the design schematics
- Used to generate schematic views.
- Specified by `symbol_library`
- If no symbol library are specified, DC searches `generic.sdb` in DC installation area. If no match found, DC displays cell instance as rectangle

- Collection of reusable circuit design building blocks that are integrated in Synopsys synthesis environment
- During synthesis, DC selects the component with the best speed and area.
- DW components implements built-in operator, for e.g., +, -, *, <=, >=, and the operations defined by if and case statement. These component are specified in standard.sldb and DC automatically uses it.
- DW components are specified by synthetic_library, but it requires DesignWare license.

- Launching DC and settings required variables
- Reading design & loading libs
- Understanding target, link & Design-Ware libraries.
- Sanity checks of Design & Libraries.

HDL Coding Overview

- **Behavioral coding:** All Verilog constructs are used. Typically used for simulating and testing of functionality

```
initial begin
```

```
y = a & b;
```

```
end
```

- **RTL coding:** Subset of Verilog constructs are used. Typically used for expressing a functionality that is intended to translate into hardware

```
assign y = a & b;
```

- **Structural coding:** Uses Verilog primitives or actual library cells. Typically used to represent exact hardware in net-lists

```
AND2X1 and_i (.Y(y), .A(a), .B(b));
```


- Most of the HDL constructs cannot be realized as hardware
- Synthesis tool is built to interpret a sub-set of verilog constructs to deduce the hardware
- Specific coding style synthesizes to specific hardware

For Example:

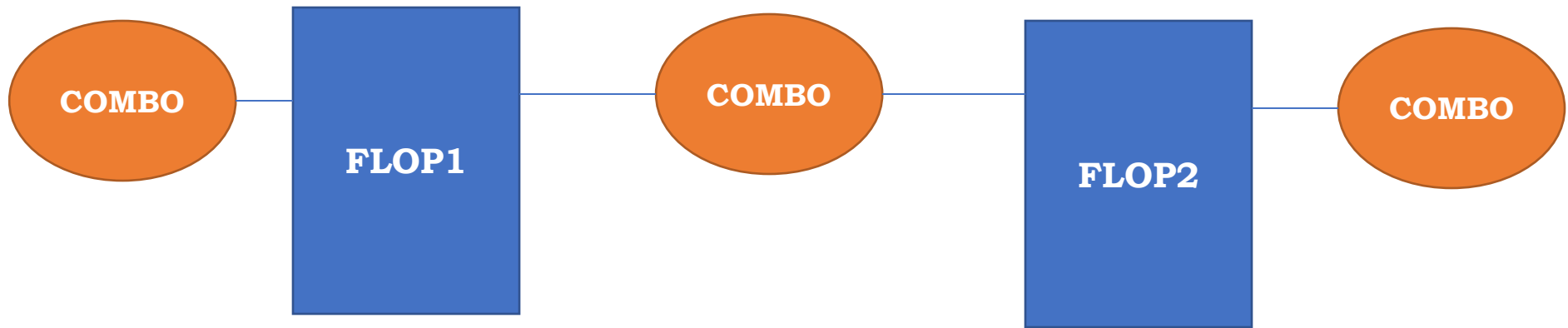
#10 ; Cannot be realized

\$display; Cannot be realized

initial; Cannot be realized

```
module name(port_name1, port_name2,.....);  
input [bus_width:0] port_name1;  
output port_name2;  
:  
wire a, y;  
:  
BUF1X1 buf_instance1 (.Y(y), .A(a));  
:  
:  
ip1 ip_instance1 (.....);  
endmodule
```

- All RTL designs are made up of the following structure



- Repeated millions of times !!! (Of course the flops/combo vary)

Some synthesis Examples

assign $y = a \wedge b$; (Synthesizes to XOR gate)

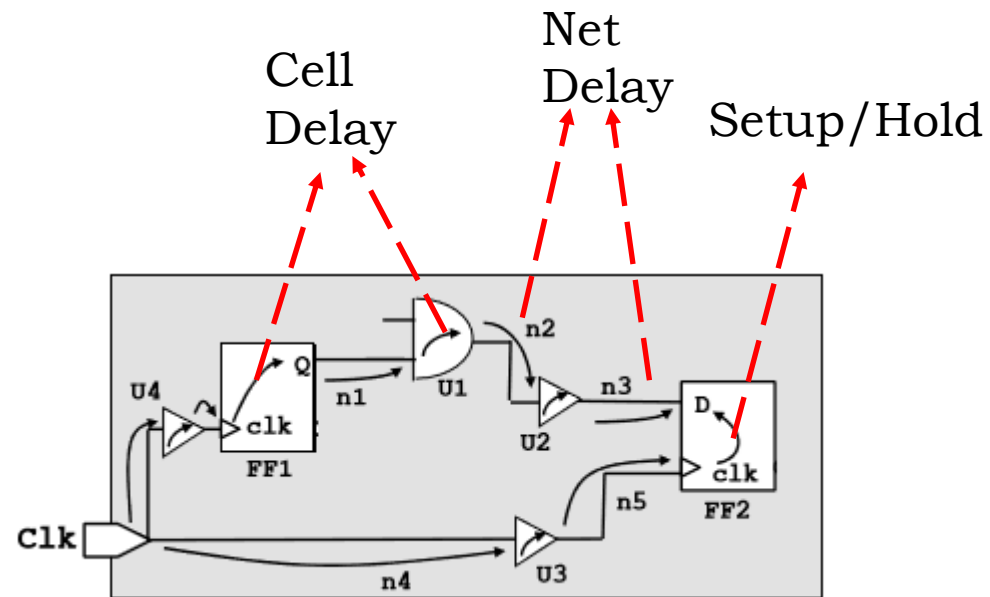
assign $y = a \& b$; (Synthesizes to AND gate)

assign $y = s ? a : b$; (Synthesizes to MUX cell)

Timing Concepts

Component of circuit timing

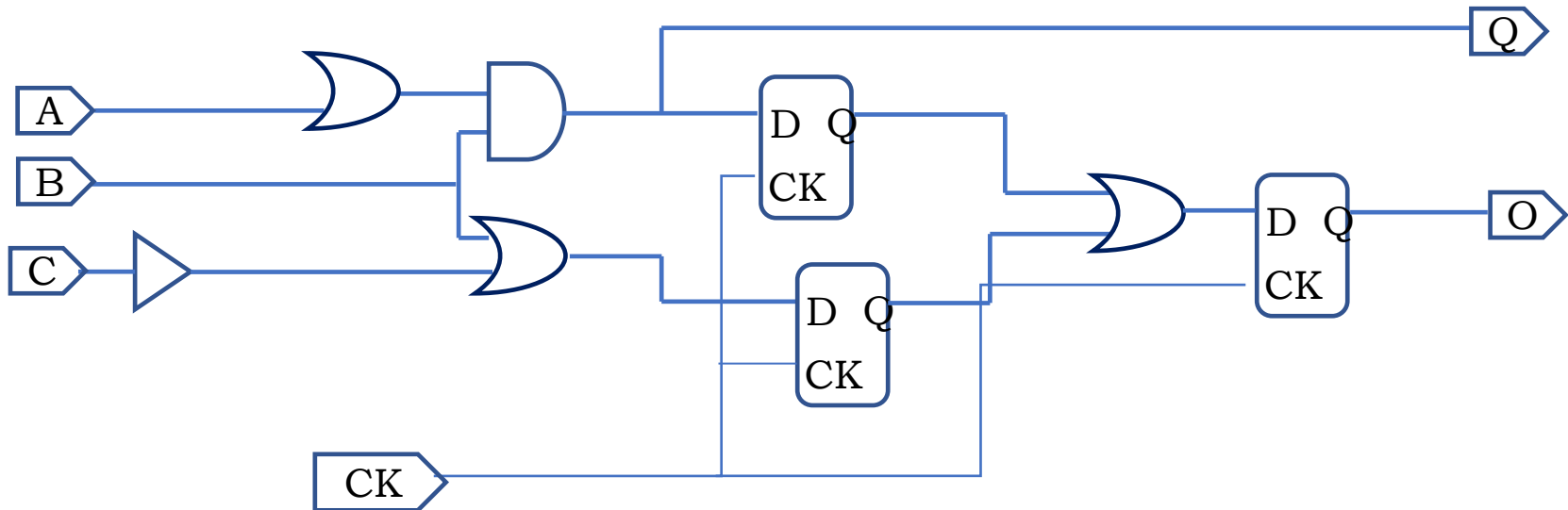
- Delay component
 - Cell delay
 - Net Delay
- Constrained Component
 - Setup/hold
 - Recovery/removal



For a circuit to operate safely, it is required that delay component at no point violate constraints of other component

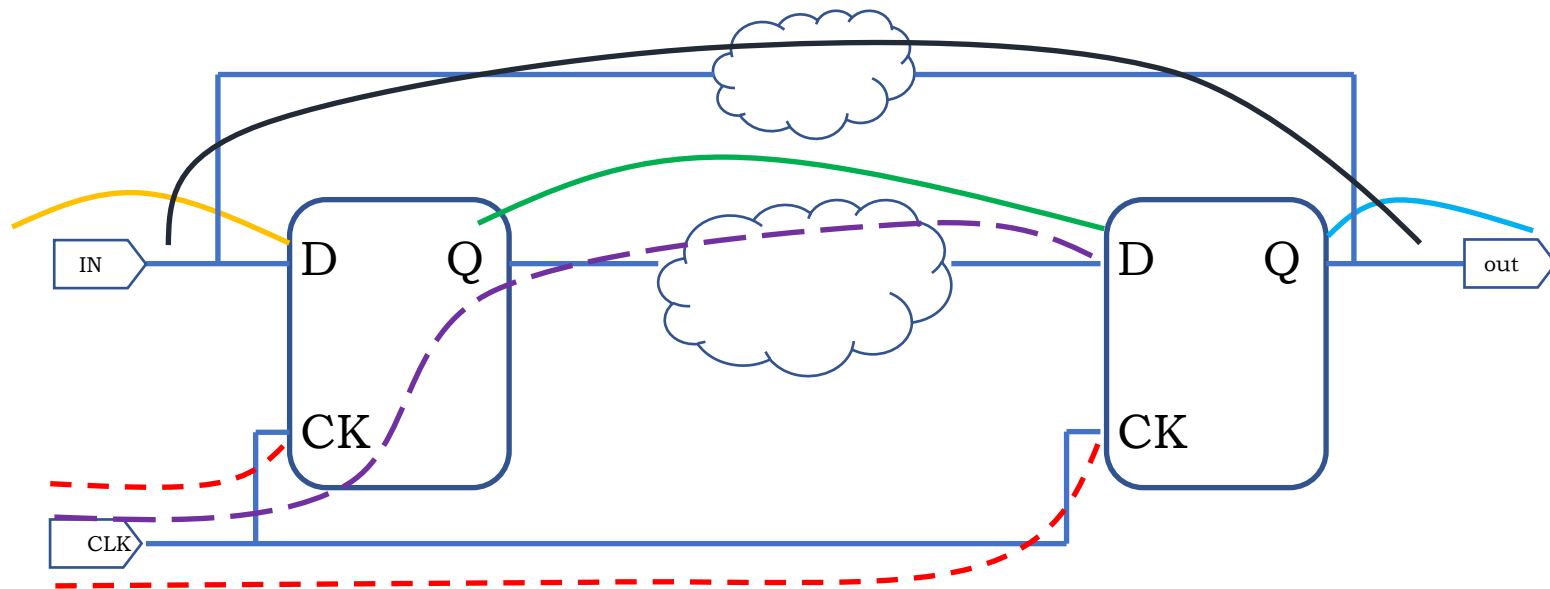
Start/End Points

- Timing Path Start-point includes
 - Input ports
 - Clock pins of flops
- Timing Path End-points includes
 - All the input pins of sequential elements except clock
 - All the output ports



Types of Timing Paths

- Input to Register
- Register to Register
- Register to output
- Input to output

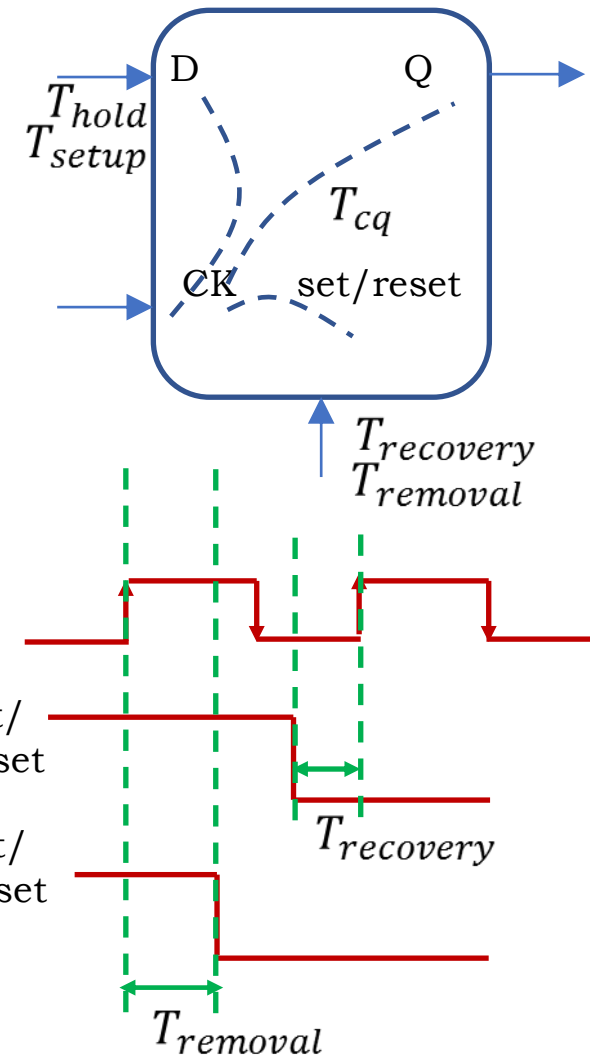
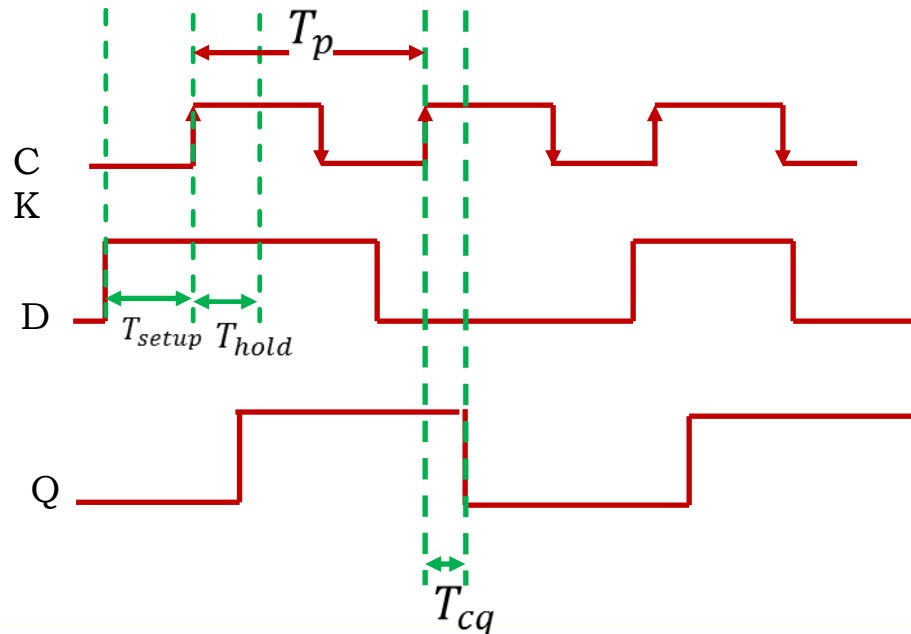


--- DATA LANE
--- CLOCK LANE

— IN2REG
— REG2REG
— OUT2REG
— IN2OUT

Sequential Timing Arcs

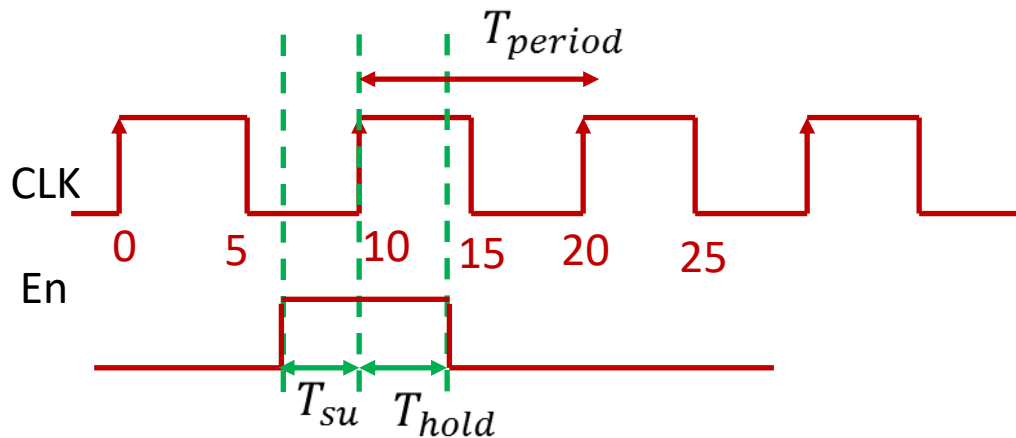
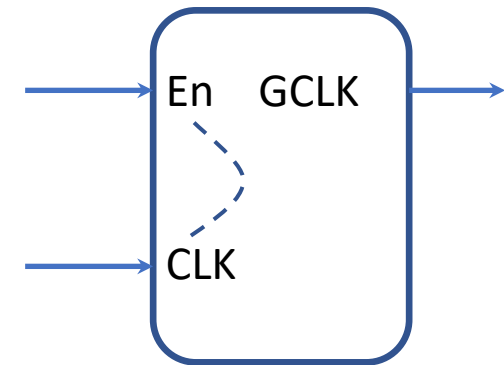
- Sequential Cells Timings (FF)
 - Cell Delay (T_{cq})
 - Setup/hold
 - Recovery/Removal



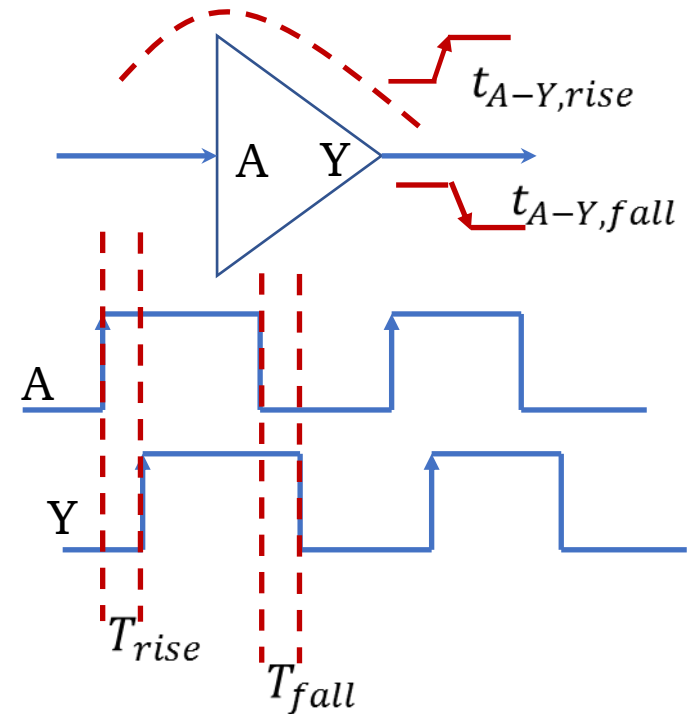
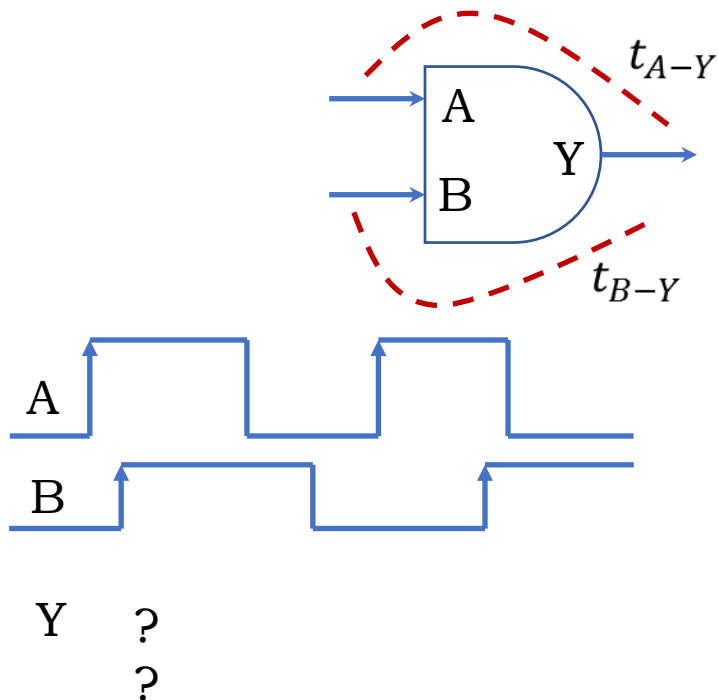
- Input pin “D” is synchronous to CK pin, there are following timing arcs
 - Setup, hold
- Input pins “set” (or reset) is asynchronous, there are following timing arcs
 - Recovery, removal

Clock Gate – Timing Arcs

- Sequential cell timings (clock gate)
 - Setup/hold timing arc for En w.r.t CLK
 - GCLK is gated clock
 - It appears in clock path only

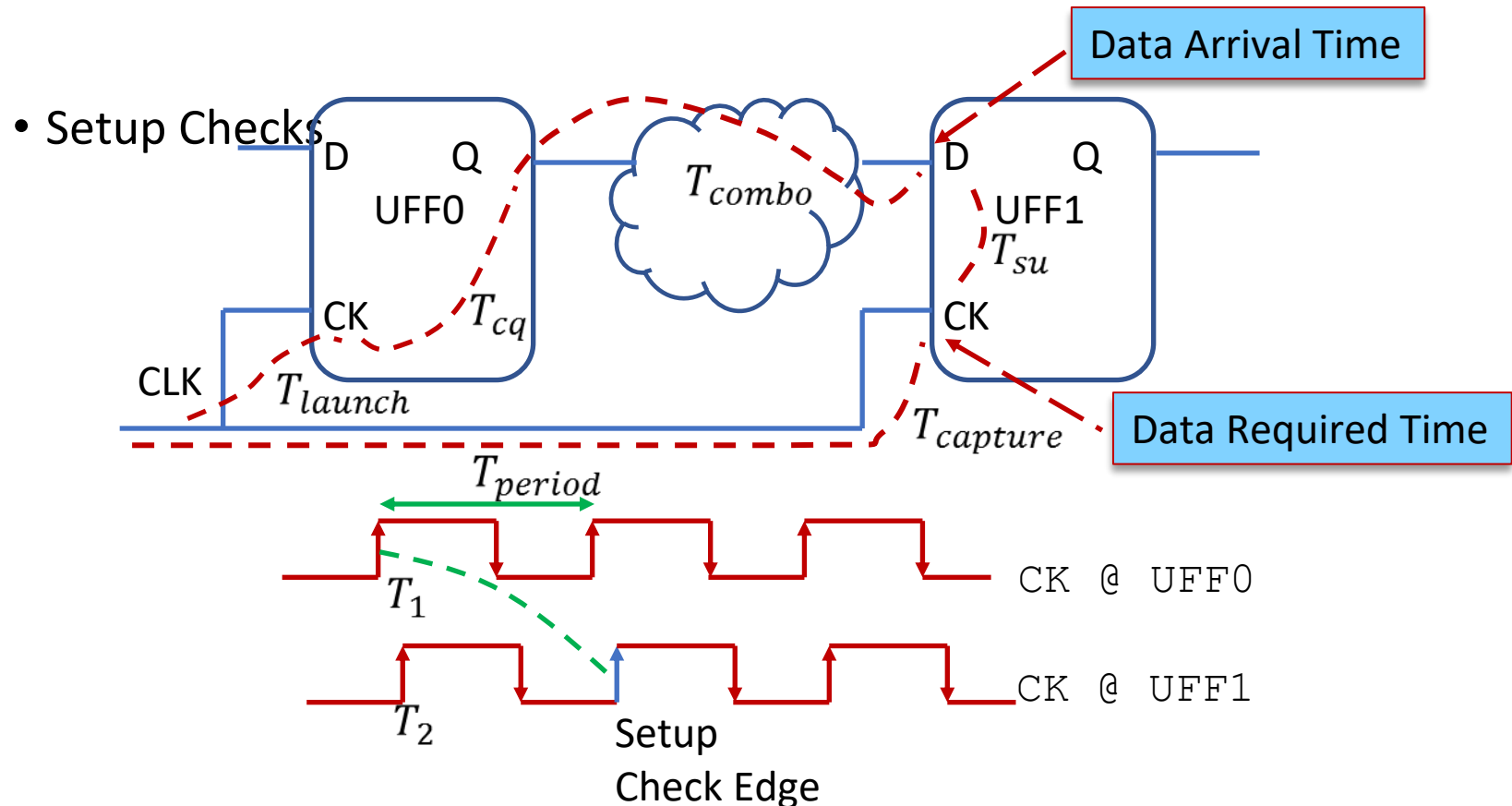


- Combinational Cells Timings
 - Cells delay (Rise delay, Fall delay)
 - Combinational timing arc



- Timing Checks – Once clock is defined, design is ready for timing checks.
- Three types of timing checks
 - Synchronous Timing Checks
 - Setup, hold
 - Asynchronous Timing Checks
 - Recovery, Removal, min pulse width
 - Clock gating checks

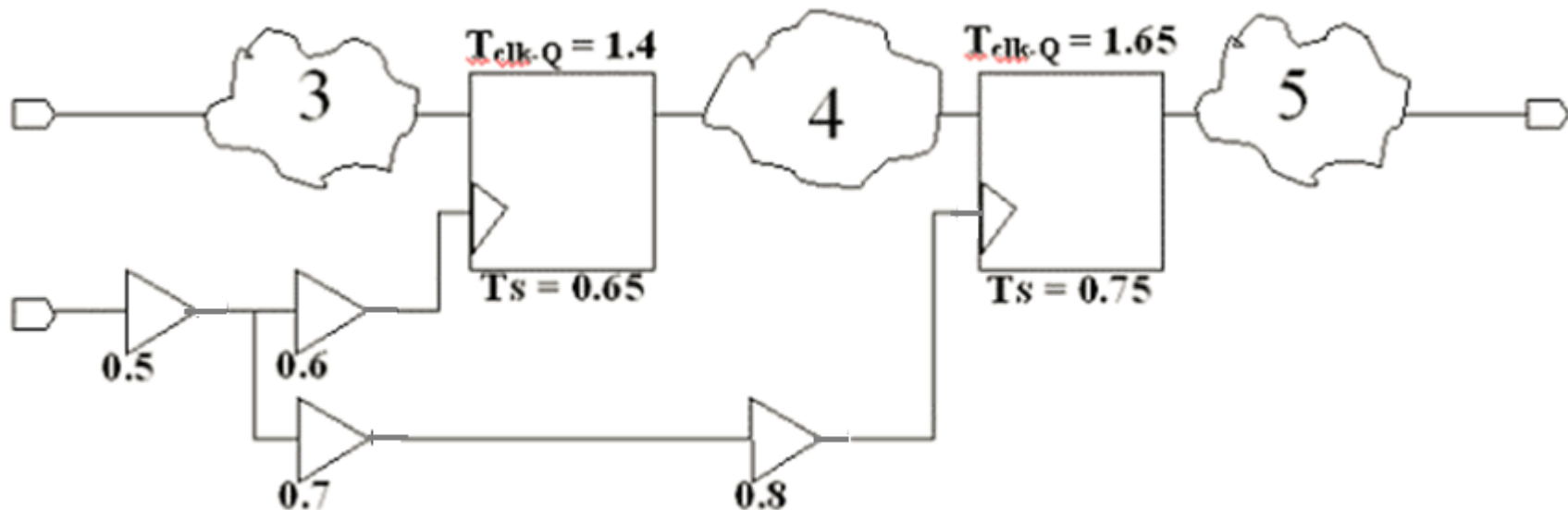
Synchronous Timing Checks



For setup, $T_{launch} + T_{cq} + T_{combo} + T_{su} \leq T_{period} + T_{capture}$

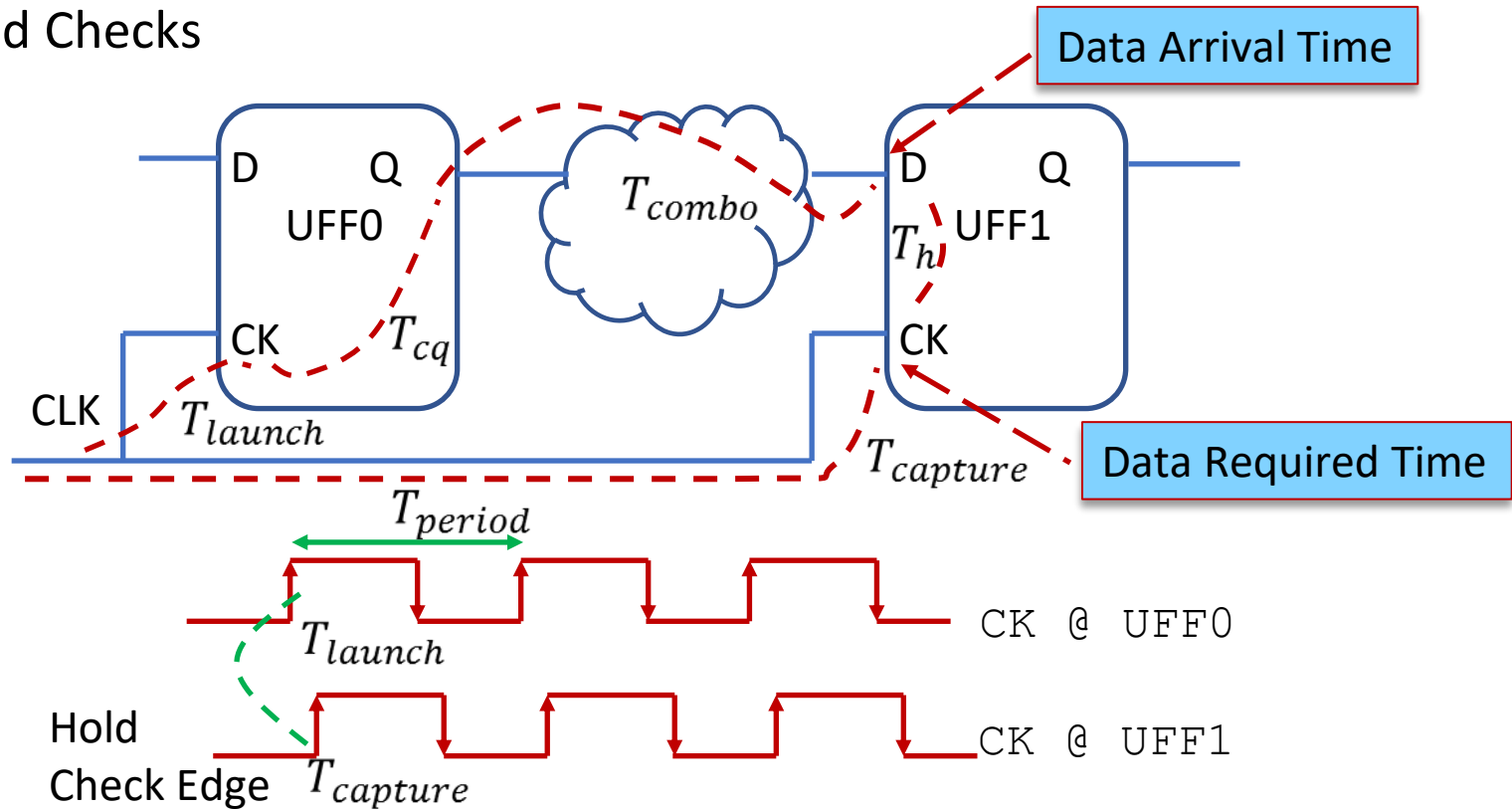
Maximum frequency calculation example

Q: Find maximum frequency of operation of below circuit?



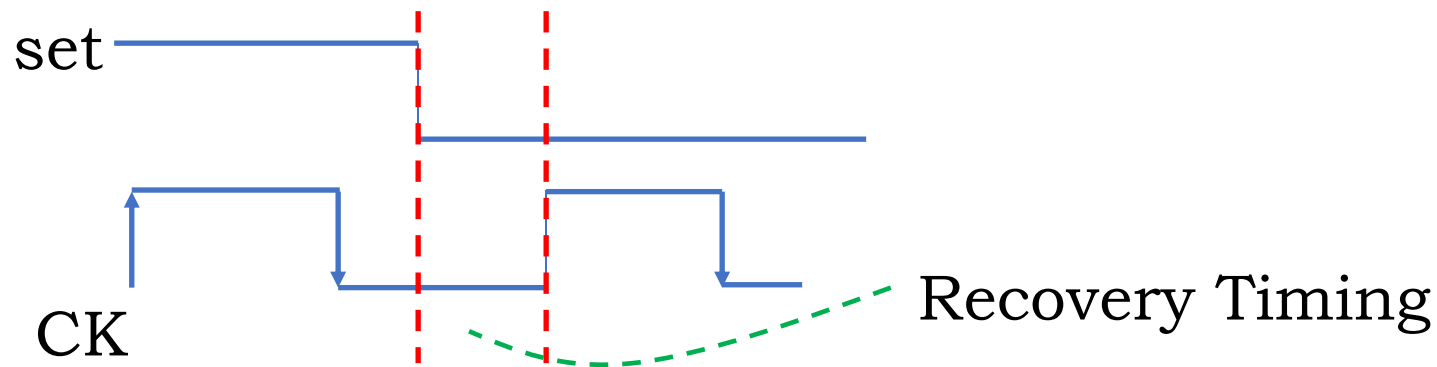
Synchronous Timing Checks

- Hold Checks

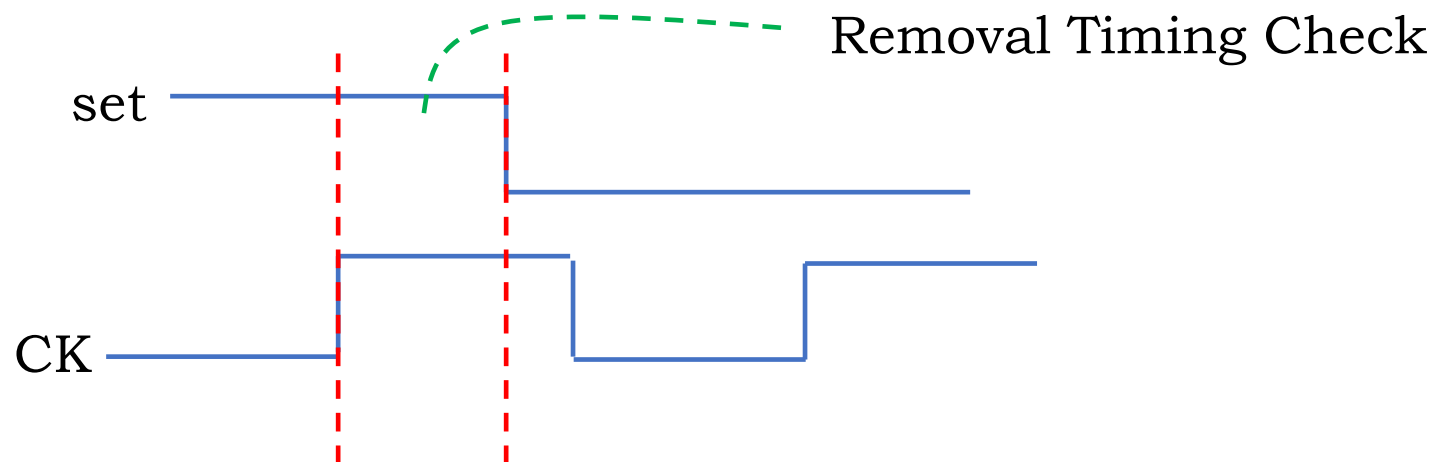


$$\text{For hold, } T_{launch} + T_{cq} + T_{combo} \geq T_h + T_{capture}$$

- Recovery Checks
 - Timing paths to “set” (or reset) pins
 - When “set” is asserted, Q changes to “1”
 - Even “set” is asynchronous to “CK”, but there is minimum amount of time between the asynchronous signal becoming inactive and the next active clock edge, called recovery timing.

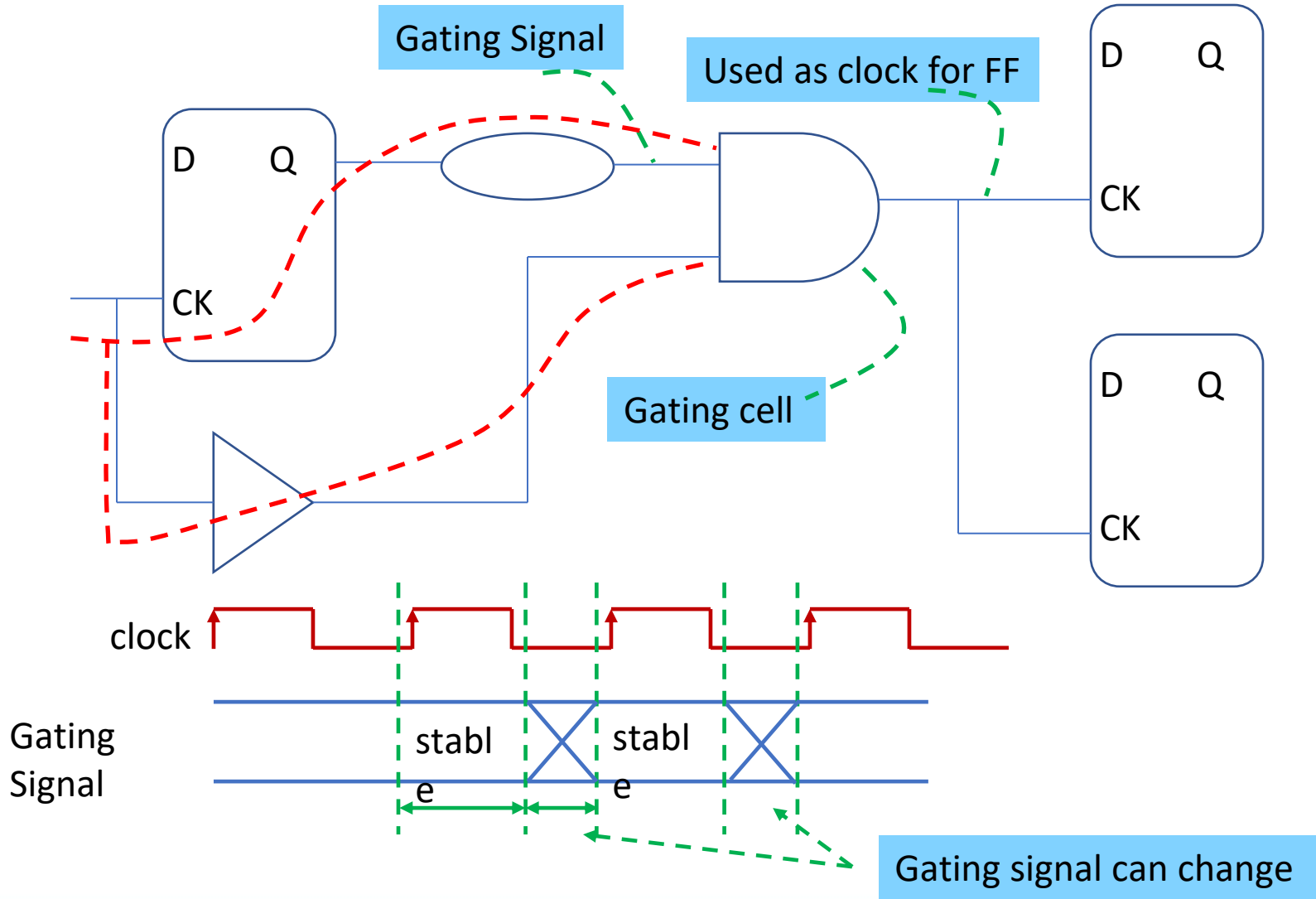


- Removal timing check ensures that there is adequate timing between an active clock edge and release of an asynchronous control signal

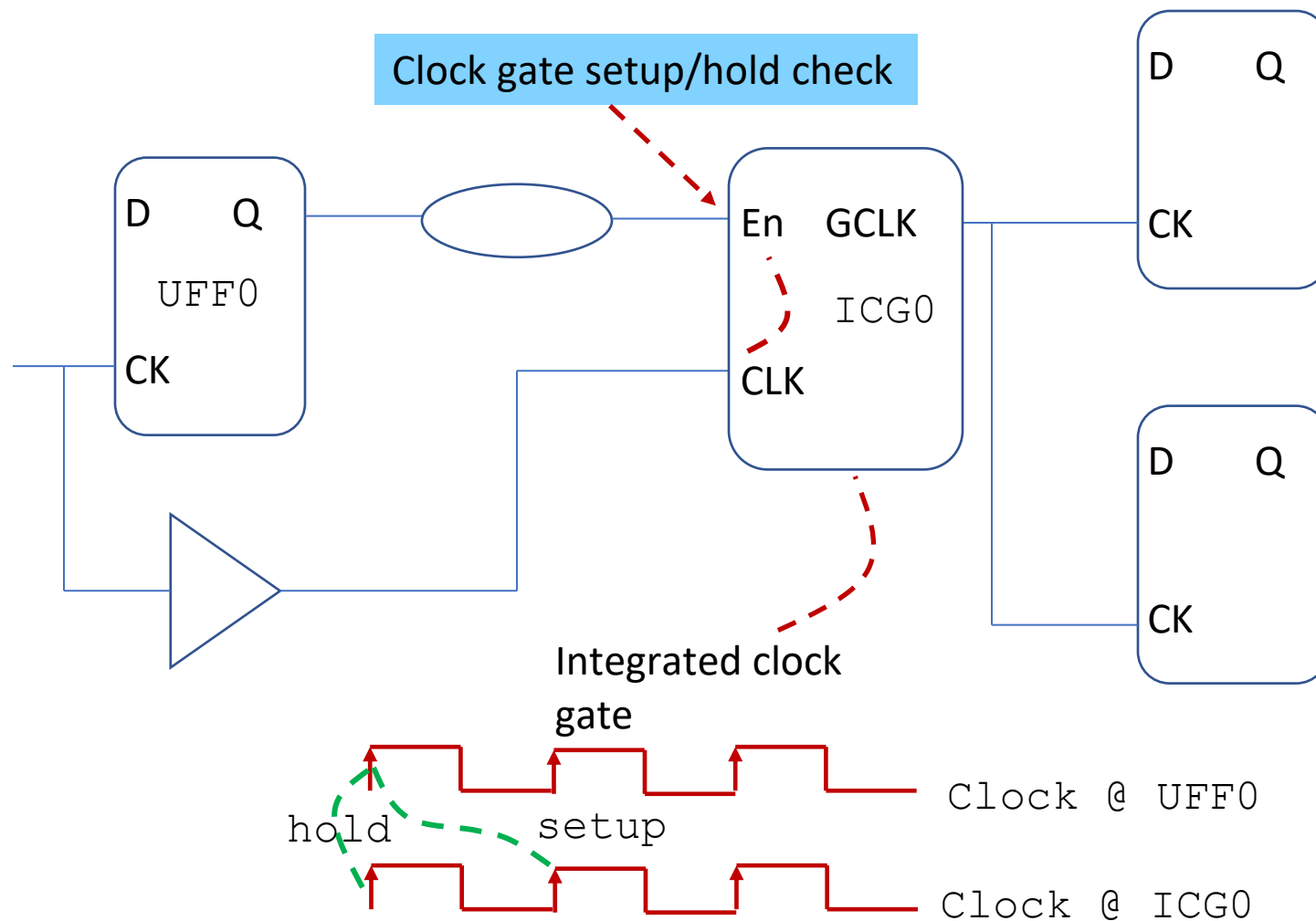


- Occurs when a gating signal control the path of clock signal at a logic level.
- Clock gating check occurs when clock that goes through the cell is used as clock downstream.

Clock gating check without ICG



Clock gating check with ICG



- Min period
- Min pulse width
- Design rule violations (DRV)
 - Max transition
 - Max capacitance
 - Max fan-out

- Synthesis overview & Flow
- Synthesis Flow using Design Compiler
- Understanding technology library
- HDL coding guidelines for Synthesis
- Timing Checks

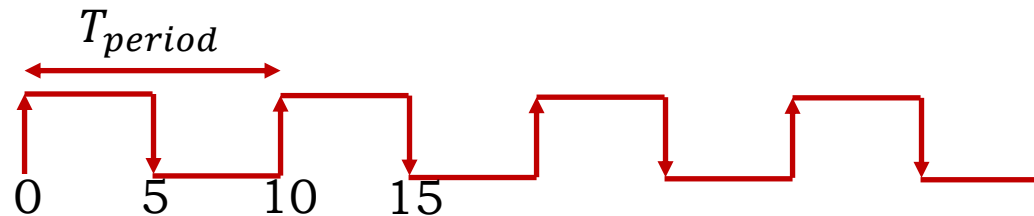
Synthesis 2

- Synthesis
 - Design Compiler User Guide – Synopsys
 - Advanced ASIC chip synthesis – Himanshu Bhatnagar
- Equivalence Checking
 - Formality User Guide – Synopsys
- Static Timing Analysis
 - Static Timing Analysis for Nanometre Designs – J.Bhaskar, Rakesh Chadhha.

Constraining the Design

- Clock definition
 - To constrain data lane w.r.t clock lane.
 - Defined on the port/pins which connects to the clock pins of sequential elements
 - DC command to create clocks “create_clock”

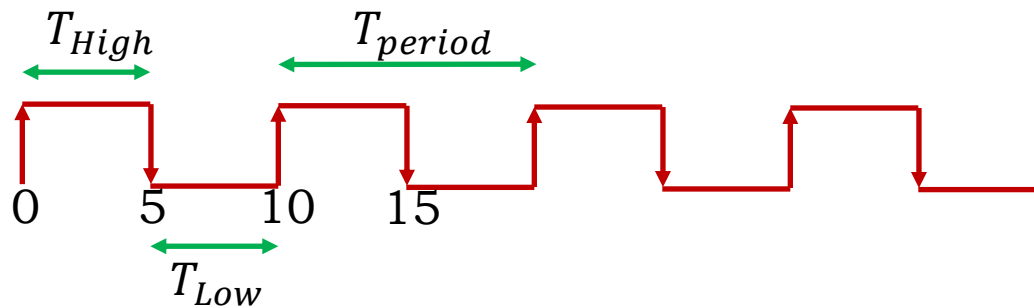
create_clock -period 10 -name HCLK -waveform {0 5} [get_ports clk]



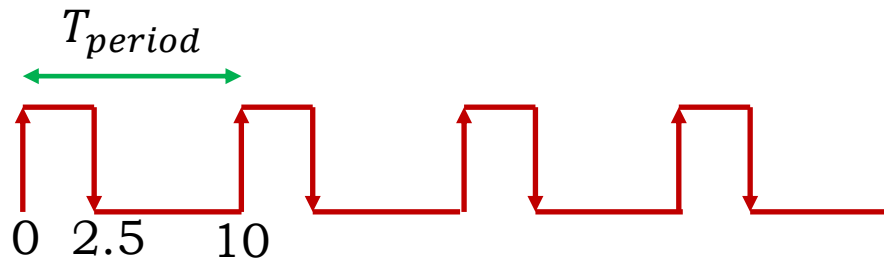
- For each clock created, a path group will be created

Clock Attributes – Duty Cycle

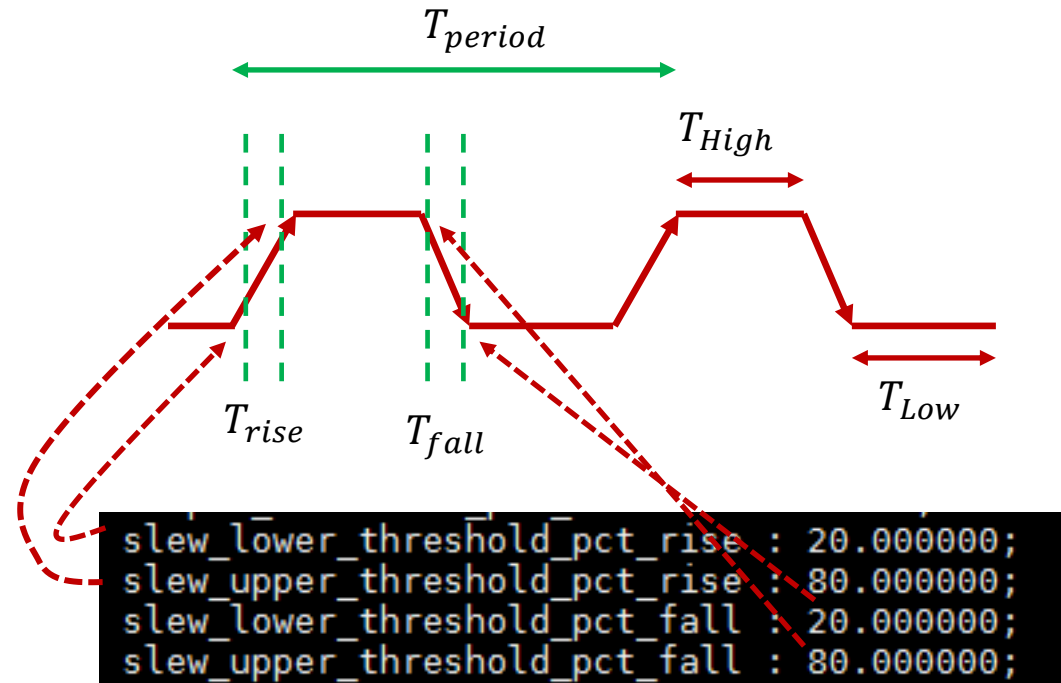
$$\text{Duty Cycle} = \frac{T_{High}}{T_{period}} \times 100 = 50\%$$



Creating clock with 25% duty cycle –
create_clock PHI2 -period 10 -waveform {0 2.5}



- Clock transitions
 - set_clock_transition

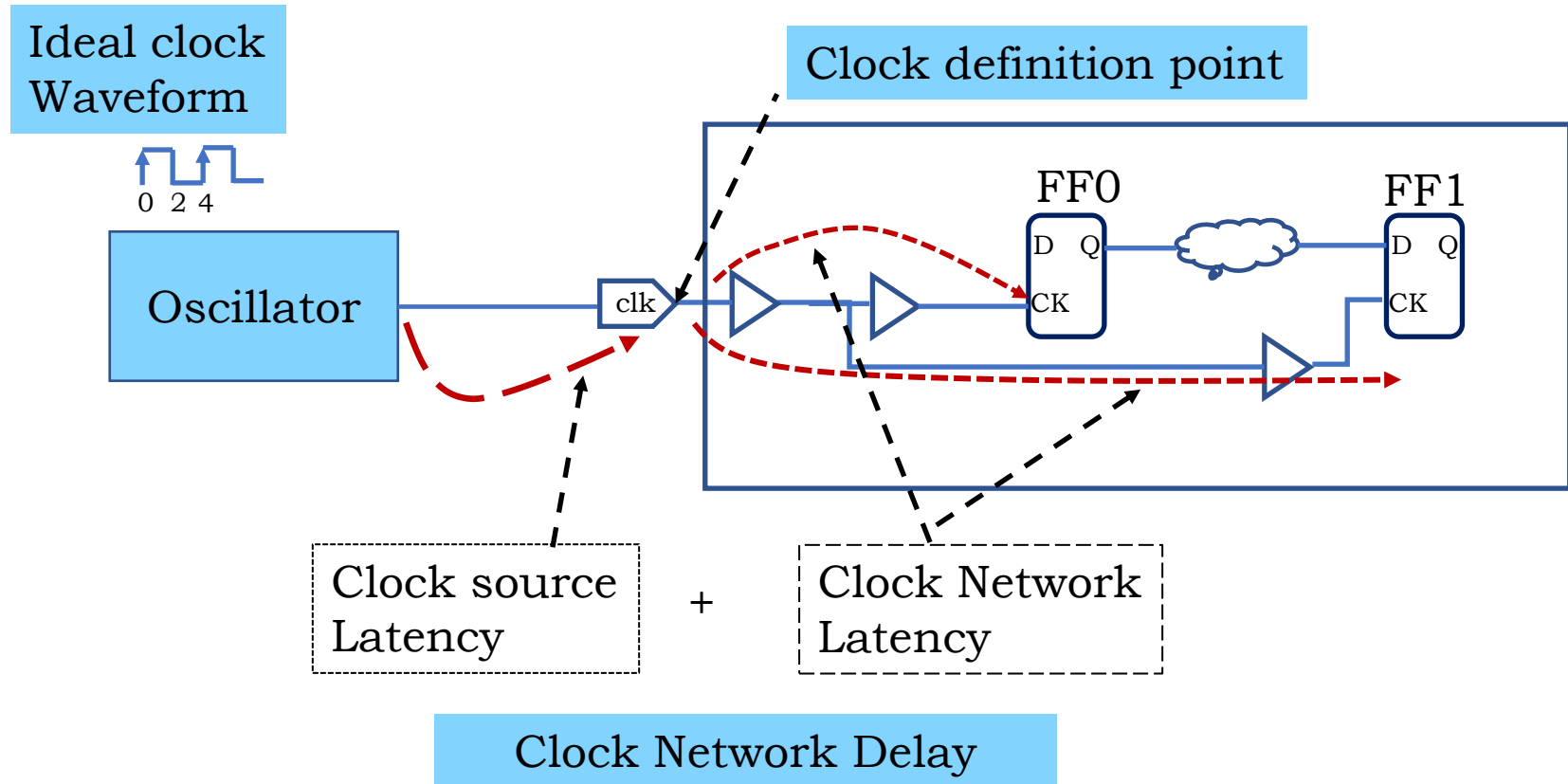


```
dc_shell> set_clock_transition 0.38 -rise [get_clocks CLK1]
```

```
dc_shell> set_clock_transition 0.25 -fall [get_clocks CLK1]
```

Note : This is useful only in pre-layout mode. Post-CTS, Timing Engine shall calculate it.

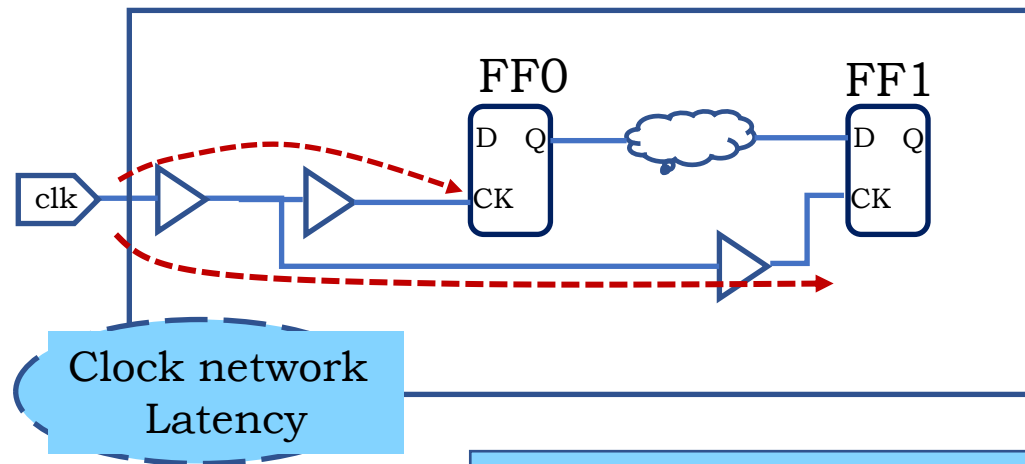
Clock Attributes – Clock Latency



```
dc_shell> set_clock_latency 1 [get_clocks clk]
dc_shell> set_clock_latency 0.4 -source [get_clocks clk]
```

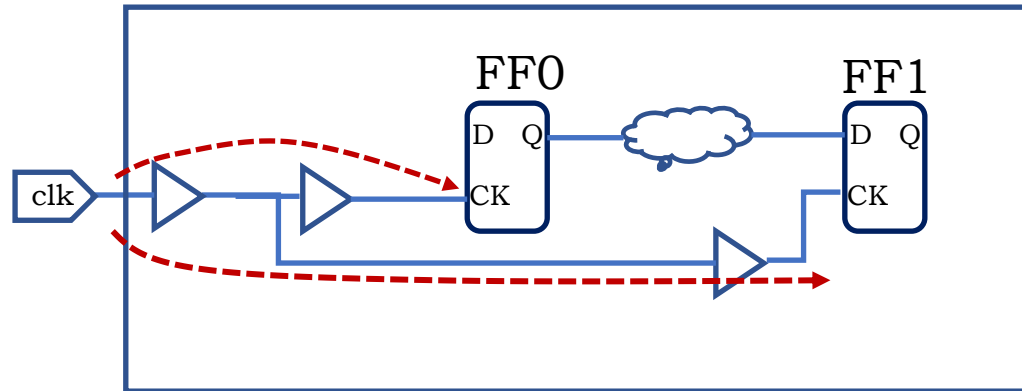
Clock Latency – Pre Vs Post CTS

Pre CTS, user specifies clock latency.
Clocks are ideal.



Post CTS, STA tool calculates clock latency.
Clocks are propagated.

```
dc_shell> set_propagated_clock [all_clocks]
```

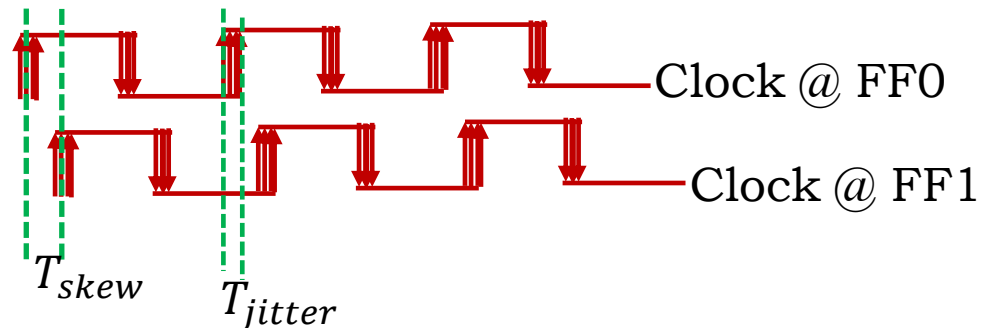


Pre CTS

Clock uncertainty = clock jitter + clock skew + margin

Post CTS

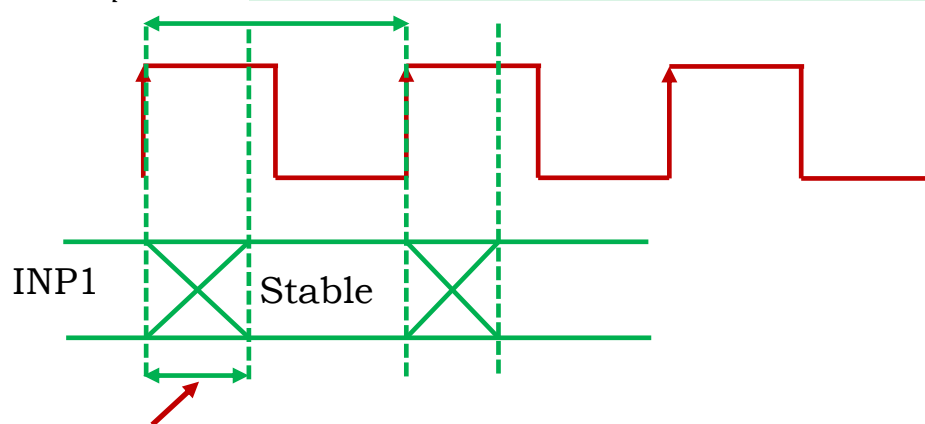
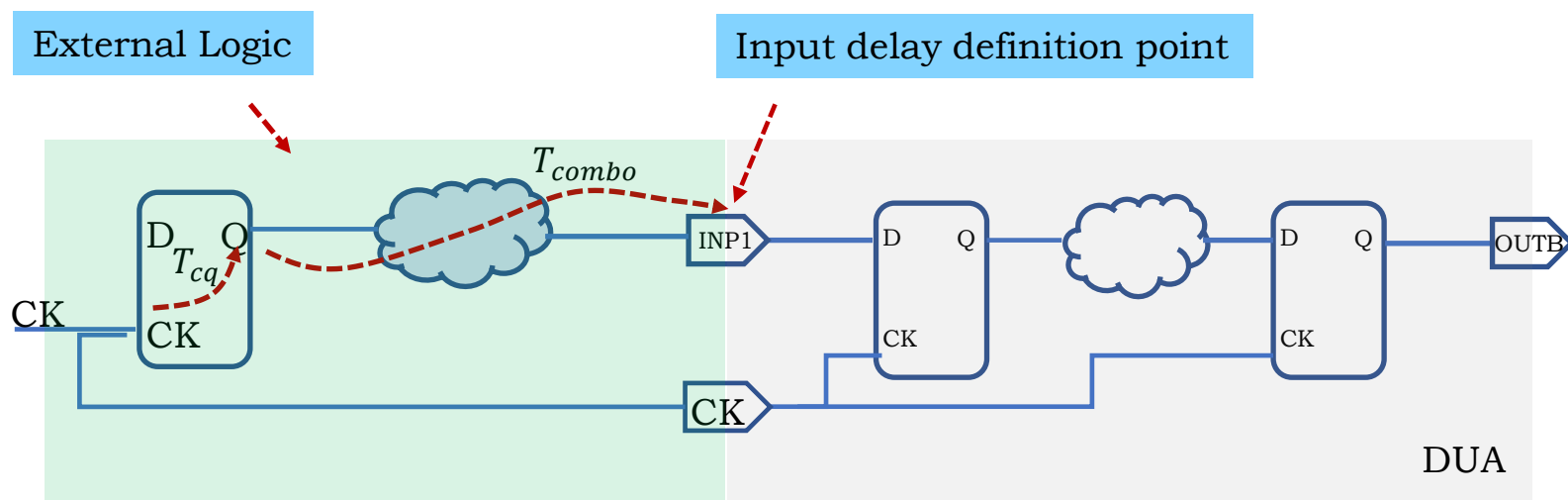
Clock uncertainty = clock jitter + margin



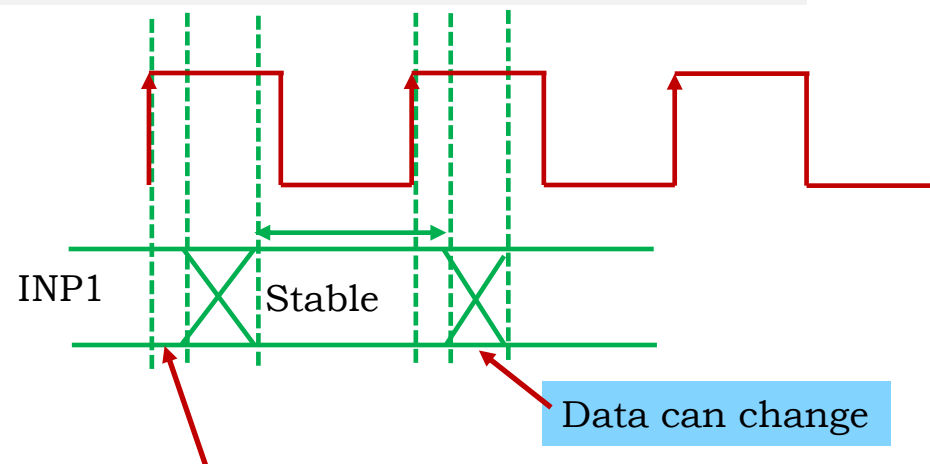
```
dc_shell> set_clock_uncertainty 0.4 [get_clocks clk]
```


- IO paths are either in2reg, in2out or reg2out.
- By default these paths are not constrained
- In order to check the timings on these IO paths, input/output ports should be constrained w.r.t to clock.
- Equivalent DC command :
 - set_input_delay
 - set_output_delay

Constraining Input paths

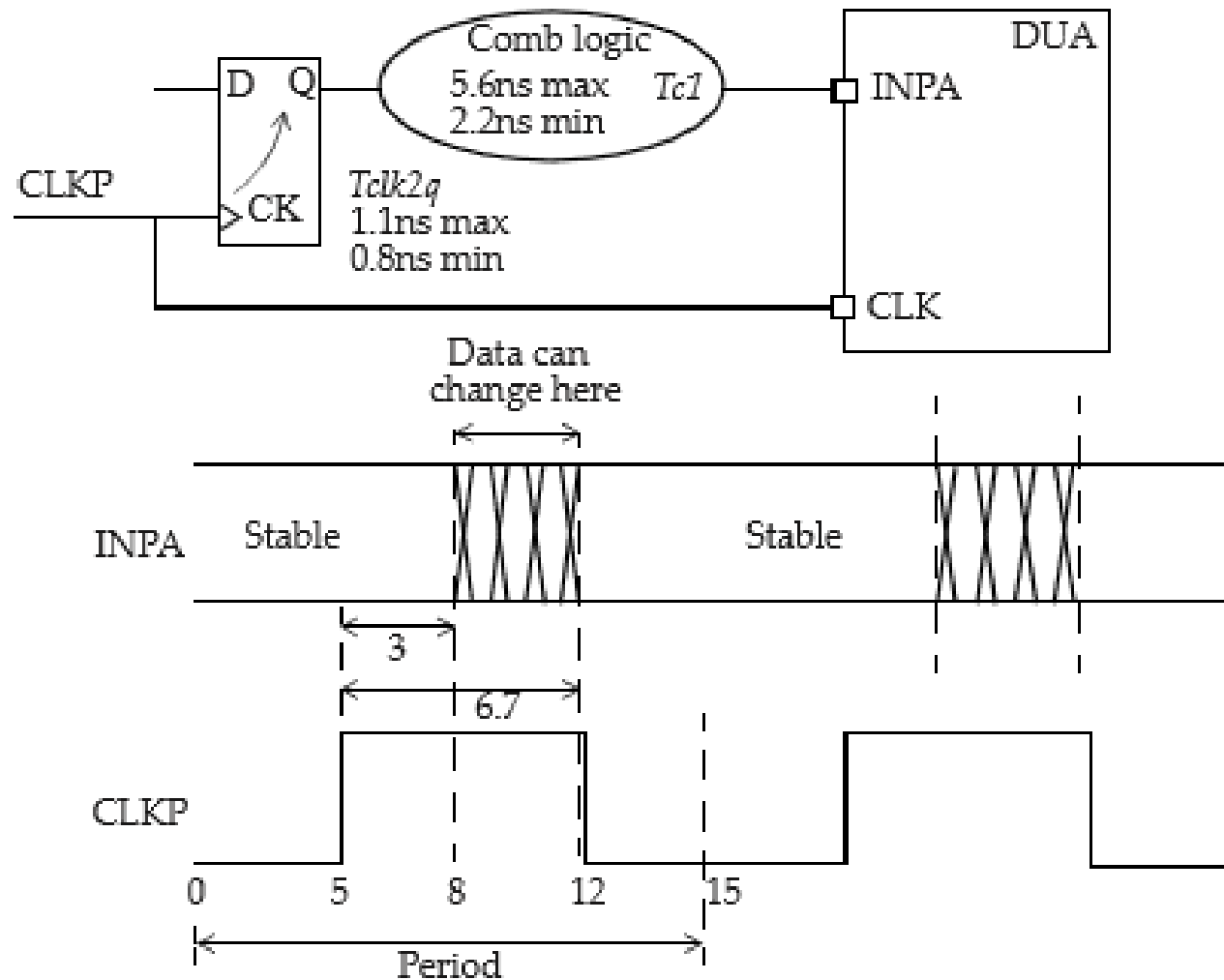


Input Delay (max) = $T_{cq,max} + T_{combo,max}$
Data can change in this region



Input Delay (min) = $T_{cq,min} + T_{combo,min}$

Constraining Input paths



Constraining Input paths

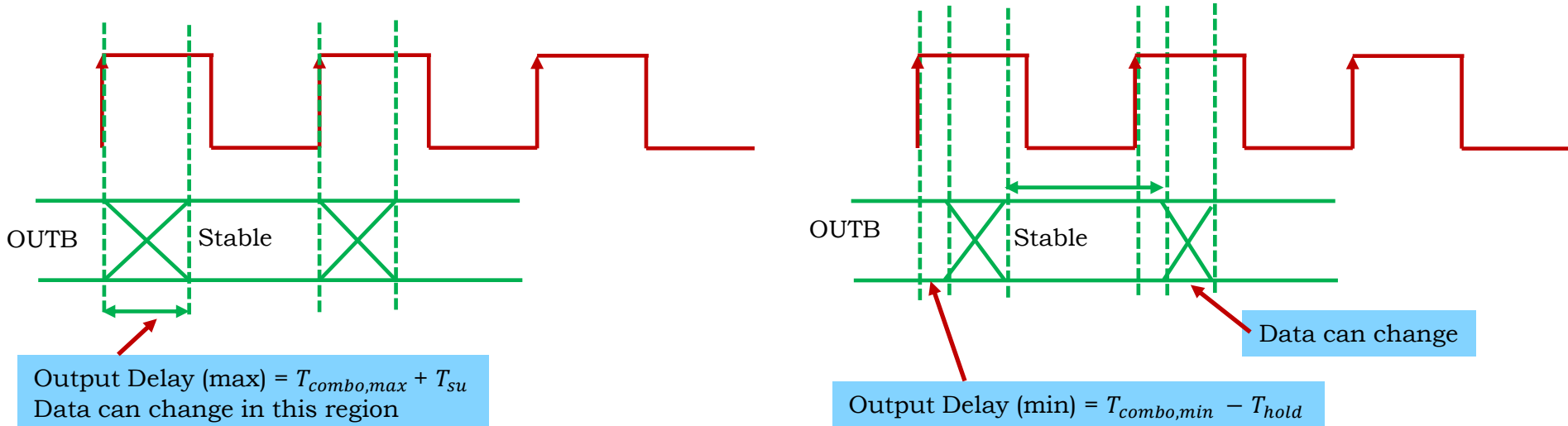
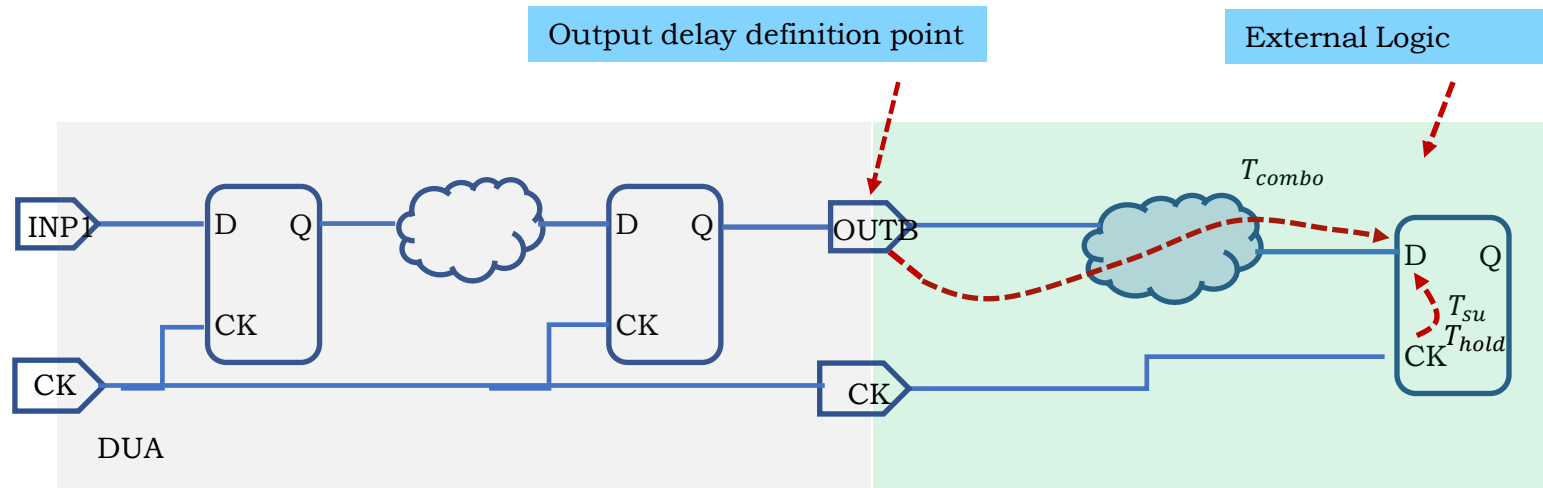
$$T_{cq,max} = 1.1ns \quad T_{combo,max} = 5.6ns \quad T_{input,max} = 6.7ns$$

$$T_{cq,min} = 0.8ns \quad T_{combo,min} = 2.2ns \quad T_{input,min} = 3.0ns$$

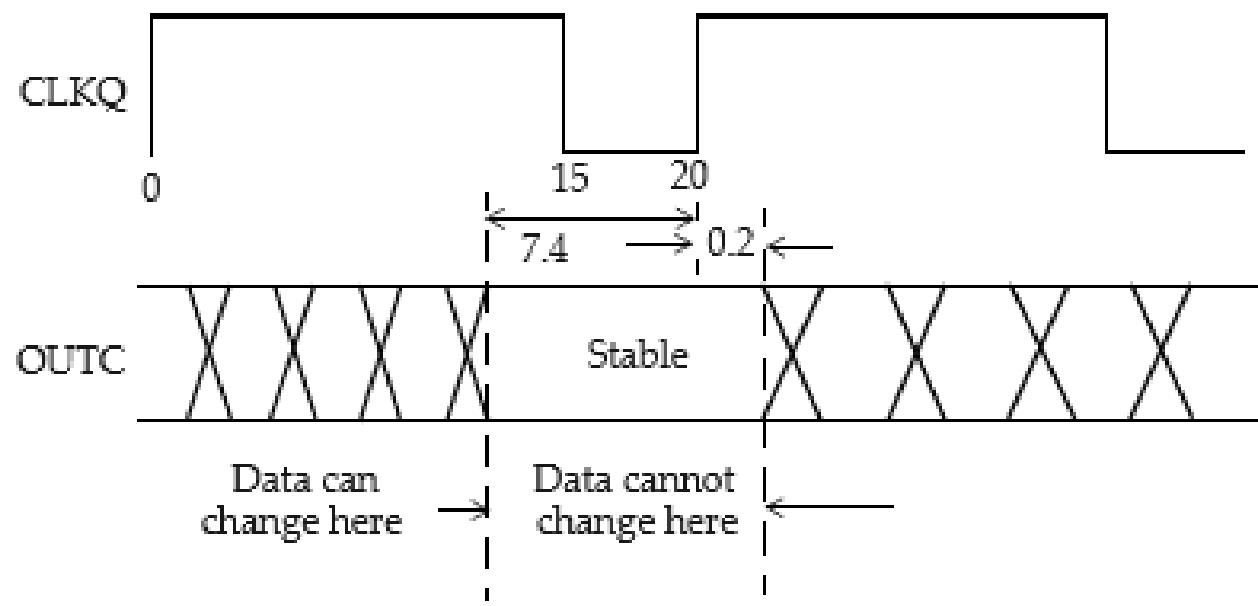
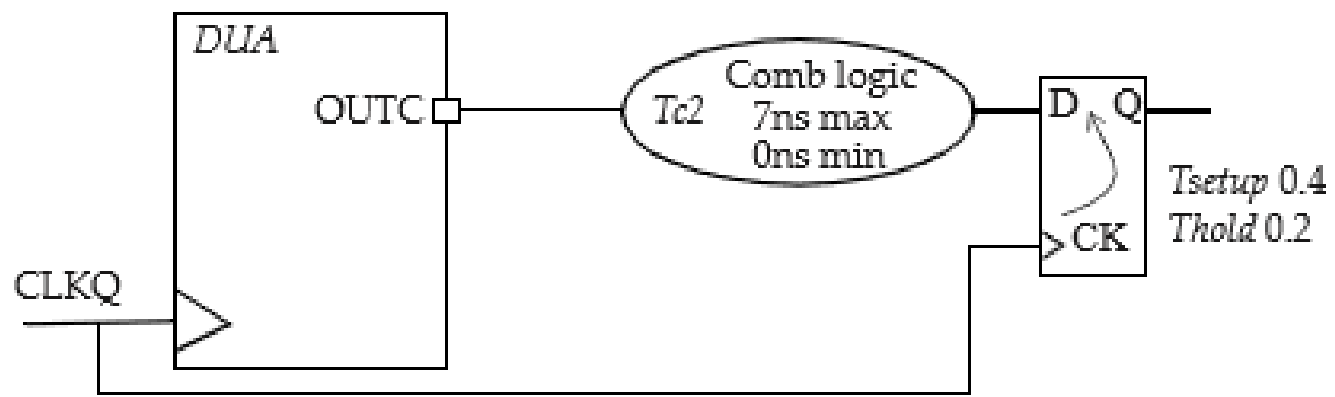
```
dc_shell> set_input_delay 6.7 -clock CLK [get_ports INPA]
```

```
dc_shell> set_input_delay 3.0 -clock CLK -min [get_ports INPA]
```

Constraining Output paths



Constraining output path



Constraining output path

$$T_{combo,max} = 7.0ns \quad T_{su} = 0.4ns \quad T_{output,max} = 7.4ns$$

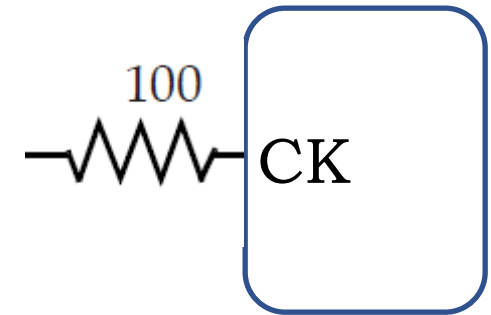
$$T_{combo,min} = 0.0ns \quad T_{hold} = 0.2ns \quad T_{output,min} = -0.2ns$$

```
dc_shell> set_output_delay 7.4 -clock CLKQ [get_ports OUTB]
```

```
dc_shell> set_output_delay -0.2 -clock CLKQ -min [get_ports OUTB]
```

- create_clock, set_input_delay & set_output_delay are sufficient for constraining all the paths in the design, but not enough to get accurate timing analysis.
- The following attribute are also required in order to model the external environment of design accurately
 - set_drive,
 - set_driving_cell,
 - set_input_transition,
 - set_load

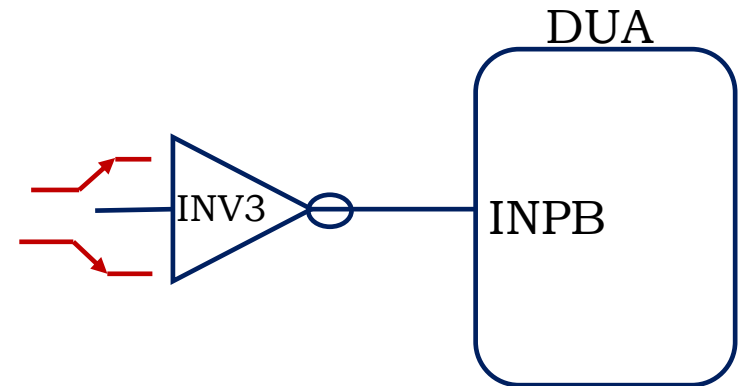
Sets the resistance to a specified value on specified input or inout ports



```
dc_shell> set_drive -rise 3 [all_inputs]
```

```
dc_shell> set_drive -fall 2 [all_inputs]
```

```
dc_shell> set_driving_cell -lib_cell INV3 \  
[get_ports INPB] \  
-input_transition_rise 0.1  
-input_transition_fall 0.1
```



- set_input_transition – used to define slew at the input ports

```
dc_shell > set_input_transition 0.85  
[get_ports INPB]
```

- Capacitive load at output ports
 - set_load

```
dc_shell> set_load 5 [get_ports OUTX]
```

- `check_timing -verbose`
 - Missing clock definitions
 - Ports with missing input delays
 - Input/output delay set without a reference clock
 - Combinational feedback loops
 - And more

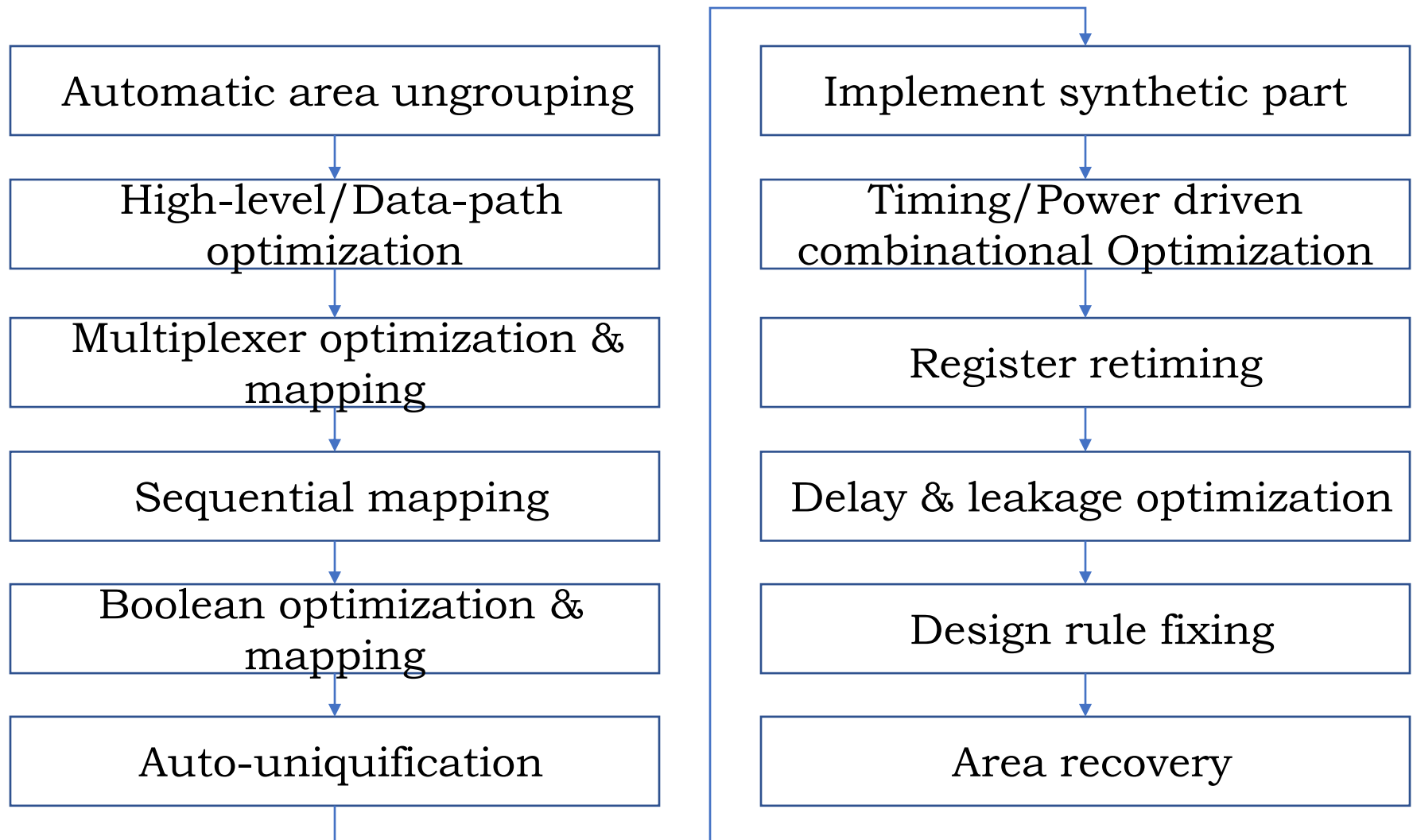
- Constraining Internal paths
 - Creating clock
 - Clock Attributes
- Constraining IO paths

Compilation and Optimization

- Compile is a process in which DC reads HDL source code and optimizes the design.
- compile or compile_ultra – Compiles the design
- Compile strategies –
 - Top down
 - Bottom up

- Optimization
 - Architectural optimization
 - Sharing common sub expressions
 - Sharing resources
 - Selecting DW implementations
 - Reordering operators
 - Identifying arithmetic expression for data-path synthesis
 - Logic level optimization
 - Structuring
 - Flattening
 - Gate level optimization
 - Mapping
 - Delay optimization
 - Design rule fixing
 - Area optimization

- Combinational optimization
 - Technology independent – Algebraic and Boolean techniques to set of logic equations and implements it.
 - Mapping – DC selects component from logic library to implement logic structure
 - Technology specific – synthesizes gate level design to meet PPA.
- Sequential optimization
 - Map sequential cells to cells in the target library.
 - Improve design timing by choosing high performance sequential cells.
 - Possible reduction in area & delay by choosing optimal sequential cells
- Local optimizations
 - Final step in gate level optimization.
 - Incremental modifications to design to adjust for timing and area



Optimizing across Hierarchical Boundaries

- Boundary optimization
 - Propagation of constant across hierarchy
 - Propagation of equal and opposite information across hierarchy
 - Propagation of unconnected port information across hierarchy
 - Pushing of inverters across the hierarchy
- Port punching
 - For more flexible load balancing across the hierarchy & improved QoR
- Automatic ungrouping
 - Ungrouping merges sub-designs of a given level of the hierarchy into parent cell.
 - It removes hierarchical boundary and improve timings by reducing level of logic.

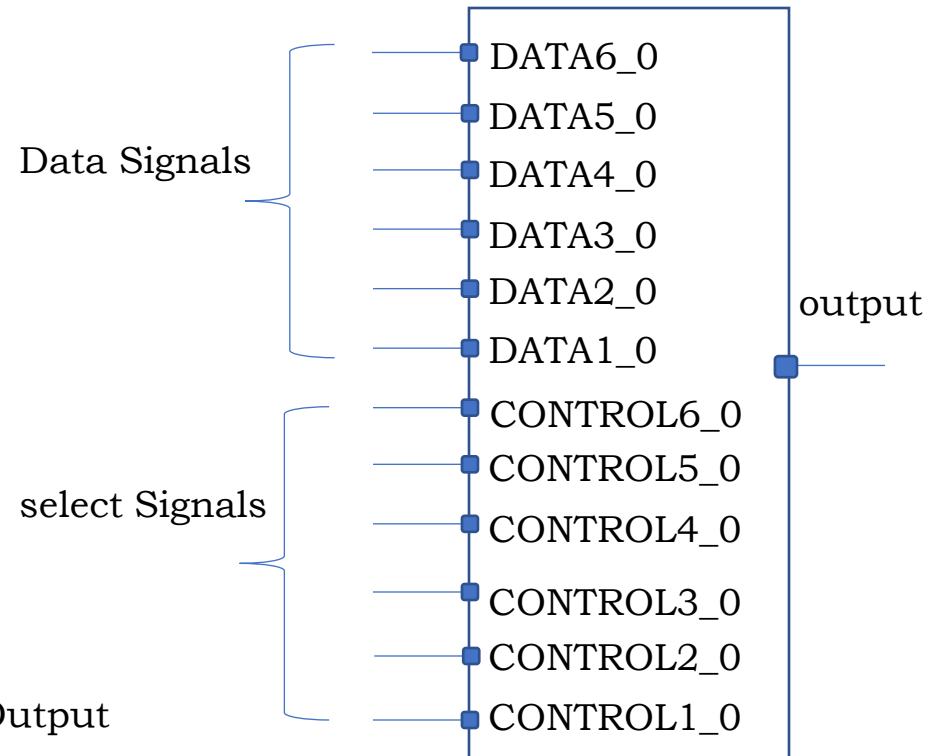
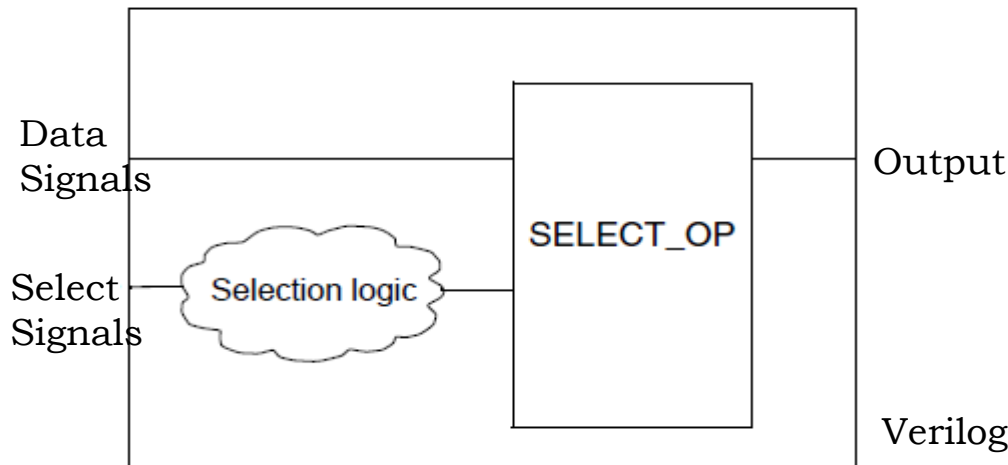
- High level optimization
 - Arithmetic simplifications to minimize hardware – e.g. expression $a+b+c+d$ implements three level of cascaded addition. But after re-arrangement $(a+b)+(c+d)$, it results faster logic.
 - Resource sharing – common subexpression elimination, sharing mutually exclusive operation.
- Data-path optimization
 - Sharing of data-path operator
 - Using carry save arithmetic technique

- DC can map combinational logic representing MUX in HDL code directly to MUX from the target library.
- MUX are commonly modeled with if and case statement.
- HDL compiler implement “if and case” logic by inferring SELECT_OP or MUX_OP cells, which DC maps to combinational logic or MUX.
- By default HDL compiler uses SELECT_OP components to implement conditional operations implied by if and case statements.

Inferring SELECT_OP

SELECT_OP behave like one-hot Multiplexer, control lines are mutually exclusive and each control input allows the data on the corresponding data input to pass to the output.

To determine which data signal is chosen, HDL compiler generates selection logic.



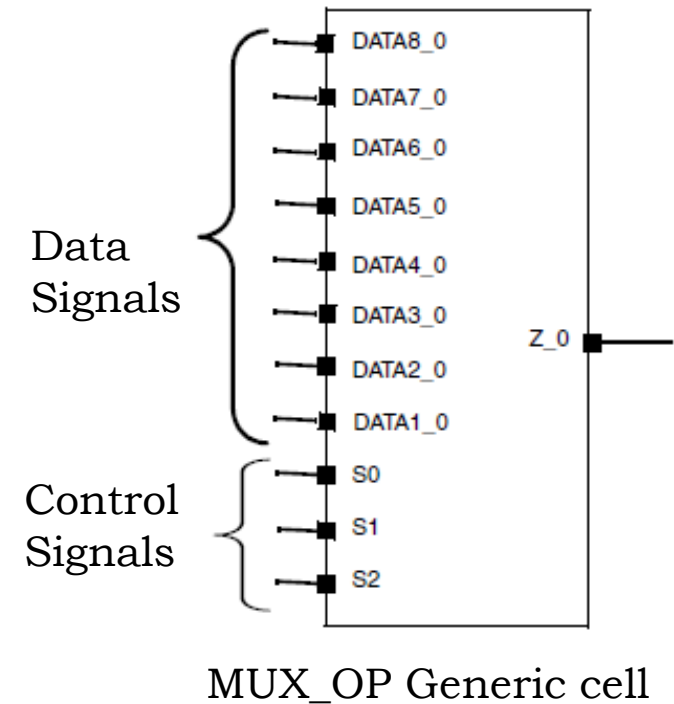
SELECT_OP generic cell

Verilog Output – SELECT_OP and Selection Logic

In order to map multiplexing logic in RTL to multiplexers – or multiplexer tree – present in logic library, HDL compiler should infer MUX_OP cells, which hierarchical generic cells optimized to use minimum number of select signals. It's typically faster than SELECT_OP, but can significantly increase area.

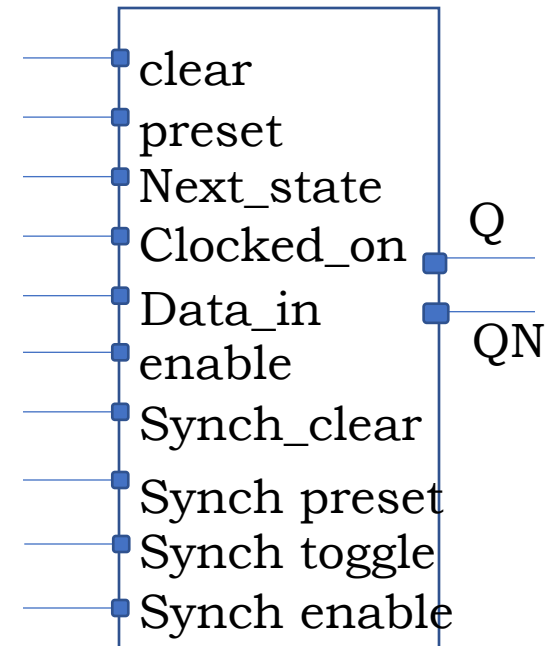
To infer MUX_OP, an attribute or directive in HDL code should be defined.

```
always @(SEL) begin  
case (SEL) //synopsys infer_mux
```



Sequential mapping

- Two step process
 - Register inference – it's process by which RTL description of a register is translated into technology independent representation called SEQGEN, which is created during elaboration and mapped to FF during compile.
 - Technology mapping – It's a process by which SEQGEN is mapped to gate from a specified target library.



Generic SEQGEN cell

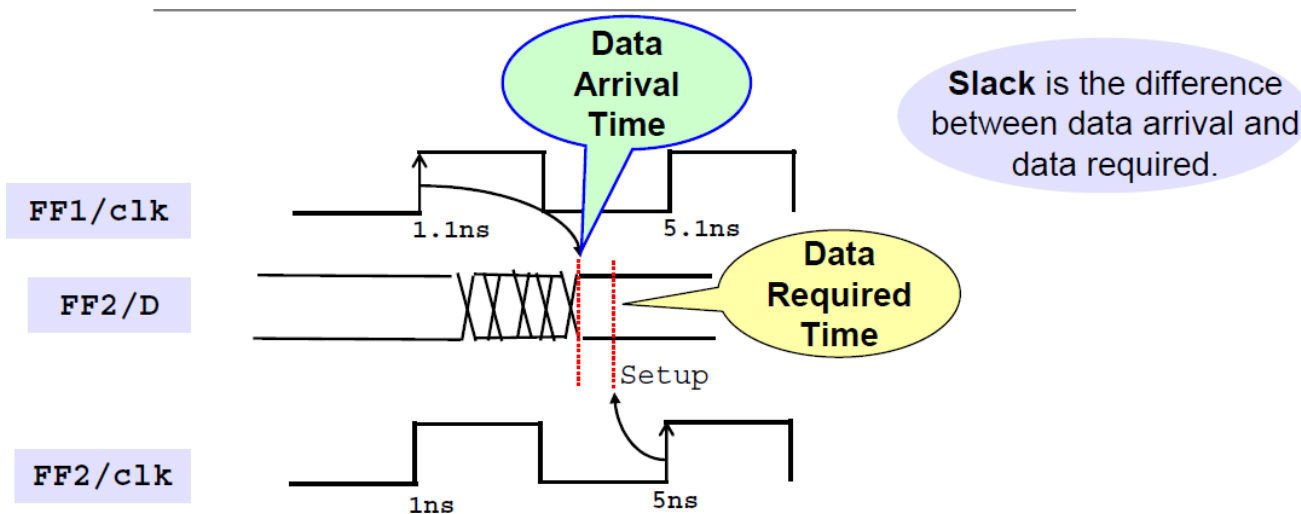
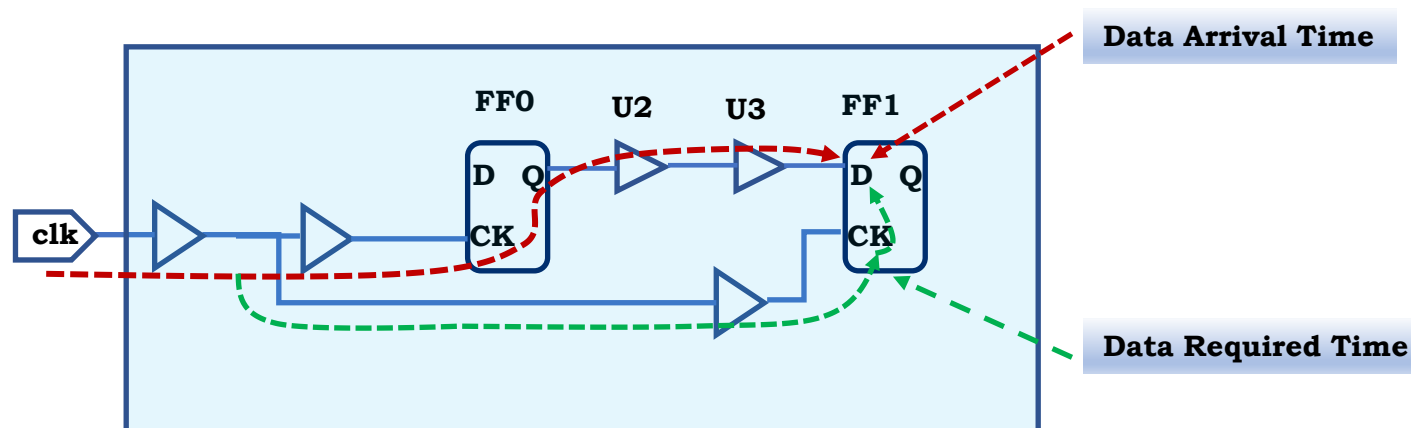
- DC implements final net-list during gate level optimization by making optimal selections of logic cells.
- DC performs following task
 - Delay optimization
 - Leakage power optimization
 - Design rule fixing
 - Area recovery

- Compiling/optimizing design
- Generating gate level net-list

Reporting and Analyzing QoR

- Timing report (report_timing) command finds all the individual timing paths in the design for analysis
- Each path is analyzed for timing twice:
 - Once for a rising edge input
 - Once with a falling edge input
- The critical path (worst violator) for each clock group is found and reported (default)

Reporting Timing Path



Sections in Timing Report

Header

Startpoint: q_reg[48] (rising edge-triggered flip-flop clocked by CLK)
Endpoint: q_reg[63] (rising edge-triggered flip-flop clocked by CLK)
Path Group: CLK
Path Type: max

Data Arrival

Point	Incr	Path

clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.40	0.40
q_reg[48]/CLK (DFFARX2_LVT)	0.00	0.40 r
q_reg[48]/Q (DFFARX2_LVT)	0.19	0.59 r
U101/Y (NAND2X0_LVT)	0.04	0.64 f
U24/Y (INVX0_HVT)	0.04	0.68 r
U25/Y (AND3X1_HVT)	0.12	0.80 r
U55/Y (NAND4X0_HVT)	0.28	1.08 f
U102/Y (NOR2X1_LVT)	0.09	1.16 r
U115/Y (NAND2X0_LVT)	0.02	1.19 f
U116/Y (NOR2X0_LVT)	0.06	1.24 r
U117/Y (XOR2X1_LVT)	0.07	1.32 f
q_reg[63]/D (DFFARX2_LVT)	0.00	1.32 f
data arrival time		1.32

Data Required

clock CLK (rise edge)	0.50	0.50
clock network delay (ideal)	0.40	0.90
clock reconvergence pessimism	0.00	0.90
q_reg[63]/CLK (DFFARX2_LVT)		0.90 r
library setup time	-0.04	0.86
data required time		0.86

Slack

data required time		0.86
data arrival time		-1.32

slack (VIOLATED)		-0.46

Analyzing Reports: Start and End Point

Startpoint: q_reg[48] (rising edge-triggered flip-flop clocked by CLK)

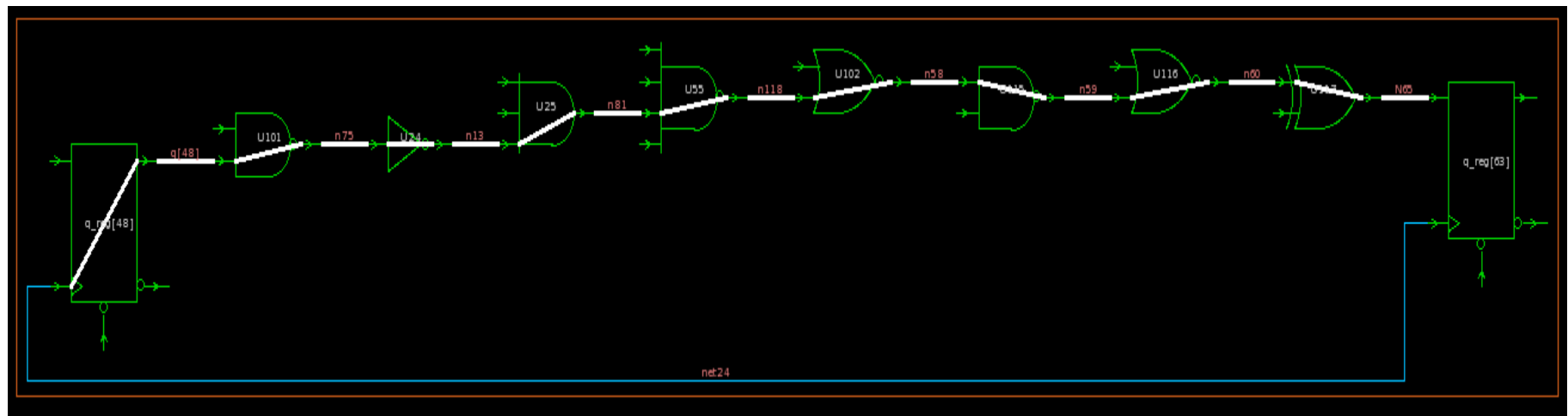
Endpoint: q_reg[63] (rising edge-triggered flip-flop clocked by CLK)

Path Group: CLK

Path Type: max

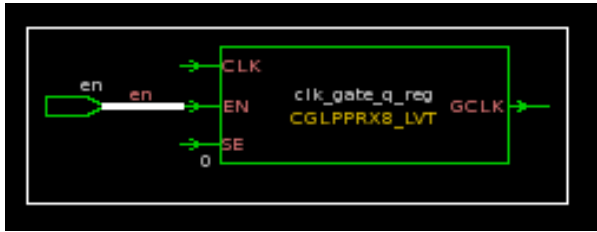
Endpoint Register Clock

Report is for Setup



Clock Gating Check

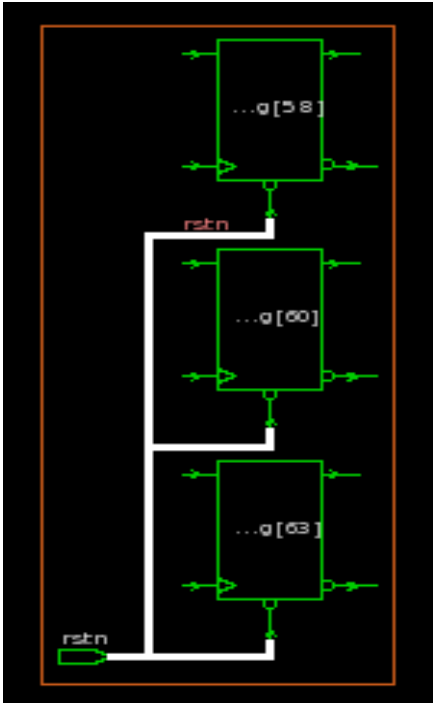
Startpoint: en (input port clocked by CLK)
Endpoint: clk_gate_q_reg
(rising clock gating-check end-point clocked by CLK)
Path Group: **clock_gating_default**
Path Type: max



Point	Incr	Path
clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.40	0.40
input external delay	1.00	1.40 f
en (in)	0.00	1.40 f
clk_gate_q_reg/EN (CGLPPRX8_LVT)	0.00	1.40 f
data arrival time		1.40
clock CLK (rise edge)	0.50	0.50
clock network delay (ideal)	0.40	0.90
clock reconvergence pessimism	0.00	0.90
clk_gate_q_reg/CLK (CGLPPRX8_LVT)		0.90 r
clock gating setup time	-0.19	0.71
data required time		0.71
data required time		0.71
data arrival time		-1.40
slack (VIOLATED)		-0.69

Asynchronous Recovery Check

Startpoint: rstn (input port clocked by CLK)
Endpoint: q_reg[63] (recovery check against rising-edge clock CLK)
Path Group: ****async_default****
Path Type: max

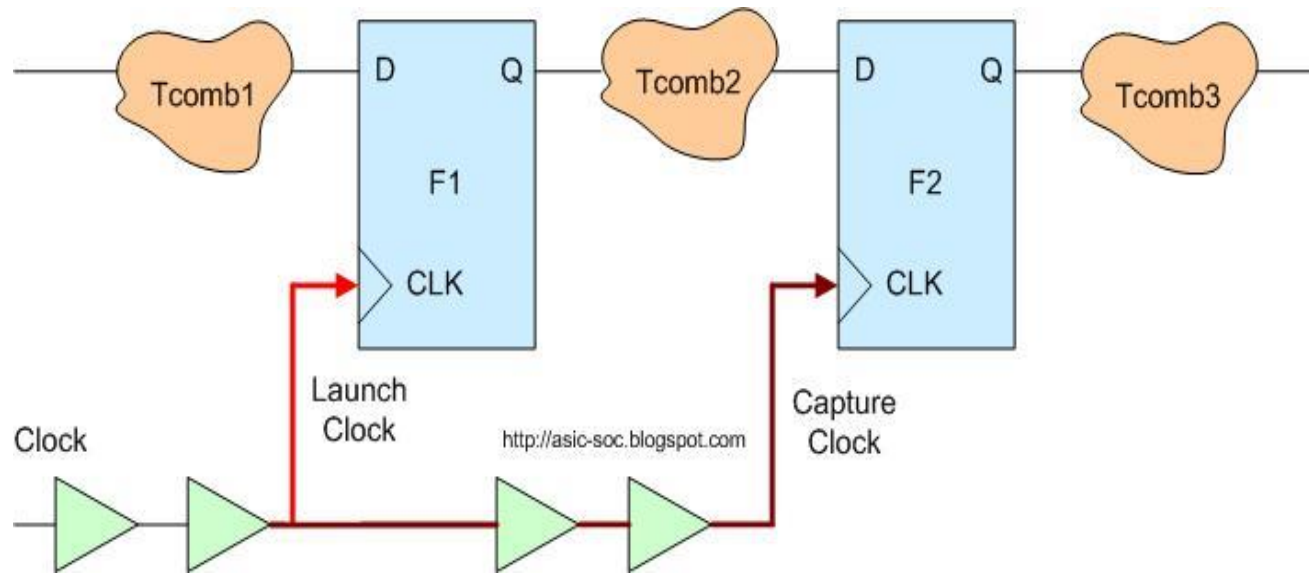


Point	Incr	Path

clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.40	0.40
input external delay	1.00	1.40 r
rstn (in)	0.00	1.40 r
q_reg[63]/RSTB (DFFARX2_LVT)	0.00	1.40 r
data arrival time		1.40
clock CLK (rise edge)	0.50	0.50
clock network delay (ideal)	0.40	0.90
clock reconvergence pessimism	0.00	0.90
q_reg[63]/CLK (DFFARX2_LVT)		0.90 r
library recovery time	0.14	1.04
data required time		1.04

data required time		1.04
data arrival time		-1.40

slack (VIOLATED)		-0.36



Capture Clock Edge

- The edge of the clock for which data is detected is known as capture edge.

Launch Clock Edge

- This is the edge of the clock wherein data is launched in previous flip flop and will be captured at this flip flop.

Analyzing Reports: Data Arrival

**Ideal Clock,
User specified
latency**

Data Arrival

clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.40	0.40
q_reg[48]/CLK (DFFARX2_LVT)	0.00	0.40 r
q_reg[48]/Q (DFFARX2_LVT) <-	0.19	0.59 r
U101/Y (NAND2X0_LVT)	0.04	0.64 f
U24/Y (INVX0_HVT)	0.04	0.68 r
U25/Y (AND3X1_HVT)	0.12	0.80 r
U55/Y (NAND4X0_HVT)	0.28	1.08 f
U102/Y (NOR2X1_LVT)	0.09	1.16 r
U115/Y (NAND2X0_LVT)	0.02	1.19 f
U116/Y (NOR2X0_LVT)	0.06	1.24 r
U117/Y (XOR2X1_LVT)	0.07	1.32 f
q_reg[63]/D (DFFARX2_LVT)	0.00	1.32 f
data arrival time		1.32

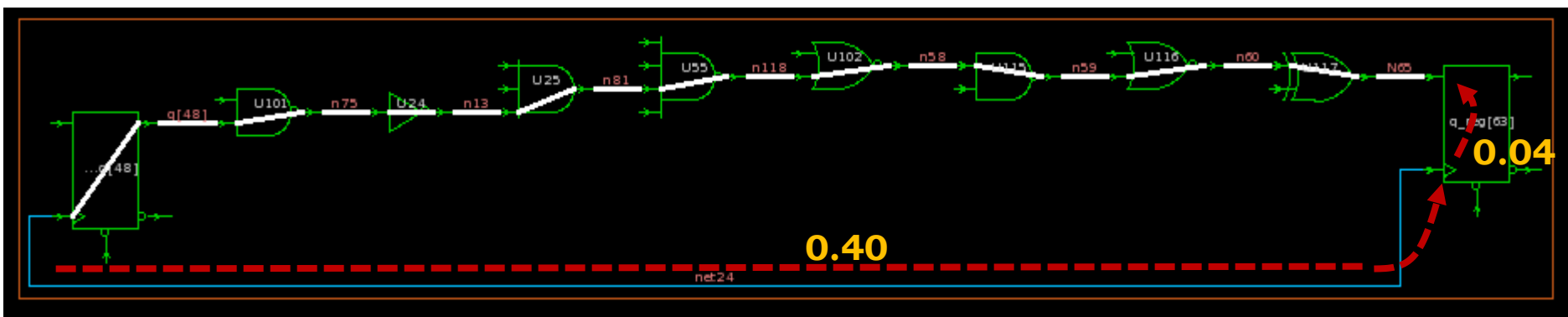
**Library
Reference**



Analyzing Reports: Data Required

Data Required

clock CLK (rise edge)	0.50	0.50
clock network delay (ideal)	0.40	0.90
clock reconvergence pessimism	0.00	0.90
q_reg[63]/CLK (DFFARX2_LVT)		0.90 r
library setup time	-0.04	0.86
data required time		0.86



Analyzing Reports: Slack

Startpoint: q_reg[48] (rising edge-triggered flip-flop clocked by CLK)
 Endpoint: q_reg[63] (rising edge-triggered flip-flop clocked by CLK)
 Path Group: CLK
 Path Type: max

Point	Incr	Path

clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.40	0.40
q_reg[48]/CLK (DFFARX2_LVT)	0.00	0.40 r
q_reg[48]/Q (DFFARX2_LVT) <-	0.19	0.59 r
U101/Y (NAND2X0_LVT)	0.04	0.64 f
U24/Y (INVX0_HVT)	0.04	0.68 r
U25/Y (AND3X1_HVT)	0.12	0.80 r
U55/Y (NAND4X0_HVT)	0.28	1.08 f
U102/Y (NOR2X1_LVT)	0.09	1.16 r
U115/Y (NAND2X0_LVT)	0.02	1.19 f
U116/Y (NOR2X0_LVT)	0.06	1.24 r
U117/Y (XOR2X1_LVT)	0.07	1.32 f
q_reg[63]/D (DFFARX2_LVT)	0.00	1.32 f
data arrival time		1.32
clock CLK (rise edge)	0.50	0.50
clock network delay (ideal)	0.40	0.90
clock reconvergence pessimism	0.00	0.90
q_reg[63]/CLK (DFFARX2_LVT)		0.90 r
library setup time	-0.04	0.86
data required time		0.86

data required time		0.86
data arrival time		-1.32

slack (VIOLATED)		-0.46

**Slack
Violated**



Analyzing Reports: All Clock Propagated

**set_propagated_clock \
[all_clocks]**

Startpoint: q_reg[48] (rising edge-triggered flip-flop clocked by CLK)
Endpoint: q_reg[63] (rising edge-triggered flip-flop clocked by CLK)
Path Group: CLK
Path Type: max

**Calculated Clock
Network Delay
For Launch**

Point	Incr	Path
clock CLK (rise edge)	0.00	0.00
clock network delay (propagated)	0.20	0.20
q_reg[48]/CLK (DFFARX2_LVT)	0.00	0.20 r
q_reg[48]/Q (DFFARX2_LVT) <-	0.21	0.41 r
U101/Y (NAND2X0_LVT)	0.04	0.45 f
U24/Y (INVX0_HVT)	0.04	0.49 r
U25/Y (AND3X1_HVT)	0.12	0.61 r
U55/Y (NAND4X0_HVT)	0.28	0.89 f
U102/Y (NOR2X1_LVT)	0.09	0.98 r
U115/Y (NAND2X0_LVT)	0.02	1.01 f
U116/Y (NOR2X0_LVT)	0.06	1.06 r
U117/Y (XOR2X1_LVT)	0.07	1.14 f
q_reg[63]/D (DFFARX2_LVT)	0.00	1.14 f
data arrival time		1.14

**Calculated Clock
Network Delay
For Capture**

clock CLK (rise edge)	0.50	0.50
clock network delay (propagated)	0.19	0.69
clock reconvergence pessimism	0.00	0.69
q_reg[63]/CLK (DFFARX2_LVT)		0.69 r
library setup time	-0.03	0.66
data required time		0.66
data required time		0.66
data arrival time		-1.14

Reporting Clock Path

Startpoint: q_reg[48] (rising edge-triggered flip-flop clocked by CLK)

Endpoint: q_reg[63] (rising edge-triggered flip-flop clocked by CLK)

Path Group: CLK

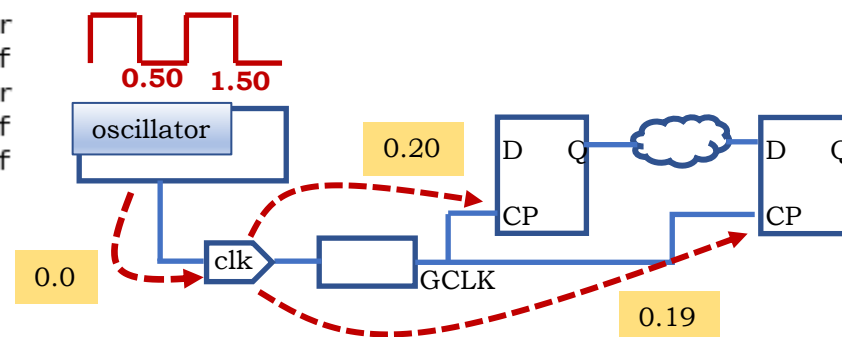
Path Type: max

Point	Incr	Path
clock CLK (rise edge)	0.00	0.00
clock source latency	0.00	0.00
clk (in)	0.00	0.00 r
clk_gate_q_reg/GCLK (CGLPPRX8_LVT)	0.20	0.20 r
q_reg[48]/CLK (DFFARX2_LVT)	0.00	0.20 r
q_reg[48]/Q (DFFARX2_LVT) <-	0.21	0.41 r
U101/Y (NAND2X0_LVT)	0.04	0.45 f
U24/Y (INVX0_HVT)	0.04	0.49 r
U25/Y (AND3X1_HVT)	0.12	0.61 r
U55/Y (NAND4X0_HVT)	0.28	0.89 f
U102/Y (NOR2X1_LVT)	0.09	0.98 r
U115/Y (NAND2X0_LVT)	0.02	1.01 f
U116/Y (NOR2X0_LVT)	0.06	1.06 r
U117/Y (XOR2X1_LVT)	0.07	1.14 f
q_reg[63]/D (DFFARX2_LVT)	0.00	1.14 f
data arrival time		1.14

Clock Ideal Waveform

Clock Source Latency

Clock Network Latency



report_timing -path_type full_clock_expanded

Timing Report: Path Delay Section

Startpoint: q_reg[48] (rising edge-triggered flip-flop clocked by CLK)
 Endpoint: q_reg[63] (rising edge-triggered flip-flop clocked by CLK)
 Path Group: CLK
 Path Type: max

Point	Incr	Path
clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.40	0.40
q_reg[48]/CLK (DFFARX2_LVT)	0.00	0.40 r
q_reg[48]/Q (DFFARX2_LVT) <-	0.19	0.59 r
U101/Y (NAND2X0_LVT)	0.04	0.64 f
U24/Y (INVX0_HVT)	0.04	0.68 r
U25/Y (AND3X1_HVT)	0.12	0.80 r
U55/Y (NAND4X0_HVT)	0.28	1.08 f
U102/Y (NOR2X1_LVT)	0.09	1.16 r
U115/Y (NAND2X0_LVT)	0.02	1.19 f
U116/Y (NOR2X0_LVT)	0.06	1.24 r
U117/Y (XOR2X1_LVT)	0.07	1.32 f
q_reg[63]/D (DFFARX2_LVT)	0.00	1.32 f
data arrival time		1.32
clock CLK (rise edge)	0.50	0.50
clock network delay (ideal)	0.40	0.90
clock reconvergence pessimism	0.00	0.90
q_reg[63]/CLK (DFFARX2_LVT)		0.90 r
library setup time	-0.04	0.86
data required time		0.86
data required time		0.86
data arrival time		-1.32
slack (VIOLATED)		-0.46

Individual contribution to path delay

Total Path Delay

Unateness

Net + Cell Delay

Constraint Report

```
prompt> report_constraint -all_violators -max_capacitance
```

```
...
```

Net	Required Capacitance	Actual Capacitance	Slack
n_6	0.0014226	0.0128121	-0.0113895 (VIOLATED)
n_16	0.0014226	0.0115580	-0.0101354 (VIOLATED)
n_121	0.0014226	0.0114569	-0.0100343 (VIOLATED)
n_21	0.0014226	0.0113311	-0.0099085 (VIOLATED)
div_18_17/n_2	0.0014226	0.0044171	-0.0029945 (VIOLATED)
div_18_17/n_43	0.0014226	0.0042607	-0.0028381 (VIOLATED)

DRC: Max Capacitance: Violators

```
prompt> report_area
```

```
*****  
Report : area  
Design : TEST_TOP  
...  
*****
```

```
Library(s) Used:  
test_lib (File: test_lib.db)
```

```
Number of ports:          565  
Number of nets:           9654  
Number of cells:          8120  
Number of combinational cells: 6594  
Number of sequential cells: 1526  
Number of macros:         0  
Number of buf/inv:        1439  
  
Number of references:     163
```

```
Combinational area:      562404.432061  
Noncombinational area:  6720840.351067  
Net Interconnect area:   undefined (Wire load has zero net area)  
Total cell area:        7283244.783128  
Total area:              undefined
```

Area Summary: Notice Interconnect area = 0; Why ?

```
prompt> report_qor
```

```
*****  
Report : qor  
Design : TEST_TOP  
...  
*****
```

Timing Path Group 'clk'

Levels of Logic:	1.00
Critical Path Length:	0.02
Critical Path Slack:	-0.68
Critical Path Clk Period:	8.00
Total Negative Slack:	-393.61
No. of Violating Paths:	1066.00
Worst Hold Violation:	0.00
Total Hold Violation:	0.00
No. of Hold Violations:	0.00

Path group Timing QoR

Cell Count

Hierarchical Cell Count:	7
Hierarchical Port Count:	2360
Leaf Cell Count:	43978
Buf/Inv Cell Count:	6764
CT Buf/Inv Cell Count:	4
Combinational Cell Count:	37212
Sequential Cell Count:	6773
Macro Count:	0

Cells utilized Statistics

```
Area
-----
Combinational Area:   562404.432061
Noncombinational Area:
                        6720840.351067
Net Area:              0.000000
Net XLength           :   2836623.25
Net YLength           :   2555007.75
-----
Cell Area:            7283244.783128
Design Area:          7283244.783128
Net Length            :   5391631.00
```

Area statistics and QoR

```
Design Rules
-----
Total Number of Nets:   47207
Nets With Violations:   5
Max Trans Violations:   3
```

DRC QoR

```
Hostname: machine
Compile CPU Statistics
-----
Resource Sharing:       21.54
Logic Optimization:     182.63
Mapping Optimization:   230.79
-----
Overall Compile Time:   631.32
Overall Compile Wall Clock Time: 288.11
```

Compile Time QoR

Power Report

```
dc_shell> report_power -hier
```

```
*****  
Report : power  
       -hier  
       -analysis_effort low  
Design : DESIGN_4  
Version: A-2007.12-SP2  
Date   : Fri Feb 22 01:51:42 2008  
*****
```

Library(s) Used:

slow (File: slow.db)

Operating Conditions: slow Library: slow
Wire Load Model Mode: Inactive.

Global Operating Voltage = 1.62

Power-specific unit information :

Voltage Units = 1V

Capacitance Units = 1.000000pf

Time Units = 1ns

Dynamic Power Units = 1mW (derived from V,C,T units)

Leakage Power Units = 1nW

Information: Reporting correlated power. (PWR-620)

Hierarchy	Switch Power	Int Power	Leak Power	Total Power	%
A7S_top	8.683	27.368	2.66e+03	36.054	100.0
CLOCK_TREE_EST	3.102	1.837	9.914	4.940	13.7
U_CORE (A7S_core)	4.318	21.712	2.23e+03	26.032	72.2

Power categories

- Sanity checks on gate net-list
- Reporting & Analyzing Timing Paths
- Multi-VT synthesis & QoR comparison

- Constraining design
- Creating clock & its Attributes
- Constraining IO paths
- Compiling/Optimizing Design
- High level/Data-path optimization
- Reporting and Analyzing QoR

Synthesis 3

RTL Coding Guidelines

```
module Continuous (A, Z);  
input A;  
output Z;  
  
assign Z = ~ A;  
  
endmodule
```

```
module Blocking (Preset, Count);  
input [0:2] Preset;  
output [3:0] Count;  
  
always @(Preset)  
    Count = Preset + 1;  
  
endmodule
```

```
module NonBlocking (clk, RegA, Mask, RegB);  
input clk;  
input [3:0] RegA, Mask;  
output [3:0] RegB;  
  
reg [3:0] RegB;  
  
always @(posedge clk)  
    RegB <= RegA & Mask  
  
endmodule
```

- Any kind of delay, delay control or intra statement delay, specified in a procedural assignment is ignored by synthesis tool.
 - #5 RegB <= RegA & Mask;
- Assignment using blocking & non-blocking both is illegal.

Count = preset+1
Count <=Mask;



- Logical operator get directly mapped onto primitive logic gate

```
module FullAdder (A, B, CarryIn, Sum, CarryOut);  
input A, B, CarryIn;  
output Sum, CarryOut;
```

```
assign Sum = (A^B)^CarryIn;  
assign CarryOut = (A&B) | (B & CarryIn) | (A &  
CarryIn);
```

```
endmodule
```

- Unsigned Arithmetic
 - If operand is of net type, they will be unsigned numbers

```
module UnsignedAdder (Arb, Bet, Lot);  
input [2:0] Arb, Bet;  
output [2:0] Lot;  
    assign Lot = Arb + Bet;  
endmodule
```

- Signed Arithmetic
 - By declaring operand as integer type

```
module SignedAdder (Arb, Bet, Lot);  
input [1:0] Arb, Bet;  
output [2:0] Lot;  
reg [2:0] Lot;  
integer A, B;  
    always @(Arb or Bet)  
    begin  
        A = ~Arb;  
        B = Bet;  
        Lot = A + B;  
    end  
endmodule
```


- >, <, <=, >=

```
module UnsignedRelation (A, B, Z);  
input [3:0] A,B;  
output Z;  
    assign Z = A[1:0]>B[3:2]  
endmodule
```

```
module SignedRelation (A,B,Z);  
input [2:0] A,B;  
output Z;  
integer AInt, Bint;  
    always @(A or B);  
    begin  
        AInt = ~A;  
        Bint = ~B;  
        Z = AInt <= Bint;  
    end  
endmodule
```

- Equality operator supported for synthesis “==”, “!=”

```
module NotEqual (A,B,Z);  
input [0:3] A,B;  
output Z;  
reg Z;  
integer IntA, IntB;
```

```
    always @(A or B)  
    begin  
        IntA = A;  
        IntB = B;  
        Z = IntA != IntB;  
    end  
endmodule
```

- Left shift (<<), Right shift (>>)

```
module ConstantShift (DataMux, Adress);  
input [0:3] DataMux;  
output [0:5] Address;  
    assign Address = (~DataMux) <<2;  
endmodule
```

```
module VariableShift (MemDataReg, Amount, InstReg);  
input [2:0] MemDataReg;  
input [1:0] Amount;  
output [2:0] InstReg;  
    assign InstReg = MemDataReg >> Amount  
endmodule
```

- Vector operand can be used in expressions

```
module VectorOperations (A, B,C,Rfile);  
input [3:0] A,B,C;  
outut[3:0] Rfile;  
    assign Rfile = (A&B) | C;  
endmodule
```

```
module VectorOperands (A,B,Z);  
input [3:0] A,B;  
output [3:0] Z;  
    assign Z = A^B;  
endmodule
```

```
module NonConstant (Data, Index, Dout);  
input [0:3] Data;  
input [1:2] Index;  
output Dout;  
    assign Dout = Data [index];  
endmodule
```

Above code synthesized to MUX

- Used to model procedural behavior of circuit

```
module EvenParity (A,B,C,D,Z);  
input A,B,C,D;  
output Z;  
reg Z, Temp1, Temp2;  
    always @ (A or B or C or D)  
    begin  
        Temp1 = A^B;  
        Temp2 = C^D;  
        Z = Temp1 ^ Temp2;  
    end  
endmodule
```

- Conditionally controlled logic.

```
module SelectOneOf (A,B,Z);  
input [1:0] A,B;  
output [1:0] Z;  
reg [1:0] Z;  
    always @(A or B)  
    if (A>B)  
        Z=A;  
    else  
        Z=B;  
endmodule
```

- If a variable is not assigned in all possible execution of an always statement, then a latch is inferred

```
module Increment (Phy, Ones, Z);  
input Phy;  
input [0:1] Ones;  
output [0:2] Z;  
reg [0:2] Z;  
    always @(Phy or Ones)  
        if (Phy)  
            Z=Ones+1;  
endmodule
```



```
module ALU (Op, A,B,Z);  
input [1:2] Op;  
input [0:1] A,B;  
output [0:1] Z;  
reg [0:1] Z;
```

```
parameter ADD='b00, SUB='b01, MUL='b10, DIV='b11;
```

```
always @(Op or A or B)
```

```
case(Op)
```

```
    ADD: Z = A+B;
```

```
    SUB: Z=A-B;
```

```
    MUL: Z=A*B;
```

```
    DIV: Z=A/B;
```

```
endcase
```

```
endmodule
```

```
module StateUpdate (CurrentState, Zip);  
input [0:1] CurrentState;  
output [0:1] Zip;  
reg [0:1] Zip;  
  
always @(CurrentState)  
case (CurrentState)  
    2'b00 : Zip = 0;  
    2'b10: Zip = 3;  
endcase  
endmodule
```

(Note: No default statement in the case)

- Incomplete if-else or case statement
- Variable assigned before use

```
module top (clock, currentState, NextState)
input clock;
input [3:0] currentState;
output [3:0] NextState;
reg [3:0] NextState;
reg [3:0] Temp;
    always @(clock or currentState or Temp);
    if(clock)
    begin
        Nextstate = Temp;
        Temp = CurrentState;
    end
endmodule
```

```
module FlipFlop (A,B,Clock,Control,Zee);  
input A,B,Clock,Control;  
output Zee;  
reg Zee;  
  
always @(negedge Clock)  
    if(Control)  
        Zee <= A;  
    else  
        Zee <= B;  
  
endmodule
```

Operators	Supported	Case equality & case inequality not supported
White space & comment	Supported	
Numbers	Supported	
Strings	Not supported	
Identifier, Keywords and System Names	Supported	

Value Set	Supported	
Registers & Nets	Supported	
Vectors	Supported	
Strengths	Ignored	
Implicit Declarations	Supported	
Net Initialization	Not supported	Wires are initially unconnected
Net types	Supported	
Memories	Supported	
Integers	Supported	
Times	Not Supported	
Real numbers	Not supported	
Parameters	Supported	

Operators	Supported	Case equality & case inequality not supported
Operands		
Net & Register bit addressing	Supported	
Memory addressing	Supported	
Strings	Not supported	
Expression Bit Length	Supported	

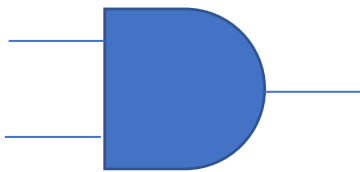
Continuous Assignment	Supported	Delay values ignored
Procedural Assignment	Supported	

Conditional Statement	Supported	
Case Statement	Supported	
Looping statement		
Forever loop	Not Supported	
Repeat loop	Supported	Repeat expression has to be constant
While loop	Not Supported	
For loop	Supported	
Task	Supported	
Function	Supported	
Procedural Timing Control		
Block Statements	Supported	
Initial Statement	Not supported	
Always Statement	Supported	

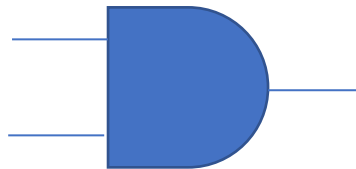
Modules	Supported	
Top-Level modules	Supported	
Module instantiation	Supported	
Overriding module parameter values	Supported	
Macro modules	Supported	
Ports	Supported	
Hierarchical names	Not supported	
Automatic naming	Supported	System generated names not supported
Scope rules	Supported	

Incomplete sensitive list

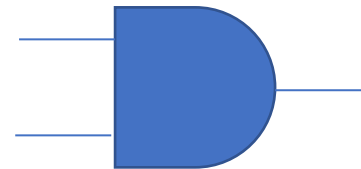
```
module uand (a,b,z);  
input a,b;  
output z;  
reg z;  
always @(a or b)  
z = a & b;  
endmodule
```



```
module uand (a,b,z);  
input a,b;  
output z;  
reg z;  
always @(a)  
z = a & b;  
endmodule
```



```
module uand (a,b,z);  
input a,b;  
output z;  
reg z;  
always  
z = a & b;  
endmodule
```



Will there be any warning/error message during synthesis ?

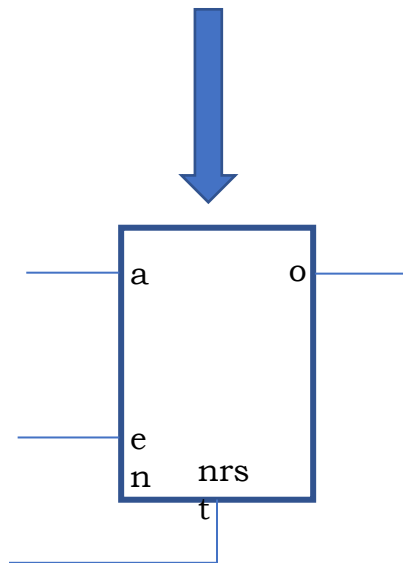
```
module code2a (o, a,b,c,d);  
output o;  
input a,b,c,d;  
reg o, temp;  
always @(a or b or c or d)  
begin  
o = a &b | temp;  
temp = c &d;  
endmodule
```

```
module code2b (o, a,b,c,d);  
output o;  
input a,b,c,d;  
reg o, temp;  
always @(a or b or c or d)  
begin  
temp = c &d;  
o = a &b | temp;  
endmodule
```

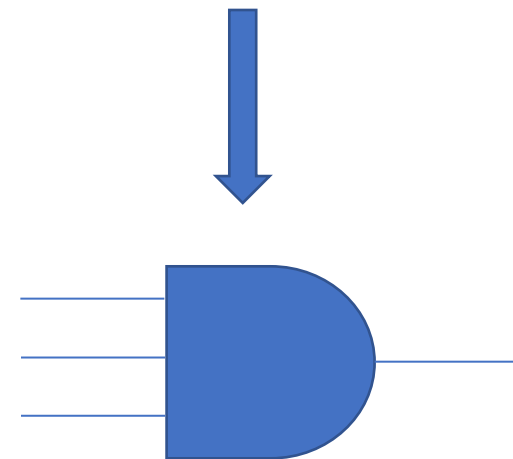


Will there be any warning/error message during synthesis ?

```
module code3a (o, a, nrst, en);  
    output o;  
    input  a, nrst, en;  
    reg    o;  
  
    always @(a or nrst or en)  
        if      (!nrst) o = 1'b0;  
        else if (en)    o = a;  
endmodule
```



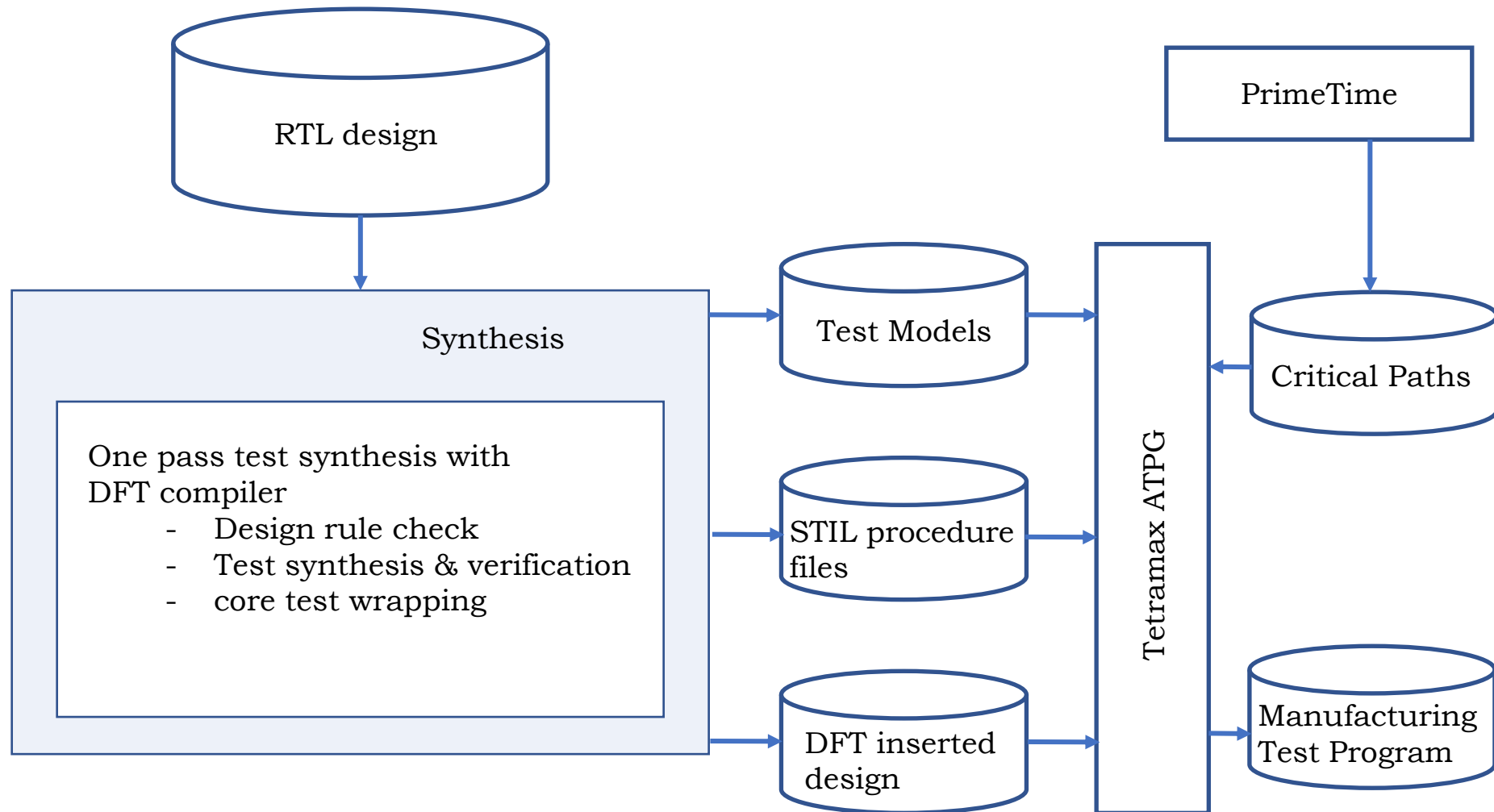
```
module code3b (o, a, nrst, en);  
    output o;  
    input  a, nrst, en;  
    reg    o;  
  
    always @(a or nrst or en)  
        o = latch(a, nrst, en);  
  
    function latch;  
        input a, nrst, en;  
        if      (!nrst) latch = 1'b0;  
        else if (en)    latch = a;  
    endfunction  
endmodule
```



Will there be any warning/error message during synthesis ?

- Synthesizing different flavor of Verilog code and observing corresponding hardware generated.

Scan Insertion



- DFT – Design for Test
 - Manufacturing testing and detecting faults
 - Stuck AT
 - Power/Ground Short
 - Open interconnect on the die caused by dust particle
 - Short circuited source/drain on the transistor

- Stuck-AT fault model
 - Stuck AT 0
 - Stuck AT 1
- Stuck AT node must be observable & Controllable in order to detect it
- To achieve best fault coverage, all the nodes in the design should be controllable & observable
- It's achieved through scan logic in the design

- Ensures that all state elements can be controlled to desired state value from the boundary of design
- It's done through scan shifting
- In full scan design, scan chain contains all state element and circuit is fully controlled.

- Ensures predictable capture of next state of the circuit and visibility at the boundary of the design
- Visibility requirement is done through capture operations in scan test
- In scan design context, a circuit is observable when the tester can successfully clock the scan cells and then shift their state to scan outputs.

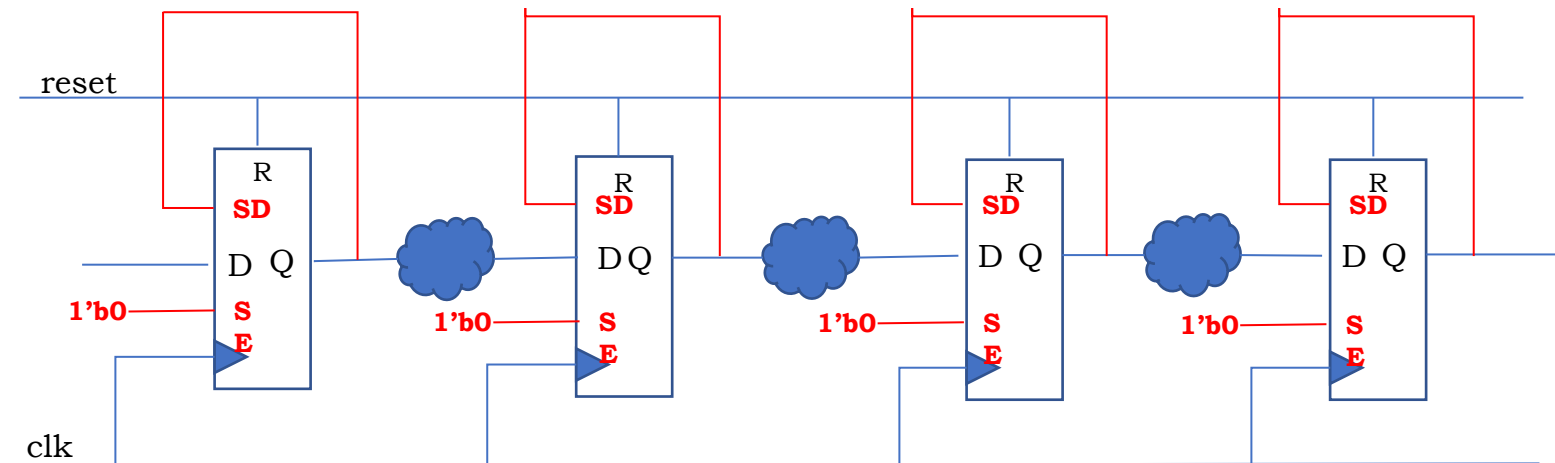
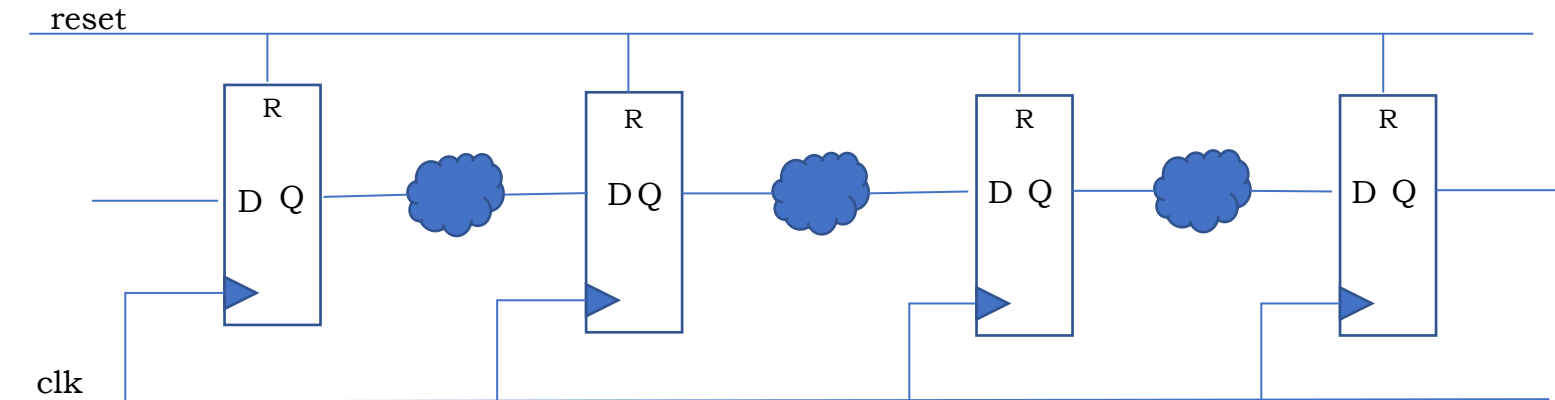
- All the flops in the design replaced with scan flops
- It provides higher fault coverage
- But, it increases design area, and decreases design performance

- Multiplexed Flip – Flop scan style
- Clocked-Scan scan style
- Level sensitive scan design style (LSSD)
- Scan enabled level sensitive scan design style

- Test Port Requirement
 - ScanIn, ScanOut, Scan clock, Scan Enable
- Test Timing Requirements
 - Test period, input timing, bidirectional timing, output strobe timing, clocking requirement
- Test clock requirement
- Test protocol requirement

Scan Chain (Cont)

dc_shell > compile

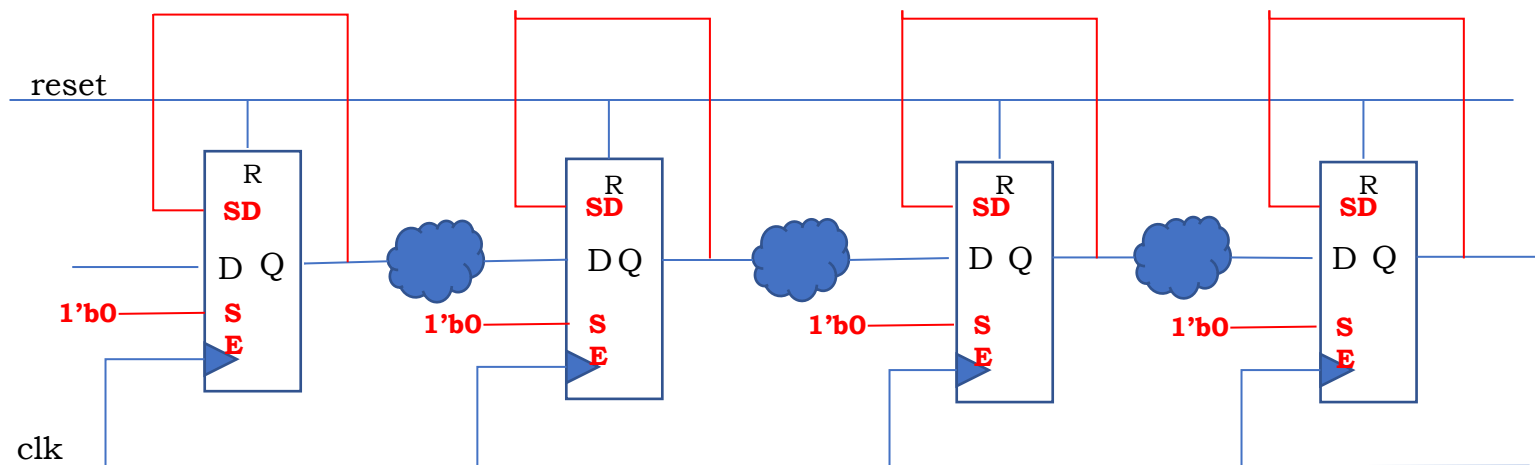


**SCAN
READY
COMPILE**

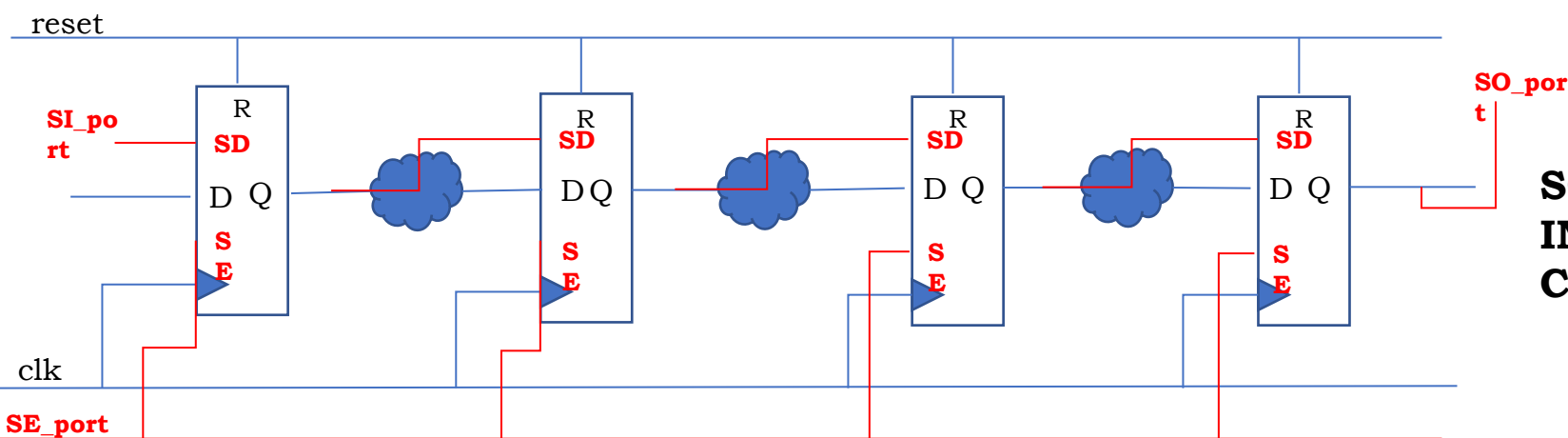
dc_shell> compile -scan

Scan Chain (Cont)

dc_shell > compile -scan



**SCAN
READY
COMPILE**



**SCAN
INSERTED
COMPILE**

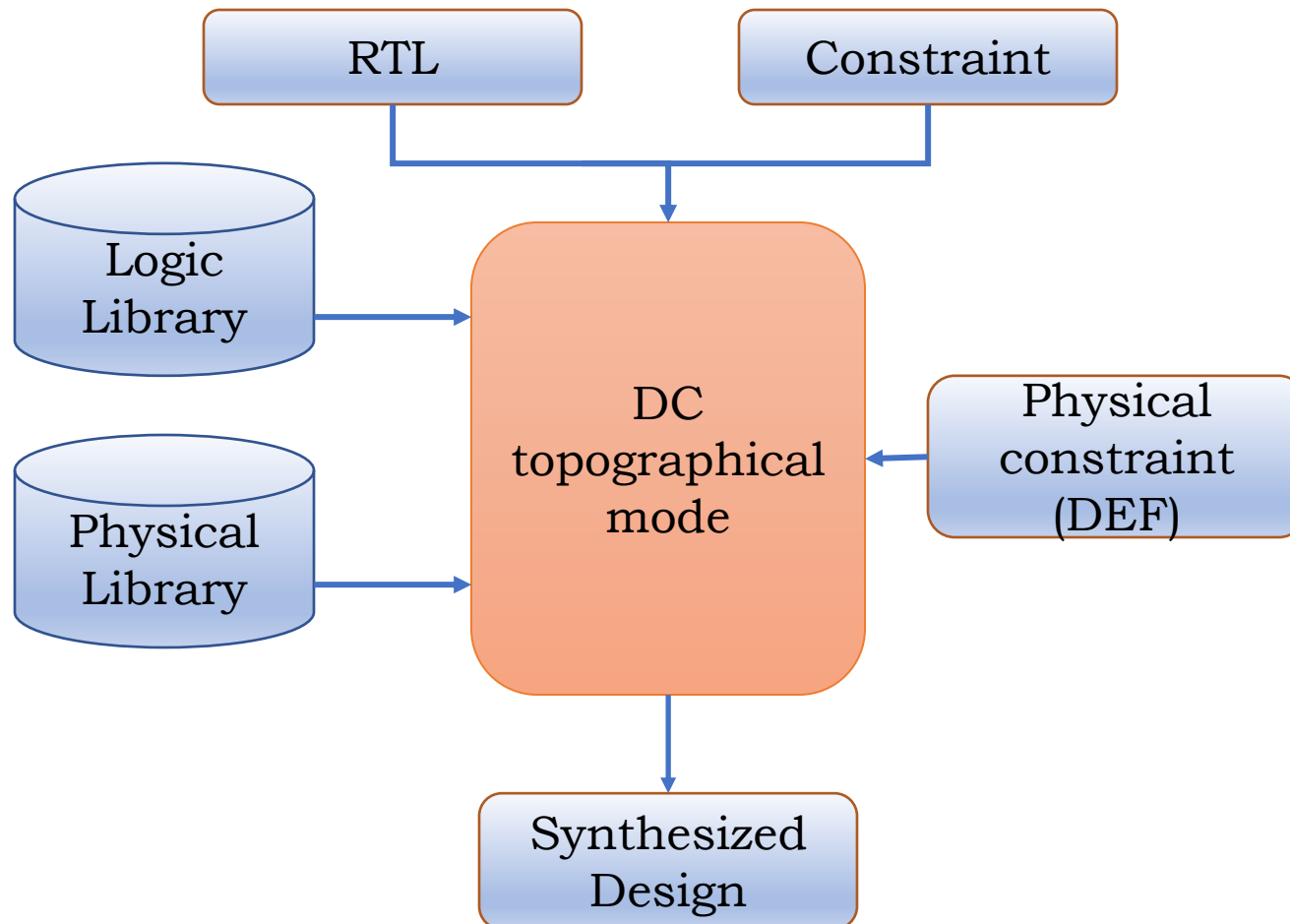
dc_shell > insert_dft

- LAB on scan insertion

Physical Aware Synthesis

- Design compiler uses topographical technology to accurately predict post layout timing, area and power during RTL synthesis
- Better timing correlation as compared to wire load mode synthesis
- Uses Synopsys' placement & optimization technique to compute timings and ensures better correlation with physical design

Synthesis flow – Topographical mode



Milky-way design library stores design data – that is, uniquified mapped netlist & constraint.

Milky-way reference library stores physical library data.

TLU+ files contains resistance & capacitance look-up tables & models process effects. Use of TLU+ provides more accurate capacitance and resistance data, thereby improves correlation with physical design.

Create Milky-way Design library

`create_mw_lib`

Open the milky-way library

`open_mw_lib`

Attach TLU+ files

`set_tlu_plus_files`

- DC supports high level physical constraint
 - Die Area
 - Core area & shape.
 - Port locations.
 - Cell locations & orientations.
 - Keep-out margin
 - Placement blockages
 - Pre-routes, Vias, tracks, wiring keep outs.
- Physical constraint to DC topographical mode can be provided –
 - By reading DEF file
 - By creating constraint manually.

- DC in topographical mode
- Comparing the QoR in topographical mode & wire load mode.

- RTL Coding Guidelines
- Assignments Statements
- Different types of operators
- Scan insertion
- DFT Flow
- Controllability and Observability of sequential cells
- Physically Aware Synthesis
- DC in topographical mode

PHYSICAL DESIGN COURSE

Module 3

Data Preparation

- In this module you will be learning about input files like Logical & Physical libraries, .tf (Technology file), TLU+ file and .sdc timing constraints files.
- Also, in addition to this, the topics like Power Specification file in upf format are learnt.

- Inputs To Block PD Implementation
- Files Required for Data Preparation
 - ✓ Logical Libraries & Physical Libraries
 - ✓ Technology Files
 - ✓ Netlist File
 - ✓ SDC (Synopsys Design Constraint) File
- Additional Information
 - ✓ Power Specification File(UPF)

Inputs to Block PD implementation

Files Required for PD Implementation

✓ Logical Libraries & Physical Libraries 

✓ Technology Files 

✓ TLU+ file (RC tech File) 

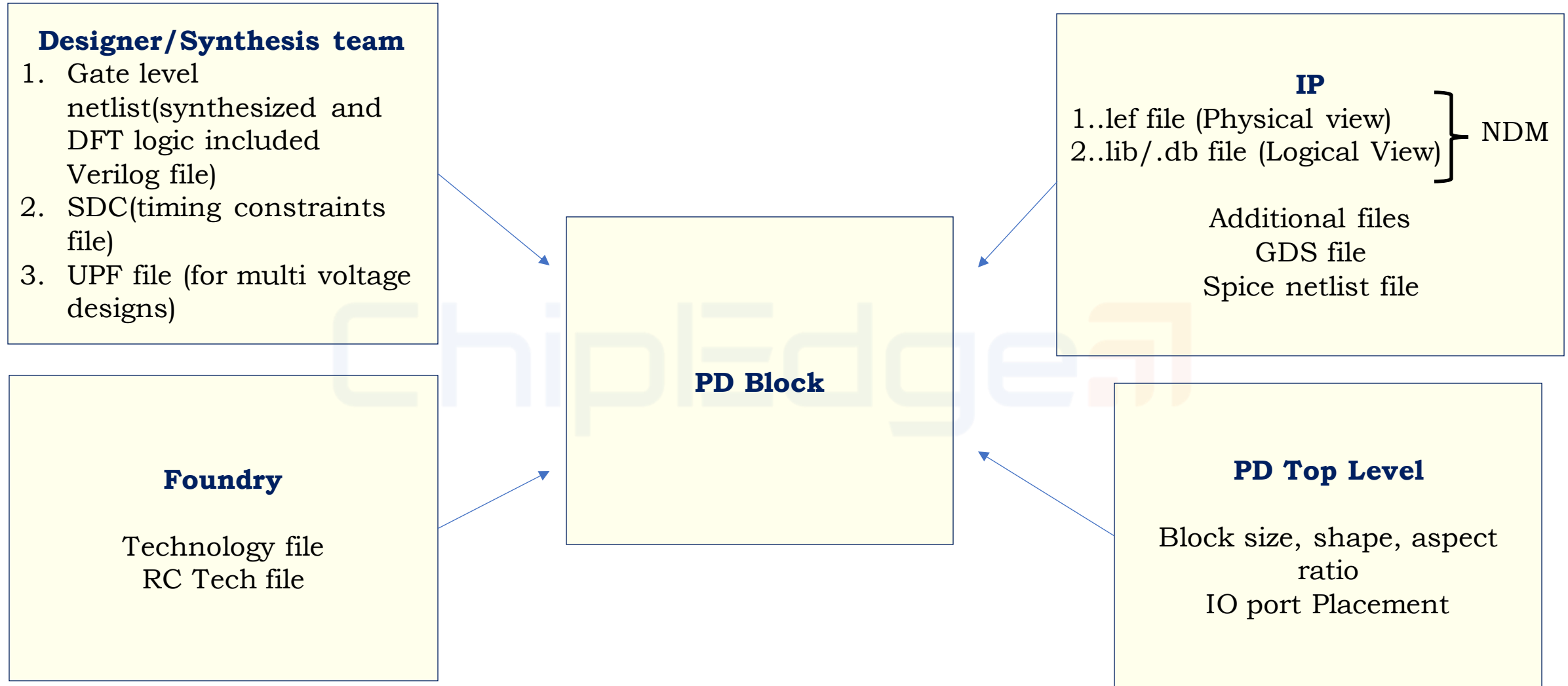
✓ Netlist File 

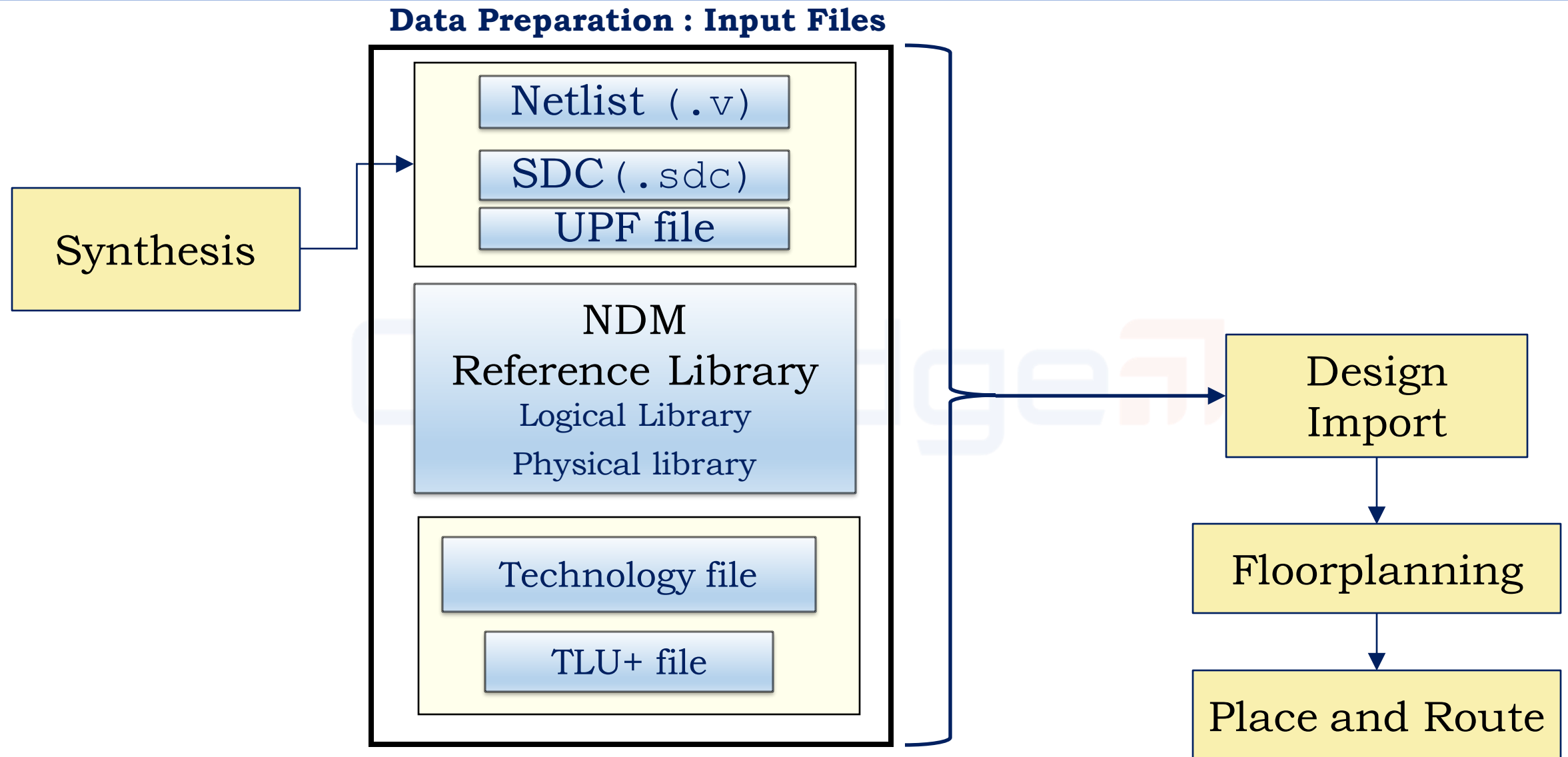
✓ SDC File 

Additional Information

✓ Power Specification File 

This section illustrates the flow of the data preparation and Inputs obtained from IP vendors, the Foundry team, and RTL Design teams.





Inputs to Block PD implementation



Files Required for PD Implementation

- ✓ Logical Libraries & Physical Libraries 
- ✓ Technology Files 
- ✓ TLU+ file (RC tech File) 
- ✓ Netlist File 
- ✓ SDC File 

Additional Information

- ✓ Power Specification File 

This section is a brief explanation of each input files (Logical & Physical library, Technology file, TLU+ file, SDC file) required for the design implementation.

- Logical and Physical Libraries (Reference Library) Files

The reference libraries are obtained from the IP vendors.

- Technology Files & TLU+ Files

The Technology and TLU+ files are obtained from the Foundry.

- RTL Code

The RTL code is primary input obtained from the Design team.

- SDC Files

sdcc files are from the RTL Design team.

Inputs to Block PD implementation



Files Required for PD Implementation



✓ Logical Libraries & Physical Libraries

This describes Logical library (.lib) and Physical Library (.lef) files with examples

✓ Technology Files



✓ TLU+ file (RC tech File)



✓ Netlist File



✓ SDC File



Additional Information

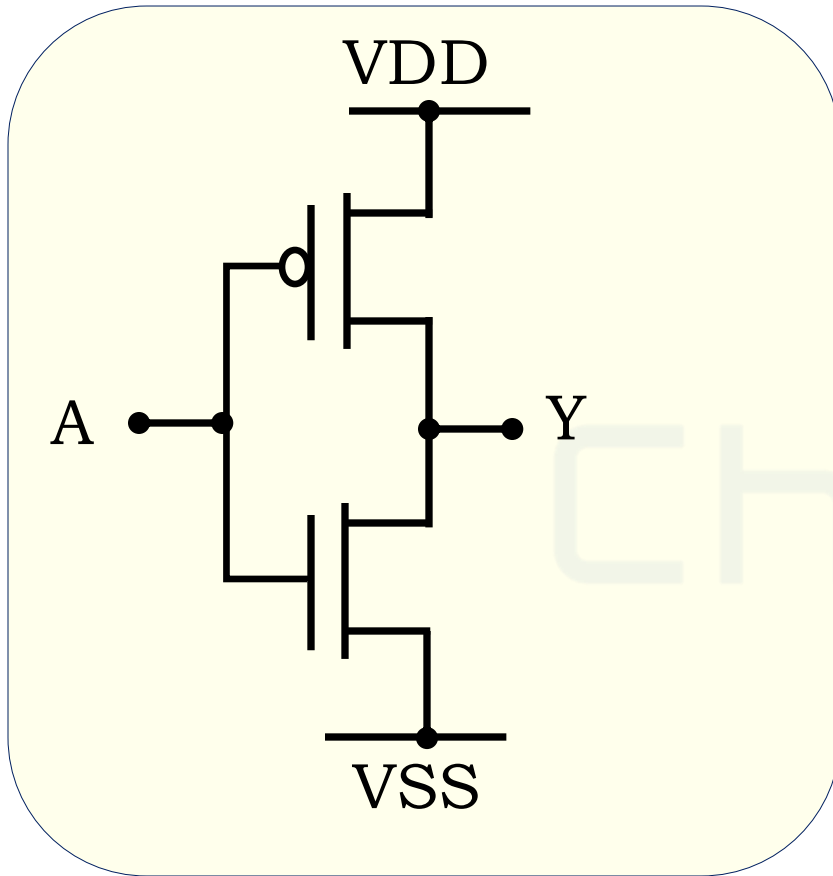


✓ Power Specification File



- Reference Library is consisting of Logical libraries and Physical Libraries.
- Reference library has contents of std cells, memories, and custom block specifications.
- Logical(.v) and Timing views (.lib) , and Physical Views (.gds, .lef) .
- Logical Library (.lib):
 - ✓ Logic Library is .lib file and is also called as Liberty Timing File.
 - ✓ The parameters related to logical views like area, pin direction, max transition, rise and fall capacitances, fanout load are mentioned in this library file.

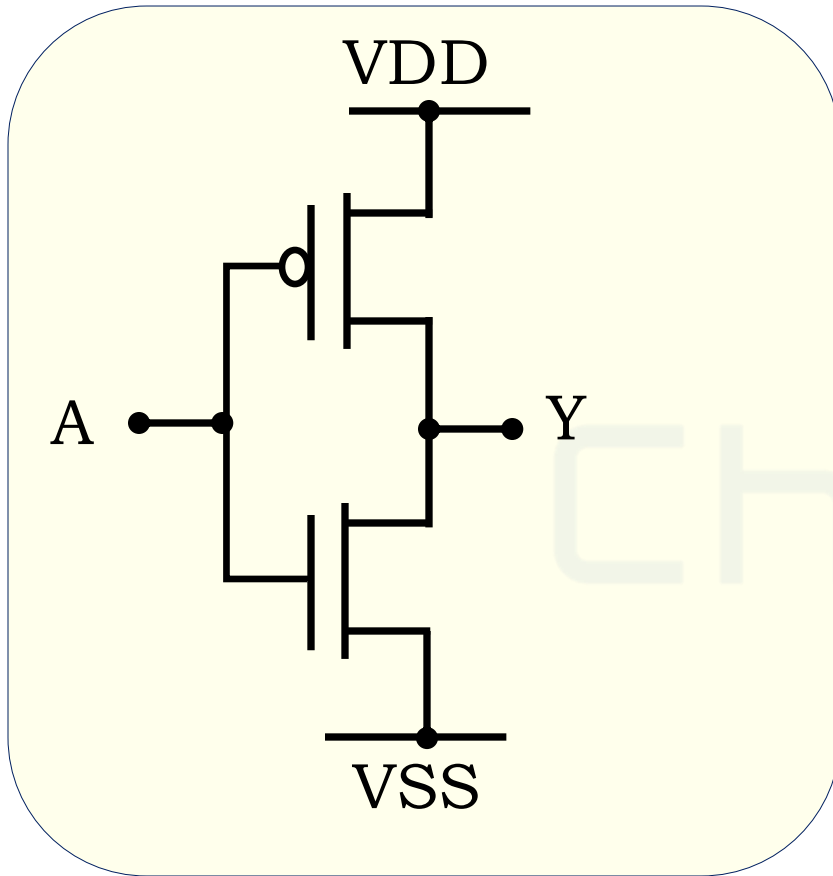
- Timing information like cell delay, output transition time for all cells.
- Setup and hold times for all the flops.
- Functionality information of Standard cells, IO's and Macros are part of this library.
- Power information like leakage power, internal power are present in .lib
- A .lib(.db) corresponds to a single corner Process Voltage Temperature(PVT)



Example of Inverter Cell
In Logical Library

Area Of The Inverter Cell

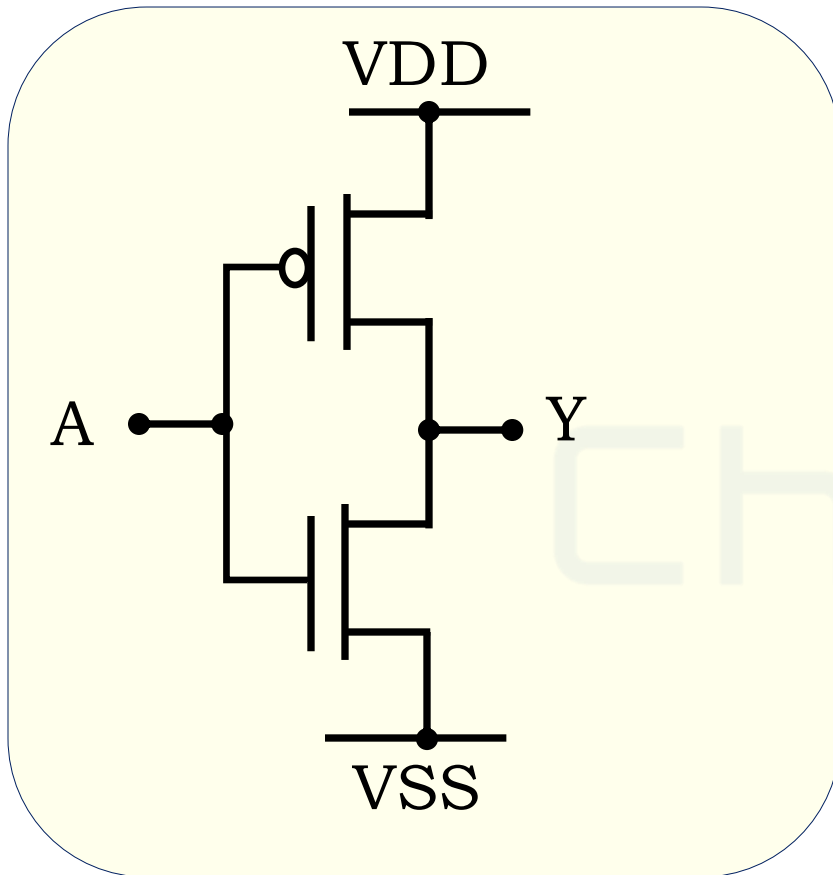
```
cell (INVX16_RVT) {  
    cell_footprint : "AV" ;  
    area : 5.08288 ;  
    cell_leakage_power : 1.635493e+04 ;  
    driver_waveform_rise : "preDrv" ;  
    driver_waveform_fall : "preDrv" ;  
    leakage_power () {  
        when : "A" ;  
        value : "1.6354926e+04" ;  
    }  
}
```



```
pg_pin (VDD) {  
    voltage_name : "VDD" ;  
    pg_type : "primary_power" ; }  
  
pg_pin (VSS) {  
    voltage_name : "VSS" ;  
    pg_type : "primary_ground" ; }
```

PG Pins
Info

Example of Inverter Cell
In Logical Library



Input Pin Information

```
pin (A) {  
    fanout load : 0.149000;  
    related_power_pin: "VDD";  
    related_ground_pin: "VSS";  
    direction : "input";  
    fall capacitance: 8.872269;  
    fall capacitance_range(6.861234, 10.630315);  
    capacitance : 9.161579;  
    rise_capacitance: 9.450888;  
    rise_capacitance_range(6.963421,  
10.719700);  
    max_transition : 1.024000; }
```

Direction
of Pin A

Example of Inverter Cell
In Logical Library


```
timing () {  
  related_pin: "A";  
  timing type: "combinational";  
  timing sense: "negative_unate";  
  cell_rise ("del_1_7_7") {  
    index_1("0.016, 0.032, 0.064, 0.128, 0.256, 0.512, 1.024");  
    index_2("0.1, 4, 8, 16, 32, 64, 82");  
    values("0.0136840, 0.0157534, 0.0174946, 0.0206528, 0.0269745, 0.0397110, 0.0468436", \  
    "0.0198602, 0.0229519, 0.0255315, 0.0298314, 0.0365002, 0.0489652, 0.0561423", \  
    "0.0289894, 0.0335950, 0.0374534, 0.0440349, 0.0537839, 0.0683675, 0.0752175", \  
    "0.0425756, 0.0493727, 0.0553501, 0.0649100, 0.0796316, 0.1009481, 0.1107494", \  
    "0.0635862, 0.0733248, 0.0818341, 0.0958451, 0.1176826, 0.1502164, 0.1646117", \  
    "0.0998563, 0.1119520, 0.1231932, 0.1430849, 0.1755651, 0.2227145, 0.2446352", \  
    "0.1679215, 0.1811684, 0.1949991, 0.2192866, 0.2627818, 0.3323777, 0.3646340");  
  }
```

Cell Rise

Index_1 indicates input transition values

Index_2 indicates Output load values

Values indicates the cell delays

Example - .lib File Power Table Power Related and Capacitance

pin (Y) {

related_power_pin: "VDD";

related_ground_pin: "VSS";

direction: "output";

output voltage: "DC_0";

power_down function: "!VDD+ VSS";

function: "(A)";

Function $Y = A'$

max capacitance: 82.000000;

max_transition : 0.447779;

Maximum Capacitance &
Maximum Transition Values

internal power () {

related_pin: "A";

rise_power ("power_outputs_1") {

index_1("0.016, 0.032, 0.864, 0.128, 0.256, 0.512, 1.024");

index_2("8.1, 4, 8, 16, 32, 64, 82");

values("6.4388667, 6.5176312, 6.5420671, 6.6468374, 6.6069970, 6.6516725, 6.6536123", \

"6.3595386, 6.4036229, 6.4634495, 6.5331329, 6.6520391, 6.5006218, 6.6989942", \

"6.3854688, 6.3931001, 6.3790128, 6.4966706, 6.5325490, 6.5476058, 6.3751805", \

"6.6826158, 6.5657397, 6.5785782, 6.5429272, 6.6211382, 6.5939931, 6.5588181", \

"7.8614849, 7.5329578, 7.2983329, 7.8475769, 6.8732230, 6.9498355, 6.9082195", \

"10.8499613, 10.2588084, 9.7958872, 9.8549845, 8.1319999, 7.6312750, 7.3876207", \

"17.4883019, 16.5710381, 15.8459899, 14.5931958, 12.7576432, 10.6570320, 9.9654296");

Example - .lib File Power Table Power Related and Capacitance

```
internal_power () {  
    related_pin: "A";  
    rise_power ("power_outputs_1") {  
        index_1("0.016, 0.032, 0.064, 0.128, 0.256, 0.512, 1.024");  
        index_2("0.1, 4, 8, 16, 32, 64, 82");  
        values("6.4380667, 6.5176312, 6.5420671, 6.6468374, 6.6069970, 6.6516725, 6.6536123", \  
            "6.3595386, 6.4036229, 6.4634495, 6.5331329, 6.6520391, 6.5006218, 6.6989942", \  
            "6.3854688, 6.3931001, 6.3790128, 6.4966706, 6.5325490, 6.5476058, 6.3751805", \  
            "6.6826158, 6.5657397, 6.5785782, 6.5429272, 6.6211382, 6.5939931, 6.5588181", \  
            "7.8614849, 7.5329578, 7.2983329, 7.8475769, 6.8732230, 6.9498355, 6.9082195", \  
            "10.8499613, 10.2588084, 9.7958872, 9.0549845, 8.1319999, 7.6312750, 7.3876207", \  
            "17.4083019, 16.5710381, 15.8459899, 14.5931958, 12.7576432, 10.6570320, 9.9654296");  
    }  
}
```



Rise Power

Index_1 indicates input transition values

Index_2 indicates Output load values

```
timing () {  
  related_pin: "A";  
  timing type: "combinational";  
  timing sense: "negative_unate";  
  cell_rise ("del_1_7_7") {  
    index_1("0.016, 0.032, 0.064, 0.128, 0.256, 0.512, 1.024");  
    index_2("0.1, 4, 8, 16, 32, 64, 82");  
    values("0.0136840, 0.0157534, 0.0174946, 0.0206528, 0.0269745, 0.0397110, 0.0468436", \  
    "0.0198602, 0.0229519, 0.0255315, 0.0298314, 0.0365002, 0.0489652, 0.0561423", \  
    "0.0289894, 0.0335950, 0.0374534, 0.0440349, 0.0537839, 0.0683675, 0.0752175", \  
    "0.0425756, 0.0493727, 0.0553501, 0.0649100, 0.0796316, 0.1009481, 0.1107494", \  
    "0.0635862, 0.0733248, 0.0818341, 0.0958451, 0.1176826, 0.1502164, 0.1646117", \  
    "0.0998563, 0.1119520, 0.1231932, 0.1430849, 0.1755651, 0.2227145, 0.2446352", \  
    "0.1679215, 0.1811684, 0.1949991, 0.2192866, 0.2627818, 0.3323777, 0.3646340");  
  }
```

Cell Rise

Index_1 indicates input transition values

Index_2 indicates Output load values

Values indicates the cell delays

Physical Library (.lef, .gds)

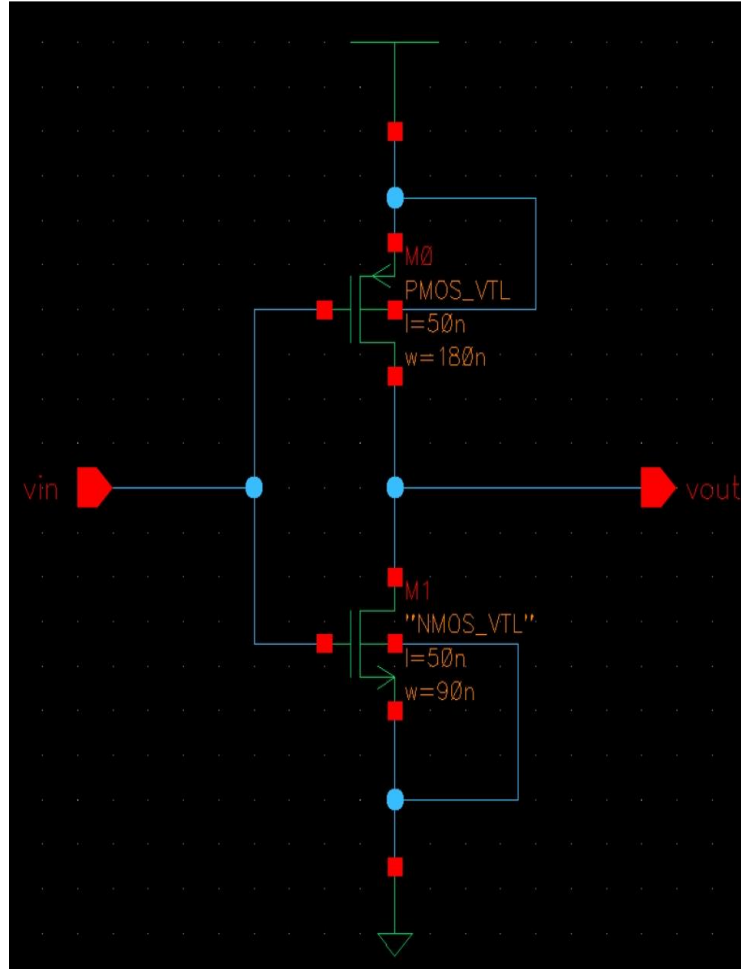
- This library consists of physical information of the standard cells, macros.
- .lef Corresponds to an abstract view which contains Width, Height, Pin location with Metal layers.
- .gds corresponds to the complete layout of std cells/memories which contains all layer information including base layers such as n-well, source, drain etc.
- Physical libraries are in library exchange format (.lef) for cadence tool and .fram or .cell format for Synopsys tool.

LEF file contains Physical information of the Library cells.

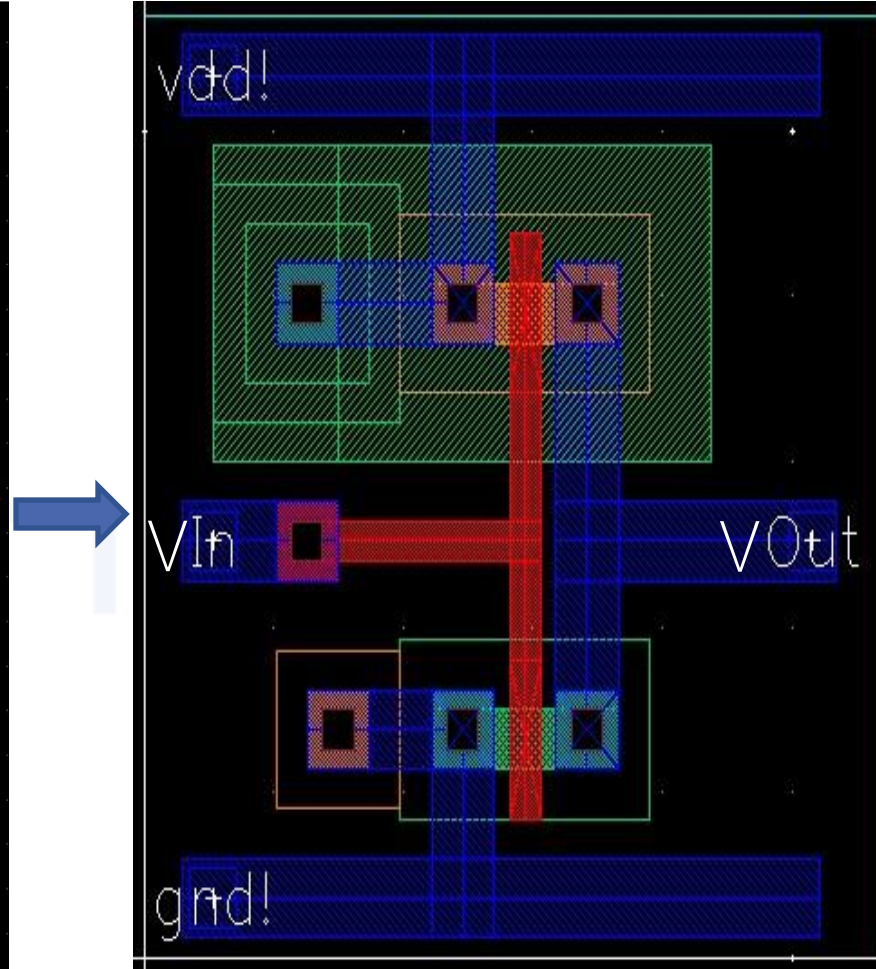
Important Information Present In LEF File Are:

- Cell Dimensions (Width * Height)
- Pin name, Direction (Input/Output/Inout)
- Pin Location of all input pins, output pin and power and ground pins.
- OBS(Routing Blockages).
- Antenna Area Information

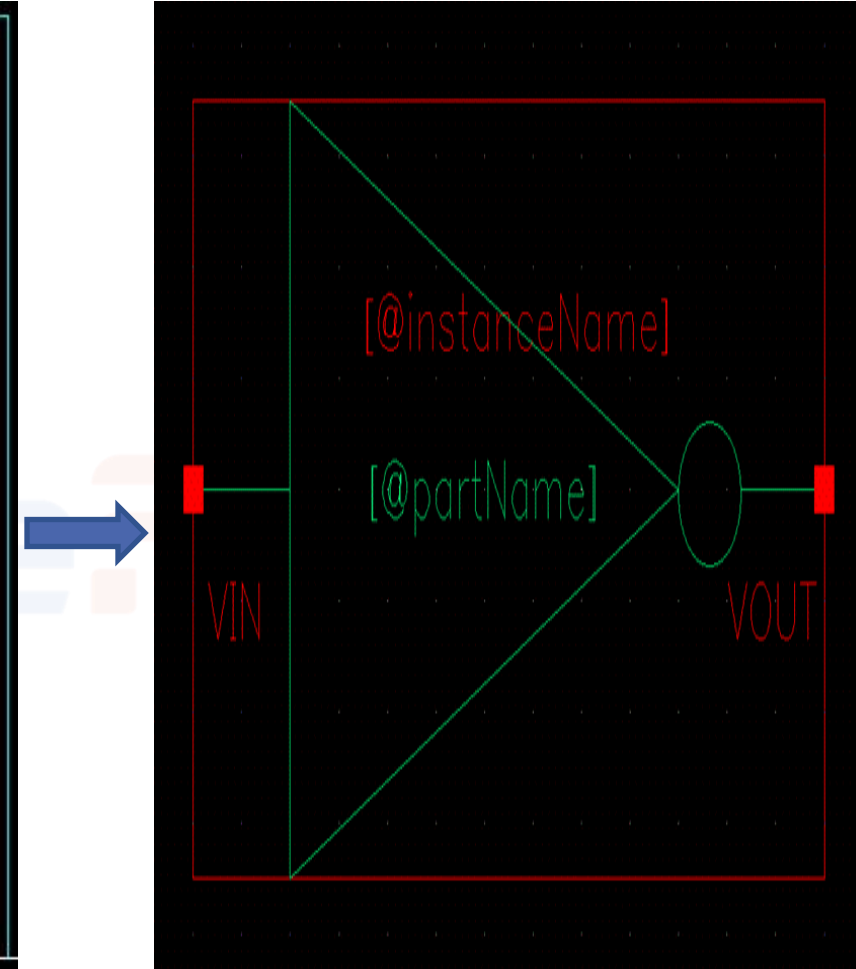
```
MACRO INVXO_RVT
CLASS CORE;
SOURCE USER ;
ORIGIN 0 0;
SIZE 0.76 BY 1.672;
SYMMETRY X Y R90;
SITE unit;
PIN A
DIRECTION INPUT;
USE SIGNAL;
PORT
  LAYER M1;
  RECT 0.2490 0.6800 0.4210 0.7300;
  RECT 0.2490 0.7300 0.3620 0.8150;
END
PORT
  LAYER CO;
  RECT 0.3590 0.6840 0.4010 0.7260;
END
  ANTENNAGATEAREA 0.0237;
END A
PIN VDD
DIRECTION INOUT;
USE POWER;
PORT
  LAYER M1;
  RECT 0.0000 1.6420 0.7600 1.7020;
  RECT 0.2790 0.9280 0.3290 1.6420;
END
PORT
  LAYER CO;
  RECT 0.2830 1.1320 0.3250 1.1740;
  RECT 0.2830 1.1320 0.3250 1.1740;
  RECT 0.2830 1.2240 0.3250 1.2660;
  RECT 0.2830 1.2240 0.3250 1.2660;
  RECT 0.2830 0.9480 0.3250 0.9900;
  RECT 0.2830 0.9480 0.3250 0.9900;
```



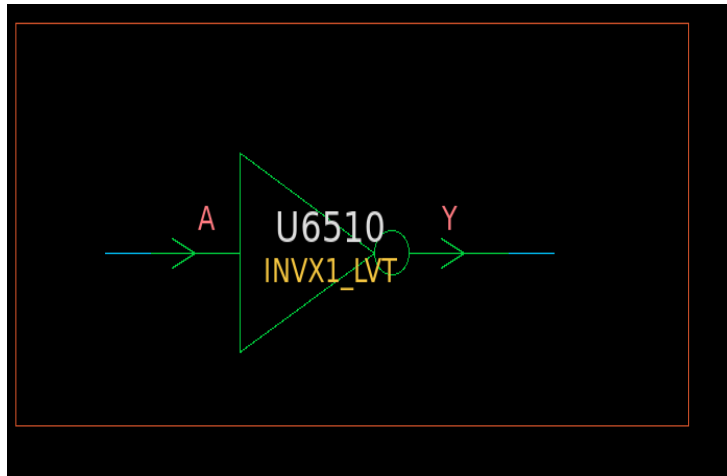
CMOS Inverter
Schematic View



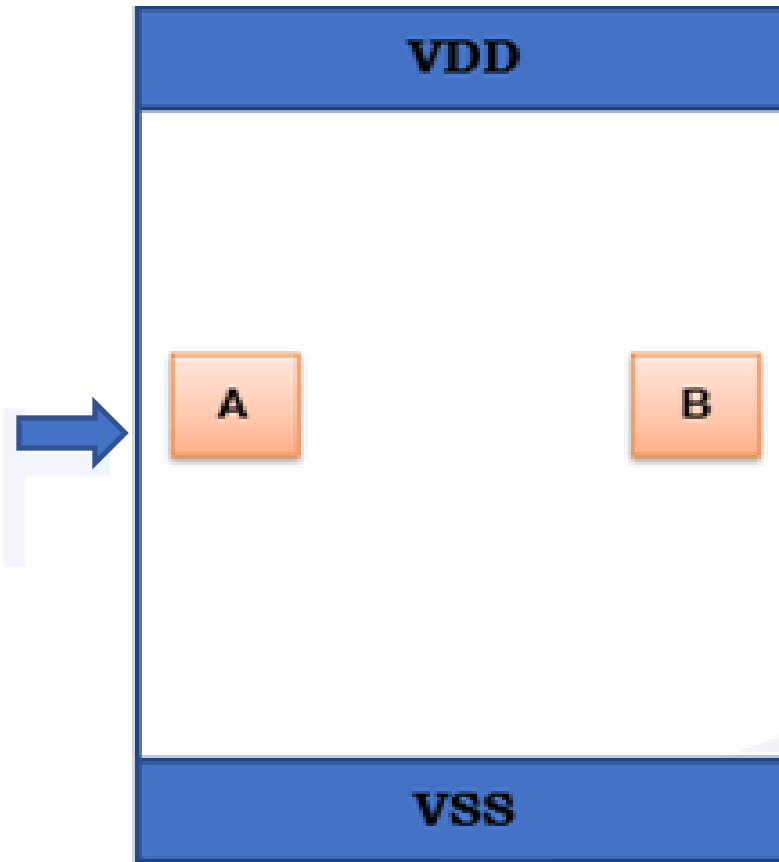
CMOS Inverter
Layout(GDS) View



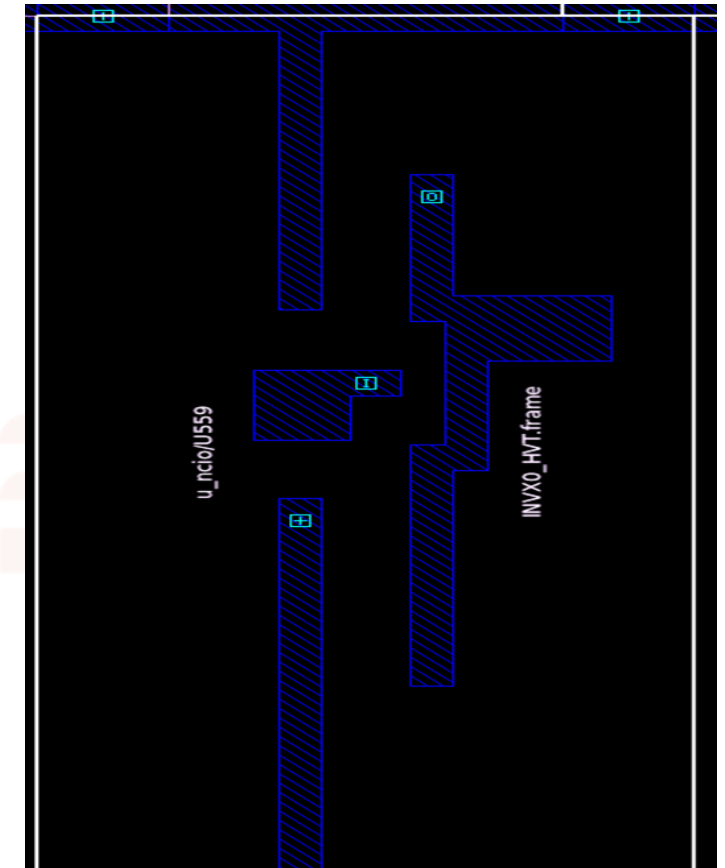
CMOS Inverter
Symbol View



CMOS Inverter
Symbol View



CMOS Inverter
FRAM(LEF) View



CMOS Inverter
GUI FRAM(LEF) View

Inputs to Block PD implementation



Files Required for PD Implementation



✓ Logical Libraries & Physical Libraries



✓ Technology Files

✓ TLU+ file (RC tech File)



✓ Netlist File



✓ SDC File



Additional Information



✓ Power Specification File



This section is an explanation about Design Rule Attributes, Display Attributes, Layout attributes of Layers and VIAs.

- Layout Attributes

Layer number, mask name and pitch information .

- Display Attributes

Details of color, line style and pattern.

- Design Rule Attributes

Max width, min width, default width and min spacing of nets.

```
"M1" {  
    /* layout attributes */  
    layerNumber          = 11  
    isDefaultLayer       = 1  
    maskName             = "metal1"  
    pitch                = 0.152  
    /* display attributes */  
    visible              = 1  
    selectable           = 1  
    blink                = 0  
    color                = "blue"  
    lineStyle            = "solid"  
    pattern              = "impdot"  
    /* design rule attributes */  
  
    minWidth             = 0.05  
    maxWidth             = 5  
    defaultWidth         = 0.05  
    minSpacing           = 0.05  
    sameNetMinSpacing    = 0.055  
}
```

- Different colors are defined with varying intensity of Red, Green and Blue. These colors are used to differentiate layers & visible in layout.
- Physical characteristics & Design rules of layers & vias like Layer Name & Number and Mask name with its appearance (pattern and color).

```
"M1" {  
    /* layout attributes */  
    layerNumber          = 11  
    isDefaultLayer       = 1  
    maskName             = "metal1"  
    pitch                = 0.152  
    /* display attributes */  
    visible              = 1  
    selectable           = 1  
    blink                = 0  
    color                 = "blue"  
    lineStyle            = "solid"  
    pattern               = "impdot"  
    /* design rule attributes */  
  
    minWidth              = 0.05  
    maxWidth              = 5  
    defaultWidth          = 0.05  
    minSpacing            = 0.05  
    sameNetMinSpacing     = 0.055  
}
```

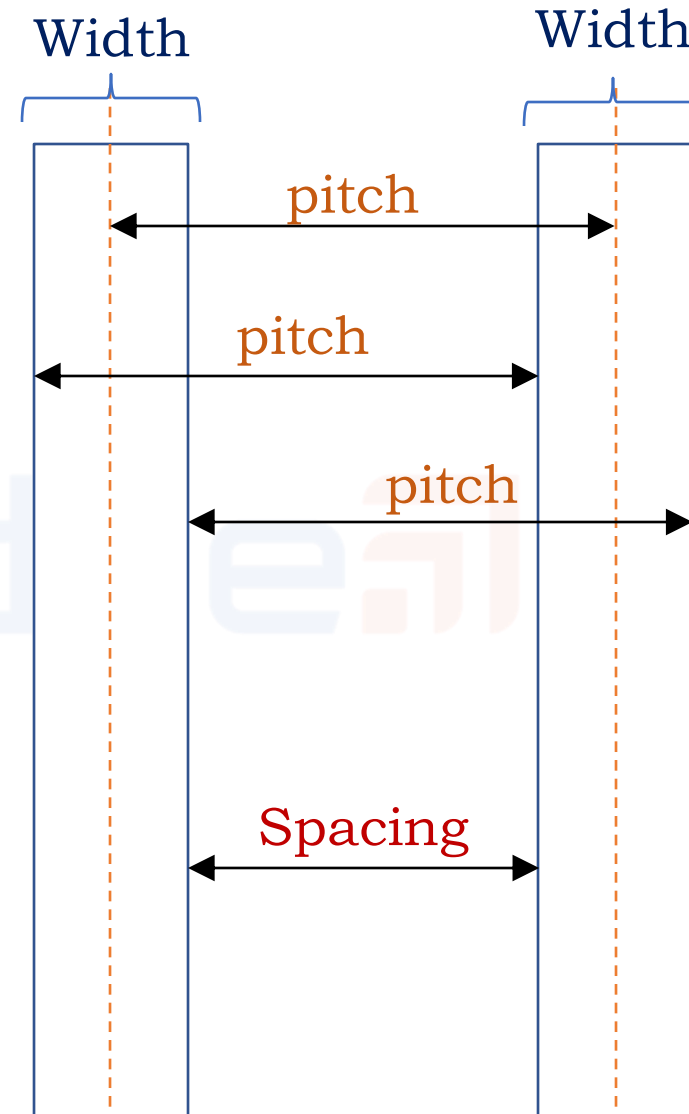
- Width W

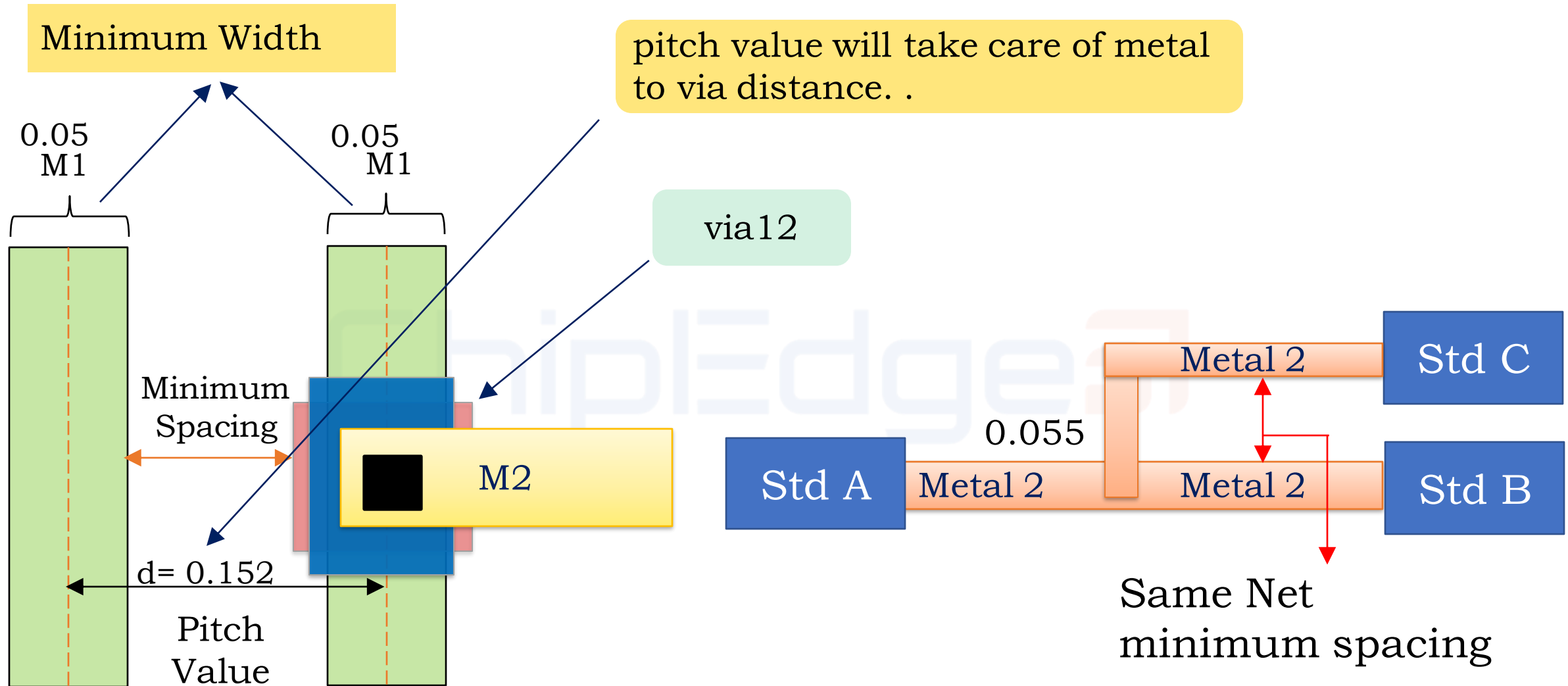
- Pitch

Centre to Centre

- Spacing

Distance between two metal layers





Inputs to Block PD implementation



Files Required for PD Implementation



✓ Logical Libraries & Physical Libraries



✓ Technology Files



✓ TLU+ file (RC tech File)

This file contains R & C information of metal layers for calculating Net delay.

✓ Netlist File



✓ SDC File



Additional Information



✓ Power Specification File



- TLU stands for Table Look Up.
- Table contains the Resistance and Capacitance values of metal layers per unit length, usually at room temp(25°C). Also has temp Coeff's, to calculate R,C at a different temp.
- These (R,C values) are used for calculating Net Delays.
- If TLU+ files are not given then these are extracted from .itf file.
- We load a map file along with the tlu file.
- The data is technology specific and its in Binary Format.

Inputs to Block PD implementation



Files Required for PD Implementation



✓ Logical Libraries & Physical Libraries



✓ Technology Files



✓ TLU+ file (RC tech File)



✓ Netlist File

Brief Explanation of synthesized gate level netlist usually written in Verilog.

✓ SDC File



Additional Information



✓ Power Specification File



- Netlist File- It is the description of gates and their logical connectivity
- The design is initially coded in the RTL and then a synthesized netlist file obtained, which is also called as gate level netlist.
- This gate level netlist is then used to create the design at the block level.
- The contents of Netlist -- cells (std, memories and IOs) and connectivity (pins and nets).

- Logical connectivity.
- Std. Cell instance – Name.
- Macros & Memories Instances.
- Interconnection Details.
- VHDL, Verilog languages used to write netlist.

module counter (clk, reset, cout);

Annotations: "Module name" points to "counter"; "Ports" points to "(clk, reset, cout)".

output [2:0] cout;

Annotation: "Output" points to "cout".

input clk, reset;

Annotation: "Input" points to "input".

wire N6, n3, n4, n6, n7, n8, n9, n10;

Annotation: "Wire names / net names" points to the list of wire names.

DFFX1_Hcout_VT reg[0] (.D(N6), .CLK(clk), .Q(cout[0]), .QN(n10));

DFFSSRX1_HVT cout_reg[1] (.D(n6), .SETB(1'b1), .RSTB(n3), .CLK(clk), .Q(cout[1]),
.QN(n9));

DFFSSRX1_HVT cout_reg[2] (.D(n4), .SETB(1'b1), .RSTB(n3), .CLK(clk), .Q(cout[2]),
.QN(n8));

```
XOR2X1_HVT U10 ( .A1(n9), .A2(n10), .Y(n6) );
```

```
XOR2X1_HVT U11 ( .A1(n7), .A2(n8), .Y(n4) );
```

```
OR2X1_HVT U12 ( .A1(n9), .A2(n10), .Y(n7) );
```

```
AND2X1_HVT U13 ( .A1(n10), .A2(n3), .Y(N6) );
```

```
INVX0_HVT U14 ( .A(reset), .Y(n3) );
```

```
endmodule
```

INVX0_HVT

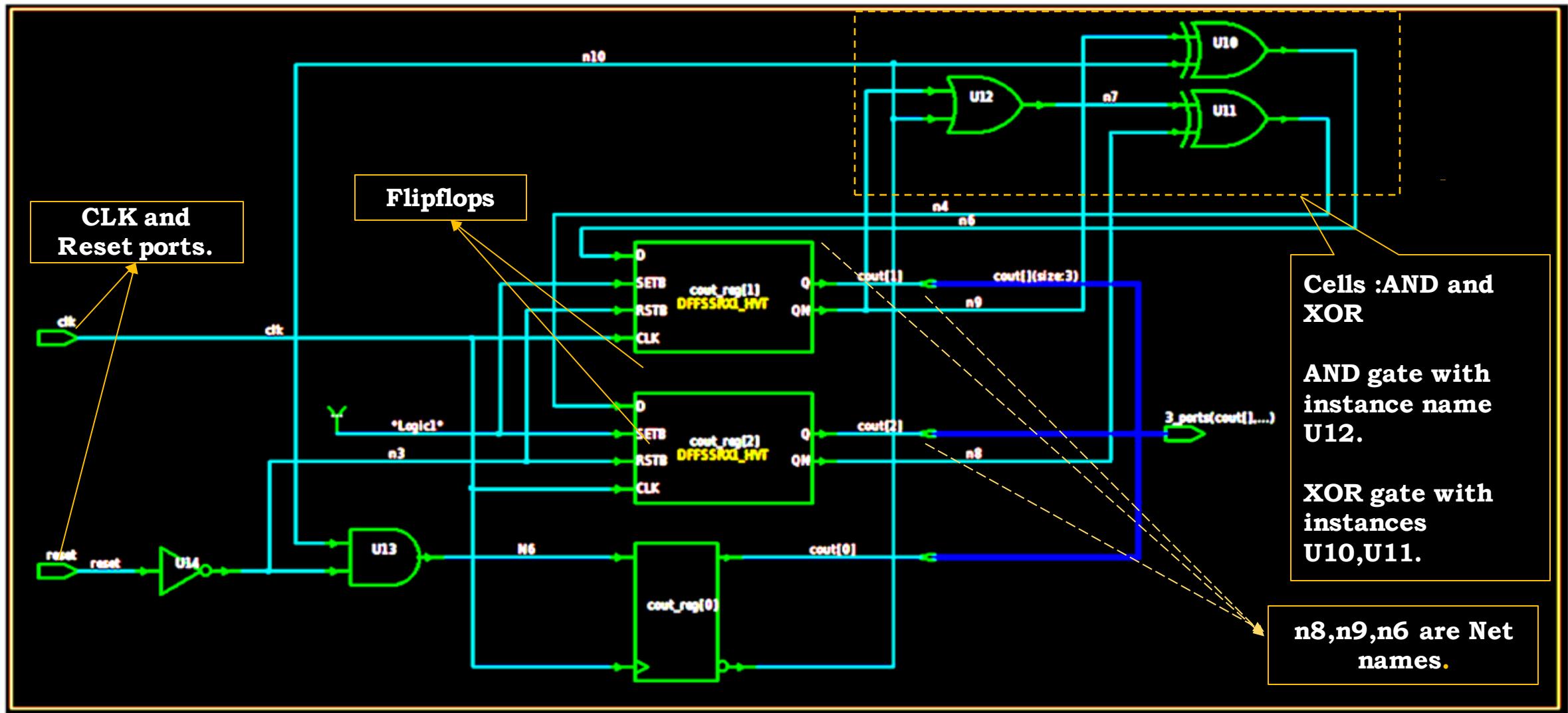
Cell/Reference
name

U14

Instance
name

(.A(reset), .Y(n3));

Pin Connectivity



Inputs to Block PD implementation



Files Required for PD Implementation



✓ Logical Libraries & Physical Libraries



✓ Technology Files



✓ TLU+ file (RC tech File)



✓ Netlist File



✓ SDC File

Additional Information



✓ Power Specification File



The explanation of timing constraints (Clock period) and non-timing constraints (operating conditions, wire load models, DRV).

- Timing Constraints are defined in sdc (Synopsys/Standard Design Constraints)file format
- Clock Definitions
 - Create_clock
 - Create_generated_clock
- Timing Constraints
 - set_input_delay
 - set_output_delay

- Design rule checks
 - `set_max_trans`
 - `set_max_cap`
 - `set_max_fanout`
- Optimization constraints (PPA)

SDC File (.sdc) Example

```
set PERIOD 0.833
set INPUT_DELAY 0.499
set OUTPUT_DELAY 0.499
set CLOCK_LATENCY 0.4
set MIN_IO_DELAY 0.499
set MAX_TRANSITION 0.150
```

Setting the variables

```
## CLOCK BASICS
create_clock -name "clock" -period $PERIOD [get_ports clock]
set_clock_latency $CLOCK_LATENCY [get_clocks clock]
set_clock_uncertainty 0.090 [get_clocks clock]
set_clock_transition 0.120 [get_clocks clock]
```

Clock Related Commands

```
## DRC
set_max_transition $MAX_TRANSITION [current_design]
set_max_capacitance 100 [current_design]
set_max_fanout 64 [current_design]
```

Commands on max_tran,
max_cap and max_fanout DRC's

```
create_clock -name scan_clock -period 8 -waveform {0 4}
[get_ports scan_clk]
create_clock -name scan_clk -period 8 -waveform {0 4}
[get_ports scan_clk]
create_clock -name clk -period 2 -waveform {0 1} [get_ports
clk]
set_clock_uncertainty -setup 0.2 [get_clocks clk]
set_clock_uncertainty -hold 0.02 [get_clocks clk]
set_clock_uncertainty -setup 0.2 [get_clocks scan_clk]
set_clock_uncertainty -hold 0.02 [get_clocks scan_clk]
```

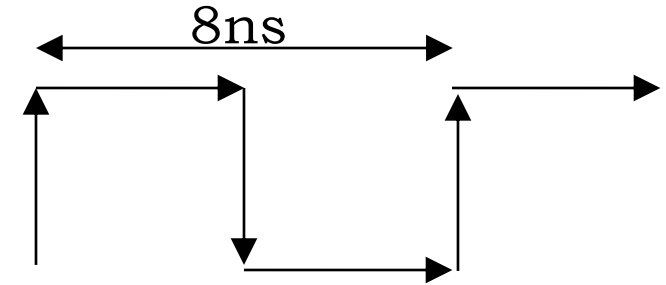
```
create_generated_clock [get_pins clk_div_reg/0] -name  
div2_clk -source [get_ports clk] -divide_by_2  
create_clock -name v_clk -period 2 -waveform {0 1}  
set_clock_uncertainty -setup 0.2 [get_clocks v_clk]  
set_clock_uncertainty -hold 0.02 [get_clocks v_clk]  
  
set_input_delay -clock v_clk -max 1.2 [all_inputs]  
set_output_delay -clock v_clk -max 1.2 [all_outputs]
```

- create_clock (#command)

create_clock is the command to define master clock

- TCL Syntax for creating clock

```
create_clock -name scan_clock -period 8 -waveform {0 4} [get_ports scan_clk]
```



-name used to declare name of the clock . From above scan_clock is the name of the clock

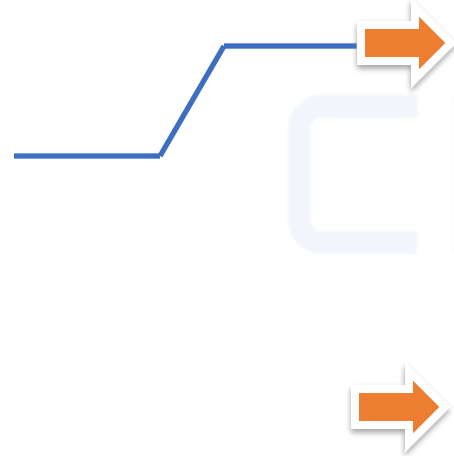
-period used to declare clock period of the clock . From below 8 is the clock period of scan clock

-waveform indicates rise time and fall time of the clock . From above , rise time starts at 0 and fall time starts at 4(i.e rise time =4 and fall time =4)

get_ports scan_clock defines the port named scan_clock .i.e(The clock named scan_clock is driven by the port named scan_clock)

Set_input_delay

Set_input_transiton



Block

Set_output_delay

Set_load



Inputs to Block PD implementation



Files Required for Data Preparation



✓ Power Specification File

Explanation of topics like Power Modes, Power Domains, Power nets and Ground Nets.

- Power Specification file is .upf – Unified Power Format contents:
- Power and Multi voltage Design aspects.
- Setting up Power switch definition, cell definition, level shifter cell definition and other low power related definition.
- Create power domain
- Define Power Modes
- Create Power Nets (VDD and VSS), GND Nets
- Define Power domains and Power rails

- Inputs to Block PD implementation
- Files Required for Data Preparation
- Additional Information



Module 4

Sanity Checks

- In this module you will understand various sanity checks.
- The checks like netlist, SDC and library are discussed in detail with examples.



- Introduction to Sanity Checks
- Netlist Check
 - ✓ Floating input pins and nets
 - ✓ Direct Path From Outputs To Ground
 - ✓ Black Box Module
 - ✓ Multi driven nets.
 - ✓ Combinational Loops
 - ✓ Pins Mismatch
- SDC Check
 - ✓ Clock not reaching the Flip Flop
 - ✓ Multiple clocks driving same register
 - ✓ Unconstrained paths.
 - ✓ Port IO delay missing.
 - ✓ Port slew /load missing.
 - ✓ Clock definition missing.
- Library Check

Introduction to Sanity Checks

The topics like Netlist, SDC and Library checks are explained in detail.

Netlist Check



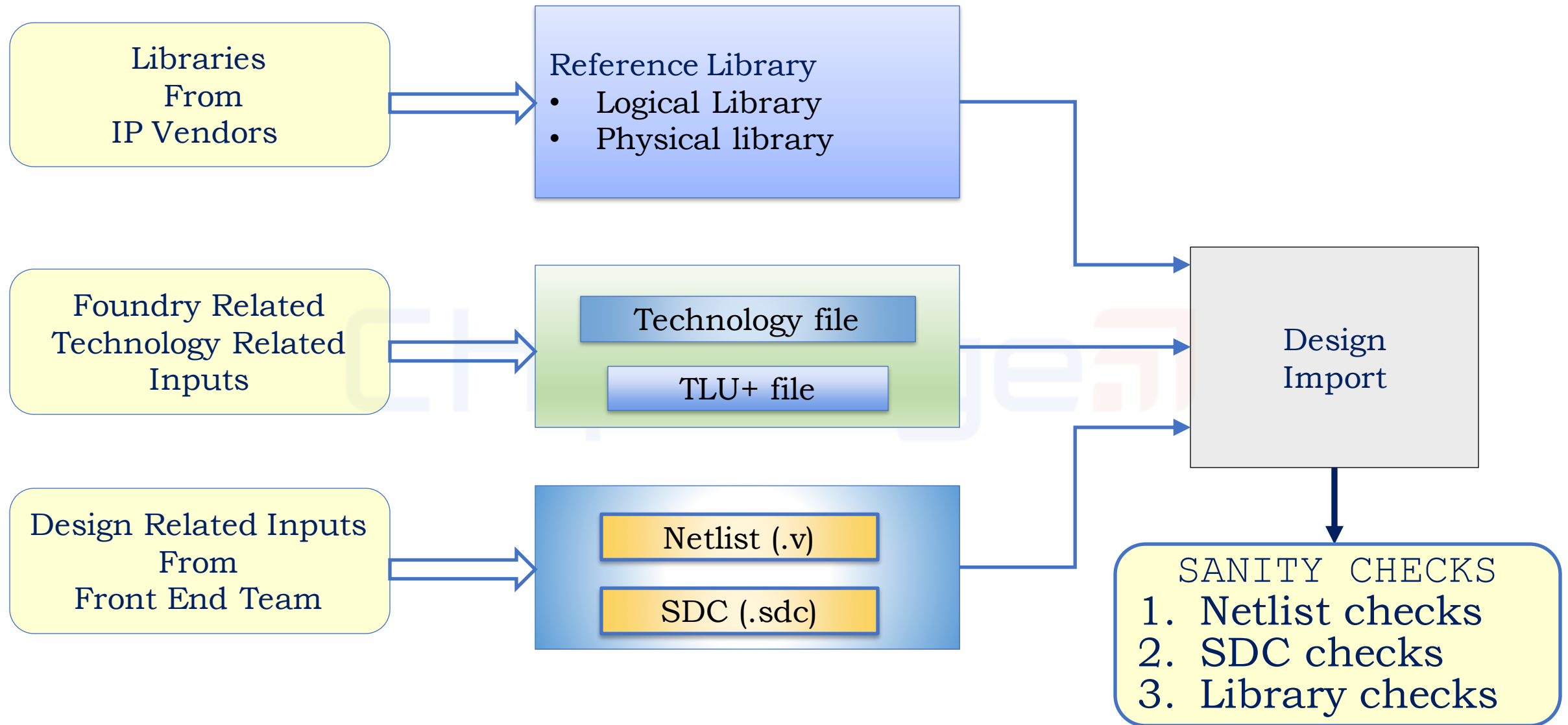
SDC Checks

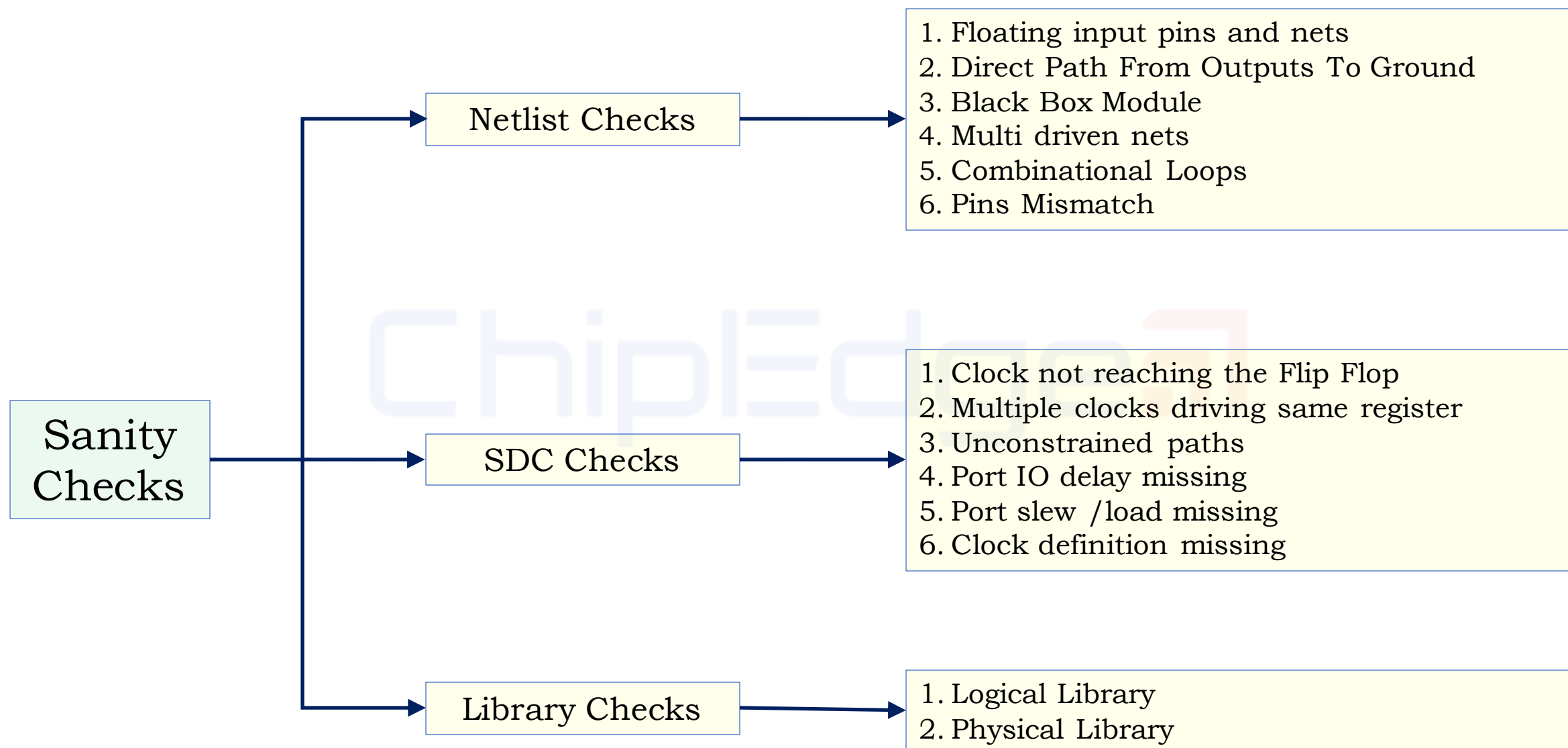


Library Check



- Sanity checks are verification of the synthesized netlist and SDC file.
- If these checks are violated, then there is a possibility of problems in later stages of P&R flow.
- The following Checks are done :
 1. Netlist checks
 2. SDC checks
 3. Library checks
- Note: These Checks should be complete without any error.





Introduction to Sanity Checks



Netlist Check

SDC Checks

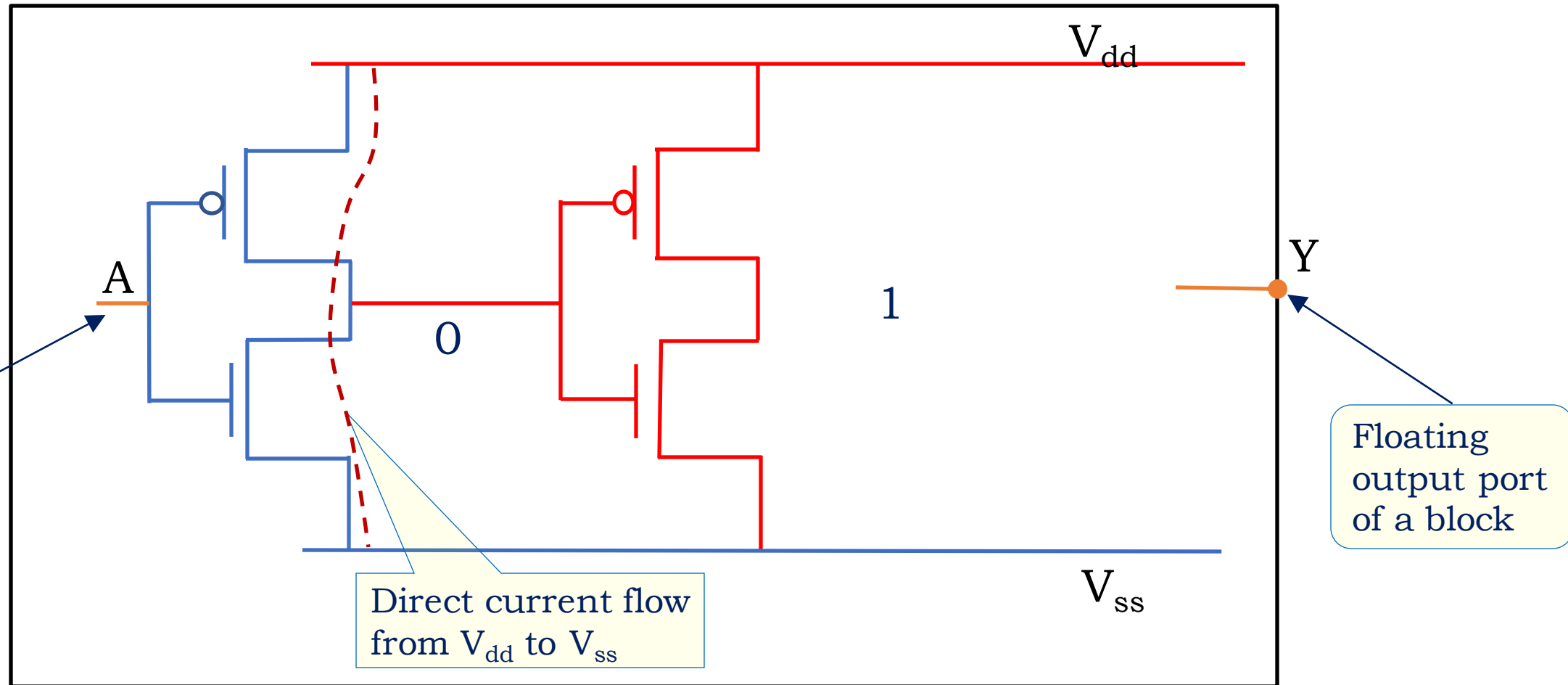
Library Check



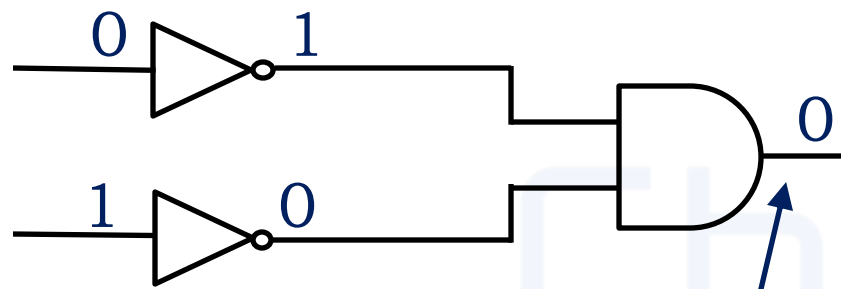
The explanation of topics like Combinational Loops, whether Clock is not reaching the Flip Flop, about Multiple clocks driving same register.

- It checks the quality of netlist and identifies:
 1. Floating input pins and nets
 2. Direct Path From Outputs to Ground
 3. Black Box Module
 4. Multi driven nets
 5. Combinational Loops
 6. Pins Mismatch

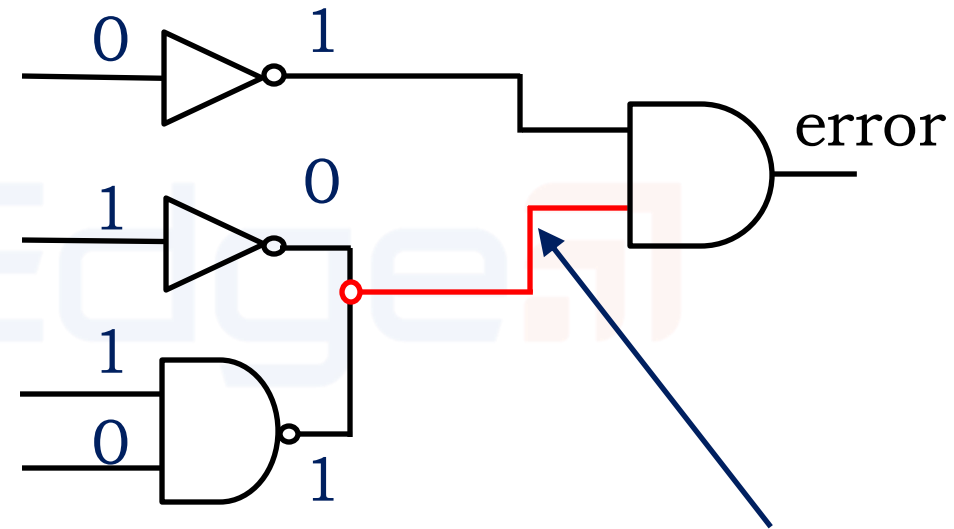
1. Floating Input pins And Output Ports



2. Multi Driven Nets



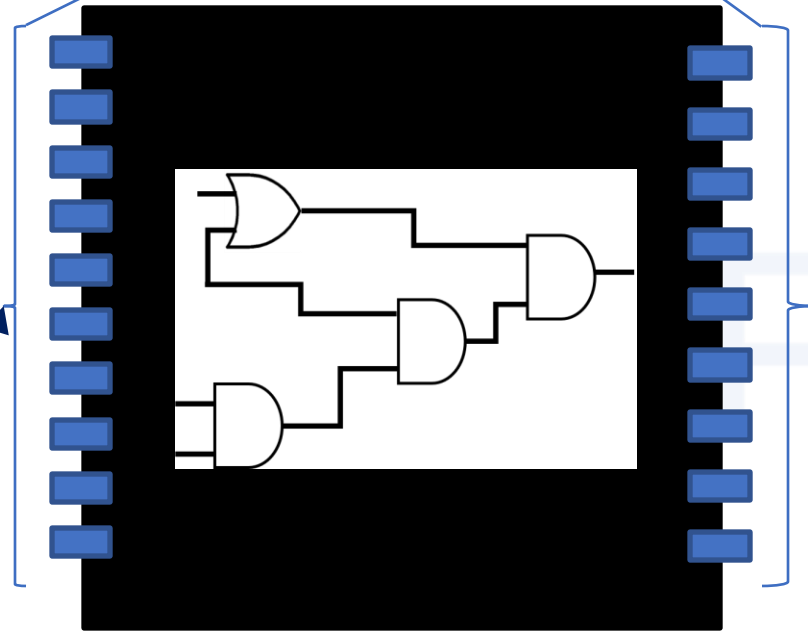
Non Multi driven net



Multi driven net, STA tools cannot efficiently time these circuits

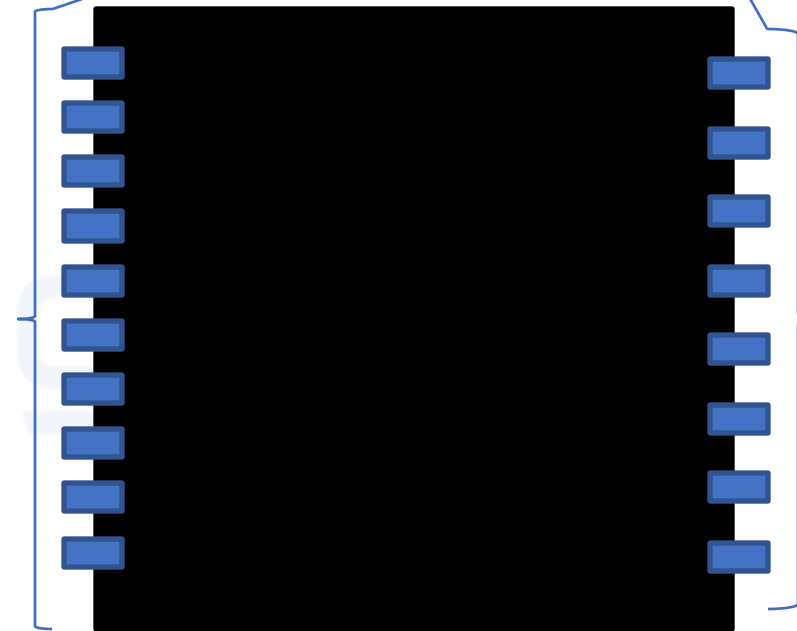
3. Black Box Module

Input and output pins



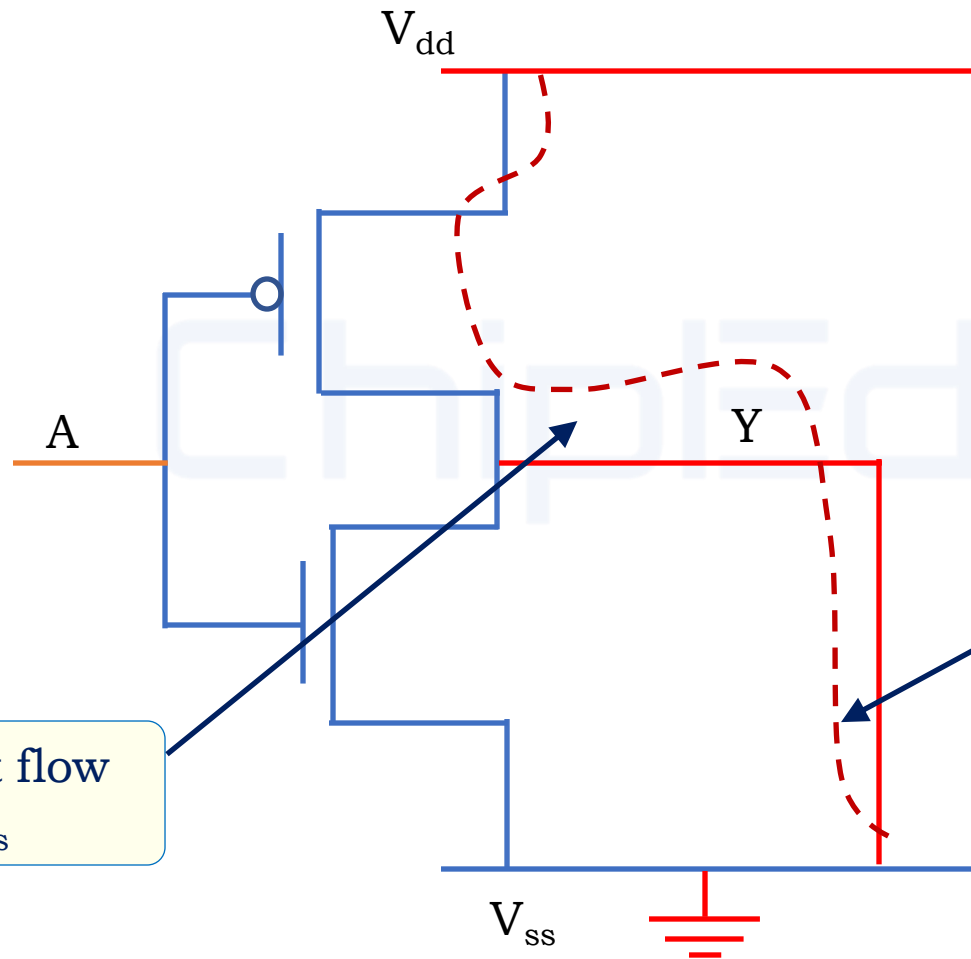
Here, the general Module is defined inside with logic, input and output ports with its functionality information in Verilog file.

Input and output pins



Black Box Module is defined as input and output ports without logic and its functionality information in Verilog file.

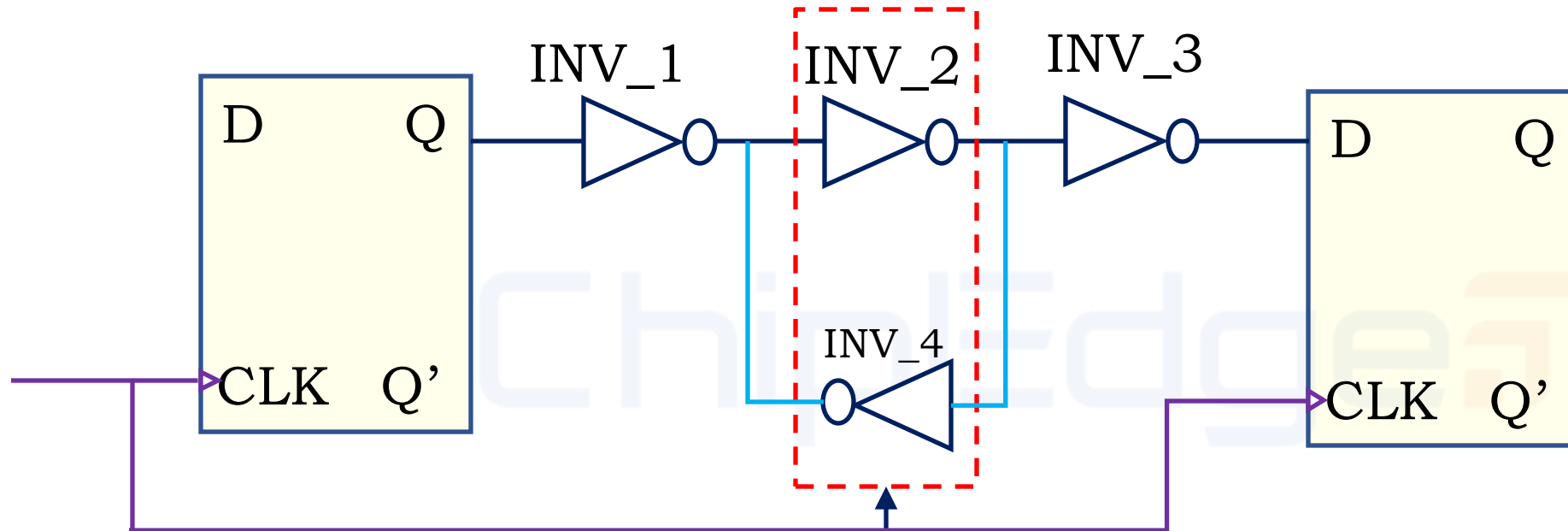
4. Direct Path From Outputs To Ground



Direct current flow from V_{dd} to V_{ss}

The current flow in the cell goes from the V_{dd} to V_{ss} directly if output is connected to ground. This leads to a design error which is under design check.

5. Combinational Loops



Combinational loops
cannot be timed
correctly by STA tools

6. Pin Direction Mismatch

The design primarily uses three types of pins: INPUT, OUTPUT, and INOUT. Pin direction mismatch in the design will occur if one of these turns into another, such as INOUT becoming INPUT or OUTPUT or vice versa. Due to this, logic propagation is affected, and we will miss the logic.


```
P/PP1[30]' has no loads. (DCHK-009)
Warning: In design 'Multiplier/ADDP', 'hierarchical' net 'Multiplier/ADDP/PP0[32]' driven by inside hier pin 'Multiplier/ADDP/PP0[32]' has no loads. (DCHK-009)
Warning: In design 'Multiplier/ADDP', 'hierarchical' net 'Multiplier/ADDP/SYNOPSYS_UNCONNECTED__0' driven by outside hier pin 'Multiplier/ADDP/CSA13/C[0]' has no loads. (DCHK-009)
Warning: In design 'Multiplier/ADDP', 'hierarchical' net 'Multiplier/ADDP/SYNOPSYS_UNCONNECTED__1' driven by outside hier pin 'Multiplier/ADDP/CSA12/C[0]' has no loads. (DCHK-009)
Warning: In design 'Multiplier/ADDP', 'hierarchical' net 'Multiplier/ADDP/SYNOPSYS_UNCONNECTED__2' driven by outside hier pin 'Multiplier/ADDP/CSA11/C[0]' has no loads. (DCHK-009)
Warning: In design 'Multiplier/ADDP', 'hierarchical' net 'Multiplier/ADDP/SYNOPSYS_UNCONNECTED__3' driven by outside hier pin 'Multiplier/ADDP/CSA10/C[0]' has no loads. (DCHK-009)
Warning: In design 'Multiplier/ADDP', 'hierarchical' net 'Multiplier/ADDP/SYNOPSYS_UNCONNECTED__4' driven by outside hier pin 'Multiplier/ADDP/CSA21/C[0]' has no loads. (DCHK-009)
Warning: In design 'Multiplier/ADDP', 'hierarchical' net 'Multiplier/ADDP/SYNOPSYS_UNCONNECTED__5' driven by outside hier pin 'Multiplier/ADDP/CSA20/C[0]' has no loads. (DCHK-009)
Warning: In design 'Multiplier/ADDP', 'hierarchical' net 'Multiplier/ADDP/SYNOPSYS_UNCONNECTED__6' driven by outside hier pin 'Multiplier/ADDP/CSA30/C[0]' has no loads. (DCHK-009)
Warning: In design 'Multiplier/ADDP', 'hierarchical' net 'Multiplier/ADDP/SYNOPSYS_UNCONNECTED__7' driven by outside hier pin 'Multiplier/ADDP/CSA40/C[0]' has no loads. (DCHK-009)
Warning: In design 'PwrCtrl', 'hierarchical' net 'PwrCtrl/memx_stat_reg_next[2]' has no drivers. (DCHK-010)
Warning: In design 'PwrCtrl', 'hierarchical' net 'PwrCtrl/memy_stat_reg_next[2]' has no drivers. (DCHK-010)
Warning: In design 'AddressGen/add_24', 'hierarchical' net 'AddressGen/add_24/CI' driven by inside hier pin 'AddressGen/add_24/CI' has no loads. (DCHK-009)
Warning: In design 'AddressGen/add_24', 'hierarchical' net 'AddressGen/add_24/B[64]' driven by inside hier pin 'AddressGen/add_24/B[64]' has no loads. (DCHK-009)
Warning: In design 'AddressGen/add_24', 'hierarchical' net 'AddressGen/add_24/A[64]' driven by inside hier pin 'AddressGen/add_24/A[64]' has no loads. (DCHK-009)
Warning: In design 'AddressGen/add_24', 'hierarchical' net 'AddressGen/add_24/C0' has no drivers. (DCHK-010)
1
icc2 shell> █
```

Introduction to Sanity Checks



Netlist Check



SDC Checks

The explanation of various SDC Checks.

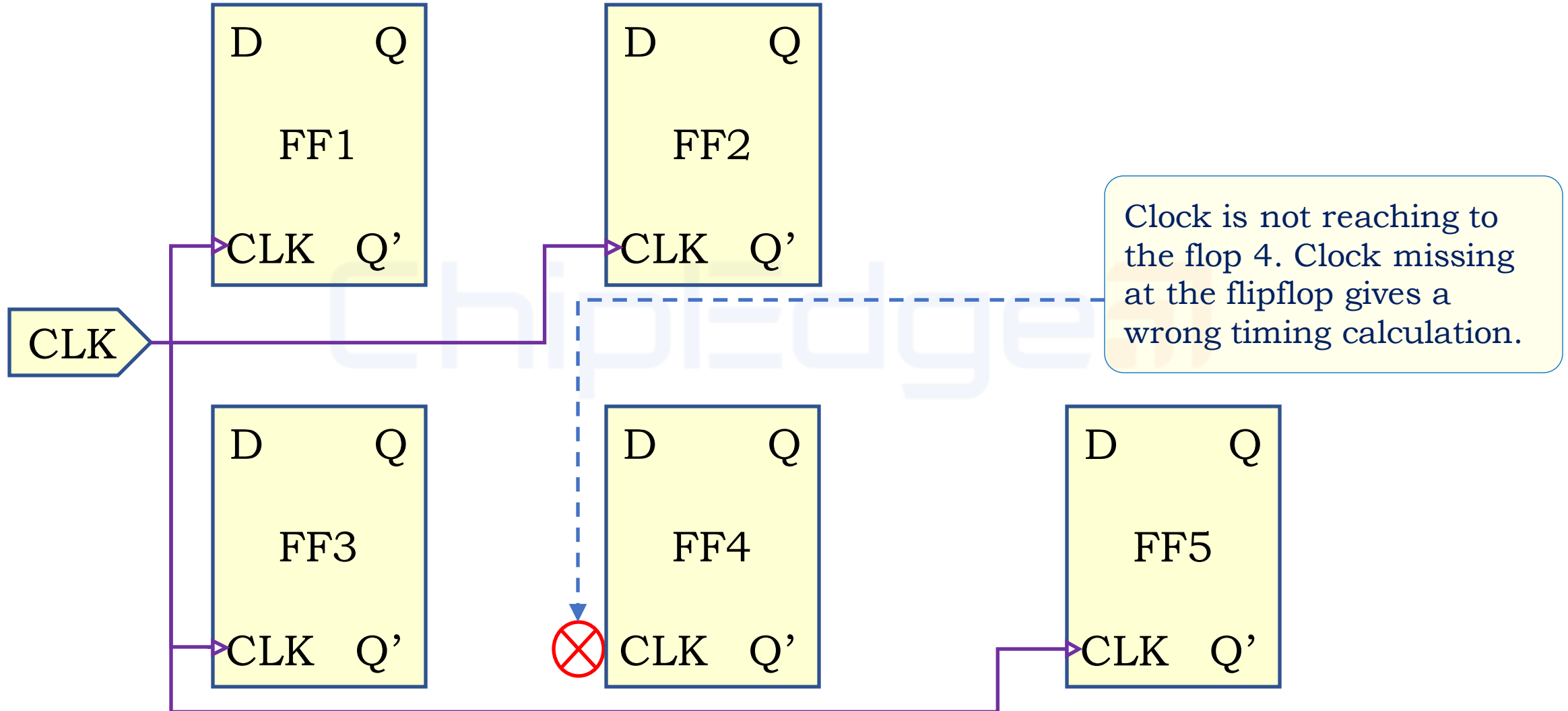
Library Check



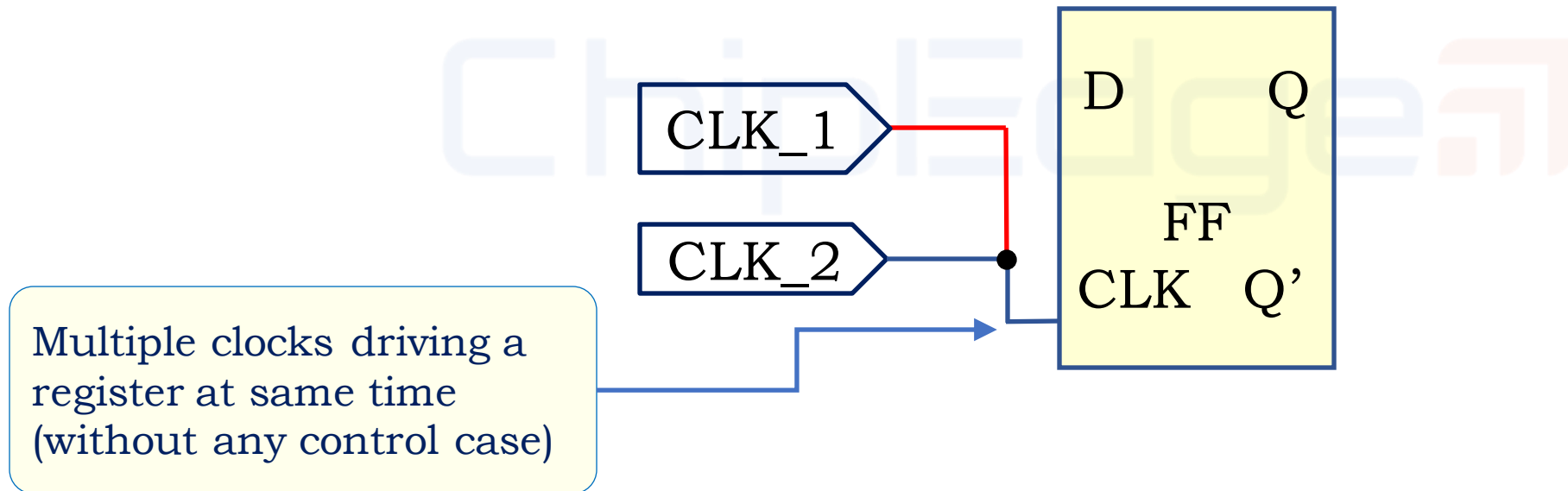
The SDC (timing related) checks are as follows:

1. Clock not reaching the Flip Flop
2. Multiple clocks driving same register
3. Unconstrained paths
4. Port IO delay missing
5. Port slew /load missing
6. Clock definition missing

1. Clock Is Not Reaching The Flipflop



2. Multiple Clocks Driving Same Register



3. Unconstrained Paths

Unconstrained paths are those not having any timing constraints, such as `set_input_delay`, `create_clock`, etc

4. Port IO delay missing

5. Port slew /load missing

6. Clock definition missing

```
icc2_shell> check_timing
Information: Timer using 16 threads
```

Rule	Type	Count	Message
TCK-001	Warn	18	The reported endpoint '%endpoint' is unconstrained. Reason: '%re...

```
Total messages: 18, Errors: 0, Warnings: 18, Info: 0
```

Check_timing command reports any issues related to clock definitions missing input output delays etc.

Messages

```
-----
Warning: The reported endpoint 'DIGIT_REG_INST/digit_out_reg[7]/SETB' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
Warning: The reported endpoint 'DIGIT_REG_INST/digit_out_reg[6]/SETB' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
Warning: The reported endpoint 'DIGIT_REG_INST/digit_out_reg[5]/SETB' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
Warning: The reported endpoint 'DIGIT_REG_INST/digit_out_reg[4]/SETB' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
Warning: The reported endpoint 'DIGIT_REG_INST/digit_out_reg[3]/SETB' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
Warning: The reported endpoint 'DIGIT_REG_INST/digit_out_reg[2]/SETB' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
Warning: The reported endpoint 'DIGIT_REG_INST/digit_out_reg[1]/SETB' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
Warning: The reported endpoint 'DIGIT_REG_INST/digit_out_reg[0]/SETB' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
Warning: The reported endpoint 'DIGIT_REG_INST/flag_out_reg/SETB' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
Warning: The reported endpoint 'tdigit[7]' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
Warning: The reported endpoint 'tdigit[6]' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
Warning: The reported endpoint 'tdigit[5]' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
Warning: The reported endpoint 'tdigit[4]' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
Warning: The reported endpoint 'tdigit[3]' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
Warning: The reported endpoint 'tdigit[2]' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
Warning: The reported endpoint 'tdigit[1]' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
Warning: The reported endpoint 'tdigit[0]' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
Warning: The reported endpoint 'tdigit_flag' is unconstrained. Reason: 'false path'. Mode:'func'. (TCK-001)
```

1

Introduction to Sanity Checks



Netlist Check



SDC Checks



Library Check

Performs consistency checks between the logical and physical library.

`check_library` command checks the following:

- Checks consistency between logic libraries
- Checks the integrity of individual logic and physical libraries
- Checks consistency between logic libraries and physical libraries
- Checks consistency between physical libraries and technology file

- Introduction to Sanity Checks
- Netlist Check
- Floating input pins and nets
- Direct Path From Outputs To Ground
- Black Box Module
- Multi driven nets.
- Combinational Loops
- Pins Mismatch
- SDC Check
- Clock not reaching the Flip Flop
- Multiple clocks driving same register
- Unconstrained paths.
- Port IO delay missing.
- Port slew /load missing.
- Clock definition missing.
- Library Check

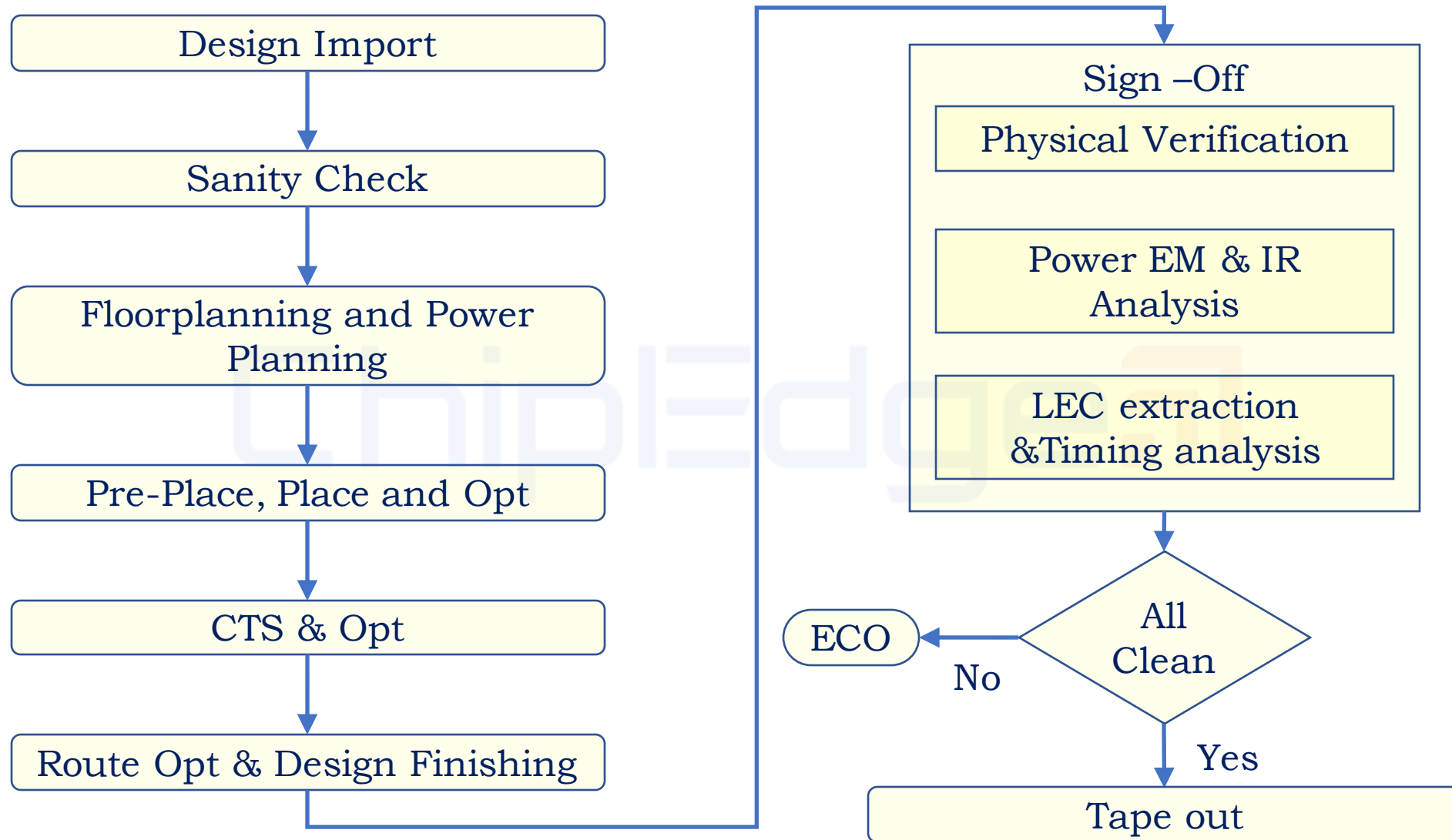
Module 5

PD Flow and Terminology

- In this module, you will learn about Physical Design Flow in detail.
- Brief overview of various topics like Full Chip Partitioning, Floorplan, Powerplan, Placement, CTS, Routing and Signoff are addressed.



- Physical Design Flow
- Full Chip Partitioning
 - ✓ Floorplanning
- Powerplan
- Placement & Optimization
- CTS & Optimization
- Routing & Optimization
- Signoff



Full Chip Partitioning

This section explains about Full Chip Partitioning and Floorplanning.

✓ Floorplanning



Powerplan



Placement & Optimization



CTS & Optimization



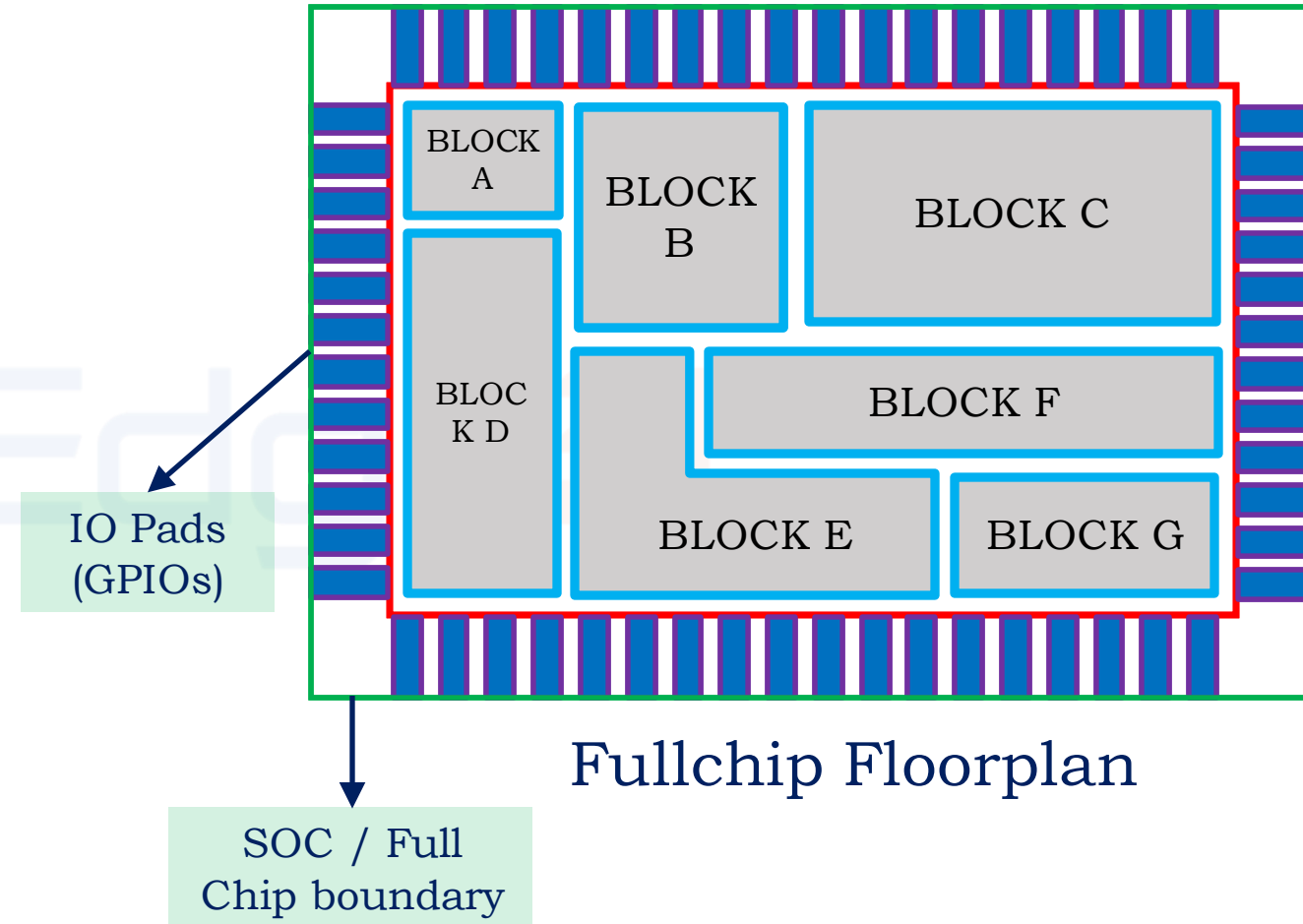
Routing & Optimization



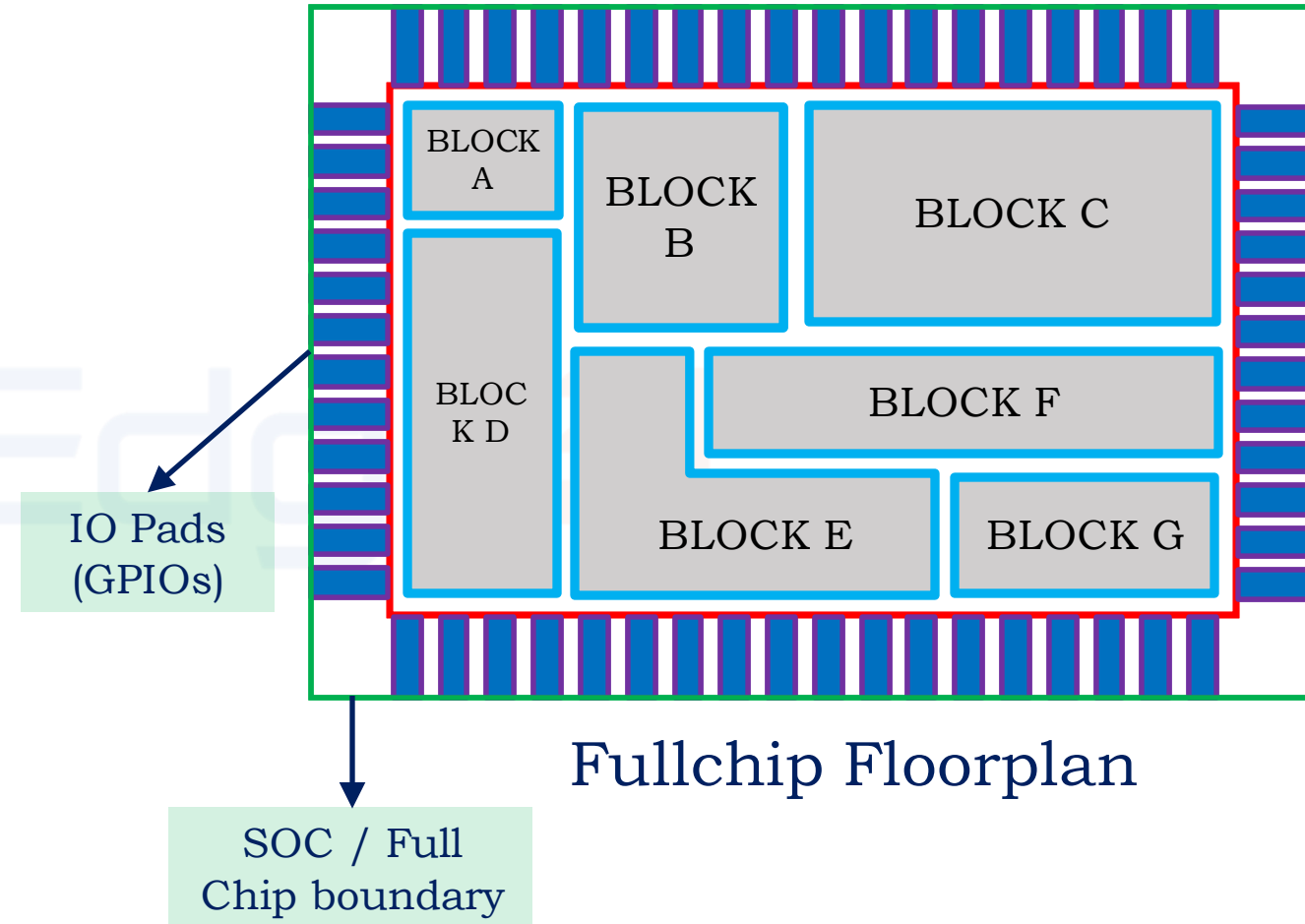
Signoff



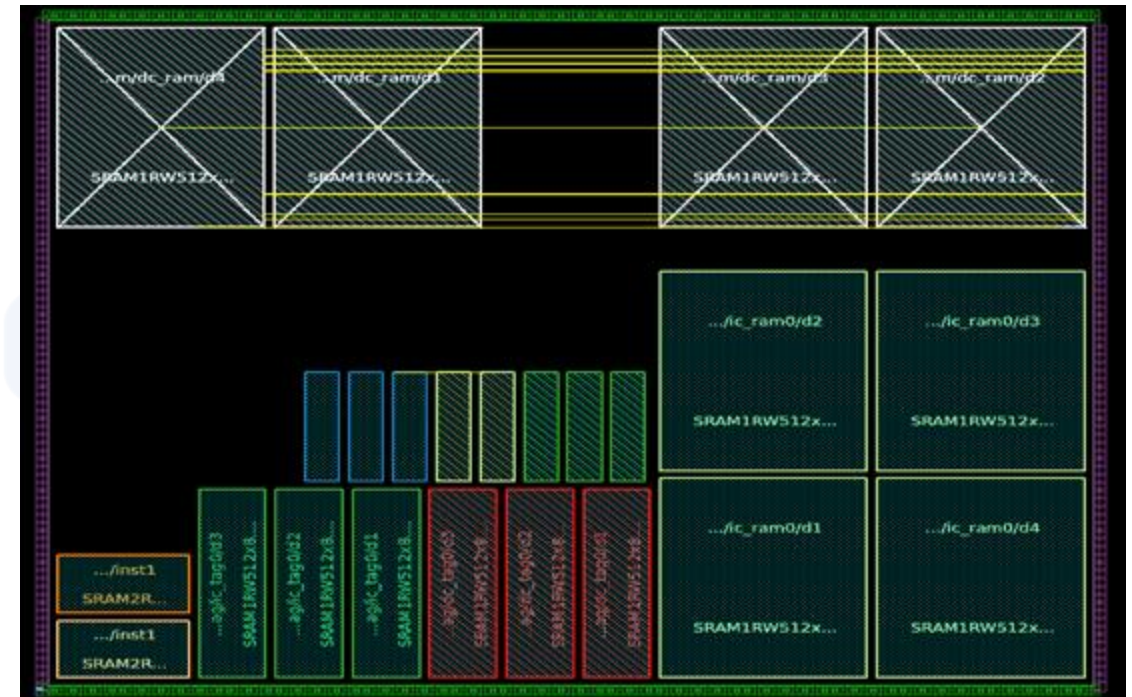
- Partitioning divides a large circuit into group of smaller blocks.
- These blocks can be designed independently and simultaneously to speed up the design process.
- An ASIC design can be partitioned logically or Physically.



- ✓ Logical partitioning is done in the early stages, the design is partitioned according to its logical functions.
- ✓ Physical partitioning is performed during the physical design activity.
- The partitioned blocks are implemented independently at block level and the outputs are provided for integration at full-chip level.



- Plan the size, shape of the block.
- Plan the positions of IO ports.
- Plan the positions of the modules/macros.
- A good floorplan helps in optimizing the circuit performance in terms of chip timing routability, wire length , and area etc.



Full Chip Partitioning



Powerplan

This section explains about power plan.

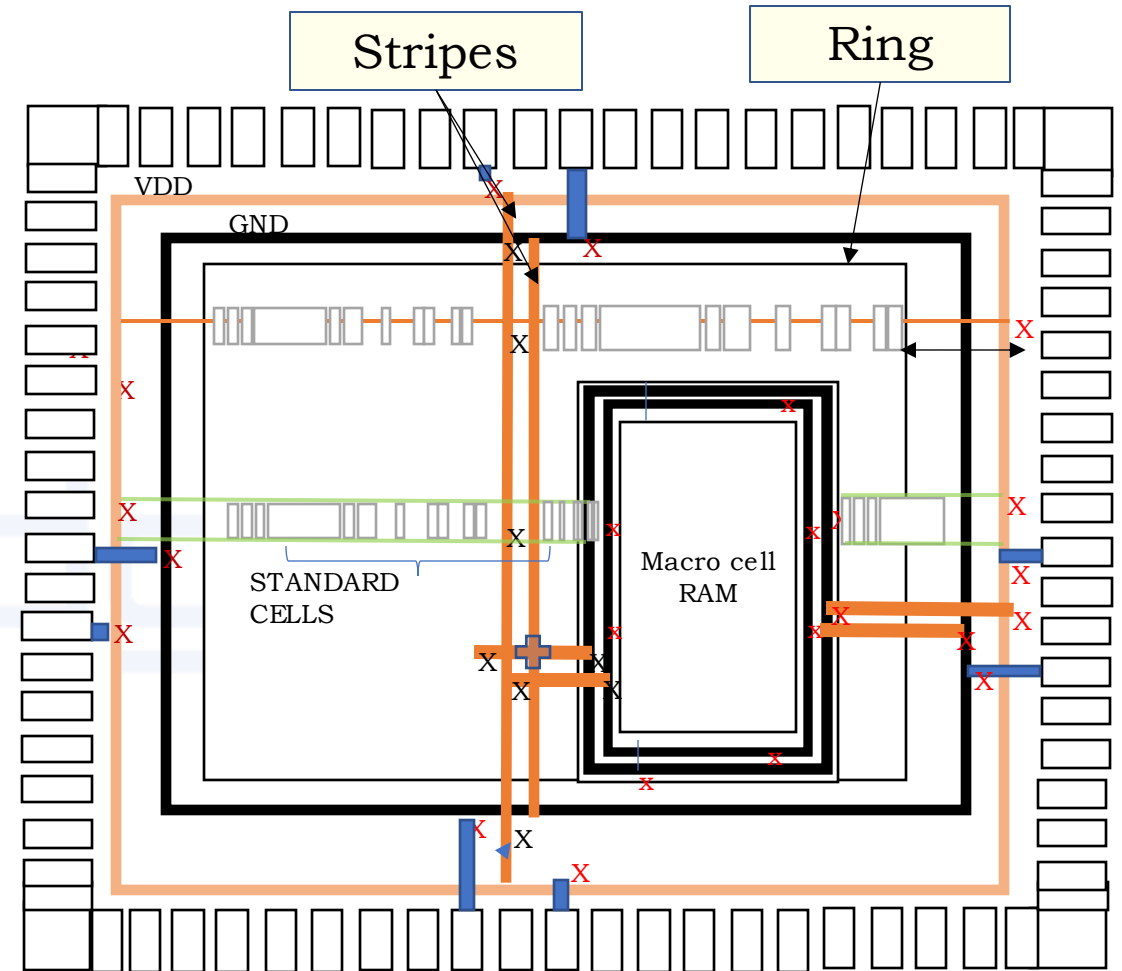
Placement & Optimization 

CTS & Optimization 

Routing & Optimization 

Signoff 

- Power grid network is created to distribute power to each part of the design efficiently.
- **Rings** : Carries VDD and VSS around the full-chip.
- **Stripes/Straps** : Carries VDD and VSS from Rings across the chip/block.
- **Rails** : Connect VDD and VSS to the standard cell VDD and VSS.



Full Chip Partitioning



Powerplan



Placement & Optimization

This section explains about Placement and Optimization.

CTS & Optimization



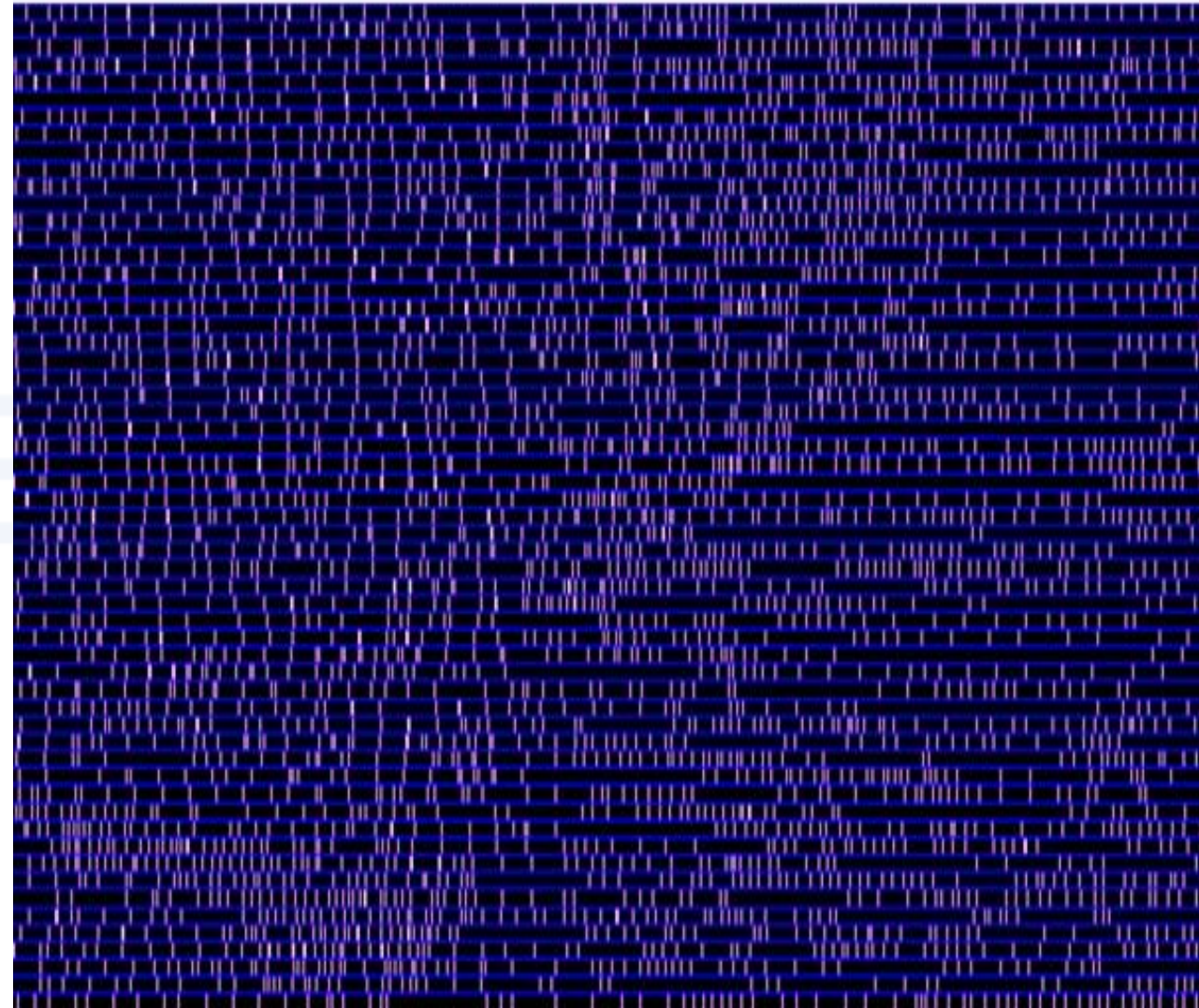
Routing & Optimization



Signoff



- Placement is the process of placing standard cells in the rows created at floorplan/powerplan stage.
- Goal is timing clean & routable design.
- Quality of routing is highly determined by placement.
- Three stages : Global Placement, Legalize Placement and Detailed Placement.
- Design is optimized for Timing, Congestion, Power, Area.





Full Chip Partitioning



Powerplan



Placement & Optimization



CTS & Optimization

This section explains about Clock Tree Synthesis

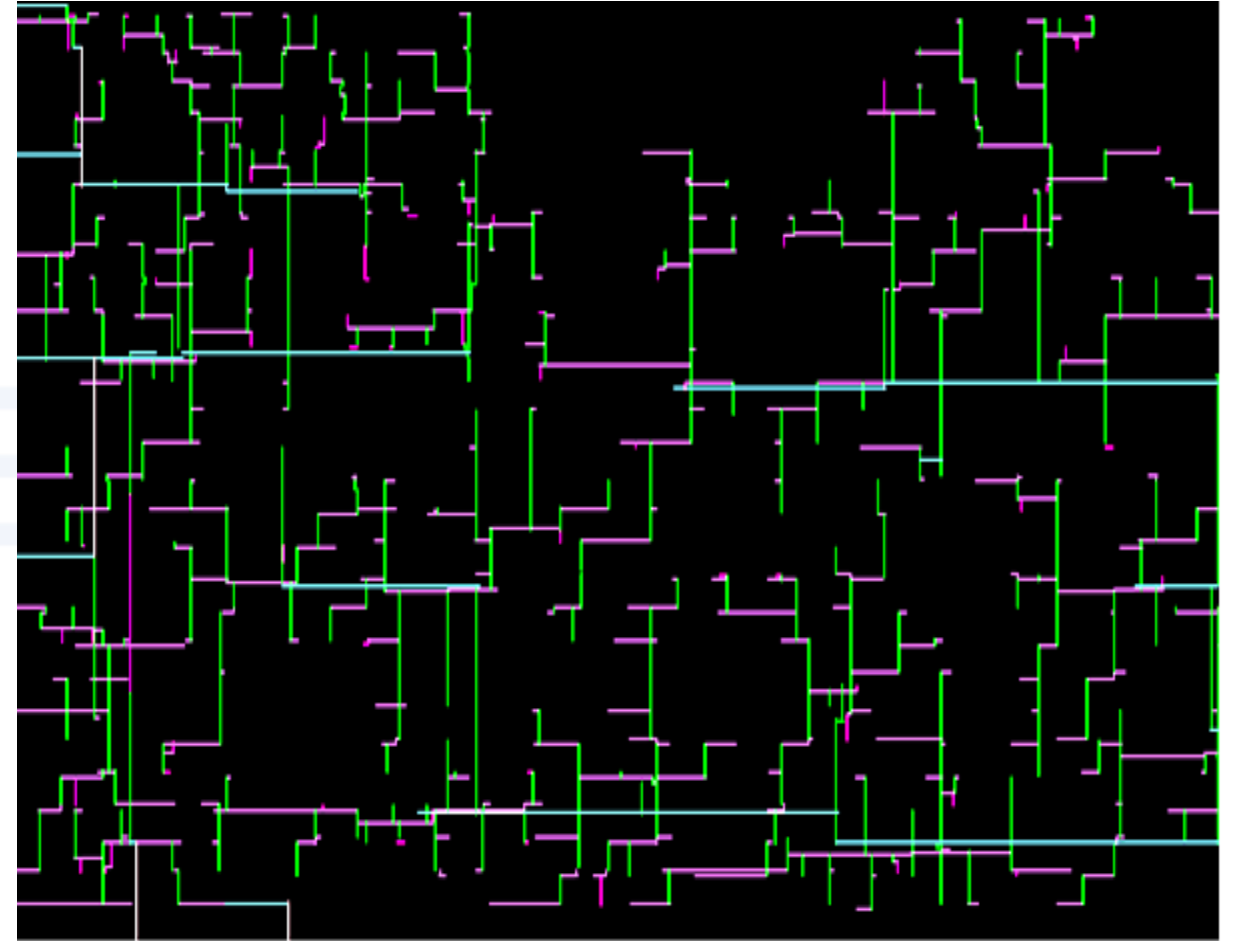
Routing & Optimization



Signoff



- Clock Tree Synthesis is a process which makes sure that the clock gets distributed evenly to all sequential elements in a design.
- The goal of CTS is to minimize skew and latency.
- The placement data will be given as input for CTS, along with the clock tree constraints.
- Design is optimized for Timing, Congestion & Power.





Full Chip Partitioning



Powerplan



Placement & Optimization



CTS & Optimization



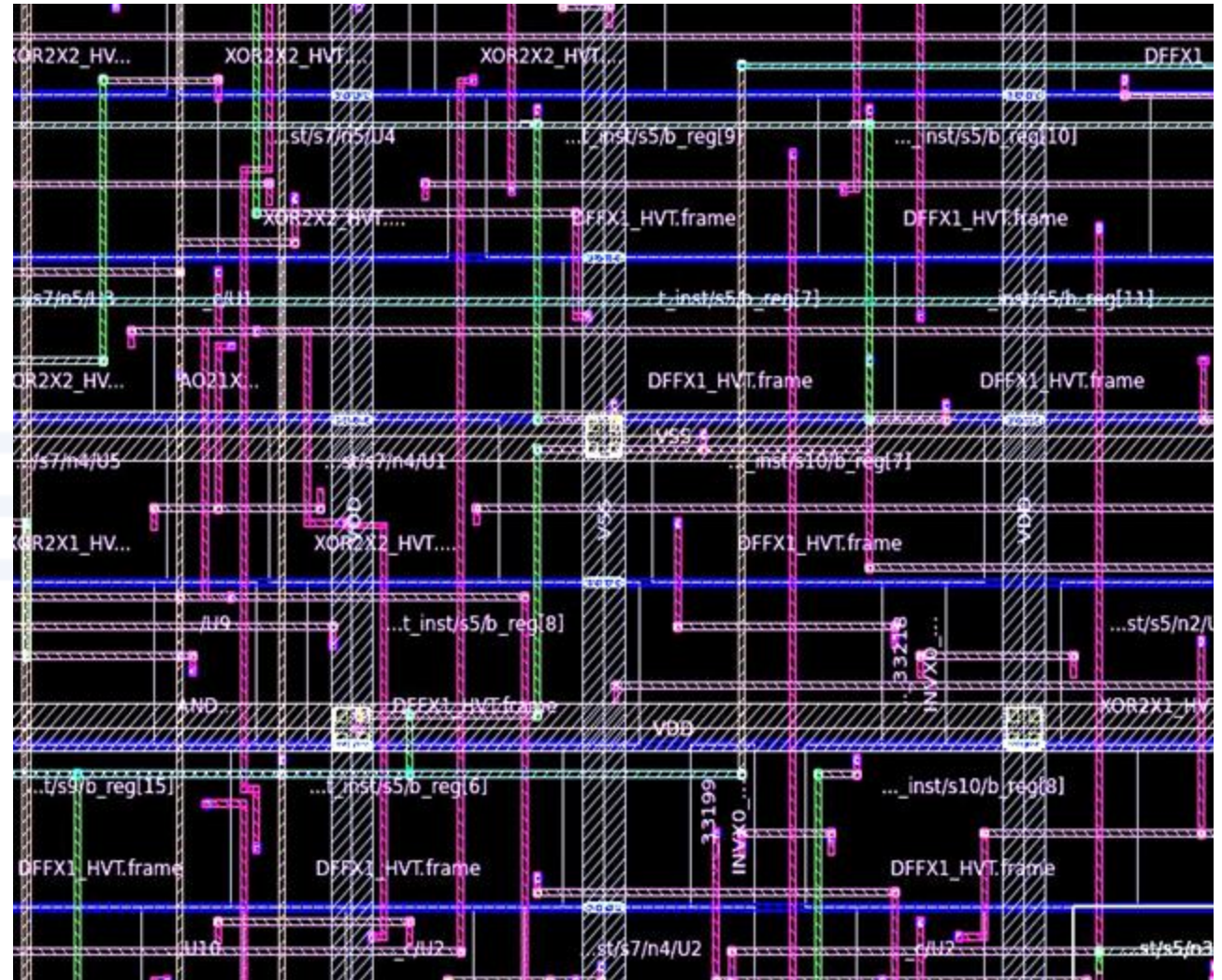
Routing & Optimization

This section explains about Routing and Optimization

Signoff



- Involves generating metal wires to connect the pins of same signal while obeying manufacturing design rules.
- There are 3 stages in routing
 - Global Routing
 - Track Assignment
 - Detail Routing
- Design is optimized for timing, power & to be DRC clean.



Full Chip Partitioning



Powerplan



Placement & Optimization



CTS & Optimization

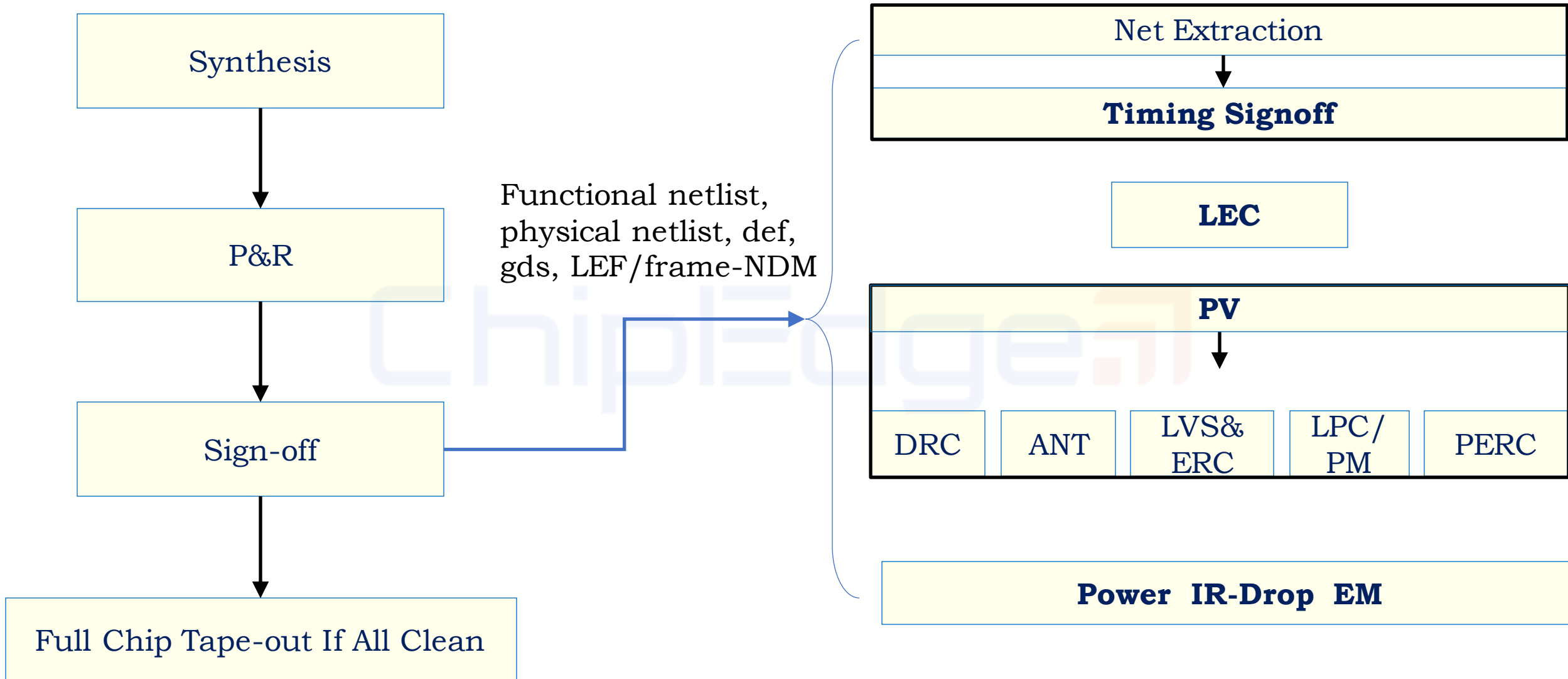


Routing & Optimization



Signoff

This section explains about Signoff



- Finally generates a gds file, having set of polygons/blueprint to manufacture.

RTL Netlist



Synthesis



Placement



PD with
Routing



Gds file -
polygons

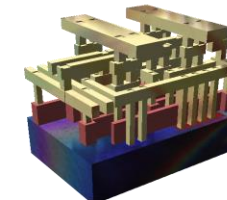
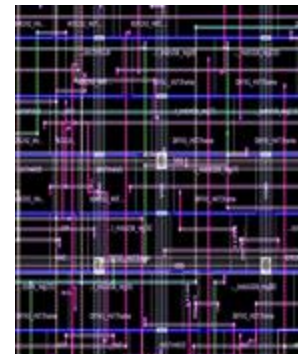


Fabrication
& Packaging



```
module  
half_adder(input a,  
b, output s, Cout);  
assign S=a^b;  
assign Cout=a&b;  
endmodule
```

```
module  
half_adder(input a,  
b, output s, Cout);  
XOR x1(S,a,b);  
AND a1(Cout,a,b);  
endmodule
```



- Physical Design Flow
- Full Chip Partitioning
- Powerplan
- Placement & Optimization
- CTS & Optimization
- Routing & Optimization
- Signoff

Module 6

Floorplanning

- In this module, you will learn about Block level floorplanning which includes boundary creation, I/O port placement, macro placement, and physical only cells placement.
- Brief overview of SoC level floorplanning which includes Hierarchical and Flat fullchip floorplanning approaches are explained.

- Introduction
- Deciding the Size of the block.
- Deciding the Shape of the Block
- I/O Ports Placement
- Macro placement Guidelines.
- Physical only cells
- Floorplanning checklist
- Types of SoC Floorplanning
 - ✓ Flat and Hierarchical Floorplanning
- Additional Information
 - ✓ Wire bond, Flip chip

Introduction

What is Floorplanning?, what are the objectives of floorplanning?

Deciding the Size of the Block



Deciding the Shape of the Block



I/O Ports Placements



Macro Placement



Physical only cells



Floorplan checklist



Types of SoC Floorplan



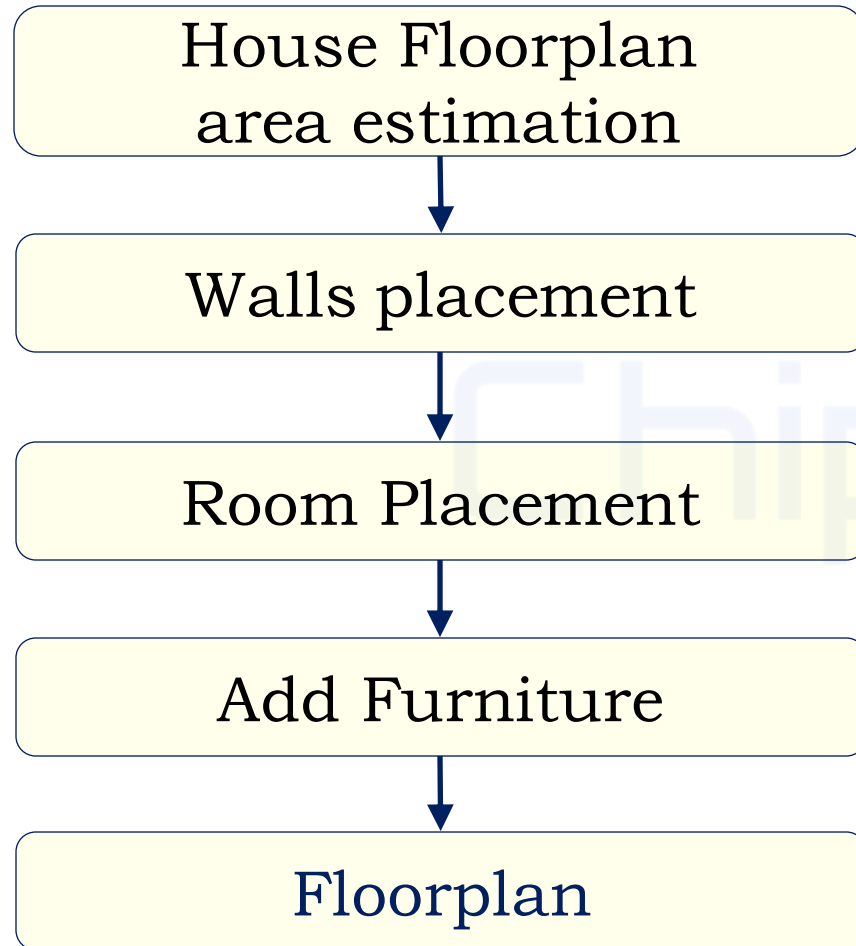
Flat & Hierarchical Floorplanning



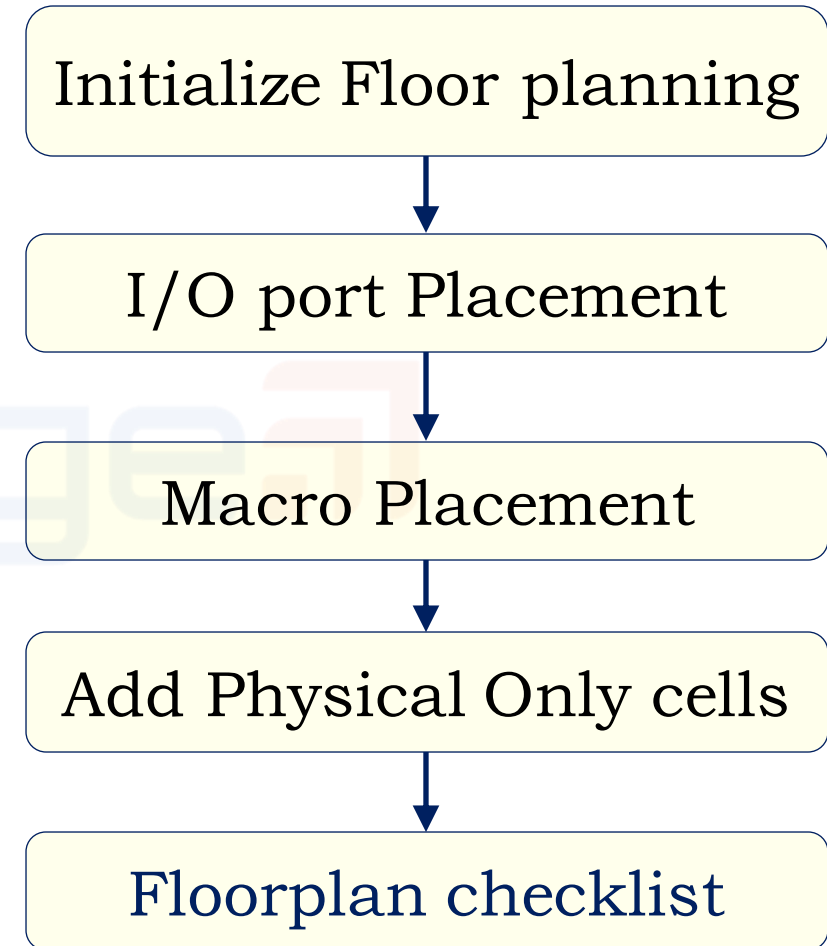
Additional Information



House Floor planning

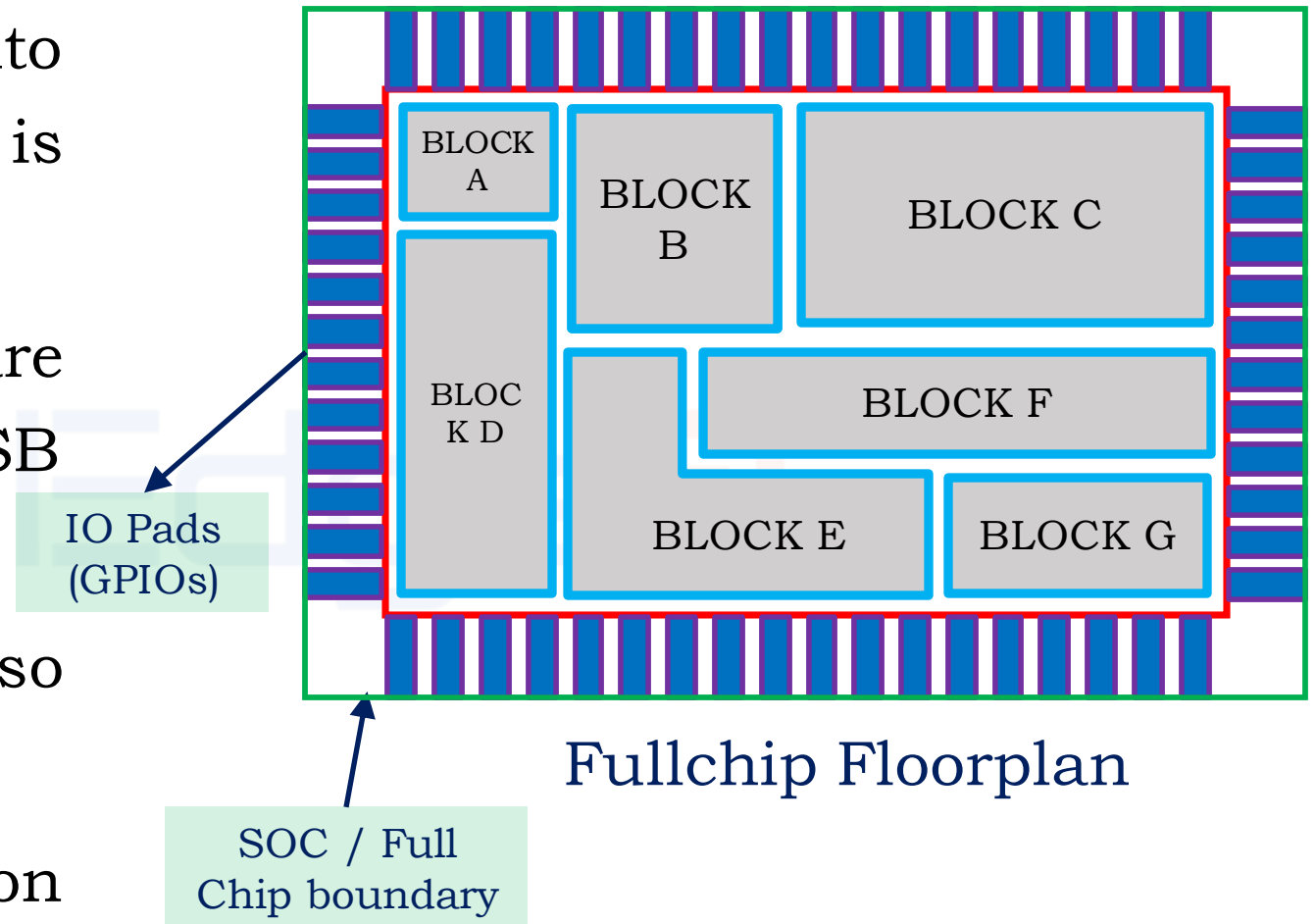


Block Floor planning

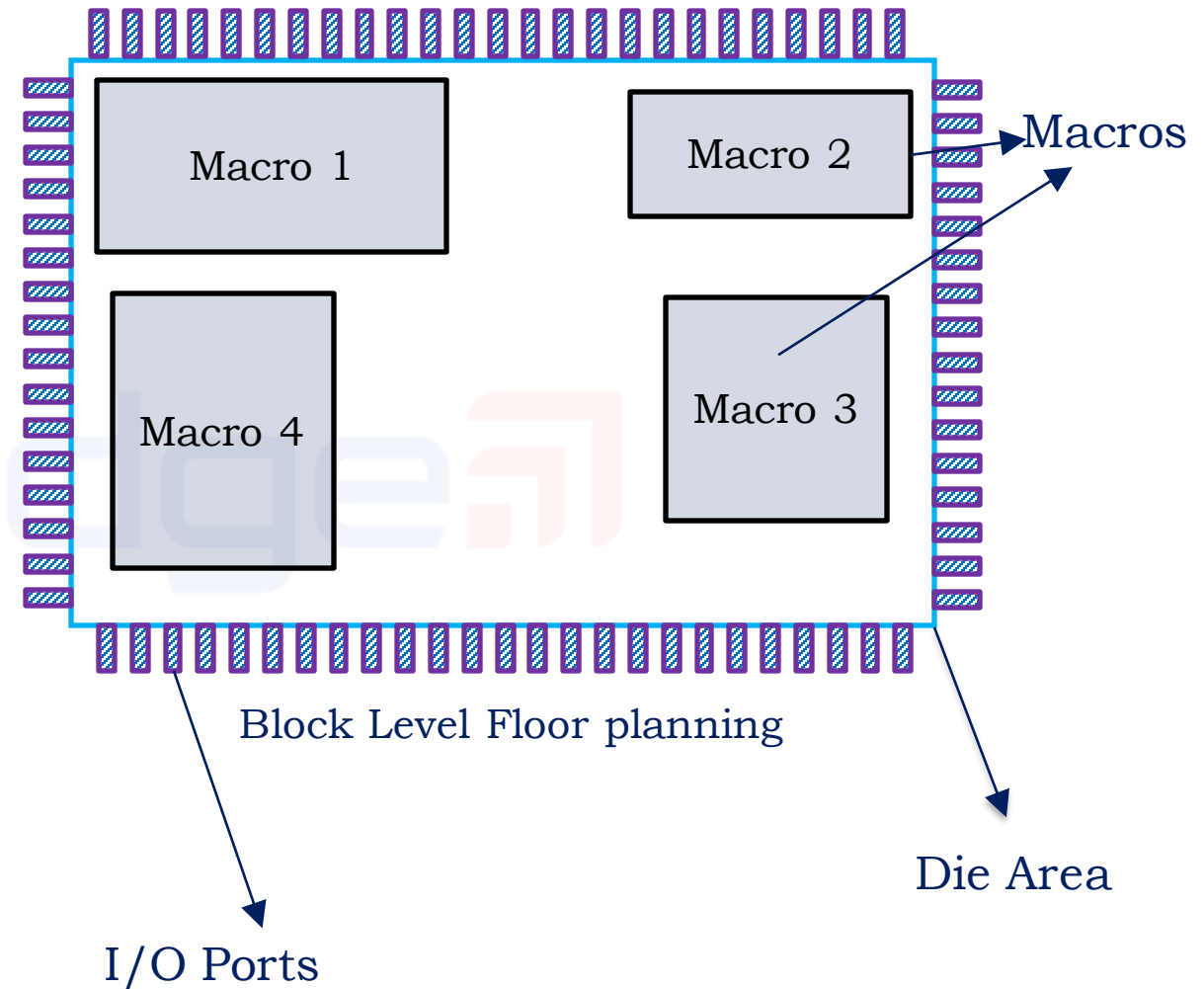


What is Floor planning ?

- Full chip is partitioned into multiple blocks, this process is called **partitioning**.
- The examples of Blocks are Power, UART, USB Interface, etc.
- Full chip Floor planning is also called **SoC Floor planning**.
- In this module we focus on **Block level Floorplanning**.

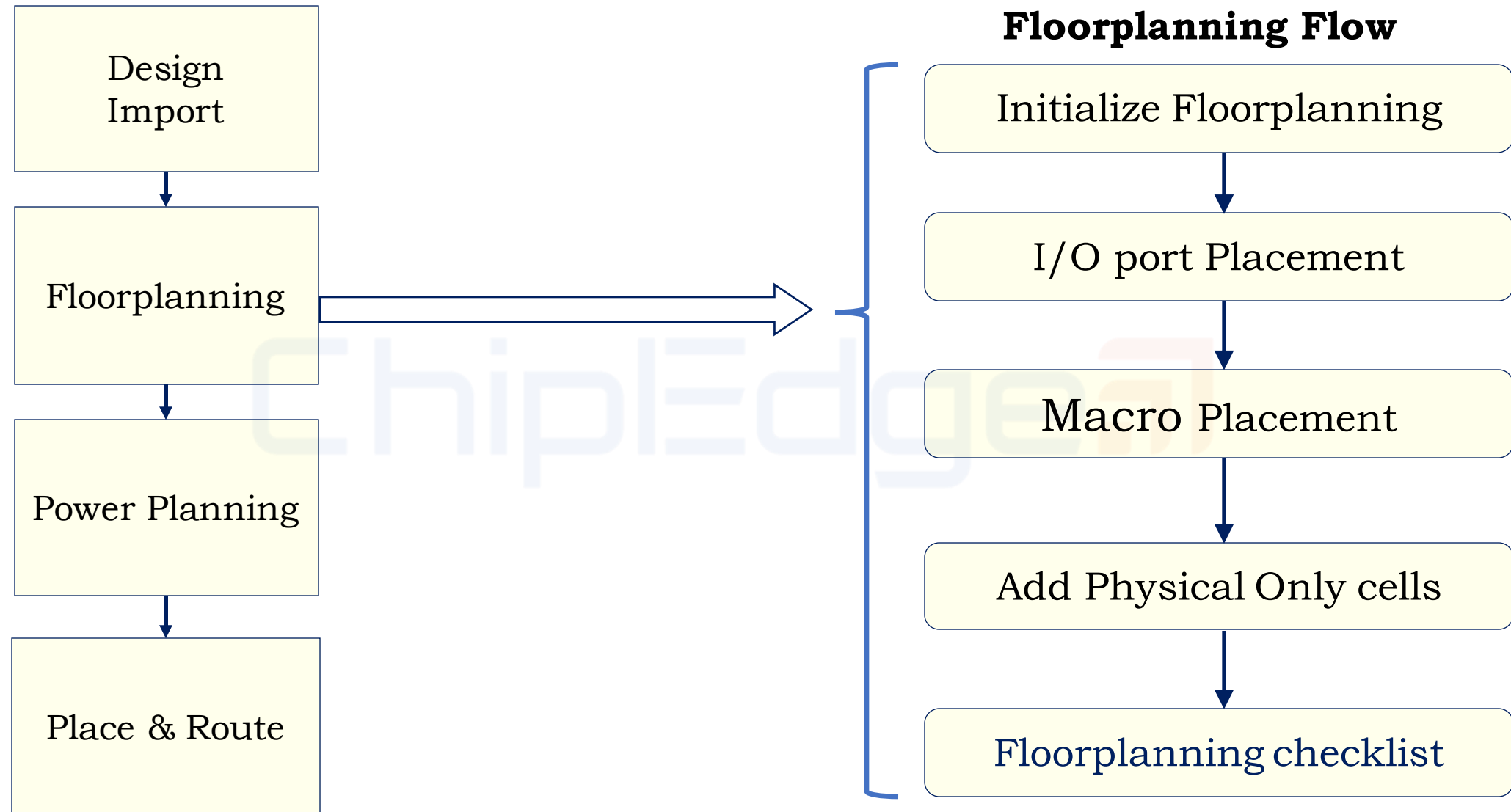


- It is the first step in the Physical Design Flow.
- This is Planning Your Floor, where floor being the **DIE**.
- Floorplanning is a process of determining the placement of Macros and I/O Ports.
- All the stages like Placement, Clock Tree Synthesis and Routing are dependent on Floorplan.



- Seed for optimal Place and Route
- Minimize Congestion
- Meet timing targets
- Optimize data-flow





Tasks

- Deciding the size of the block
- Deciding the shape of the block
- IO port placement
- Macro placement
- Physical only cells placement

Introduction



Deciding the Size of the Block

Deciding the Shape of the Block

I/O Ports Placements

Macro Placement

Physical only cells

Floorplan checklist

Types of SoC Floorplan

Flat & Hierarchical Floorplanning

Additional Information

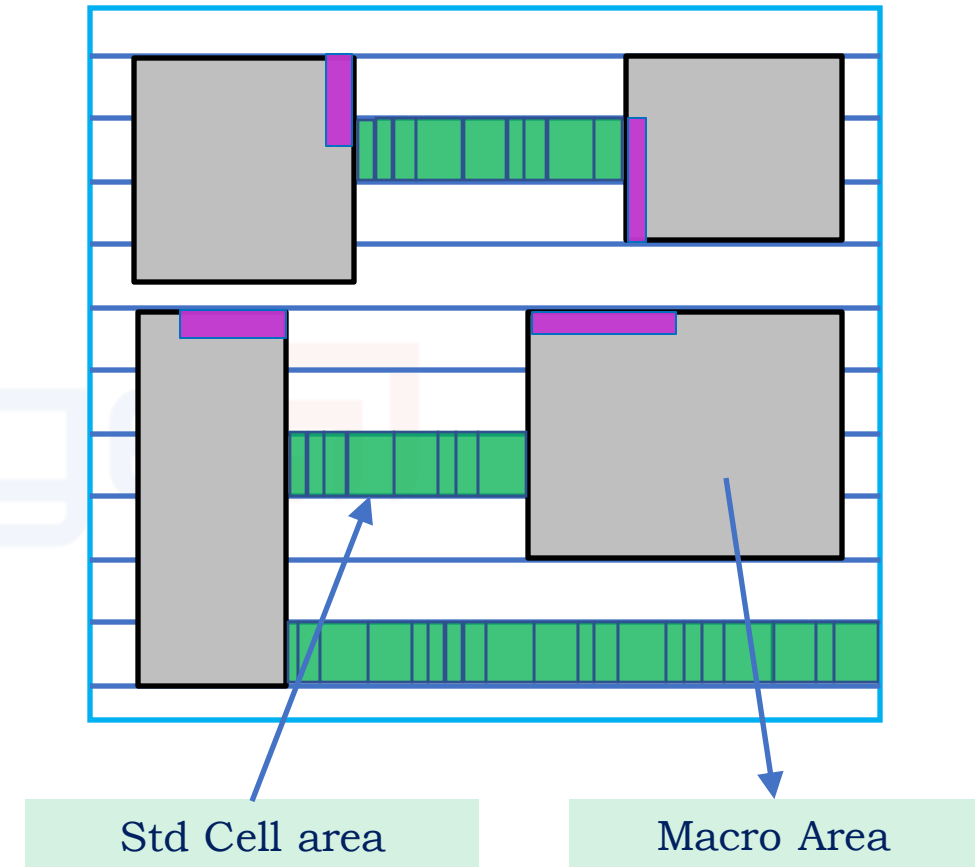


In this section die size estimation, blockage area and components in the Block area are discussed with examples.

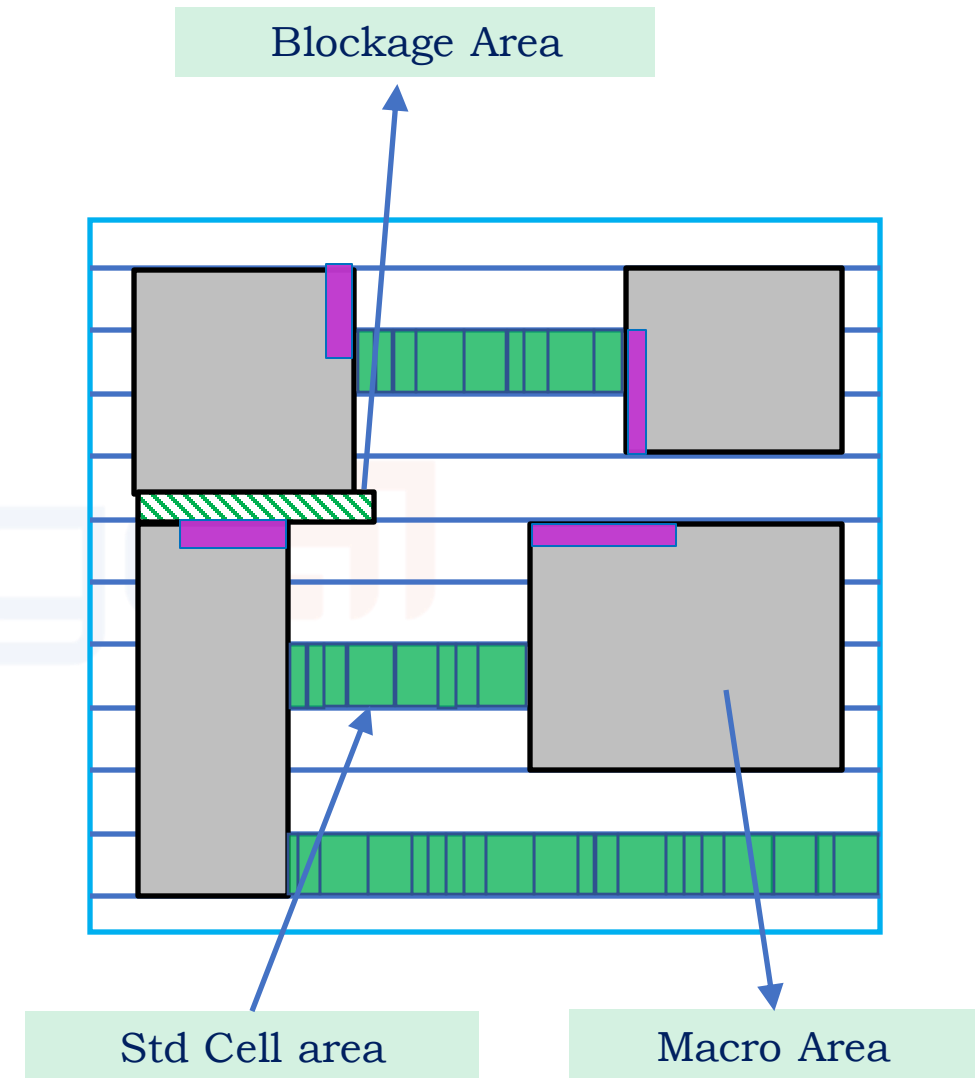
- Die Utilization = $\text{Netlist Area} / \text{Die-Area}$
- Netlist Area = Sum of Areas of all the cells in Netlist.(std cells + macros)
- Initial utilization is different from block to block and depends on complexity of the block etc.



- **Die Area:**
Die Area = Width x Height of the block.
- **Standard cell area :** All the standard row area, excluding macros and blockages is standard cell area.
- **Macro area:** Area occupied by macros

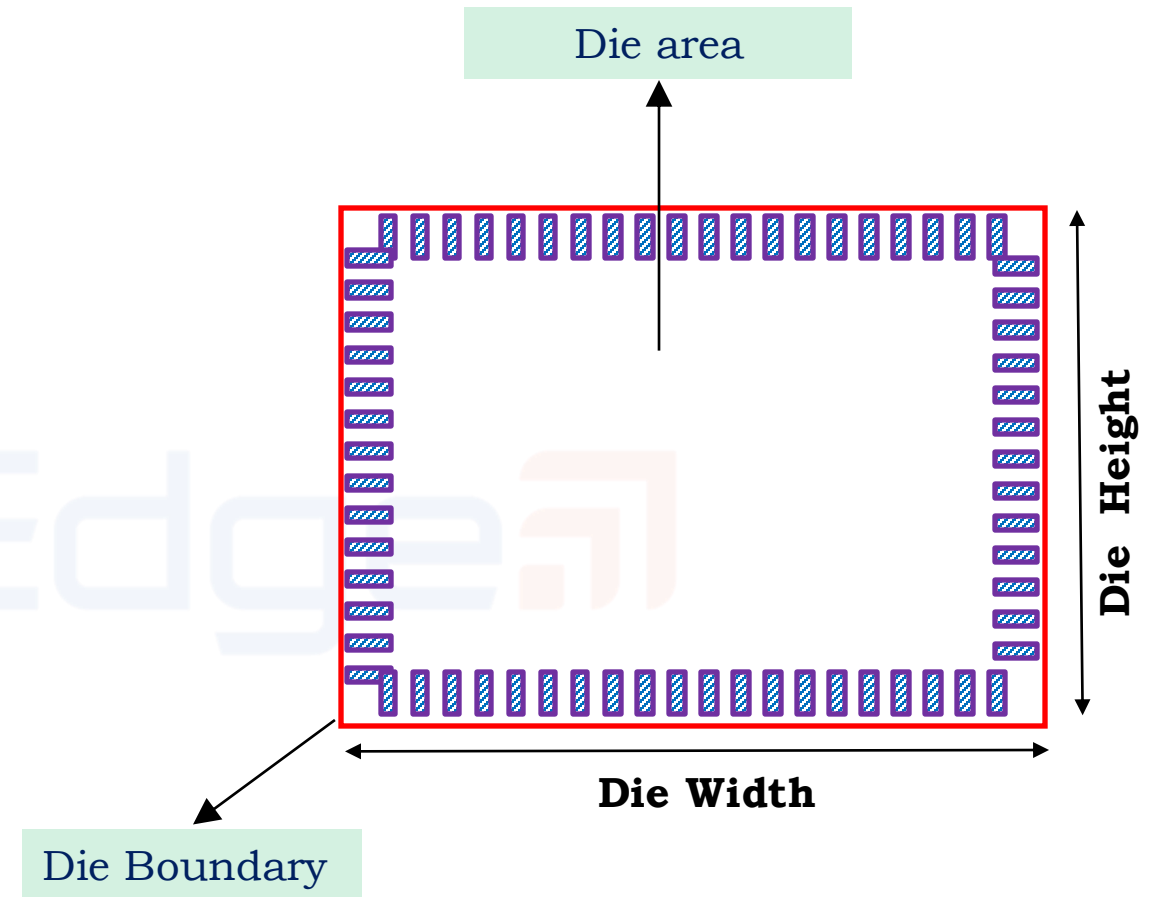


- Blockage area: sum of all the blockages in the design
- Blockages are used to reserve channels for buffer insertion.
- It prevents cells from being placed nearer to macros & avoid congestion near macro.



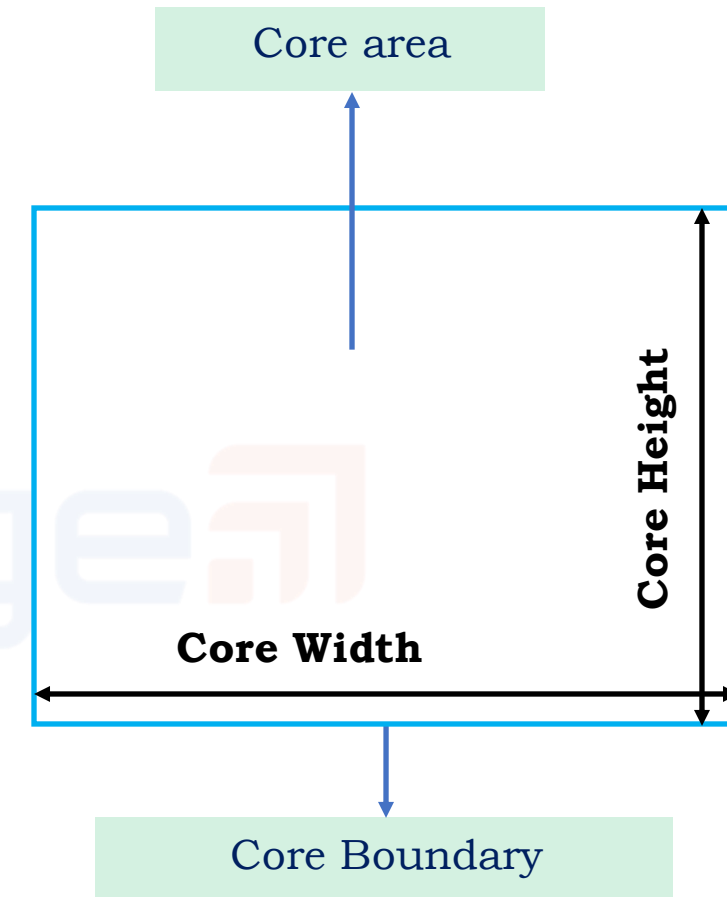
Die Boundary

- Die is the Boundary of the block which contains all the components of the block like cells, ports, macros etc.

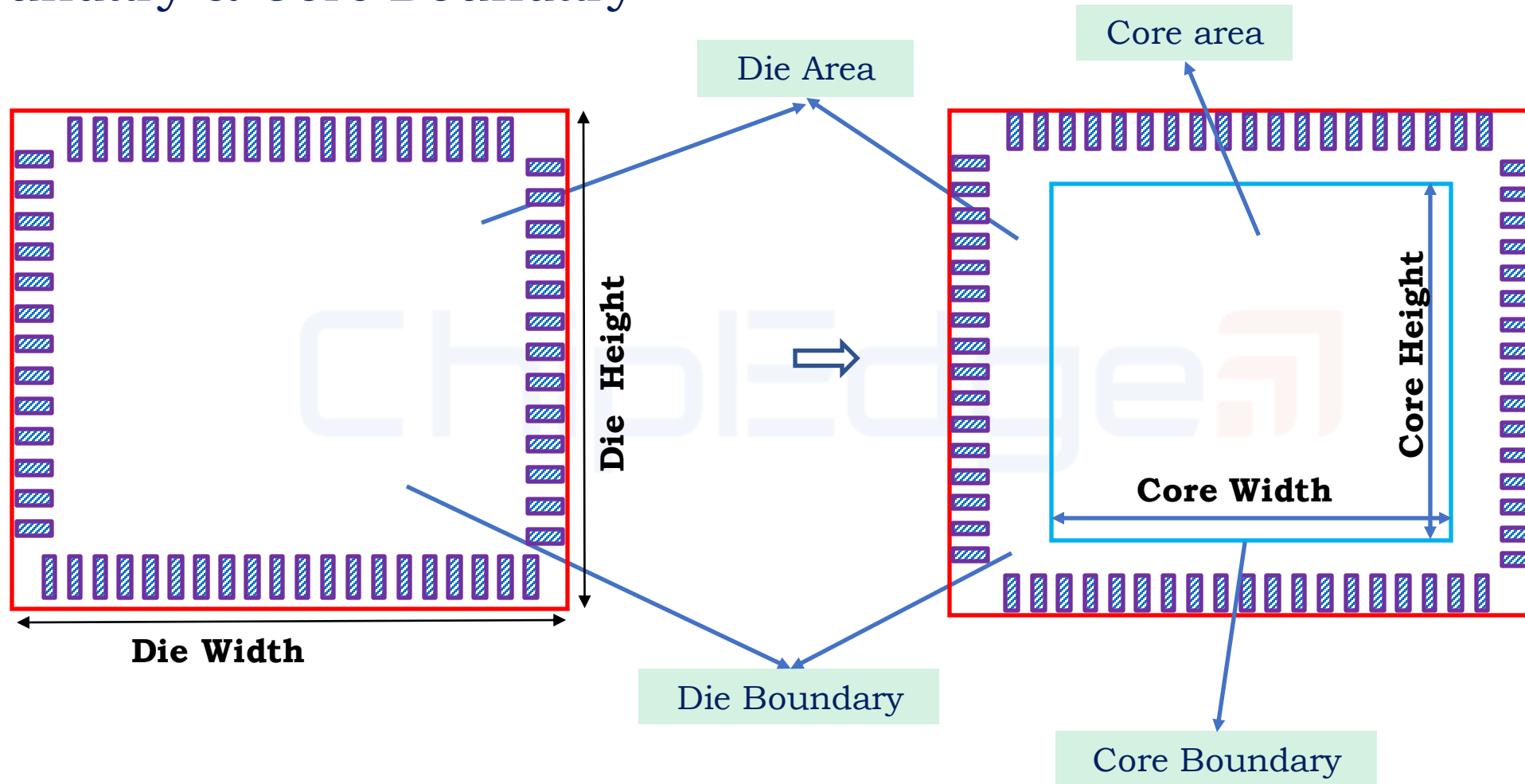


Core Boundary

- Core is defined as the region inside the die which contains macros, standard cells, and blockages.
- Core boundary refers to the boundary of core.



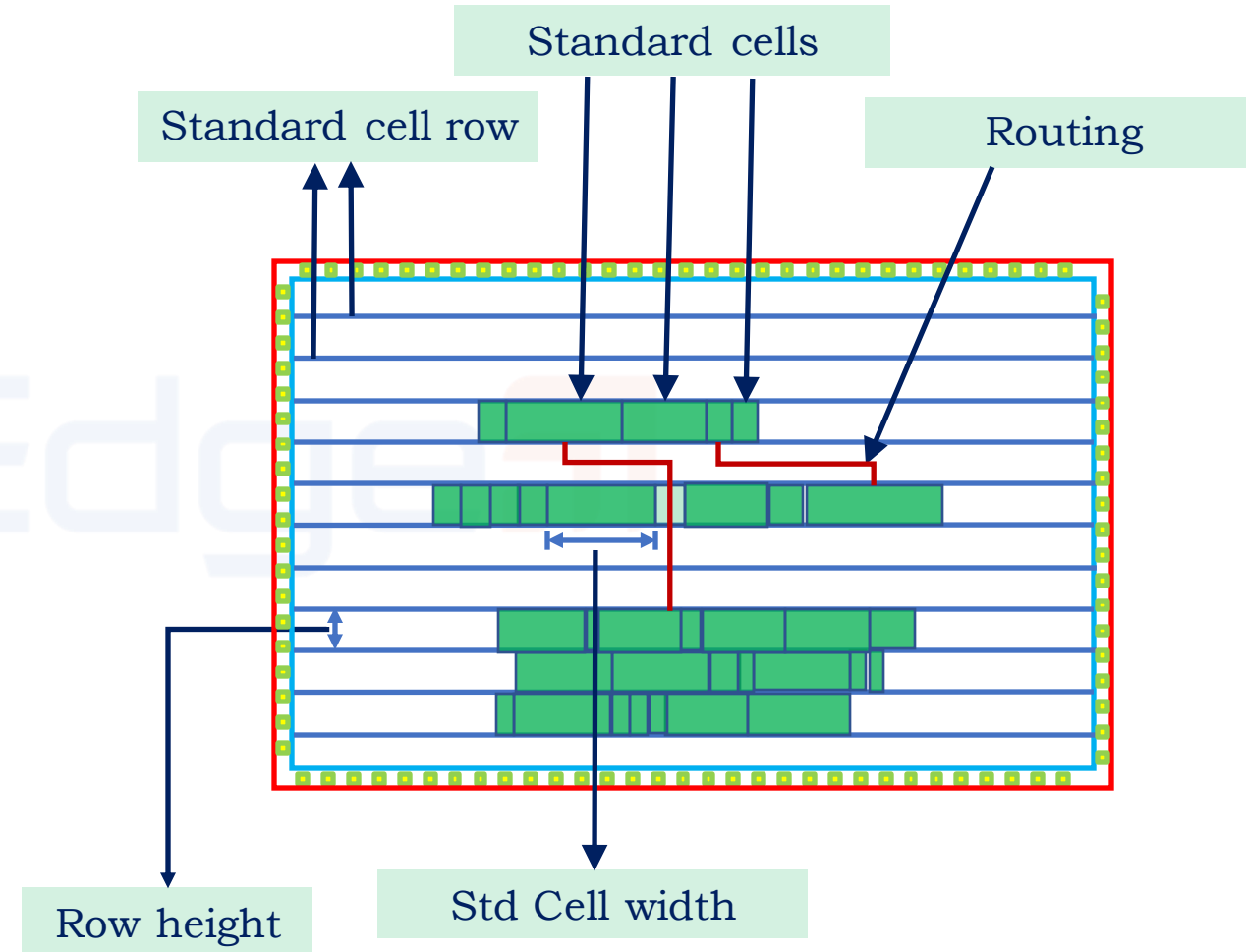
Die Boundary & Core Boundary



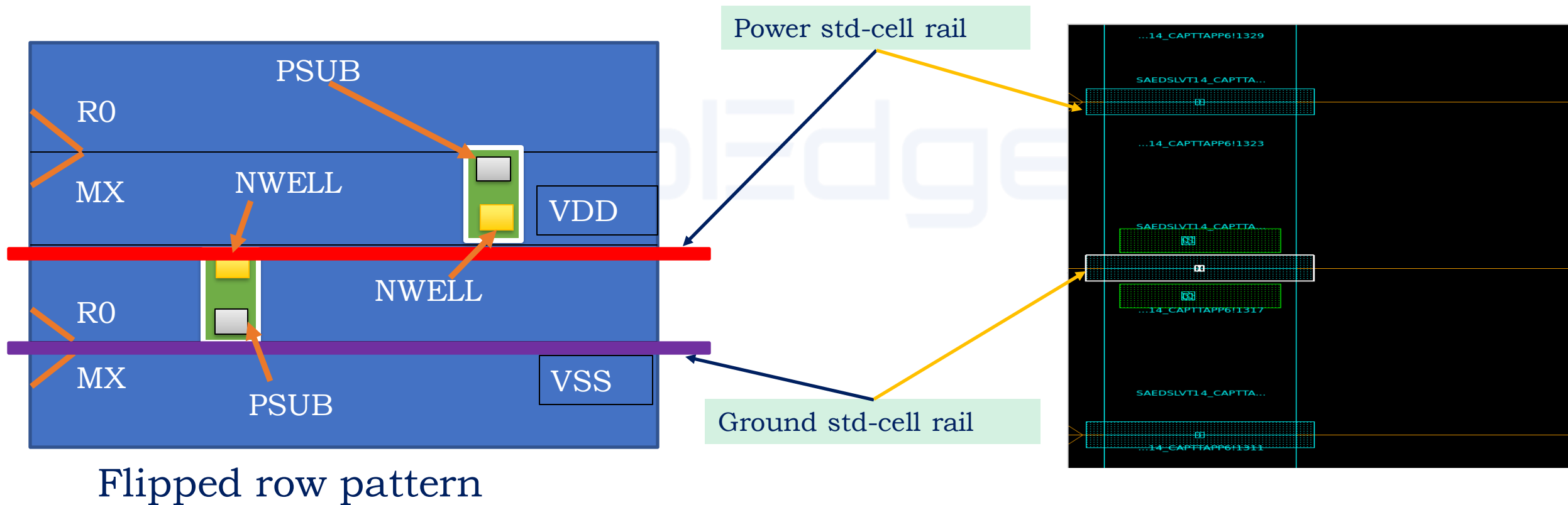
Standard cell Rows:

The standard cells in the design are placed in rows.

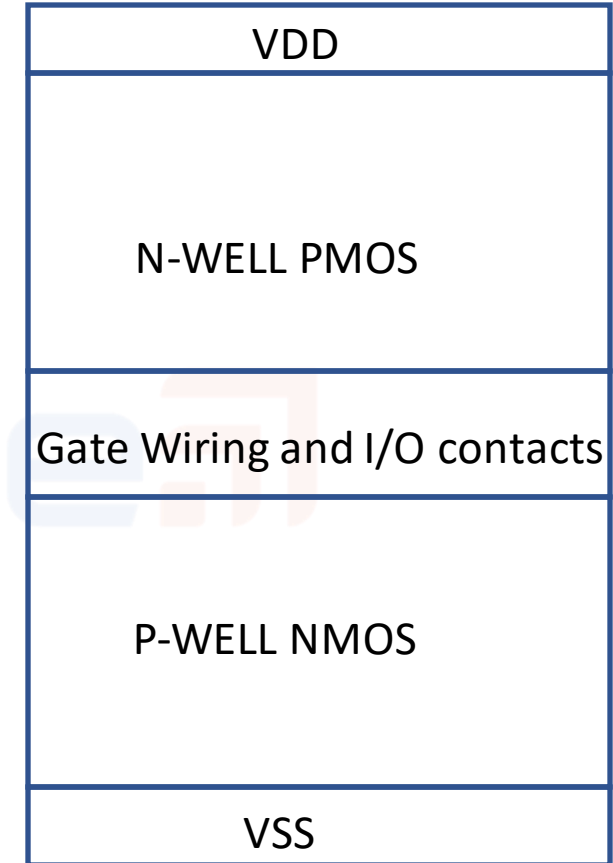
- The standard cells in the rows get ground and power connection from the VSS and VDD rails which are placed on either side of the cell rows.
- The technology allows the rows to be **flipped** because the cells can share the power and ground rails.



Flipped Standard Cell rows: Rows are flipped because std cells share the power and ground rails in vdd-vss-vdd pattern.

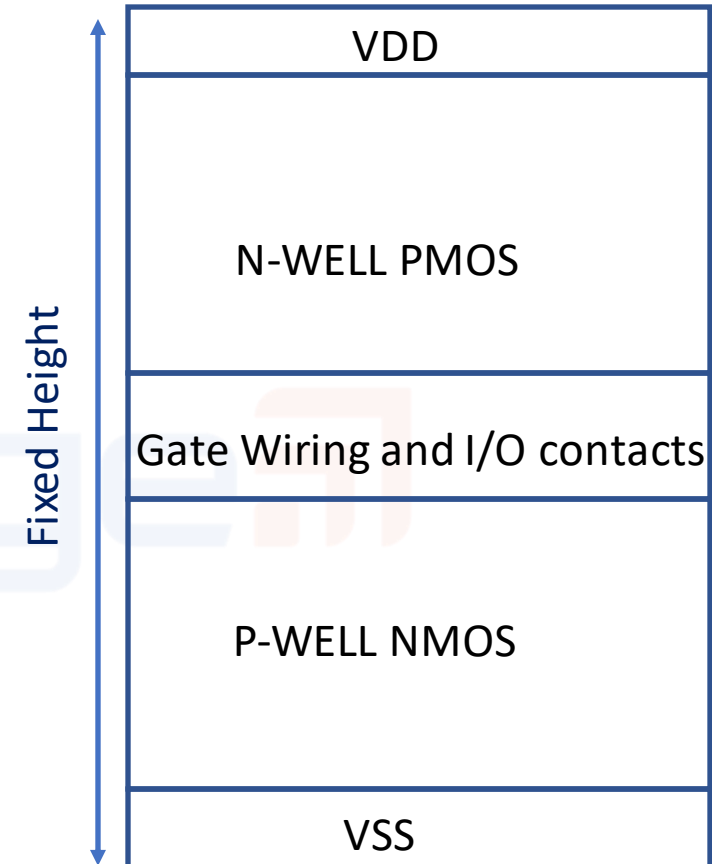


- These are well defined and pre-characterized cells used in ASIC Design flow as basic building blocks.
- Some examples are Inverters, AND gates, Buffers.



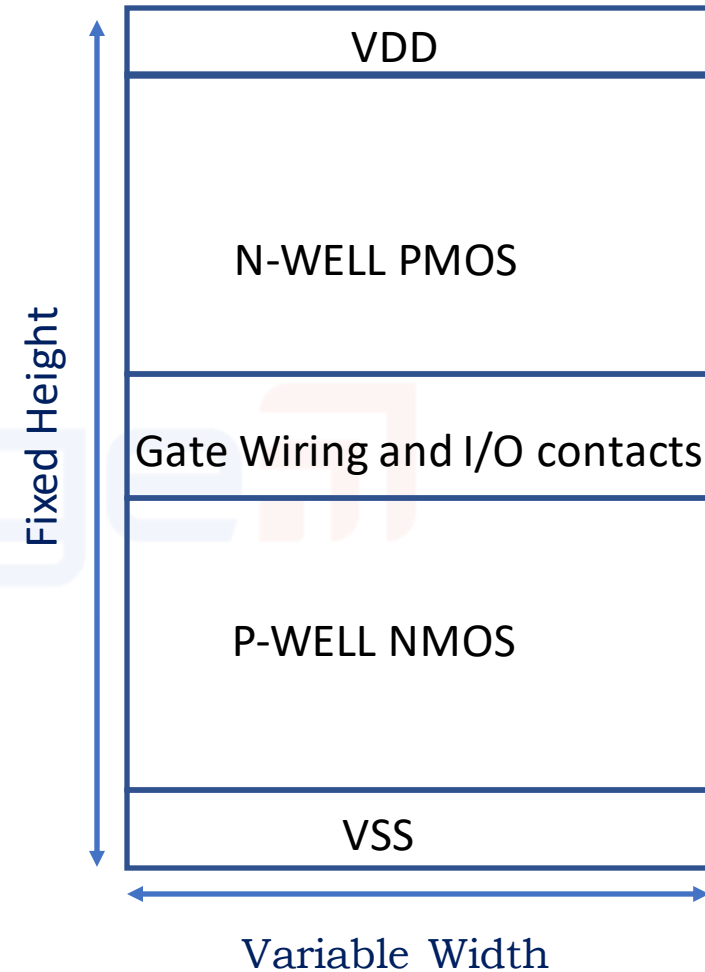
Height of Standard Cell:

- Height of std cells is always an integral multiple of row height.
- Most std cells have height equal to row height.
- For few complex Standard cells, the height can be more: 2x or 3x of row height.



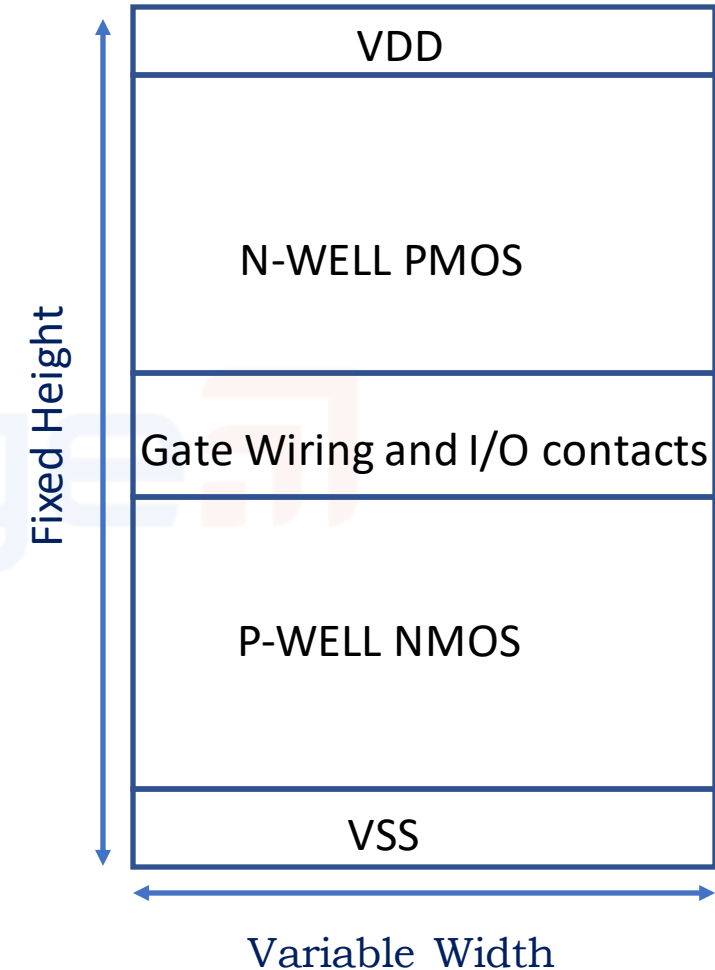
Width of Standard cell:

- Unlike height, width of the standard cells is variable.
- The width changes according to the number and dimensions of the CMOS transistors present inside the std cells.

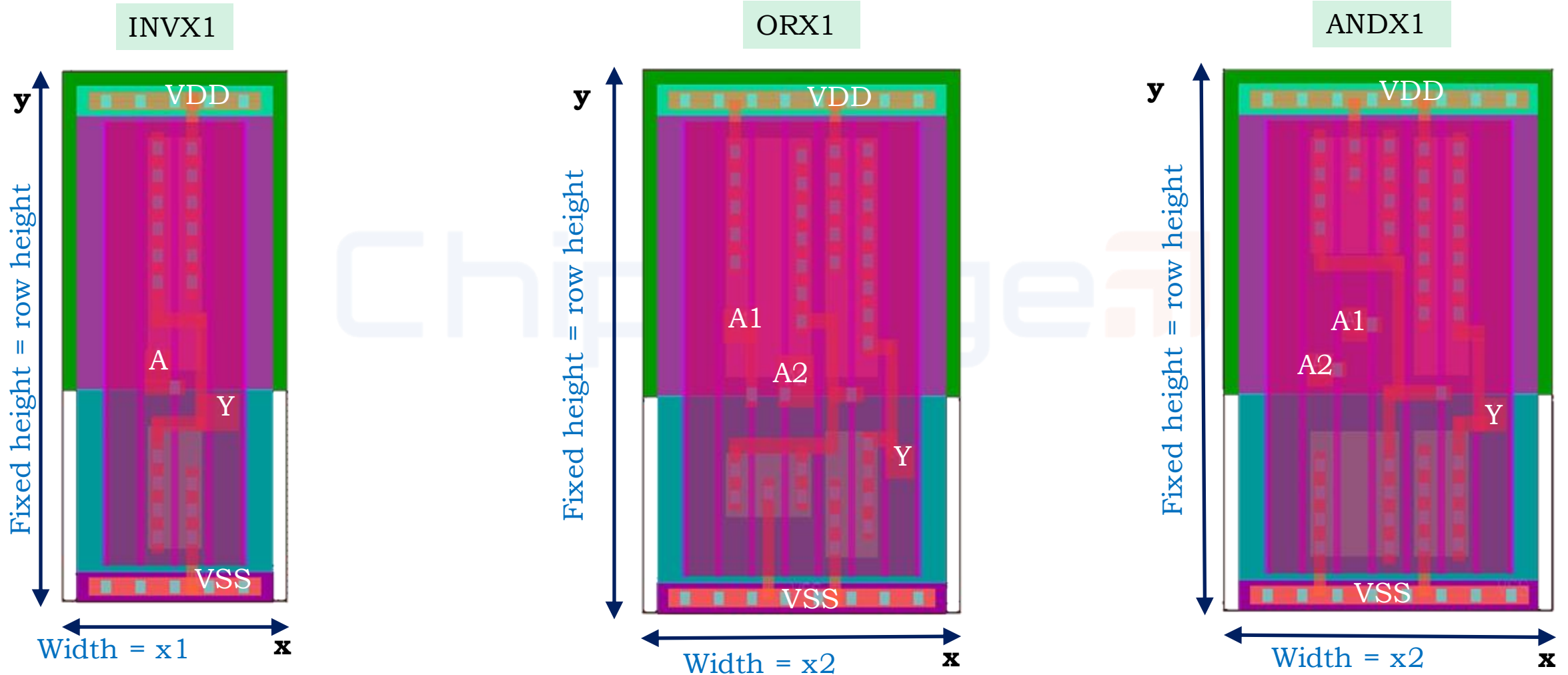


Width of Standard cell:

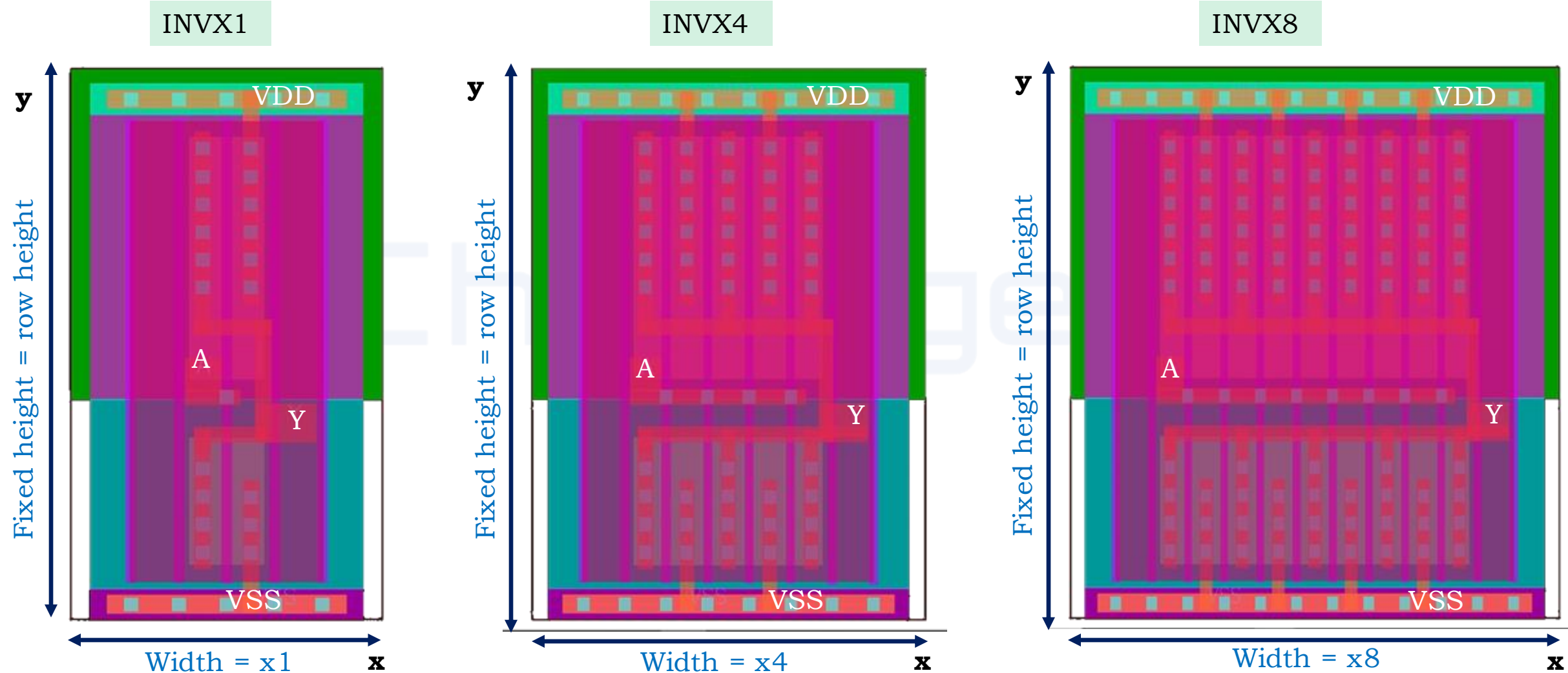
- This in turn depends on the Function of the std cell (Inverter or AND or Flop) and the drive strength of the std cell.
- E.g. one std cell library will have multiple Inverters which can drive different loads. (this will be discussed in more detail later)



Example of different Width of std cells with different functionality



Example of different widths of std cell Inverter with varying drive strengths (X1, X4, X8)



Introduction



Deciding the Size of the Block



Deciding the Shape of the Block

In this section, the importance of choosing the right shape for the block is discussed.

I/O Ports Placements



Macro Placement



Physical only cells



Floorplan checklist



Types of SoC Floorplan



Flat & Hierarchical Floorplanning



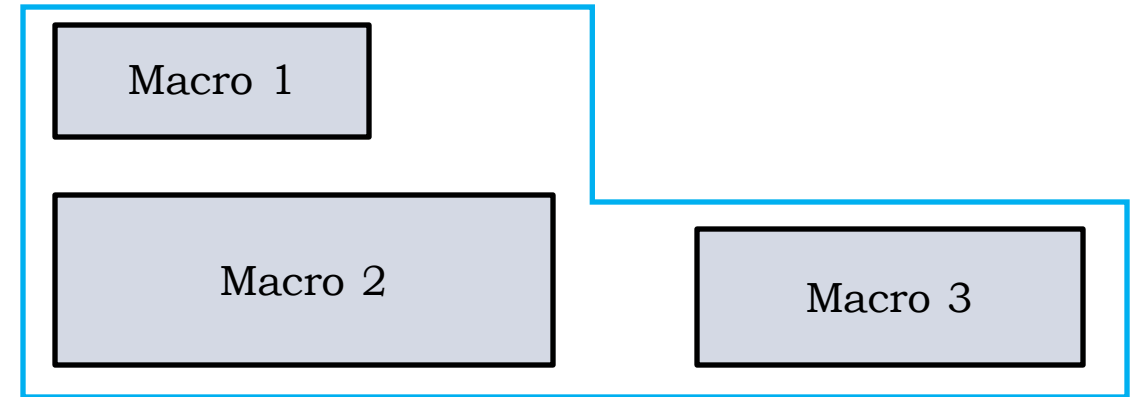
Additional Information



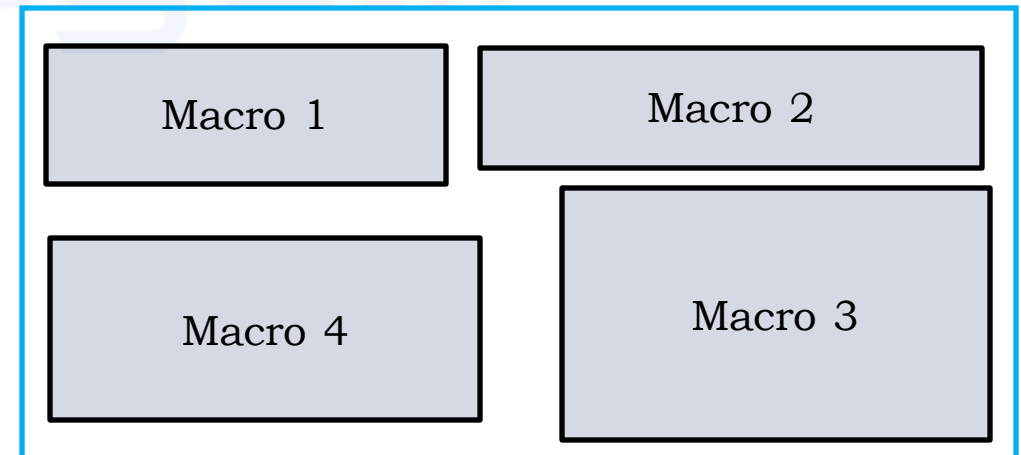
Rectilinear:

- If The aspect ratio of the blocks is not equal to 1, then these block shapes are called as Rectilinear.
- Example of rectilinear block shape are Rectangle, T shape, L shape etc.,

L-Block Shape



Rectangular Block Shape

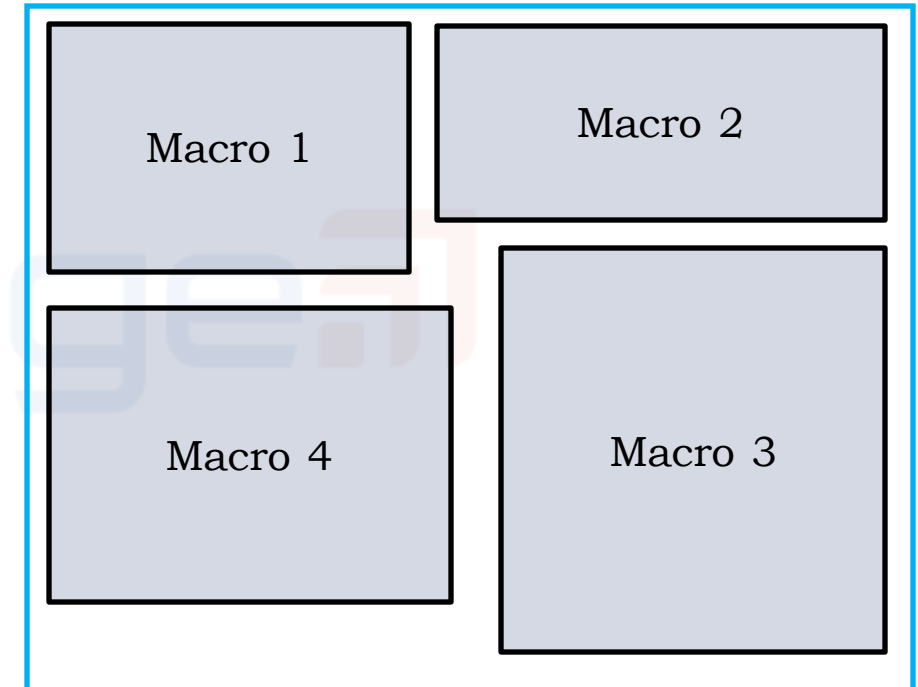


Square:

- If the aspect ratio of the block is 1, then this block shape is called as Square.

Note: We will focus majorly on rectangular/Square in our work.

Square Block Shape



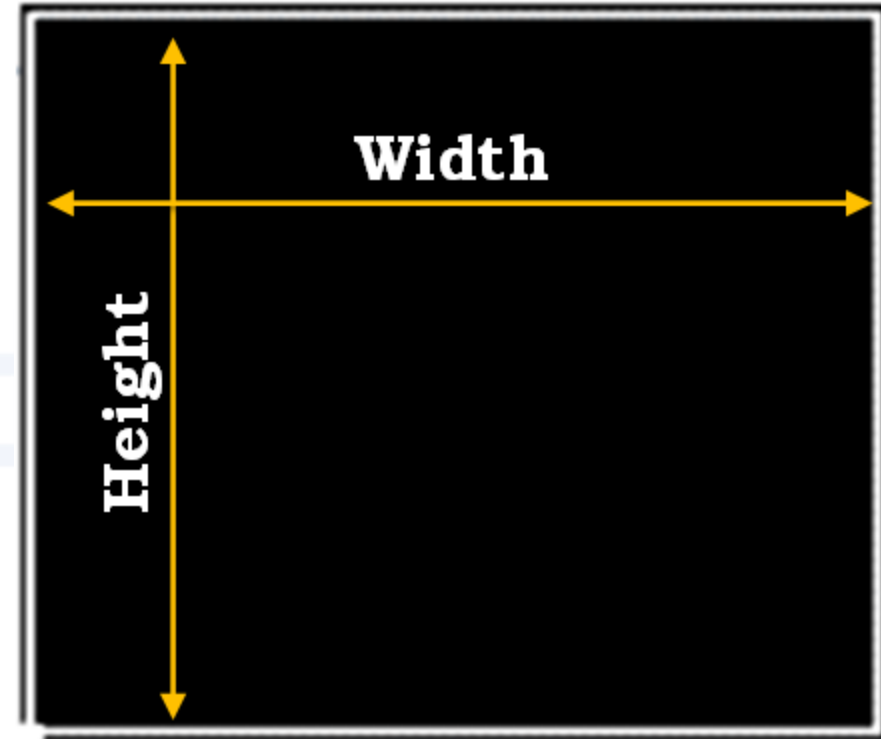
Aspect Ratio

In this section, the height and width of the floorplanning are explained. The importance of aspect ratio in the physical design flow is discussed.

- The ratio of the height and width is called as the aspect ratio.

$$\text{Aspect Ratio} = \frac{\text{Height of the Block}}{\text{Width of the Block}}$$

- Height of the block will determine the number of horizontal routing resources available.
- Width of the block will determine the number of vertical routing resources available.



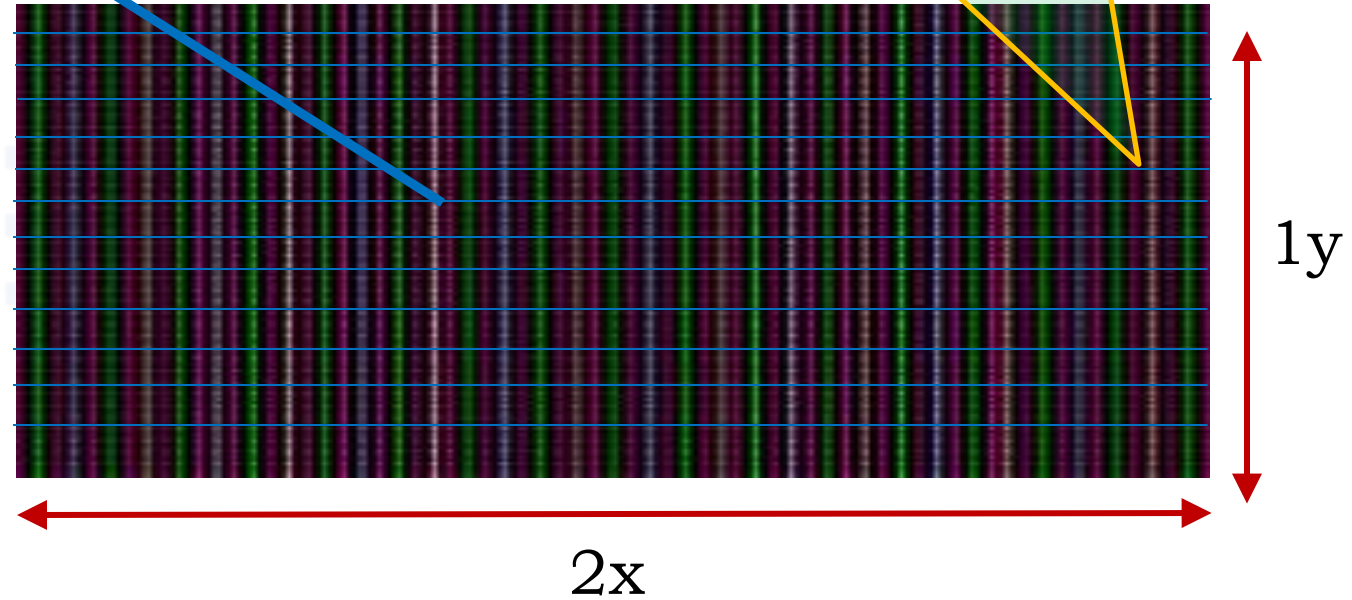
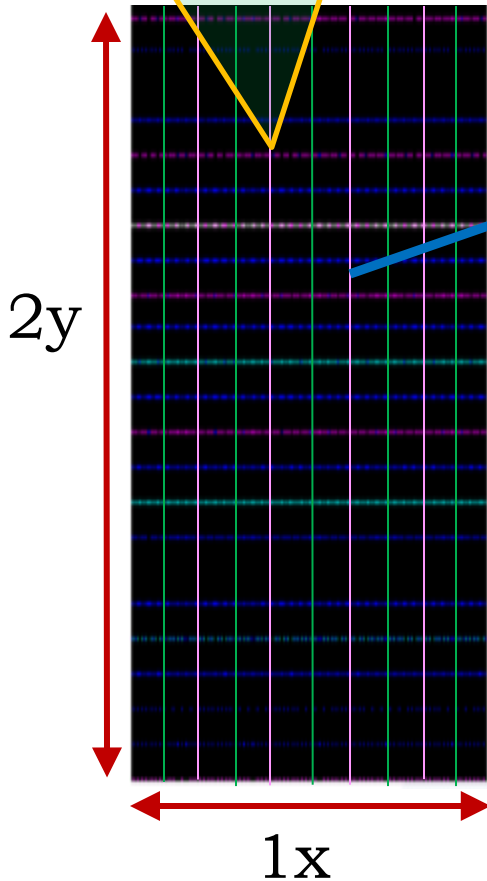
Aspect Ratio

Example 1: Aspect ratio is 2 .
In this scenario ,the block may get vertical routing congestion.
(Horizontal tracks are more)

Track : Track is defined as the path in which the metal routes can pass through

Example 2 : Aspect ratio is $1/2$. In this scenario ,the block may get Horizontal routing congestion (vertical resources will be more)

The ideal aspect ratio is 1, which gives the design similar number of horizontal and vertical routing resources, reducing chances of congestion.



Introduction



Deciding the Size of the Block



Deciding the Shape of the Block



I/O Ports Placements

Macro Placement

Physical only cells

Floorplan checklist

Types of SoC Floorplan

Flat & Hierarchical Floorplanning

Additional Information

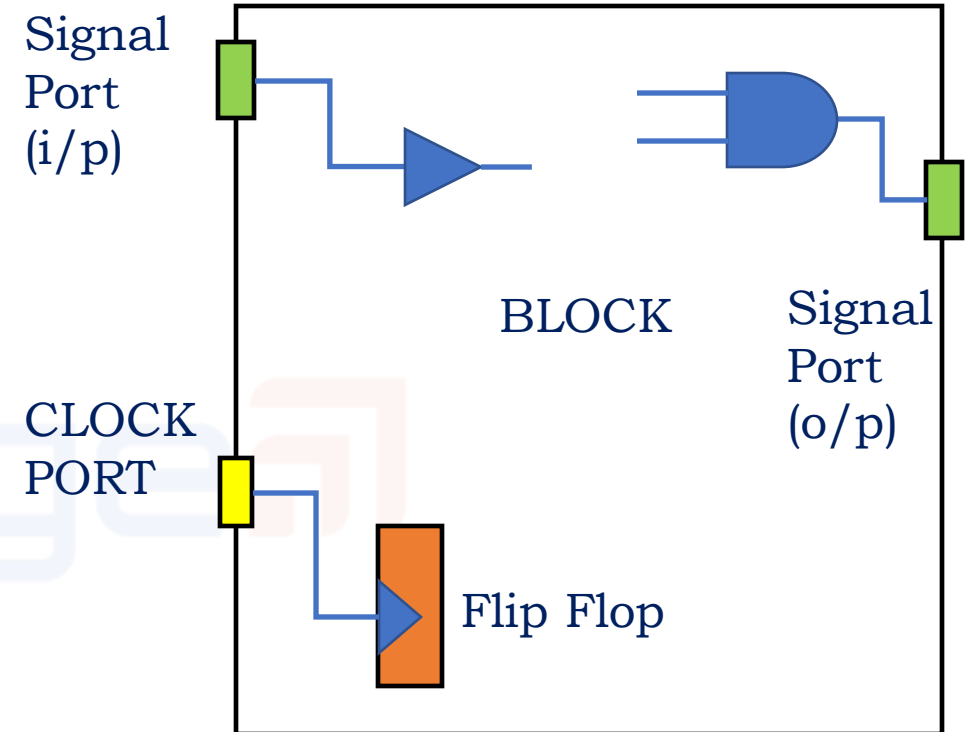


In this section, types of ports like signal ports, clock ports and their placement is illustrated with examples.

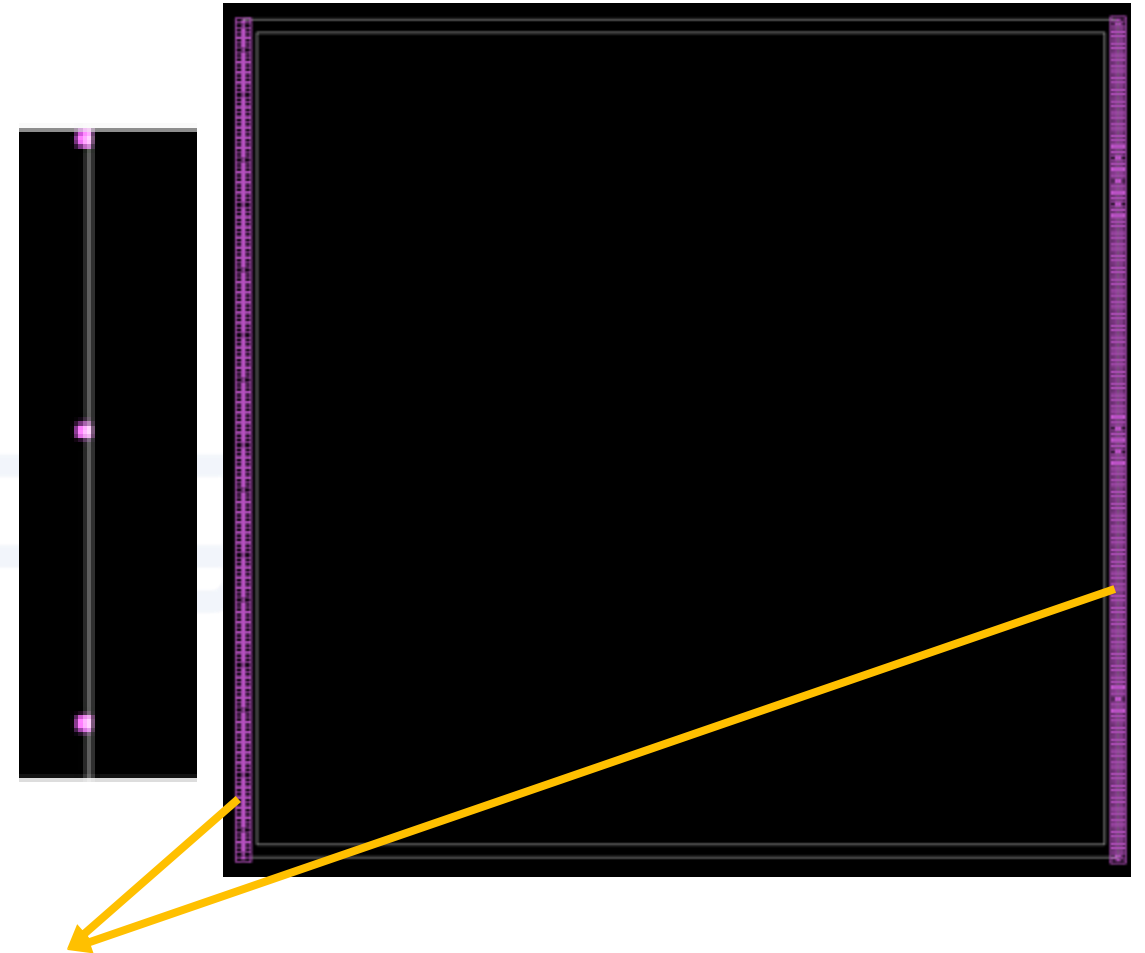
I/O Port: It is the interface point of the block through which the cells inside the block connects to the cells outside the block at SoC level.

- Types of IO ports
 - Clock ports
 - Signal ports
- At Chip level it is I/O Pads and at block level it is I/O ports.

- Clock ports: Clock ports are used to propagate the clock pulse and connect to the clock pins of sequential cells.
- Signal ports: Signal ports are used to transfer data signals from external top-level blocks to the internal macros and cells in the block.

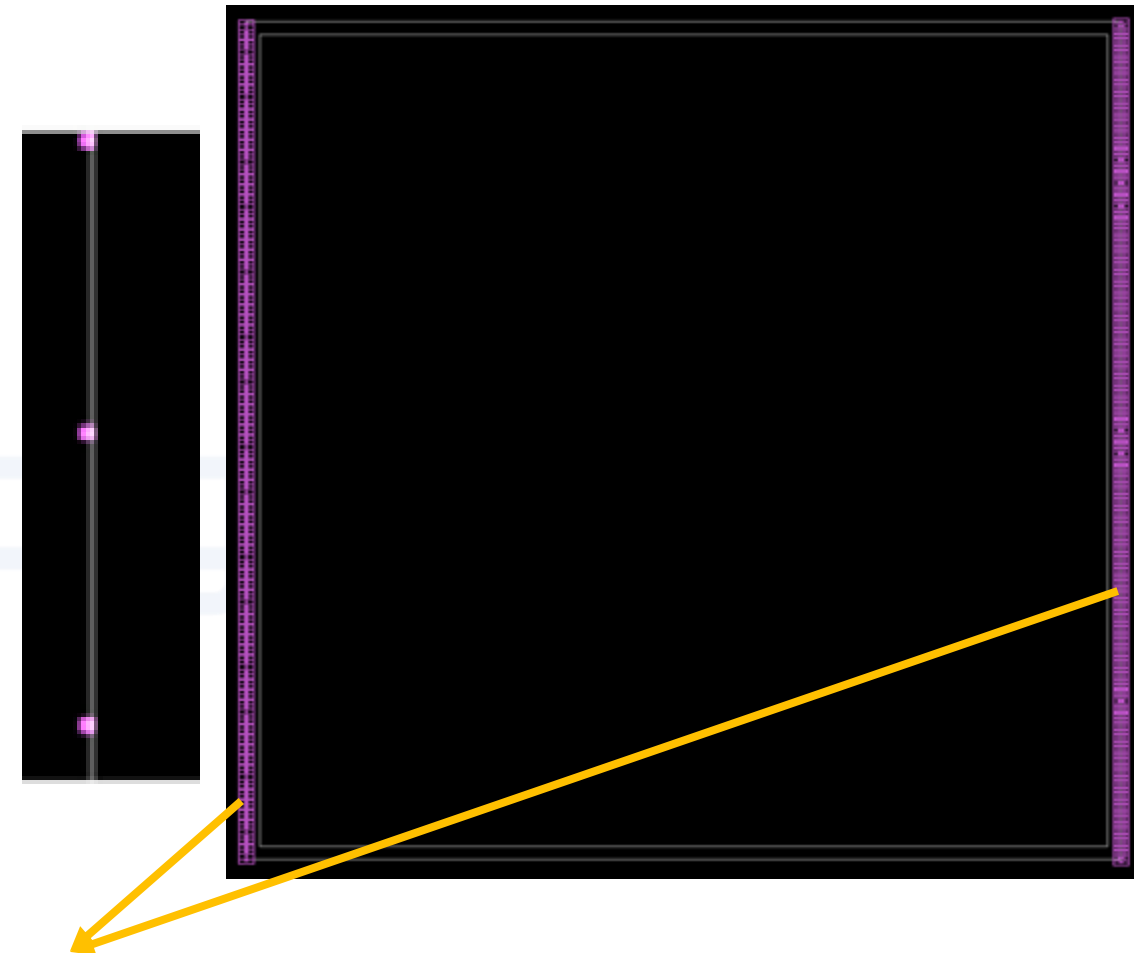


- Port placement refers to assigning Physical location for the I/O ports present in the design
- Placement of I/O ports plays a vital role in the design.



Input and output ports

- The Locations of ports is majorly dictated by the **interconnections** of **our block** with the other blocks in the SOC.
- Port Locations are generally shared by **top level team** in the form of below files
 1. DEF (Design Exchange Format)
 2. Tcl file
 3. Excel sheet locations



Input and output ports

Introduction



Deciding the Size of the Block



Deciding the Shape of the Block



I/O Ports Placements



Macro Placement

This section illustrates the guidelines for macro placement.

Physical only cells



Floorplan checklist



Types of SoC Floorplan



Flat & Hierarchical Floorplanning



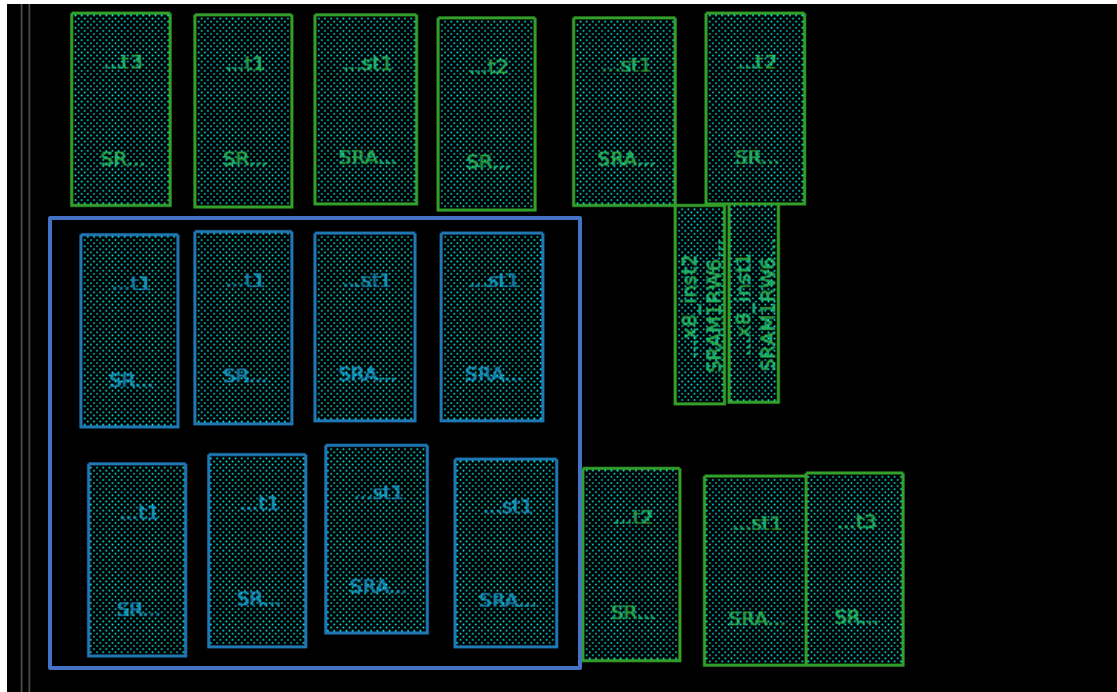
Additional Information



- Macro Examples : Memories (RAM, ROM), PLL, ADC, DAC, GPIOs, Thermal Sensor Diode, Voltage Monitor cells, etc.
- Macro Placement is done based on Connectivity information.
- Macro Placement should result in uniform standard cell area.
- Macro placement is very critical for congestion and timing.

- Macro Placement Requires
 - Fly-line Analysis
 - Design Module Hierarchy Analysis
 - Channel Length Calculation
 - Data-Flow Analysis

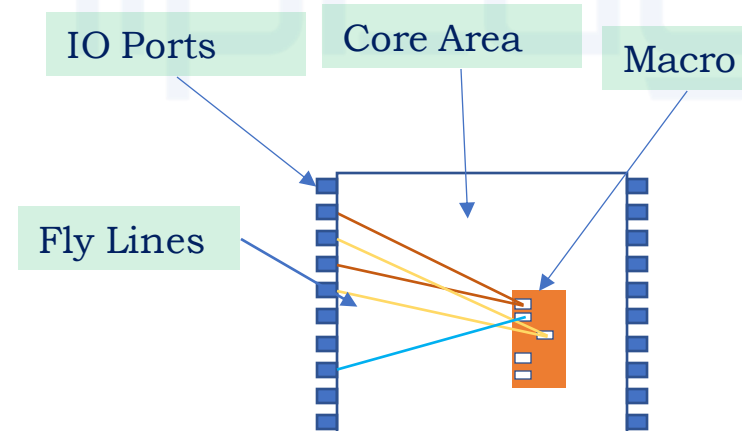
1. Check the logical grouping/hierarchy of macros and place together as per hierarchy



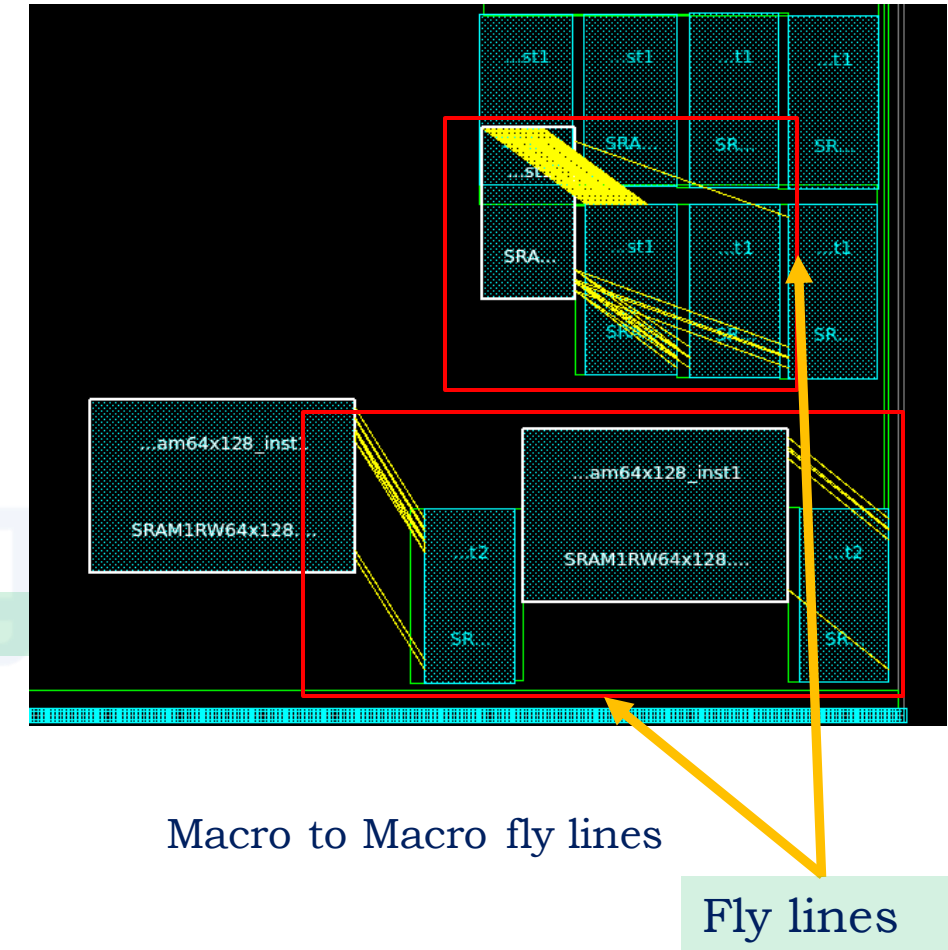
2. Check connectivity of macros through Fly-lines and place macros as per Fly-line analysis

- Priority of Fly-line connections

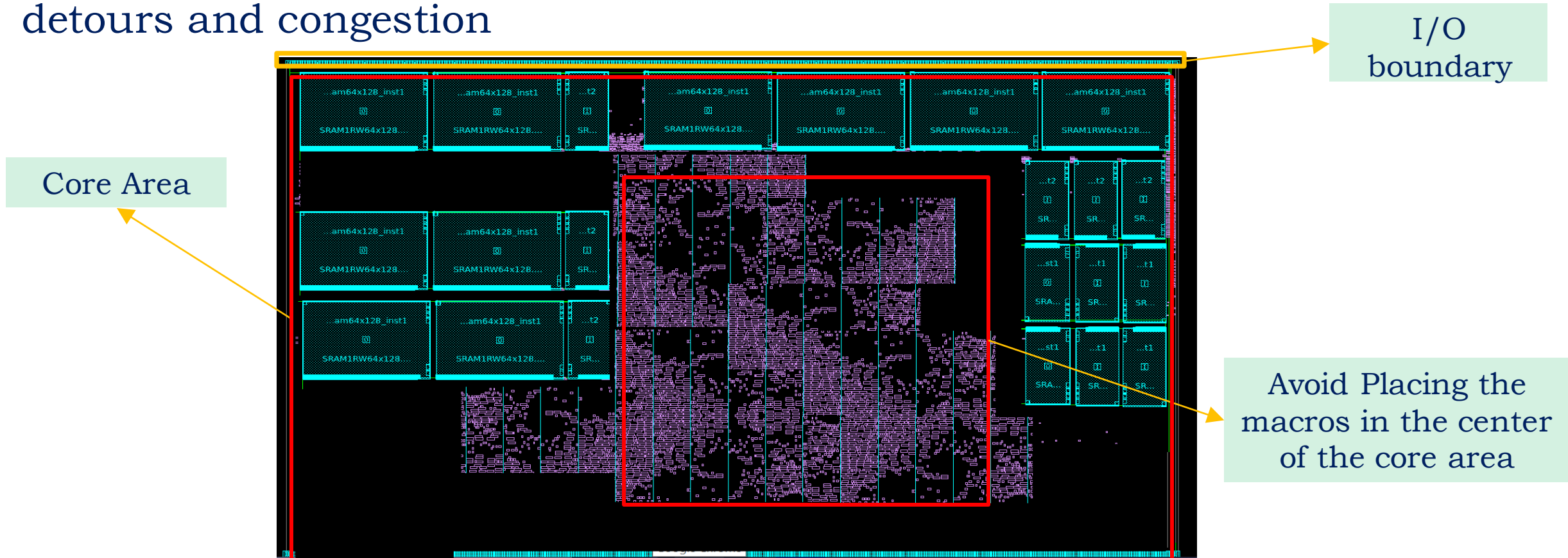
- Macro to IO ports
- Macro to Macro
- Macro to std-cells



Macro to IO port fly lines

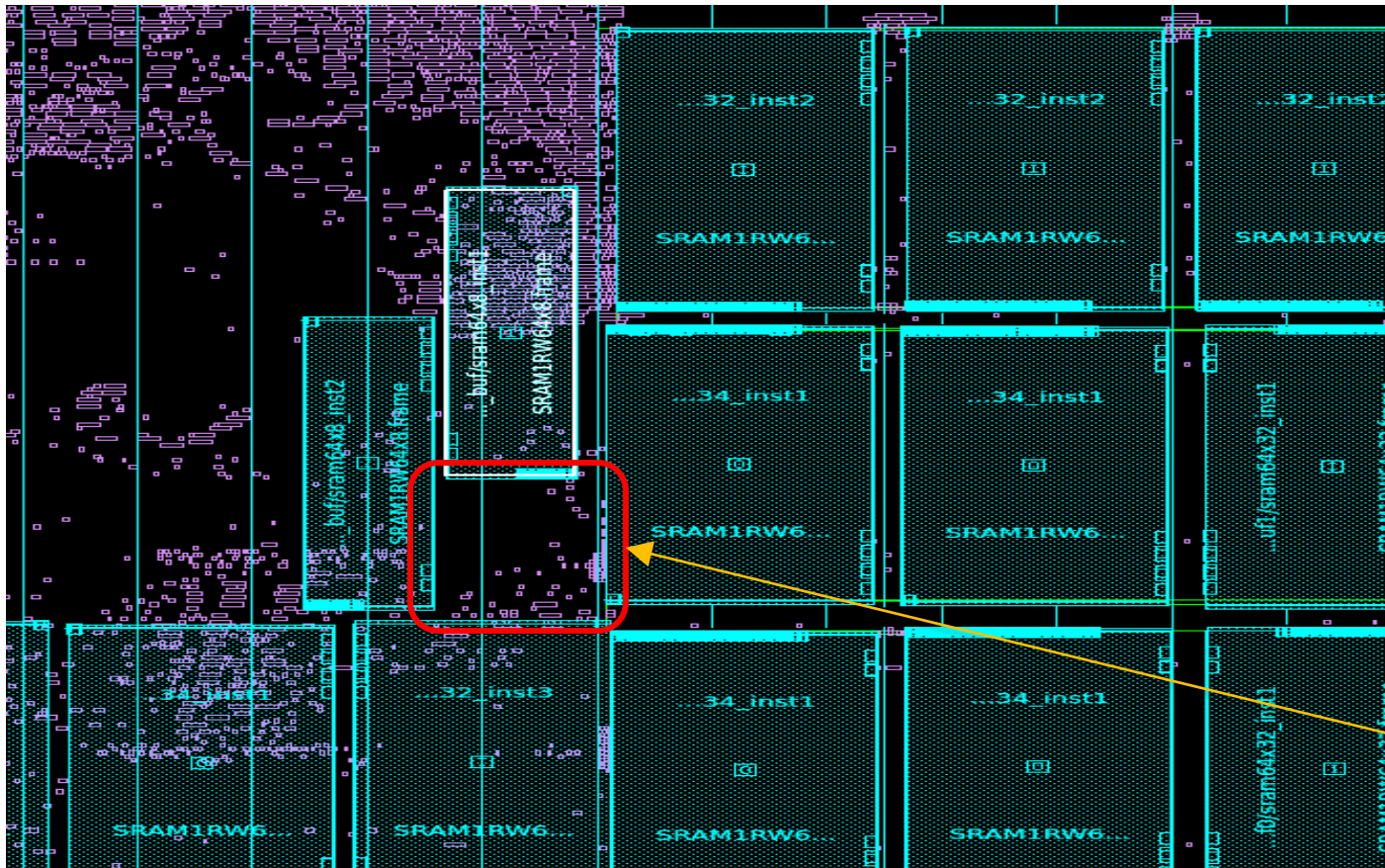


3. Avoid placing macros in the center of the core area. Else, it may lead to detours and congestion

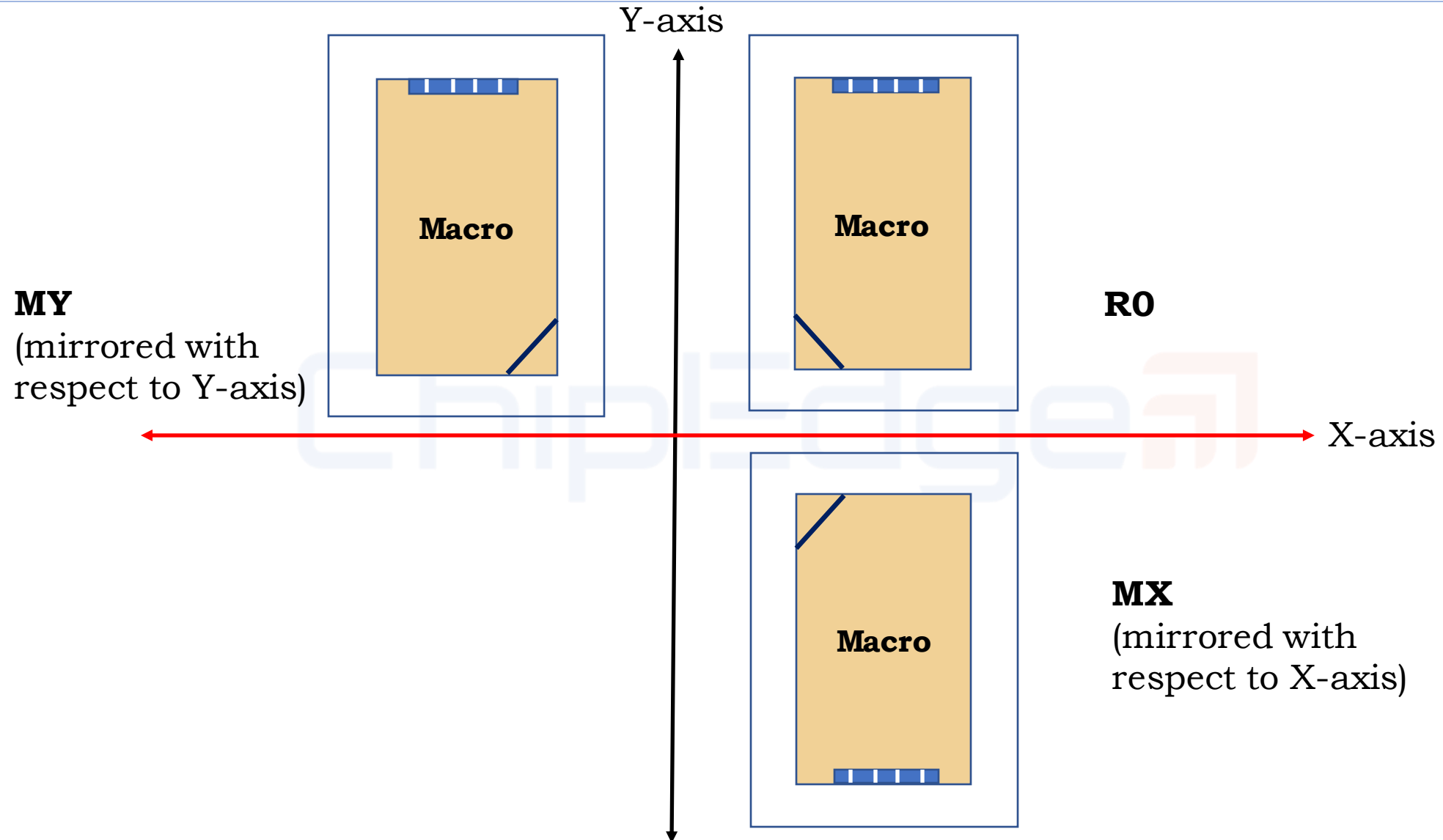


4. Avoid notches while placing macros

- Notches are a problem because they block routing of all std cells in notch area and also routing of nearby macro pins in notch area.



Example of
Notch area



5. Check flyline connectivity and avoid criss-cross connections

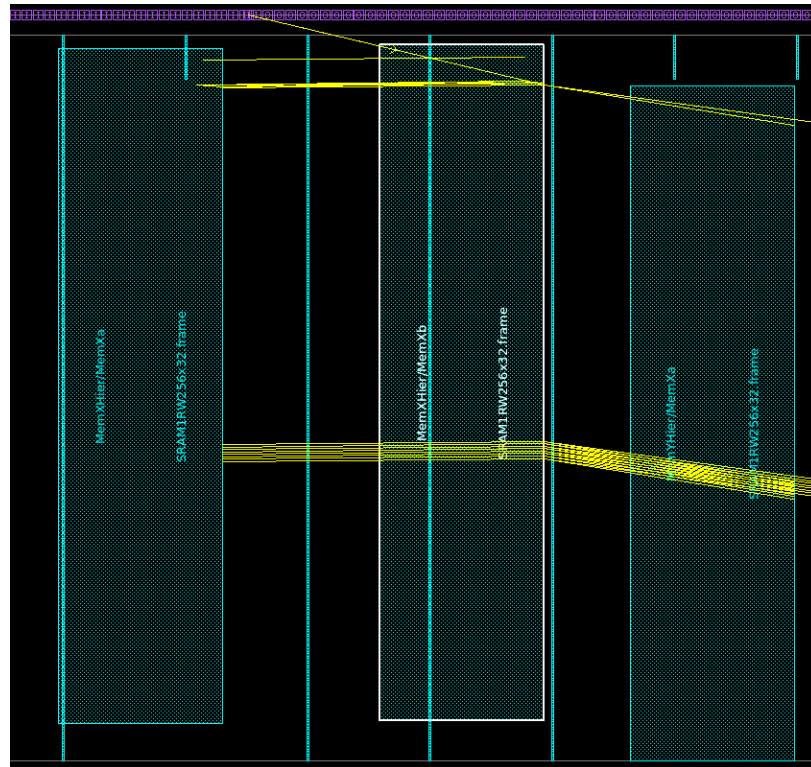


Image For No Crisscross Flylines

Avoid
Criss-Cross
connections

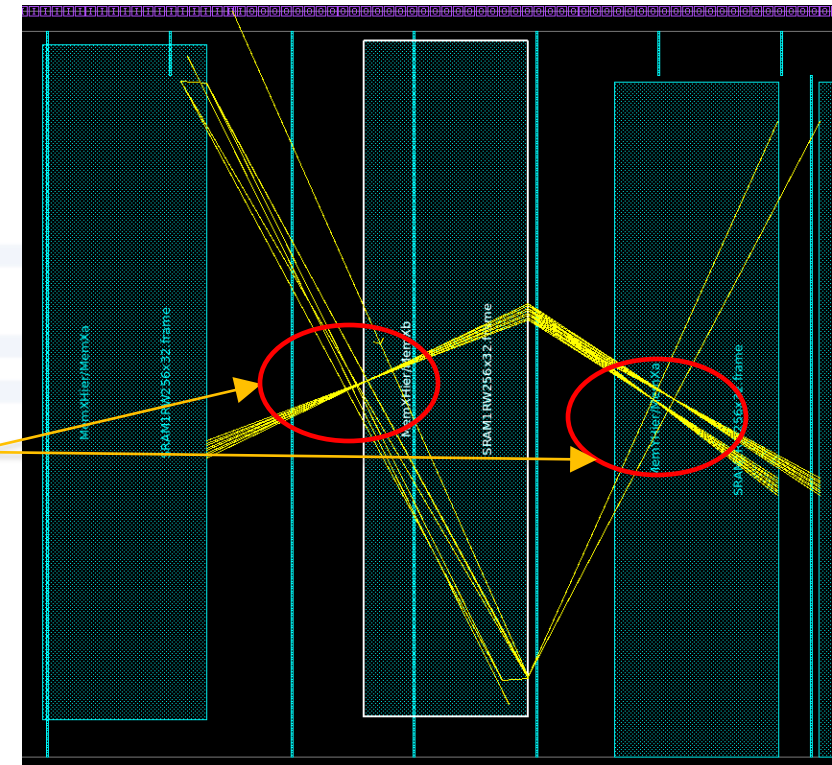
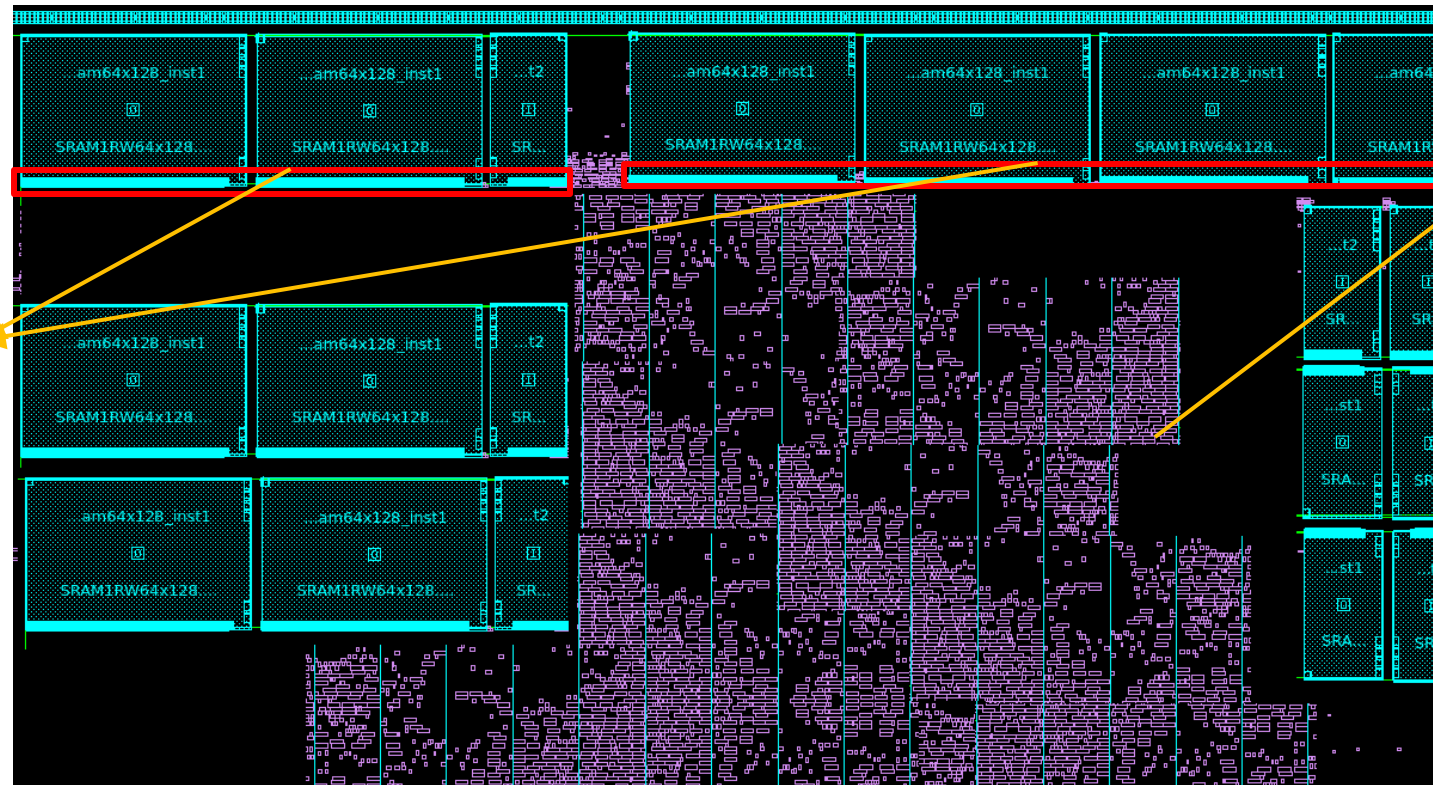


Image For Crisscross Flylines

5. Place macros so their pins are facing towards the center / core area and not towards the block boundary.

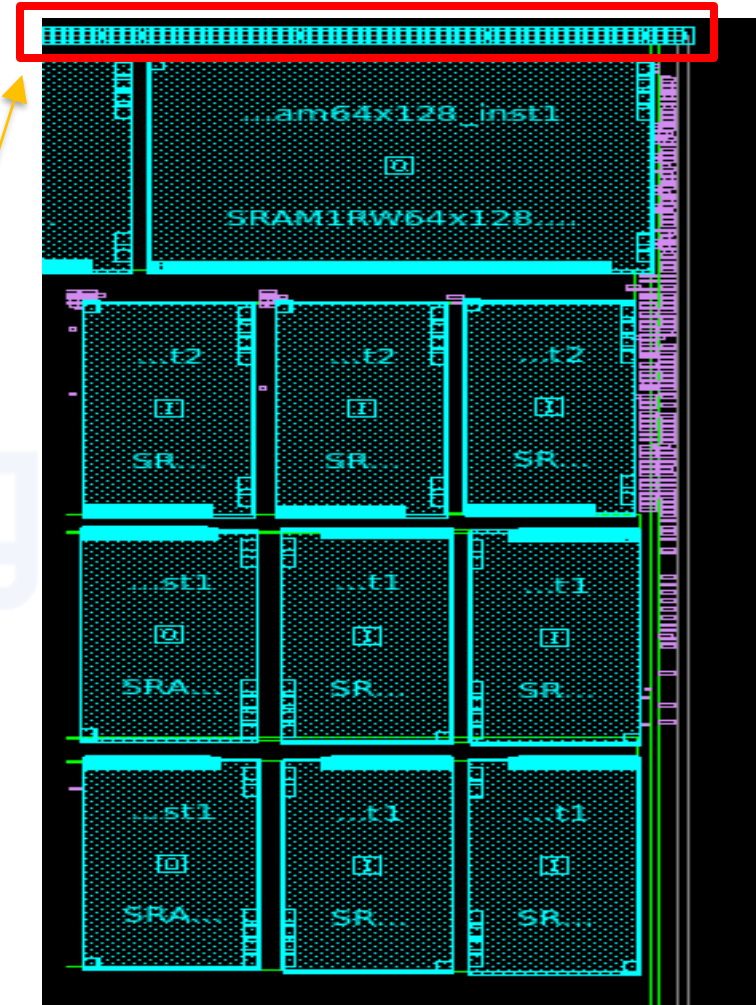


Macro pins
should face Core
area

Std cells are placed
in the center of the
core area
Do not place macro

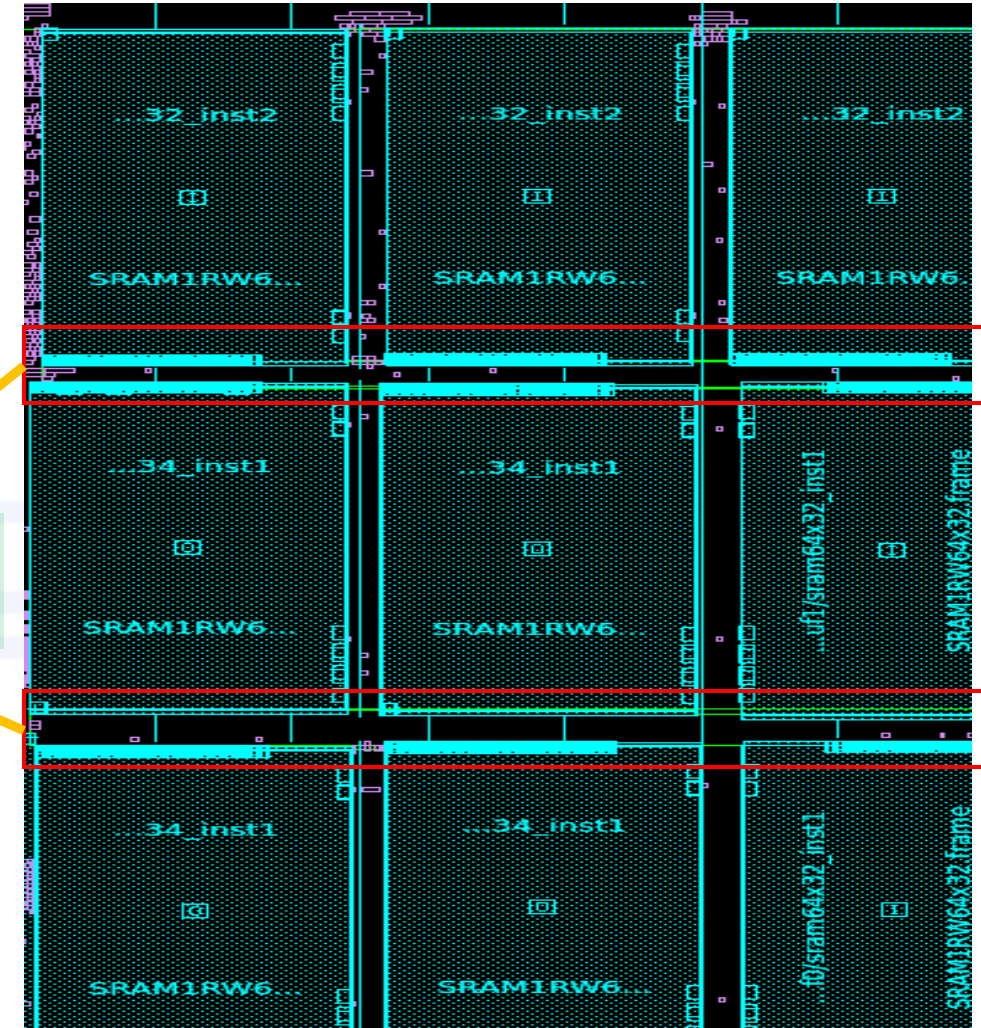
7. Avoid placing macros near to I/O ports which might block the routing for I/O ports.
- If you place macros where more I/O ports are present, provide enough space between I/O ports and Macros to avoid routing congestion.

Avoid placing macro close to ports

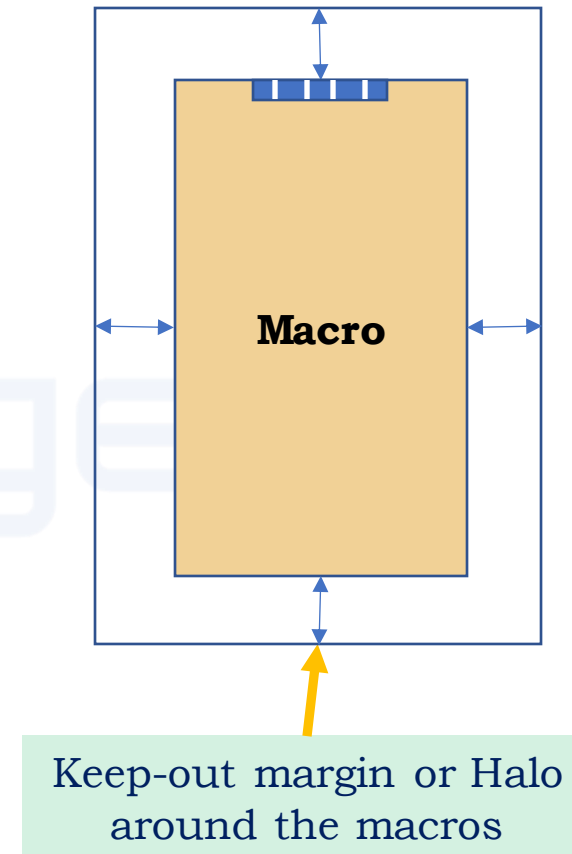


- Leave enough channel spacing between macros, to route the macro pins.
- The space left between two macros is generally referred as channel.
- The minimum spacing required depends upon the number of macro pins to be routed in the channel.

Channel
spacing

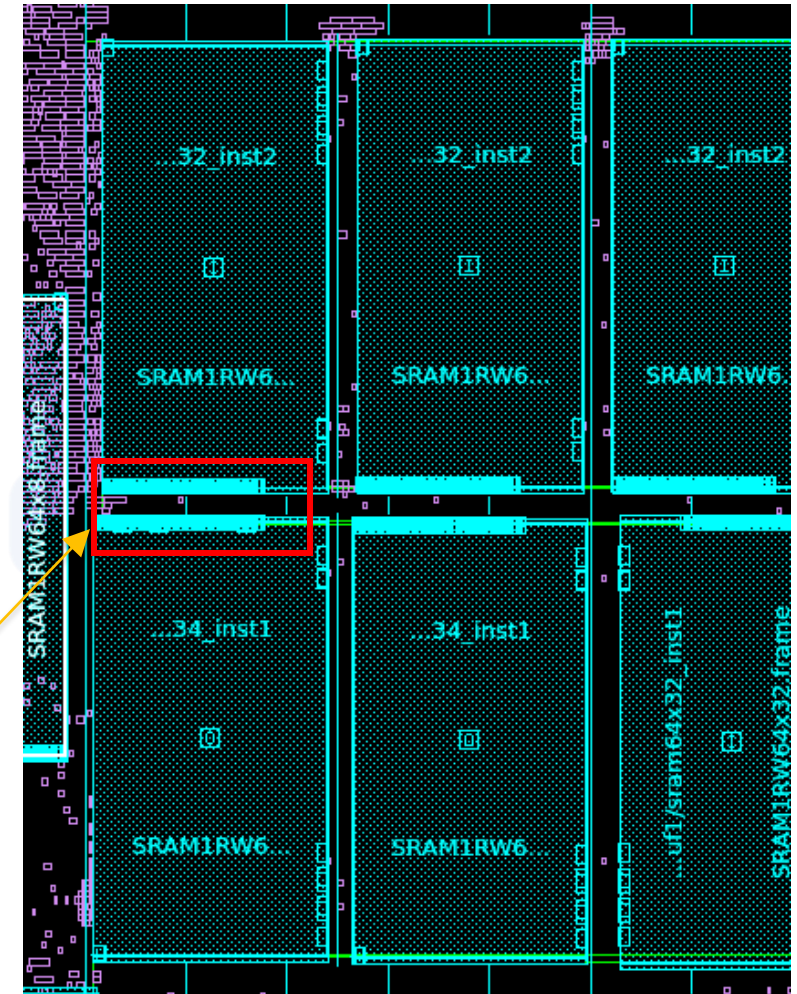


3. Create Keep-out Margin around macro
- Provide enough space around the Macro (especially on the side where macro pins are there), to prevent standard cells placement too close to the macros.
 - This helps in avoiding congestion in the channels.





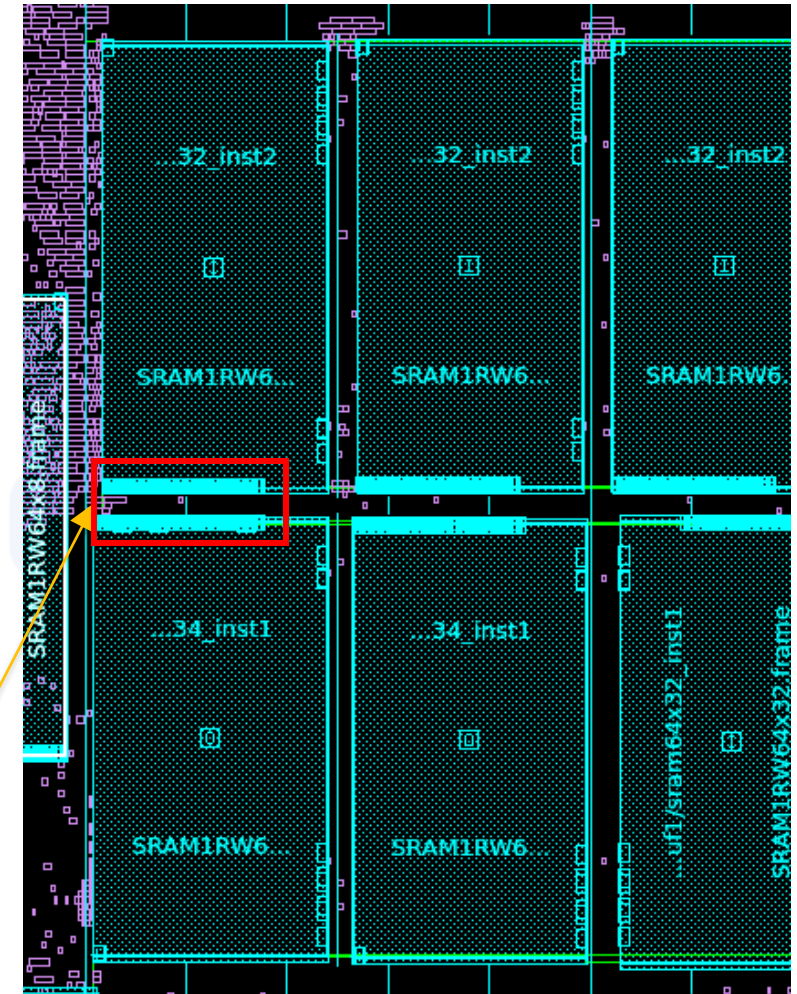
- Have keepout margin (halo), usually 0.5um-1um around macros as a protection.
- We need more empty space around macros, and in channel area to provide sufficient routing space to route the macro pins.



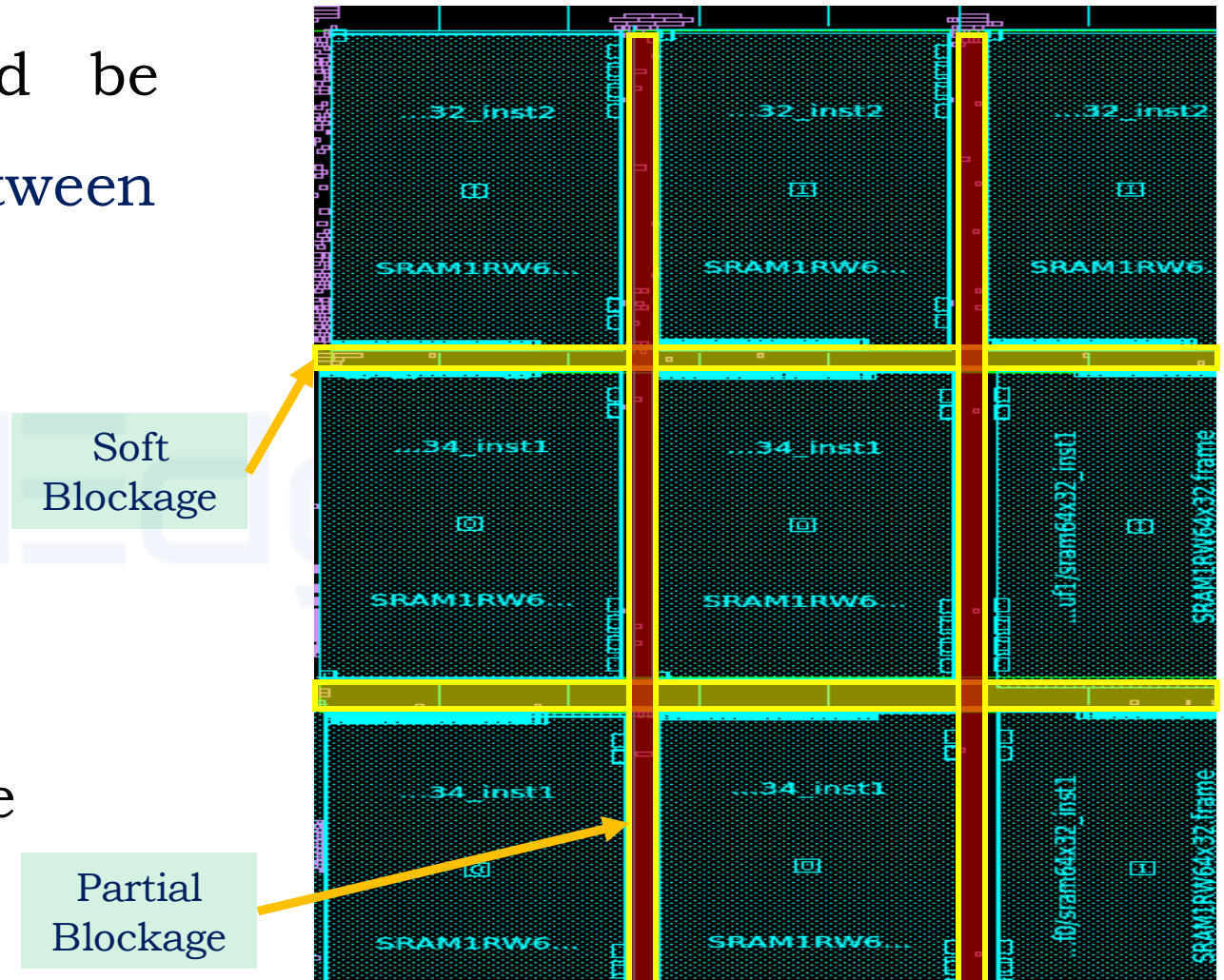
Less std cells in channel

- In this area, minimal std cells are placed which are needed for buffering the long nets.

Less std cells in channel



- Soft/Partial Blockages should be created in all the channels between macros.
- Types of placement blockages
 - 1. Hard placement Blockage
 - 2. Soft Placement Blockage
 - 3. Partial Placement Blockage



11. Make sure the macros are oriented to match the std-cell poly orientation. Usually, this info is provided in the docs supplied by the IP vendor.

12. Need to make sure there is proper PG supply available to std cells placed in the macro channels.



Introduction



Deciding the Size of the Block



Deciding the Shape of the Block



I/O Ports Placements



Macro Placement



Physical only cells

In this section, the physical only cells like endcap and tap cells are discussed in detail.

Floorplan checklist



Types of SoC Floorplan



Flat & Hierarchical Floorplanning

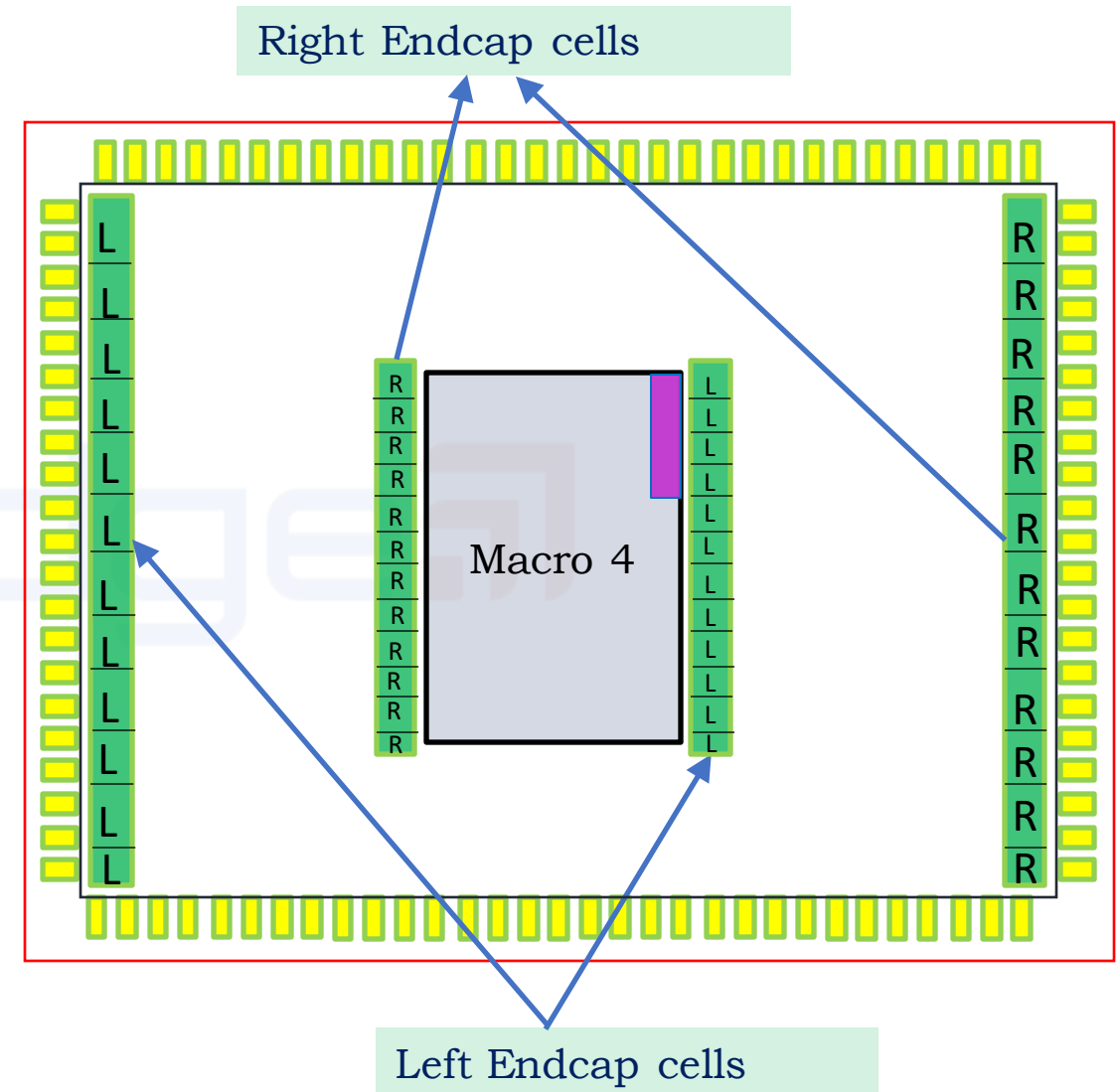


Additional Information



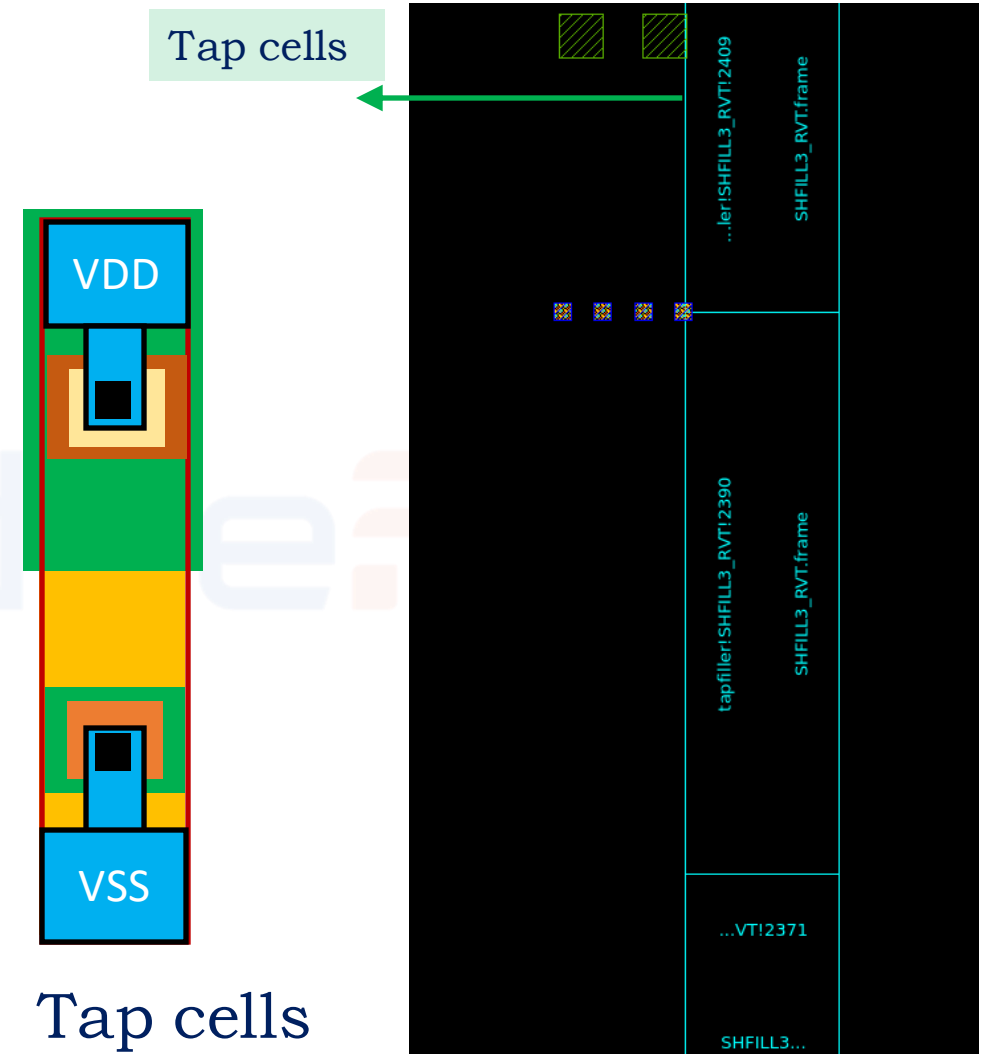
- Physical only cells are those which are present in Place and Route database, but not present in the design netlist as they have no logical function.
- These cells are added at different stages in the Place & Route Physical Design flow.
- Types of Physical only cells added at floorplan stage:
 - Endcap/boundary cells
 - Tap cells

- This cell doesn't have any timing model. Placed at the start and end of std cell row
- Placed to avoid Well proximity effect.
- At the end of the ROWs, where NWELL ends, doping concentration of NWELL is higher leading to V_t being higher for cells sitting at the end of the ROW
- When base DRC checks are run as part of signoff DRC, any missing end-cap situations are caught.



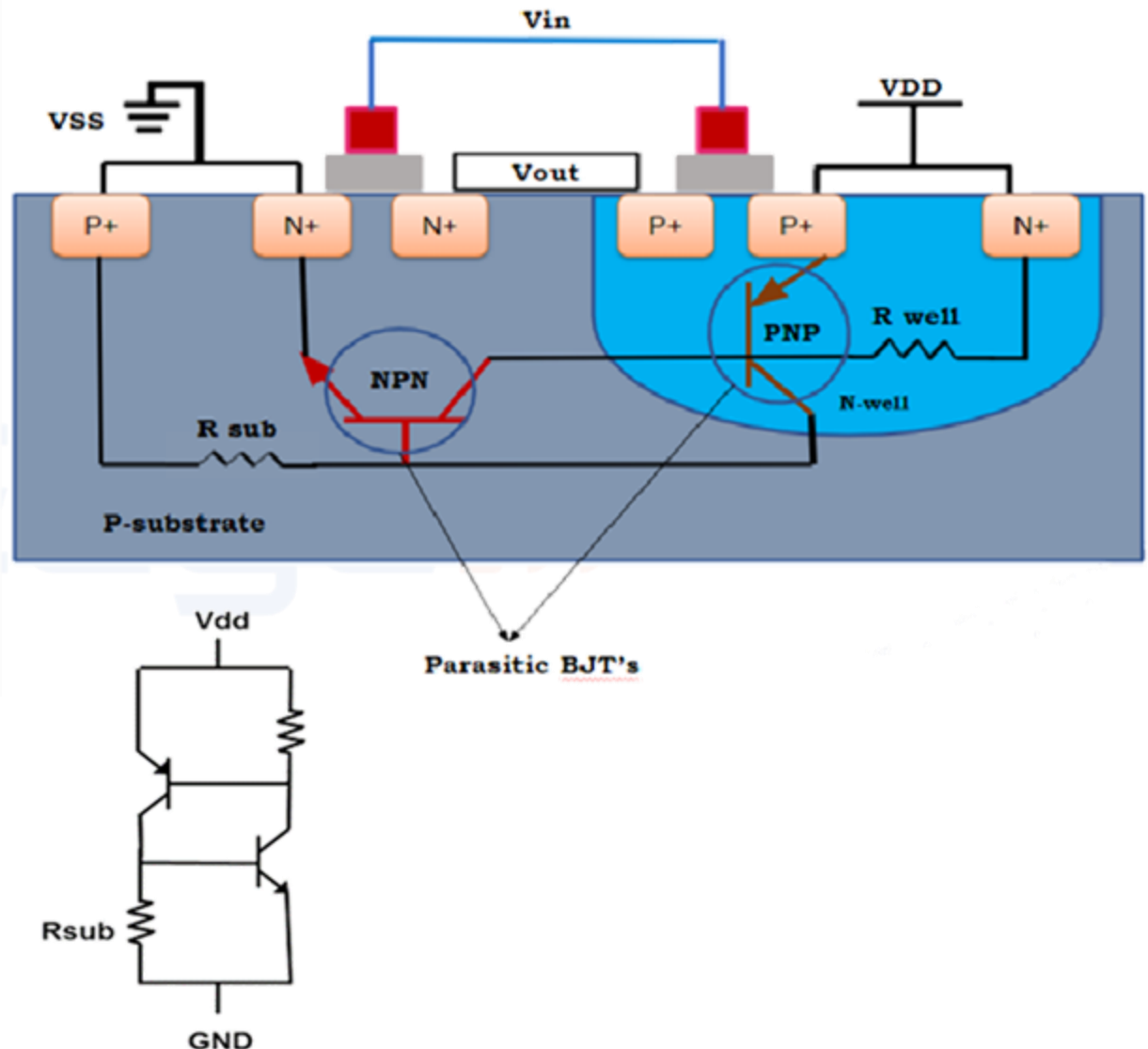
Well tap cells have no logical functions, it has only two connections (VDD, VSS).

- Tap cell is a special nonlogic cell.
- Tap cells are placed in the regular intervals in standard cell row.
- Well tap cells are used to connect the Nwell to the VDD and Pwell to the VSS.
- Well tap cells are used to prevent the latch-up problem in the CMOS design



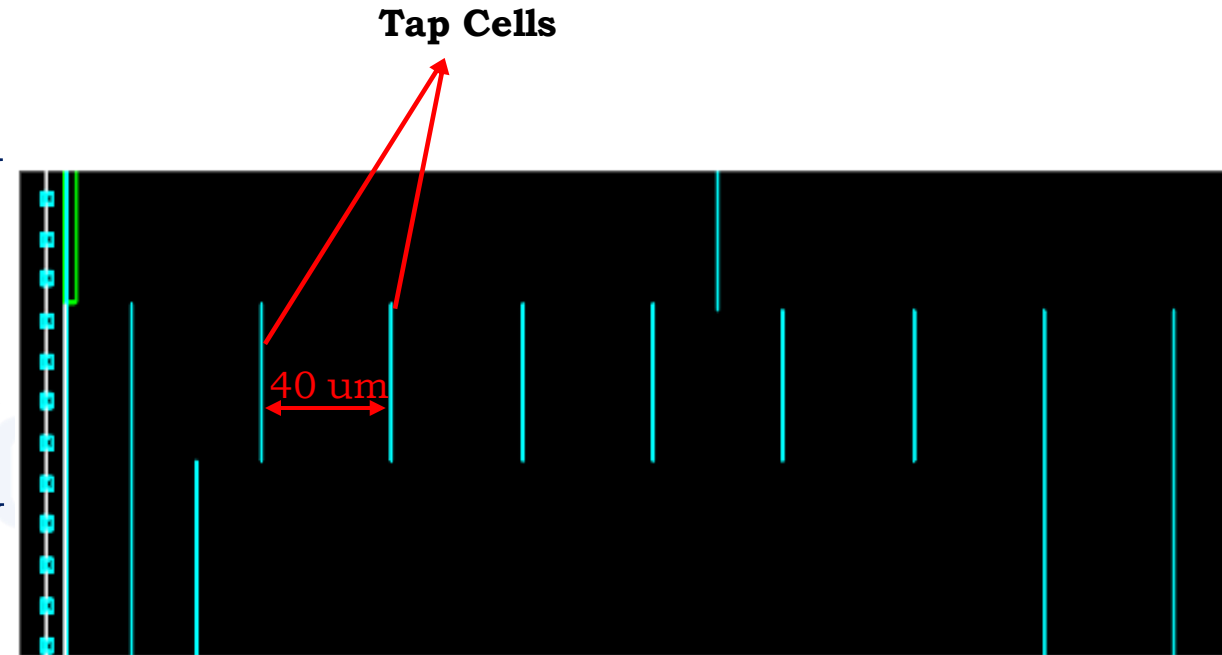
Latch-up issue:

This issue is caused when there is a low impedance path between VDD and VSS, the direct flow of current occurs in the region which might result in a complete failure of chip.



Solution for latch-up issue:

- To avoid latch-up tap cells should be added in regular intervals.
- The spacing between the well-tap insertion in the designs will vary accordingly to the technology.
 - In 28nm, the distance is 40um
 - In 16nm, the distance is 25um



Introduction



Deciding the Size of the Block



Deciding the Shape of the Block



I/O Ports Placements



Macro Placement



Physical only cells



Floorplan checklist

Types of SoC Floorplan



Flat & Hierarchical Floorplanning



Additional Information



In this section, the final floorplanning checklist like legality checks etc., are explained in detail.

- All I/O Ports are **Placed** and in **fixed state**. They should be on **track**
- All **Macros** are placed in correct orientation and in **fixed state**
- No **Macros** are **overlapping**.
- **IO port routing** should not be blocked by macro placement
- **Blockages** should be used as needed
- Use `check_legality`.

Introduction



Deciding the Size of the Block



Deciding the Shape of the Block



I/O Ports Placements



Macro Placement



Physical only cells



Floorplan checklist



Types of SoC Floorplan

Flat & Hierarchical Floorplanning



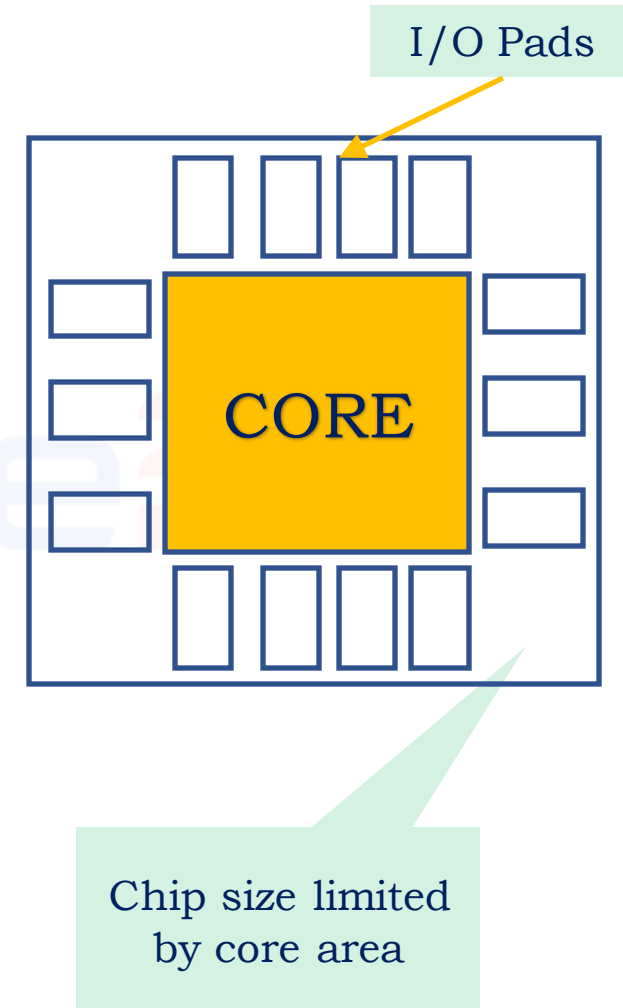
Additional Information



Core limited vs I/O limited SoC Floorplan. Flat SoC vs Hierarchical SoC Floorplanning

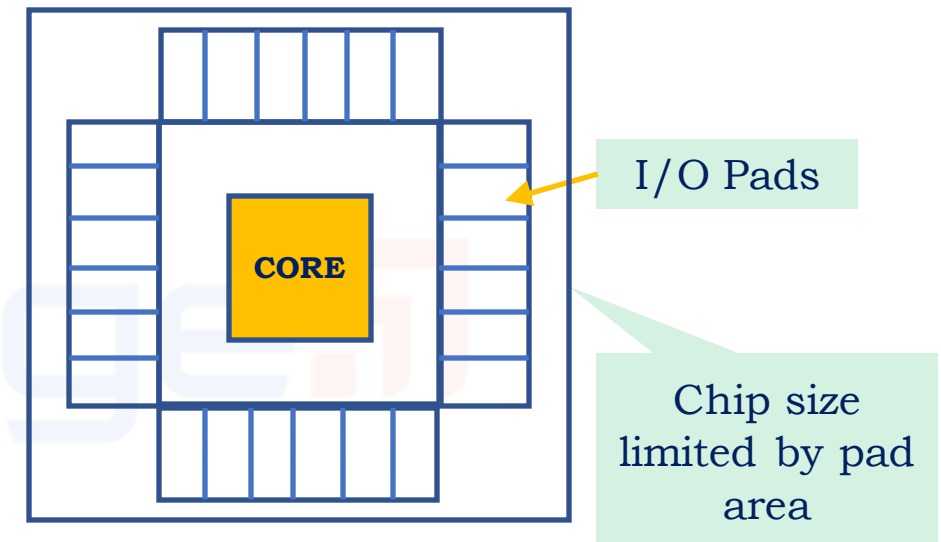
Core limited design: The chip size is limited by the core size.

- Core size determines the overall die size.
- IOs are less. Pads can be placed around the core.
- Porting target to denser process might reduce core size.



I/O limited design: The chip size is limited by the number of IO pads in the design.

- Small Core with large number of pads.
- The pad number determines necessary circumference thus the fixed die size which cannot be reduced later.
- Number of IO pads are More or larger in size (Technology dependent).



Introduction



Deciding the Size of the Block



Deciding the Shape of the Block



I/O Ports Placements



Macro Placement



Physical only cells



Floorplan checklist



Types of SoC Floorplan



Flat & Hierarchical Floorplanning

Additional Information



In the fullchip, the flat and Hierarchical floorplanning are explained with examples. The advantages and disadvantages of the hierarchical floorplanning are discussed in detail.

SoC Fullchip level

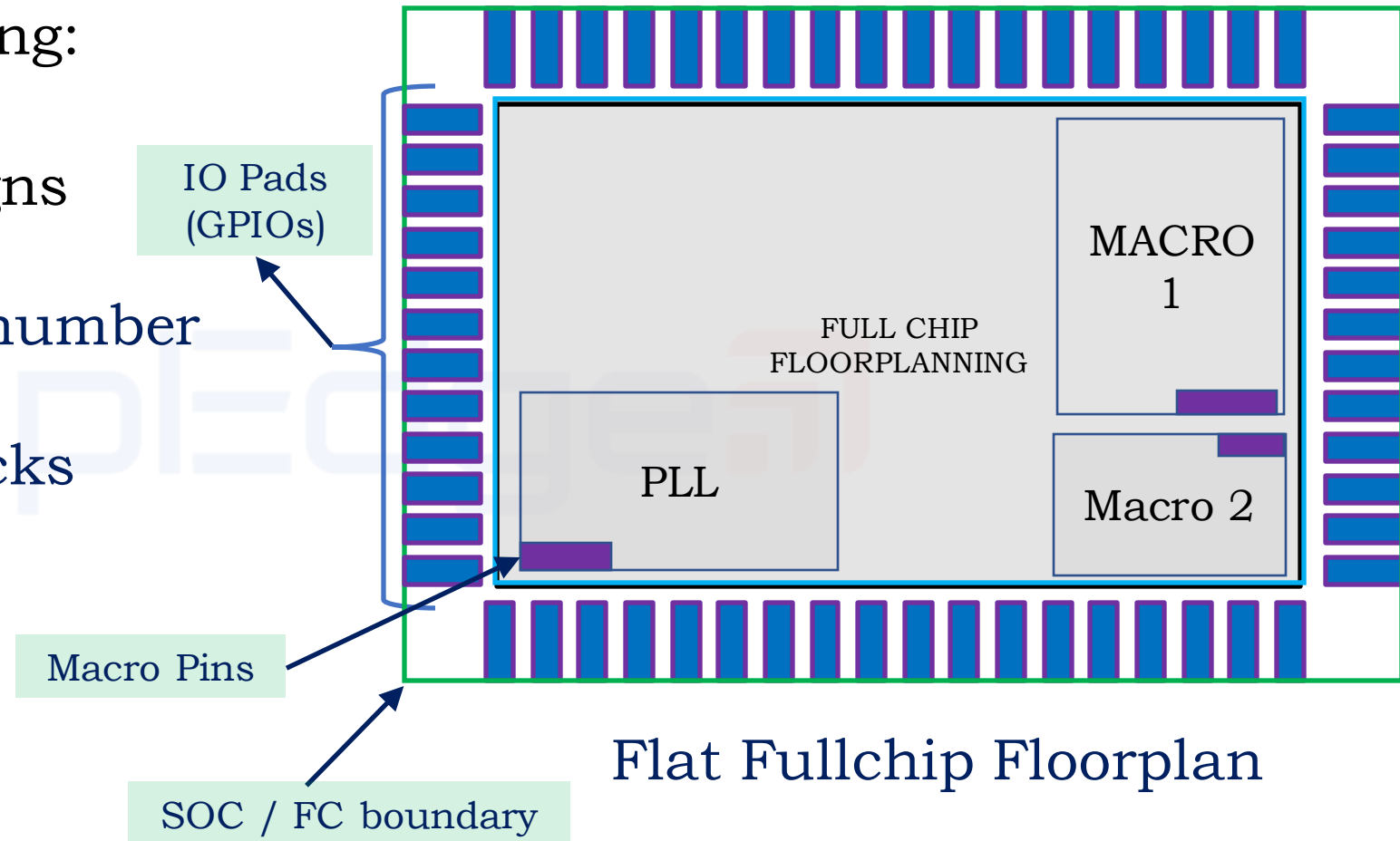
- What is Flat Floorplanning?

The whole SoC / Fullchip is considered a single block, to be implemented in Physical Design cycle

- What is Hierarchical Floorplanning?

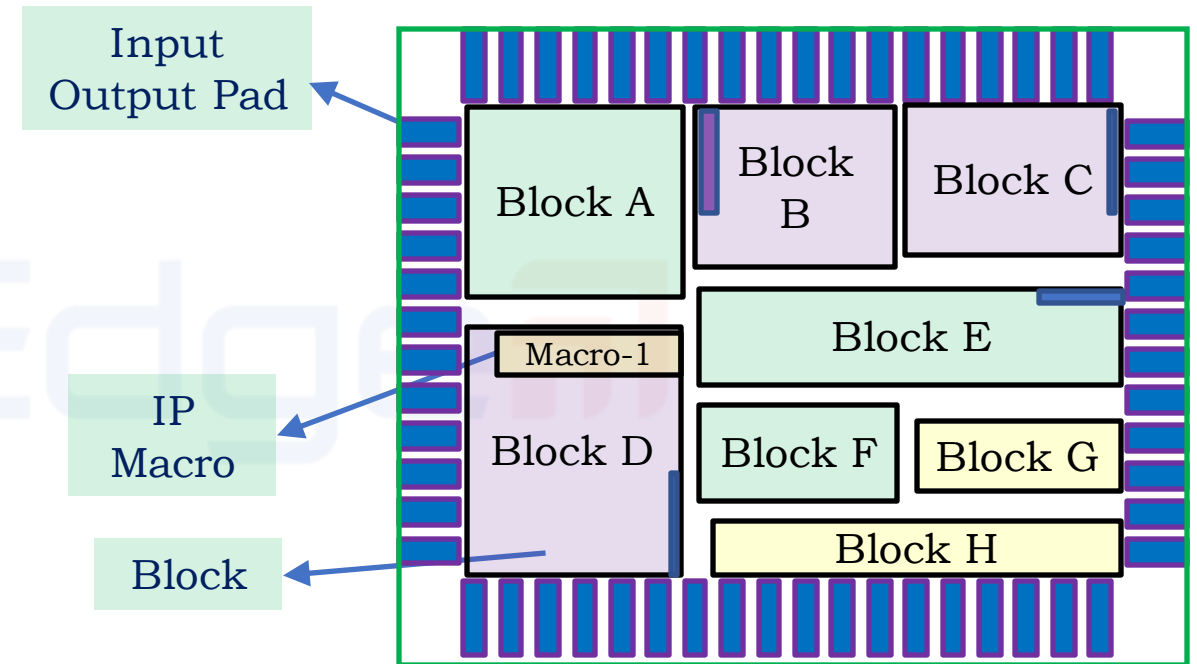
The SoC / Fullchip is quite big in size (typically > 3M cells in design, *or high number of I/O and analog blocks*), so it needs to be broken into smaller blocks. Each block is executed separately as PnR implementation and then assembled at Fullchip/SoC level for final tapeout.

- Flat Full chip Floorplanning:
- For smaller, simpler designs
- Less gate count, lower number of macros and analog blocks



Flat Fullchip Floorplan

- Hierarchical floorplanning is mostly used for complex designs with a cell count greater than 3 million.
- Hierarchical partitioning is done before the block level physical design flow.



Note: Hier Floorplanning could be either abutted or channel based design.

- Table shows Gate count and Type of Floorplanning used:

Hierarchical floorplanning Cell count > 3 million
Hierarchical OR Flat floorplanning Cell count = 1 to 3 million
Flat floorplanning Cell count < 1 million

Advantages

- Parallel Placement and routing implementation of different blocks
- Smaller blocks (with smaller total gate count) have faster runtimes
- Target should be to finish one Placement and Routing iteration in <2 days

Disadvantages

- Execution complexity increases with Hierarchical designs
- Faster Timing closure in Flat design compared to Hierarchical design
- Better Chip Area in Flat Floorplanning implementation compared to Hierarchical Floorplan.

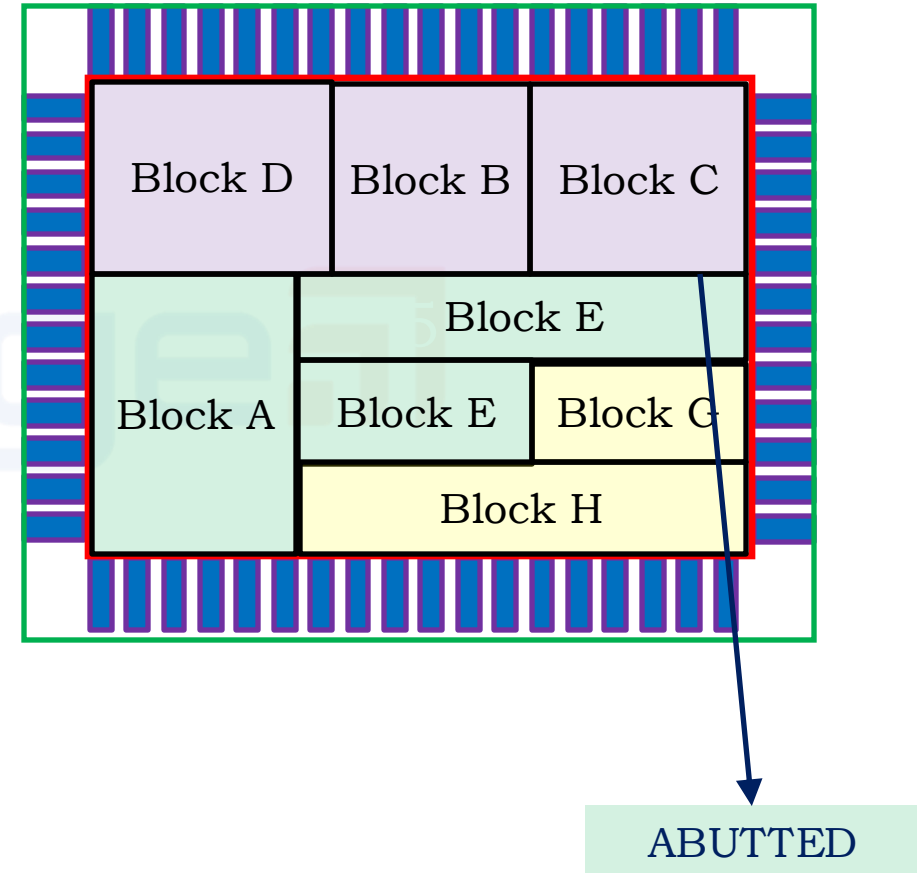
Advantages

- Examples of runtimes: ~2-3M gates design with more macros, complex design and fast clocking can take up to 7-10 days for 1 Placement and routing cycle (synth to post-route)
- Easier debug in case of issues

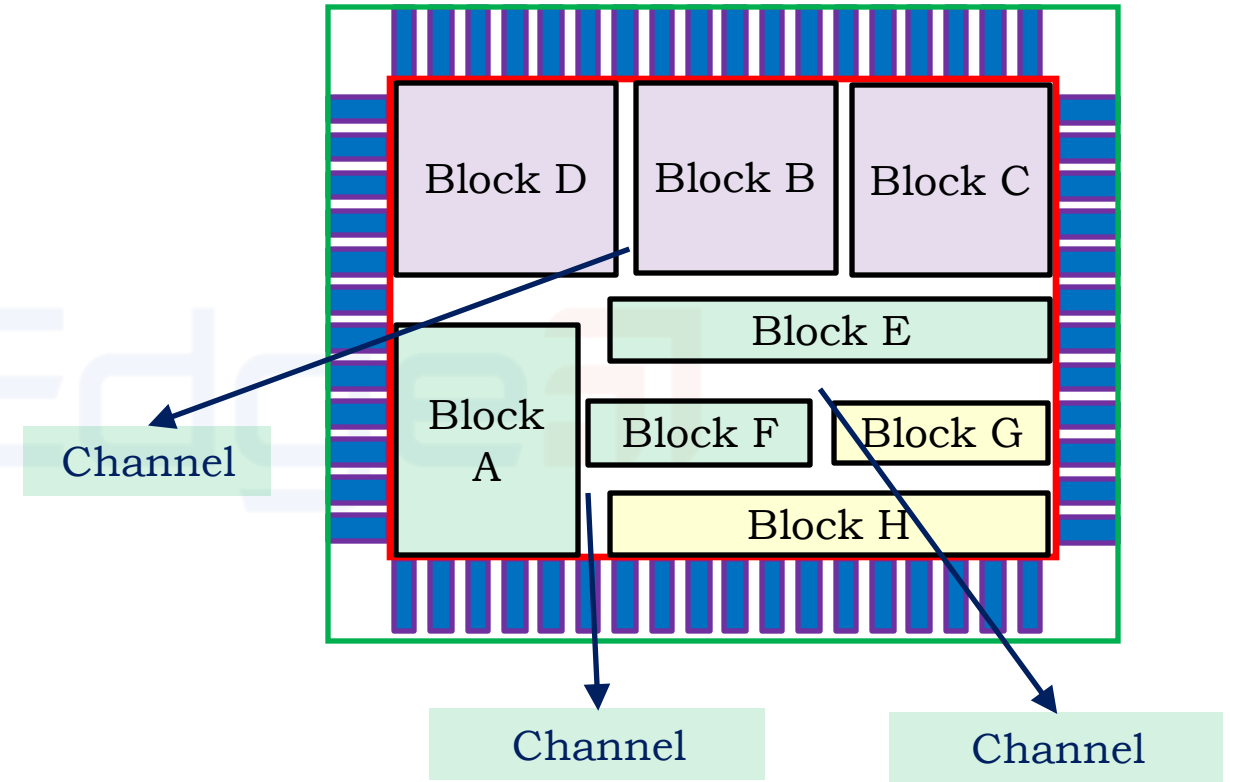
Disadvantages

- One Physical design engineer can handle the Flat Fullchip whereas having multiple blocks mean more Physical Design engineers needed to work (1-3 blocks per Physical Design engineer).

- All blocks touch each other
- No space at SoC (Fullchip) level outside of blocks
- No logic at SoC level
- All Blocks and Analog macros/IO pads, assembled at Fullchip level for tape-in db.



- Channel based SOC Floorplanning, has channel space between blocks at top-level.
- This space is used for placement of small logic and routing buffers at SoC level.



Channel based SoC FP

Introduction



Deciding the Size of the Block



Deciding the Shape of the Block



I/O Ports Placements



Macro Placement



Physical only cells



Floorplan checklist



Types of SoC Floorplan



Flat & Hierarchical Floorplanning



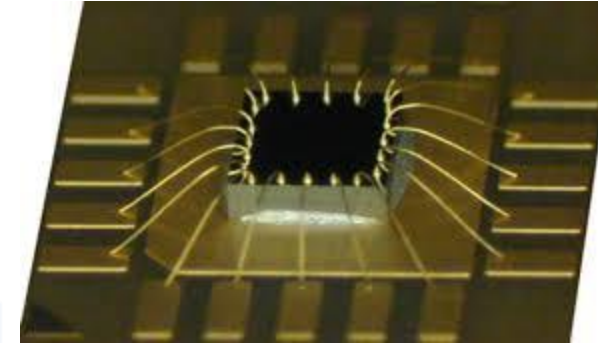
Additional Information

Wire bond, Flip chip

Full chip packaging techniques differ in the way the chip connects with the package.

There are two types of packaging.

- ❑ Wire Bond Design
- ❑ Flip Chip Design

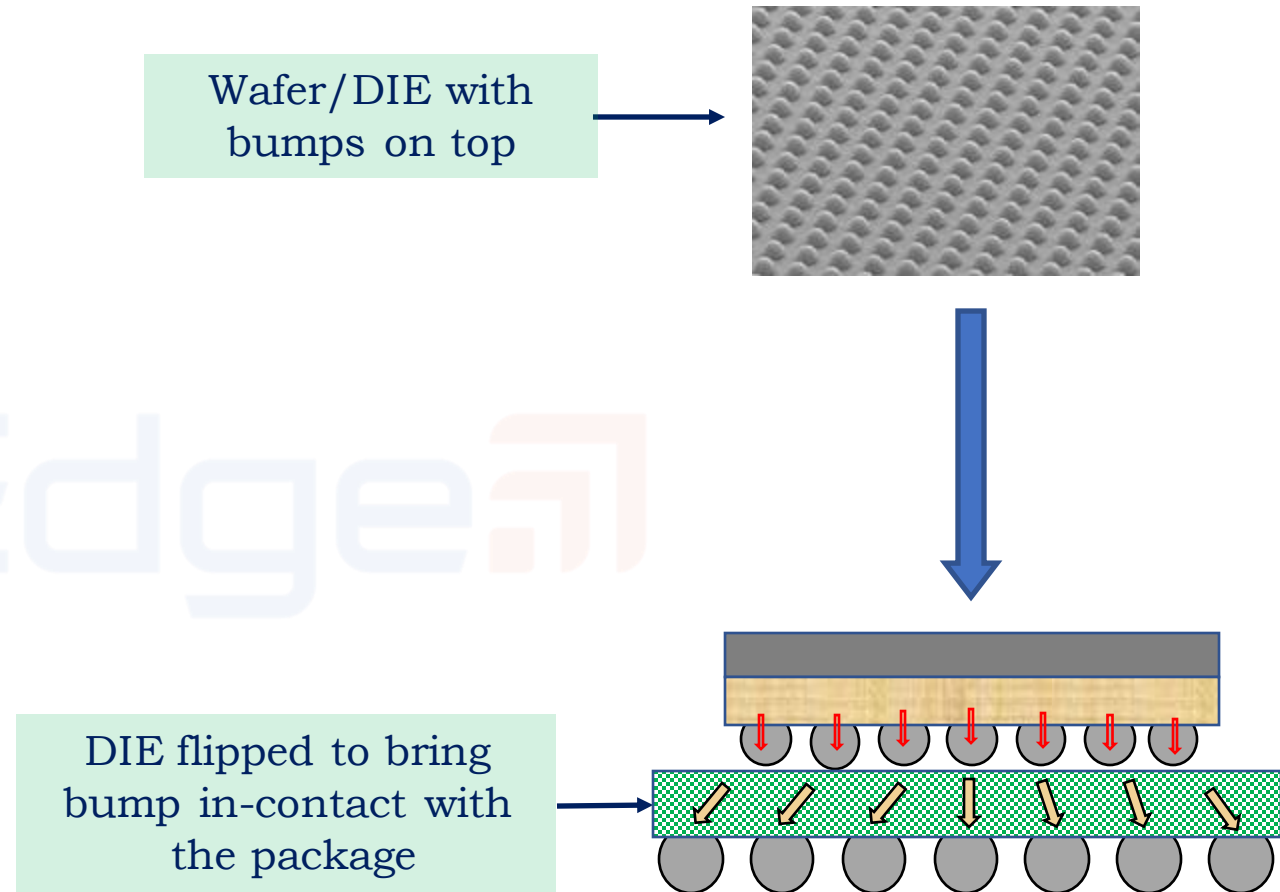


Wire bonded way of connecting the DIE to the package

Flip chip means die flipped over the circuit board facing downward.

Advantages of Flip chip:

- Small package size
- High I/O density
- Highly reliable
- Lower stress on a given area
- Disadvantages of Flip chip:
- Flip chip fabrication process is costly.



- Block Level Floorplanning
- Macro Placement & Guidelines
- Physical Only Cells
- SoC Level Floorplan types
- Types of SoC packaging



Module 7

Power Planning

- In this module you will learn about Power Planning, IR Drop, Electro-Migration and Low Power Techniques.
- You will understand about the concepts of Leakage Power, Internal Power and Switching Power.

- Power Plan Objectives
- Power Consumption in IC
 - ✓ Leakage power
 - ✓ Internal Power
 - ✓ Switching Power
- Power Planning
- Low Power Techniques
 - ✓ Clock Gating
 - ✓ Power Gating
 - ✓ Multi VDD and DVFS

Power Planning Objectives

In this module Efficient power supply, IR drop targets, electromigration targets and optimal resource usage are discussed.

Power Consumption in IC 

Power Planning

Low Power Techniques



- Provide efficient power supply to each component in the design
- Power grid should meet the set IR drop targets
- Power grid should be able to meet all Electromigration targets
- Power grid should be optimal in terms of resource usage, as too pessimistic power routes might lead to routing congestion issues

Power Planning Objectives



Power Consumption in IC

In this section total power, leakage power, internal power and switching power are discussed in detail

✓ Leakage Power



✓ Internal Power



✓ Switching Power

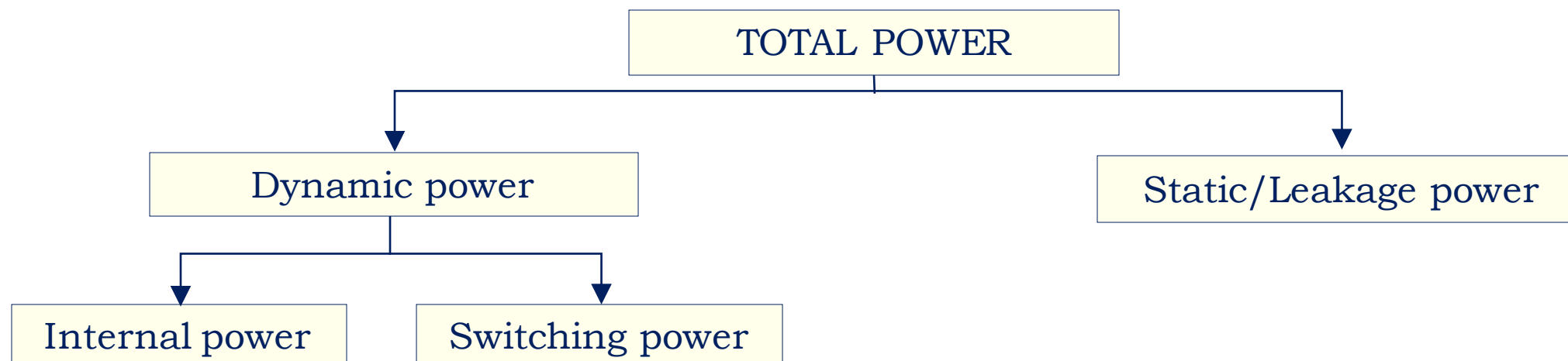


Power Planning



Low Power Techniques





$$\text{Total Power} = \text{Leakage Power} + \text{Internal Power} + \text{Switching Power}$$

- Sub-threshold leakage
- Gate leakage, Gate induced drain leakage
- Drain induced barrier lowering

The Power consumed internal to the std-cells, memories, IPs etc. It also Includes power consumed due to internal switching and short circuit current/crow-bar current

It is defined as the Power consumed due to switching of load on the output pins of the std-cells/ macros/ memories/ Ips.

Power Planning Objectives



Power Consumption in IC



✓ Leakage Power

This explains the leakage power with inverter as an example.

✓ Internal Power



✓ Switching Power



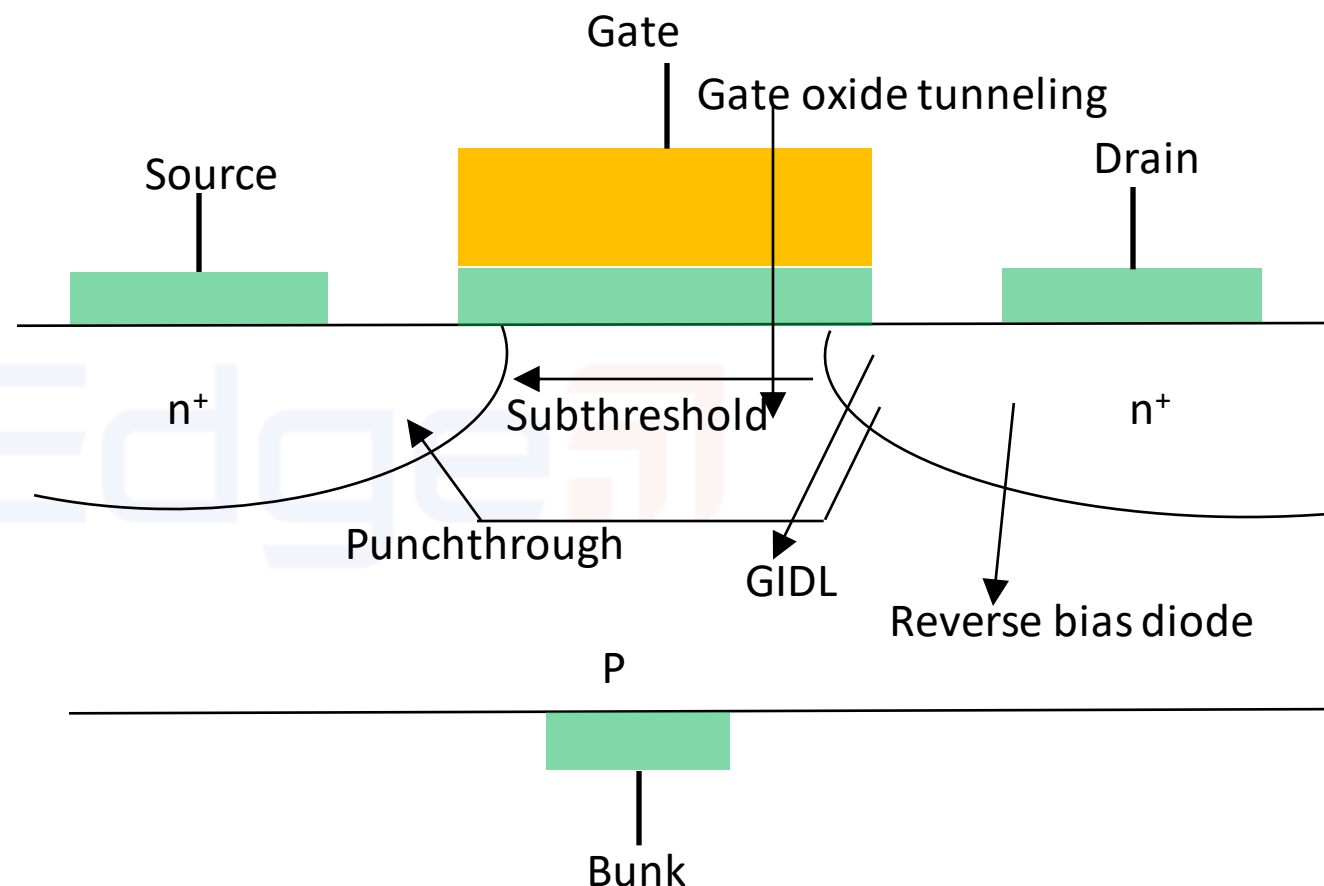
Power Planning



Low Power Techniques



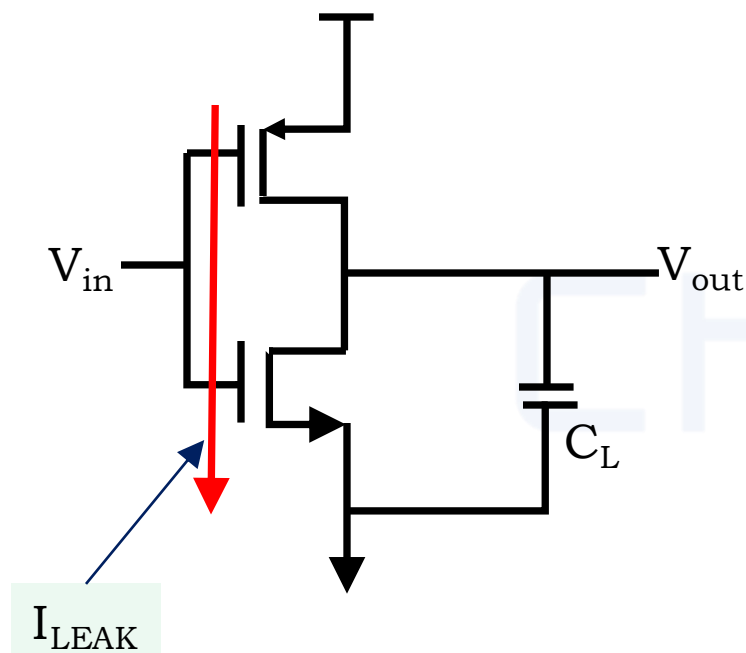
- Sub threshold is major component of leakage power
- Current Equation I_D
$$I_D = \mu_n C_{ox} \frac{W}{L} (V_{GS} - V_{TH})^2$$
- Leakage is high with LVT cells and LVT cells are used for fast corner.



STD-CELL	LEAKAGE POWER	PERFORMANCE	GATE DELAY
HVT	LOW	SLOW	HIGH GATE DELAY
LVT	HIGH	FAST	LOW GATE DELAY

- Sub threshold leakage varies exponentially with V_{TH} with respect to the decreased dependency of delay over V_{TH} .
- HVT cells are used to decrease the leakage power in non-critical paths.
- And LVT cells are used in critical paths to meet timing.
- Sub leakage w.r.t V_{th} is $I_{sub} \propto e^{\left(\frac{V_{GS}-V_{TH}}{nV_{TH}}\right)}$

- State dependent leakage power of the cell is present in .lib



Example Of Inverter Cell

```
cell (INVX16_RVT) {  
    cell_footprint : "AV" ;  
    area : 5.08288 ;  
    cell_leakage_power : 1.635493e+04 ;  
    driver_waveform_rise : "preDrv" ;  
    driver_waveform_fall : "preDrv" ;  
    leakage_power () {  
        when : "A" ;  
        value :  
            "1.6354926e+04" ;  
    }  
}
```

Leakage Power of the
Inverter Cell

Power Planning Objectives



Power Consumption in IC



✓ Leakage Power



✓ Internal Power

This Section explains the internal power with inverter as an example.

✓ Switching Power



Power Planning



Low Power Techniques



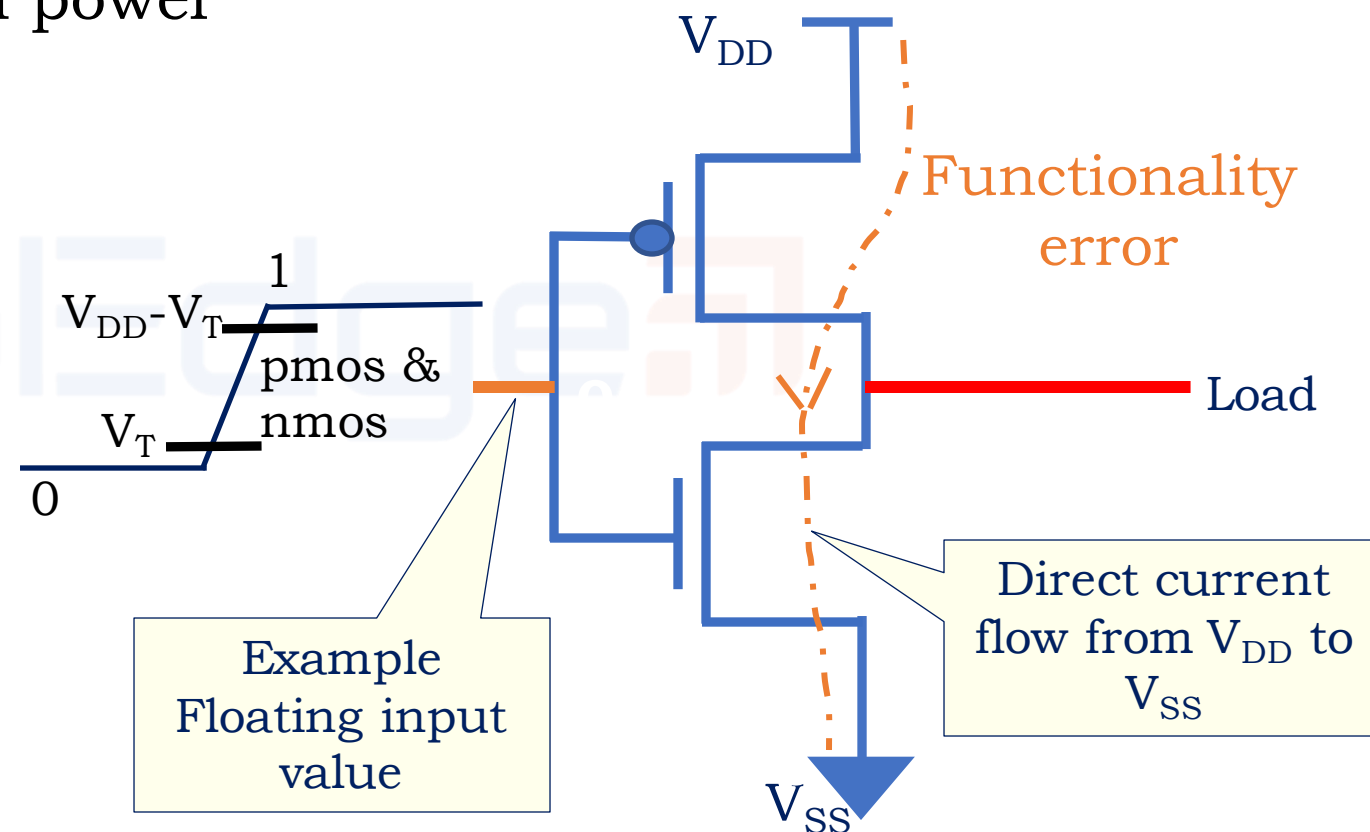
Internal energy taken from timing libs and power & is expressed as

$$E \times F \times \text{Activity_factor}$$

where E- Internal Energy ; F is Frequency of Operation.

- ✓ SVSS Through nMOS and pMOS During certain period of input transition.
- ✓ During switching, a circuit dissipates internal power by the charging or discharging of any existing capacitance to internal to the cell.
- ✓ In most simple library cells internal power is primarily due to short circuit power.
- ✓ Short circuit current is the current which flows directly from VDD to VSS

- Power Dissipation due to this temporary short circuit path and the internal capacitance is internal power
- Depends on some factors
 - ✓ Input Transition
 - ✓ Output Capacitance



Power Planning Objectives



Power Consumption in IC



✓ Leakage Power



✓ Internal Power



✓ Switching Power

In this the switching power is explained in detail with inverter as an example.

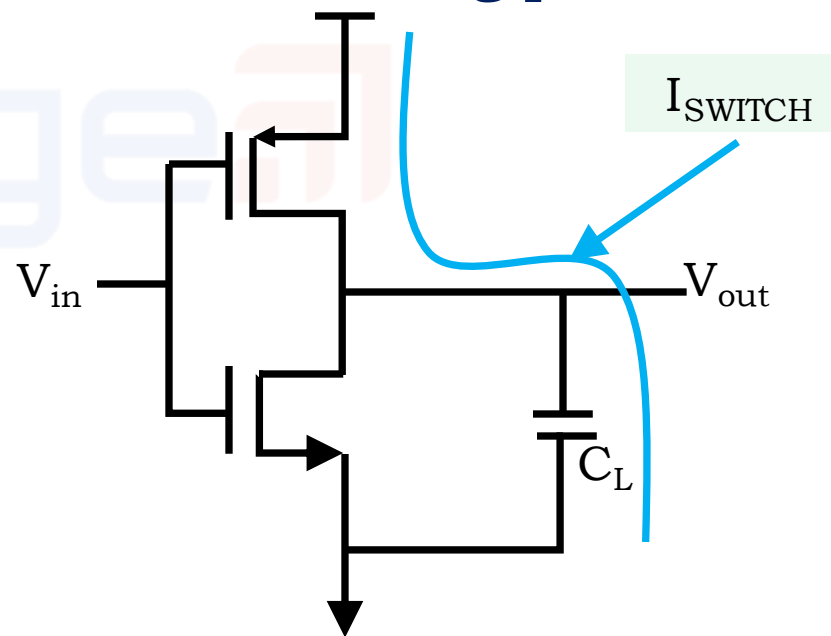
Power Planning



Low Power Techniques



- It is the **power consumed** when the device is active, when signals are changing values (by switching logic states).
- Primary source of **dynamic power** consumption is **switching power**
- $P_{DYN} = A \times C \times V^2 \times F$
 - ✓ A is activity factor
 - ✓ C is Load capacitance
 - ✓ V is supply voltage
 - ✓ F is clock frequency



Power Planning Objectives 

Power Consumption in IC 

Power Planning

In this section detailed information of power planning of Fullchip and Block level are discussed.

Low Power Techniques 

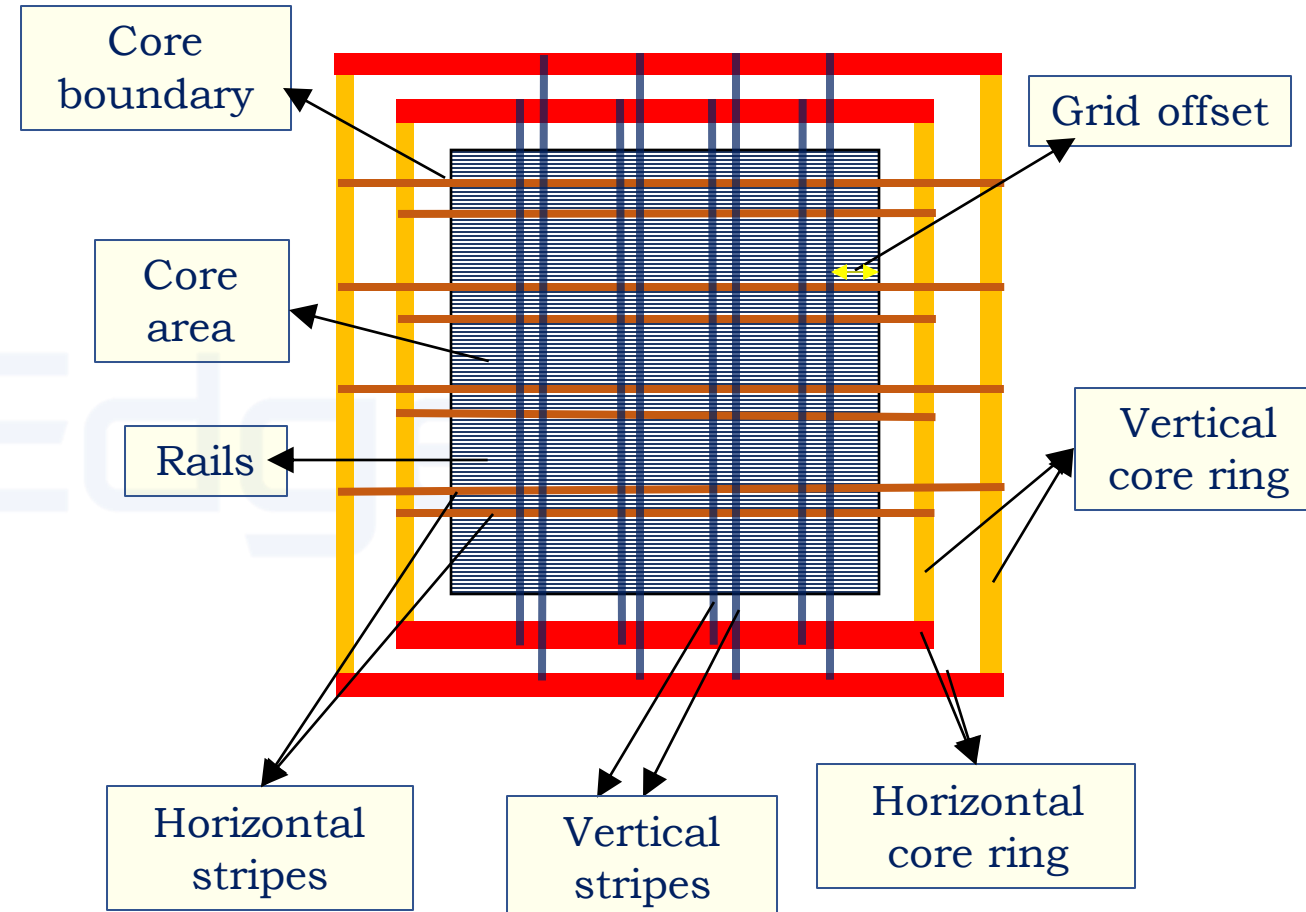
- Power planning is meant to provide power to every macro, standard cell and all the other elements present in the design.
- To connect Power to the Chip by considering issues like EM and IR Drop
- Power Routing also called Pre-Routing.
- Pre-Routing includes creating Power Ring, Stripes/Mesh/Grid, and Standard Cell Power Rails
- Power Planning also includes Power VIA insertion

Power Rings:

- V_{DD} and V_{SS} Rings are created around the Core and Macro.

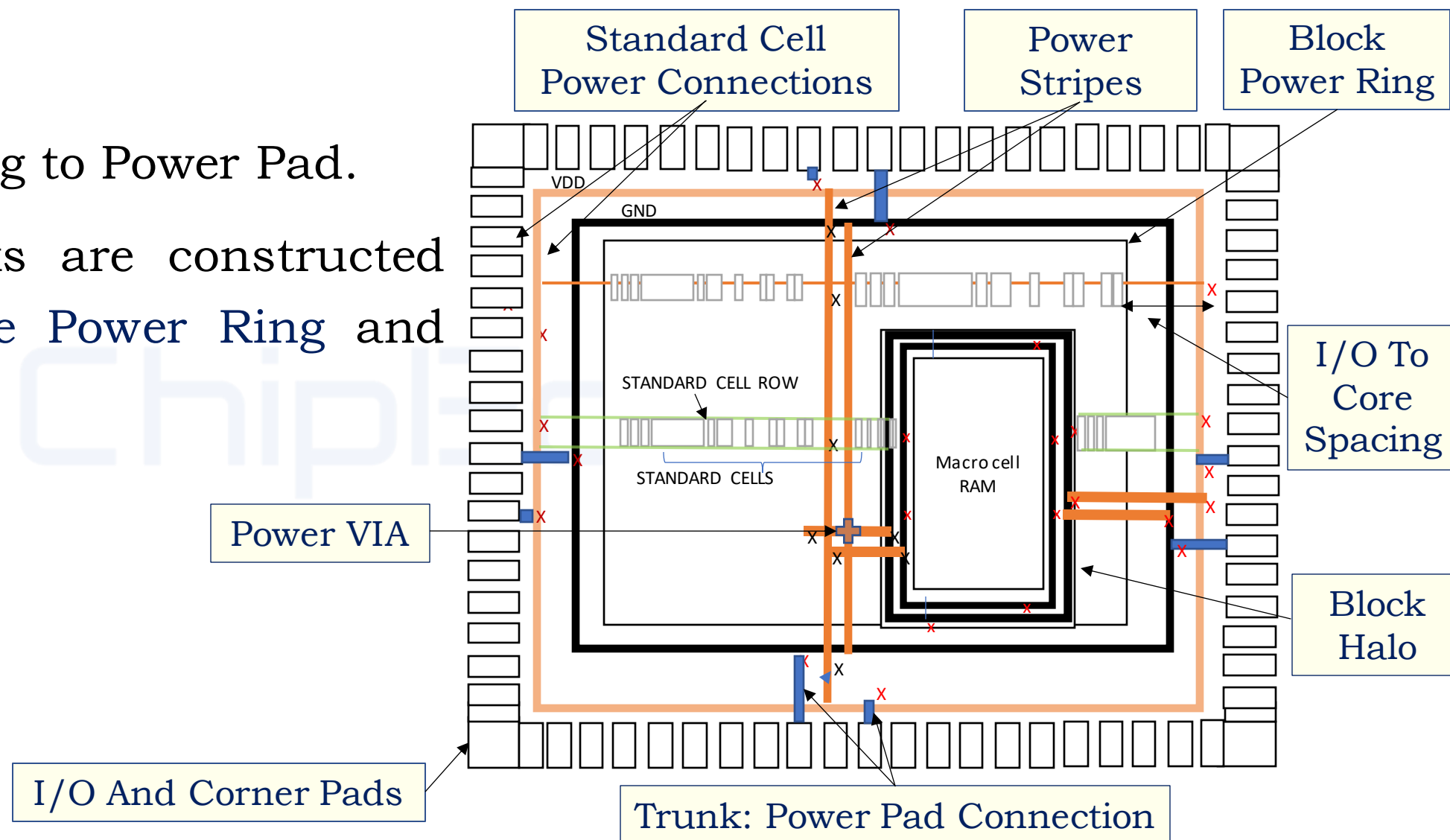
Stripes:

- Power Stripes are created in the Core Area to tap power from Core Rings to the core area.



Trunks

- Connects Ring to Power Pad.
- Power Trunks are constructed between Core Power Ring and Power Pads.

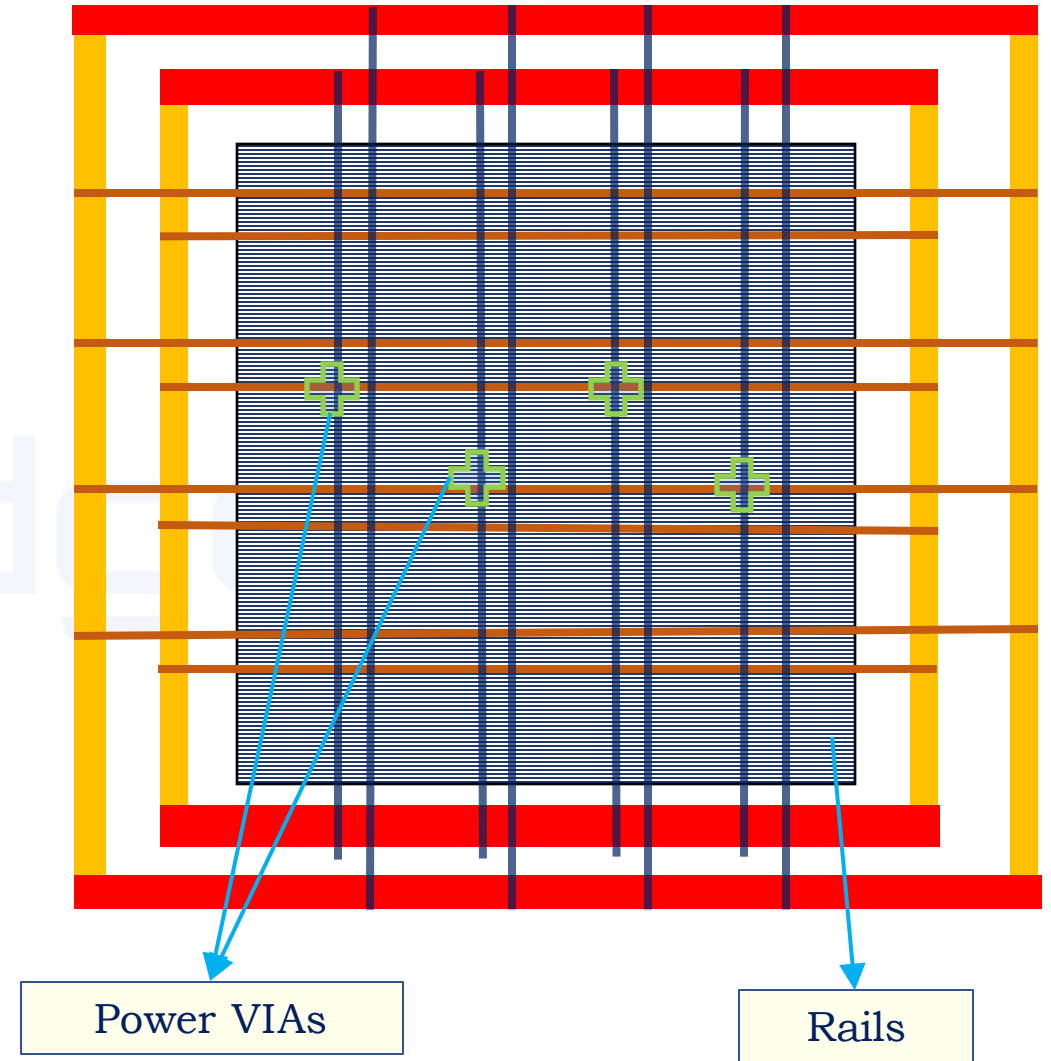


Rails (Special Route)

- Connect VDD and VSS to the standard cell.
- Standard Cell Rails are created to tap power from Power Stripes to Standard Cell Power/Ground Pins.

Power VIAs

- Insert all Power VIAs between Ring & Grid, Grid & Rail and Vertical Grid & Horizontal Grid

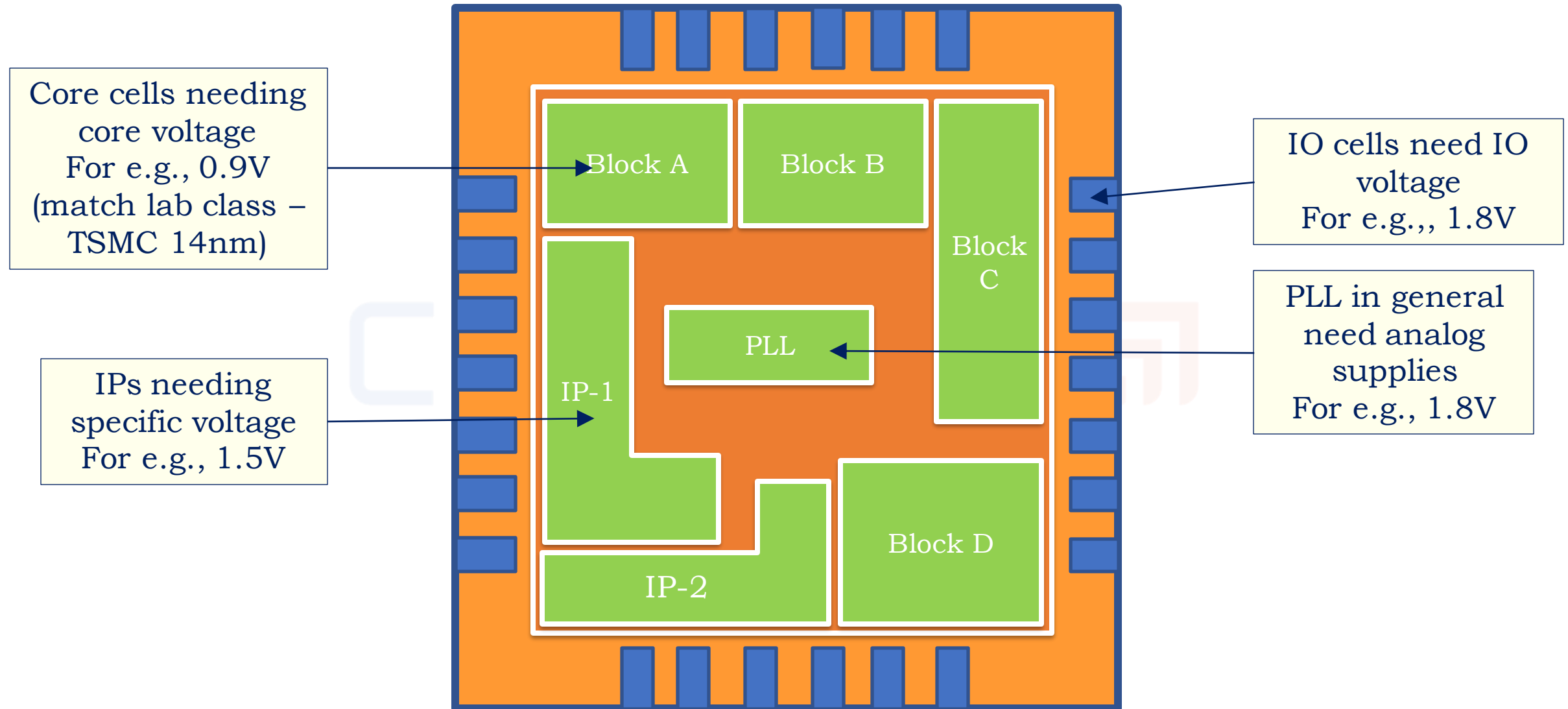


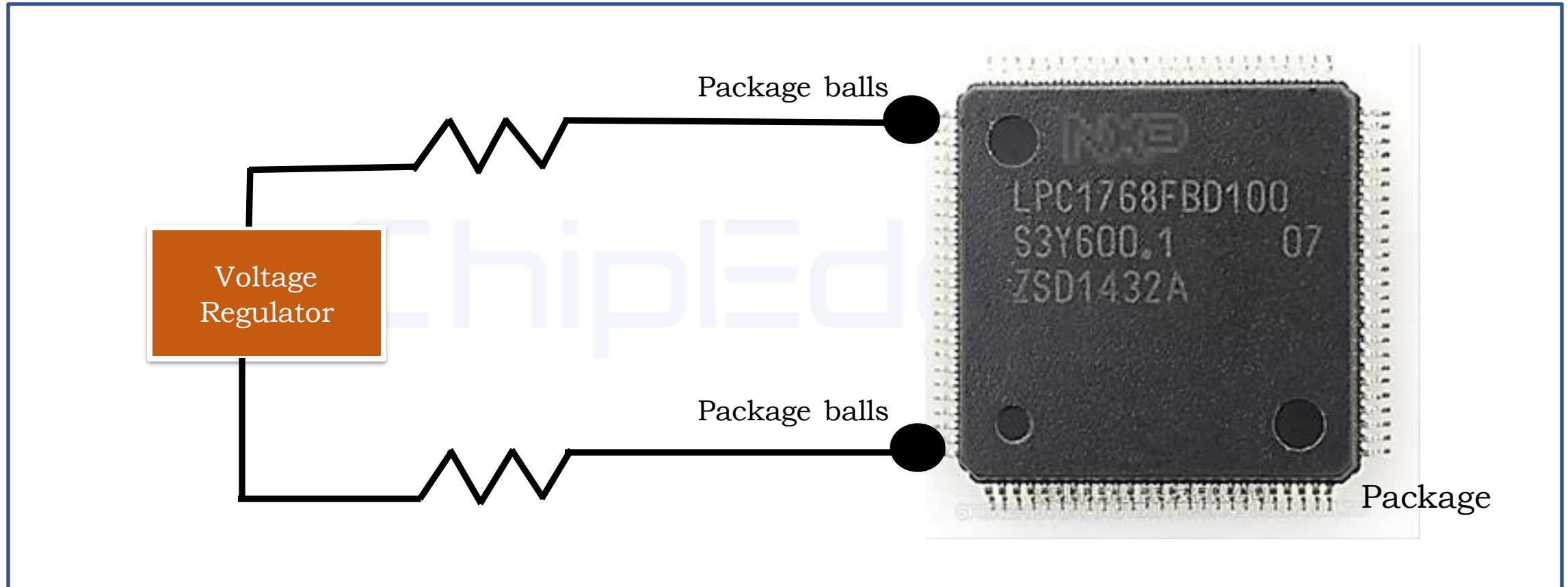
Factors to be considered for power planning

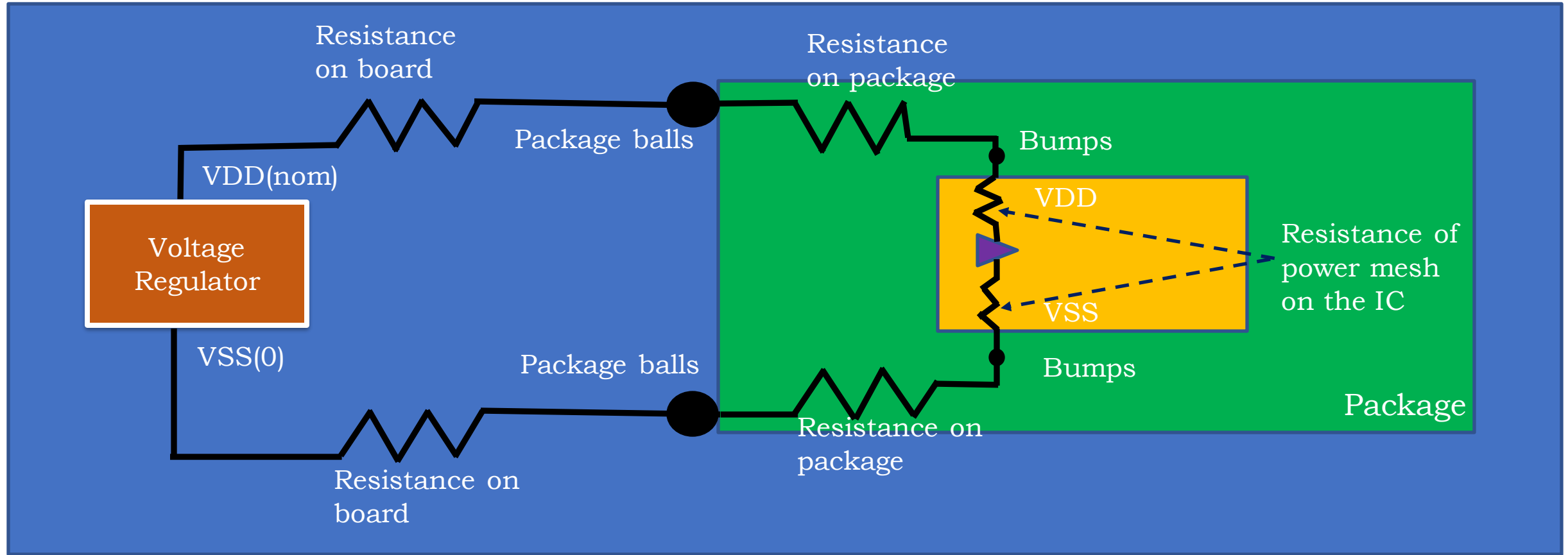
- Multiple voltages in the same die.
- IR drop
- Electro-migration (EM) effects
- Routability



Multiple Voltages In The Same Die



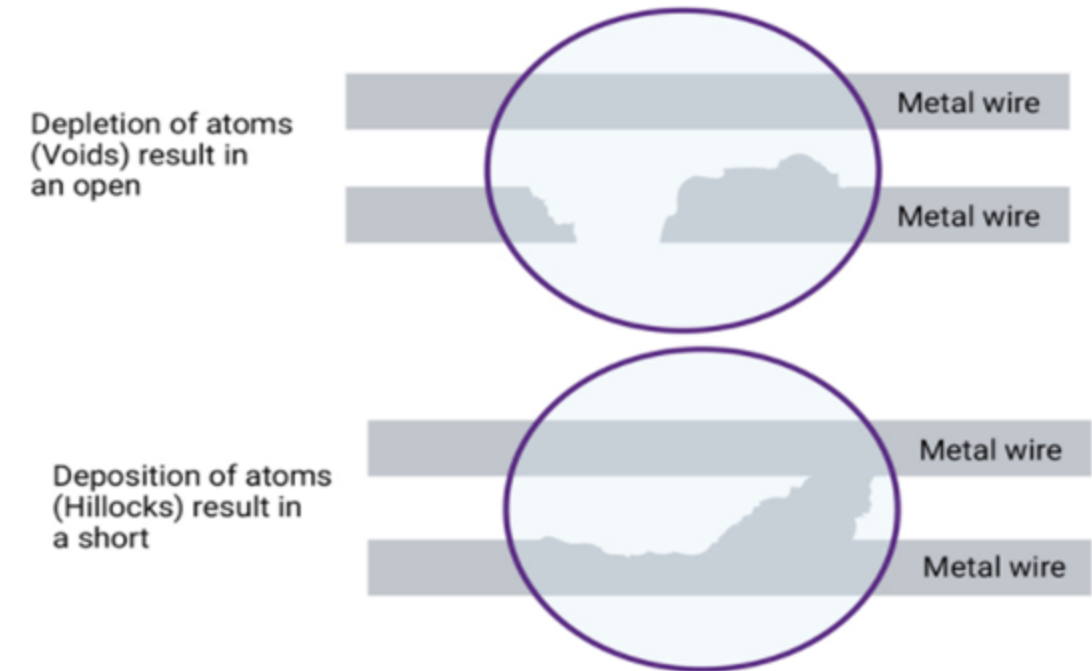
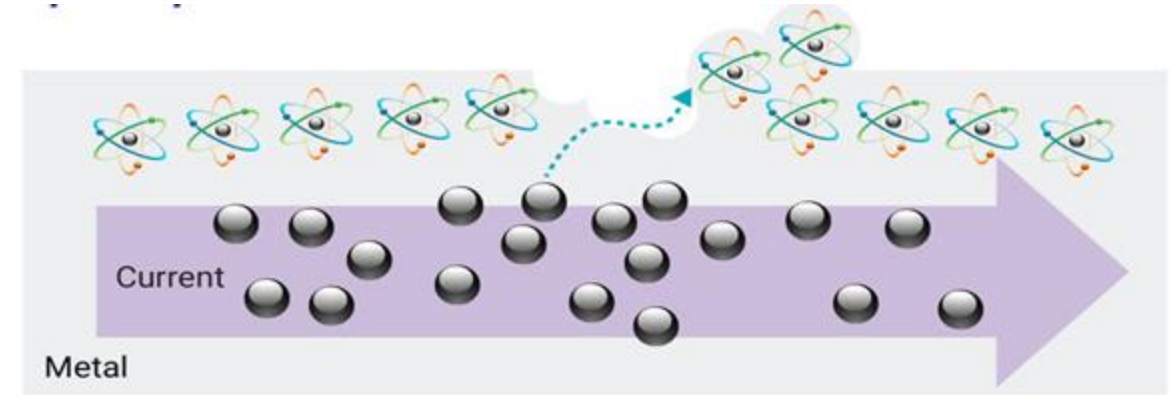




- Voltage Regulator(VR) on the board supplies power to the IC
- IR drop from Voltage Regulator on board to the Package balls
- IR drop within the package: from the package balls to the Die bumps
- IR drop within the die: from the bumps to the std cells/memories
- General IR drop budgets : 5% for Voltage Regulator fluctuations, 2% for resistances from Voltage Regulator to the IC bumps and then have 3% for drop on the power grid inside the IC.

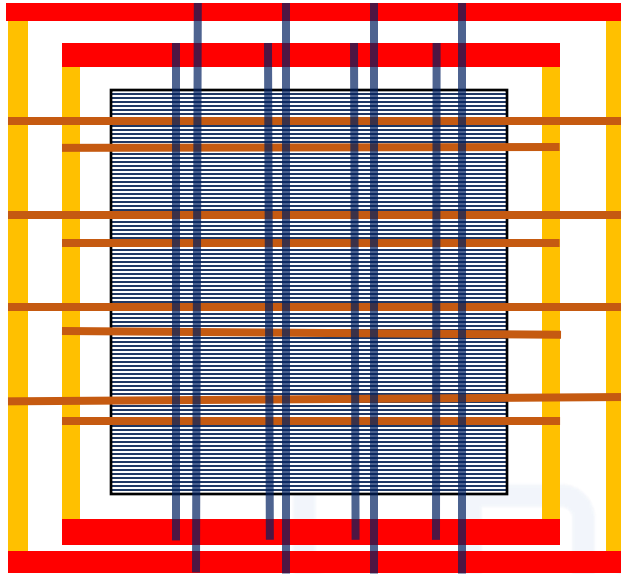
- **IR Drop** is the voltage drop in the metal wires constituting the power grid before it reaches the power/gnd pins of cells.
- On die IR-drop target: less than 3%
- IR drop = IR drop in the **VDD grid** + IR drop in the **VSS grid**

- Electro-migration is the movement of atoms based on the flow of current through a material.
- If the current density is high enough, the heat dissipated will break atoms from the structure and move them.
- Will create both ‘vacancies’ and ‘deposits’ leading to Shorts, Opens.



- Higher current density in a metal may lead to EM.
- For a Technology node, Maximum current density (EM Limit) for Each metal layer is provided by Foundry.
- In Power planning width of the metal layer is decided based on the EM Limit.
- Increasing the metal width will help in avoiding EM Violation.

F=1GHz



Low Frequency Design

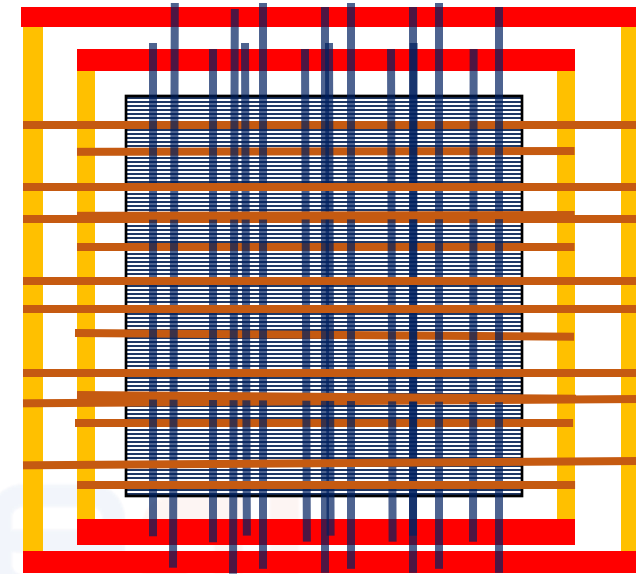
Less Power -> $I \downarrow$ -> IR Drop $\downarrow$

Lesser Power Stripes

Low congestion

Routing will be Easy

F=2GHz



High Frequency Design

More Power -> $I \uparrow$ -> IR Drop $\uparrow$

More Power strips

High congestion

Routing will be Difficult

Power Planning Objectives



Power Consumption in IC



Power Planning



Low Power Techniques

This section explains about Low power and why it is required, followed by various low power techniques.

✓ Clock Gating



✓ Power Gating



✓ Multi VDD and DVFS



- 1) **Battery:** Mobile devices, no dedicated current supply through mains.
Only battery.
- 2) **Power costs money:** Cost of providing power, lower power saves money.
- 3) **Power generates heat, which needs to be dissipated:** Technical Challenges and cost for heat dissipation.

- Heat dissipation is important, as higher temperatures lead to Die-Functionality failures.
- **Technical challenges:** Concern on heat dissipation in all places: server farms, desktops, laptops, mobile devices. Complex mechanisms in server farms, water-cooled solutions in desktops, heat-sinks typically used in laptops and mobile devices.
- **Heat sinks:** Large power dissipation requires larger heat sinks which needs more space, which is difficult in mobile devices.

Physical Design	Design/ Architectural	Process Technology
Multi Vt	Pipelining	Multi Vt
Clock Gating	Asynchronous	PD SoI FD SoI
Power Gating	Multi VDD	Finfet
Multi channel length	DVFS *	Body bias

- DVFS - Dynamic Voltage and Frequency Scaling
- PD SOI - **Partially Depleted** Silicon On Insulator
- FD SOI - **Fully Depleted** Silicon On Insulator
- Multi VT - It Uses Different VT in the circuit to reduce leakage but until it satisfies timing constraints

Dynamic power	Leakage power
Clock gating	Multi Vt
Multi VDD	Multi channel length
DVFS	Power gating
	Process tech improvements

- Multi channel length-uses Gates with different channel lengths in the **same VT class**. **Longer channel length** leads to **lesser leakage power**.

Power Planning Goals



Power Consumption in IC



Power Planning



Low Power Techniques



✓ Clock Gating

✓ Power Gating

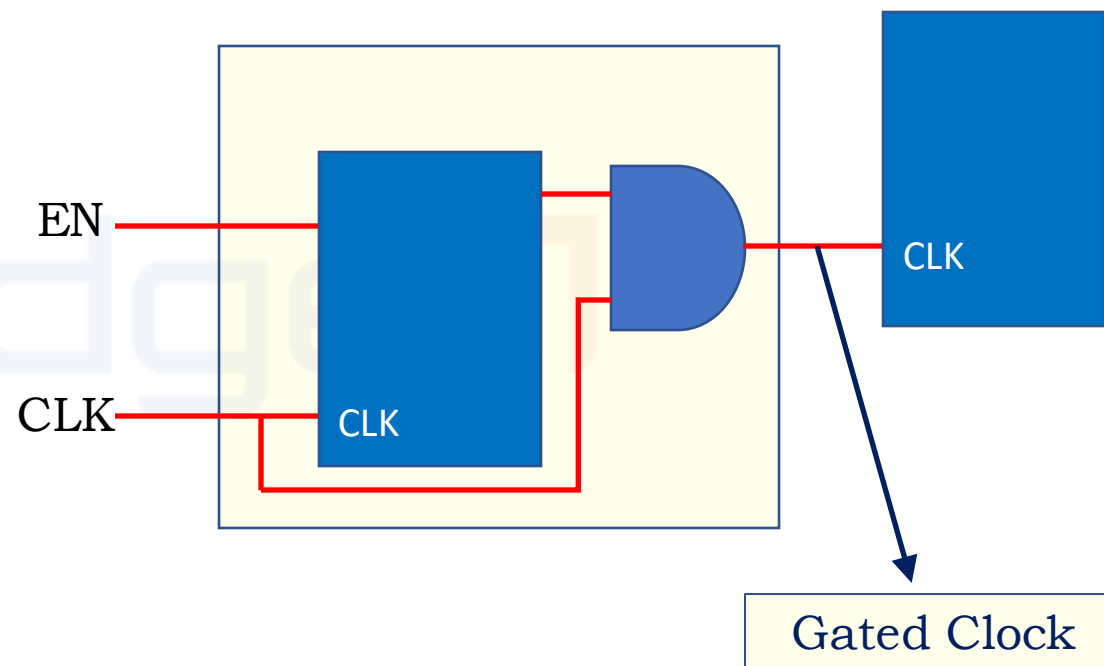


✓ Multi VDD and DVFS



It explains the reduction technique for dynamic power by controlling switching activities on clock signals.

- Clock gating is a technique to turn off the clock to certain parts of the digital design when not required.
- Clock consume more than 30% of dynamic power, as clock buffers toggle with every clock edge



- In most designs, data is loaded into registers very infrequently, but the clock signal continues to toggle at every clock cycle. Often, the clock signal drives a large capacitive load, making these signals a major source of dynamic power dissipation.
- Clock gating reduces Dynamic power dissipation as the Gated clock circuitry will not Switch.
- Enable conditions for clock gating is coded in RTL.

Power Planning Goals



Power Consumption in IC



Power Planning



Low Power Techniques



✓ Clock Gating



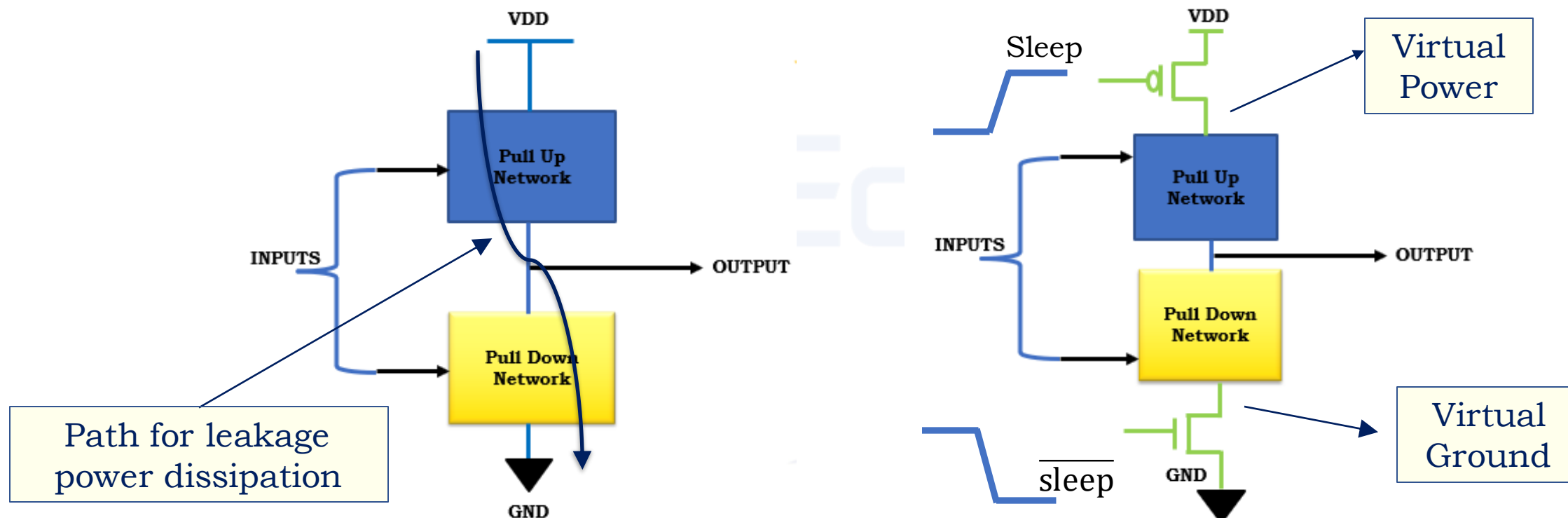
✓ Power Gating

✓ Multi VDD and DVFS



This explains the Leakage power reduction techniques by gating the power supply using power switches.

- Blocks that are not in use, Power supply is temporarily turned off



Power Planning Goals



Power Consumption in IC



Power Planning



Low Power Techniques



✓ Clock Gating



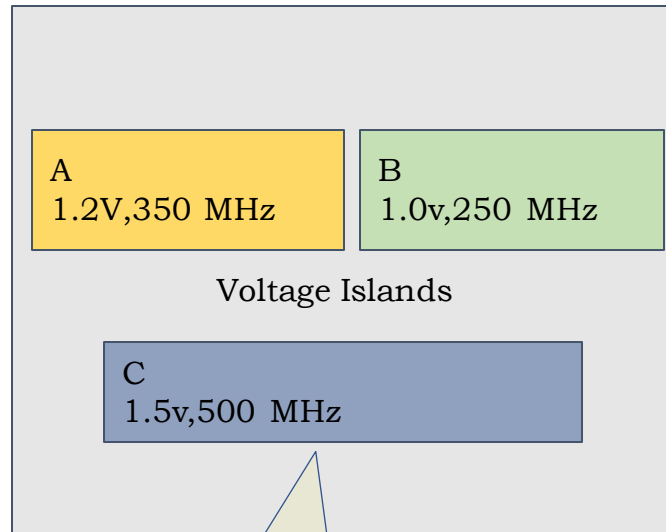
✓ Power Gating



✓ Multi VDD and DVFS

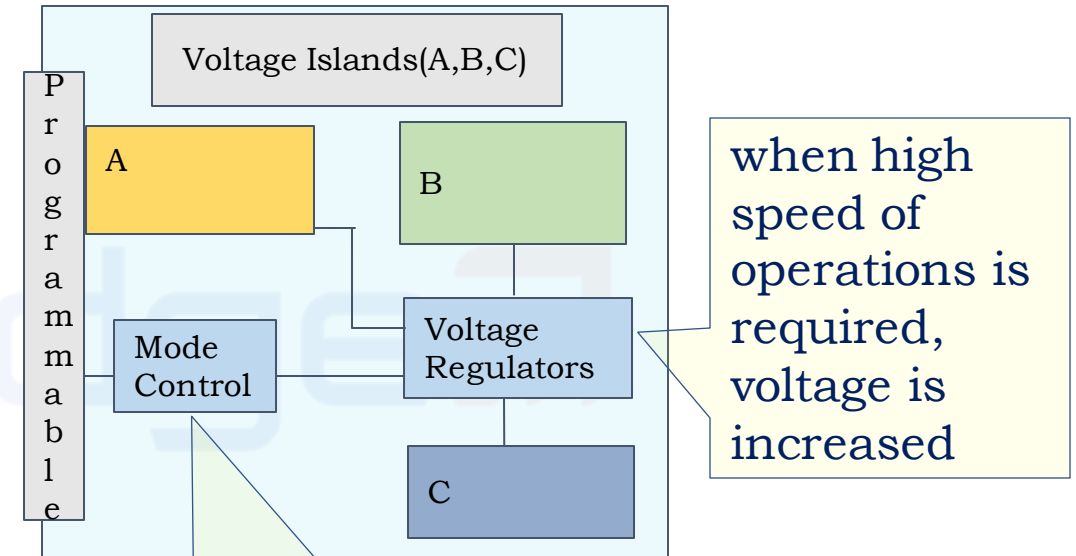
This explains power reduction techniques using multi VDD and Dynamic voltage frequency scaling.

Static Voltage Scaling



Different but fixed voltage is applied to different blocks or subsystems of the SoC design

Dynamic Voltage and Frequency scaling



Voltage as well as frequency is dynamically varied as per the different working modes of the design

- Power Consumption in IC with example.
- Power Planning – Various voltages
- Power Planning – Power Delivery & IR Drop
- Electro-Migration (EM)
- Low Power Techniques
- Clock Gating
- Power Gating
- Multi VDD & DVFS

Module 8

Static Timing Analysis -1

- In this module you will learn about introduction to Static Timing Analysis and other topics related to STA which includes Setup time, Hold time and Timing paths.
- Setup & Hold time Analysis and Timing Reports can be understood in detail with some examples.
- In addition to these, Clock skew can also be learnt with respect to the Static Timing Analysis.

- Introduction to STA-1
 - ✓ What Is Timing Analysis?
 - ✓ Static Timing Analysis Flow
 - ✓ STA in RTL to GDSII Flow
 - ✓ Terminologies Used in STA
- Setup
 - ✓ Setup Timing Diagram
- Hold
 - ✓ Hold Timing Diagram
- Timing Checks – Setup & Hold
 - ✓ Sequential Cells - Timing Arcs
 - ✓ Data Flow in a Chip
 - ✓ Timing Checks – Setup
 - ✓ Timing Checks - Hold
- Timing reports
 - ✓ Schematic View of Timing Path

- ✓ Reg-to-Out & In-to-Reg
- ✓ In-to-Reg timing Path
- ✓ Reg-to-Out Timing Path
- Clock Skew
 - ✓ Calculation Of Setup Slack with Skew (Reg to Reg)
 - ✓ Calculation Of Hold Slack with Skew (Reg to Reg)
- ✓ Positive Skew vs Negative Skew
- ✓ STA Table – Factors Affecting Setup & Hold
- ✓ DRV (Design Rule Violations - Logic DRC)
- Additional Information

Introduction to STA-1

Setup



This section explains about of what is static timing analysis, static timing analysis flow, timing paths and terminologies used in the STA

Hold



Timing Checks – Setup & Hold



Timing reports



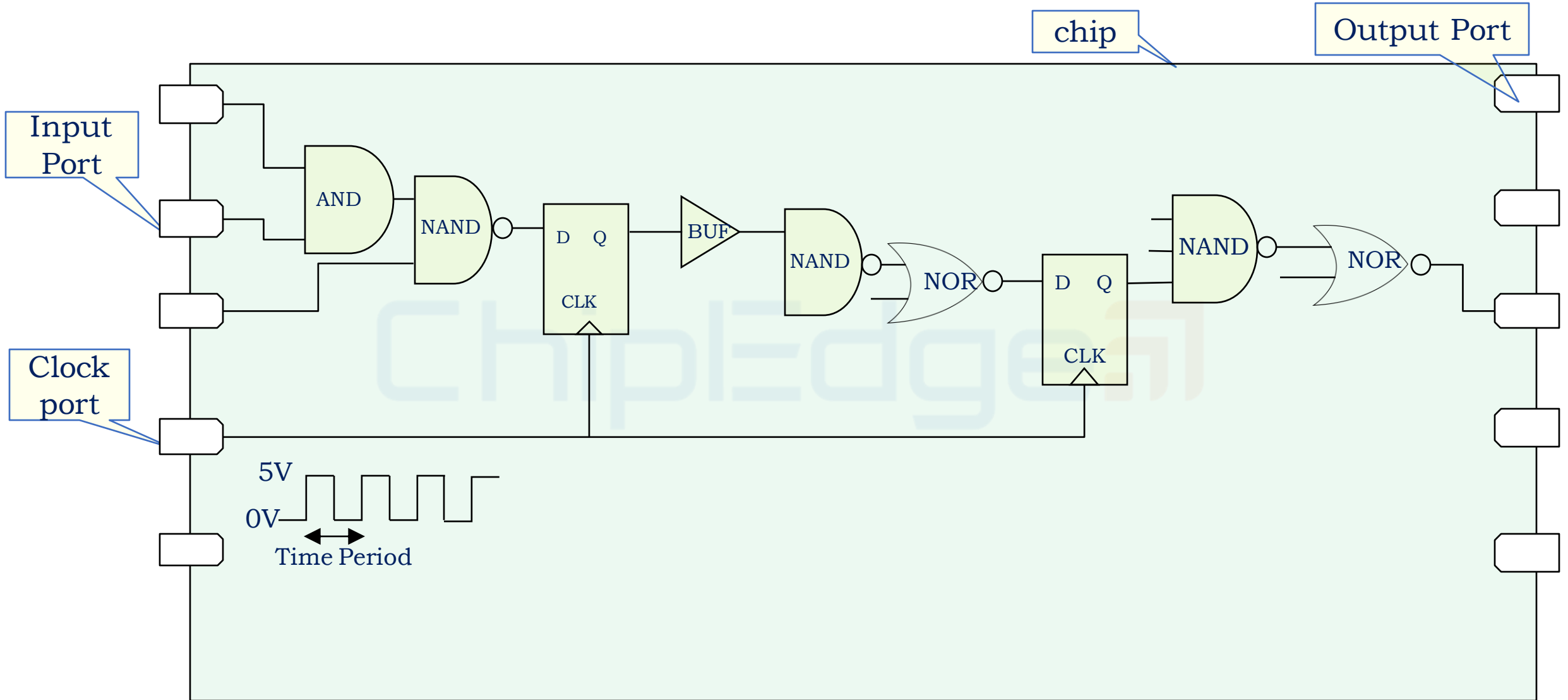
Clock Skew



Additional Information

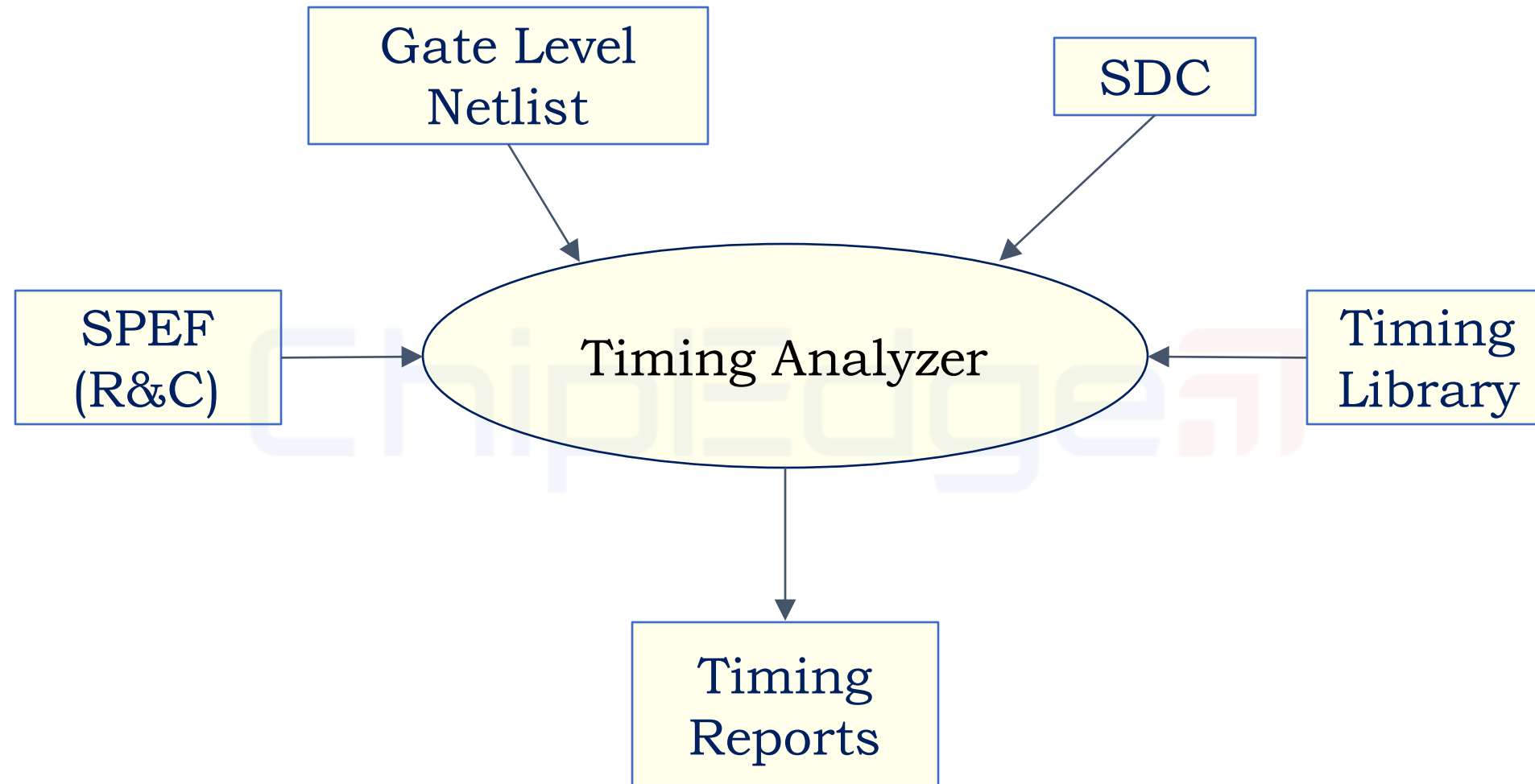


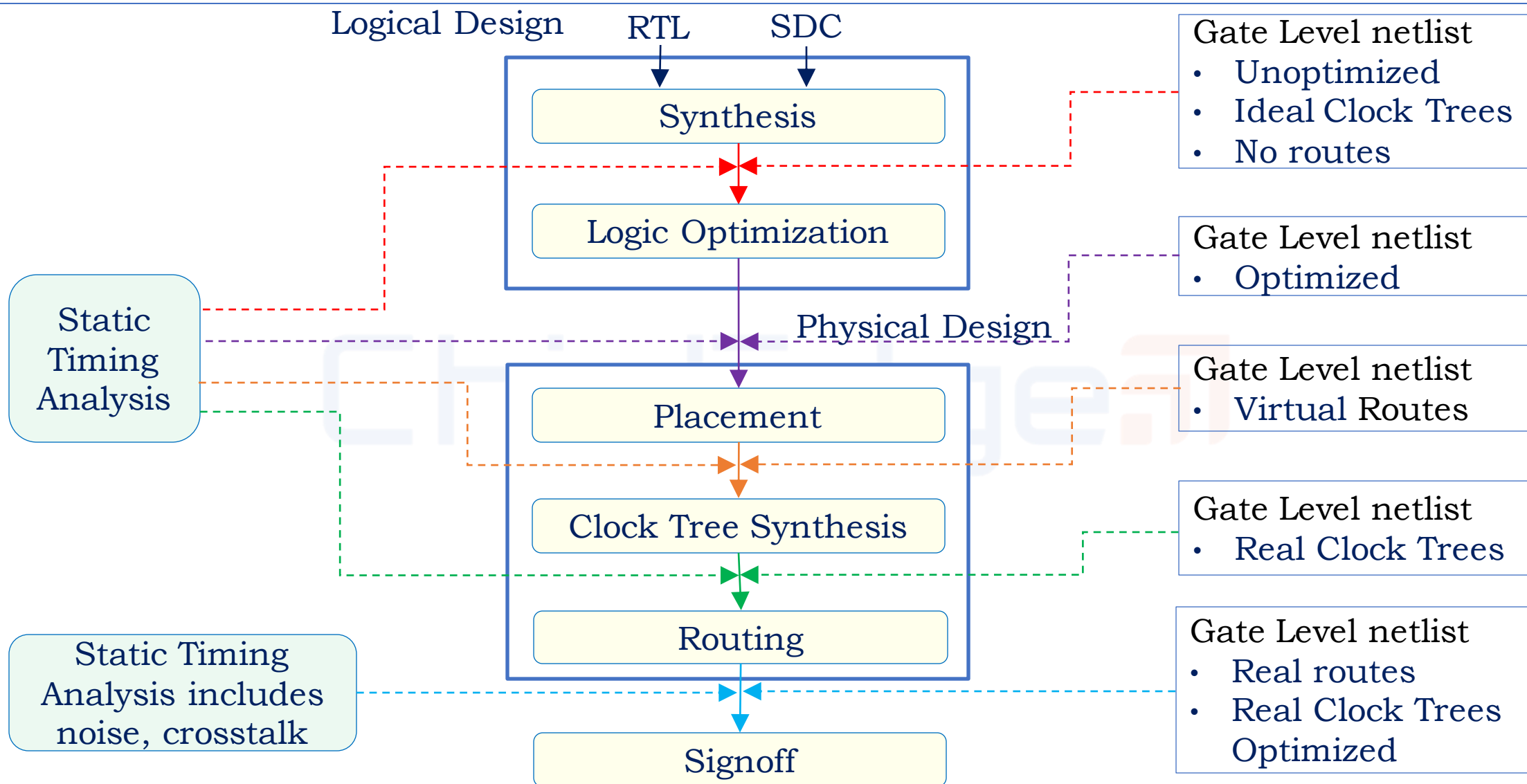
What Is Timing Analysis?

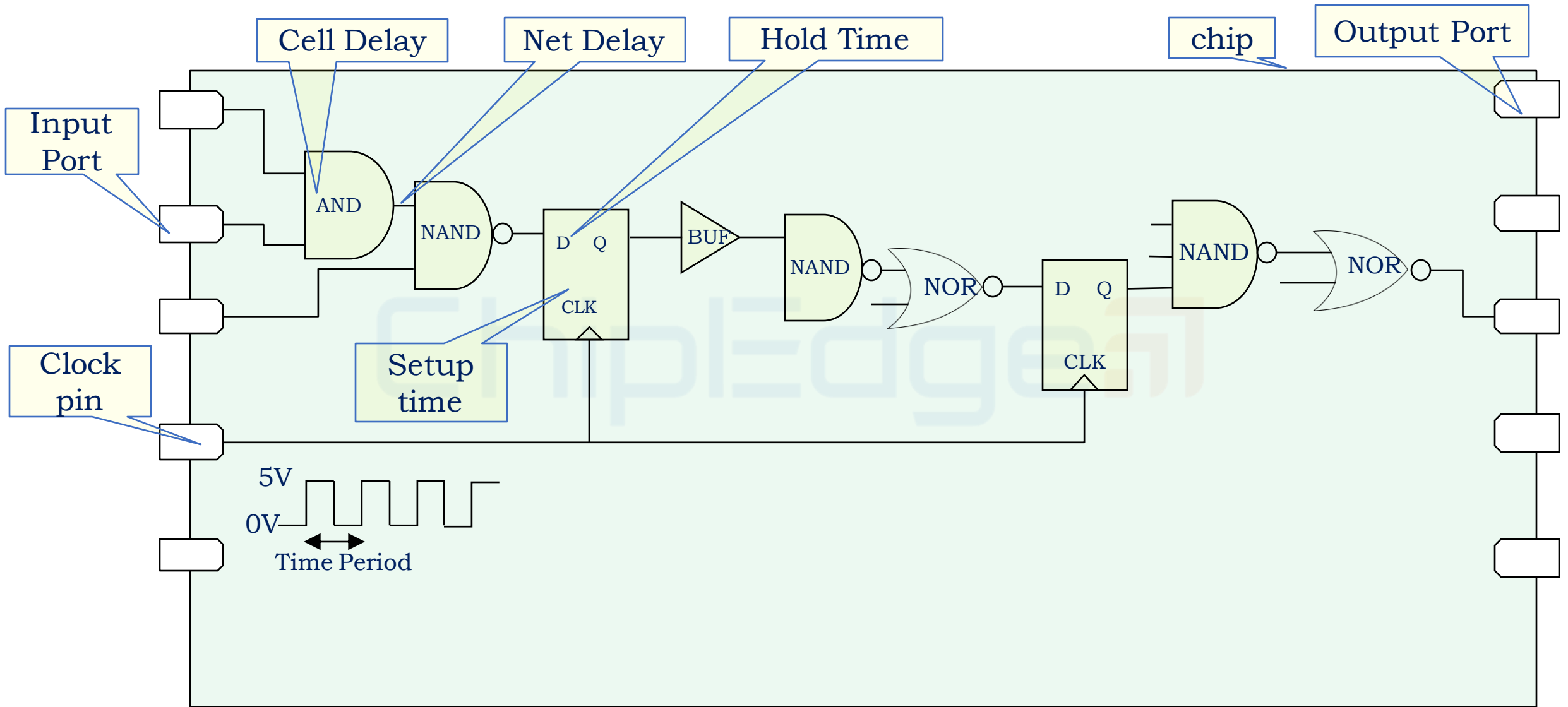


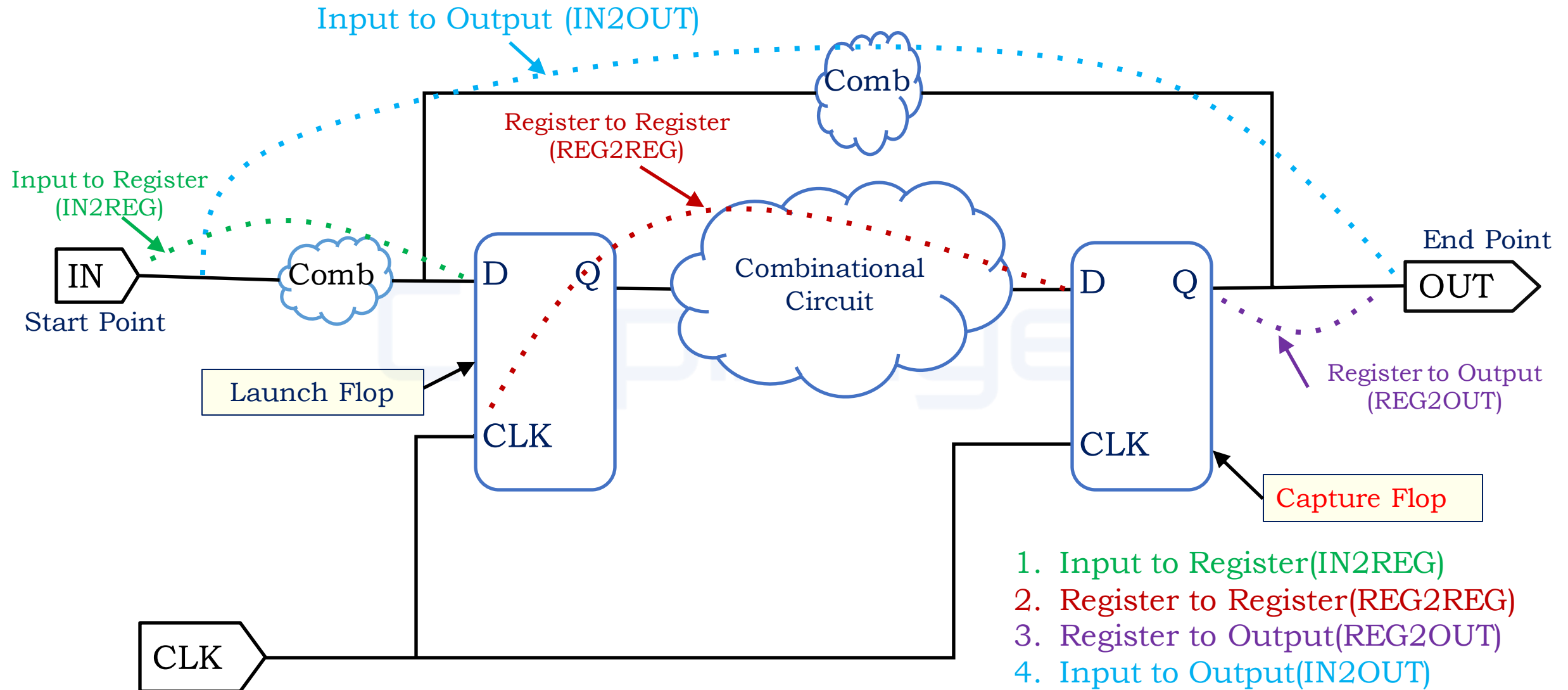
- Static Timing Analysis (STA) is a method for determining if the design meets timing constraints without having to simulate
- STA is used for digital designs.
- Designs are so big that simulations will not work. (simulations require more Runtime)

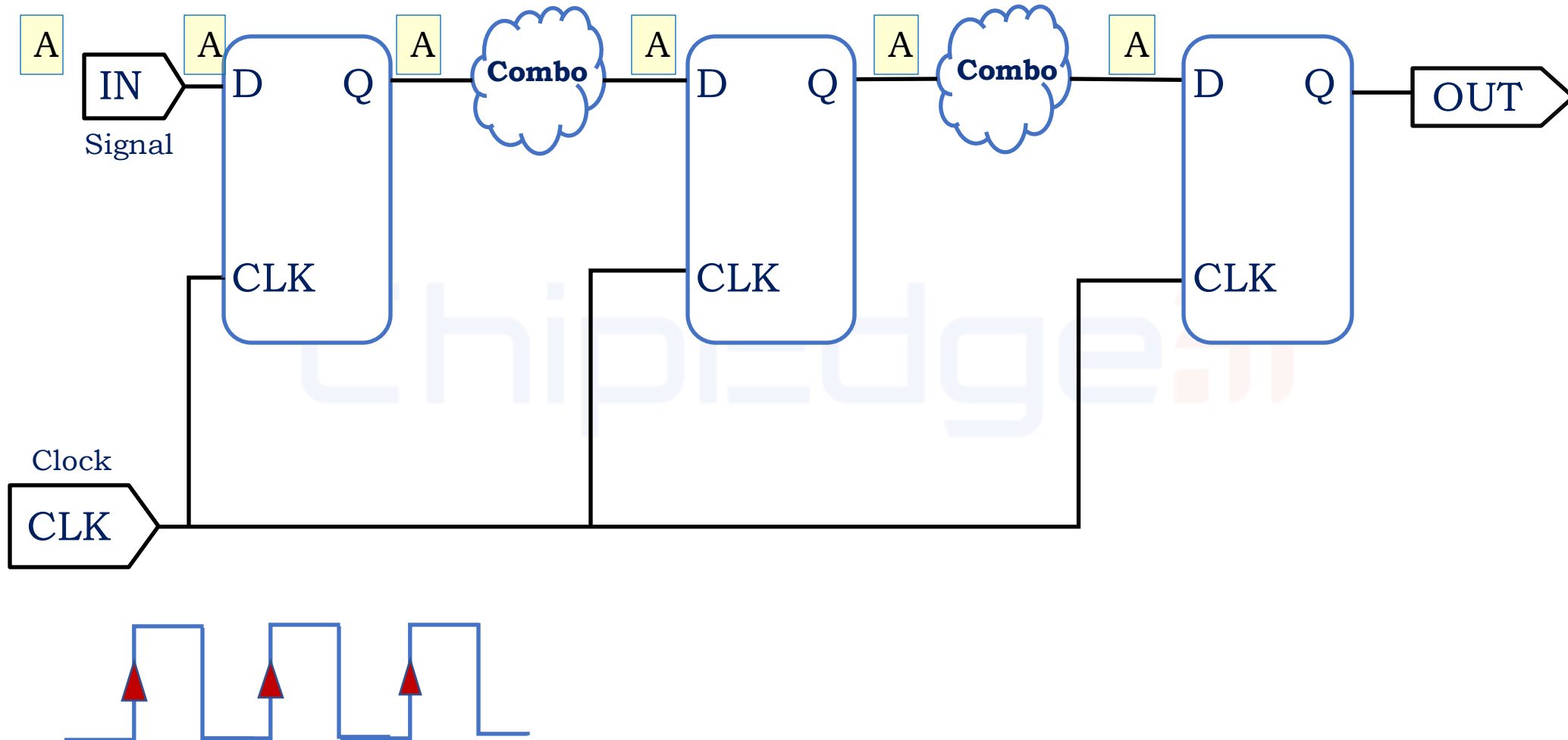
- Static Timing Analysis is a complete and exhaustive verification of all timing checks of a design.
- It is much faster than timing-driven, gate-level simulation
- Proper circuit functionality is not checked
- All timing paths are considered for the timing analysis. This is not the case in simulation.



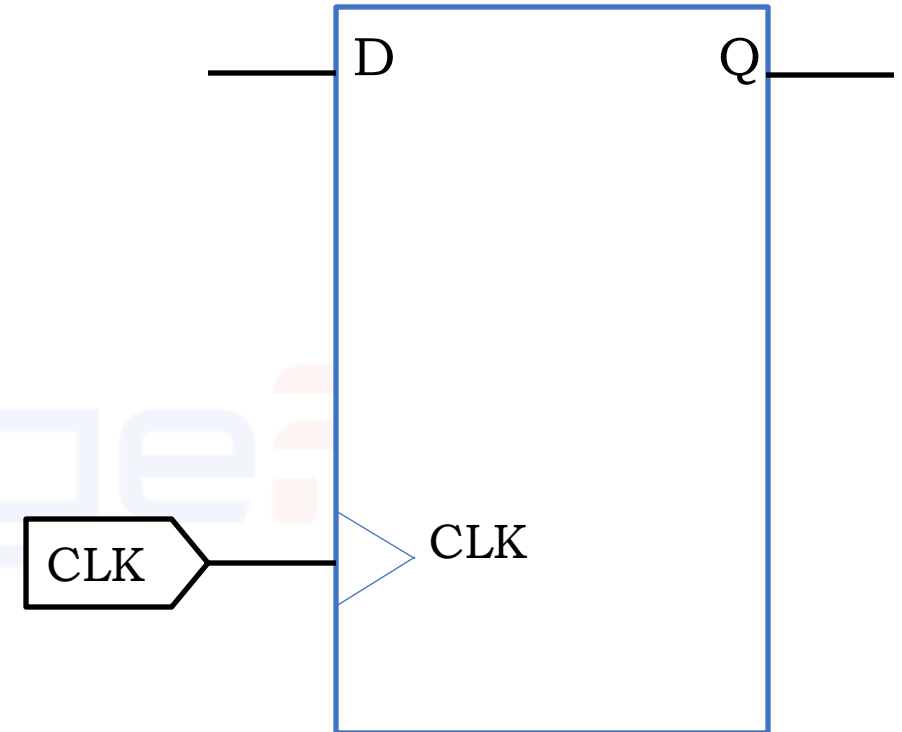


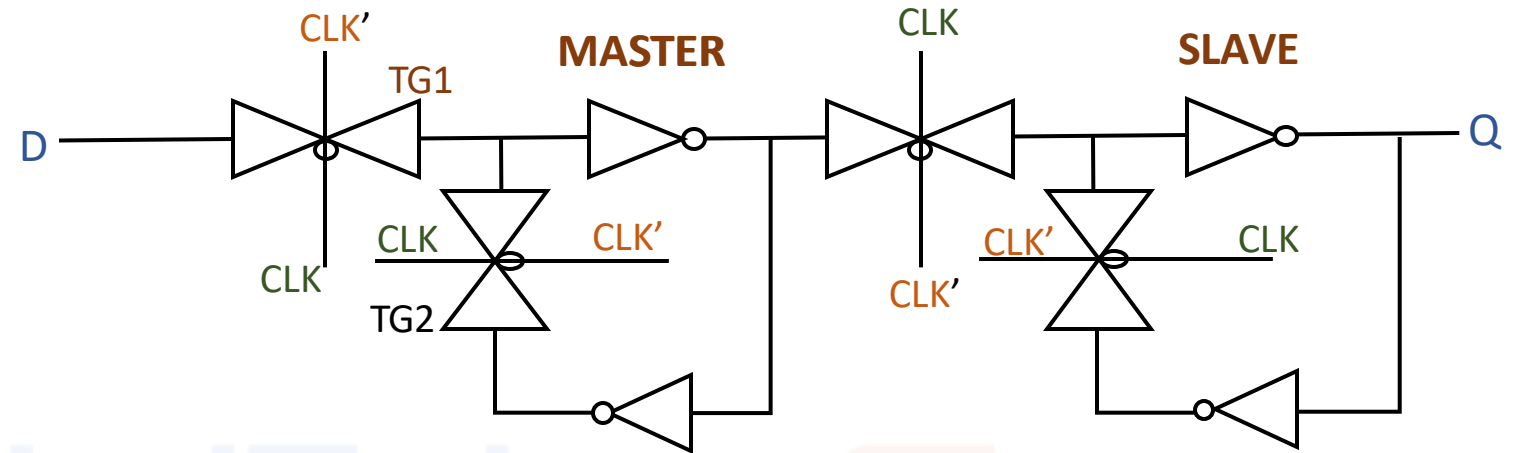
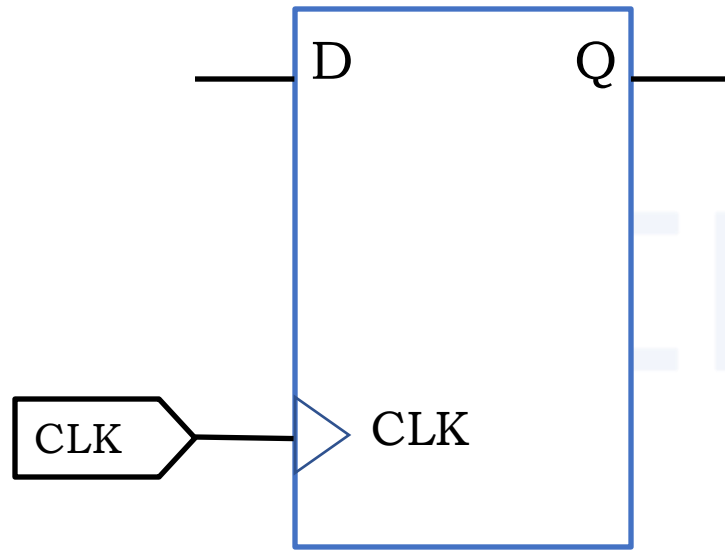






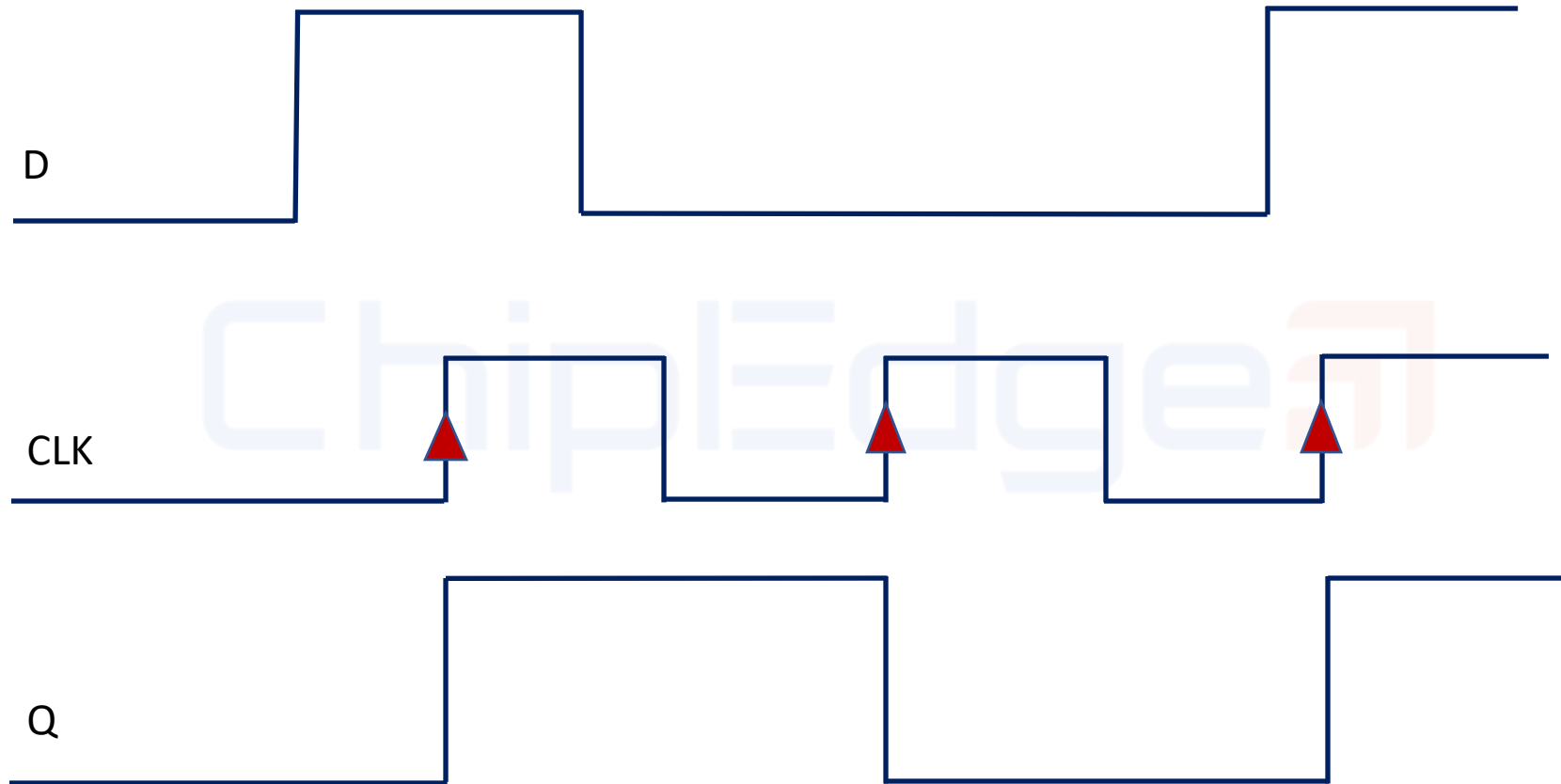
- D flip-flop is also called a **delay flipflop**. D flip-flop has one data input that is 'D' and one clock input 'CLK' and an output Q.
- When an input data is provided at D pin, the data at D is transferred to Q only at the rising edge of the clock. It holds the data at Q until next clock edge.





Truth Table

D	CLK	Q
X	0	Q
X	0	Q
0		0
1		1
X	1	Q
X	1	Q



Introduction to STA-1



Setup

In this section the definition of setup with timing diagrams are explained in detail.

Hold



Timing Checks – Setup & Hold



Timing reports



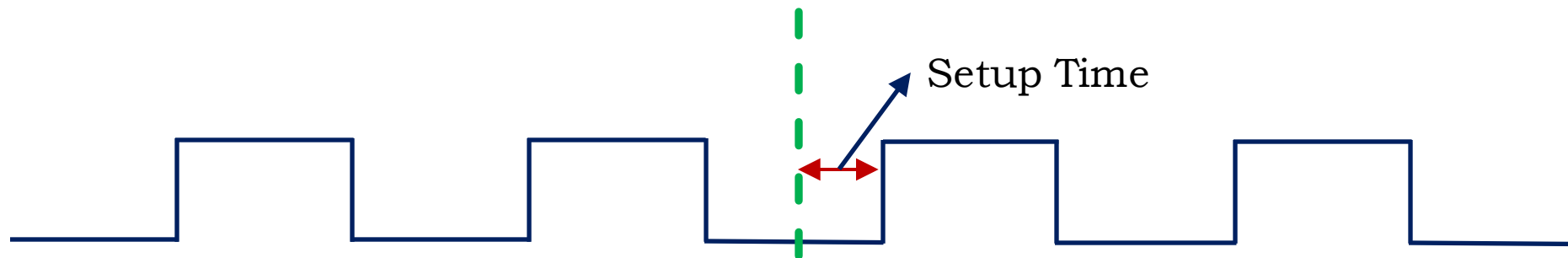
Clock Skew

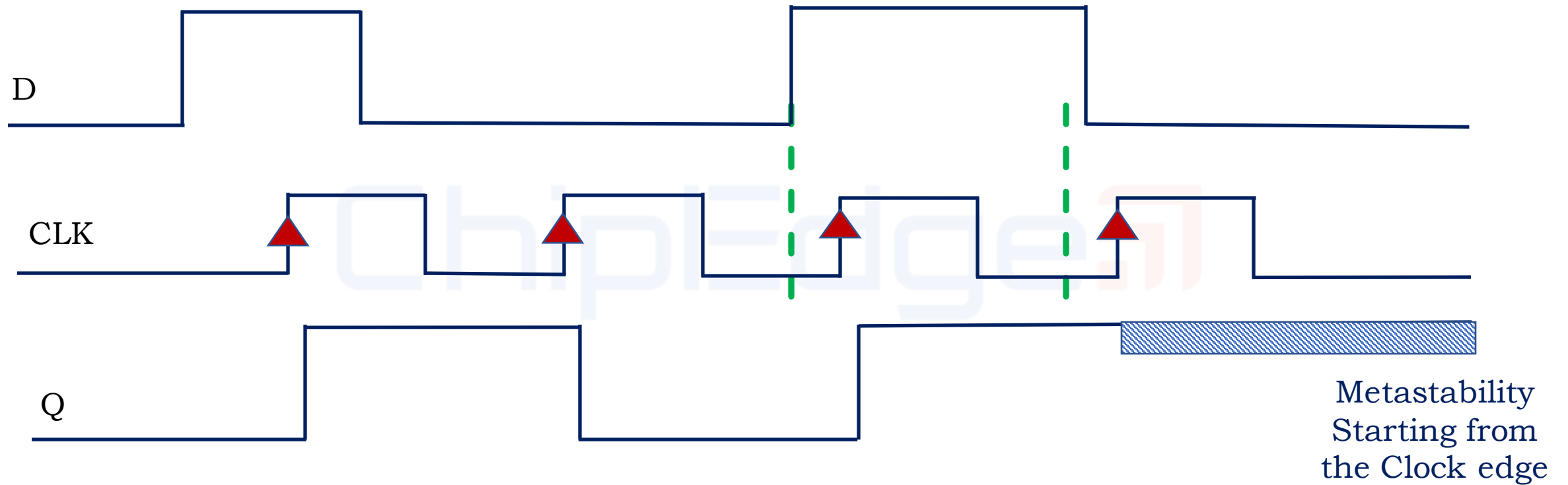


Additional Information



- Every flop in the library has certain **setup time** and **hold time** defined in `.lib`
- Setup time is the amount of time the data at the synchronous input (D) must be **stable** before the active edge of clock
- If the **data** reaches the **D input** **before** the defined **setup time** of the flop, then only it will be **captured** properly.





Introduction to STA-1



Setup



Hold

In this section the definition of hold with timing diagrams are explained in detail.

Timing Checks – Setup & Hold



Timing reports



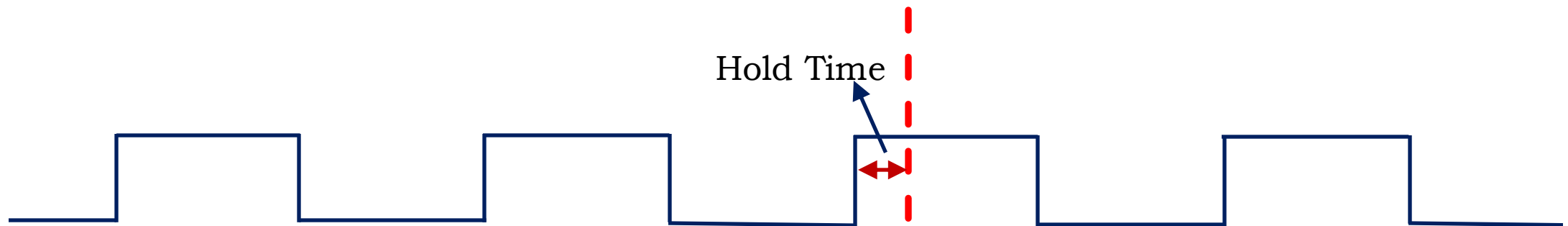
Clock Skew

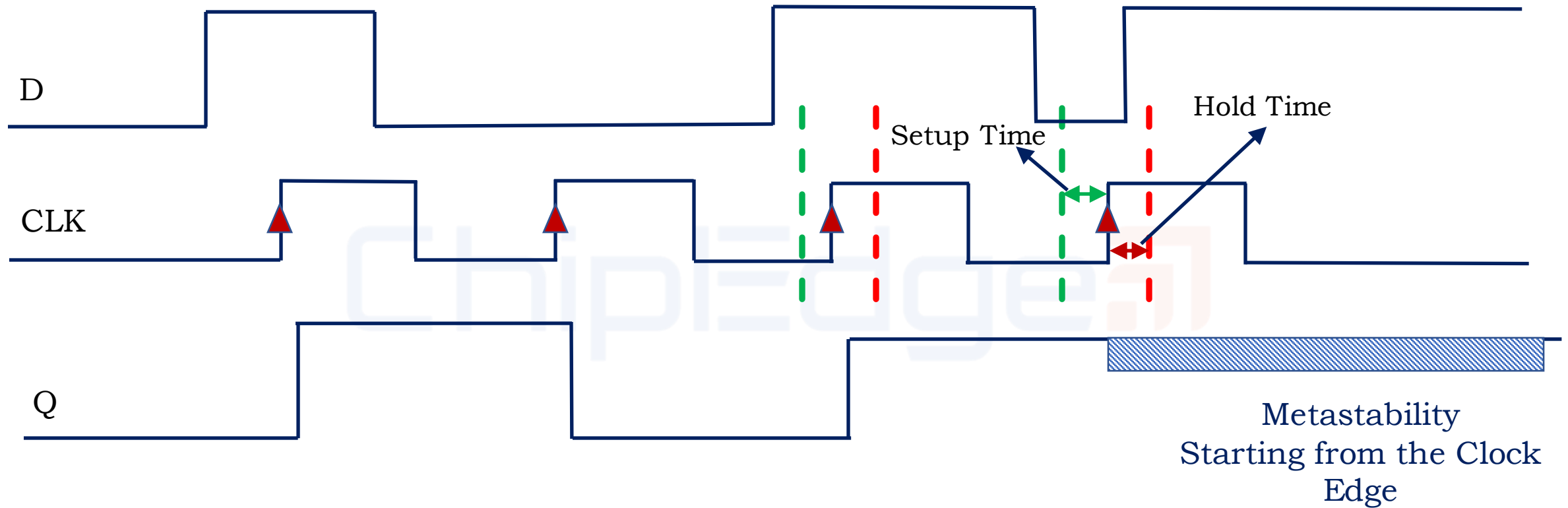


Additional Information



- Hold is the amount of the time the **synchronous input (D)** should stay stable **after** the **clock edge**, so that the data can be captured properly.
- The **data** which reached at **D input** should stay stable after the clock edge for at least the **time equal to the defined hold time** of a flop. Only then the data can be captured properly.





Introduction to STA-1



Setup



Hold



Timing Checks – Setup & Hold

The description of timing arcs, setup and hold slack calculations are explained and in detail with examples.

Timing reports



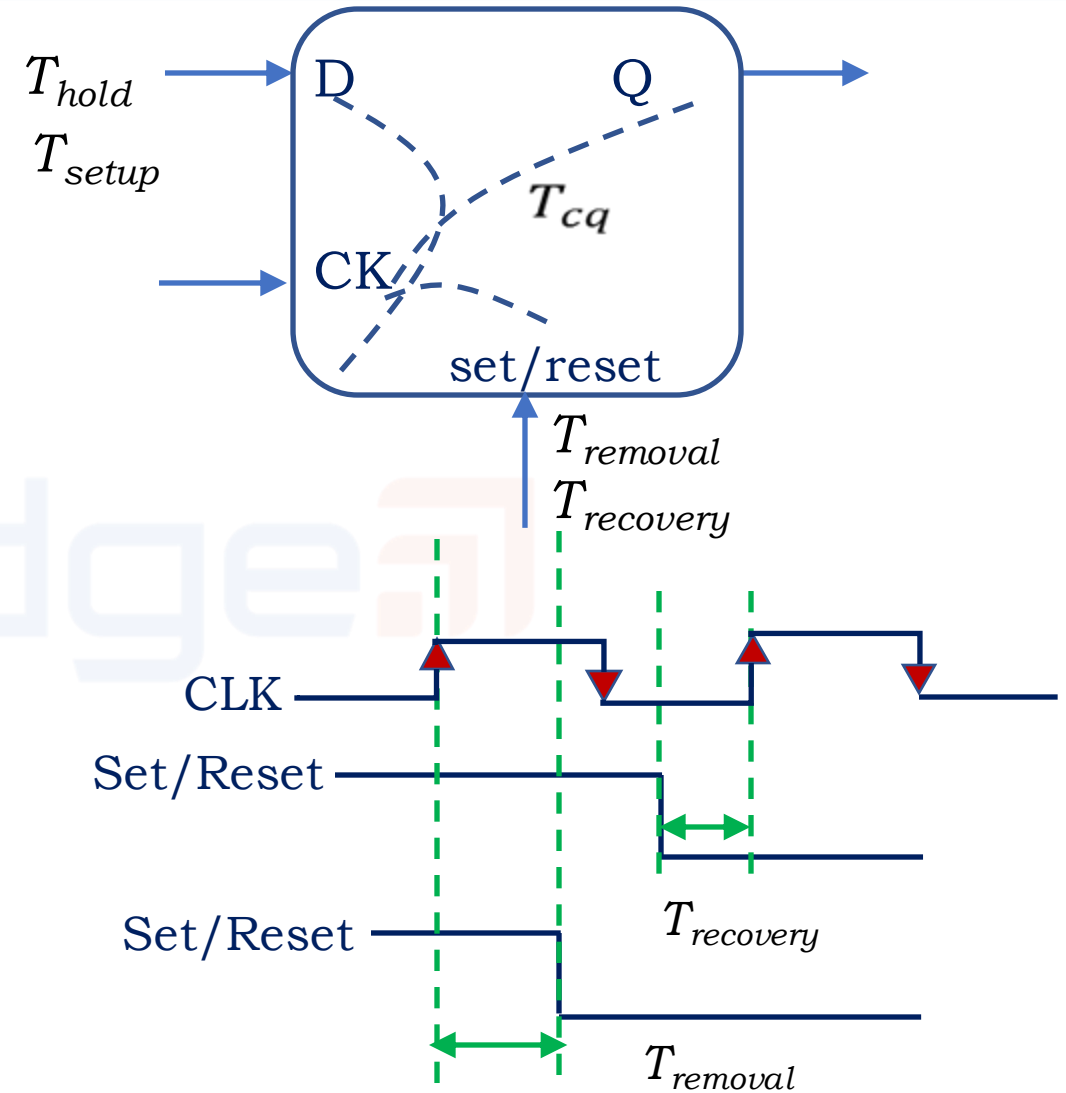
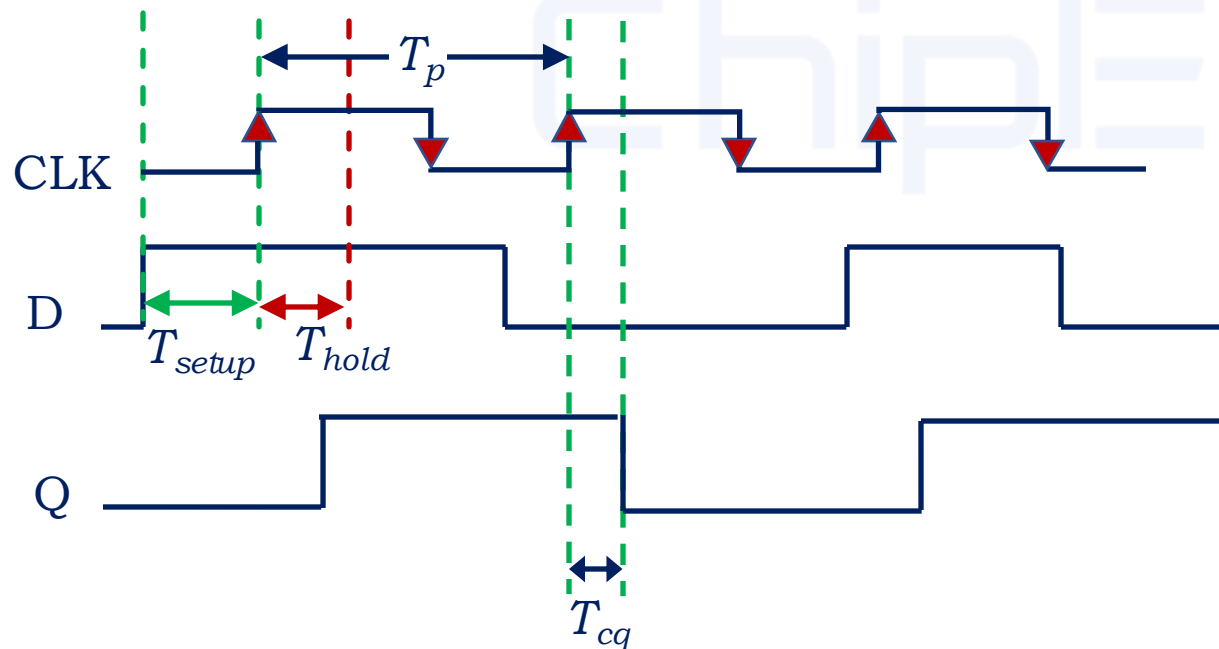
Clock Skew

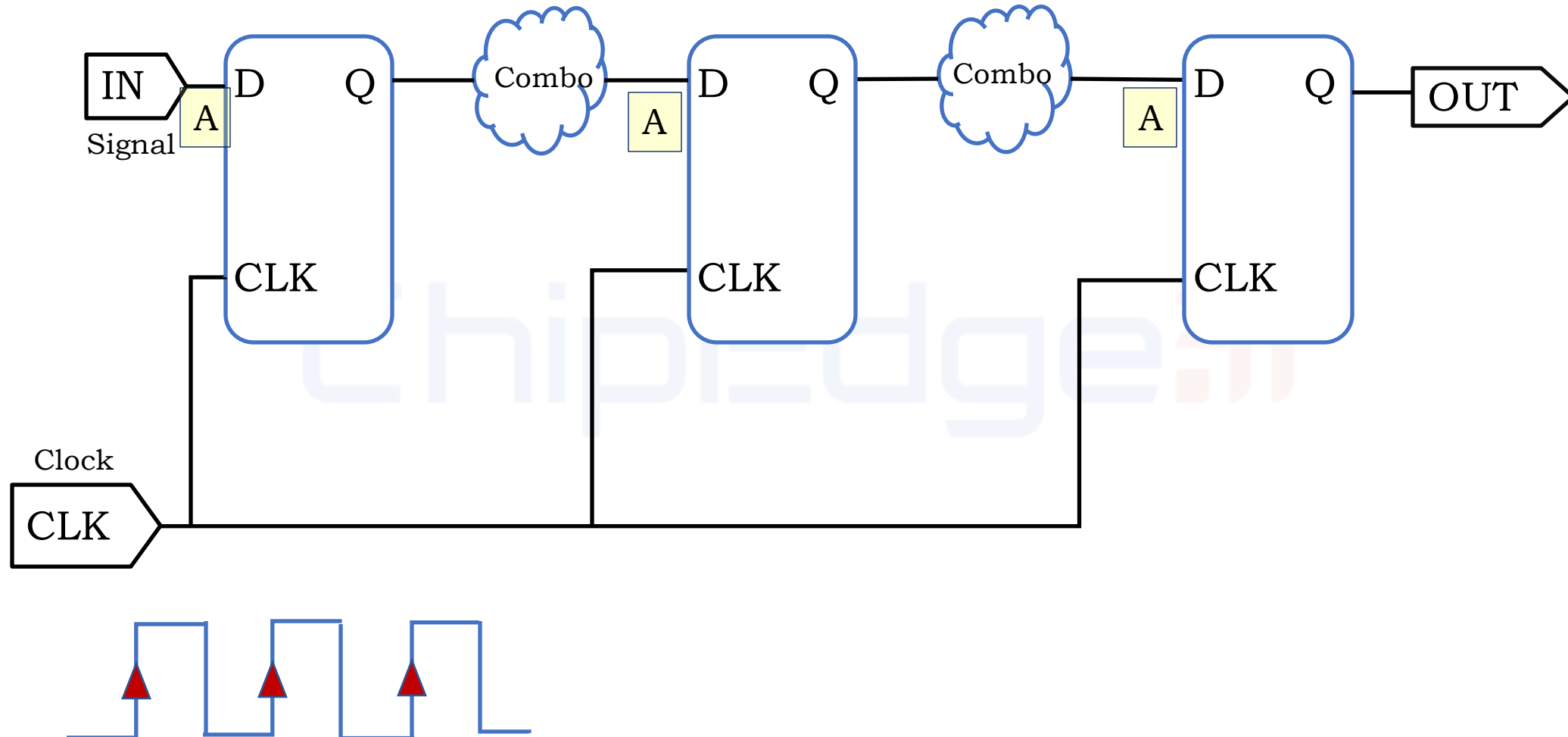


Additional Information

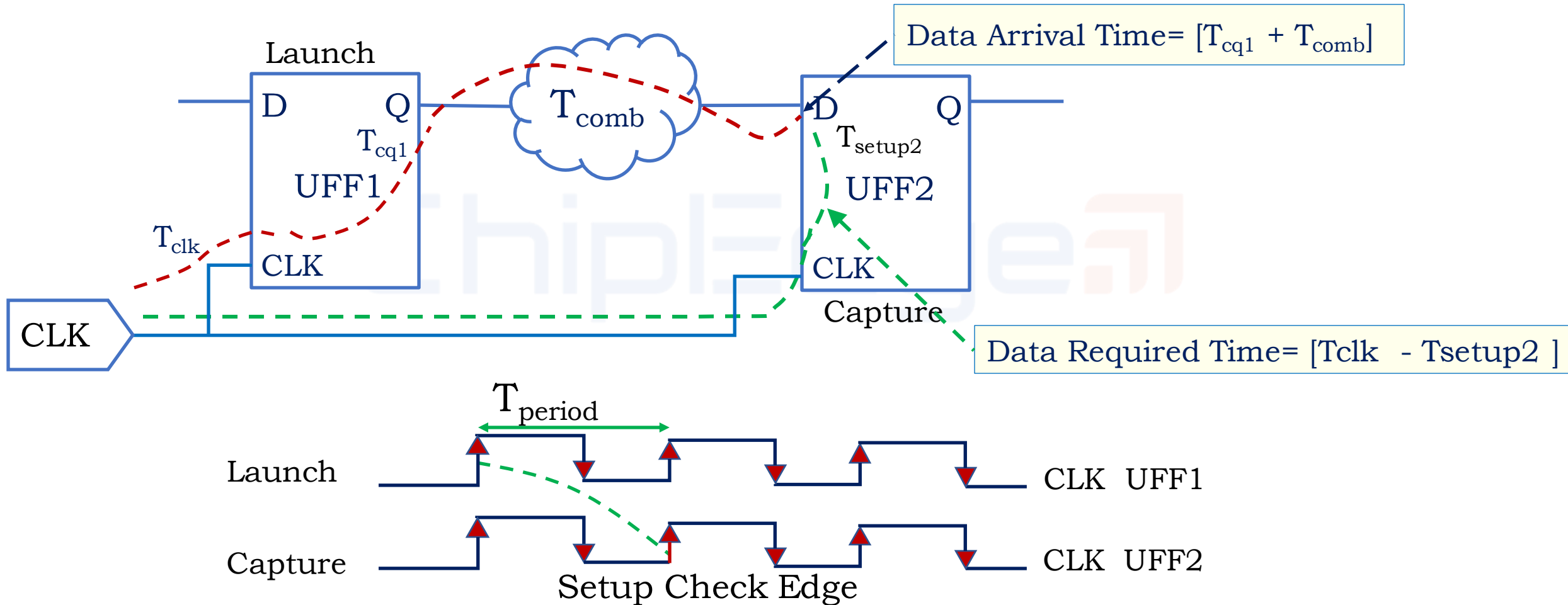


- Cell Delay (T_{cq})
- Setup and hold
- Recovery and Removal

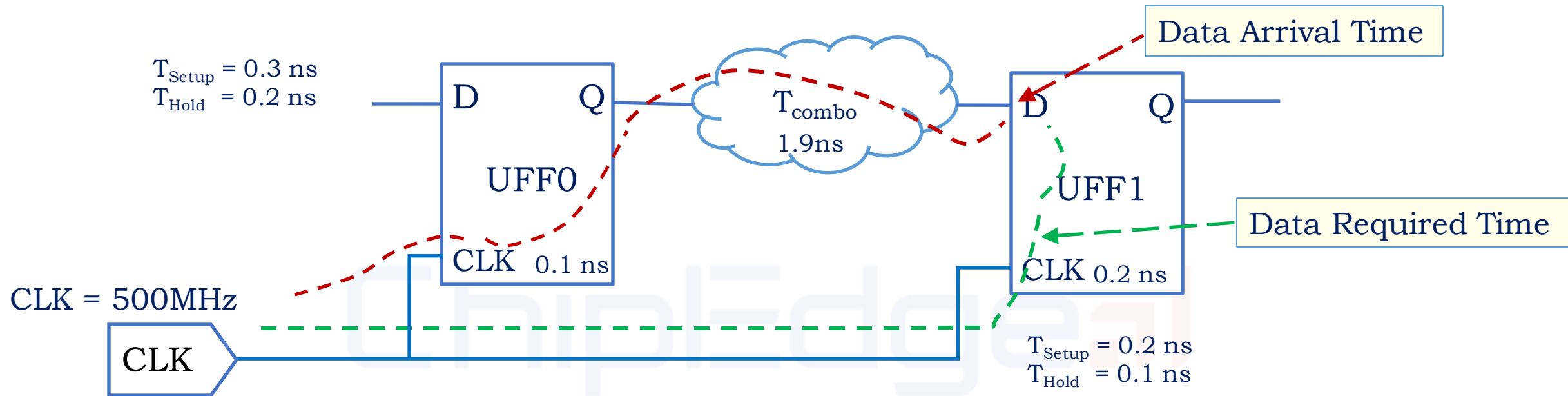




For Setup Check to meet $T_{cq1} + T_{comb} \leq T_{clk} - T_{setup2}$



Calculation Of Setup Slack (Reg to Reg)



Data Arrival Time: It is the actual time at which the data is reaching at the D input of the capture flop.

Data Required Time: This is the time before which the data should reach the D input of the capture flop so that it pass the setup timing check.

Condition for setup check -- Met

Date Arrival Time < Data Required time

$$T_{cq1} + T_{comb} \leq T_{clk} - T_{setup2}$$

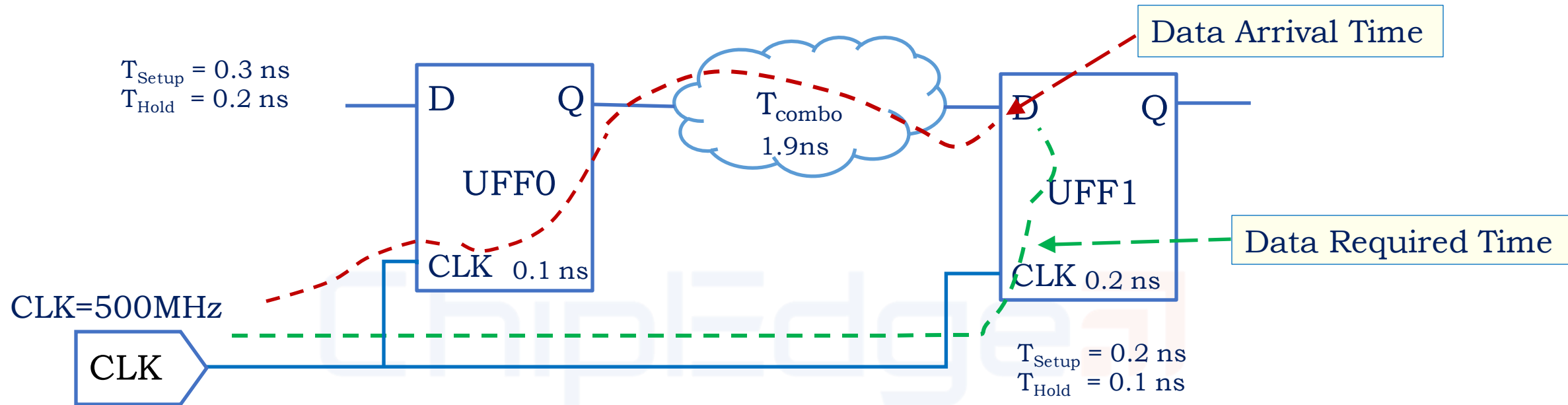
Setup Slack = Data Required time – Data Arrival time

$$\begin{aligned} &= [T_{clk} - T_{setup2}] - [T_{cq1} + T_{comb}] \geq 0 \\ &= ?? \end{aligned}$$

If setup slack $\geq 0 \rightarrow$ setup is **MET**

If setup slack $< 0 \rightarrow$ setup is **VIOLATED**

WNS (worst negative slack) for setup is the worst of all the negative setup slacks in the design.



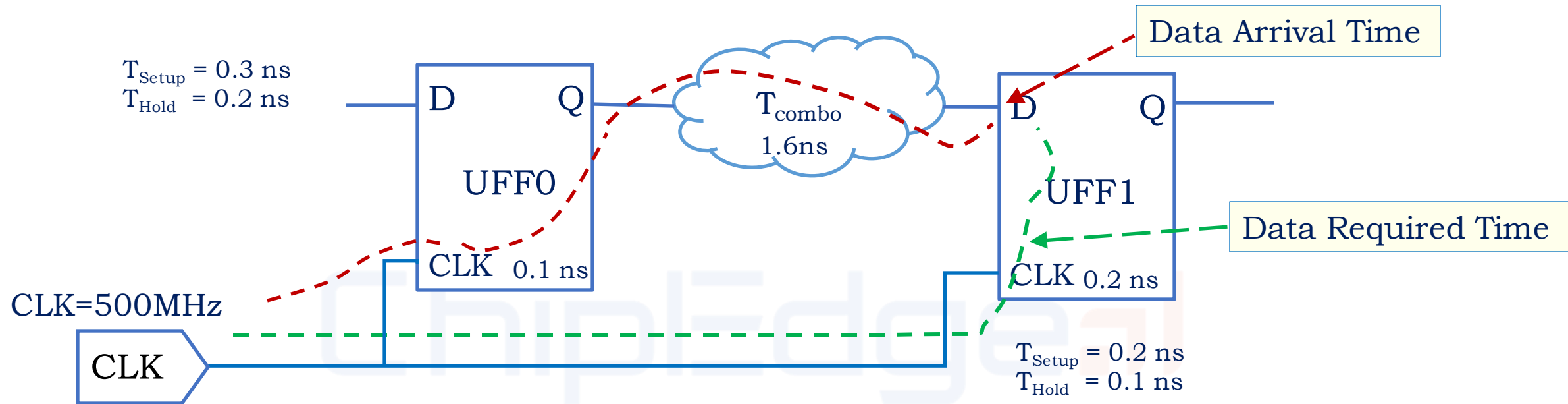
$$\text{Setup Slack} = \text{RAT} - \text{AT}$$

$$= [T_{clk} - T_{setup2}] - [T_{cq1} + T_{comb}]$$

$$= [2 - 0.2] - [0.1 + 1.9] = -0.2$$

Setup timing check is failing as the slack is negative (-0.2)

Calculation Of Setup Slack Met (Reg to Reg)

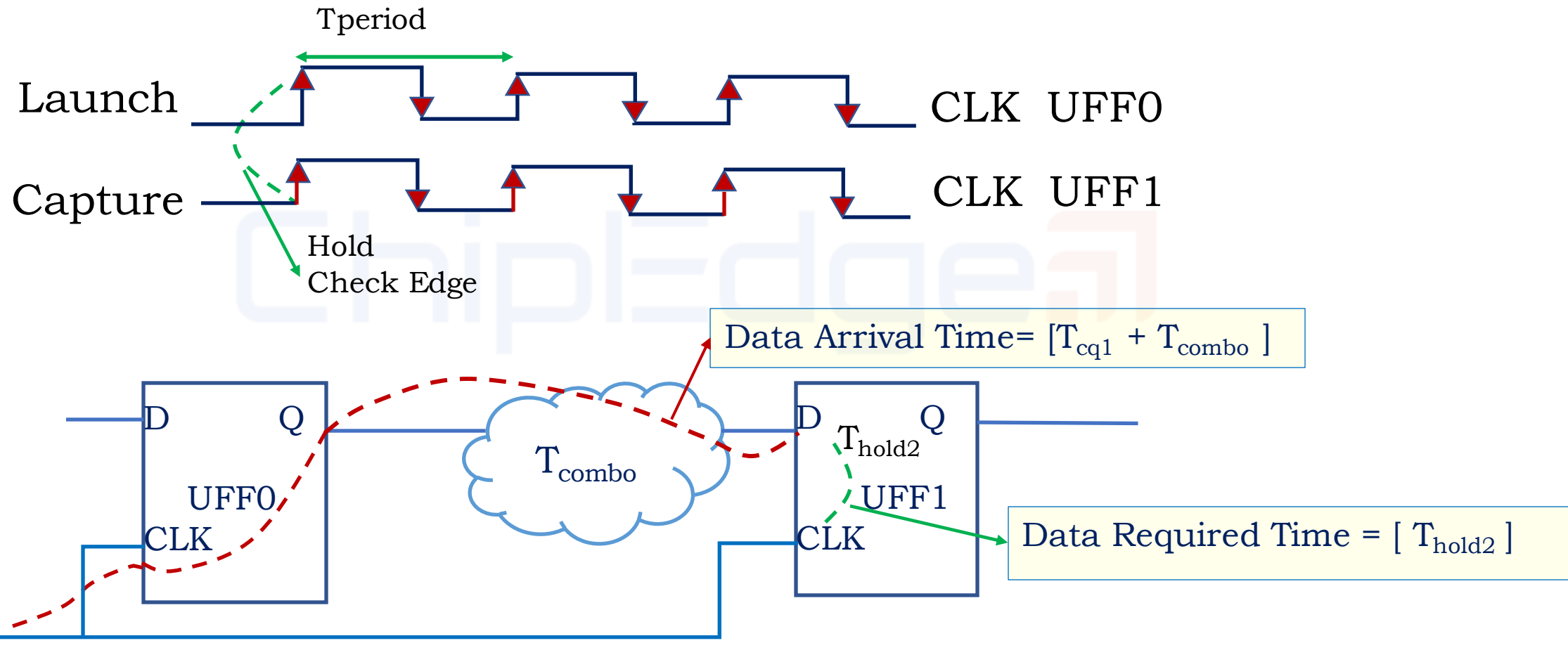


$$\text{Setup Slack} = \text{RAT} - \text{AT}$$

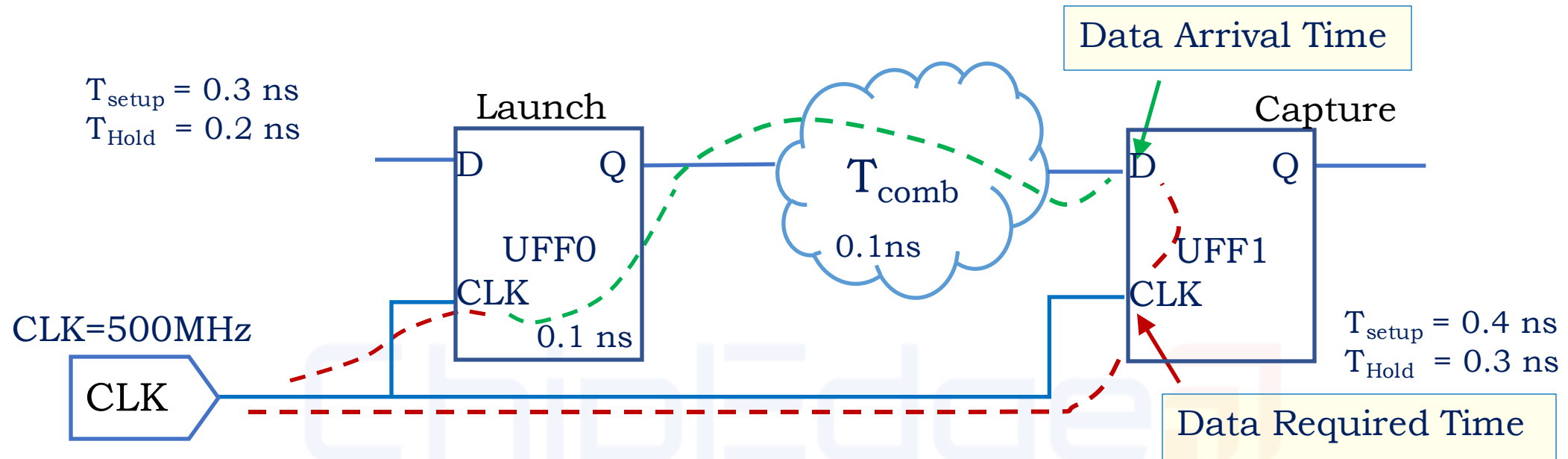
$$= [T_{clk} - T_{setup2}] - [T_{cq1} + T_{comb}]$$
$$= [2 - 0.2] - [0.1 + 1.6] = 0.1$$

Setup timing check is passing as the slack is positive (+0.1)

For hold check to meet $\rightarrow T_{cq1} + T_{combo} \geq T_{hold2}$



Calculation Of Hold Slack (Reg to Reg)



Data Arrival Time: It is the actual time at which the data is reaching at the D input of the capture flop.

Data Required Time: This is the time after which the data should reach the D input of the capture flop so that it pass the hold timing check.

Condition for Hold check -- Met

$$T_{cq1} + T_{comb} \geq T_{hd2}$$

Data Arrival Time > Data Required time

Hold Slack = Data Arrival time - Data Required time

$$= T_{cq1} + T_{comb} - T_{hd2} \geq 0$$

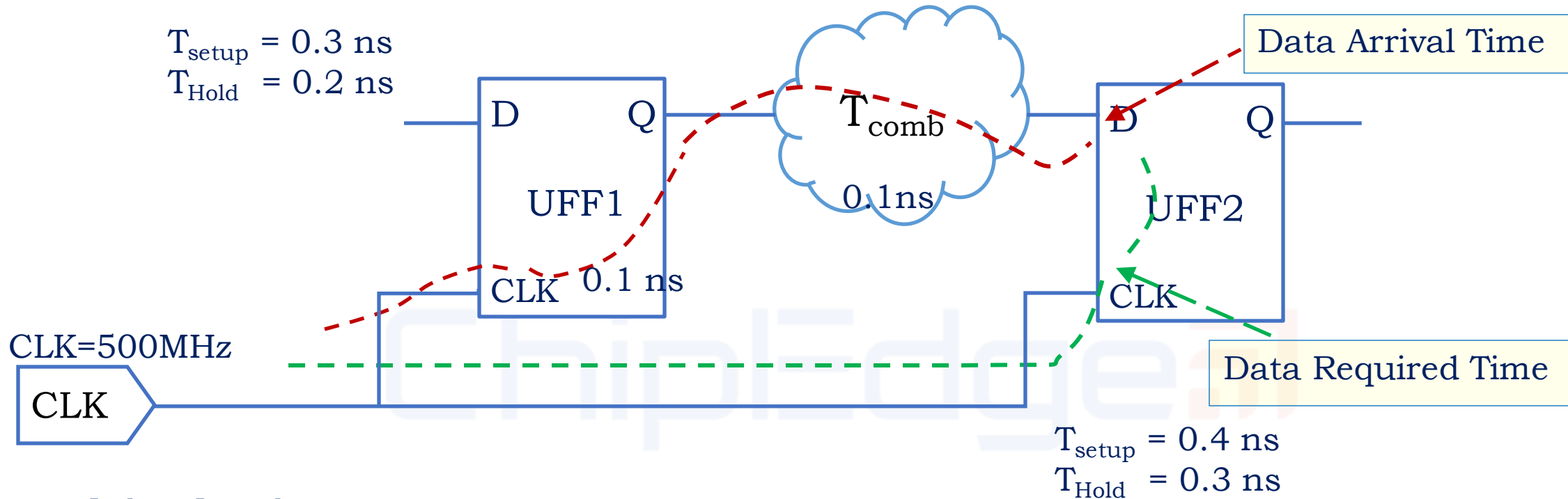
$$= ??$$

If Hold slack $\geq 0 \rightarrow$ Hold is MET

If Hold slack $< 0 \rightarrow$ Hold is VIOLATED

WNS (worst negative slack) for Hold is the worst of all the negative Hold slacks in the design.

Calculation Of Hold Slack Violated (Reg to Reg)



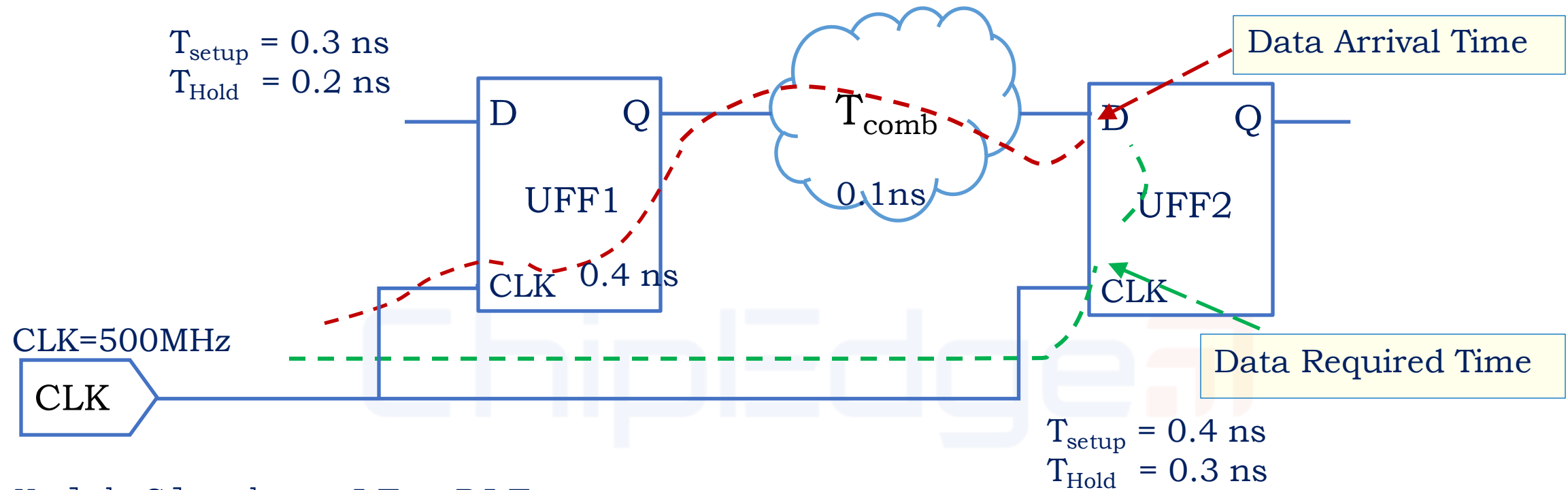
$$\text{Hold Slack} = \text{AT} - \text{RAT}$$

$$= T_{cq1} + T_{comb} - T_{hold2}$$

$$= 0.1 + 0.1 - 0.3 = -0.1$$

Hold check is failing for this path as slack is negative (-0.1)

Calculation Of Hold Slack Met (Reg to Reg)



$$\text{Hold Slack} = \text{AT} - \text{RAT}$$

$$\begin{aligned} &= T_{\text{cq1}} + T_{\text{comb}} - T_{\text{hold2}} \\ &= 0.4 + 0.1 - 0.3 = 0.2 \end{aligned}$$

Hold check is passing for this path as slack is positive (+0.2)

Introduction to STA-1



Setup



Hold



Timing Checks – Setup & Hold



Timing reports

In this section the timing report in different timing paths like reg to reg, in to reg or reg to out are discussed in detail.

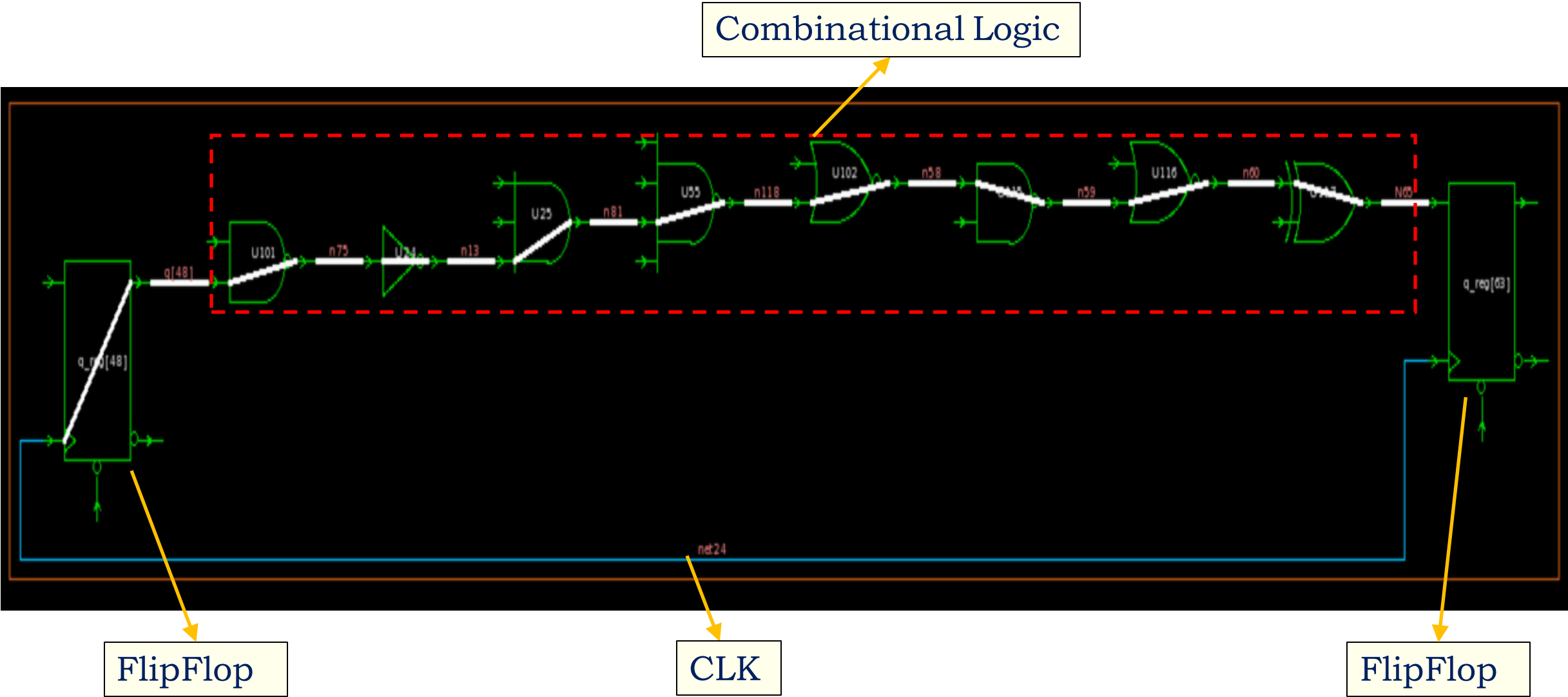
Clock Skew



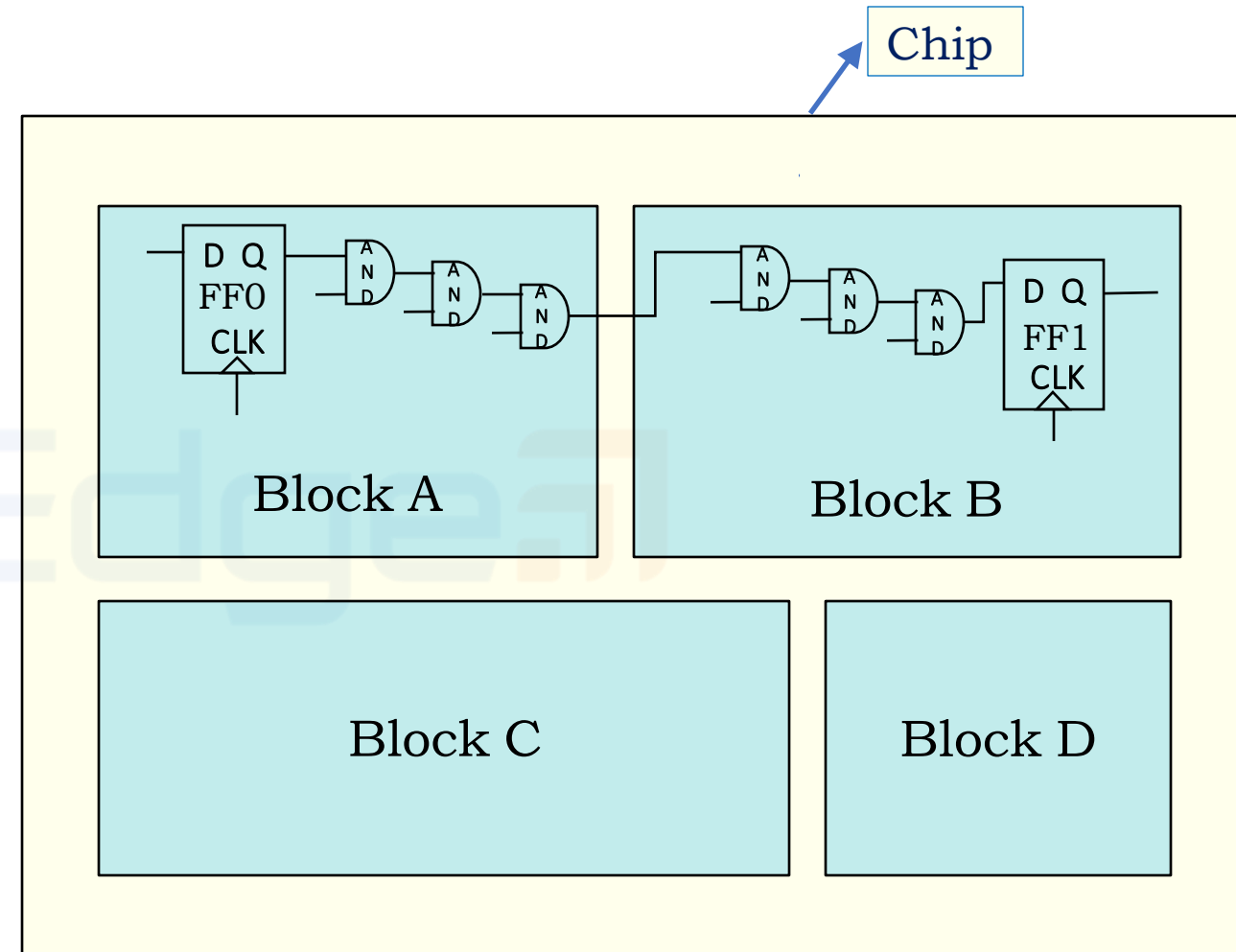
Additional Information

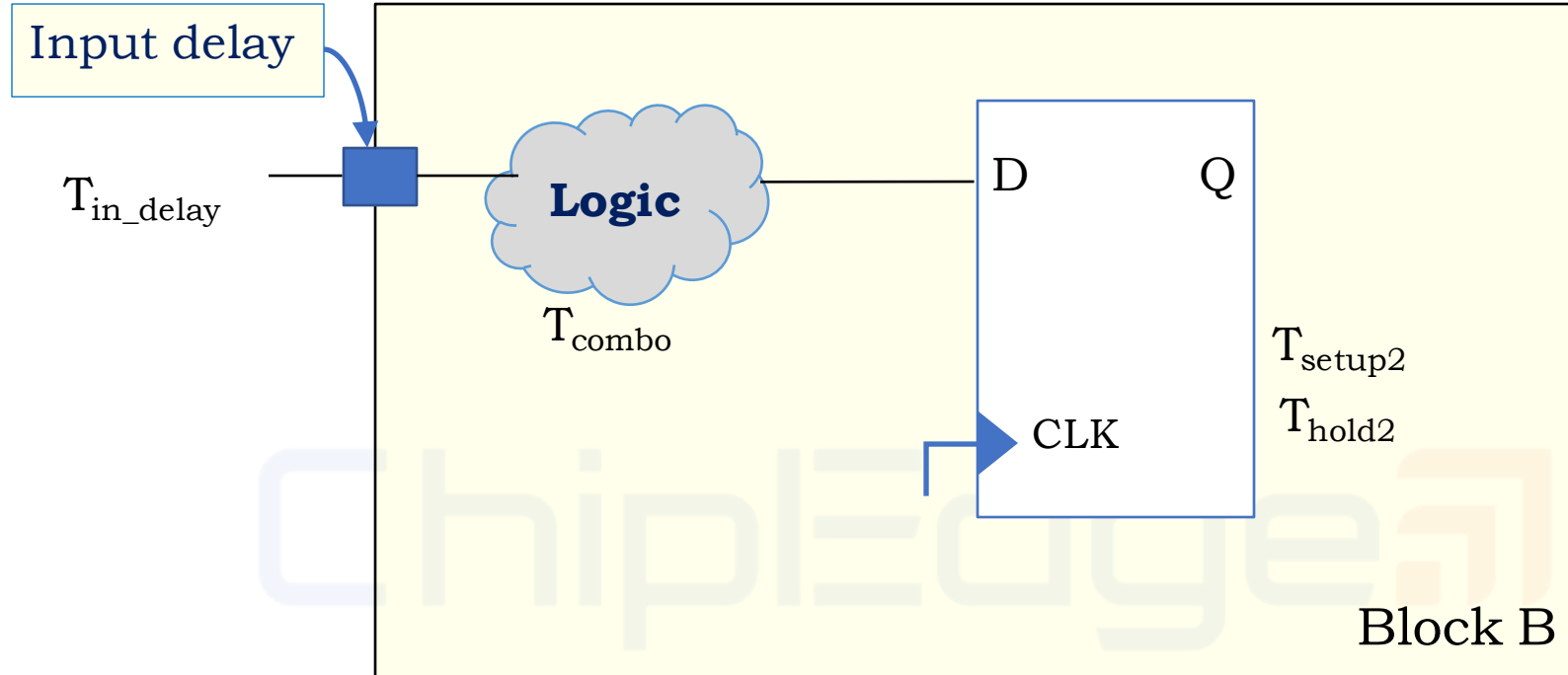


Startpoint: q_reg[48] (rising edge-triggered flip-flop clocked by CLK)			
Endpoint: q_reg[63] (rising edge-triggered flip-flop clocked by CLK)			
Path Group: CLK			
Path Type: max			
Header	Point	Incr	Path
	clock CLK (rise edge)	0.00	0.00
	clock network delay (ideal)	0.40	0.40
	q_reg[48]/CLK (DFFARX2_LVT)	0.00	0.40 r
	q_reg[48]/Q (DFFARX2_LVT)	0.19	0.59 r
	U101/Y (NAND2X0_LVT)	0.04	0.64 f
	U24/Y (INVX0_HVT)	0.04	0.68 r
	U25/Y (AND3X1_HVT)	0.12	0.80 r
	U55/Y (NAND4X0_HVT)	0.28	1.08 f
	U102/Y (NOR2X1_LVT)	0.09	1.16 r
	U115/Y (NAND2X0_LVT)	0.02	1.19 f
	U116/Y (NOR2X0_LVT)	0.06	1.24 r
	U117/Y (XOR2X1_LVT)	0.07	1.32 f
	q_reg[63]/D (DFFARX2_LVT)	0.00	1.32 f
	data arrival time		1.32
Data Arrival	clock CLK (rise edge)	0.50	0.50
	clock network delay (ideal)	0.40	0.90
	clock reconvergence pessimism	0.00	0.90
	q_reg[63]/CLK (DFFARX2_LVT)		0.90 r
	library setup time	-0.04	0.86
	data required time		0.86
	data required time		0.86
	data arrival time		-1.32
	slack (VIOLATED)		-0.46
Data Required			
Slack			



- The path from FF0 to FF1 is a **Reg-to-Reg** path when we are looking at it in the top level.
- At **block level** it is a Reg-to-Out path in block A, it is a In-to-Reg path in Block B.
- When the timing is analyzed at Block level, the delay of that part of the path present outside the block is provided in SDC (set_input_delay, set_output_delay)





Setup Slack = Required Arrival Time – Arrival Time

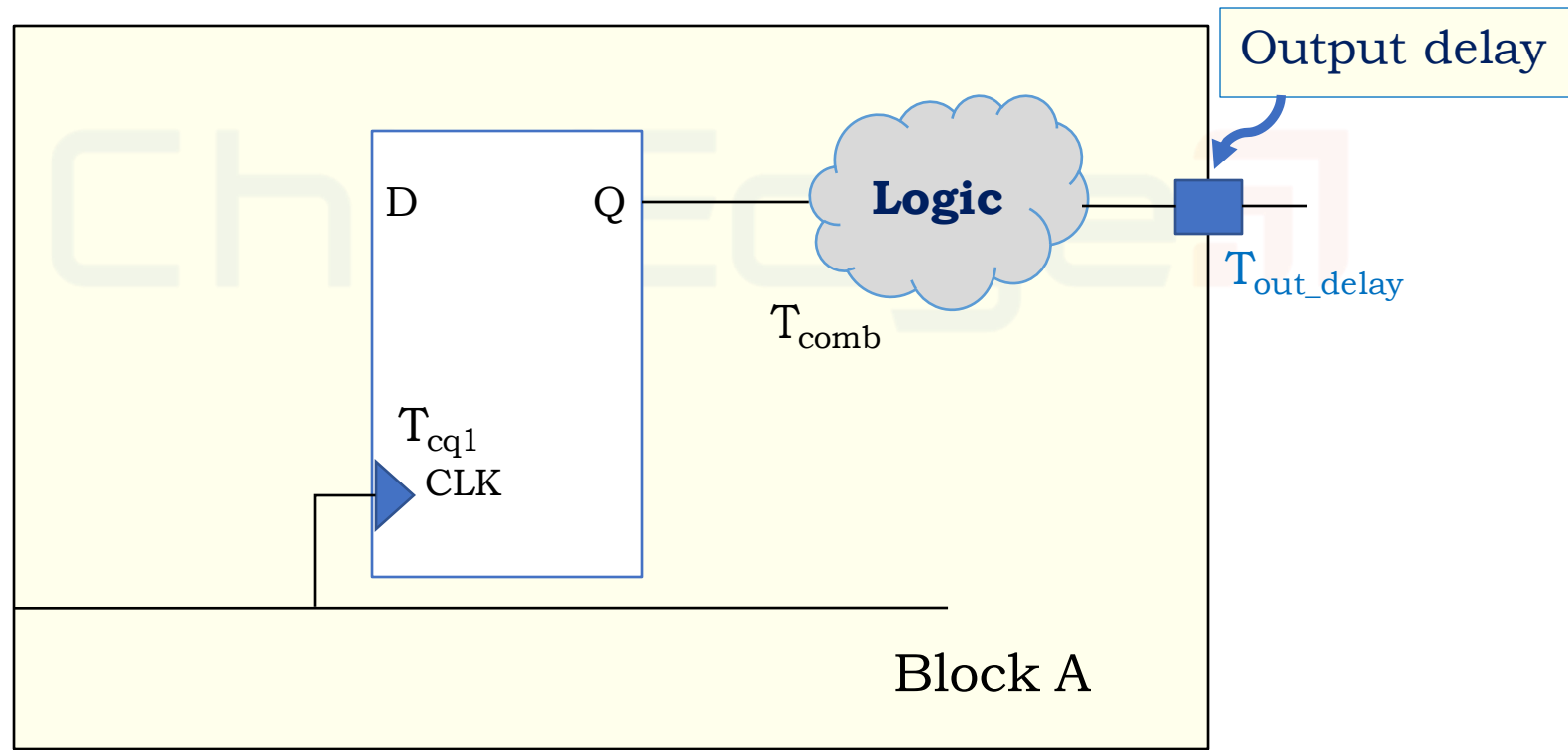
$$= [T_{clk} - T_{setup2}] - [T_{in_delay} + T_{comb}]$$

Hold Slack = Arrival Time – Required Arrival Time

$$= [T_{in_delay} + T_{comb}] - T_{hold2}$$

Setup Slack = Required Time – Arrival Time

$$= [T_{\text{clk}} - T_{\text{out_delay}}] - [T_{\text{cq1}} + T_{\text{combo}}]$$



Introduction to STA-1



Setup



Hold



Timing Checks – Setup & Hold



Timing reports

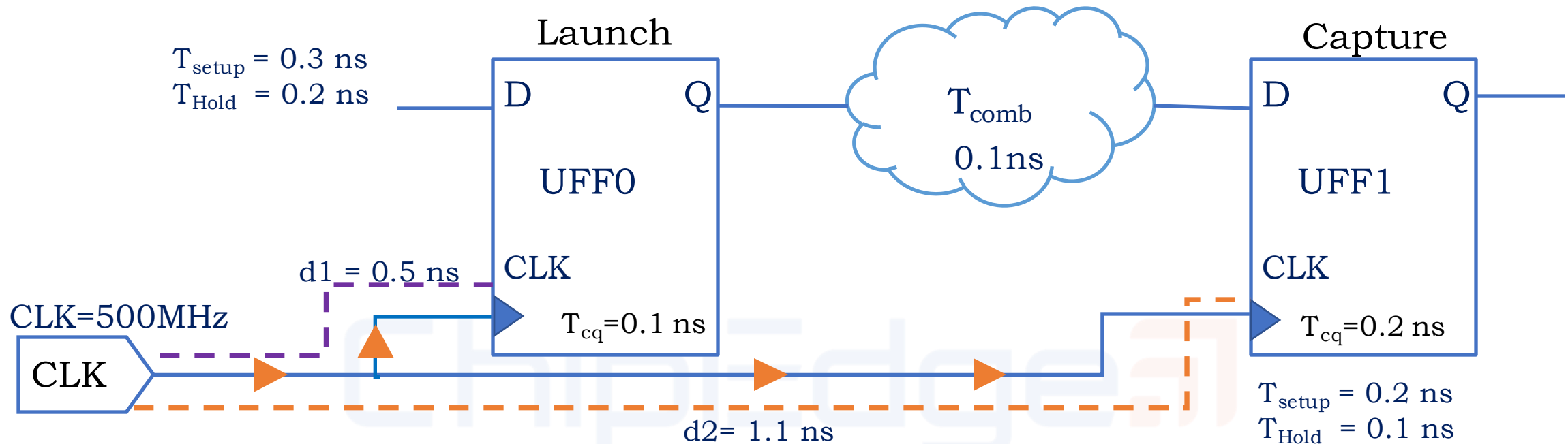


Clock Skew

Additional Information



This section explains about the definition of skew, Positive vs Negative skew and addition of skew into the Setup – Hold slacks.



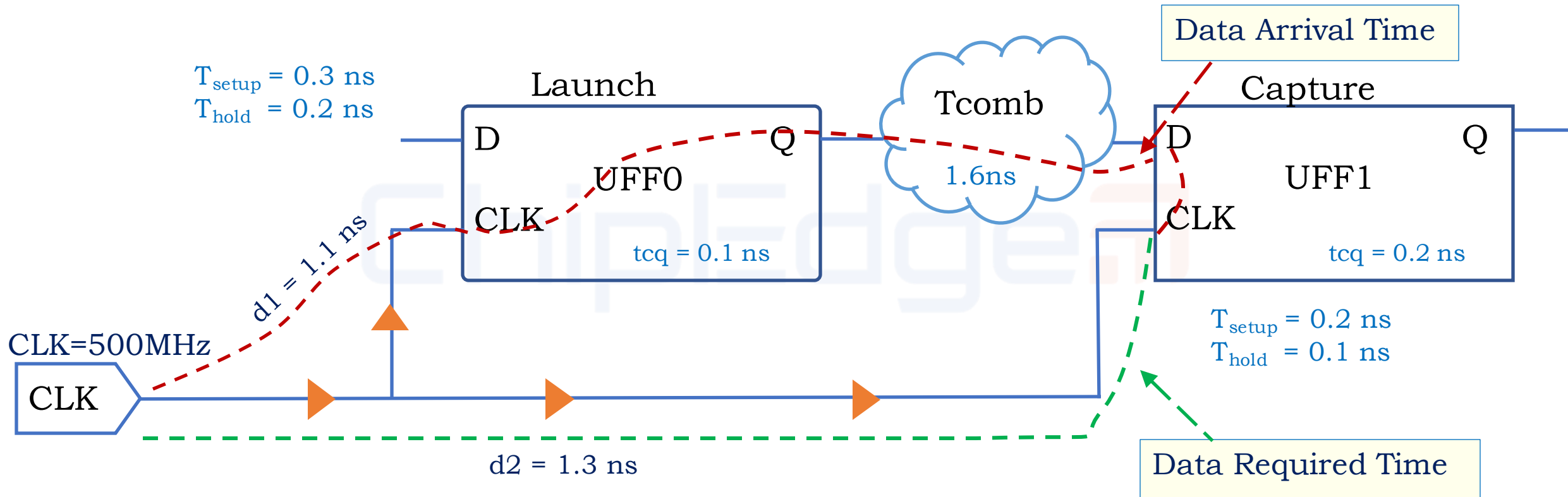
Launch latency is the delay from the clock port to the launch clock pin = $d1$

Capture Latency is the delay from the clock port to the capture clock pin = $d2$

Skew is the difference between Capture Latency and Launch Latency.

$$\text{Skew} = \text{Capture Latency} - \text{Launch Latency}$$

Calculation Of Setup Slack with Skew (Reg to Reg)



Setup Slack = Required Arrival Time - Arrival Time

$$= [T_{cap} + T_{clk} - T_{setup}] - [T_{lan} + T_{cq} + T_{combo}] \quad \text{..... 1}$$

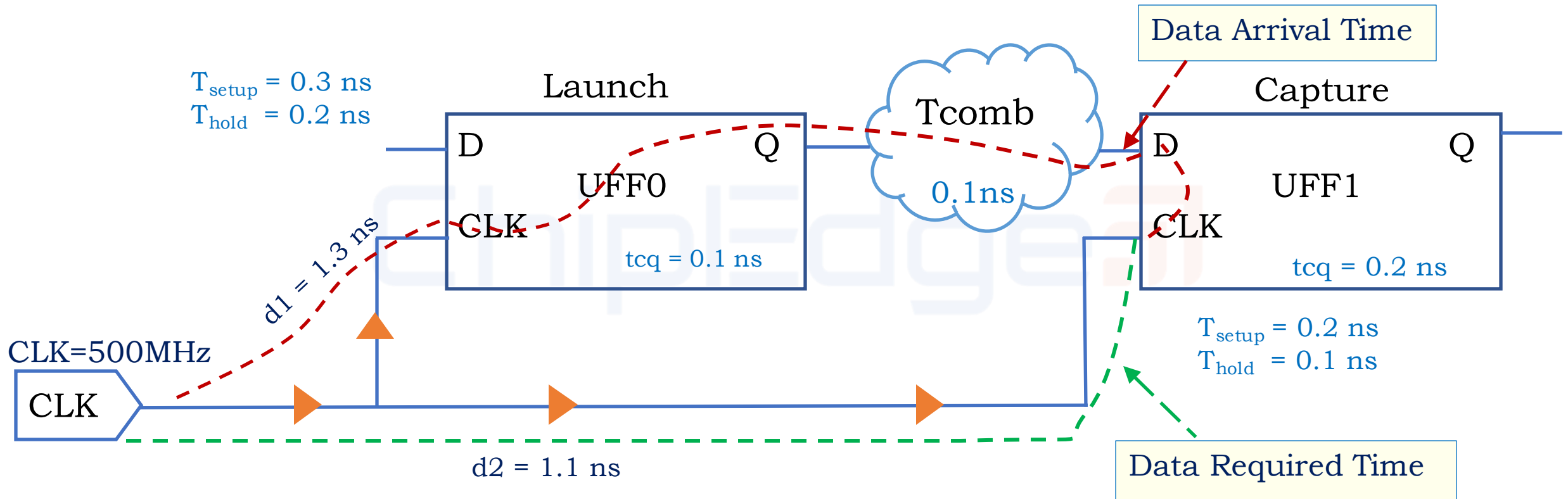
$$= \{ [T_{clk} - T_{setup}] - [T_{cq} + T_{combo}] \} + skew \quad \text{..... 2}$$

$$= \{ slack \text{ without skew (pre_CTS slack)} \} + skew$$

$$= [1.3 + 2 - 0.3] - [1.1 + 0.1 + 1.6]$$

$$= \{ [2 - 0.3] - [0.1 + 1.6] \} - (1.3 - 1.1)$$

Setup Slack = 0.8



Hold slack = Arrival Time - Required Arrival Time

$$= [T_{lan} + T_{cq} + T_{combo}] - [T_{cap} + T_{hold2}] \dots\dots\dots 1$$

$$= \{ [T_{cq} + T_{combo}] - [T_{hold2}] \} - skew \dots\dots\dots 2$$

$$= \{ slack \text{ without skew (pre_CTS slack)} \} - skew$$

$$= [1.3 + 0.1 + 0.1] - [1.1 + 0.1]$$

$$= \{ [0.1 + 0.1] - [0.1] \} - (1.3 - 1.1)$$

$$\text{Hold slack} = 0.3$$

$$\text{SETUP Slack} = \{ [T_{\text{clk}} - T_{\text{setup}}] - [T_{\text{cq}} + T_{\text{combo}}] \} + \text{skew}$$

$$\text{HOLD Slack} = \{ [T_{\text{cq}} + T_{\text{combo}}] - [T_{\text{hold2}}] \} - \text{skew}$$

$$\text{Skew} = \text{Capture Latency} - \text{Launch Latency}$$

- If the skew is positive, the setup slack will improve and hold slack degrades.
- If the skew is negative, the setup slack degrades but hold slack gets better.

STA Table – Factors Affecting Setup & Hold

S.No.	Factors	Setup	Hold
1.	Data path delay ($T_{cq} + T_{combo}$) ↑	X	✓
2.	T_{cq} ↑	X	✓
3.	T_{combo} ↑	X	✓
4.	T_{setup} ↑	X	N/A
5.	T_{hold} ↑	X	X
6.	T_{launch} ↑	NA	✓
7.	$T_{capture}$ ↑	✓	X
8.	Skew ↑	✓	X
9.	T_{clock} ↑	✓	N/A
10.	Frequency ↑	X	N/A

↑ = If High
 X = Not Affected
 ✓ = Affected
 N/A = Not Applicable

Introduction to STA-1



Setup



Hold



Timing Checks – Setup & Hold



Timing reports



Clock Skew



Additional Information

This section explains about max transition and max capacitance explained in detail.

- **Max_Transition** violation: If the transition time on any pin is more than maximum transition target, it is reported as Max_Transition violation.
- **Max_Capacitance** violation: The capacitance load on any pin is more than maximum capacitance target it is reported as Max_Capacitance violation

Note: the maximum transition target ,maximum capacitance target are generally set in SDC(set_max_transition, set_max_capacitance)

- Static Timing Analysis
- Setup
- Hold
- Timing Checks & Slack Calculations
- Timing Report Analysis
- Clock Skew

Module 9

Placement

- In this Module you will understand What is Placement and its importance in the Physical design flow.
- The typical concepts like Physical cells, Scan chain, QoR analysis, and Congestion are discussed in detail with examples.

- Introduction to Placement
 - ✓ Before Placement
 - ✓ Pre-place Stage
- Physical Cells
 - ✓ End cap Cells
 - ✓ Tap Cells
 - ✓ Spare Cells
 - ✓ I/O Buffers
- Placement Methodologies
 - ✓ Timing Driven
 - ✓ Congestion Driven
 - ✓ Power Driven
 - ✓ Area Driven
 - ✓ Timing, Congestion Priority

- Steps During Placement
 - ✓ Coarse placement
 - ✓ Legalized placement or Detailed placement
 - ✓ Trial Route/global route
 - ✓ Optimization
- Post Placement
 - ✓ Post Place - Tie Cells
- Scan Chain Reordering
 - ✓ Post Place - Scan Chain Reordering
 - ✓ What is a Scan chain
 - ✓ Scan Chain Reordering
- QoR Analysis
 - ✓ Congestion & Timing

- Congestion Analysis
 - ✓ Congestion Map
 - ✓ Vertical Congestion
 - ✓ Horizontal Congestion
 - ✓ Fixing Congestion
 - ✓ Placement Bounds
- Timing Analysis
 - ✓ Fixing Timing Issues
- Placement Checks
 - ✓ Placement Quality Checks

Introduction to Placement

This section gives the introduction to placement and different steps involved in placement stage

Physical Cells



Placement Methodologies



Steps During Placement



Post Placement



Scan Chain



QoR Analysis



Congestion Analysis

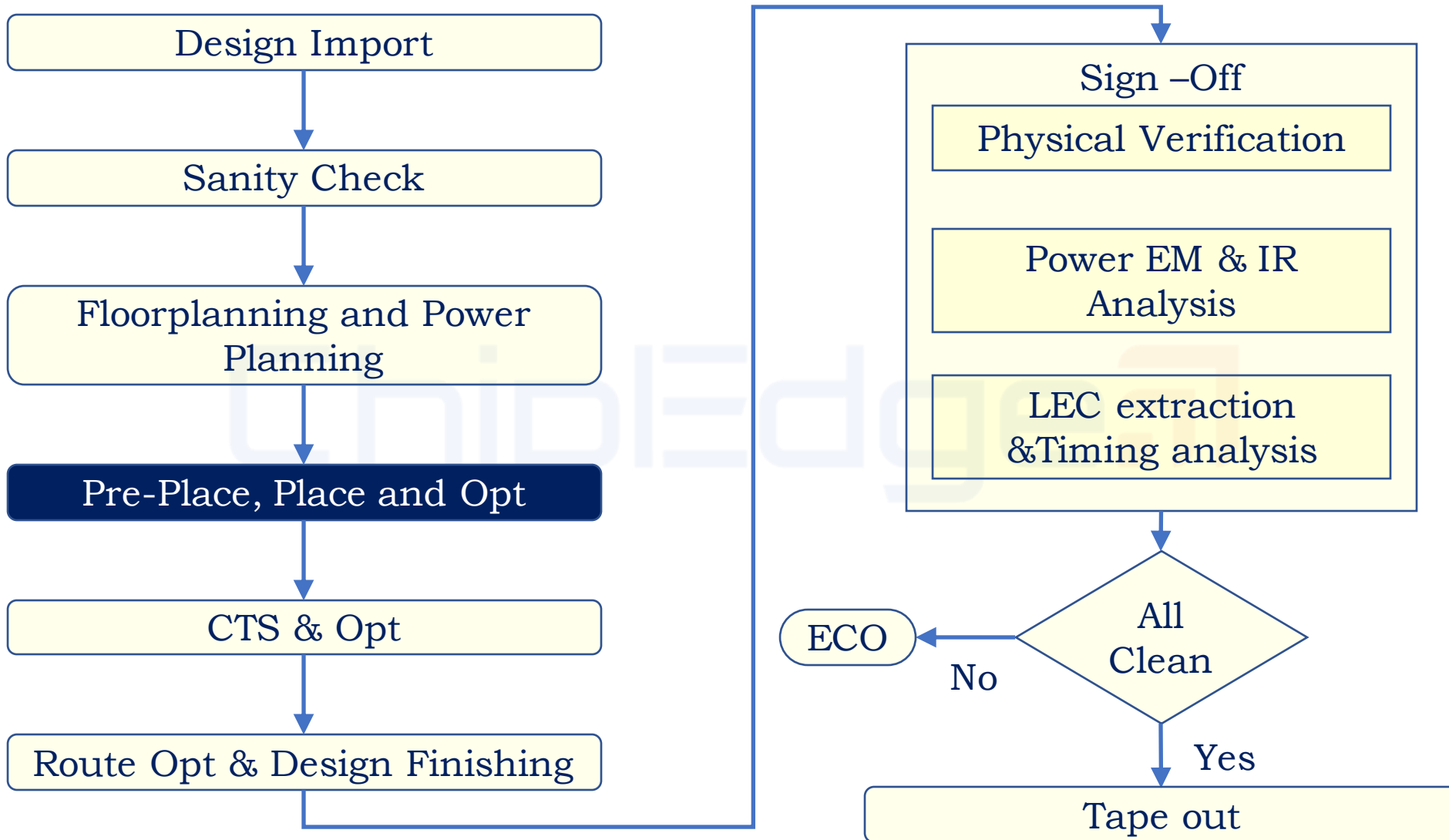


Timing Analysis



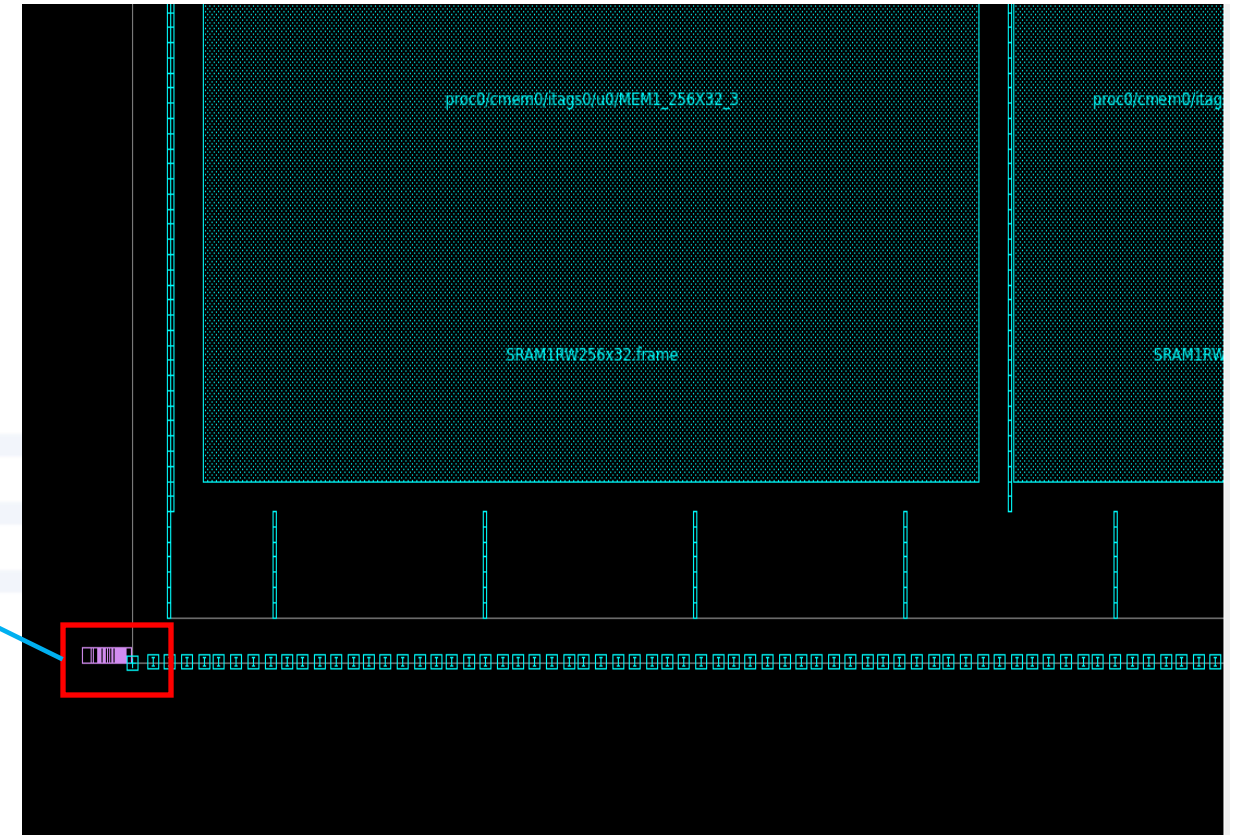
Placement Checks





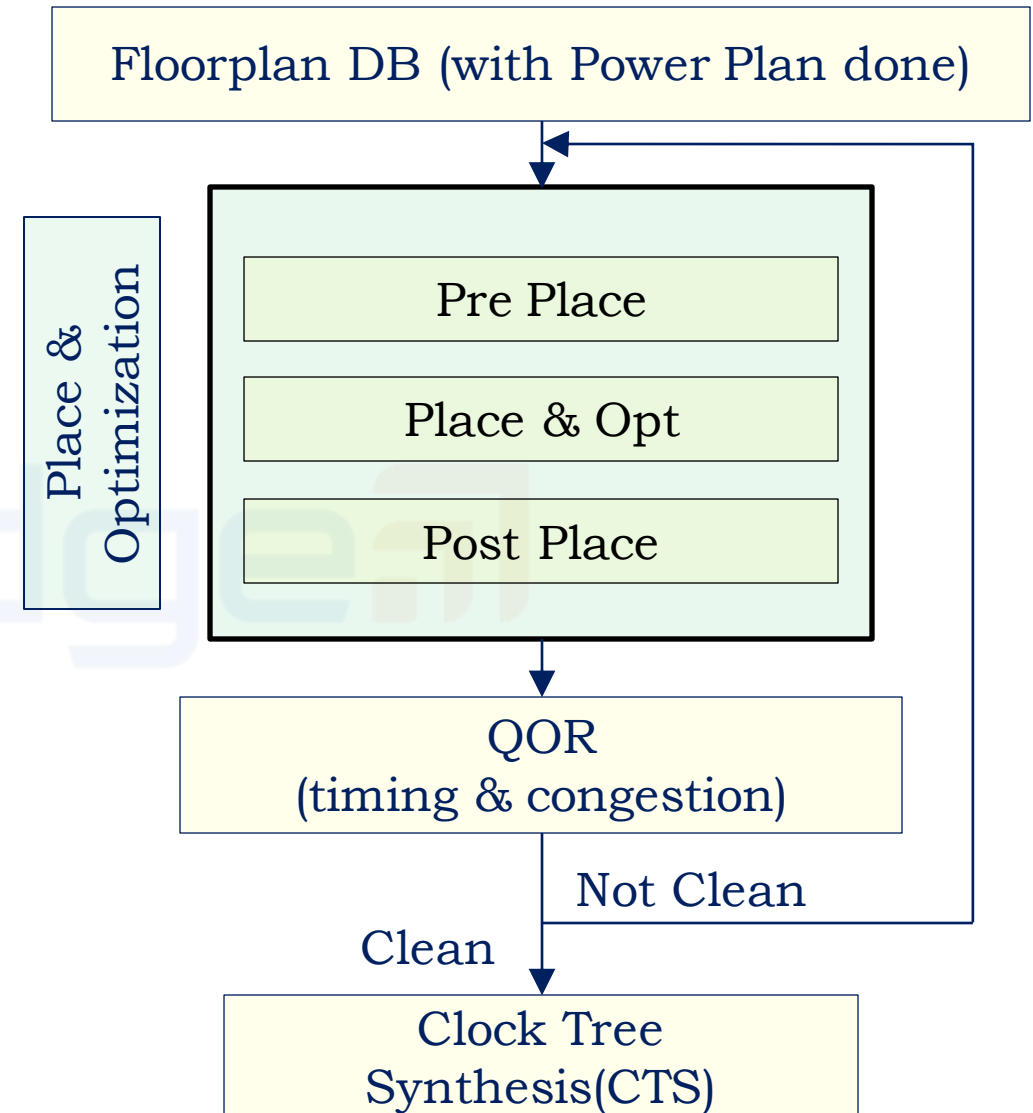
At this stage, from Floorplan db:

1. All the macros are placed
2. All the I/o Ports are placed
3. Standard Cells are unplaced
(all the Std cells are at the origin).



What Is Placement?

- Definition: Legal location to all standard cells, optimized for best routability, timing, power, area(PPA)
- Goals: Achieve a database that is routable and timing clean
- Stages in Placement:
 - ✓ Preplace
 - ✓ Place & Opt
 - ✓ Post place
 - ✓ QOR analysis



- Pre-place cells should be placed in design as per foundry requirement.
- Placing some special cells before asking the tool to place all the standard cells.
- Examples of pre-placed cells :
 - ✓ EndCap cells → To mitigate WPE (well proximity effect)
 - ✓ Tap-Cells → For well tapping and latch-up removal.
 - ✓ I/O Buffers → For Signal strengthening/slew rate improvement
 - ✓ Spare-Cells → For Post tapeout or last moment ECOs

- ✓ Other optional preplace activities.
- ✓ Placement bounds (as needed) → soft, hard, exclusive
- ✓ Blockages, keep-outs/padding.
- ✓ Group paths (as needed) → to give better weightage for critical path optimization.

Introduction to Placement



Physical Cells

Placement Methodologies



Steps During Placement



Post Placement



Scan Chain



QoR Analysis



Congestion Analysis



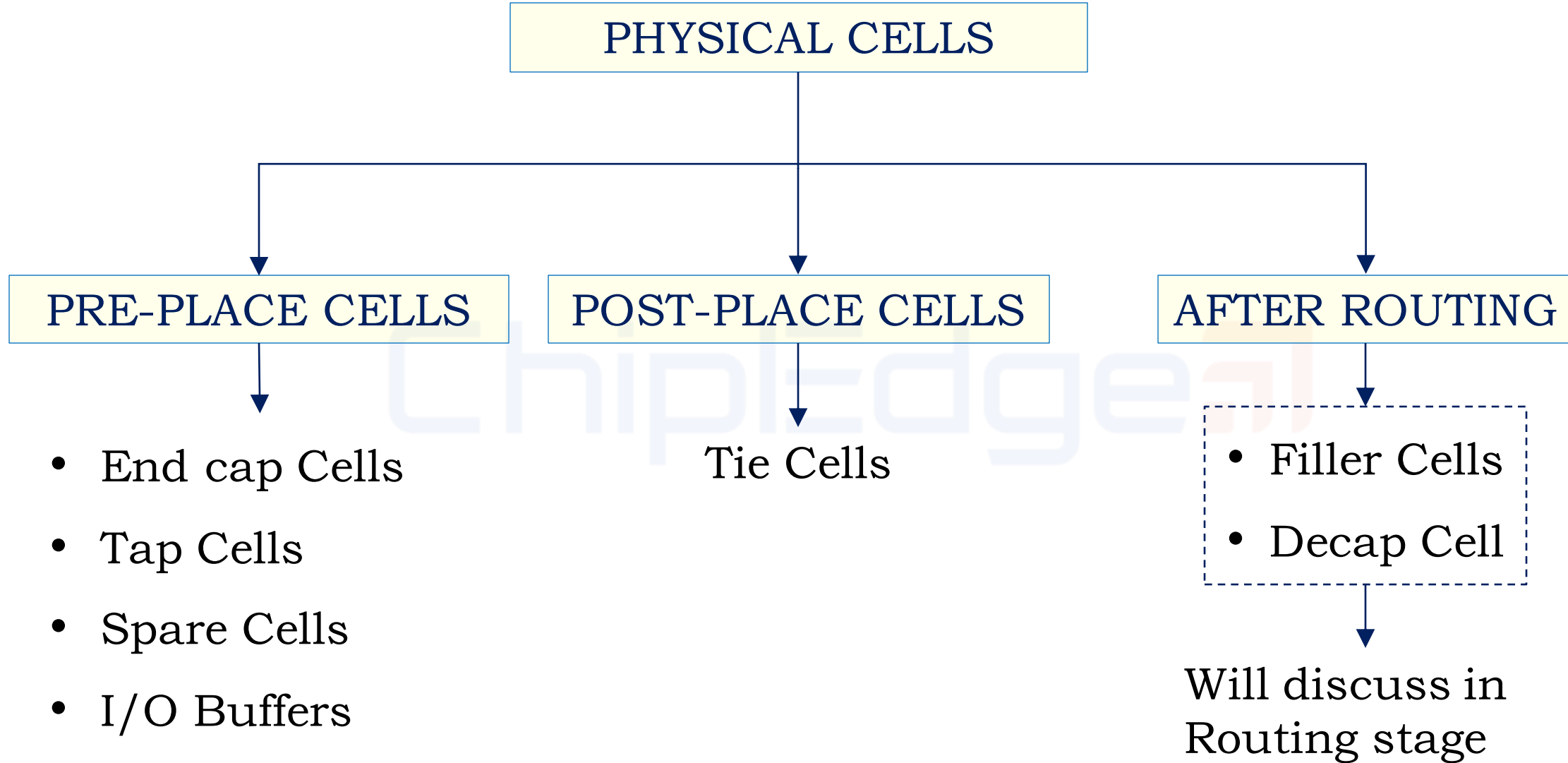
Timing Analysis



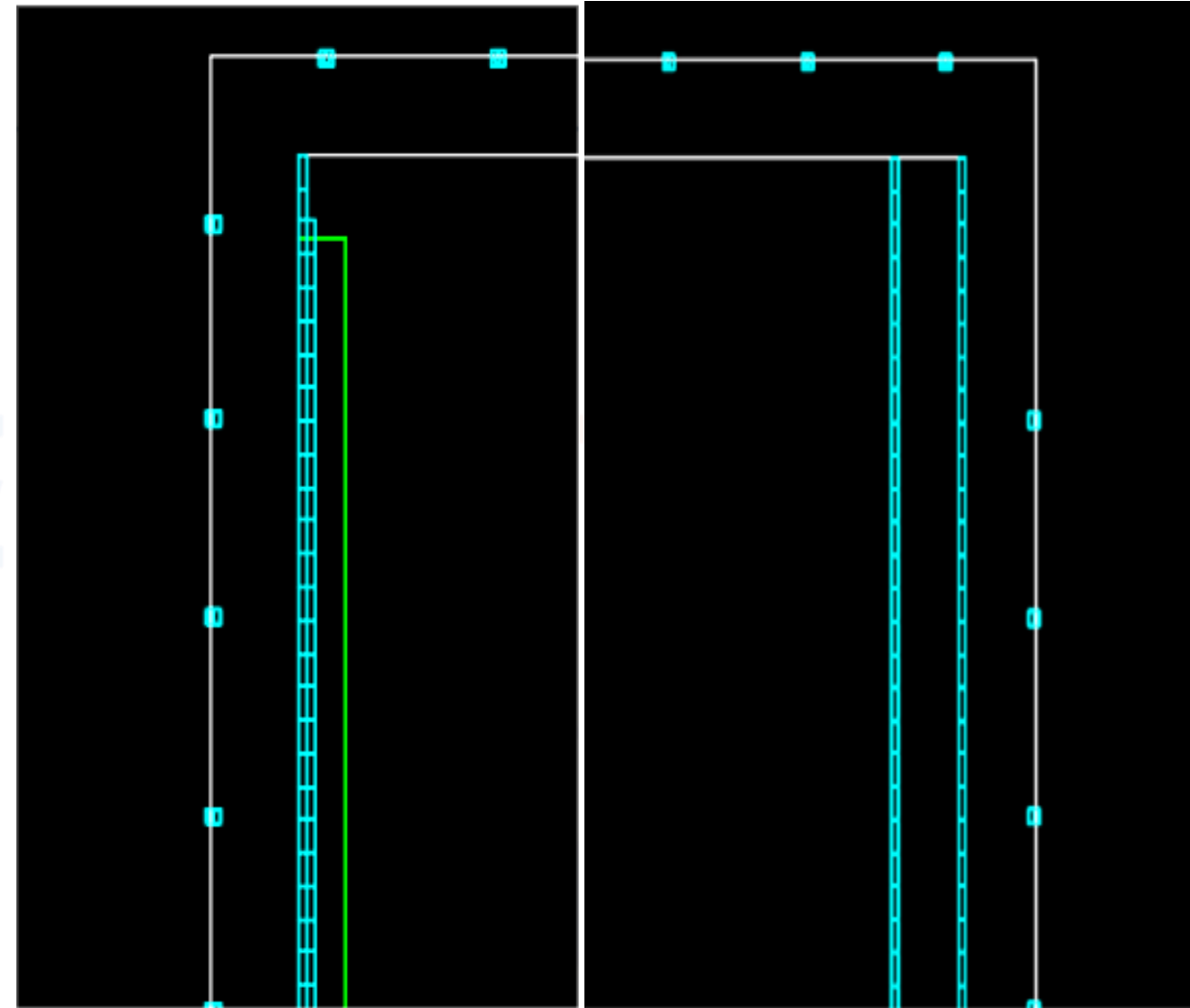
Placement Checks



In this section, Cells used in cts, the end-cap cells, tap cells, base tapeout and metal tapeout are explained in detail. The concept of spare cells are also explained with illustrations.

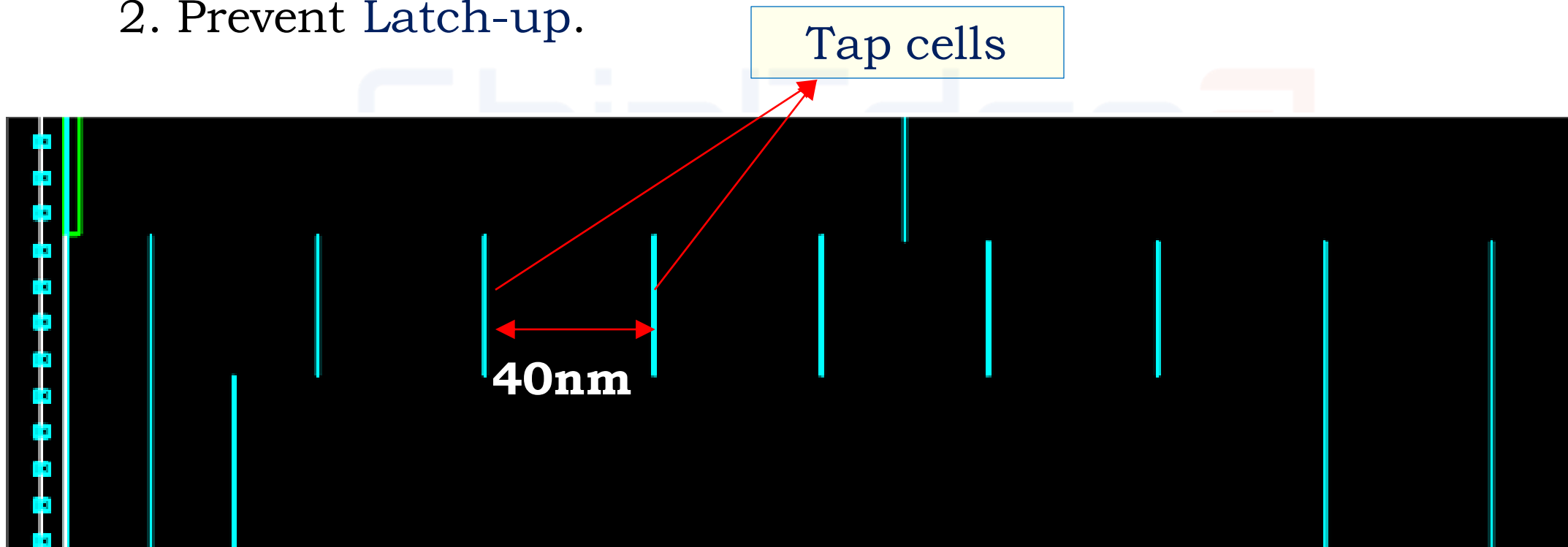


- This cell doesn't have any timing model. Placed at the start and end of std cell row
- Placed to avoid Well proximity effect.
- At the end of the ROWs, where NWELL ends, doping concentration of NWELL is higher leading to V_t being higher for cells sitting at the end of the ROW
- When base DRC checks are run as part of signoff DRC, any missing end-cap situations are caught

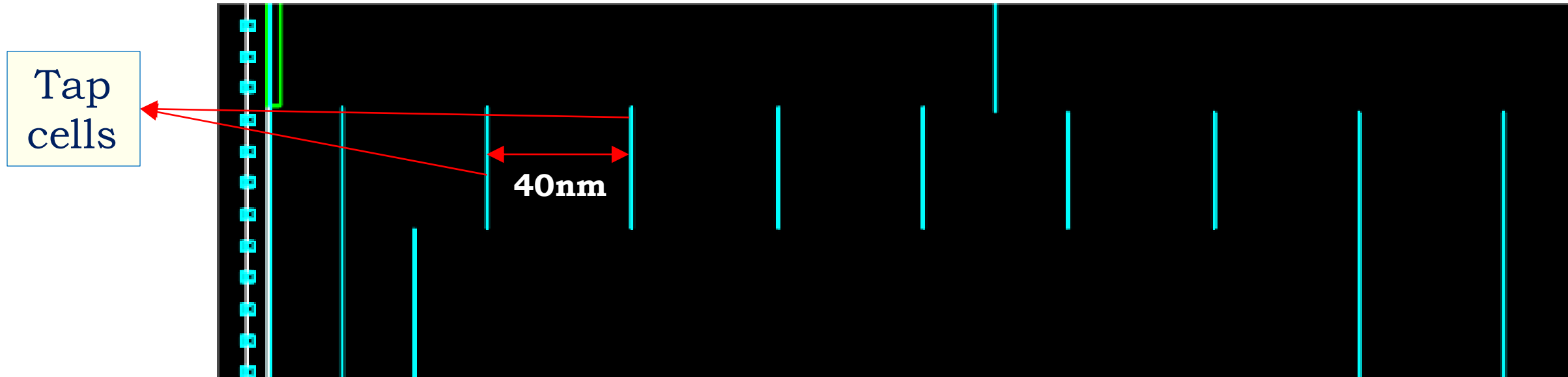


These cells are used to:

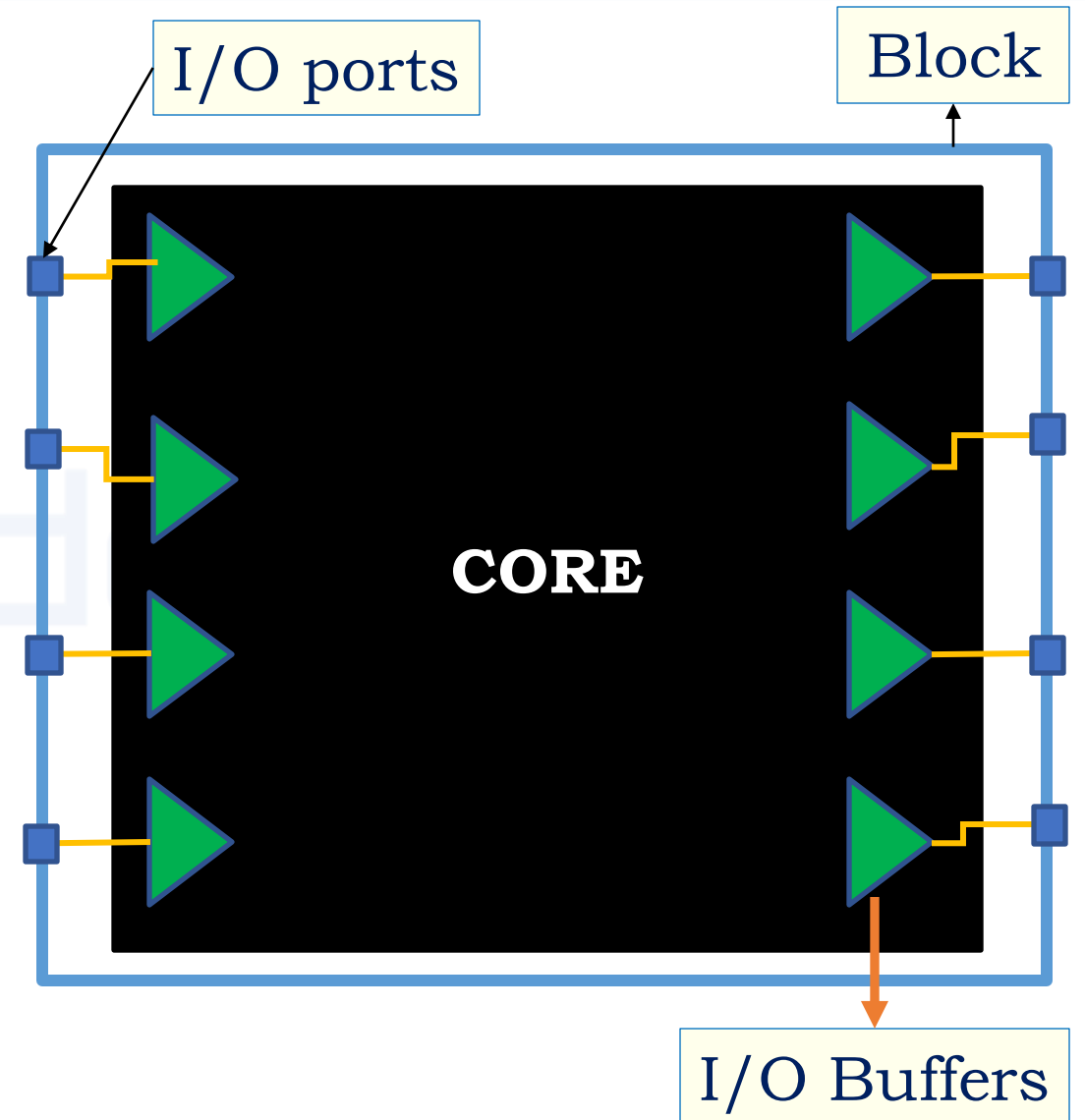
1. Connect Nwell to Power and Psub to GND at regular intervals
2. Prevent Latch-up.



- The spacing between the well-tap insertion in the designs will vary according to the technology,
 - ✓ In 28nm, the distance is 40um
 - ✓ In 16nm, the distance is 25um
- *This is must do. There is a DRC check on this – later this will need to be fixed if not correct.*



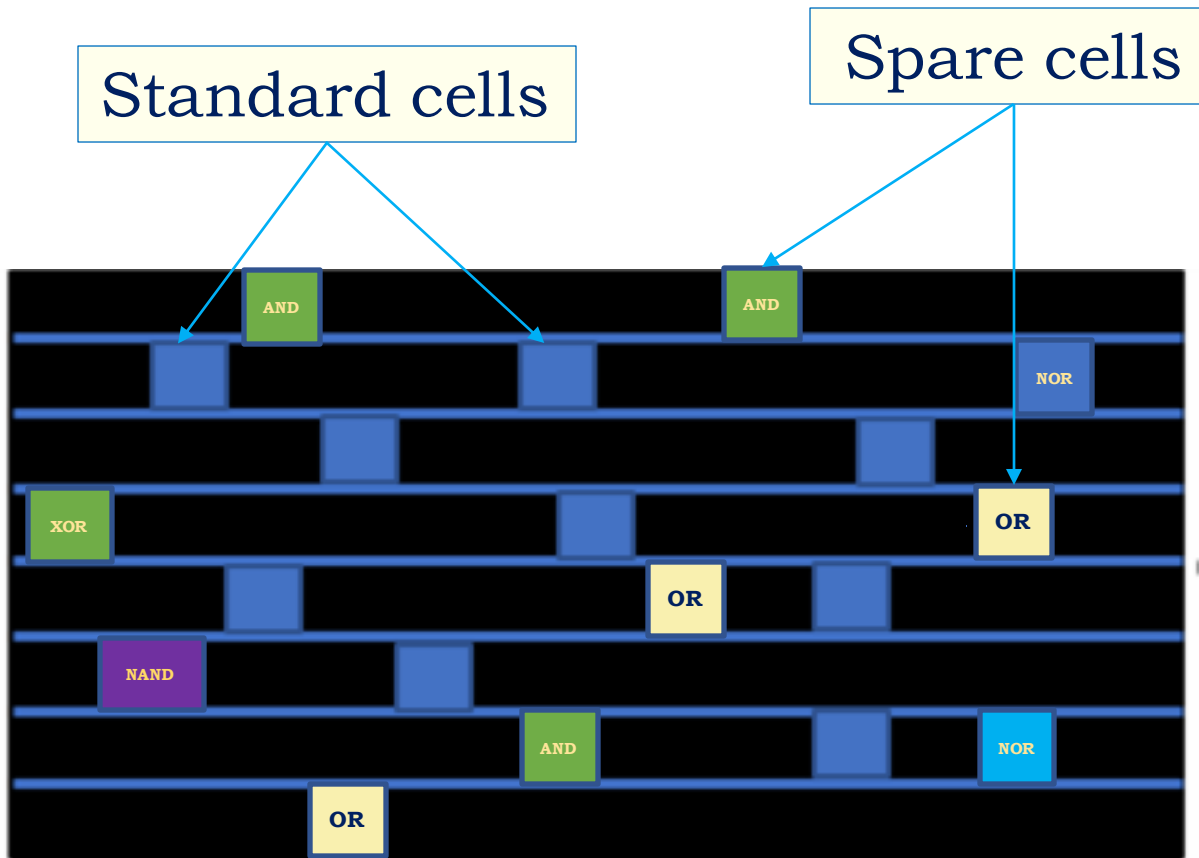
- The Inputs to the block may be coming from another block which is sitting far away in a SoC, so the transition may be bad on the signals.
- Buffers are added at the Input and Output signal ports to improve the transition on the signal entering the block and exiting the block.



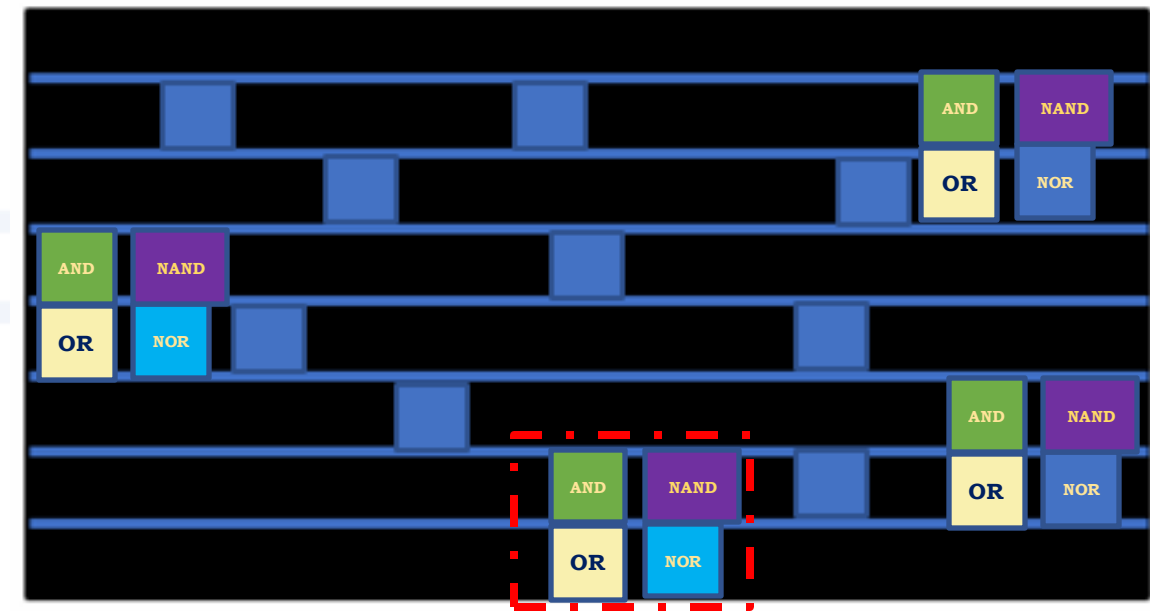
- Normal functional std cells, added as “extra cells” in the design for later use.
- Even if they are not present, the design will work without any issues.
- Will be used to implement ECO after Base-Tape Out & Before Metal-Tape Out.
- If any bugs reported/found after the base tape-out, we can use these spare cells to fix the bugs without adding “new cells” to the database.
- Using these spare cells, we can make bug-fixes using Metal layers only because the base layers of the functional cells are already present.

- Base layers are below M1 (metal1)
- M1 and above are called Metal layers.
- Fab can take Base layers first to manufacture
- Base layers are released : Base Tape-out
- Metal layers are typically released later (~2-4 weeks, depends on Fab and process)

- After Base Tape-out, any new bug fixes can be done using Spare cells and metal layers only.
- For any subsequent bug-fixes (after the chip is tested and back), spare cells can be used for next Metal tape-out. Rather than changing all layers.
- Metal tape-out saves money (Mask cost for new set of Base layers) and time (it takes to manufacture Base layers)



Individual Sprinkling of Spare cells



Group Sprinkling of Spare cells

Introduction to Placement



Physical Cells



Placement Methodologies

This section explains the methodologies used to achieve a good standard cell placement.

Steps During Placement



Post Placement



Scan Chain



QoR Analysis



Congestion Analysis



Timing Analysis

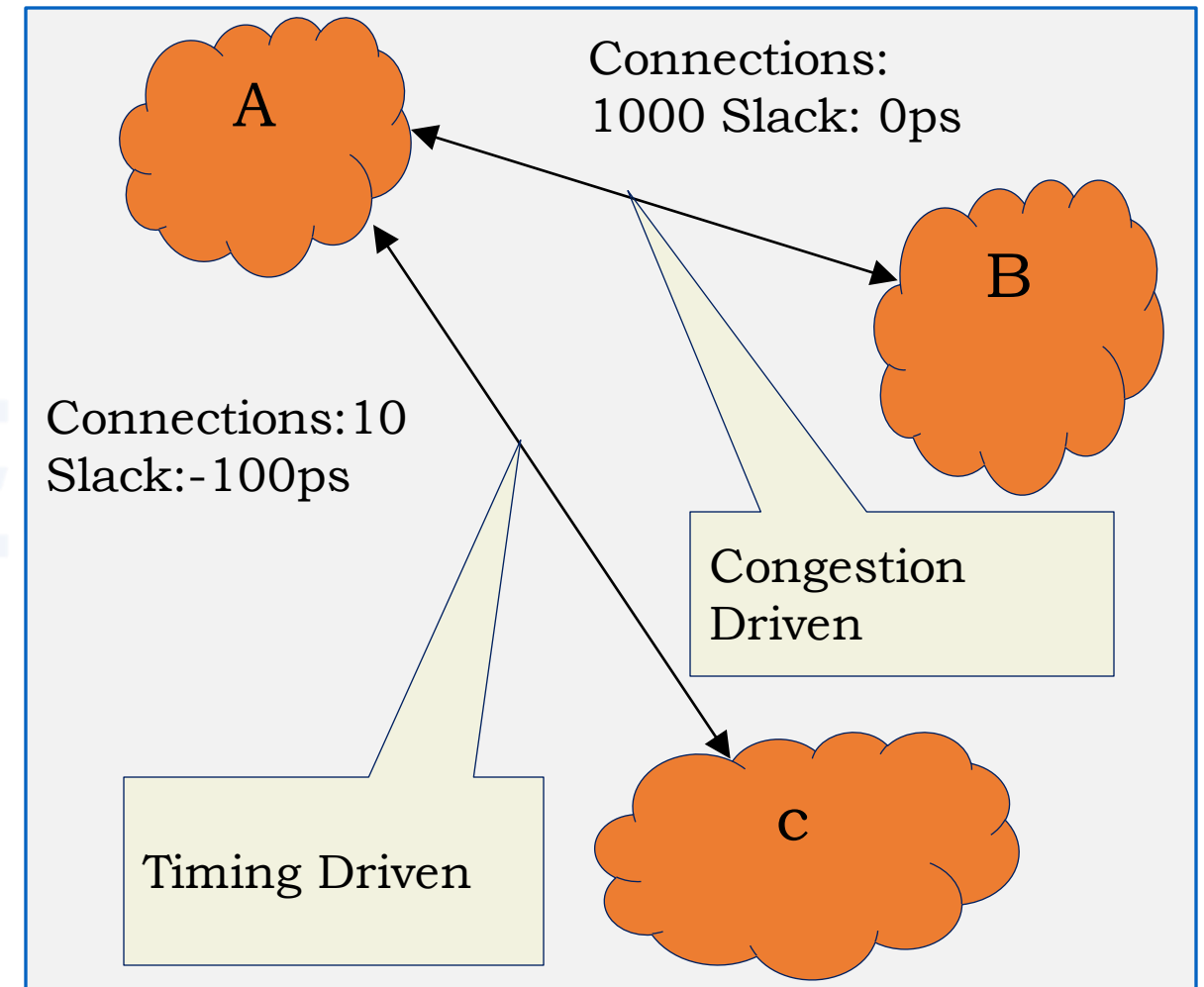


Placement Checks



Standard Cell Placement based on

1. Timing Driven
2. Congestion Driven
3. Power Driven
4. Area Driven



ICC-II command for priority →

```
> set_app_options -name place_opt.place.congestion_effort -  
value high /medium /low  
  
> set_app_options -name opt.timing.effort -value high /medium  
/low
```

Introduction to Placement



Physical Cells



Placement Methodologies



Steps During Placement

Post Placement



Scan Chain



QoR Analysis



Congestion Analysis



Timing Analysis



Placement Checks



This section illustrates the process of placement through each individual steps like coarse placement, detail placement, trial route and optimization.

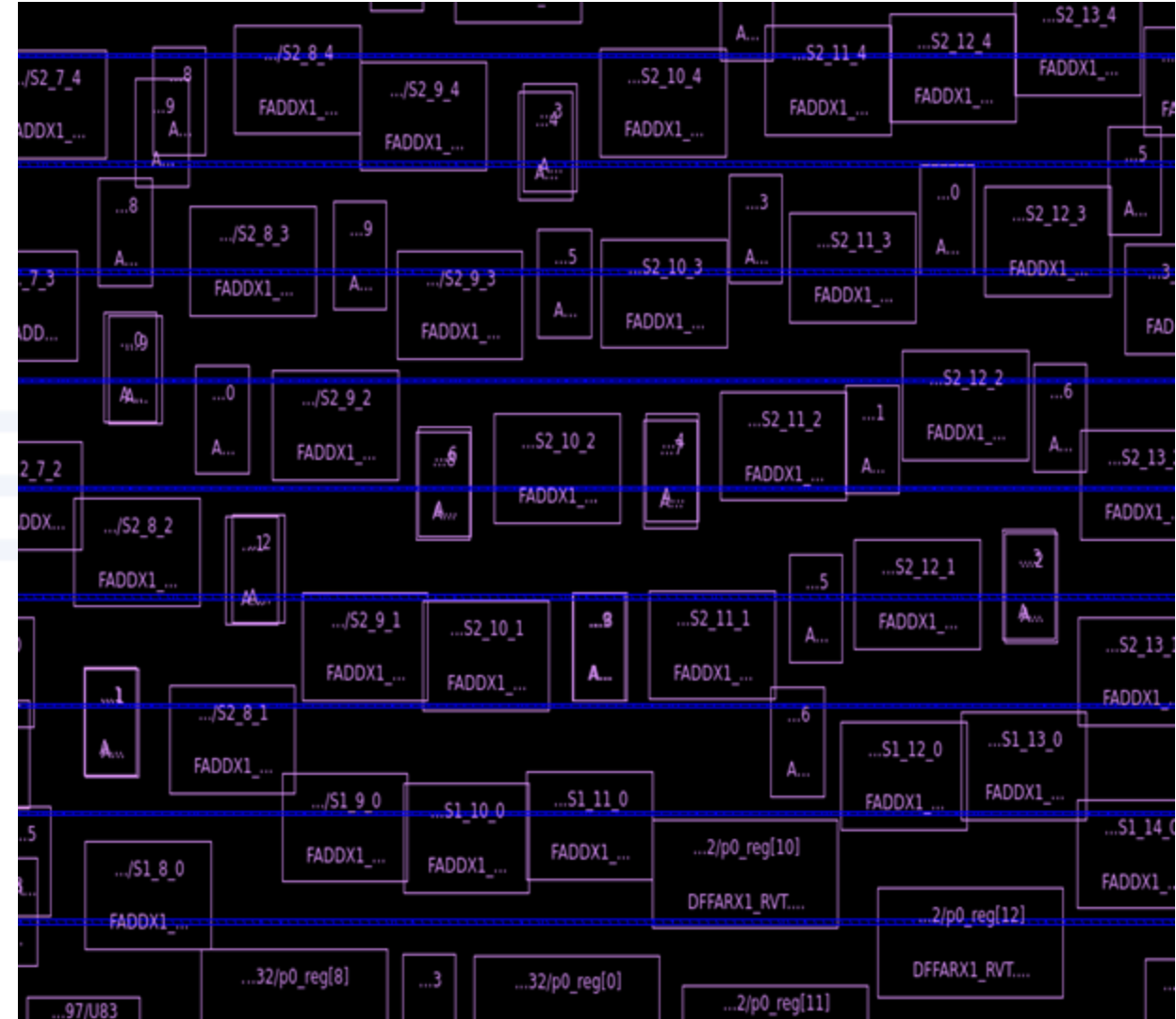
`place_opt` is the command. Following steps are run internally

1. Coarse placement
2. Legalized placement or Detailed placement
3. Trial Route/global route
4. Optimization

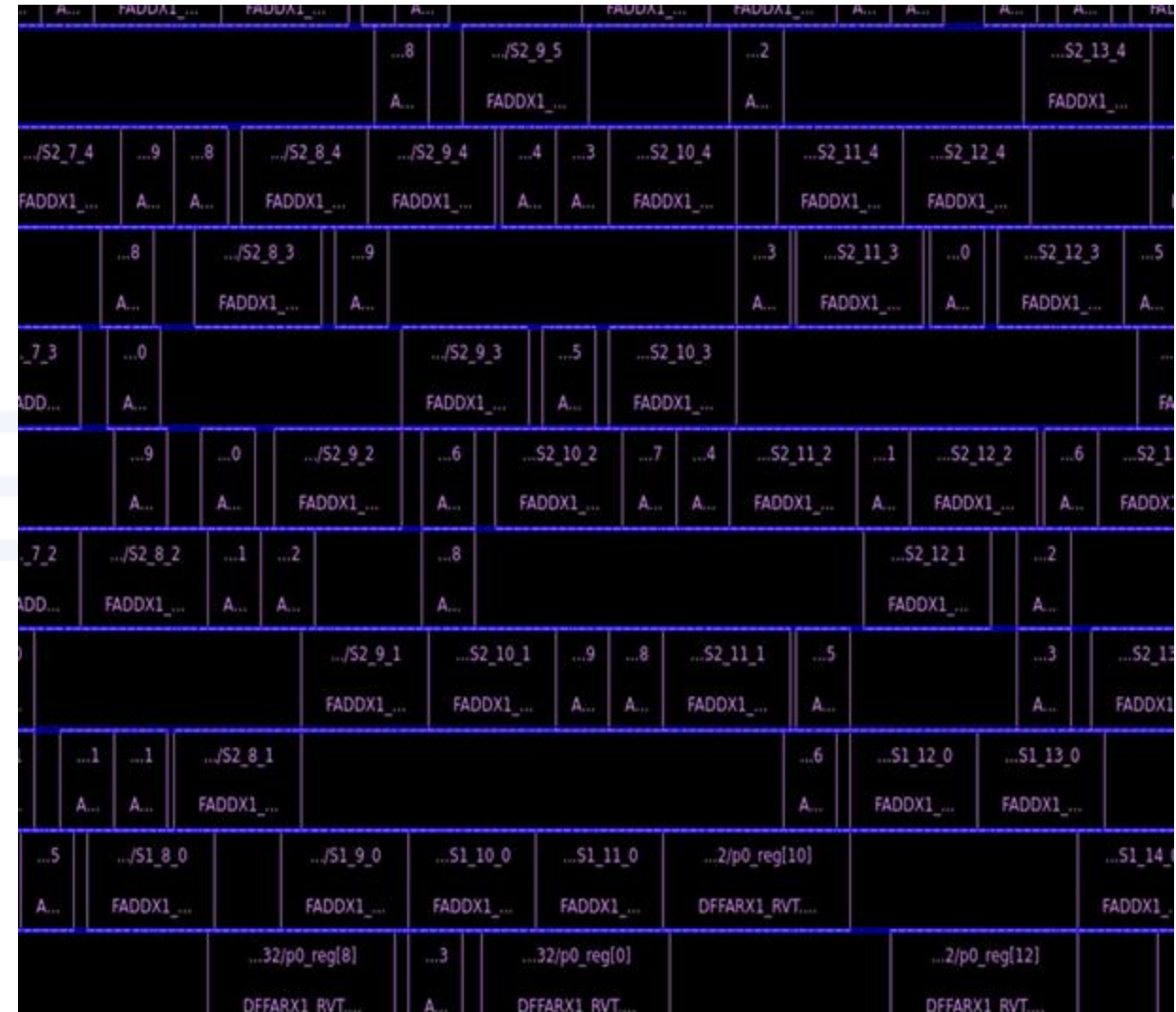
Note:

After step-1, the other steps run iteratively to improve the placement quality considering timing and congestion

- Initial fast placement of standard cells without logical optimization
- Makes a good seed for further placement
- Some cells may not be legally placed and there can be overlapping



- It is the refined placement over coarse-placement
- Legalize the standard cell locations to avoid overlapping and congestion
- Overlapping cells not allowed because this may lead to shorts and other DRC errors



Trial Route

- This is a quick route to estimate wire parasitic and congestion
- Tool does not try for a DRC clean route.
- Based on global/trial route, tool estimates congestion.
- Based on congestion scores, tool make changes to placement.

Optimization

- In this stage, tool performs physical optimization to meet PPA targets along with working towards a database that is routable

Introduction to Placement



Physical Cells



Placement Methodologies



Steps During Placement



Post Placement

Scan Chain



QoR Analysis



Congestion Analysis



Timing Analysis

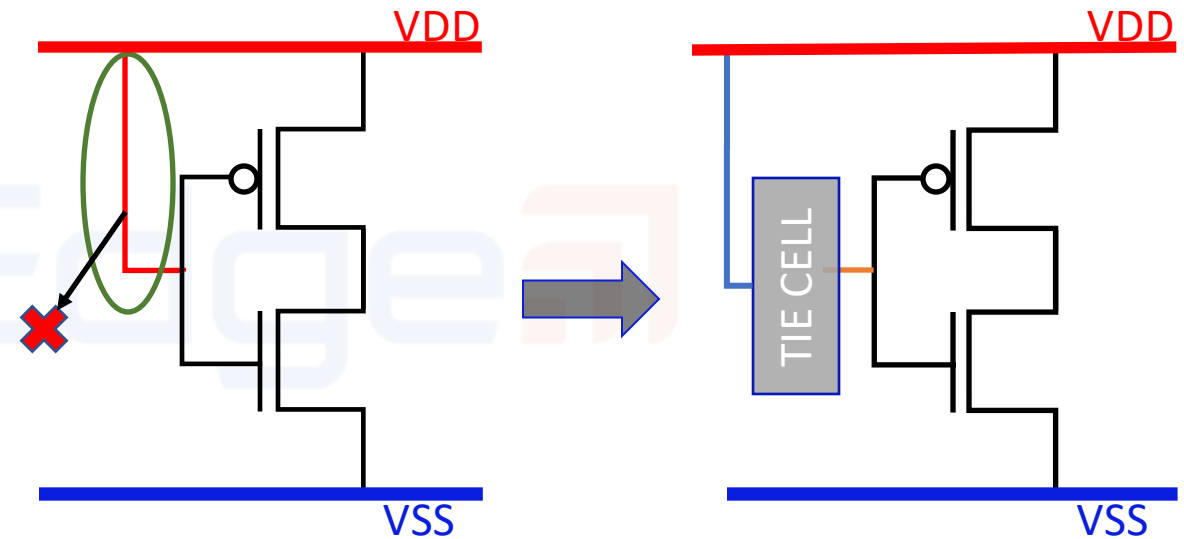


Placement Checks

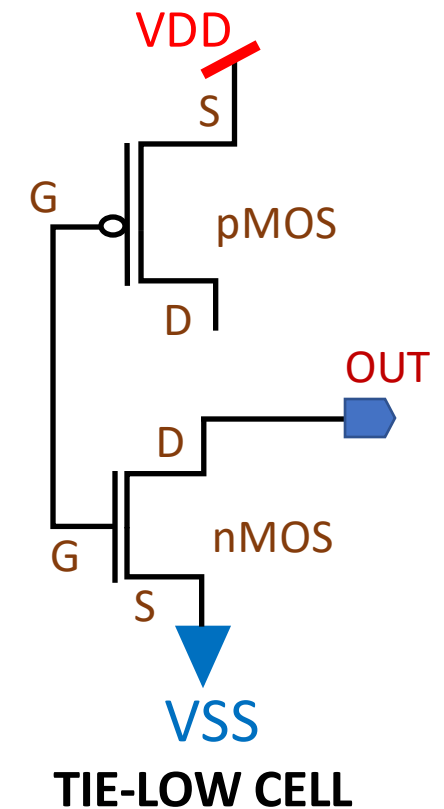
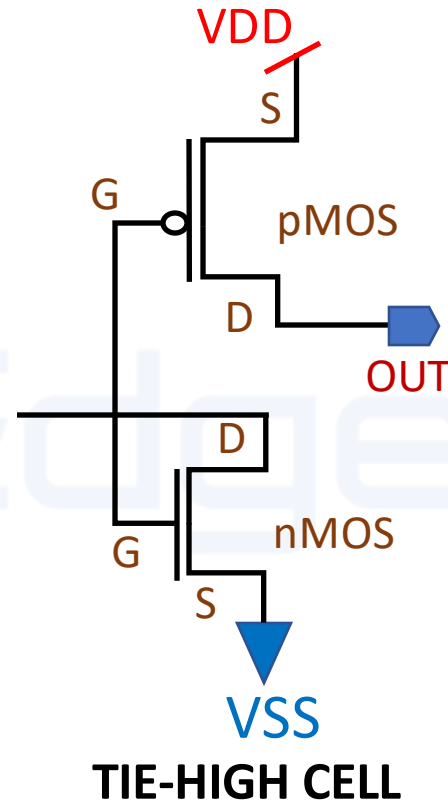


In this section, the importance of the tie cells and scan chain reordering after the placement is discussed with illustrations.

- Some pins of std cells need to be connected to logic-0 (VSS) or logic-1 (VDD) based on netlist.
- In Lower technology nodes, if the gate is connected directly to power/ground the transistor might be turned on/off or damage due to power/ground bounce



- Hence Tie High and Tie Low cells are used to connect Power/Ground to input pins of the standard cells
- They act as low pass RC filters and will filter out any high charge that could have potentially damaged the thin gate oxide



Introduction to Placement



Physical Cells



Placement Methodologies



Steps During Placement



Post Placement



Scan Chain Reordering

QoR Analysis



Congestion Analysis



Timing Analysis



Placement Checks



In this section, the brief introduction to scan chain, scan chain reordering and its applications are explained with examples.

Scan chain reordering is an optimization technique to ensure scan chains are connected in more efficient way – based upon the placement of the flip-flops

Congestion reduction:

- Initially in the RTL / synthesized netlist, Scan chain connections are built without having any placement information.
- Initial placement in the tool will be done without considering Scan chain.
- Then, the scan chains are reordered as per the placement of flops



In Functional mode

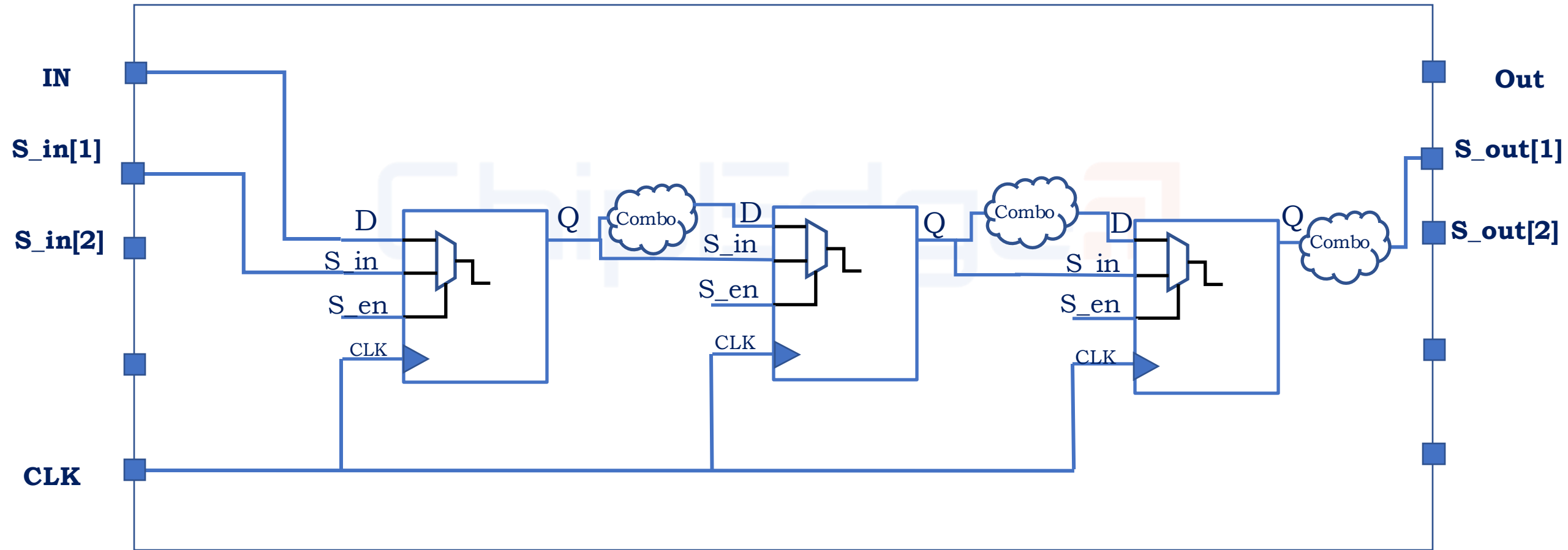
Scan Enable = 0, so input at **D** is transferred to **Q**.

In Test mode/ Scan mode

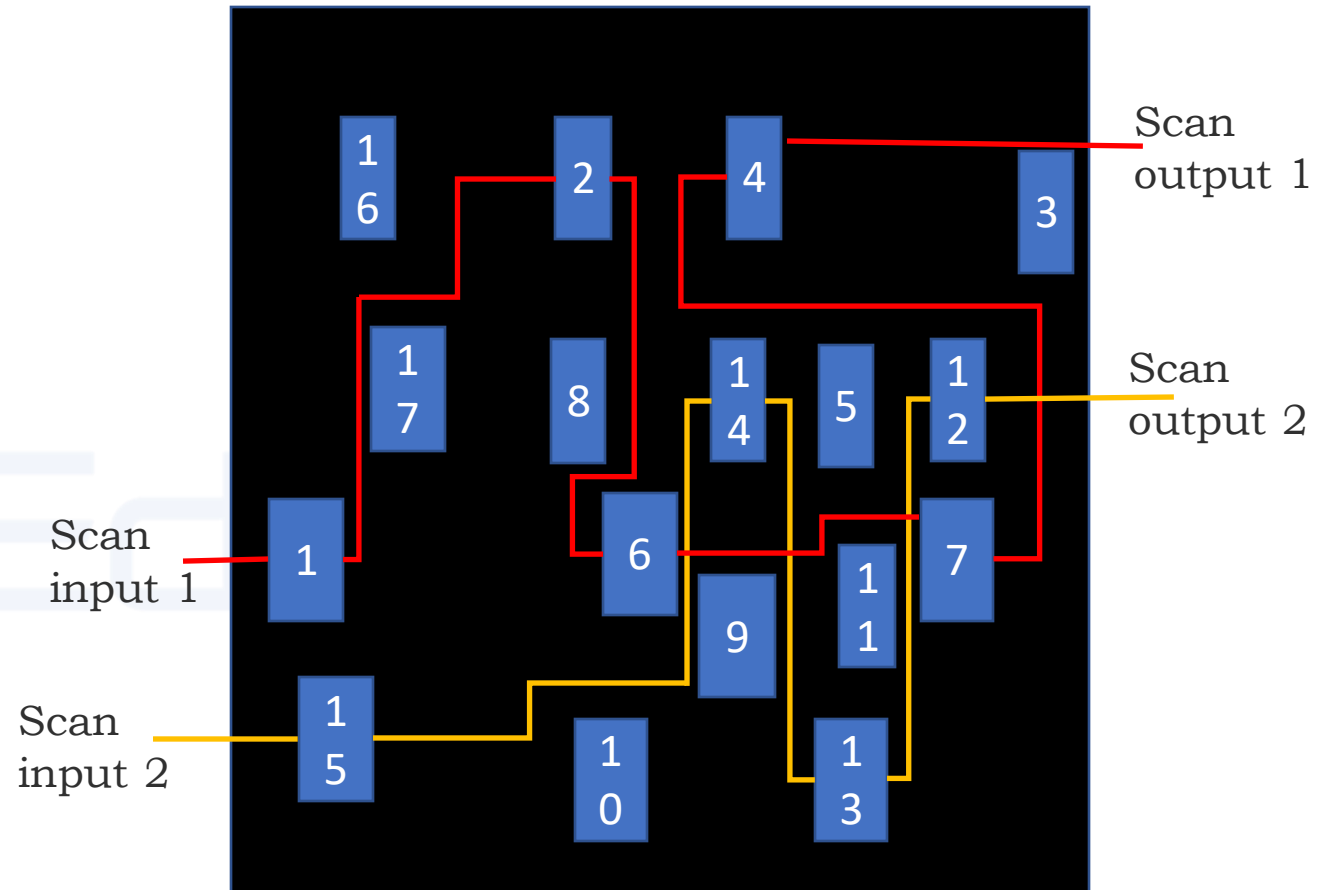
Scan Enable = 1, so input at **S_in** is transferred to **Q**.

What is a Scan chain

Scan chains are used to pass the test vectors to find out any manufacturing defects during testing

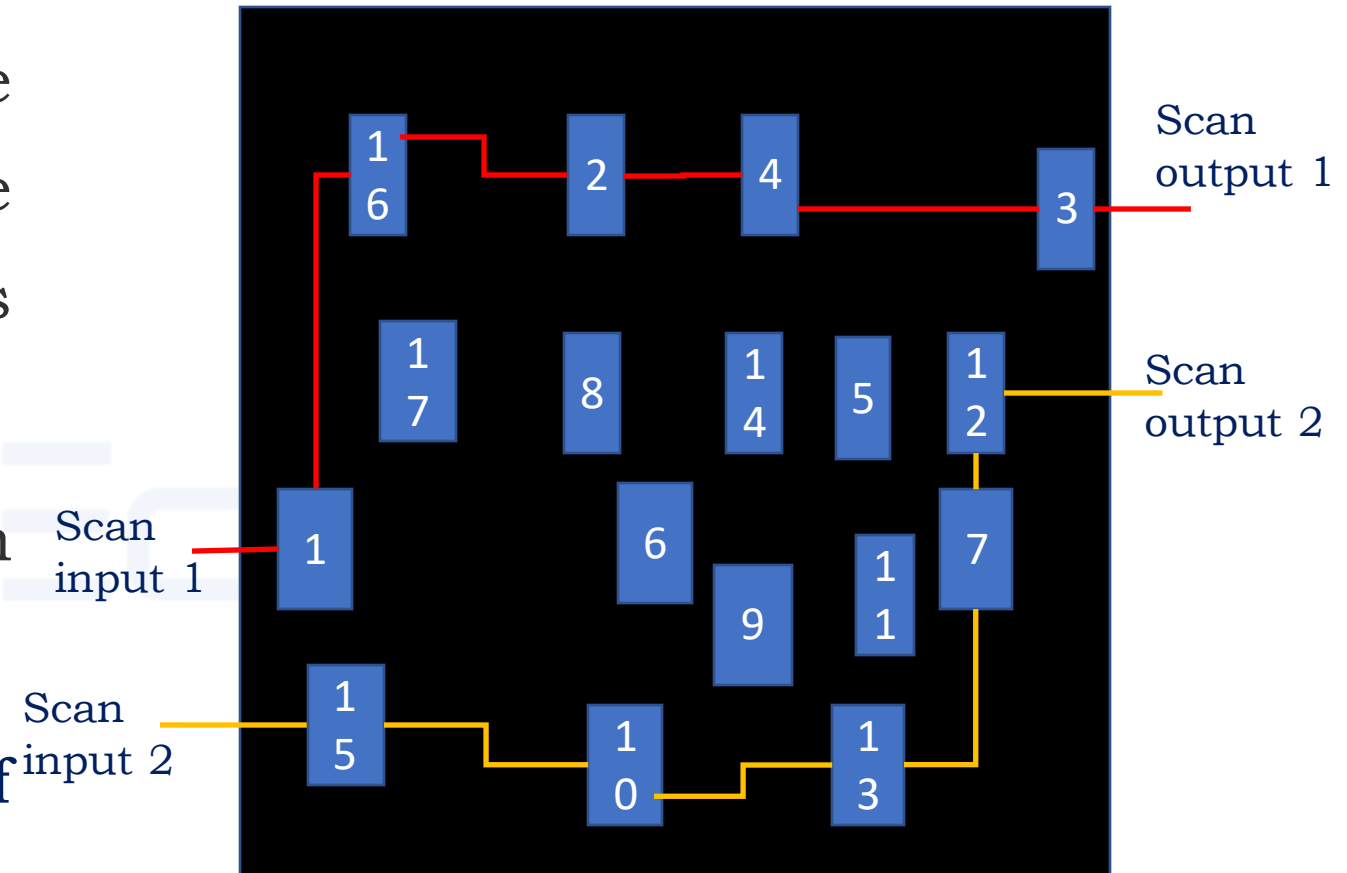


- Scan chain reordering is an optimization technique to ensure scan chains are connected in the more efficient way based upon the placement of the flip-flops
- After placement it might be possible that the two flops stitched are placed apart from each other, so we will face the placement congestion and timing issues.



Scan chain connections after placement.

- SCANDEF file contains the Scan Chain information of the design (from DFT/ Synthesis team).
- The tool disconnects the scan chain in the design.
- After placement, reordering of scan chains is done to reduce congestion.



Scan chain connections after reordering

Introduction to Placement



Physical Cells



Placement Methodologies



Steps During Placement



Post Placement



Scan Chain



QoR Analysis

Congestion Analysis



Timing Analysis



Placement Checks



The brief explanation of QoR reports in terms of Timing and Congestion are explained in this section.

- `report_congestion`

Layer Name	overflow		# GRCs has	max overflow (%)	max overflow
	total	max			
Both Dirs	1340	21	1130	(0.14%)	1
H routing	1057	21	927	(0.22%)	1
V routing	283	17	203	(0.05%)	1

- `report_global_timing`

Setup violations

	Total	reg->reg	in->reg	reg->out	in->out
WNS	-0.60	0.00	0.00	0.00	-0.60
TNS	-56.19	0.00	0.00	0.00	-56.19
NUM	144	0	0	0	144

- In trial route, the block is virtually divided into small boxes (portions) referred as Global Route Cell (GRC).
- Congestion refers to available Routing Tracks vs Required Tracks for Routing the design (Demand vs Supply).
- Technology decides Available Tracks.
 - Number of metal layers (horizontal/vertical).
 - Number of metal lines per um.
- Nets inside Global Route Cell (GRC) decide Required Tracks.
 - Horizontal Congestion
 - Vertical Congestion

Introduction to Placement



Physical Cells



Placement Methodologies



Steps During Placement



Post Placement



Scan Chain



QoR Analysis



Congestion Analysis

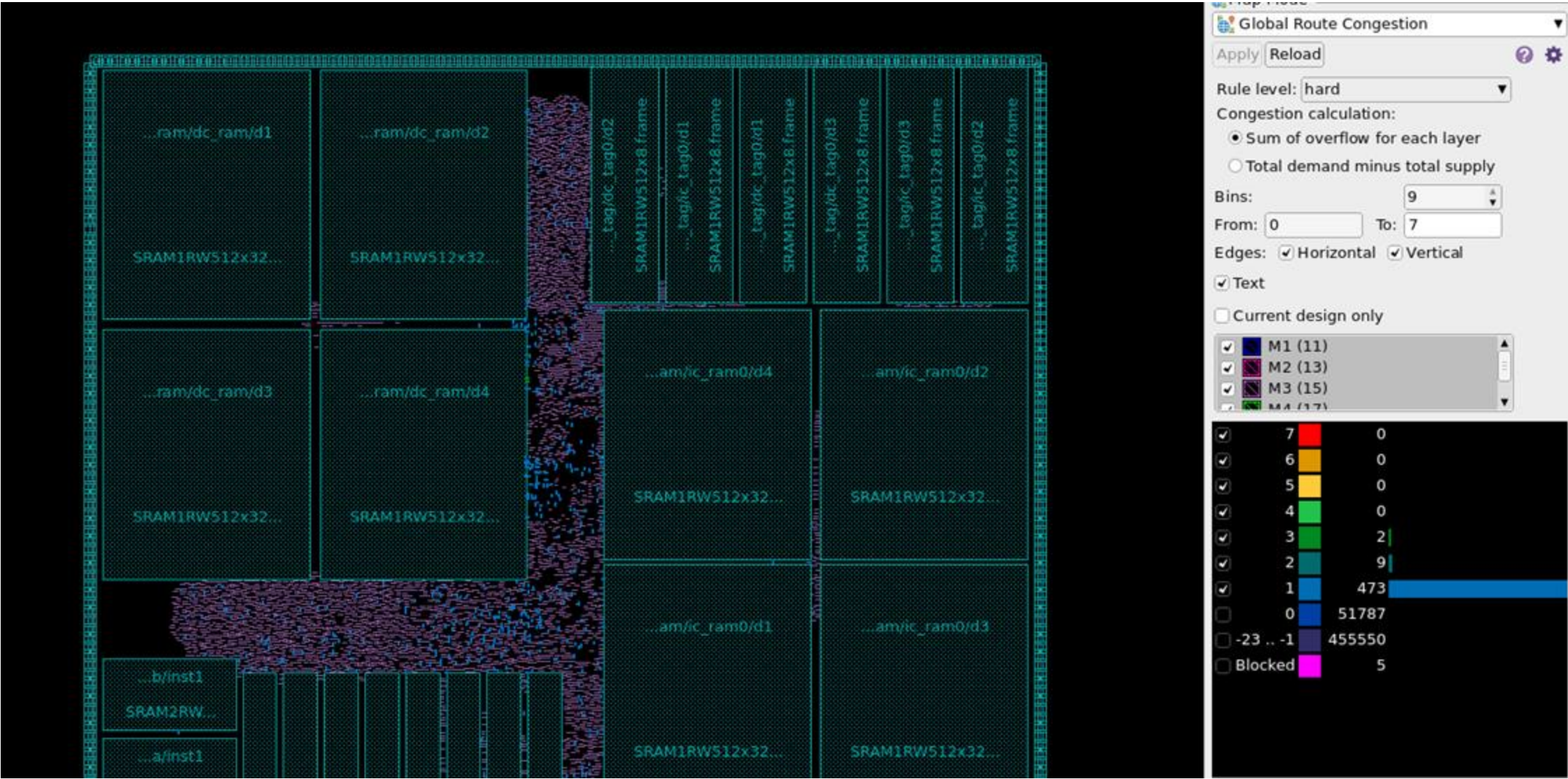
This section explains about Global congestion, Local congestion with examples.

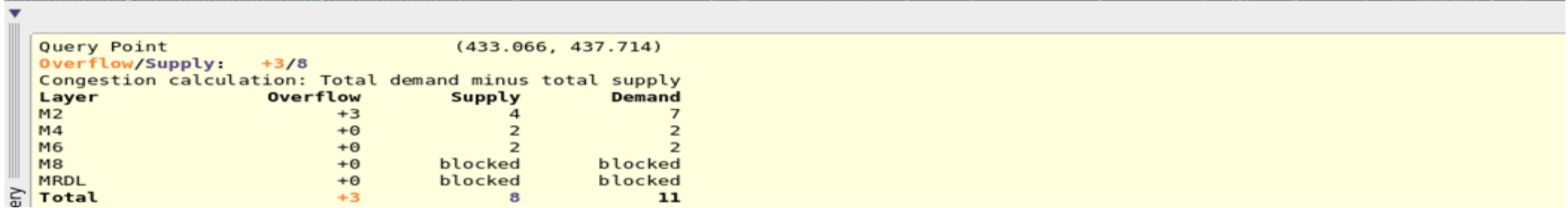
Timing Analysis

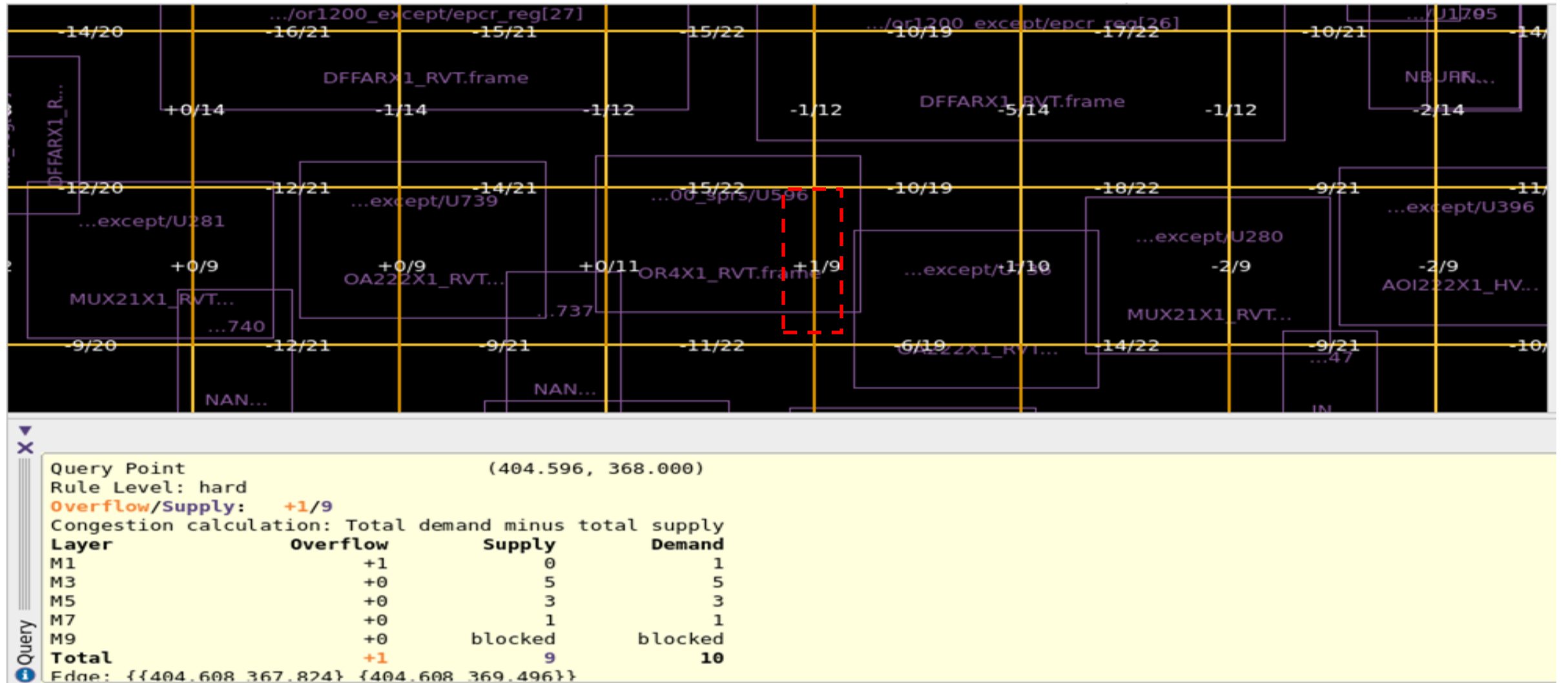


Placement Checks

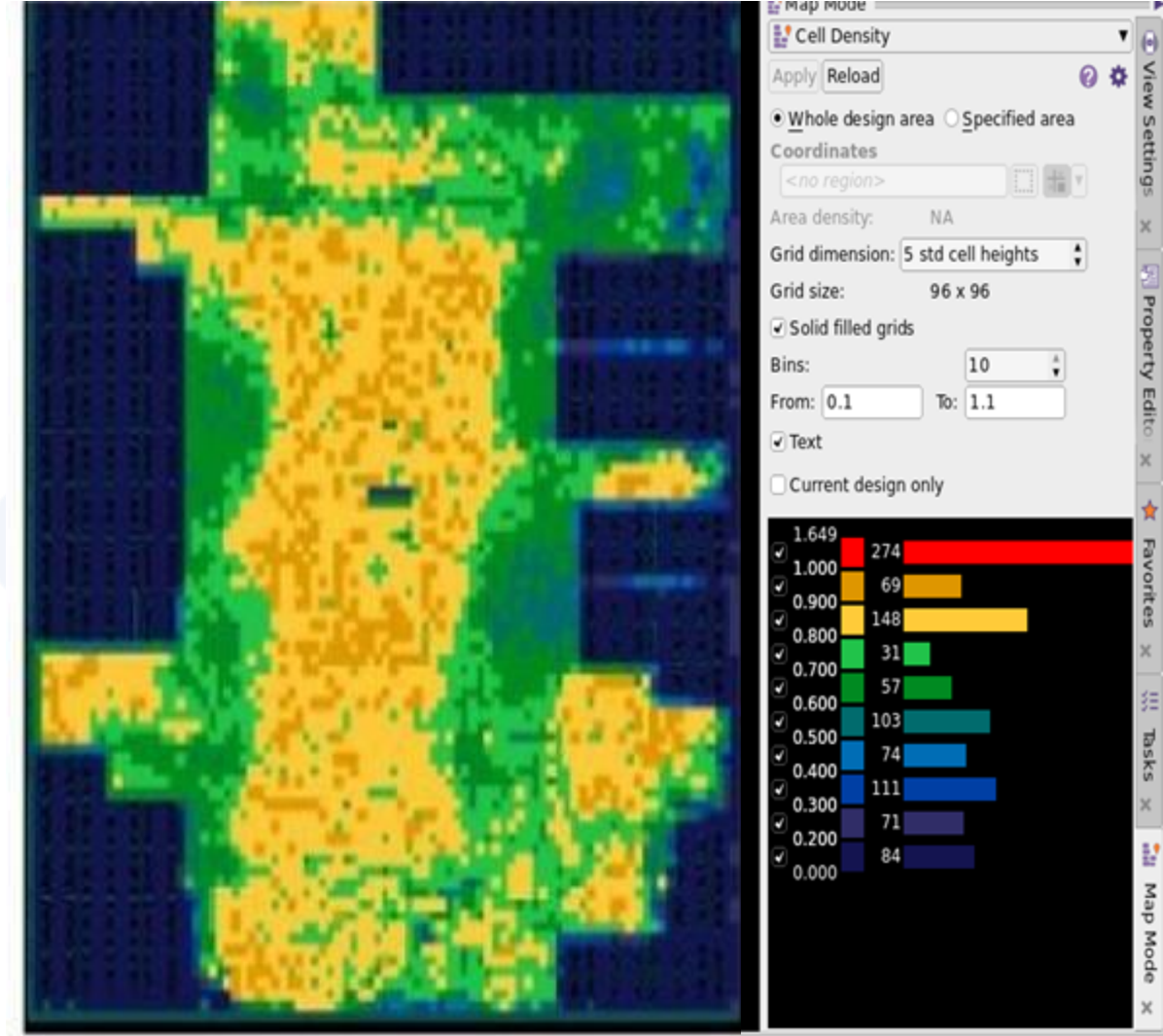




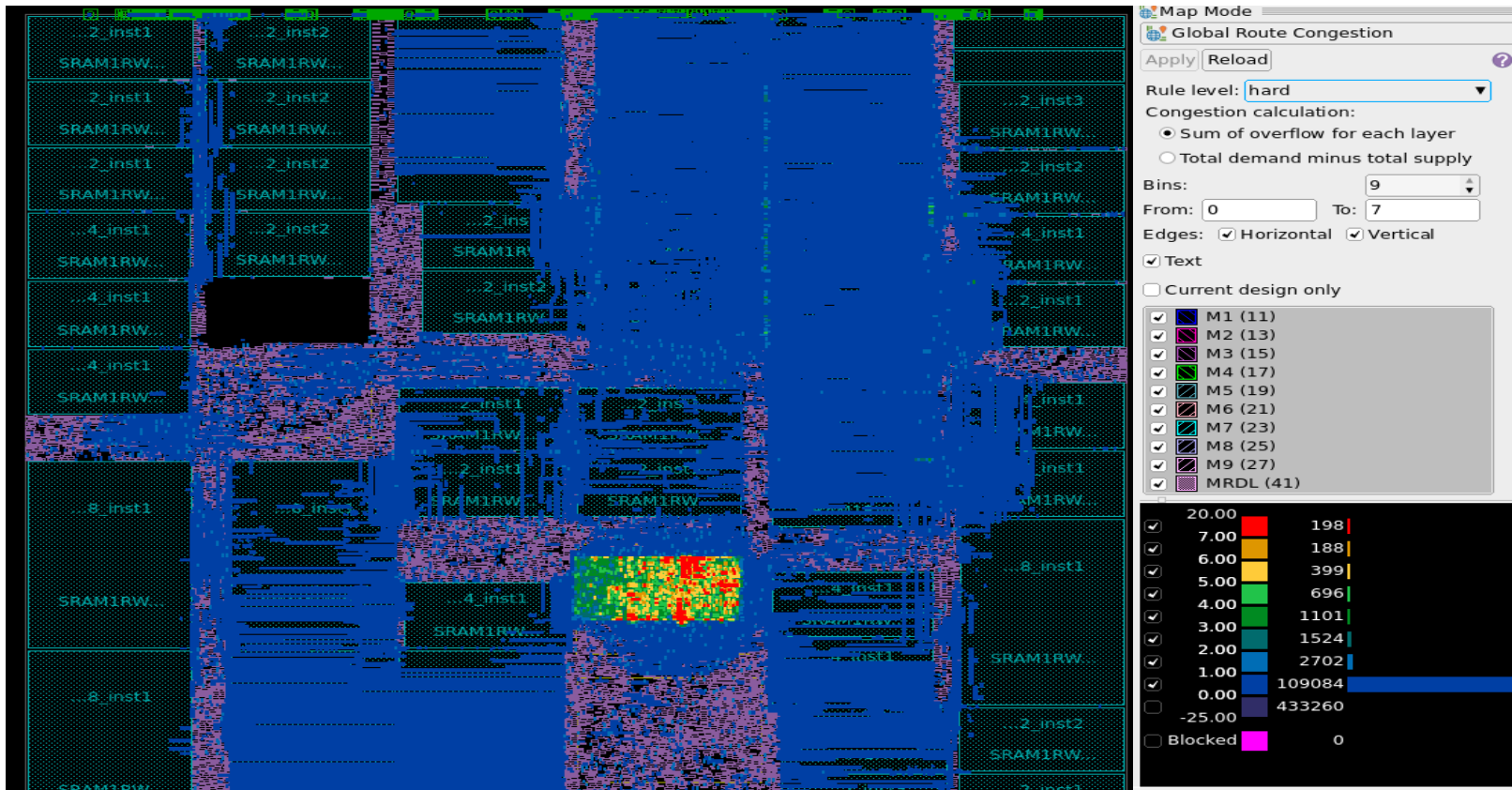




- Global Congestion:
If the congestion is present in the entire block or it is spread over most of the block, it is referred as Global congestion.



Local congestion: If the congestion is present in a small area in any part of the block, it is referred as Local congestion.



- Few options to resolve Global Congestion:
 - ✓ Could be floorplan issues → Might have to redo floorplan based on congestion analysis
 - ✓ If the utilization is very high, need to increase the size of the block.
 - ✓ If it is Horizontal or vertical congestion, make more space in X/Y direction accordingly.
 - ✓ Refer to data-flow diagram and check if the placement matches with it.
 - ✓ Try placement bounds for specific modules.
 - ✓ Tool options to guide the tool to put more effort in resolving congestion.

Local congestion → Few cases

- High cell density
 - ✓ In case of high placement density in few areas leading to congestion
→ partial placement blockages or constraints on max utilization might help
- High pin density (cell pins)
 - ✓ In case of high pin density in few areas, possibly high pin count cells might be clustered → cell based keep outs or cell padding might help

- High Port density (I/O Ports)
 - ✓ Spread the ports if possible.
 - ✓ Partial placement blockages.
- If modules splitting leading to more congestion
 - ✓ Then in this scenario, Placement bounds might help.

- Placement Bounds are used to restrict the placement of specific cells to a specific region.
- Used to achieve better placement, to optimize congestion and timing.
- There are three types of Placement Bound.
 - ✓ Soft placement bound
 - ✓ Hard placement bound
 - ✓ Exclusive placement bound

Introduction to Placement



Physical Cells



Placement Methodologies



Steps During Placement



Post Placement



Scan Chain



QoR Analysis



Congestion Analysis



Timing Analysis

Placement Checks



In this section the QoR timing analysis and fixing the timing issues are discussed in detail.

- Timing metrics analyzed at this stage are

- ✓ Setup Timing

If timing is very bad / broken, need to review, make some corrections and redo the placement step.

- ✓ DRV (Design Rule Violations - Logic DRC)

- Max_Transition violation:
- Max_Capacitance violation:
- Max_Fanout violation:

Ideally both of these should be clean after placement. If any big errors, then review is needed, and placement needs to be redone.

- Path grouping
- Placement with timing effort high
- Placement bounds for few critical modules.
- Solve Congestion issues which are leading to timing issues

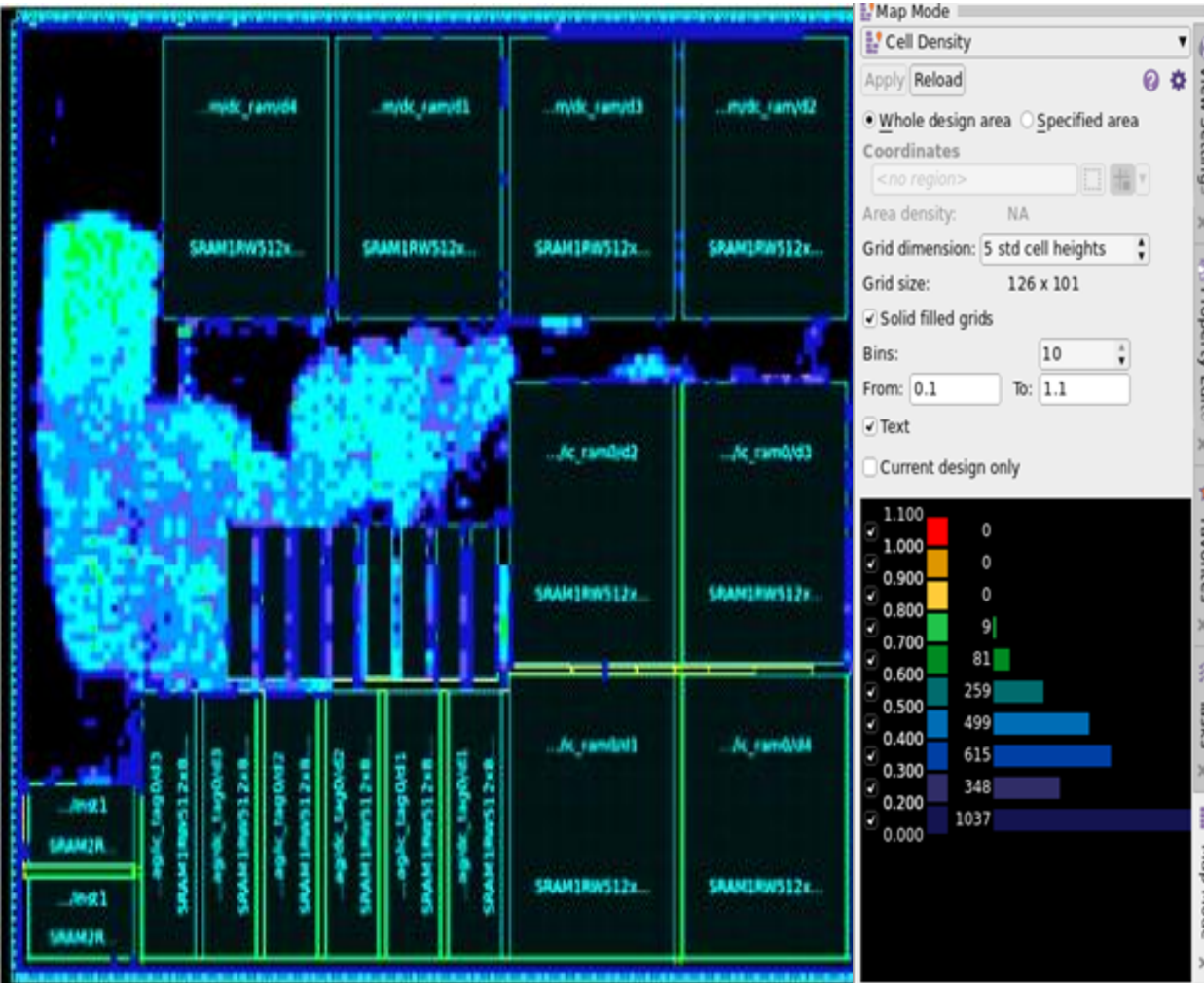
Introduction to Placement	✓
Physical Cells	✓
Placement Methodologies	✓
Steps During Placement	✓
Post Placement	✓
Scan Chain	✓
QoR Analysis	✓
Congestion Analysis	✓
Timing Analysis	✓
Placement Checks	

In this section, factors required for checking the quality of the design are explained.

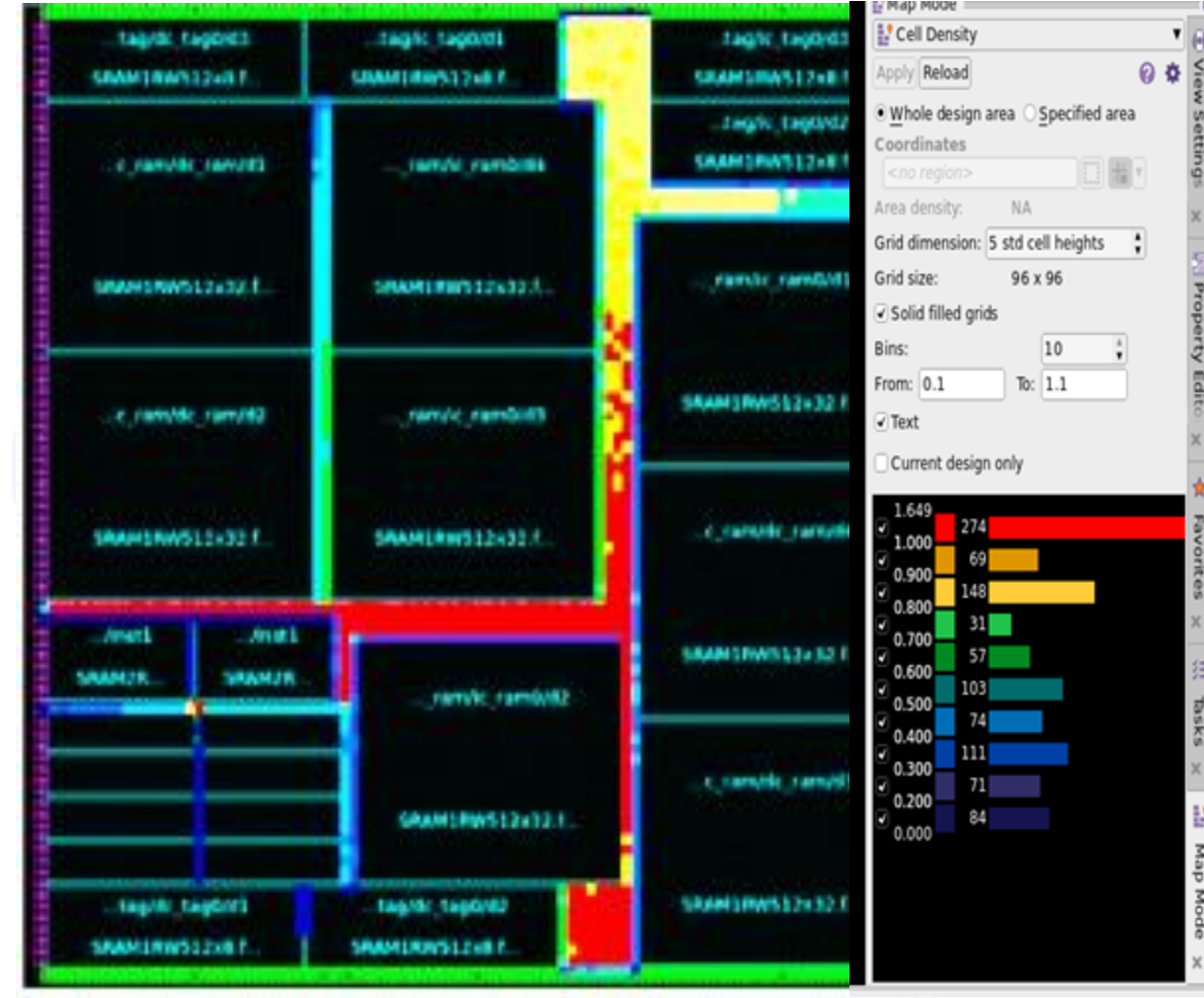
- Timing
 - ✓ `report_qor -summary`
 - ✓ Ideally design should not have any DRVs and setup timing violations.
- Congestion
 - ✓ `report_congestion`
 - ✓ Check congestion map from the GUI.
 - ✓ Should have minimal congestion and design should be routable.

- Legalization Checks
 - ✓ `check_legality`
 - ✓ Unplaced Cells should be 0 & Cells Overlap should be 0
- Utilization
 - ✓ If high jump in utilization from pre-place to post-place, check for reasons
- Cell density maps
 - ✓ Should not have many areas with $> 80\%$ cell density

Good Placement



Bad Placement



- Physical Cells
- Placement Methodologies
- Steps During Placement
- Post Placement
- QoR Analysis
- Congestion Analysis
- Timing Analysis
- Placement Checks



TCL Scripting

- Tcl - **Tool Command Language**

Tcl is a string-based command language. The language has only a few fundamental constructs and relatively little syntax, which makes it easy to learn.

- ❑ An interpreted programming language
 - Build on-the-fly commands, procedures
 - Platform-independent
 - Easy to use for building GUIs

➤ **Application Development**

Using TCL, complex applications can be developed quickly and easily.

➤ **Embeddable**

TCL scripting can be easily embedded into other applications written in C or C++

➤ **Extendible**

New functionality can be added easily by defining new TCL commands using TCL or C/C++

➤ **Easy to Learn**

TCL has basic syntax and few basic Language rules.

Note: TCL is case Sensitive

Why TCL ??

- ❖ **Rapid development**-Tcl is that it gets their job done faster.
- ❖ **Graphical user interfaces-Tcl provides facilities for creating GUIs that are incredibly simple yet remarkably powerful**
- ❖ **Cross-platform applications -Tcl/Tk runs on Windows, Macintosh, and nearly every imaginable Unix platform**
- ❖ EDA tools are developed to be compatible with TCL language constructs.
- ❖ Much easier and simple syntax compared to other languages
- ❖ Tcl is an ideal language to use for automated hardware and software testing
- ❖ Tcl is open source — freely available

- Tcl is interpreted, it can be run interactively through its shell by command, **tclsh**
- When Tcl is run interactively, the system responds to each command that is entered. For example, shown below, You can experiment with **tclsh** by simply opening a terminal and entering the command **tclsh**

Example

```
$ tclsh
```

```
% set a 35
```

```
35
```

```
% expr 35 * $a
```

```
1225
```

```
% puts "35 * a is: [ expr 35 * $a ]"
```

```
35 * a is: 1225
```

```
% exit
```

```
$
```

The basic syntax for a Tcl command is:

command arg1 arg2 arg3 ...

The command is either the name of a built-in command or a Tcl procedure. White space (i.e., spaces or tabs) is used to separate the command name and its arguments, and a newline (i.e., the end of line character) or semicolon is used to terminate a command

The behavior of the Tcl command processor can be summarized in three basic steps:

- Argument grouping.
- Value substitution of nested commands, variables, and backslash escapes.
- Command invocation. It is up to the command to interpret its arguments.

Hello, World!

Example

puts stdout {Hello, World!}

=> Hello, World!

- Tcl scripts:
 - Sequence of **commands**.
 - Commands separated by newlines, semi-colons.
- Tcl command:
 - One or more **words** separated by white space.
 - First word is **command name**, others are **arguments**.
 - Returns **string result**.

Examples:

```
set myName Saul
```

```
puts "My Name is $myName"
```

Summary of Tcl Command Syntax

- Command: words separated by whitespace
- First word is a function, others are arguments
- Only functions apply meanings to arguments
- \$ causes variable interpolation
- [] causes command interpolation
- “” prevents word breaks
- { } prevents all interpolation
- \ escapes special characters
- TCL HAS NO GRAMMAR!

Examples:

- puts "hello (world)) "
- puts "hello " world "" "
- puts "hello \"world \"\" \" "
- puts "hello "world!" "
- puts { hello world }
- puts { hello (world)) }
- puts { hello " world "" }
- puts { hello } world! }

The set command is used to assign a value to a variable. It takes two arguments: The first is the name of the variable, and the second is the value.

When you want to use a variable, you can directly assign a value to it, there is no step of declaration in Tcl.

Ex: `set myVariable 45`

- We can access any variable by using \$. (That variable should be already defined)

- `$myVariable`

Accessing the variables, Word Grouping & Printing

Use “” or {} for word Grouping

- Ex: `puts “Hello World”`
- EX: `puts { Hello World}`

Variables and Arguments

Items of data passed to a script from the command line are known as **arguments**.

For example, take the simple script as shown below

```
puts [expr 3 + 2]
```

The method by which numbers can be passed into, and used by a script, is as follows.

```
argc argv argv0
```

All Tcl scripts have access to three predefined variables.

\$argc - number items of arguments passed to a script.

\$argv - list of the arguments.

\$argv0 - name of the script.

Example:

Create a text file named args.tcl that contains the following commands.

```
# If no command line arguments are passed perform no actions
if {$argc > 0} {
# Print out the filename of the script
puts "The name of the script is: $argv0"
# Print out the count of the arguments passed
puts "Total count of arguments passed is: $argc"
# Print out a list of the arguments
puts "The arguments passed are: $argv"
# Using the List Index of argv print a specific argument
puts "The first argument passed was [lindex $argv 0]"
}
```

Example (contd)

After you have created the file invoke the script with the following command line:

```
Tclsh args.tcl ONE 2 3
```

The name of the script is: args.Tcl

Total count of arguments passed is: 3

The arguments passed are: ONE 2 3

The first argument passed was ONE

%

How it works...

As you can see, the script accepts any number of arguments and using the Tcl global variables allows access to the arguments passed as either a list or individual values. Keep in mind that when passing control characters, they must be escaped using the backslash character.

Example (contd...)

Invoke the script with the following command line:

```
tclsh args.Tcl \home \etc
```

The name of the script is: args.Tcl

Total count of arguments passed is: 2

The arguments passed are home etc

The first argument passed was home

%

In the above example you can see that the backslash characters are removed. This is NOT done by Tcl, but rather by the shell from which Tcl was invoked.

```
tclsh args.Tcl \\home \\etc
```

The name of the script is: args.Tcl

Total count of arguments passed is: 2

The arguments passed are: {\home} {\etc}

The first argument passed was \home %

By adding the escape character, the backslash characters are retained, and curly braces have been appended to define the values as strings.

- Syntax: *\$varName*
- Variable name is letters, digits, underscores.
 - This is a little white lie, actually.
- May occur anywhere in a word.

Sample command Result

set b 66	66
set a b	b
set a \$b	66
set a \$b+\$b+\$b	66+66+66
set a \$b.3	66.3
set a \$b4	<i>no such variable</i>

- Syntax: *[script]*
- Evaluate script, substitute result.
- May occur anywhere within a word.

<u>Sample command</u>	<u>Result</u>
-----------------------	---------------

set b 8	8
---------	---

set a [expr \$b+2]	10
--------------------	----

set a "b-3 is [expr \$b-3]"	b-3 is 5
-----------------------------	----------

- Words break at white space and semi-colons, except:
 - Double-quotes prevent breaks:
set a 4; set y 5
set a "x is \$x; y is \$y"
-> x is 4; y is 5
 - Curly braces prevent breaks and substitutions:
set a {[expr \$b*\$c]}
->[expr \$b*\$c]
 - Backslashes quote special characters:
set a word\ with\ \\$\ and\ space
->word with \$ and space

- Backslashes can escape newline (continuation)

```
set aLongVariableNameIsUnusual \  
  "This is a string"
```

-> This is a string

- Substitutions don't change word structure:

```
set a "two words"  
set b $a
```

-> two words

- The # is the comment command
- Tcl parsing rules apply to comments as well

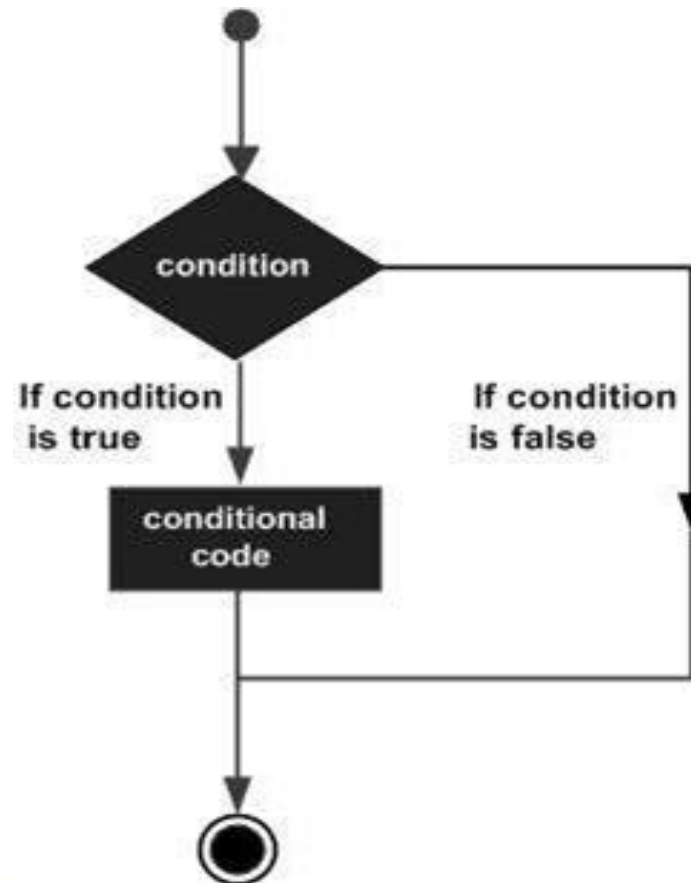
set a 22; set b 33 <- OK

this is a comment <- OK

set a 22 # same thing? <- Wrong!

set a 22 ;# same thing <- OK

- Decision making structures require that the programmer specifies one or more conditions to be evaluated or tested by the program, along with a statement or statements to be executed if the condition is determined to be true, and optionally, other statements to be executed if the condition is determined to be false.



SL.No	Statement & Description
1	<u>if statement</u> An 'if' statement consists of a Boolean expression followed by one or more statements.
2	<u>if...else statement</u> An 'if' statement can be followed by an optional 'else' statement, which executes when the Boolean expression is false.
3	<u>nested if statements</u> You can use one 'if' or 'else if' statement inside another 'if' or 'else if' statements.
4	<u>switch statement</u> A switch statement allows a variable to be tested for equality against a list of values.
5	<u>nested switch statements</u> You can use one switch statement inside another switch statements.

- An if statement can be followed by an optional else statement, which executes when the Boolean expression is false.

Syntax

The syntax of an 'if...else' statement in Tcl language is –

```
if {boolean_expression}
{
# statement(s) will execute if the boolean expression is
true
} else { # statement(s) will execute if the boolean
expression is false
}
```

Examples:

```
#tclsh
```

```
set a 100
```

```
#check the boolean condition
if {$a < 20 } {
    #if condition is true then
    print the following
    puts "a is less than 20"
} else {
    #if condition is false then
    print the following
    puts "a is not less than 20"
}
puts "value of a is : $a"
```

OUTPUT:
a is not less than 20;
value of a is : 100

The if...else if...else Statement

- An 'if' statement can be followed by an optional else if...else statement, which is very useful to test various conditions using single if...else if statement.

Syntax

The syntax of an 'if...else if...else' statement in Tcl language is –

```
if {boolean_expression 1} {  
# Executes when the boolean expression 1 is true  
} elseif {boolean_expression 2} {  
# Executes when the boolean expression 2 is true  
} elseif {boolean_expression 3} {  
# Executes when the boolean expression 3 is true  
} else {  
# executes when the none of the above condition is  
true  
}
```

Examples:

```
#tclsh
set a 100
#check the boolean condition
if { $a == 10 } {
    # if condition is true then print the following
    puts "Value of a is 10"
} elseif { $a == 20 } {
    # if else if condition is true
    puts "Value of a is 20"
} elseif { $a == 30 } {
    # if else if condition is true
    puts "Value of a is 30"
} else {
    # if none of the conditions is true
    puts "None of the values is matching"
}
puts "Exact value of a is: $a"
```

OUTPUT:
None of the values is
matching
Exact value of a is: 100

It is always legal in Tcl to nest if-else statements, which means you can use one if or else if statement inside another if or else if statement(s).

Syntax

The syntax for a nested if statement is as follows –

```
if { boolean_expression 1 } {  
# Executes when the boolean expression  
1 is true if {boolean_expression 2} {  
# Executes when the boolean expression  
2 is true } }
```

```
#tclsh
set a 100
set b 200
# check the boolean condition
if { $a == 100 } {
    # if condition is true then check the following
    if { $b == 200 } {
        #if condition is true then print the following
        puts "Value of a is 100 and b is 200"
    }
}
puts "Exact value of a is : $a"
puts "Exact value of b is : $b"
```

OUTPUT:

Value of a is 100 and b is 200
Exact value of a is : 100
Exact value of b is : 200

Switch Statement

- A switch statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each switch case.

Syntax

The syntax for unquoted switch statement in Tcl language is as follows –

```
switch switchingString matchString1 {body1} matchString2 {body2} ...
matchStringn {bodyn}
```

```
switch switchingString {
    matchString1 {
        body1
    }
    matchString2 {
        body2
    }
    ...
    matchStringn {
        bodyn
    }
}
```

Example unquoted version

```
#tclsh  
set grade C;  
switch $grade  
A { puts "Well done!" } B { puts "Excellent!" } C { puts "You  
passed!" } F { puts "Better try again" } default { puts  
"Invalid grade" }  
puts "Your grade is $grade"
```

OUTPUT:

You passed!

Your grade is C

Example quoted version

```
#tclsh
set grade B;
switch $grade {
  A {
    puts "Well done!"
  }
  B {
    puts "Excellent!"
  }
  C {
    puts "You passed!"
  }
}
```

```
F {
  puts "Better try again"
}
default {
  puts "Invalid grade"
}
puts "Your grade is $grade"
```

OUTPUT:
Excellent!
Your grade is B

```
puts " Welcome to Basic interactive script"
```

```
while {1} {  
  puts " \nCommand: "  
  gets stdin line  
  if {$line == ""} { continue }  
  switch [lindex $line 0] {  
    dir - ls { puts [glob *] }  
    quit - exit { break }  
    default { puts "Error : unrecognized command " }  
  }  
}  
  
puts " Thank you "
```

Nested Switch Statement

It is possible to have a switch as part of the statement sequence of an outer switch. Even if the case constants of the inner and outer switch contain common values, no conflicts will arise.

Syntax

The syntax for a nested switch statement is as follows –

```
switch switchingString {
  matchString1 {
    body1
    switch switchingString {
      matchString1 {
        body1 }
      matchString2 {
        body2 } ...
      matchStringn {
        bodyn }
    } }
  matchString2 { body2 }
```

Example

```
#tclsh
set a 100
set b 200
switch $a {
  100 {
    puts "This is part of outer switch"
    switch $b {
      200 {
        puts "This is part of inner
switch!"
      }
    }
  }
}
puts "Exact value of a is : $a"
puts "Exact value of a is : $b"
```

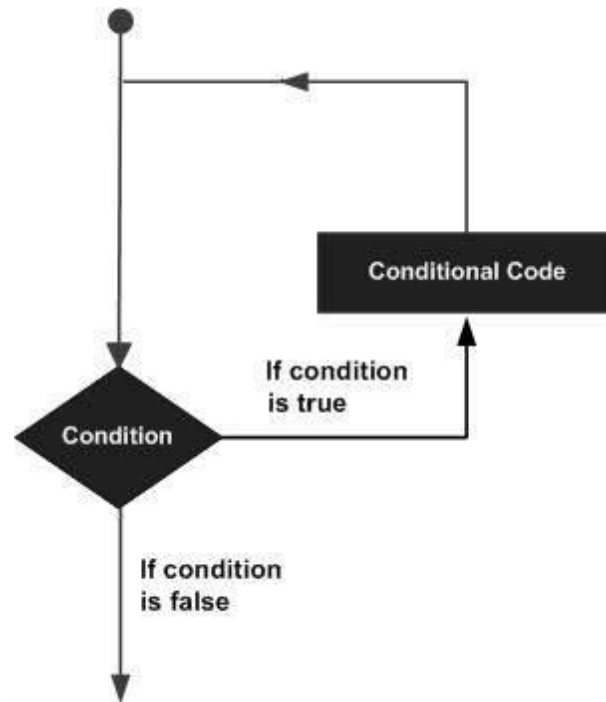
OUTPUT:

This is part of outer switch This
is part of inner switch!

Exact value of a is : 100

Exact value of a is : 200

- There may be a situation, where you need to execute a block of code several number of times. In general, statements are executed sequentially: The first statement in a function is executed first, followed by the second, and so on.
- A loop statement allows us to execute a statement or group of statements multiple times and following is the general form of a loop statement in most of the programming languages –



- [while loop](#) Repeats a statement or group of statements while a given condition is true. It tests the condition before executing the loop body.
- [for loop](#) Executes a sequence of statements multiple times and abbreviates the code that manages the loop variable.
- [nested loops](#) You can use one or more loop inside any another while, for or do..while loop.

```
while { condition } {  
    body }
```

```
for { var initialization } {condition}  
    {increament} {  
        Body }
```

```
Ex: for { set k 0 } { $k< 10 } { incr k } {  
    puts $k }
```

A while loop statement in Tcl language repeatedly executes a target statement as long as a given condition is true.

Syntax

The syntax of a while loop in Tcl language is –

```
while {condition} {  
statement(s) }
```

Write a Program to print numbers multiple of 4 between 1 to 20 using while loop

```
set i 1
while {$i < 20} {
  if {$i%4==0} {
    puts $i
  }
  incr i
}
```

Write a Program to accept a number and print its reverse using while loop

```
puts "enter a number"
gets stdin n
while {$n>0} {
set rem [expr $n%10]
puts $rem
set n [expr $n/10]
}
```

Write a Program to accept a number and print sum of its digits

```
puts "Enter a number "
gets stdin n
set sum 0
while {$n>0} {
  set rem [expr $n%10]
  set sum [expr $sum+$rem]
  set n [expr $n/10]
}
puts "Sum of digits : $sum"
```

```
#tclsh
```

```
set a 10
```

```
#while loop execution  
while { $a < 20 } {  
    puts "value of a: $a"  
    incr a  
}
```

OUTPUT:

value of a: 10

value of a: 11

value of a: 12

value of a: 13

value of a: 14

value of a: 15

value of a: 16

value of a: 17

value of a: 18

value of a: 19

A for loop is a repetition control structure that allows you to efficiently write a code that needs to be executed for a specific number of times.

Syntax

The syntax of a for loop in Tcl language is –

```
for {initialization} {condition}  
{increment} { statement(s) ;  
}
```


Example:

```
#tclsh
```

```
# for loop execution
```

```
for { set a 10} {$a < 20} {incr  
a} {  
    puts "value of a: $a"  
}
```

OUTPUT:

```
value of a: 10  
value of a: 11  
value of a: 12  
value of a: 13  
value of a: 14  
value of a: 15  
value of a: 16  
value of a: 17  
value of a: 18  
value of a: 19
```

Tcl allows to use one loop inside another loop. The following section shows a few examples to illustrate the concept.

Syntax

The syntax for a **nested for** loops statement in Tcl language is as follows –

```
for{initialization}{condition} {increment}{  
  for {initialization} {condition} {increment}{ statement(s);  
  }  
  statement(s);  
}
```

The syntax for a nested while loop statement in Tcl language is as follows –

```
while {condition} { while  
  {condition} { statement(s);  
  }  
  statement(s);  
}
```

Example:

```
#tclsh

set j 0;
for {set i 2} {$i<100} {incr i} {
    for {set j 2} {$j <= [expr $i/$j]} {incr j} {
        if { [expr $i%$j] == 0 } {
            break
        }
    }
    if {$j >[expr $i/$j]} {
        puts "$i is prime"
    }
}
```

OUTPUT:

```
2 is prime
3 is prime
5 is prime
7 is prime
11 is prime
13 is prime
17 is prime
19 is prime
23 is prime
29 is prime
31 is prime
37 is prime
41 is prime
43 is prime
47 is prime
53 is prime
59 is prime
61 is prime
67 is prime
71 is prime
73 is prime
79 is prime
83 is prime
89 is prime
97 is prime
```

- Loop control statements change execution from its normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed.

Control Statement & Description

- [break statement](#) Terminates the loop or switch statement and transfers execution to the statement immediately following the loop or switch.
- [continue statement](#) Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.

The break statement in Tcl language is used for terminating a loop. When the break statement is encountered inside a loop, the loop is immediately terminated and program control resumes at the next statement following the loop.

If you are using nested loops (i.e., one loop inside another loop), the break statement will stop the execution of the innermost loop and start executing the next line of code after the block.

Syntax

The syntax for a break statement in Tcl is as follows –

```
break;
```

Example

```
#tclsh
```

```
set a 10
```

```
# while loop execution
```

```
while {$a < 20} {
```

```
    puts "value of a: $a"
```

```
    incr a
```

```
    if { $a > 15} {
```

```
        # terminate the loop using break statement
```

```
        break
```

```
    }
```

```
}
```

OUTPUT:

value of a: 10

value of a: 11

value of a: 12

value of a: 13

value of a: 14

value of a: 15

- The **continue** statement in Tcl language works somewhat like the **break** statement. Instead of forcing termination, however, **continue** forces the next iteration of the loop to take place, skipping any code in between.
- For the **for** loop, **continue** statement causes the conditional test and increment portions of the loop to execute. For the **while** loop, **continue** statement passes the program control to the conditional tests.

Syntax

The syntax for a continue statement in Tcl is as follows –

```
continue;
```

Example

```
#tclsh
```

```
set a 10
# do loop execution
while { $a < 20 } {
    if { $a == 15 } {
        #skip the iteration
        incr a
        continue
    }
    puts "value of a: $a"
    incr a
}
```

OUTPUT:

```
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 16
value of a: 17
value of a: 18
value of a: 19
```


A loop becomes infinite loop if a condition never becomes false. The **while** loop is traditionally used for this purpose. You can make an endless loop by leaving the conditional expression as 1.

```
while {1}
{ puts "This loop will run
  forever." }
```

- List is one of the basic data-type available in Tcl. It is used for representing an ordered collection of items. It can include different types of items in the same list. Further, a list can contain another list.
- An important thing that needs to be noted is that these lists are represented as strings completely and processed to form individual items when required.

The general syntax for list is given below –

```
set listName { item1 item2 item3 .. itemn }  
# or  
set listName [list item1 item2 item3]  
# or  
set listName [split "items separated by a character"  
split_character]
```

Creating a list

```
#tclsh
```

```
set colorList1 {red green blue}  
set colorList2 [list red green blue]  
set colorList3 [split  
"red_green_blue"_ ]  
puts $colorList1  
puts $colorList2  
puts $colorList3
```

OUTPUT:

```
red green blue  
red green blue  
red green blue
```

Appending Item to a List

The syntax for appending item to a list is given below –

```
append listName split_character value  
# or  
lappend listName value
```

Example:

```
#tclsh  
set var orange  
append var " " "blue"  
lappend var "red"  
lappend var "green"  
puts $var
```

OUTPUT:
orange blue red green

The syntax for length of list is given below –

```
llength listName
```

Example:

OUTPUT:4

```
#tclsh
```

```
set var {orange blue  
red green}  
puts [llength $var]
```

The syntax for selecting list item at specific index is given below –

```
lindex listname index
```

Example:

```
#tclsh
```

```
set var {orange blue  
red green}  
puts [lindex $var 1]
```

OUTPUT:
blue

The syntax for inserting list items at specific index is given below.

```
linsert listname index value1 value2..valuen
```

Example:

```
#tclsh  
  
set var {orange blue red  
green}  
set var [linsert $var 3  
black white]  
puts $var
```

OUTPUT:
orange blue red black white
green

Replace Items at Indices

The syntax for replacing list items at specific indices is given below –

```
lreplace listname firstindex lastindex value1 value2..valuen
```

Example:

```
#tclsh
```

```
set var {orange blue red green}  
set var [lreplace $var 2 3 black white]  
puts $var
```

OUTPUT:
orange blue black white

The syntax for setting list item at specific index is given below –

```
lset listname index value
```

Example:

```
#tclsh
```

```
set var {orange blue red  
green}  
lset var 0 black  
puts $var
```

OUTPUT:
black blue red green

Transform List to Variables

The syntax for copying values to variables is given below –

```
lassign listname variable1 variable2.. variablen
```

Example:

```
#tclsh
```

```
set var {orange blue red green}  
lassign $var colour1 colour2  
puts $colour1  
puts $colour2
```

OUTPUT:
orange
blue

The syntax for sorting a list is given below –

```
lsort listname
```

Example:

```
#tclsh
```

```
set var {orange blue red  
green}  
set var [lsort $var]  
puts $var
```

OUTPUT:
blue green orange red

An array is a systematic arrangement of a group of elements using indices. The syntax for the conventional array is shown below.

```
set ArrayName(Index) value
```

Example:

```
tclsh  
set languages(0) Tcl  
set languages(1) "C Language"  
puts $languages(0)  
puts $languages(1)
```

OUTPUT:
Tcl
C Language

The syntax for calculating size array is shown below.

```
[array size variablename]
```

Example:

```
tclsh
```

```
set languages(0) Tcl  
set languages(1) "C  
Language"  
puts [array size languages]
```

OUTPUT:
2

Array Iteration

- Array indices can be non-continuous like values specified for index 1 then index 10 and so on. But, in case they are continuous, we can use array iteration to access elements of the array.
- A simple array iteration for printing elements of the array is shown below

Example:

```
tclsh
```

```
set languages(0) Tcl
set languages(1) "C Language"
for { set index 0 } { $index < [array size
languages] } { incr index } {
    puts "languages($index) :
$languages($index)"
}
```

OUTPUT:

```
languages(0) : Tcl
languages(1) : C Language
```

Associative Arrays

- In Tcl, all arrays by nature are associative. Arrays are stored and retrieved without any specific order. Associative arrays have an index that is not necessarily a number, and can be sparsely populated.
- A simple example for associative array with non-number indices is shown below.

Example:

```
tclsh
```

```
set personA(Name) "sonali"
set personA(Age) 21
puts $personA(Name)
puts $personA(Age)
```

OUTPUT:
sonali
21

The syntax for retrieving indices of array is shown below.

```
[array names variablename]
```

Example:

```
tclsh
```

```
set personA(Name) "sonali"  
set personA(Age) 21  
puts [array names personA]
```

OUTPUT:
Age Name

Iteration of Associative Array

You can use the indices of array to iterate through the associative array. An example is shown below.

Example:

```
tclsh
```

```
set personA(Name) "sonali"  
set personA(Age) 21  
foreach index [array names personA] {  
    puts "personA($index):  
$personA($index)"  
}
```

OUTPUT:

```
personA(Age): 21  
personA(Name): sonali
```

List Vs Arrays

Lists	Arrays
Can store various elements	Can store Various elements
Order of elements preserved	Order of elements is not preserved
Elements are retrieved using Integer index	Elements are retrieved using String index
set a [list r g b] llength \$ a puts \$a gives r g b	array set a { r red g green b blue} array names a puts \$a doesn't give r g b

Strings

- The primitive data-type of Tcl is string and often we can find quotes on Tcl as string only language. These strings can contain alphanumeric character, just numbers, Boolean, or even binary data. Tcl uses 16 bit unicode characters and alphanumeric characters can contain letters including non-Latin characters, number or punctuate String Representations
- Unlike other languages, in Tcl, you need not include double quotes when it's only a single word. An example can be -on.

Example:

```
tclsh
```

```
set myVariable hello
puts $myVariable
```

OUTPUT:
hello

- When we want to represent multiple strings, we can use either double quotes or curly braces. It is shown below –

Example:

```
tclsh
```

```
set myVariable "hello world"  
puts $myVariable  
set myVariable {hello world}  
puts $myVariable
```

OUTPUT:

```
hello world
```

```
hello world
```

String Methods & Description

1	compare string1 string2 Compares string1 and string2 lexicographically. Returns 0 if equal, -1 if string1 comes before string2, else 1
2	first string1 string2 Returns the index first occurrence of string1 in string2. If not found, returns -1.
3	index string index Returns the character at index.
4	last string1 string2 Returns the index last occurrence of string1 in string2. If not found, returns -1.
5	length string Returns the length of string.
6	match pattern string Returns 1 if the string matches the pattern.
7	range string index1 index2 Return the range of characters in string from index1 to index2.
8	tolower string Returns the lowercase string.
9	toupper string Returns the uppercase string.
10	trim string ?trimcharacters? Removes trimcharacters in both ends of string. The default trimcharacters is whitespace.

String Comparison-Examples

```
set string1 "Hello"  
set string2 "World"  
set string3 "World"  
puts [string compare $string1 $string2]
```

```
if {[string compare $string2 $string3] == 1} {  
  puts "String string2 and string3 are same."; }
```

```
if {[string compare $string1 $string2] == -1} {  
  puts "String $string1 comes before $string2."; }
```

```
if {[string compare $string2 $string1] == 1} {  
  puts "String $string2 comes after $string1"; }
```

OUTPUT:

String Hello comes before World

String World comes after Hello

Index of String-Example

```
tclsh
```

```
set s1 "Hello World"
```

```
set s2 "o"
```

```
puts "First occurrence of $s2 in s1"
```

```
puts [string first $s2 $s1]
```

```
puts "Character at index 0 in s1"
```

```
puts [string index $s1 0]
```

```
puts "Last occurrence of $s2 in s1"
```

```
puts [string last $s2 $s1]
```

```
puts "Word end index in s1"
```

```
puts [string wordend $s1 20]
```

```
puts "Word start index in s1"
```

```
puts [string wordstart $s1 20]
```

OUTPUT:

First occurrence of o in
s1

4

Character at index 0 in
s1

H

Last occurrence of o in
s1

7

Word end index in s1

11

Word start index in s1

6

Length of String-Example

```
tclsh  
set s1 "Hello World"  
puts "Length of string s1"  
puts [string length $s1]
```

OUTPUT:
Length of string s1
11


```
tclsh  
set s1 "Hello World"  
puts "Uppercase string of s1"  
puts [string toupper $s1]  
puts "Lowercase string of s1"  
puts [string tolower $s1]
```

OUTPUT:

```
Uppercase string of s1  
HELLO WORLD  
Lowercase string of s1  
hello world
```

Trimming Characters- Example

```
tclsh
```

```
set s1 "Hello World"
```

```
set s2 "World"
```

```
puts "Trim right $s2 in $s1"
```

```
puts [string trimright $s1 $s2]
```

```
set s2 "Hello"
```

```
puts "Trim left $s2 in $s1"
```

```
puts [string trimleft $s1 $s2]
```

```
set s1 " Hello World "
```

```
set s2 " "
```

```
puts "Trim characters s1 on both sides of s2"
```

```
puts [string trim $s1 $s2]
```

OUTPUT:

Trim right World in Hello World
Hello

Trim left Hello in Hello World
World

Trim characters s1 on both sides of s2
Hello World

Matching Strings-Example

```
tclsh
set s1 "test@test.com"
set s2 "*@*.com"
puts "Matching pattern s2 in s1"
puts [string match "*@*.com" $s1 ]
puts "Matching pattern tcl in s1"
puts [string match {tcl} $s1]
```

OUTPUT:

Matching pattern s2 in s1

1

Matching pattern tcl in s1

0

Append Command- Example

```
tclsh
```

```
set s1 "Hello"
```

```
append s1 " World"
```

```
puts $s1
```

OUTPUT:
Hello World

- Scan command is used for parsing a string based to the format specifier.

```
tclsh
```

```
puts [scan "90" {%[0-9]} m]
```

```
puts [scan "abc" {%[a-z]} m]
```

```
puts [scan "abc" {%[A-Z]} m]
```

```
puts [scan "ABC" {%[A-Z]} m]
```

OUTPUT:

1

1

0

1

The following table shows the list of format specifiers available in Tcl –

Specifier	Use
%s	String representation
%d	Integer representation
%f	Floating point representation
%e	Floating point representation with mantissa-exponent form
%x	Hexa decimal representation

Format command -Example

```
puts [format "%f" 43.5]  
puts [format "%e" 43.5]  
puts [format "%d %s" 4 tuts]  
puts [format "%s" "Tcl Language"]  
puts [format "%x" 40]
```

OUTPUT:

```
43.500000  
4.350000e+01  
4 tuts  
Tcl Language  
28
```

Write a program to accept two strings from user and print their difference

```
puts "enter a number"  
gets stdin fst  
puts "enter another number"  
gets stdin snd  
set c [expr $fst - $snd]  
puts "difference between both the numbers is $c"
```

Enter a number

89

Enter another number

90

Difference between both the
numbers is -1

Write a program to accept two strings from user and print their product(multiply) and quotient(divide)?

```
puts "enter a number"
gets stdin fst
puts "enter another number"
gets stdin snd
set c [expr $fst * $snd]
set d [expr $fst/$snd]
puts "Product of both the numbers is $c"
puts "Quotient of both the numbers is $d"
```

Enter a number

10

Enter another number

10

Product of both the numbers is 100

Quotient of both the numbers is 1

Exercise

- 1) Write a program to find if its even or odd.
- 2) Write a program to check if it is >10 , <10 or equals 10.
- 3) Write a program to find if positive or negative.
- 4) Write a program to print area of a circle. $A(\text{circle}) = 3.142 * R * R$?
- 5) Write a program to print area of a triangle $A(\text{Triangle}) = 0.5 * B * H$
- 6) Write a program to accept 2 numbers from user and compare them.

- Tcl supports file handling with the help of the built in commands open, read, puts, gets, and close.
- A file represents a sequence of bytes, does not matter if it is a text file or binary file.

Opening Files

- Tcl uses the open command to open files in Tcl. The syntax for opening a file is as follows –

```
open fileName accessMode
```

Modes and Description

Sr.No.	Mode & Description
1	r Opens an existing text file for reading purpose and the file must exist. This is the default mode used when no accessMode is specified.
2	w Opens a text file for writing, if it does not exist, then a new file is created else existing file is truncated.
3	a Opens a text file for writing in appending mode and file must exist. Here, your program will start appending content in the existing file content.
4	r+ Opens a text file for reading and writing both. File must exist already.
5	w+ Opens a text file for reading and writing both. It first truncate the file to zero length if it exists otherwise create the file if it does not exist.
6	a+ Opens a text file for reading and writing both. It creates the file if it does not exist. The reading will start from the beginning, but writing can only be appended.

To close a file, use the close command. The syntax for close is as follows -

```
close fileName
```

Any file that has been opened by a program must be closed when the program finishes using that file. In most cases, the files need not be closed explicitly; they are closed automatically when File objects are terminated automatically.

Puts command is used to write to an open file.

```
puts $filename "text to write"
```

A simple example for writing to a file is shown below.

```
tclsh  
set fp [open input.txt w]  
puts $fp " Sample for TCL "  
puts $fp " This is sonali "  
close $fp
```

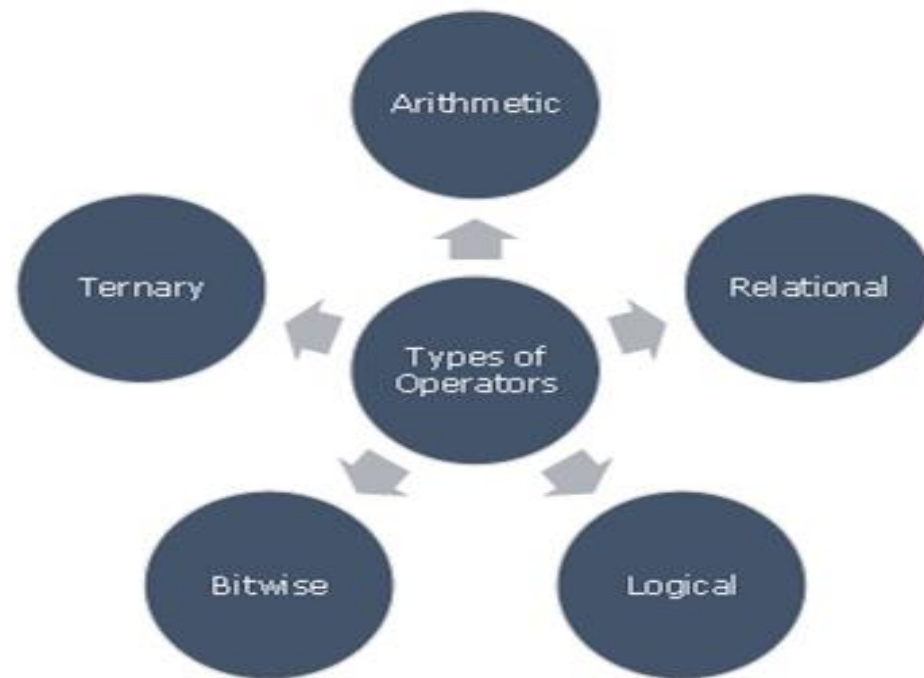
When the above code is compiled and executed, it creates a new file input.txt in the directory that it has been started under (in the program's working directory).

Following is the simple command to read from a file –

```
# Reading from file
set fp [open input.txt r]
set file_data [read $fp]
puts $file_data
close $fp
```

```
#Reading File line by line
set fp [open input.txt r]
set k 1
while { [gets $fp data ] >= 0 } {
  puts " line-$k : $data"
  incr k
}
close $fp
```

- An operator is a symbol that tells the compiler to perform specific mathematical or logical manipulations. Tcl language is rich in built-in operators and provides the following types of operators –



Assume variable 'A' holds 10 and variable 'B' holds 20, then –

Operator	Description	Example
+	Adds two operands	$A + B$ will give 30
-	Subtracts second operand from the first	$A - B$ will give -10
*	Multiplies both operands	$A * B$ will give 200
/	Divides numerator by denominator	B / A will give 2
%	Modulus Operator and remainder of after an integer division	$B \% A$ will give 0

Example

```
set x1 10  
set x2 20  
set xdiff [$x1-$x2]
```

NOTE: Use the keyword **expr** for executing the arithmetic operations

```
set xdiff [expr $x1-$x2] → -10  
set xdiff [expr{$x1-$x2}] → Syntax error ( after expr space should be given)  
set xdiff [expr {$x1-$x2}] → -10
```

Relational Operators

Assume variable A holds 10 and variable B holds 20, then –

Operator	Description	Example
==	Checks if the values of two operands are equal or not, if yes then condition becomes true.	A==BA==B is not true.
=	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	A!=BA!=B is true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	A>BA>B is not true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	A<BA<B is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	A>=BA>=B is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	A<=BA<=B is true.

Assume variable A holds 1 and variable B holds 0, then –

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then condition becomes true.	A && B is false.
	Called Logical OR Operator. If any of the two operands is non-zero, then condition becomes true.	A B is true.
!	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	!A && B is true.

Procedures are nothing but code blocks with series of commands that provide a specific reusable functionality. It is used to avoid same code being repeated in multiple locations. Procedures are equivalent to the functions used in many programming languages and are made available in Tcl with the help of proc command.

The syntax of creating a simple procedure is shown below –

```
proc procedureName {arguments}  
{ body  
}
```

A simple example for procedure is given below –

```
tclsh
```

```
proc helloWorld { } {  
    puts "Hello, World!"  
}  
helloWorld
```

OUTPUT:
Hello, World!

An example for procedure with arguments is shown below –

```
tclsh
```

```
proc add {a b} {  
    return [expr $a+$b]  
}  
puts [add 10 30]
```

An example for procedure with arguments is shown below –

```
tclsh
```

```
proc avg {numbers} {  
    set sum 0  
    foreach number $numbers {  
        set sum [expr $sum + $number]  
    }  
    set average [expr $sum/[llength $numbers]]  
    return $average  
}  
puts [avg {70 80 50 60}]  
puts [avg {70 80 50 }]
```

OUTPUT:

65

66

Default arguments are used to provide default values that can be used if no value is provided. An example for procedure with default arguments, which is sometimes referred as implicit arguments is shown below –

tclsh

```
proc add {a {b 100}} {  
    return [expr $a+$b]  
}  
puts [add 10 30]  
puts [add 10]
```

OUTPUT:

40
110

Write a procedure to find the factorial of a number

```
tclsh
proc factorial {number} {
    if {$number <= 1} {
        return 1
    }
    return [expr $number * [factorial [expr $number - 1]]]
}
puts [factorial 3]
puts [factorial 5]
```

OUTPUT:

6

120

- list out all the Data cells which are having transition time more than 400 ps and fanout more than 24
- List out all the nets in the design and write it into a file
`get_nets -hier`
- list out all the partially annotated nets and write them into a file.
- `get_nets -hier -top_net_of_hierarchical_group -f
"has_detailed_parasitics == true && has_valid_parasitics == false"`

- The "regexp" command is used to match a regular expression in Tcl. A regular expression is a sequence of characters that contains a search pattern.

Syntax

The syntax for regex is given below –

```
regexp optionalSwitches patterns searchString fullMatch  
subMatch1 ... subMatchn
```

Example:

```
set a [regexp sample "sample test"]  
puts $a  
regexp {sample} "sample test" line  
puts $line
```

Regex (Cont..)

Sr.No	Rule & Description
.	
1	x : Exact match.
2	[a-z] : Any lowercase letter from a-z.
3	. : Any character.
4	^ : Beginning string should match.
5	\$: Ending string should match.
6	\^ : Backlash sequence to match special character ^. Similarly you can use for other characters.
7	() : Add the above sequences inside parenthesis to make a regular expression.
8	x* : Should match 0 or more occurrences of the preceding x.
9	x+ : Should match 1 or more occurrences of the preceding x.
10	[a-z]? : Should match 0 or 1 occurrence of the preceding x.
11	{digit} : Matches exactly digit occurrences of previous regex expression. Digit that contains 0-9.
12	{digit,} : Matches 3 or more digit occurrences of previous regex expression. Digit that contains 0-9.

```
regexp {[A-Z,a-z]*} "Tcl Tutorial" a b
```

```
puts "Full Match: $a"                      →      Full Match: Tcl
```

```
puts "Sub Match1: $b"                      →      Sub Match: Tcl
```

```
regexp {[A-Z,a-z]*}([A-Z,a-z]*) "Tcl Tutorial" a b c
```

```
puts "Full Match: $a"                      →      Full Match: Tcl Tutorial
```

```
puts "Sub Match1: $b"                      →      Sub Match1: Tcl
```

```
puts "Sub Match2: $c"                      →      Sub Match2: Tutorial
```

```
regexp {[A-Z,a-z]*}([A-Z,a-z]*) "Tcl Tutorial" a b c
```

```
puts "Full Match: $a"
```

```
puts "Sub Match1: $b"                      →      Full Match: Tcl Tutorial
```

```
puts "Sub Match2: $c"                      →      Sub Match1: Tcl Tutorial
```

```
                                            →      Sub Match2: Tutorial
```

- **nocase** – Used to ignore case.
- **indices** – Store location of matched sub patterns instead of matched characters.
- **line** – New line sensitive matching. Ignores the characters after newline.
- **start index** – Sets the offset of start of search pattern.

Procedures and Scope

- Scoping: local and global variables.
 - Interpreter knows variables by their name and scope
 - Each procedure introduces a new scope
- `global` procedure makes a global variable local

```

set outside "I'm outside"
set inside "I'm really outside"
proc whereAmI {inside} {
    global outside
    puts $outside
    puts $inside
}
whereAmI "I wonder where I will be"
-> I'm outside
    I wonder where I will be
  
```


Examples (Cont..)

Write a proc to find errors with pattern: **ERROR: (IMPCCOPT-<any_number>)

```
gets stdin file
proc errors { in_file } {
  set fp [open $in_file r ]
  while { [gets $fp line ] >= 0 } {
    if { [regexp ERROR $line ] } {
      if { [regexp -nocase {\*\*[a-z]*: *\ (IMPCCOPT-[0-9]*)} $line] } {
        puts $line
      }
    }
  }
  puts " file1 : $file"
  errors $file
}
```

```
entity add is
port {
cin : in std_logic;
a   : in std_logic_vector { 3 downto 0}
b   : in std_logic_vector { 3 downto 0}
cout: out std_logic
sum : out std_logic_vector { 3 downto 0 }
}
end add;
```

- Findout how many multi-bit buses are there in the above vhdl file.
- Replace the pin name with pin_bus

Tcl - Built-in Functions

abs arg : Calculates the absolute value of arg.

acos arg:Calculates the arccosine of arg.

asin arg :Calculates the arcsine of arg.

atan arg :Calculates the arctangent of arg.

atan2 y x :Calculates the arctangent of the quotient of its arguments(y/x).

ceil arg :Calculates the smallest integer greater than or equal to a number.

cos arg:Calculates the cosine of arg.

cosh arg:Calculates the hyperbolic cosine of arg.

double arg:Calculates if arg is a floating-point value, returns arg, otherwise

converts: arg to floating-point and returns the converted value.

exp arg:Calculates an exponential function (e raised to the power of arg).

int arg:Calculates if arg is an integer value of the same width as the machine word, returns arg, otherwise converts arg to an integer.

Log: Calculates the natural logarithm of arg.

log10 arg:Calculates the base 10 logarithm of arg.

pow x y:Calculates the value of x raised to the power y. If x is negative, y must be an integer value.

sqrt arg: Calculates the square root of arg. arg must be positive.

Example

Some examples using math functions are given below –

```
tclsh
```

```
namespace import
```

```
::tcl::mathfunc::*
```

```
puts [tan 10]
```

```
puts [pow 10 2]
```

```
puts [ceil 10.34]
```

```
puts [hypot 10 20]
```

```
puts [srand 45]
```

```
puts [log 10]
```

```
puts [srand 45]
```

OUTPUT:

0.6483608274590866

100.0

11.0

22.360679774997898

0.0003521866166741525

2.302585092994046

0.0003521866166741525

System Functions

The important system functions in Tcl includes,

- **clock** – seconds function, which returns current time in seconds.
- **clock** – format function, which formats the seconds into date and time.
- **clock** – scan function, which scans the input string and converts it into seconds.
- **open** – function, which is used to open a file.
- **exec** – function, which is used to execute a system command.
- **close** – function, which is used to close a file.

```
#get seconds
```

```
set currentTime [clock seconds]
```

```
puts $currentTime
```

```
#get format
```

```
puts "The time is: [clock format $currentTime -format  
%H:%M:%S]"
```

```
puts "The date is: [clock format $currentTime -format %D]"
```

```
set date "Jun 15, 2014"
```

```
puts [clock scan $date -format {%b %d, %Y}]
```

```
puts [exec ls]
```

```
puts [exec dir]
```

```
set a [open input.txt]
```

```
puts [read $a];
```

```
puts $a
```

```
close $a
```

Example :

1. Write all the even numbers below 100.

```
set number 100
proc even_number {arg} {
  for { set i 1 } { $i < $arg } { incr i } {
    if { [expr $i%2] == 0 } {
      append list " $i"
    }
  }
  puts $list
}
even_number $number
```

2. Fibonacci Series:

```
set a 0; set b 1;  
for {set i 1} { $i <= [expr $n]} {incr i} {  
  set c [expr $a + $b]  
  puts $c  
  set a $b  
  set b $c  
}
```


3. List out all the nets in the design and write into a file

```
set fp [open nets.rpt w] ; set i 0
set nets [get_object_name [get_flat_nets]]
foreach net $nets {
  puts $fp $net ; incr i
}
puts "Total no of nets: $i"
close $fp
```

4. Write a procedure to find factorial of given number

```
proc factorial {number} {  
  if {$number <= 1} {  
    return 1  
  }  
  return [expr $number * [factorial [expr $number - 1]]]  
}  
factorial 5
```

- Features of TCL
- Invoking TCL
- Basic Syntax
- Scripts and Commands
- Variables
- Decisions
- Loops
- Arrays
- Strings
- Lists
- Reg-Exp

Module 10

Clock Tree Synthesis

- In this module you will understand what is clock tree synthesis and its importance in the physical design flow.
- Clock tree specifications, clock tree methodologies, concept of global and local skew, etc are discussed.



- Introduction to CTS
 - ✓ Clock Distribution in A Block
 - ✓ CTS in ASIC Flow - Stages
- CTS Specifications
 - ✓ Cells to use in CTS
 - ✓ Target Skew, Target Latency, Max Transition Target
 - ✓ Clock Routing Rules
 - ✓ Exceptions
 - ✓ Stop pins
 - ✓ Non-Stop Pins
 - ✓ Float Pins
 - ✓ Exclude Pins
- CTS QoR
 - ✓ QoR Summary
 - ✓ Clock Jitter

- Clock Uncertainty
 - ✓ Setup Slack(reg2reg) with Uncertainty
 - ✓ Hold Slack(reg2reg) with Uncertainty
- CTS & Opt-Quality Checks
 - ✓ CTS & Opt - Care about
 - ✓ CTS Distribution Methodologies
- CTS Distribution Methodologies
 - ✓ H-tree
 - ✓ Clock Mesh

Introduction to CTS

This section gives the introduction to Clock Tree Synthesis and various steps involved in Clock Tree Synthesis stage

CTS Specifications



CTS QoR



Clock Uncertainty



CTS & Opt-Quality Checks

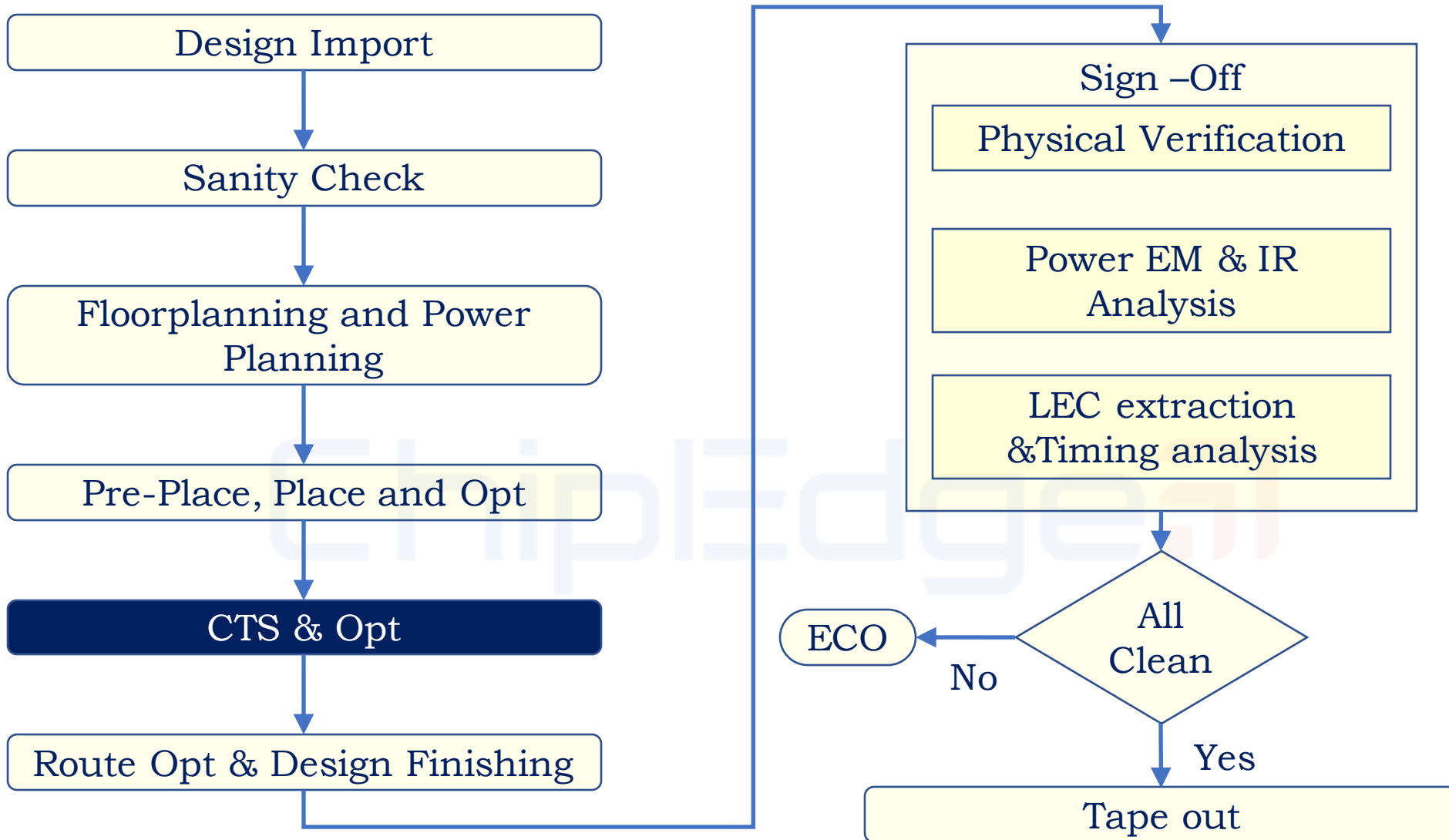


Additional Topics

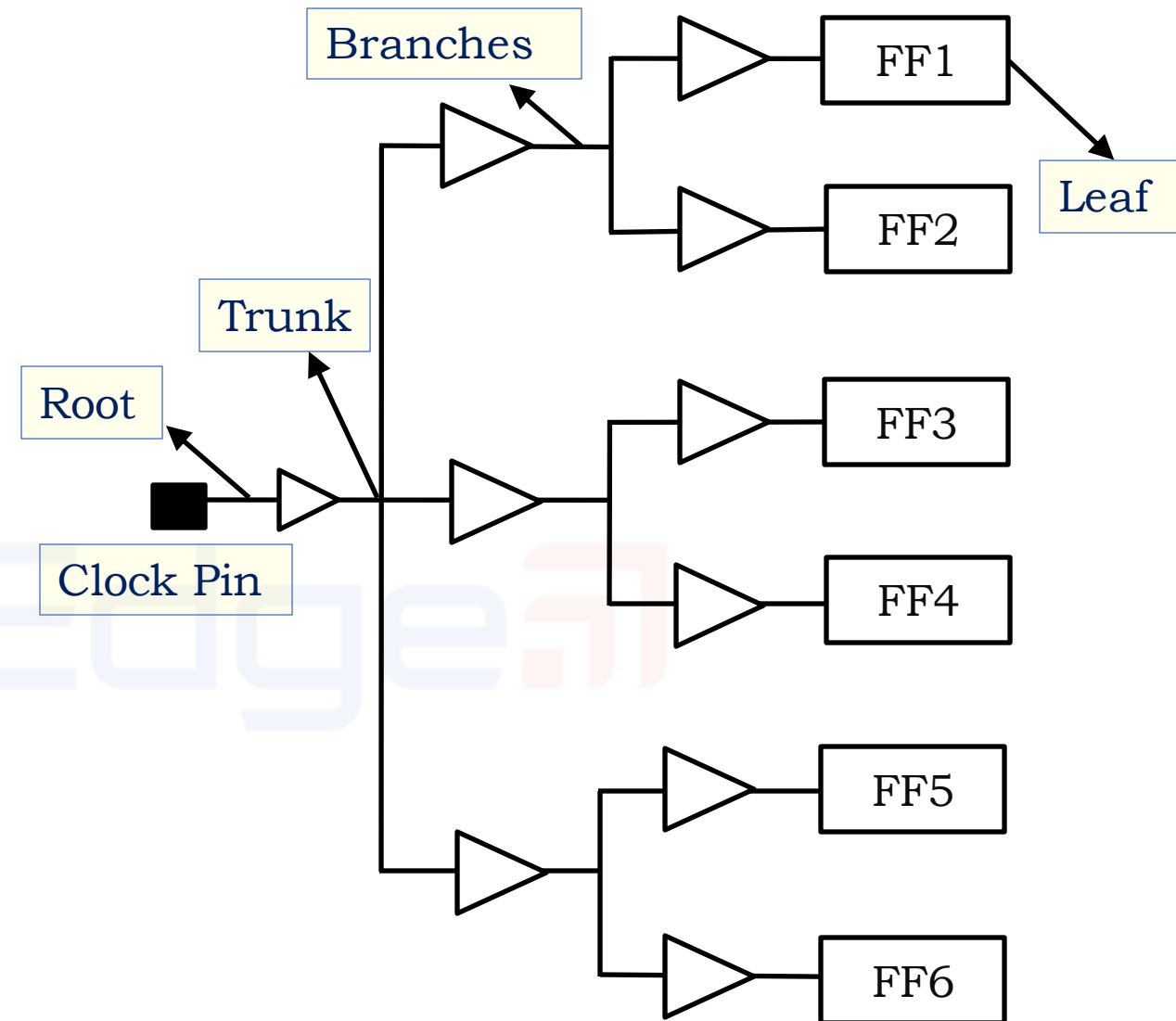


CTS Distribution Methodologies

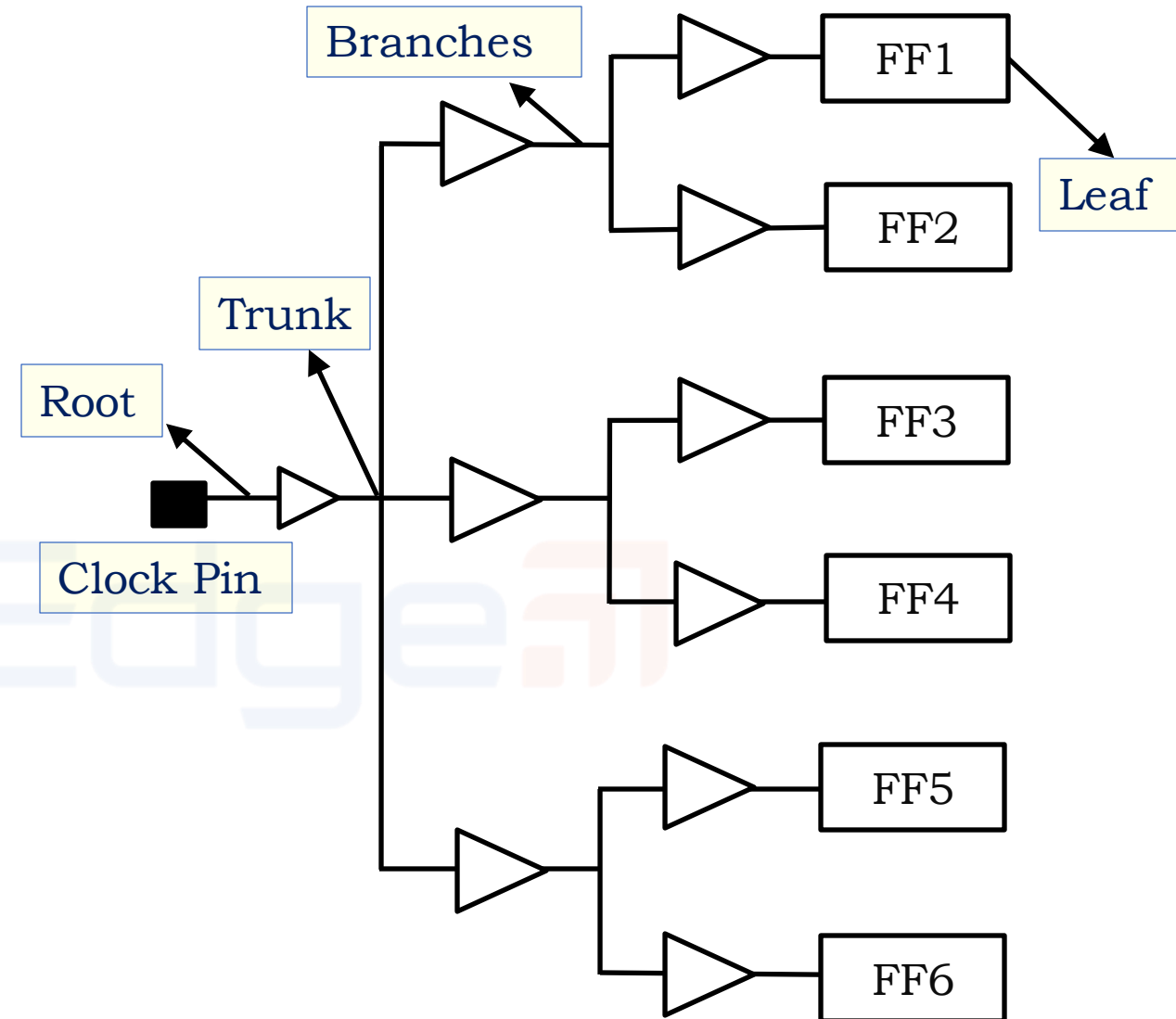


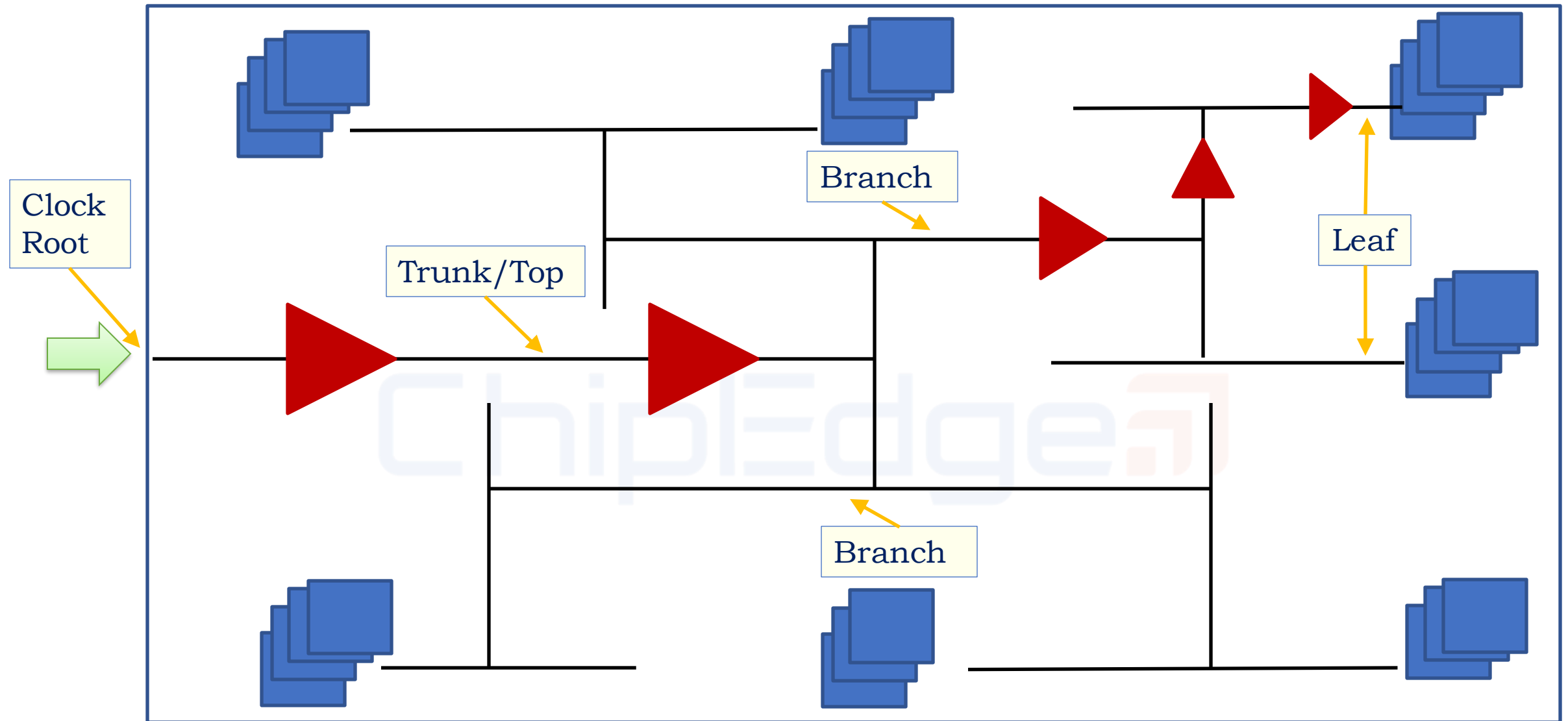


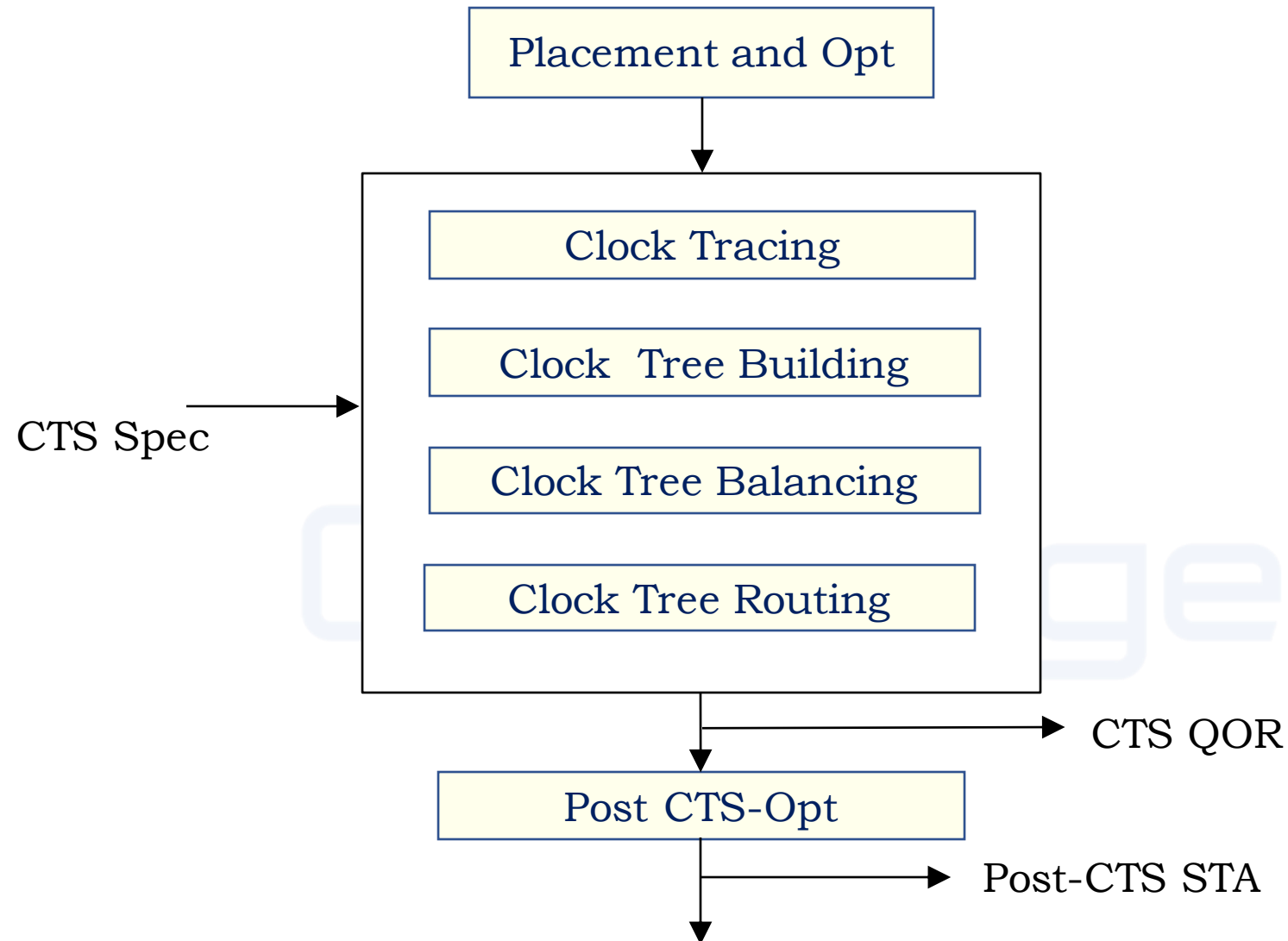
- Building clock tree from clock port to the all-clock pins of sequential cells.
- Goals of CTS
 - Efficient Clock reaches all the flops (transition time, min pulse width)
 - Minimize Clock Skew
 - Minimize Clock latency



- Before CTS the clock is treated to be ideal. After CTS the clock is said to be propagated clock.
- Clock tree balancing is done using buffers and inverters







Introduction to CTS



CTS Specifications

CTS QoR



Clock Uncertainty



CTS & Opt-Quality Checks



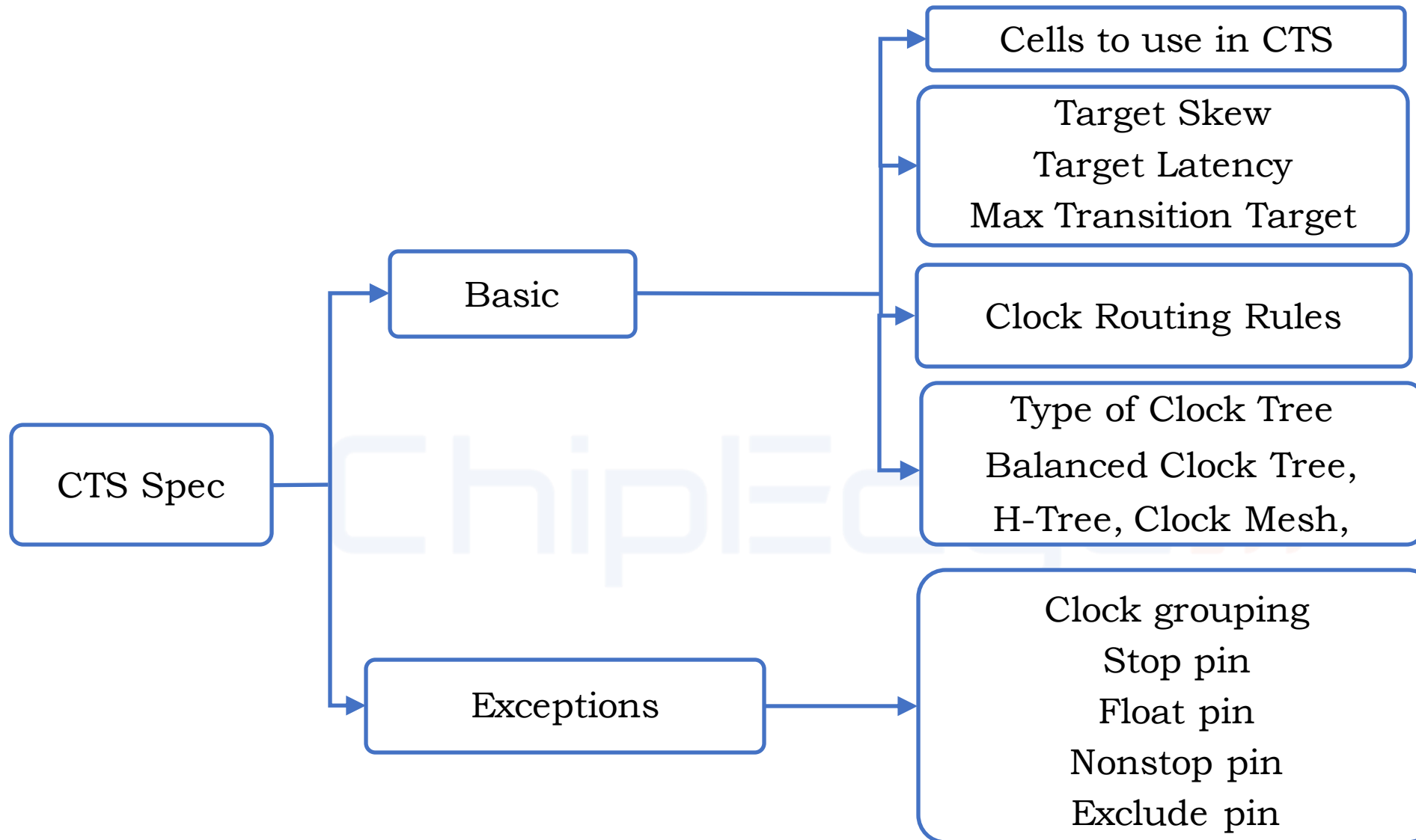
Additional Topics



CTS Distribution Methodologies

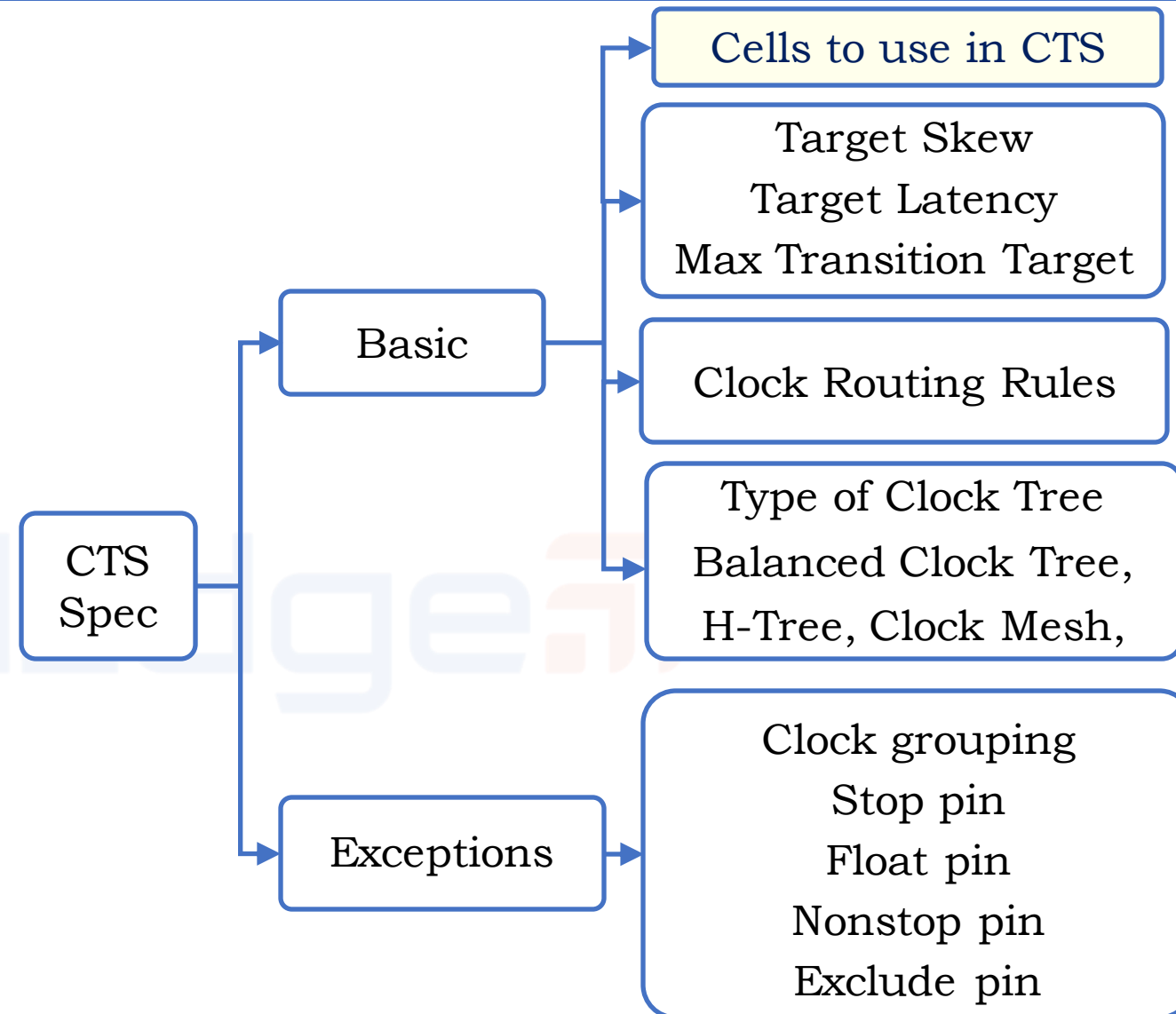


In this section, clock cells (inverters, buffers etc.), clock routing rules, types of clock trees and exceptions are explained in detail.



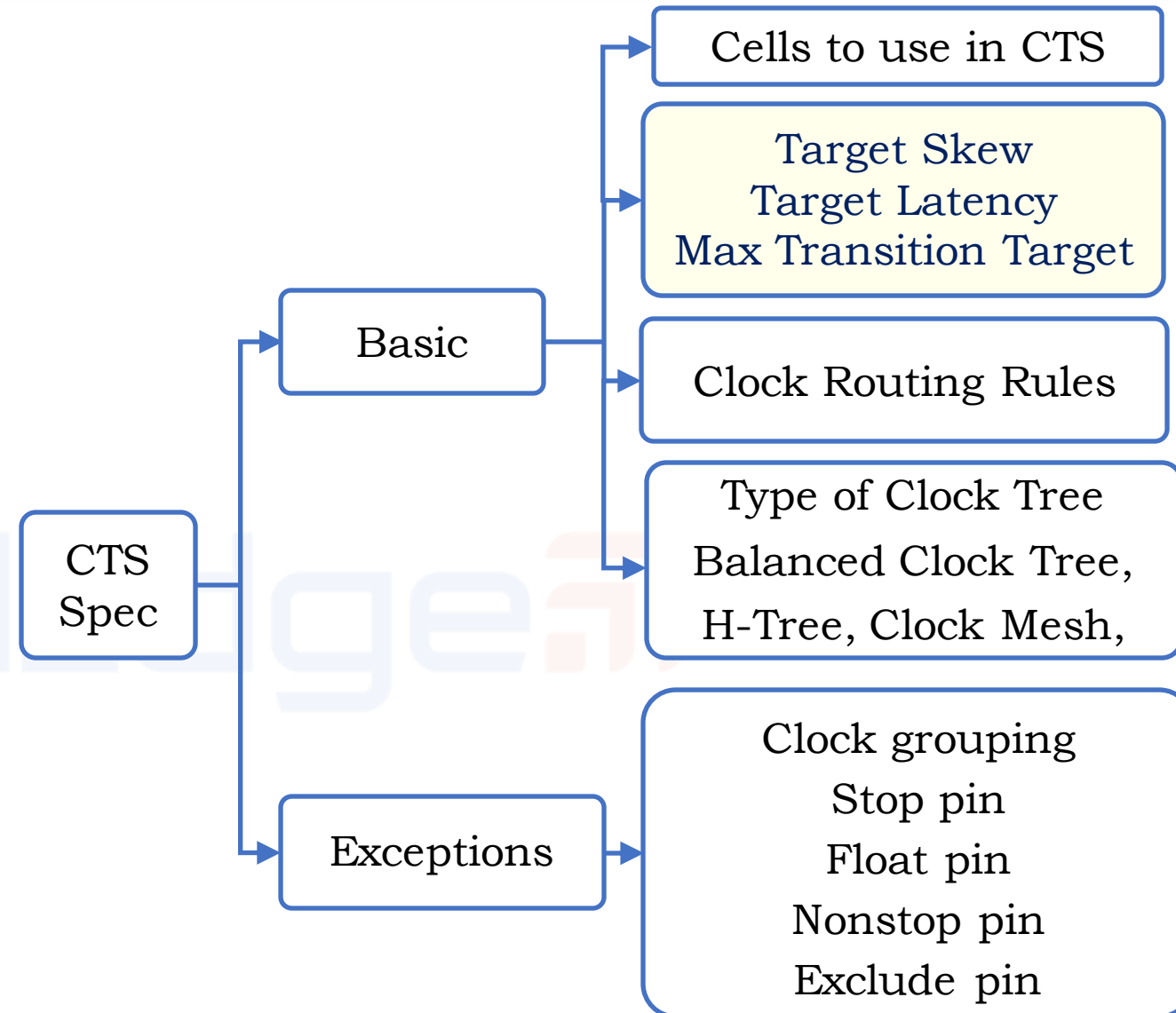
CTS Spec-Cells to use in CTS

- Clock Cells-Inverters / Buffers: Equal rise and fall times.
- LVT: Lesser Delay
- Single VT: Less Variations between Different PVT.
- Mid Drive Strength
- Low Drive strength cells are not preferred because of higher delay, more variations.
- High Drive Strength cells are not preferred because of power density

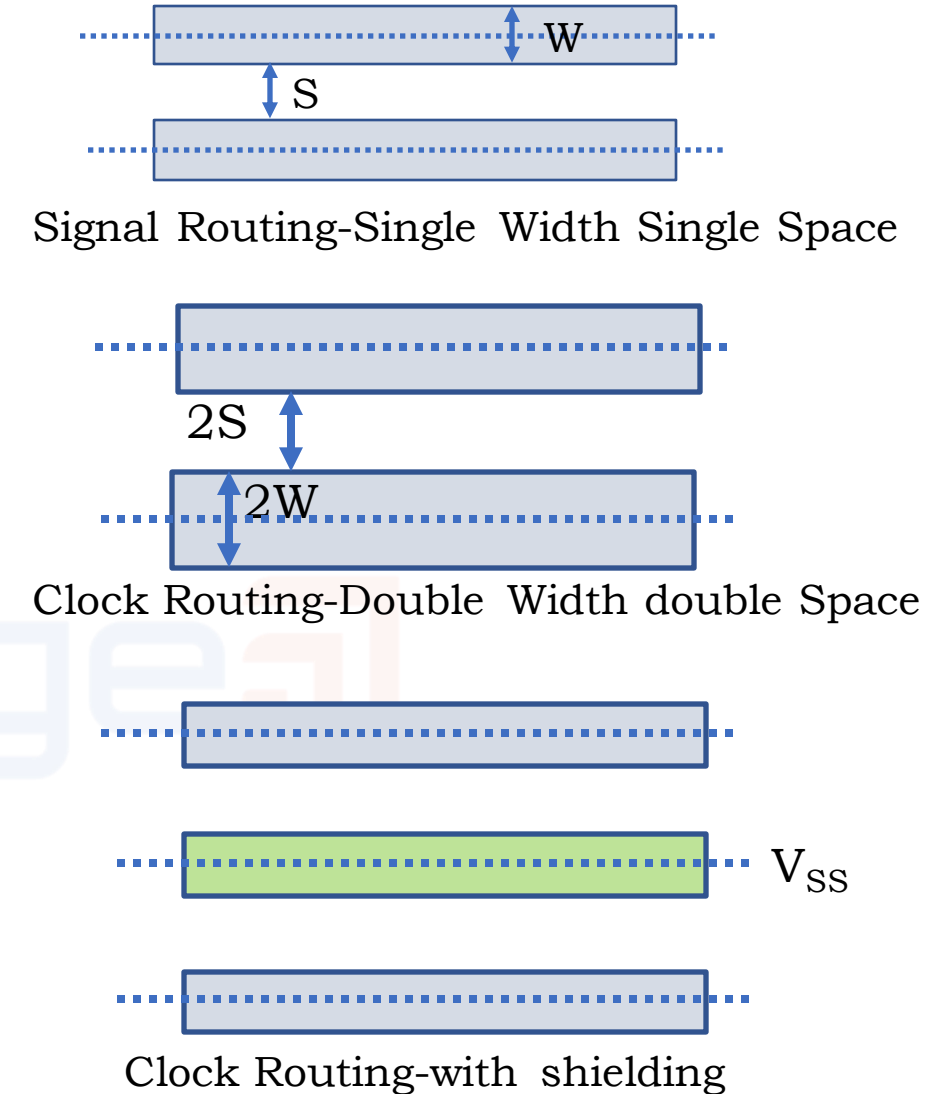


CTS Spec – Target Skew, Target Latency, Max Transition Target

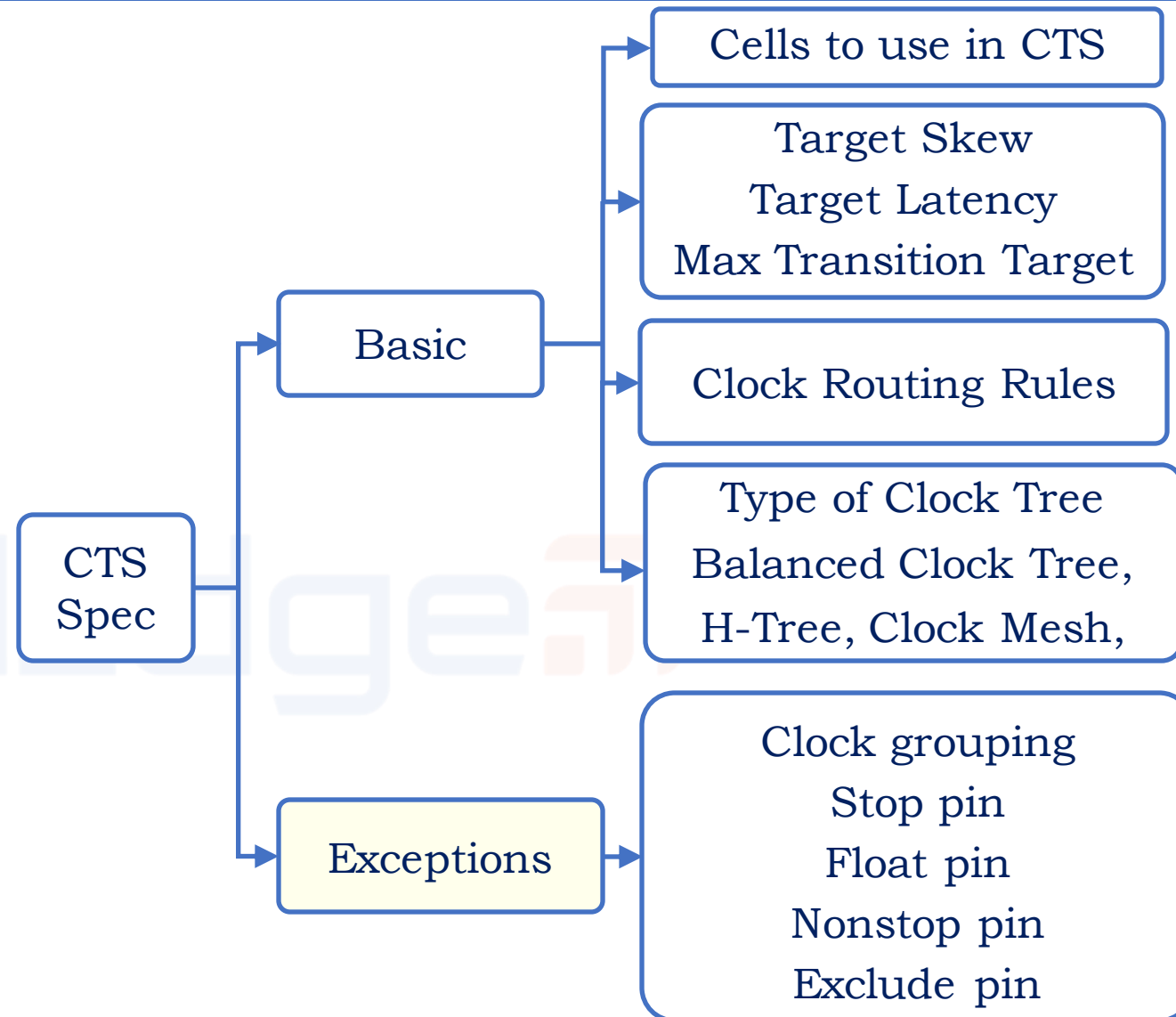
- Target skew: CTS will balance the clock tree such that it achieves the given target skew value.
- Target latency: CTS will build the clock tree such that it achieves the given target latency value.
- Max transition target: CTS will build the clock tree such that the transition time on any clock net is less than the given max transition target.



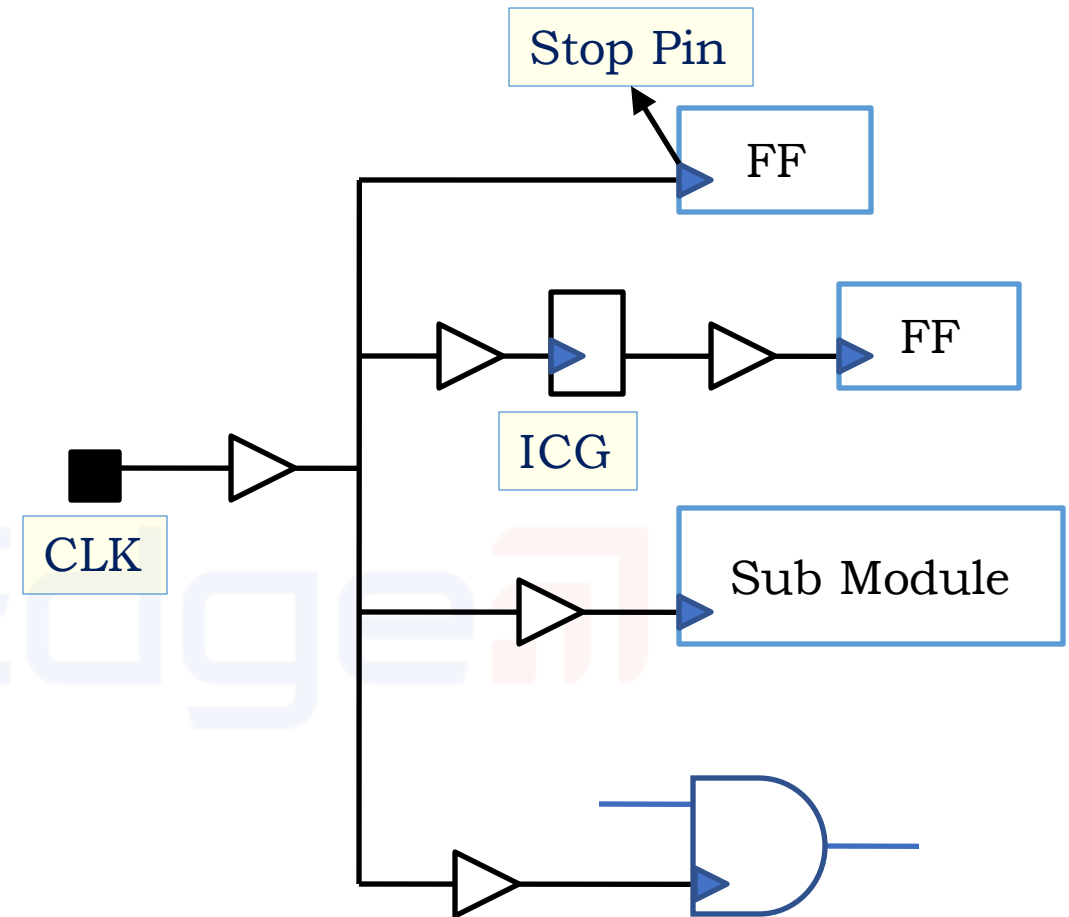
- Metal layers to be used for CTS
- Width and spacing of metal layers
- Shielding
- Metal Layers : Top 2 metal layers below the power mesh, (low resistance, improves latency)
- Double width : Avoid EM, low resistance (faster, improves latency)
- Double spacing : Avoid crosstalk
- Shielding with V_{SS} grid is also used to avoid crosstalk



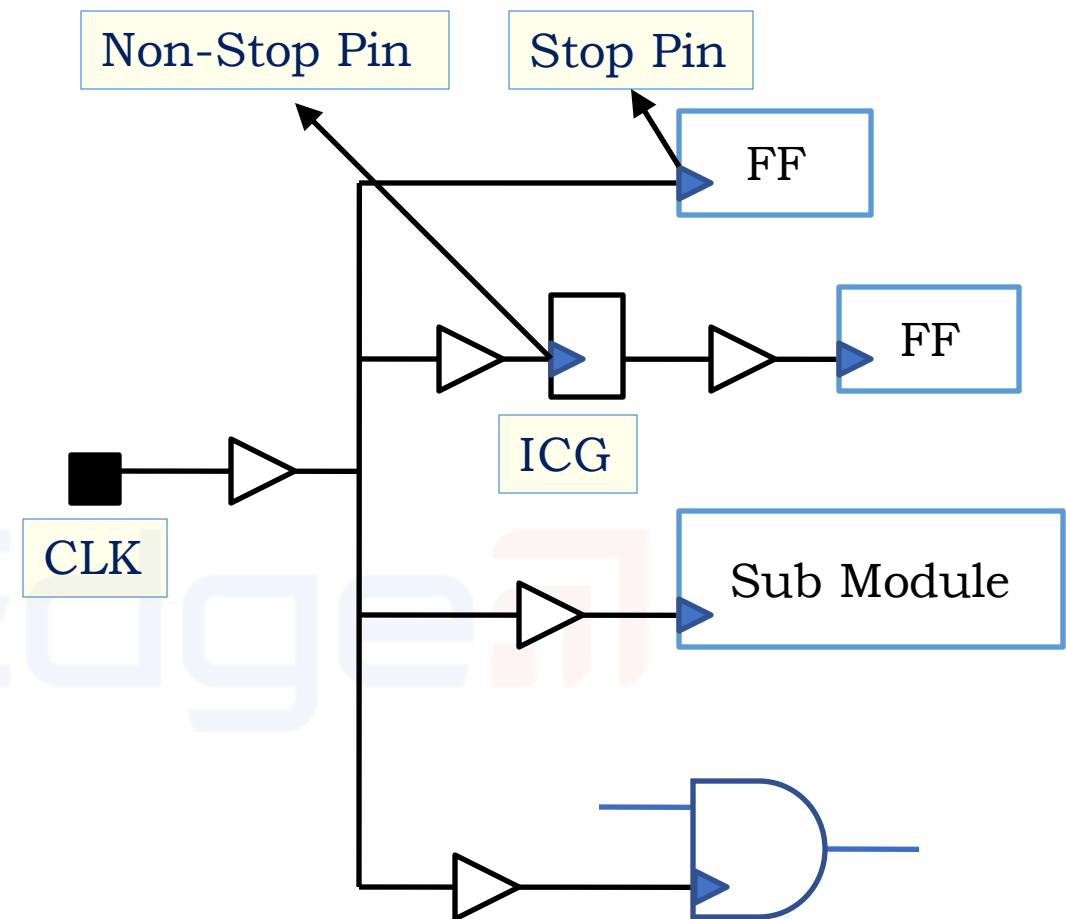
- By default, CTS Balances the latency of all sink flops.
- This default behavior can be altered by defining exceptions like float pin, exclude pin, etc...



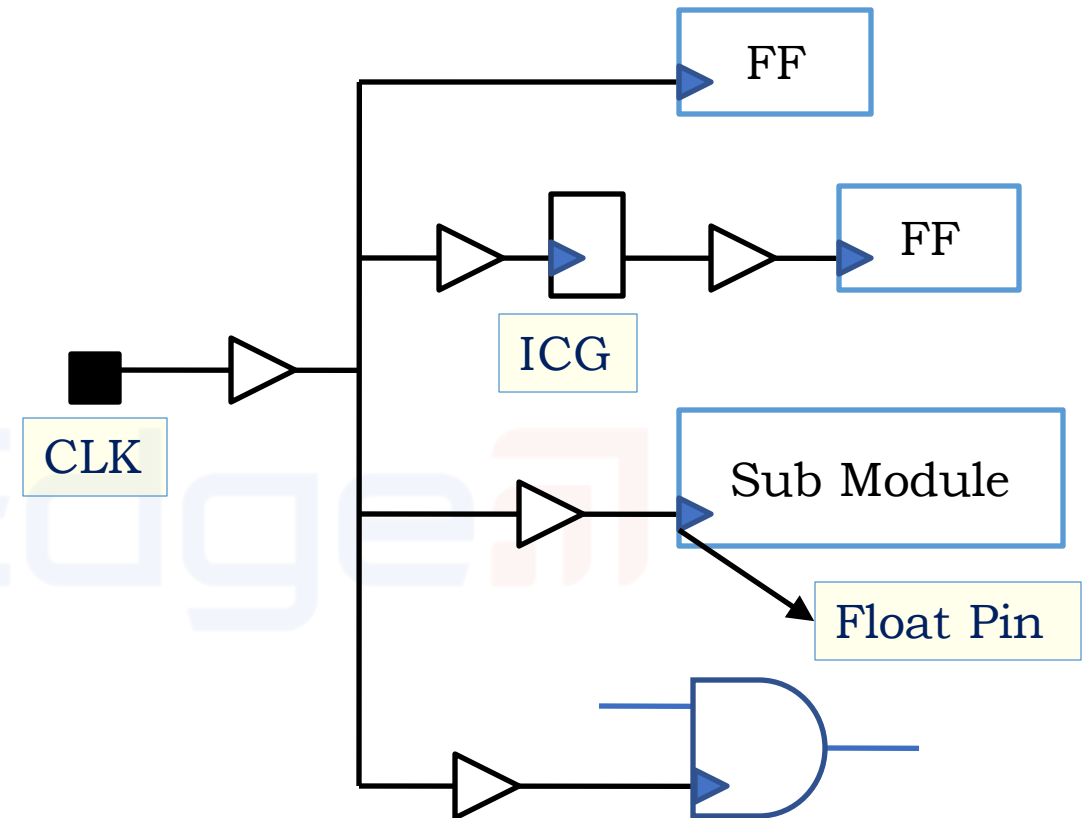
- Stop pins are the endpoints of the clock tree that are used for delay balancing.
- During clock tree synthesis, the tool uses stop pins in calculations and optimizations for both design rule constraints and clock tree timing (skew and insertion delay).



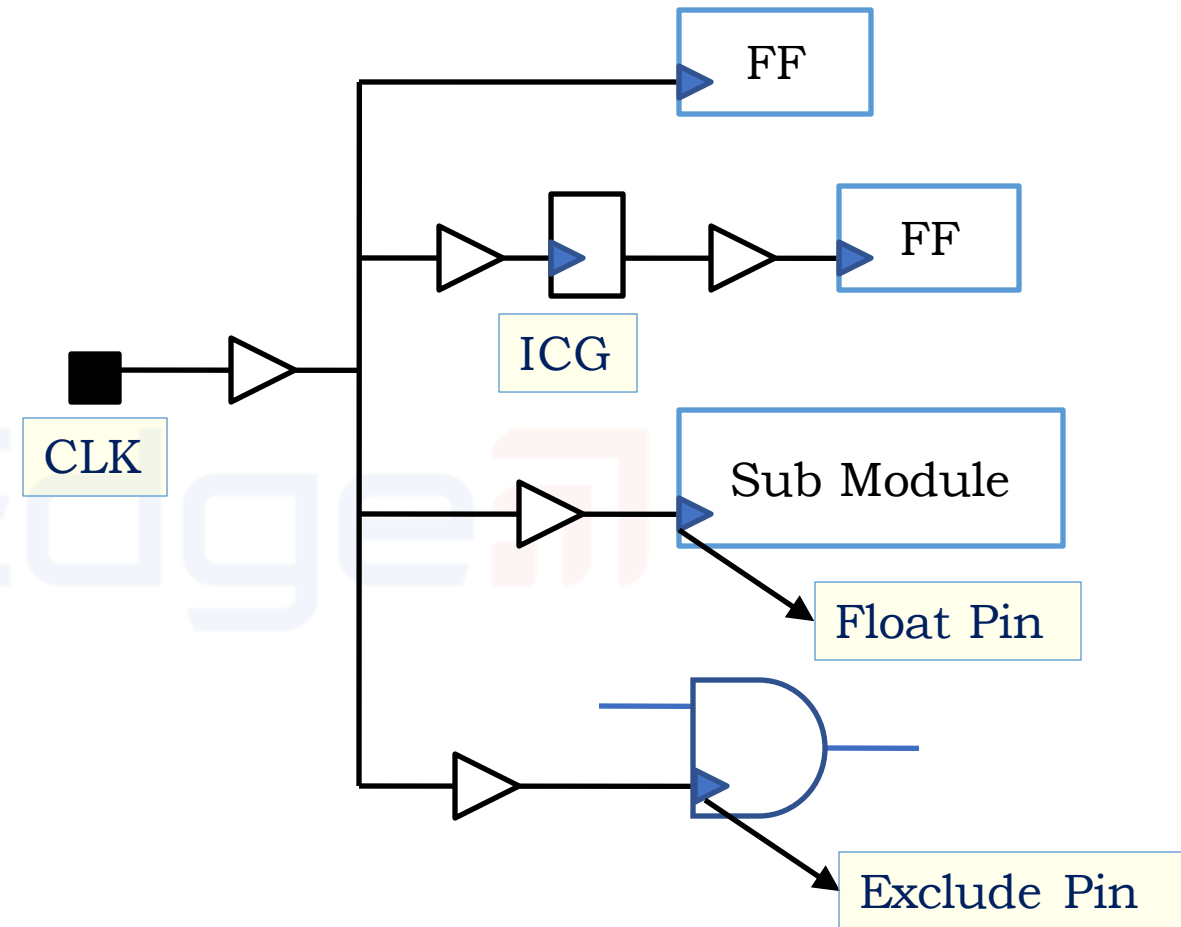
- Nonstop pins are pins that would normally be considered endpoints of the clock tree, but instead the tool traces through them to find the clock tree endpoints.
- The clock pins of ICG (Integrated Clock Gating cell) are nonstop pins.
- The clock pins of sequential cells driving generated clocks are implicit nonstop pins.



- Float pins are clock pins that have special insertion delay requirements.
- The tool adds the float pin delay (positive or negative) to the calculated insertion delay up to this pin.

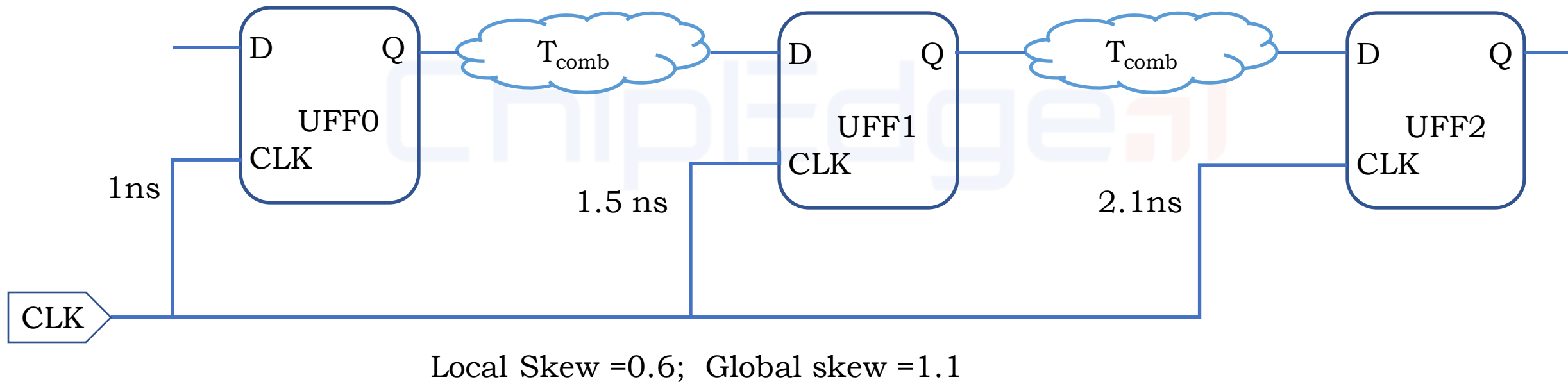


- Exclude pins are clock tree endpoints that are excluded from clock tree timing calculations and optimizations.
- The tool uses exclude pins only in calculations and optimizations for design rule constraints.



- The tool implicitly identifies stop pins ,non-stop pins, exclude pin etc..
- Clock tree exceptions are user-defined changes to the default endpoints derived by the tool.
- For example, the clock pin identified as a stop pin by the tool can be changed to exclude pin by defining clock exception on the pin

- Global Skew: The skew between any two flops irrespective of whether they are interacting or not.
- Local Skew : The skew between two interacting flops .



Introduction to CTS



CTS Specifications



CTS QoR

Clock Uncertainty



CTS & Opt-Quality Checks



Additional Topics



CTS Distribution Methodologies



The brief explanation of QoR reports in terms of Global skew, Local skew, Insertion delay/Latency and Max clock transition violations are explained in this section.

- Global skew
- Local skew
- Insertion delay/Latency
- Max clock transition violations
- Number of levels
- Number of Clock cells added

```
icc2_shell> report_clock_qor
```

```
=====
==== Summary Reporting for Corner fast ====
=====
```

===== Summary Table for Corner fast =====											
Clock / Skew Group	Attrs	Sinks	Levels	Clock Repeater Count	Clock Repeater Area	Clock Stdcell Area	Max Latency	Global Skew	Trans DRC Count	Cap DRC Count	Wire Length

### Mode: func, Scenario: func_fast clock	M,D	723	9	32	11.81	40.80	0.56	0.28	0	0	3136.69

All Clocks		723	9	32	11.81	40.80	0.56	0.28	0	0	3136.69

Warning: Please use -largest / -smallest / -all switches with -show_verbose_paths / -show_paths to report the clock paths. (CTS-956)

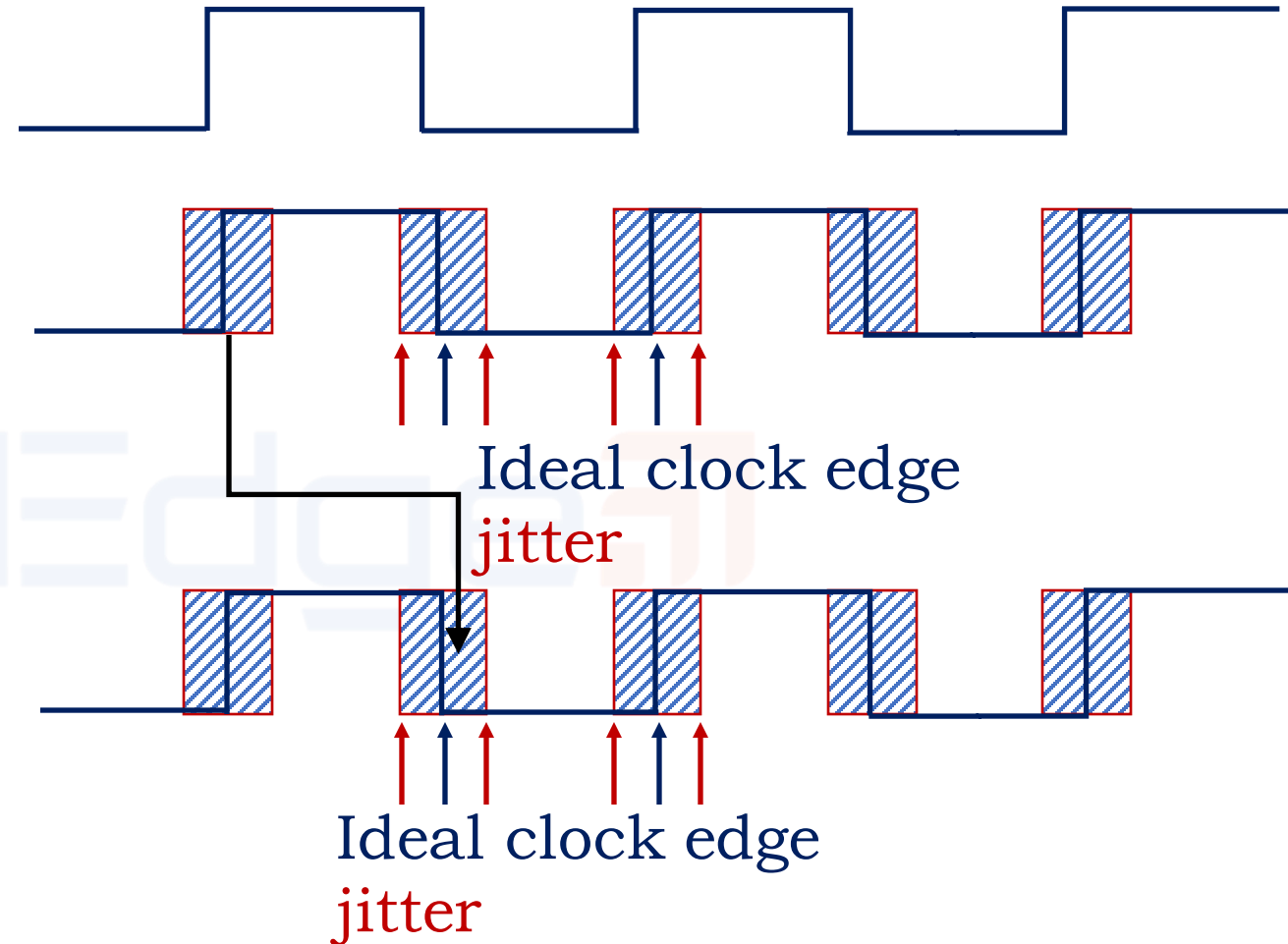
```
=====
==== Summary Reporting for Corner slow ====
=====
```

===== Summary Table for Corner slow =====											
Clock / Skew Group	Attrs	Sinks	Levels	Clock Repeater Count	Clock Repeater Area	Clock Stdcell Area	Max Latency	Global Skew	Trans DRC Count	Cap DRC Count	Wire Length

### Mode: func, Scenario: func_slow clock	M,D	723	9	32	11.81	40.80	0.56	0.28	0	3	3136.69

All Clocks		723	9	32	11.81	40.80	0.56	0.28	0	3	3136.69

- Clock Jitter is deviation of a clock edge from its ideal location
- Clock source (PLL) Jitter.
- Clock tree introduced Jitter.



Introduction to CTS



CTS Specifications



CTS QoR



Clock Uncertainty

This section explains about clock uncertainty, stage wise timing margins, calculations of set-up slack and hold slack using uncertainty with the example problems.

CTS & Opt-Quality Checks



Additional Topics



CTS Distribution Methodologies



- Clock Uncertainty: Extra margin introduced into timing analysis to consider the impacts seen in later stages
 - Clock Uncertainty at block level components
 - Clock source (PLL) Jitter
 - Clock tree introduced Jitter
 - Sign-off timing margins
 - IR drop impact on timing
- Applied all through the flow and for signoff

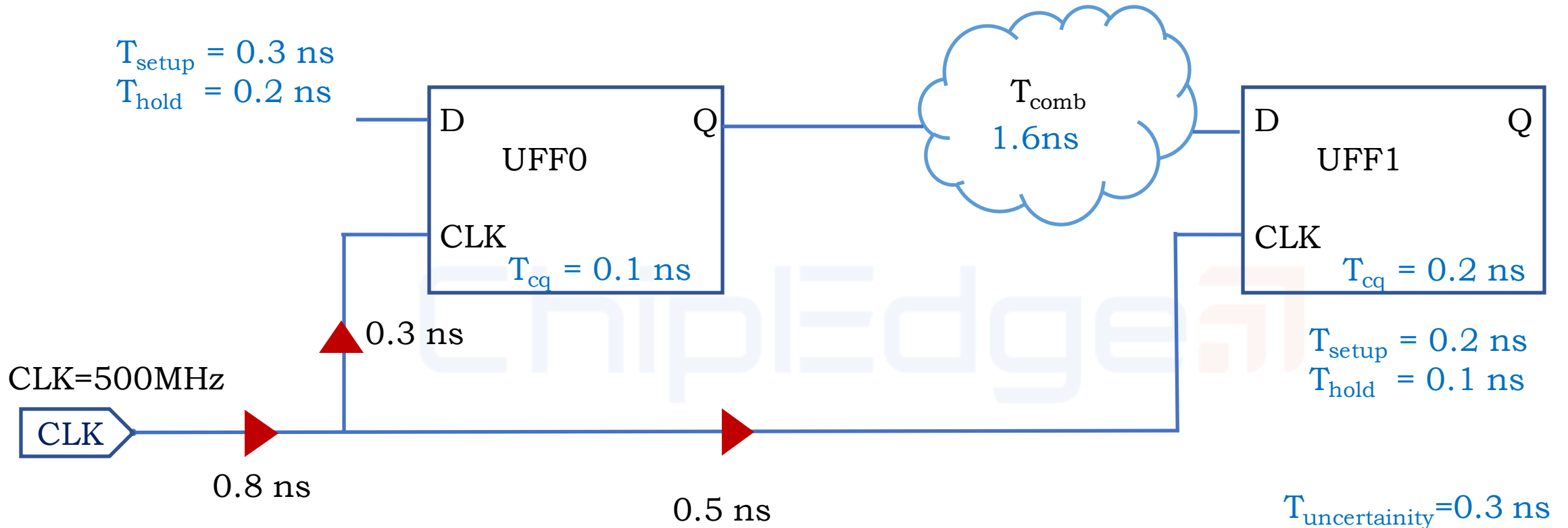
Stage wise timing margins to account for uncertainties

- Synthesis stage: Net delays + skew + cross talk + top level impacts on blocks.
- At placement stage: skew + cross talk + top level impacts on blocks (due to crosstalk, clock divergence etc)
- CTS stage: cross talk + top level impacts on blocks
- Routing stage : top level impacts on blocks

Applied
based on
stages of the
flow

- Setup Equation with Uncertainty
- Setup Slack = Required time – Arrival time
 - $= [T_{cap} + T_{clk} - T_{uncertainty} - T_{setup}] - [T_{lan} + T_{cq} + T_{comb}]$
 - $= [T_{clk} - T_{setup}] - [T_{cq} - T_{comb}] + skew - T_{uncertainty}$
- Hold equation = Arrival time – Required time
 - $= [T_{lan} + T_{cq} + T_{comb}] - [T_{cap} + T_{uncertainty} + T_{hold}]$
 - $= [T_{cq} + T_{comb}] - [T_{hold}] - skew - T_{uncertainty}$

Setup Slack(reg2reg) with Uncertainty



Condition to be MET for setup check (Setup Equation with Uncertainty)

$$T_{\text{clk}} - T_{\text{uncertainty}} + T_{\text{skew}} \geq T_{\text{cq1}} + T_{\text{comb}} + T_{\text{setup2}}$$

$$T_{\text{clk}} - T_{\text{uncertainty}} + [T_{\text{capture}} - T_{\text{launch}}] \geq T_{\text{cq1}} + T_{\text{comb}} + T_{\text{setup2}}$$

- Setup Slack = Required time – Arrival time

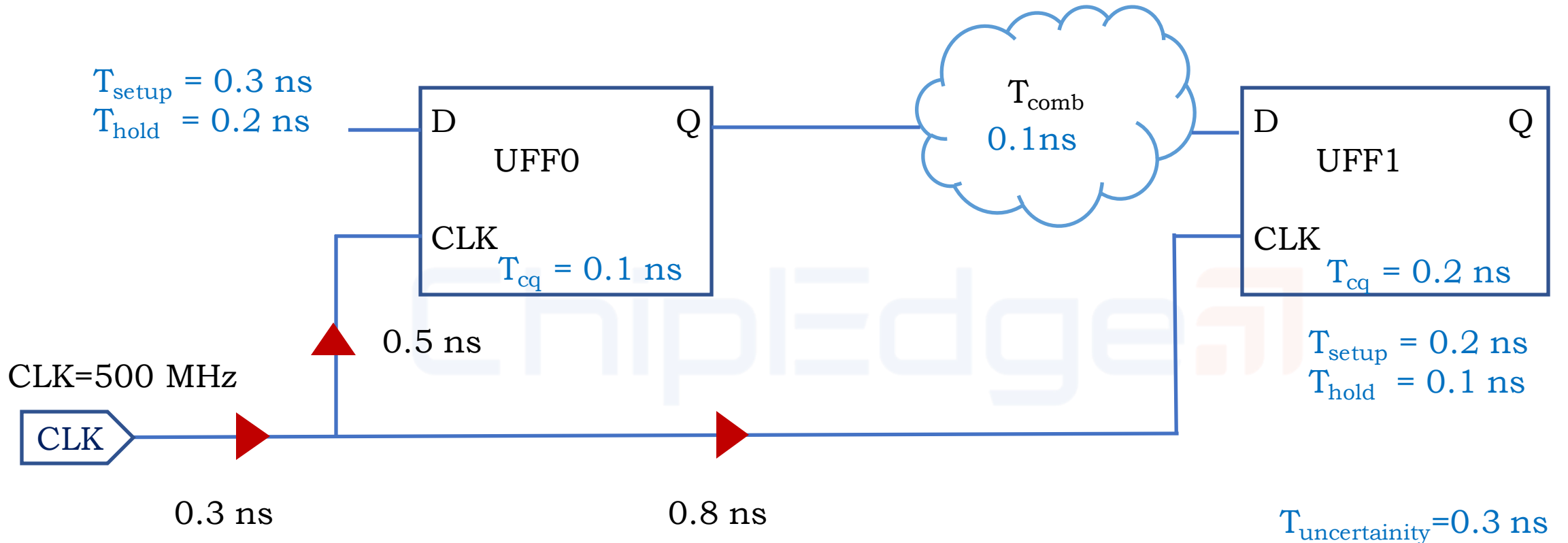
$$= [T_{\text{cap}} + T_{\text{clk}} - T_{\text{uncertainty}} - T_{\text{setup}}] - [T_{\text{lan}} + T_{\text{cq1}} + T_{\text{comb}}]$$

$$= [T_{\text{clk}} - T_{\text{setup}}] - [T_{\text{cq}} + T_{\text{comb}}] + [T_{\text{cap}} - T_{\text{lan}}] - T_{\text{uncertainty}}$$

$$= [2 - 0.3] - [0.1 - 1.6] + [1.3 - 1.1] - 0.3 = -0.1$$

Since setup slack < 0 setup is VIOLATED

Hold Slack(reg2reg) with Uncertainty



Condition to be MET for hold check (Hold Equation with Uncertainty)

$$T_{\text{hold}} + T_{\text{uncertainty}} + T_{\text{skew}} \leq T_{\text{cq1}} + T_{\text{comb}}$$

$$T_{\text{hold}} + T_{\text{uncertainty}} + [T_{\text{capture}} - T_{\text{launch}}] \leq T_{\text{cq1}} + T_{\text{comb}}$$

- Hold equation =Arrival time –Required time

$$= [T_{\text{lan}} + T_{\text{cq}} + T_{\text{comb}}] - [T_{\text{cap}} + T_{\text{uncertainty}} + T_{\text{hold}}]$$

$$= [T_{\text{cq}} + T_{\text{comb}}] - [T_{\text{hold}}] - [T_{\text{capture}} - T_{\text{lan}}] - T_{\text{uncertainty}}$$

$$= 0.1 + 0.1 - 0.1 - [1.1 - 0.8] - 0.3 = -0.5$$

Since hold slack < 0 hold is VIOLATED

Introduction to CTS



CTS Specifications



CTS QoR



Clock Uncertainty



CTS & Opt-Quality Checks

This section explains about setup and hold timing, DRV, congestion, utilization and CTS care abouts

Additional Topics



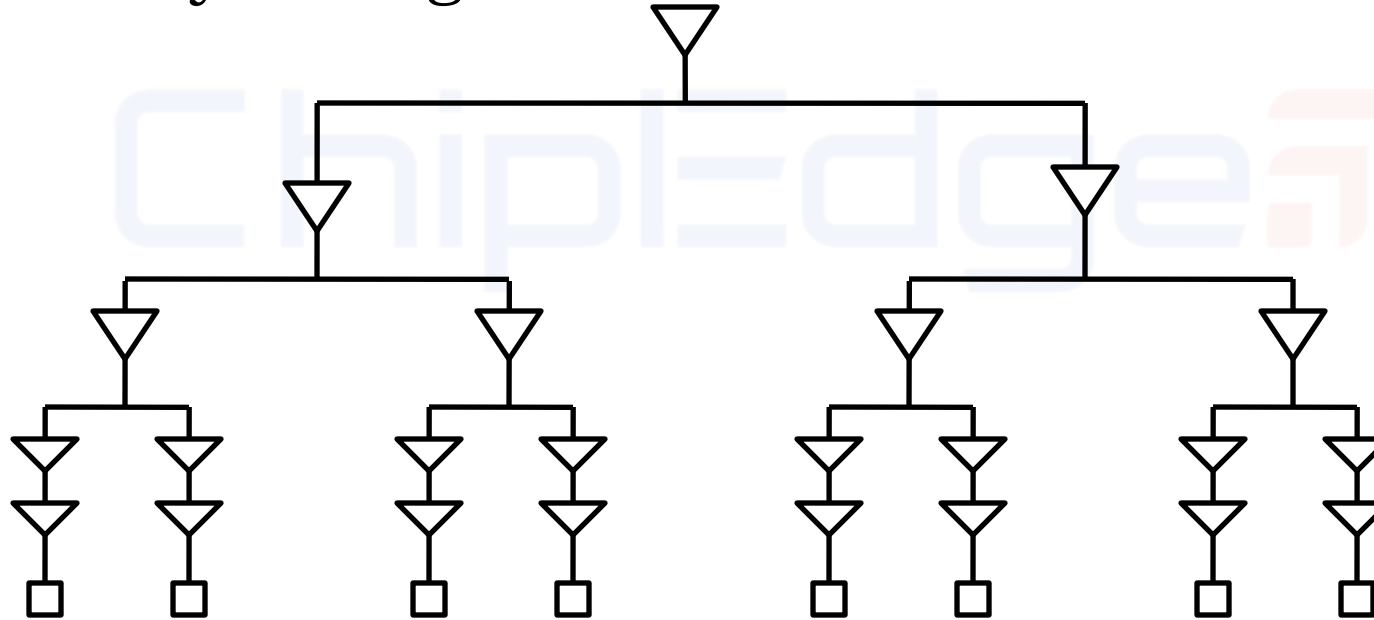
CTS Distribution Methodologies



- Setup & hold timing
- DRV (Clock and Data nets)
- Congestion
 - Clock tree leading to congestion
 - Setup/hold optimization leading to congestion
- Utilization
 - Utilization increase after CTS
 - Utilization increase after setup/hold optimization
 - Number of hold buffers added

- CTS buffer clustering
- CTS buffers leading to power density/dynamic IR issues Decaps
- Clock duty cycle issues
- Clock tree induced jitter
- Cross-talk on Clock trees

- Balanced clock tree - Most commonly used approach
- Based on location of clock sinks, group them into clusters
- Build Tree for all individual clusters
- Balance clusters by adding buffers at the root of the cluster



Introduction to CTS



CTS Specifications



CTS QoR



Clock Uncertainty



CTS & Opt-Quality Checks



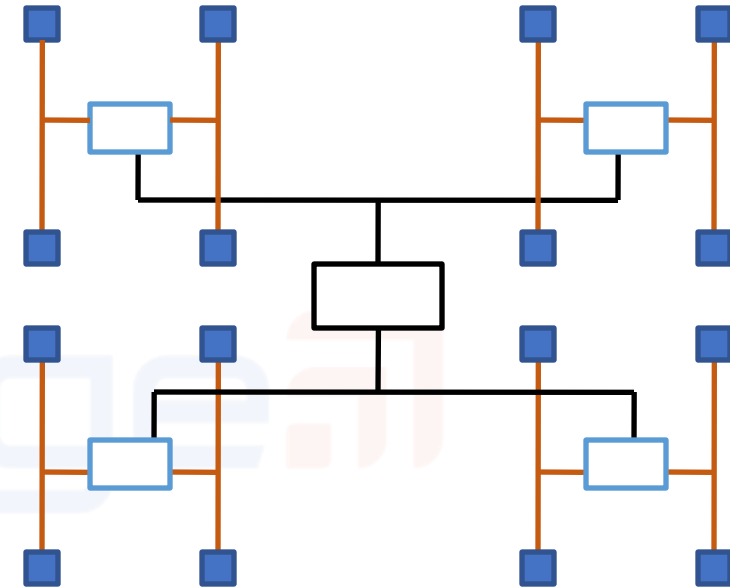
Additional Topics



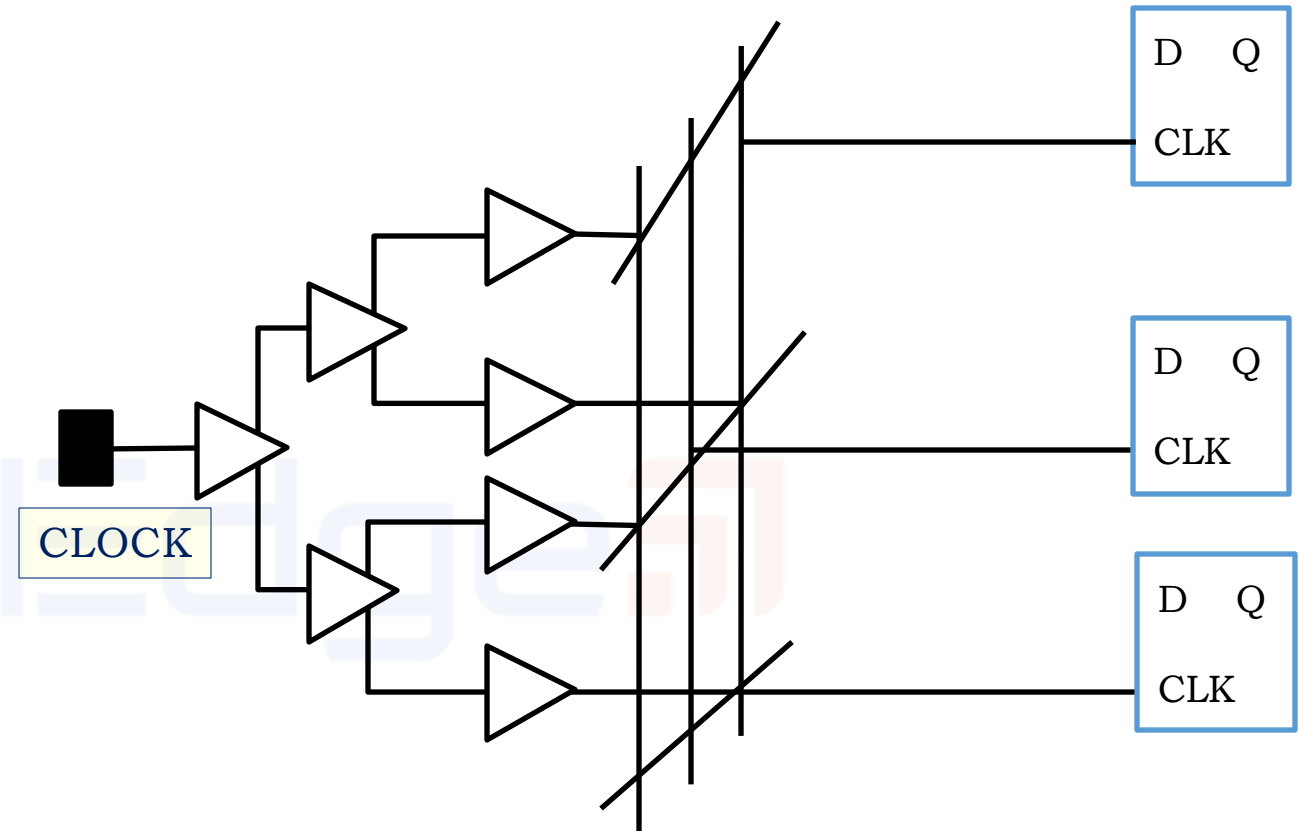
CTS Distribution Methodologies

This section explains the methodologies used to achieve balanced skew and minimum insertion latency.

- Also Called Binary tree
- Each driver has 2 symmetric sinks
- Load/Routing all aspects should be matched at every level.
- Can achieve very low skew with reasonable buffer/inv count
- May not be suitable when related logic on same clock domain have to be scattered



- Generally used on very high-speed design (like Microprocessors)
- Clock gating cells are spread uniformly in design area irrespective of clock sinks.
- Based on placement, clock sinks are connected to clock gating cells.
- Require advanced SPICE analysis to find actual insertion delay and skew



- Impact of Clock tree delays on timing
- Understanding Clock distribution - CTS
- CTS in ASIC Flow - Stages
- CTS Spec
- CTS QOR
- CTS care-about
- Clock distribution methodologies

Module 11

Routing

- In this module you will understand what is Routing and its importance in the Physical design flow.
- The concept of cross talk, antenna effect are discussed.

ChipEdge

- Introduction to Routing
 - ✓ Prerequisites
 - ✓ Check for Routability
- Routing Flow
 - ✓ Global Route
 - ✓ Track Assignment
 - ✓ Detail Route
 - ✓ Search and Repair
- Post Route Optimization
 - ✓ Routing Switches
- Cross Talk
- Antenna Violation
 - ✓ Antenna Fixes
 - ✓ Redundant via Insertion effort
 - ✓ Routing Quality Summary
- Routing Quality Summary

Introduction to Routing

This section gives the introduction to routing and different steps involved in routing stage

Routing Flow



Post Route optimization



Cross Talk



Antenna Violation

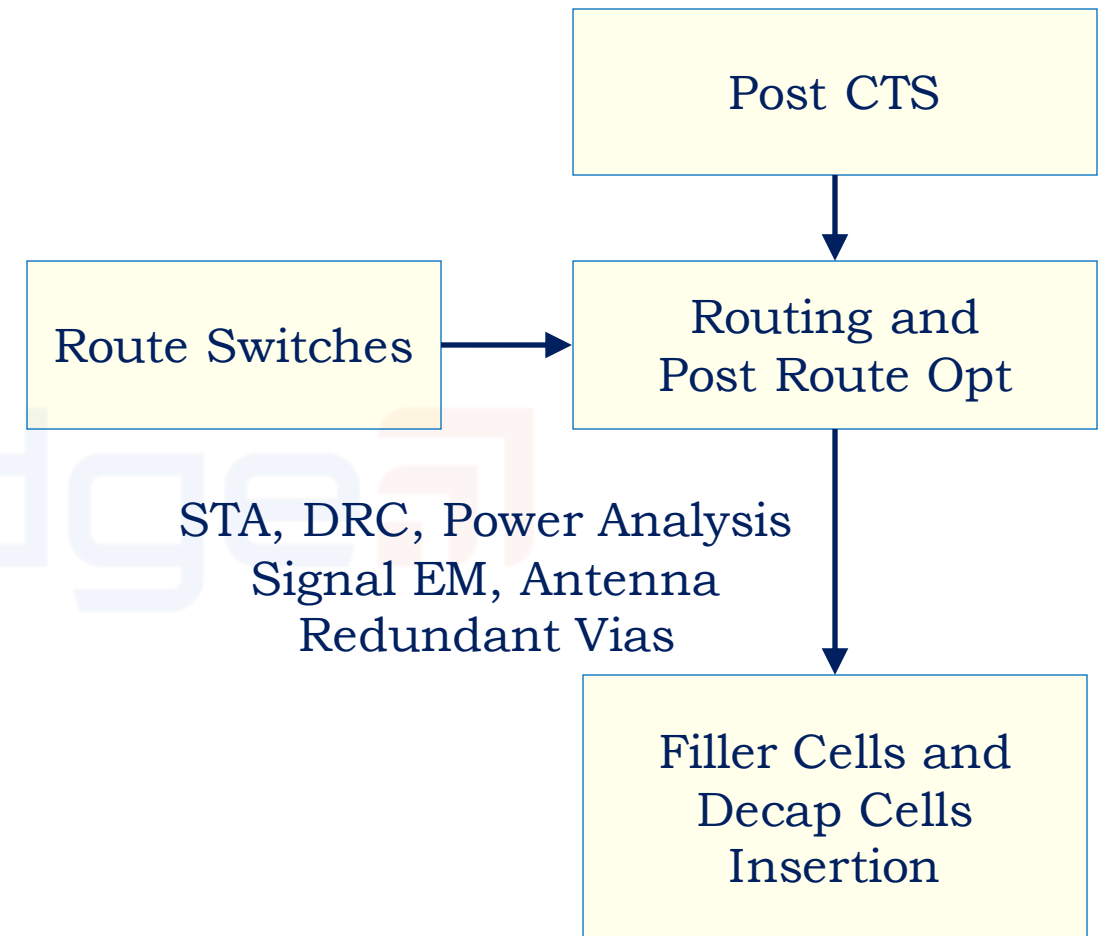


Routing Quality Summary



What is Routing?

- Routing is the process of creating physical connections based on logical connectivity.
- Signal pins are connected by routing metal interconnects.
- Routed metal paths must meet timing, clock skew, max trans/cap requirements and also physical DRC requirements.



- All ports should be legally placed and fixed.
- All cells should be legally placed.
- Special routing data
 - ✓ Non-Default Routing (NDR)
 - ✓ Routing blockages
- Technology data
 - ✓ Metal layers (LEF, Tech file)
 - ✓ DRC rules
 - ✓ Via creation rules
 - ✓ Grid rules (Metal pitch)

- The design must be placed and optimized. CTS and optimization should be complete.
- The PG nets must be pre-routed and physically connected to all macro and standard cells.
- The timing DRC Violations and the timing QoR, estimated after CTS must be acceptable.
- The measured congestion should be tolerable.
- There should not be any ideal nets.

- Checks if the ports of the standard cells are blocked
- Checks for overlapping cells.
- Checks for pins underneath PG routes (they may be inaccessible and cause violations on metals).
- Check if the ports of the top level or macro cell are blocked and physically inaccessible.
- Check for pins that are outside the design boundary.

```
== Check for design ==  
=====
```

```
>>> No net contains a large number of ports
```

```
>>> No port contains a large number of pins
```

```
=====
```

```
== Check for PG PreRoute setting ==
```

```
=====
```

```
== Check for overlap of standard cells ==
```

```
>>>>> No overlap of standard cells found
```

```
=====
```

```
== Check for min-grid violations ==
```

```
>>>>> No min-grid violations found
```

```
=====
```

```
== Check for out-of-boundary ports ==
```

```
>>>>> No out-of-boundary error found
```

```
-----
```

```
=====
```

```
== Check for blocked ports ==
```

```
=====
```

```
>>>>> No blocked nets found
```

```
=====
```

```
== Check for shielding violations ==
```

```
=====
```

```
>>>>> The following Non-PG net has a shape with shape_use shield_route
```

```
>>> No non-PG net has a shape with shape_use shield_route.
```

```
>>>>> The following PG net has shape with shape_use detail_route and shape length is too long.
```

```
>>> No PG net has shape with shape_use detail_route and shape length is too long.
```

```
>>>>> The following nets with shield non-default rule has no internal data representation in Zroute.
```

```
>>> No nets with shield non-default rule has no internal data representation in Zroute.
```

```
=====
==      Check for via cut blockage violation      ==
=====

>>>>> The following via cut blockages are wrongly treated as real vias.

>>> No via cut blockage is wrongly treated as real via.

=====
==      Check for custom via definition      ==
=====

>>> No custom via definition has too many cuts

=====
==      Check for via master      ==
=====

>>> The total number of via master definition is 306

=====
==      Check non-default rule setting      ==
=====

>>> No NDR width is larger than signalRouteMaxWidth

>>> No NDR shield width is larger than signalRouteMaxWidth
```

```
=====
==      Check library cell has non-routing layer pins      ==
=====

>>> No cell has non-routing layer pins to be connected.

=====
==      Check over promoted nets      ==
=====
Updating congestion ...
Updating congestion ...

>>> No over promoted nets.

End of check_routability
```

Introduction to Routing



Routing Flow

This section illustrates the process of Routing Flow through each individual steps like

Post Route optimization



Global Route, Track Assignment, Detail Route and Search and Repair.

Cross Talk



Antenna Violation



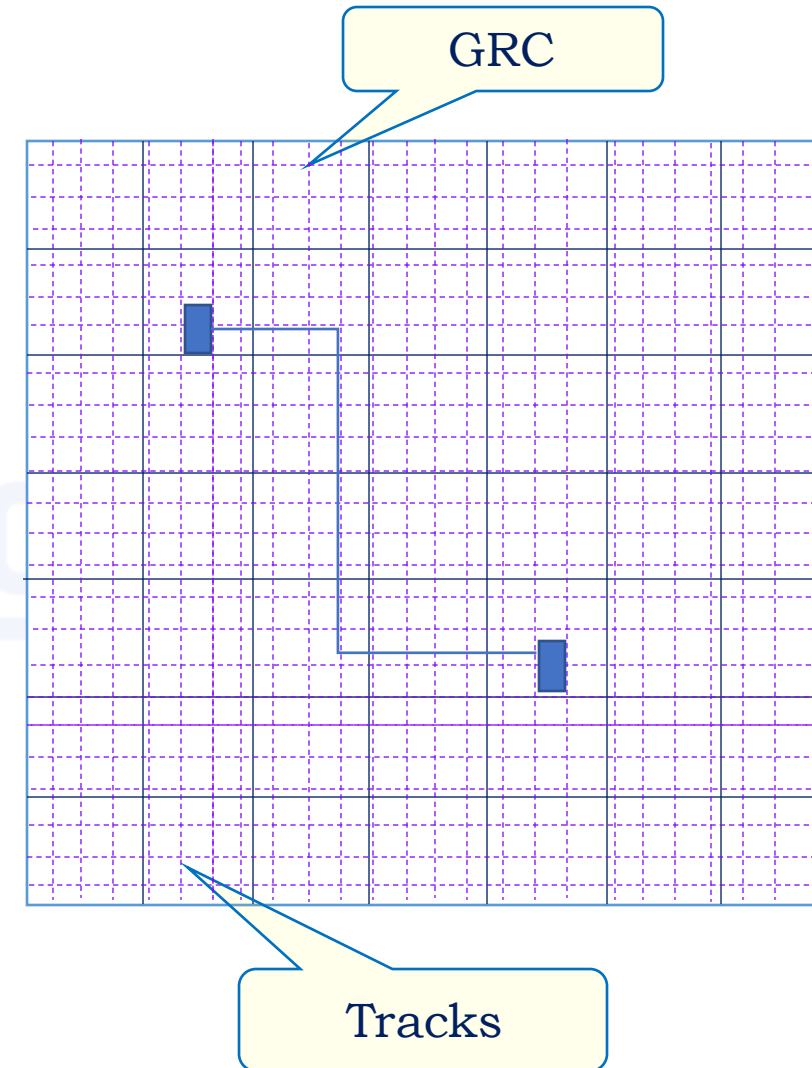
Routing Quality Summary



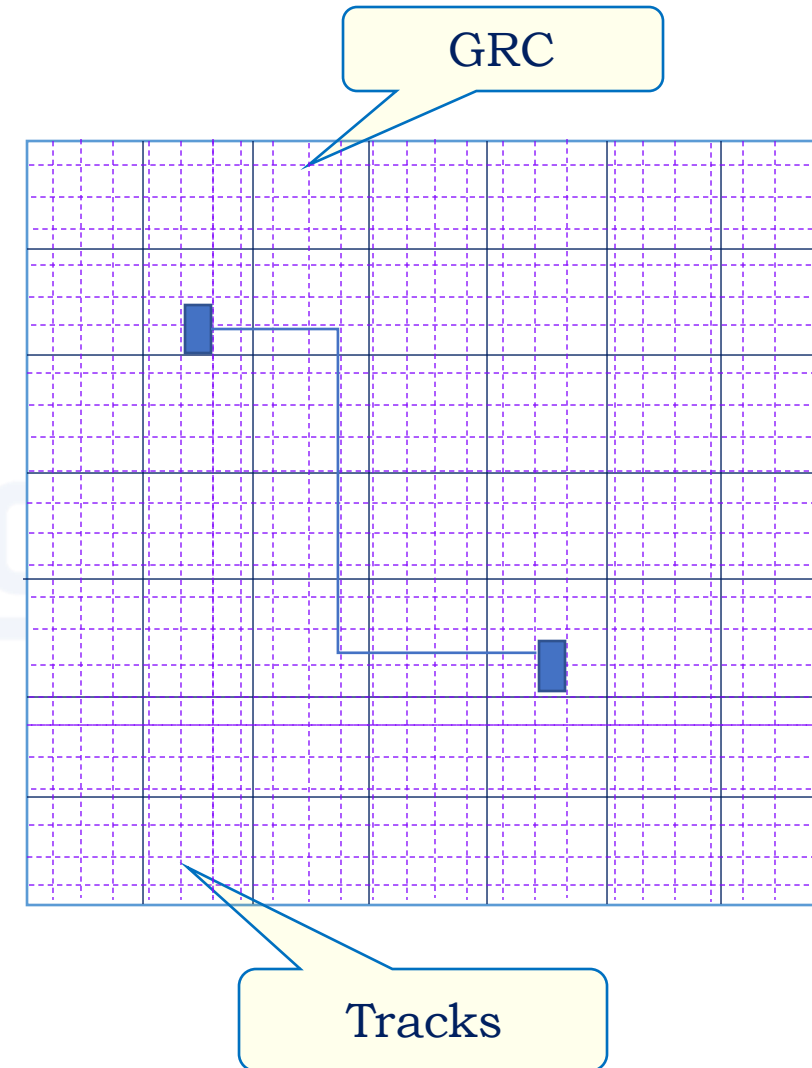
`route_auto` is the command for routing. Internally following steps are run as part of routing.

1. Global Route
2. Track Assignment
3. Detail Route
4. Search and Repair

- Global routing is an estimate of how the routing might take place with the current placement.
- The chip area is virtually divided into small boxes (GRC) .
- Wire segments are tentatively assigned(embedded) within the chip layout.
- Chip area is represented by a coarse routing grid.



- Assign layers to nets, although loosely.
- Assign net segments to GRCs
- Avoid long detours.
- Repetitive global route runs to reduce congestion.
- Avoid routing over blockages.
- Avoid routing pre-route nets.



- Track assignment is a stage where the routing tracks are assigned for each global routes.
- The tasks are performed during the stage are
 - ✓ Assigning tracks in vertical and horizontal partitions
 - ✓ Rerouting of overlapped wires
- It does not follow the physical DRC rules.
- It will do the timing aware Track Assignment.
- It tries to do Via Minimization (reduce metal layer switching).
- Track Assignment replaces all global routes with actual metal layers.

- The Detail router uses the routing plan laid by the router during the global routing and track assignment.
- Detailed routing follows up with the track routed net segments and performs the complete DRC aware and timing driven routing.
- The violations that were created during the track assignment stage are fixed in this stage.
- The main goal of detailed routing is to complete all of the required interconnect without leaving open, shorts or DRCs.

- The search and repair stage fixes the DRC Violations left out at the end of detail routing.
- In search and repair, shorts and DRCs are located and rerouting of affected areas to fix all possible violation is executed .

Introduction to Routing



Routing Flow



Post Route optimization

This section gives the detailed explanation of route_opt command.

Cross Talk



Antenna Violation



Routing Quality Summary



- `route_opt` is the command used for post route optimization.
- After completing the Routing, the design is optimized to meet setup timing and hold timing.
- Optimization techniques like Vt cell swapping, cell sizing, buffer insertion etc are used to meet the timing targets.
- Incremental routing is done to fix the routing disturbances caused due to optimization.
- Apart from timing, the post route optimization also works to meet Power, Area targets.

1. Set of priority nets to route first
2. Specify metal routing layers for Signal nets
3. Selected layers for few priority nets
4. Non-default routing (NDR) for few nets : higher width and/or spacing
5. Timing and SI effort
6. Redundant via insertion effort
7. Antenna diodes

Introduction to Routing



Routing Flow



Post Route optimization



Cross Talk

In this section, the brief introduction to cross talk, cross talk noise and cross talk delay are explained with examples.

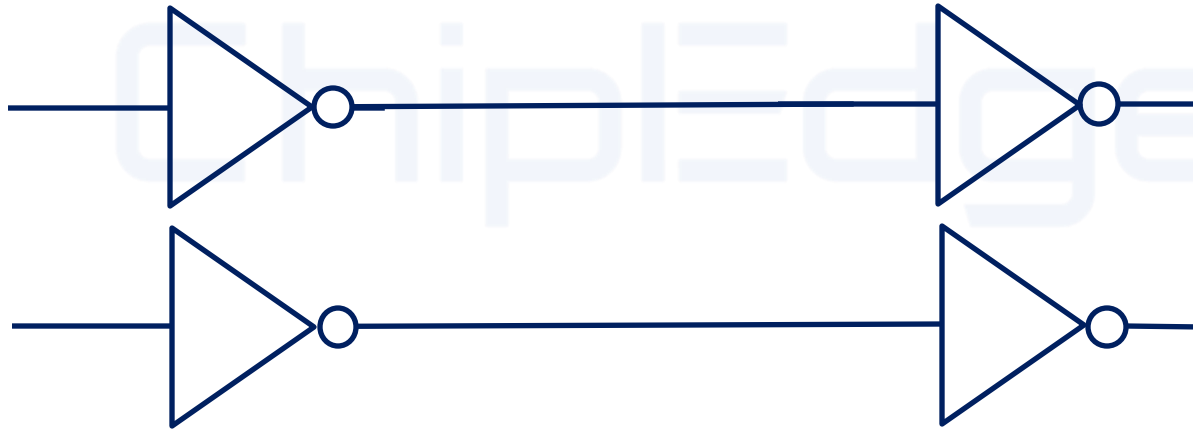
Antenna Violation



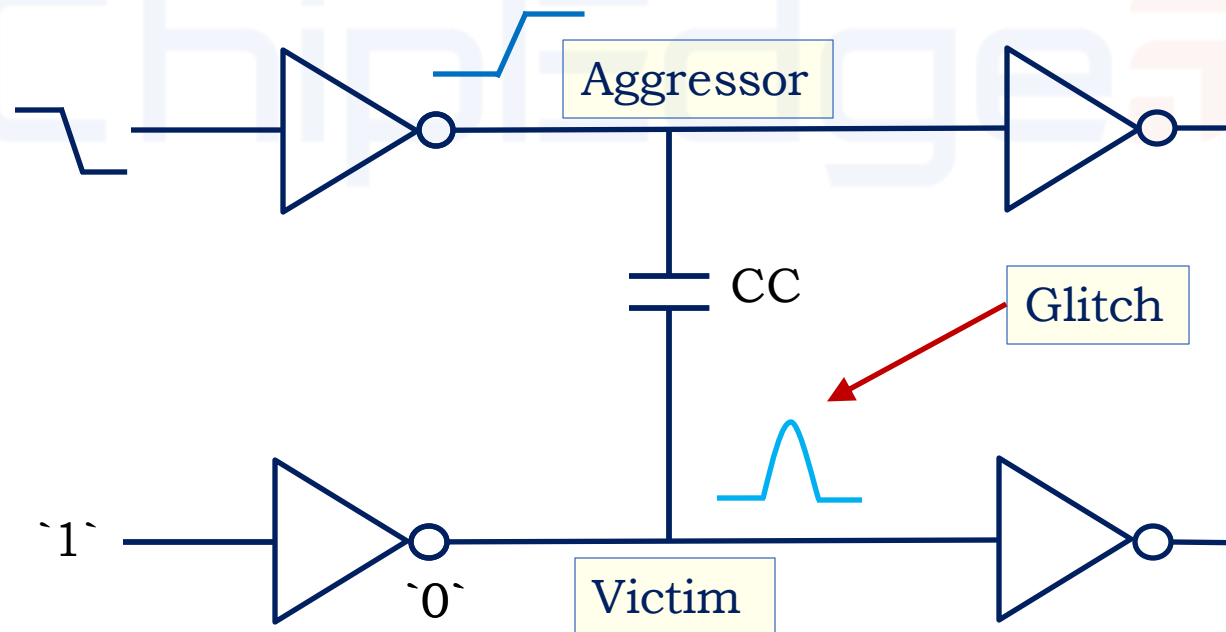
Routing Quality Summary



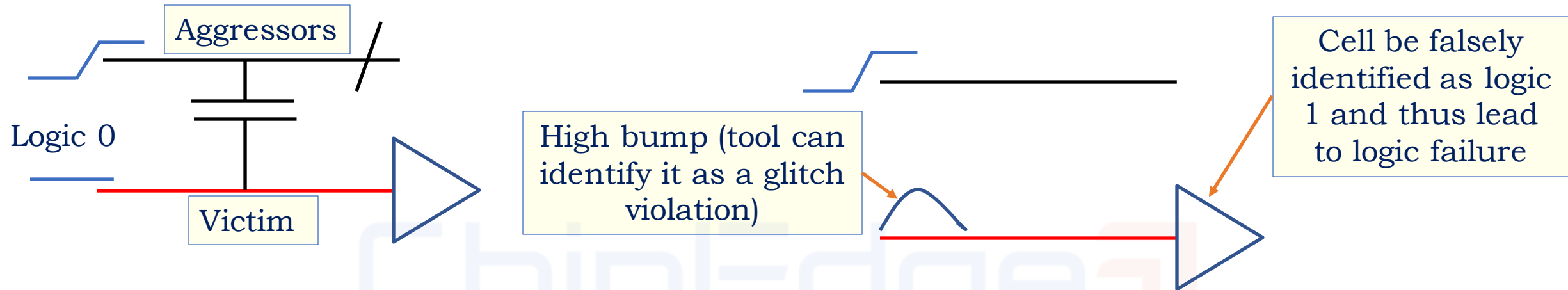
Cross Talk: Switching of the signal in one net can interfere with the neighboring net due to cross-coupling capacitance known as cross-talk.



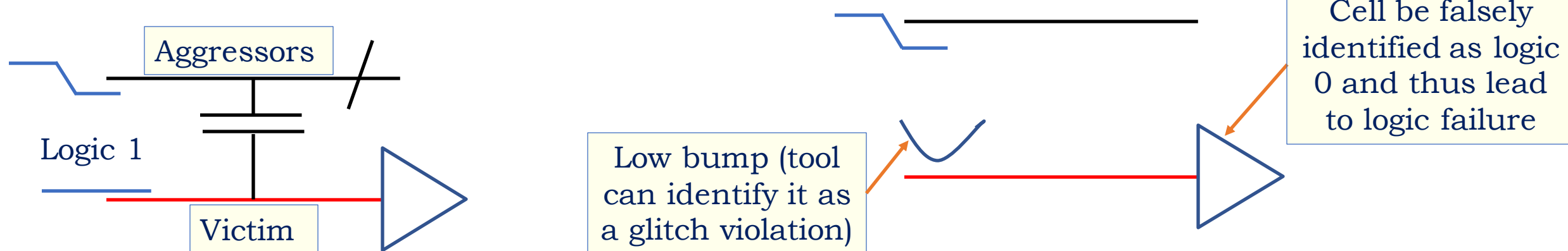
Cross Talk Noise: If one net is switching and the other is at a constant value, the switching net may cause voltage spikes on the other Net. This is called cross-talk noise.



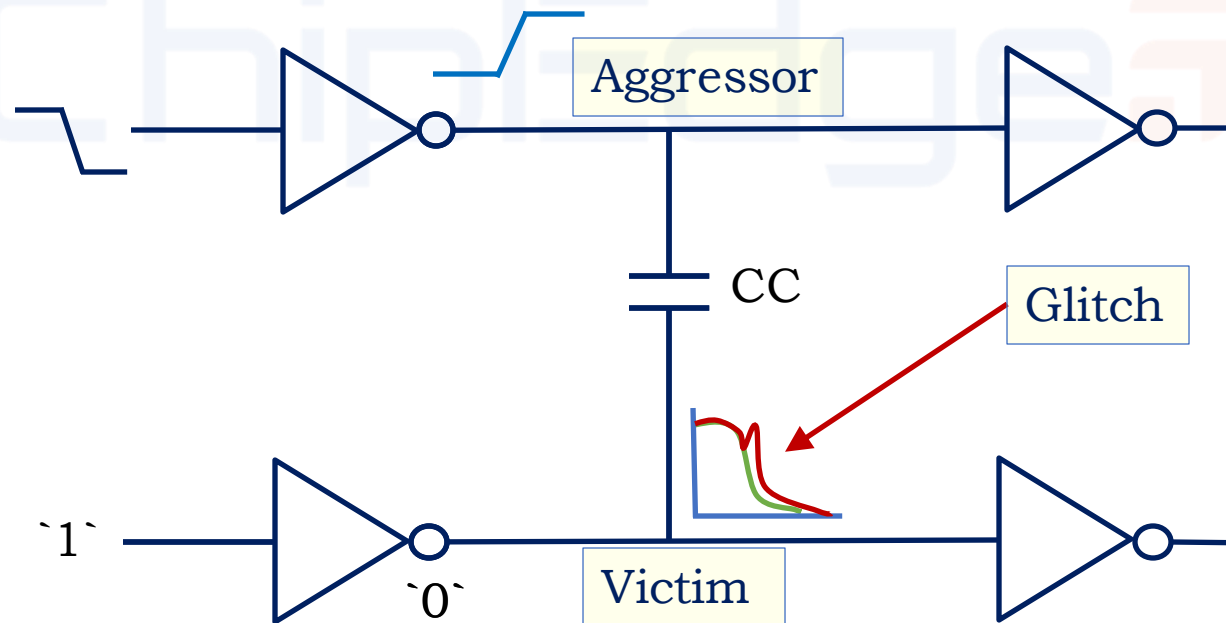
Aggressor switching & Victim at static LOW :



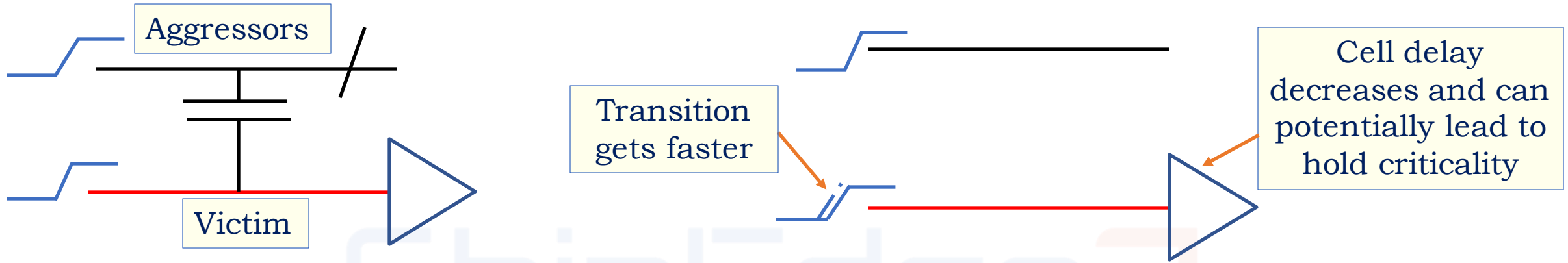
Aggressor switching & Victim at static HIGH :



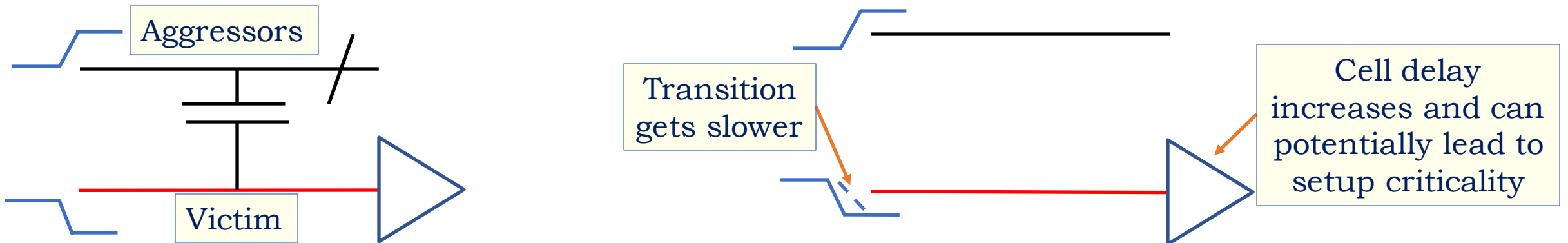
Cross Talk Delay: When both the nets are switching it leads to additional delay on the net called as cross talk delay. Cross talk delay can be positive or negative based on the switching directions of the nets. Cross-talk may lead to setup or hold violation.



Aggressor & Victim switching in same direction :



Aggressor & Victim switching in different direction :



To avoid cross-talk or minimal cross-talk effect:

- Increase the spacing between wires.
- Insert buffer if wire run length is too long.
- Jump to different metal layers.
- Shielding.
- Improve transition time on victim net .

Introduction to Routing



Routing Flow



Post Route optimization



Cross Talk



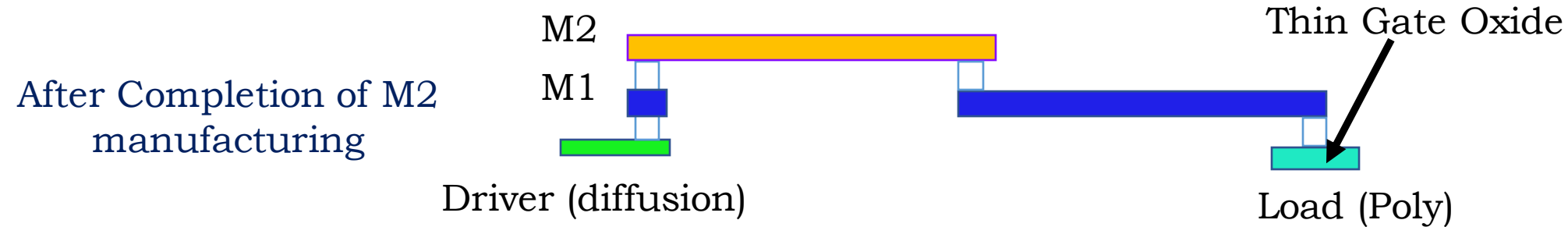
Antenna Violation

In this section, the Antenna Violation and Antenna violation fixes are discussed in detail.

Routing Quality Summary



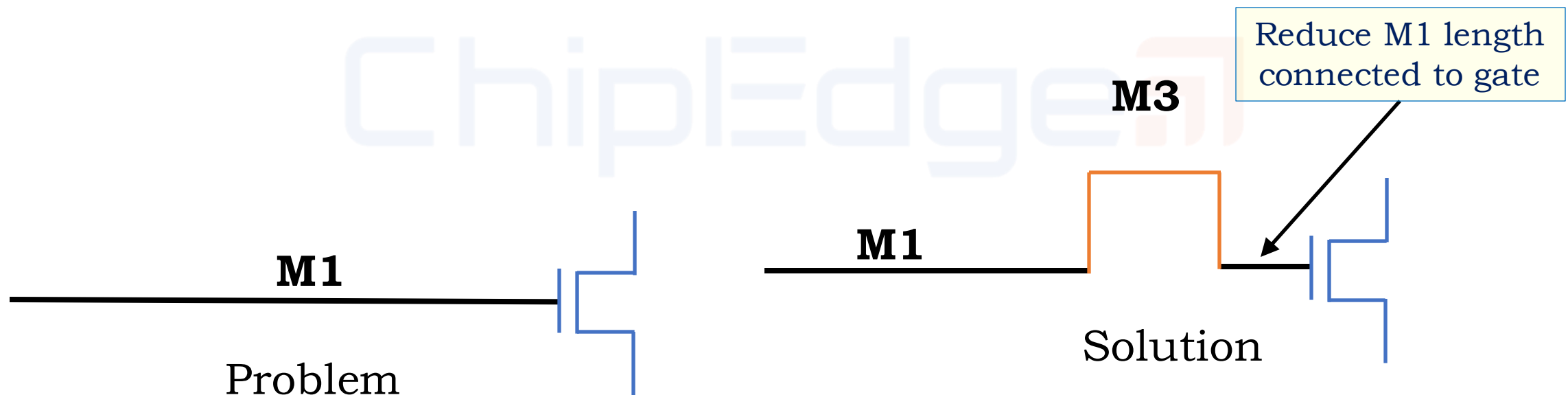
- The antenna effect, more formally plasma induced gate oxide damage, is an effect that can potentially damage the Gate of the MOSFET during Fabrication process.
- During etching process of metal layers, lot of charge is generated on these accumulated on the metals.
- When a long metal shape is connected to Gate, the accumulated charge will create high potential difference across gate thus damaging it.
- In design, this is represented by Antenna Ratio (metal to gate-area ratio), which if higher than a limit, leads to antenna violation.



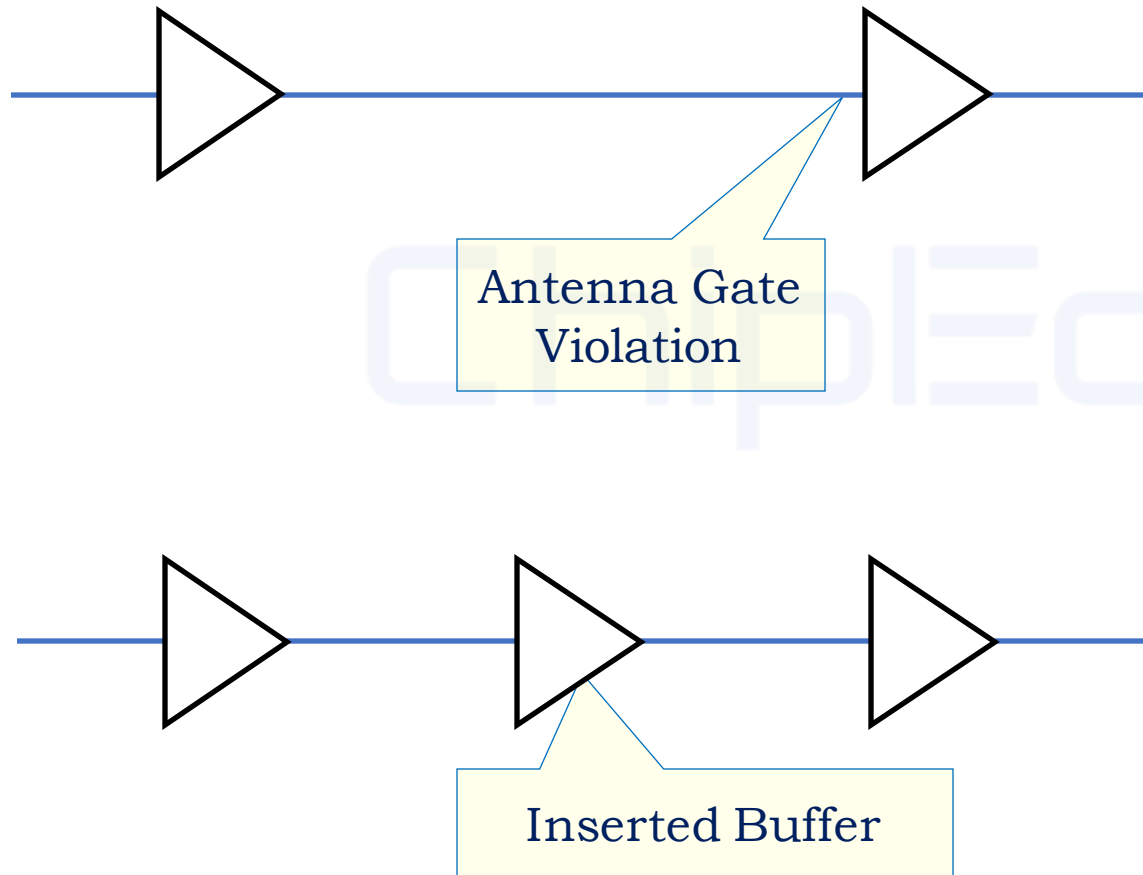
Antenna Ratio (Design) $\leq$ Antenna Ratio (Limit)

Antenna Ratio (Design) = Metal Area / (Gate Area + Diff Area)

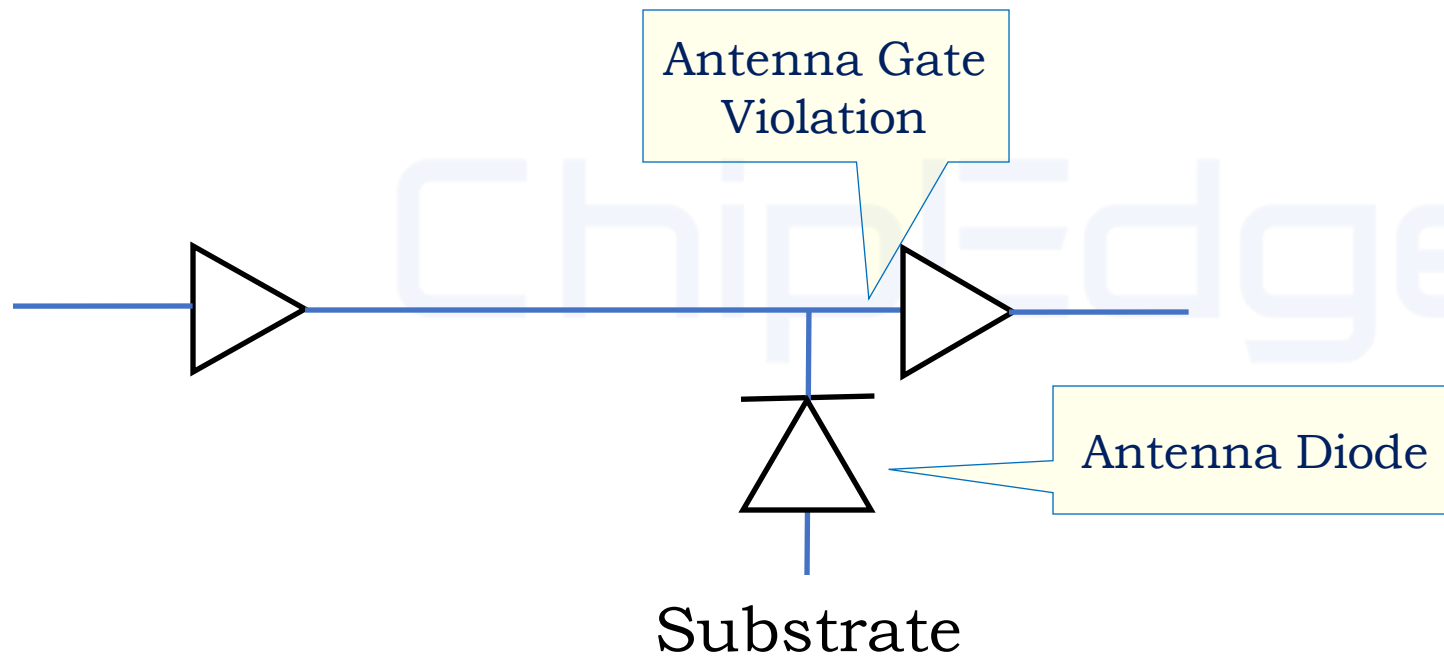
1. Metal Hopping – Wherever a long route is connected to the gate, try to break and route a segment in higher layer. This reduces the metal area connected to gate.

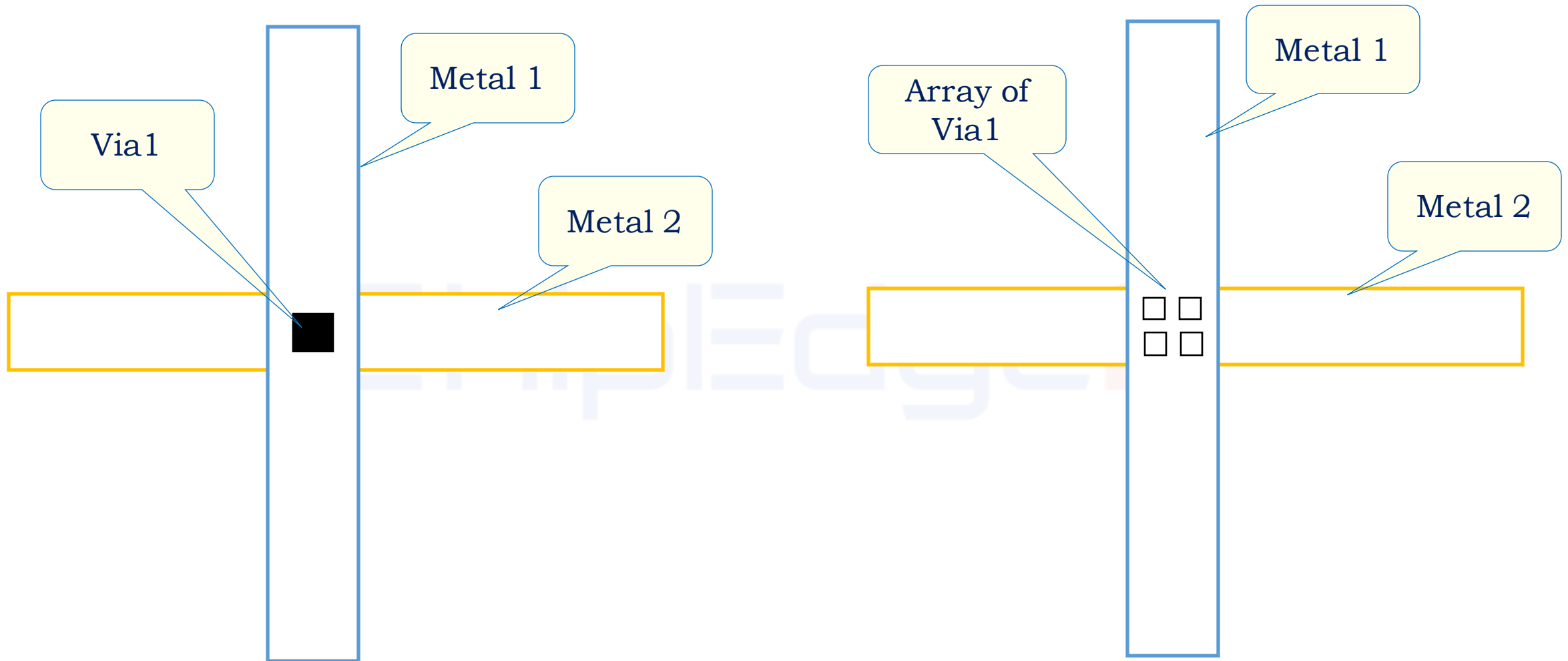


2. Insert Buffer to reduce the net length.



3. Add Antenna Diode.





Introduction to Routing



Routing Flow



Post Route optimization



Cross Talk



Antenna Violation



Routing Quality Summary

The brief explanation of Routing Quality Summary in terms of DRC and Timing are explained in this section.

- Analyze Physical DRC : `check_routes`.
 - ✓ Std cells pin-density – M1/M2 pins connectivity
- Check Timing QoR: `report_qor -summary`.
 - ✓ Setup and Hold Timing and DRVs (maxTran/MaxCap)
 - ✓ If issues are present
 1. routing option tweaks: metal layers, NDR / pre-routes
 2. Revisit Placement
 3. go back to Floorplanning

- What is Routing
- Prerequisites
- Check for Routability
- Routing Flow
- Routing Switches
- Antenna Violation
- Antenna Fixes
- Redundant via Insertion effort
- Routing Quality Summary

Module 12

Optimization Techniques

- In this module you will learn about overview of optimization and various Optimization techniques.
- The brief overview of timing optimization which include Setup optimization and Hold optimization are explained.
- In addition to these area and power optimization are explained.

- Introduction to Optimization
- Timing Optimization
 - ✓ Hold Fix
 - ✓ Setup Fix
- Power Optimization
 - ✓ Leakage Power
 - ✓ Dynamic Power
- Area Optimization

Introduction to Optimization

This section gives the introduction to optimization process

Timing Optimization



Power Optimization



Area Optimization



- Optimization is the process of iterating through a design such that it meets timing, area and power specifications. The design must satisfy these multiple design objectives.
- In general, optimization can be broken down into the following areas
 - Timing
 - Power
 - Area

Introduction to Optimization 

Timing Optimization

This section gives the detailed explanation of Hold Optimization, DRV optimization, setup and hold fixes.

Power Optimization



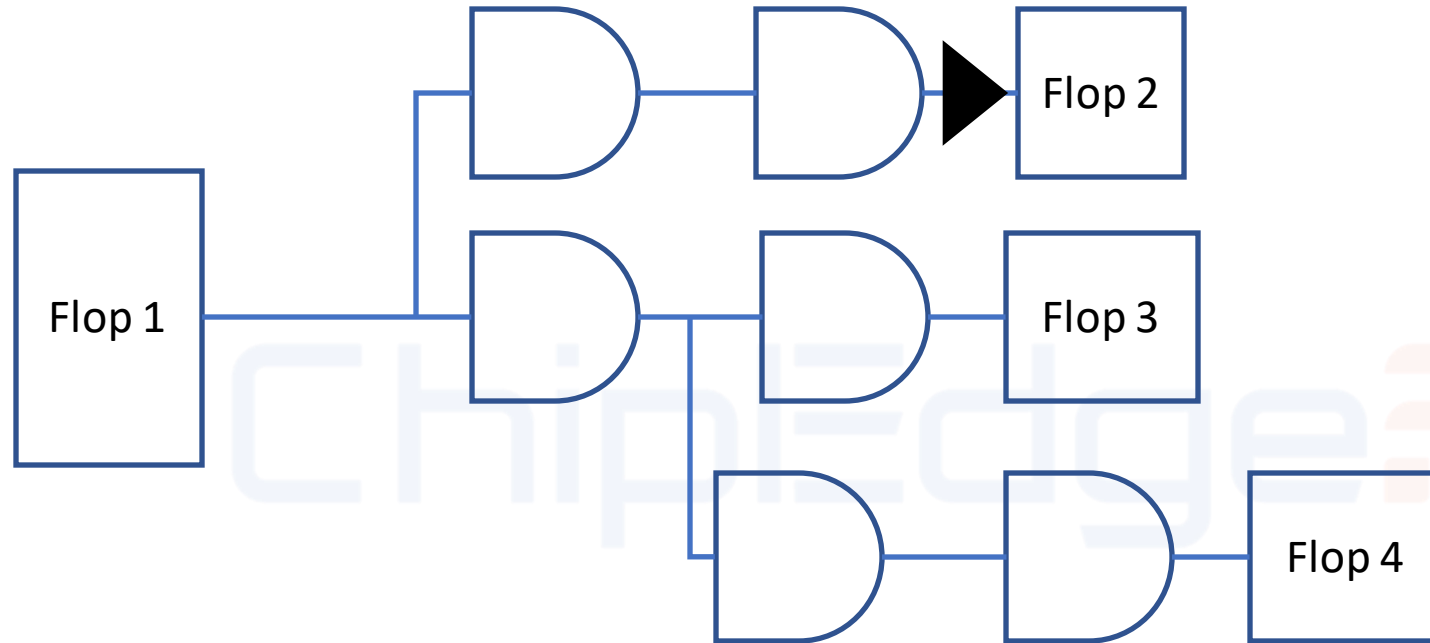
Area Optimization

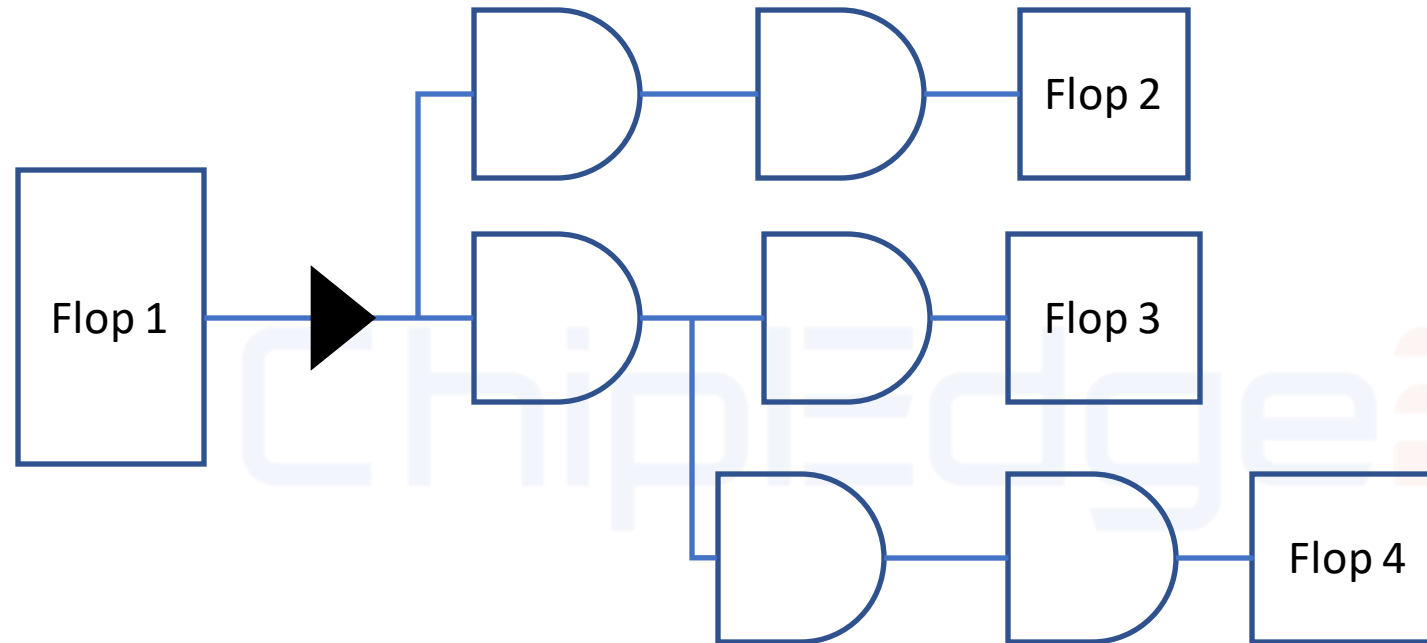


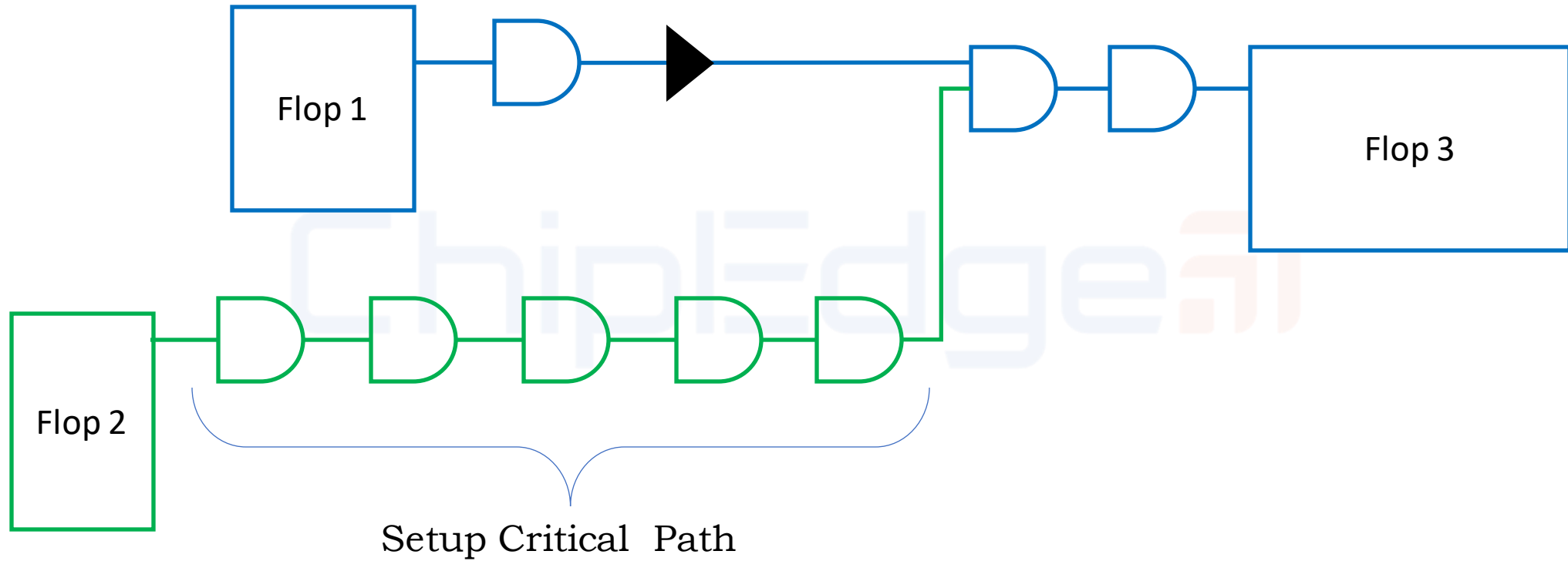
- Setup Optimization – Data Path Optimization.
- V_t Swapping (to LVT) for cells in data path to make them faster.
- Buffering of nets.
- Fanout splitting.
- Avoiding module splitting.
- Better floorplan for optimized routing.
- Routing of critical nets in thicker and wider layers: Reduced resistance, faster path.

Hold Optimization - Data path

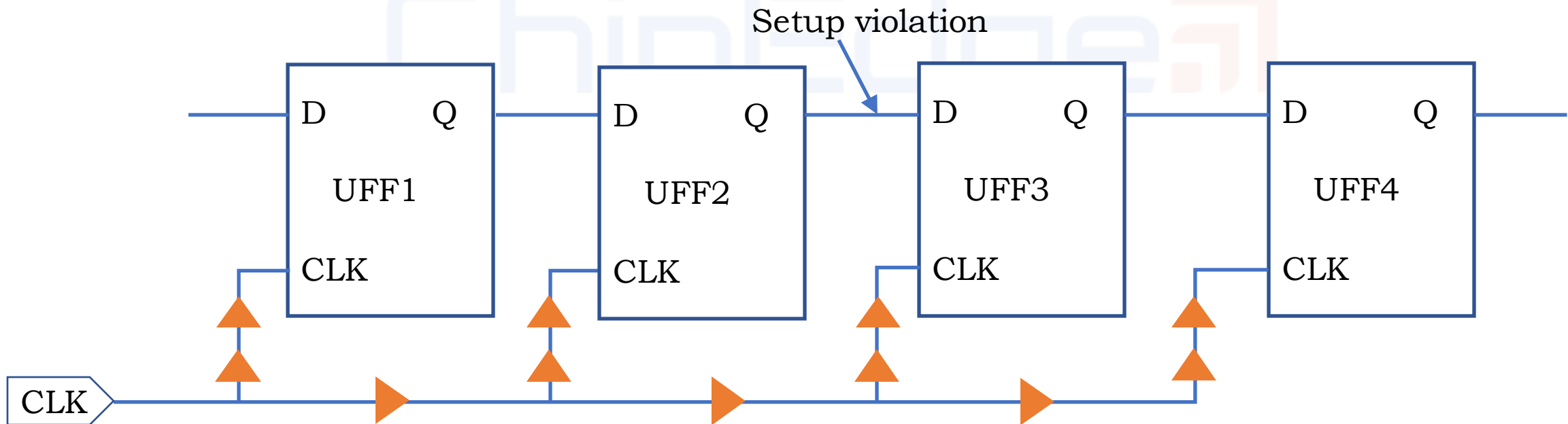
- End point buffering.
- Divergent point buffering.
- Start point buffering in few cases.
- Net detouring to fix marginal hold violations – Extra metal routing adds delays in data path.

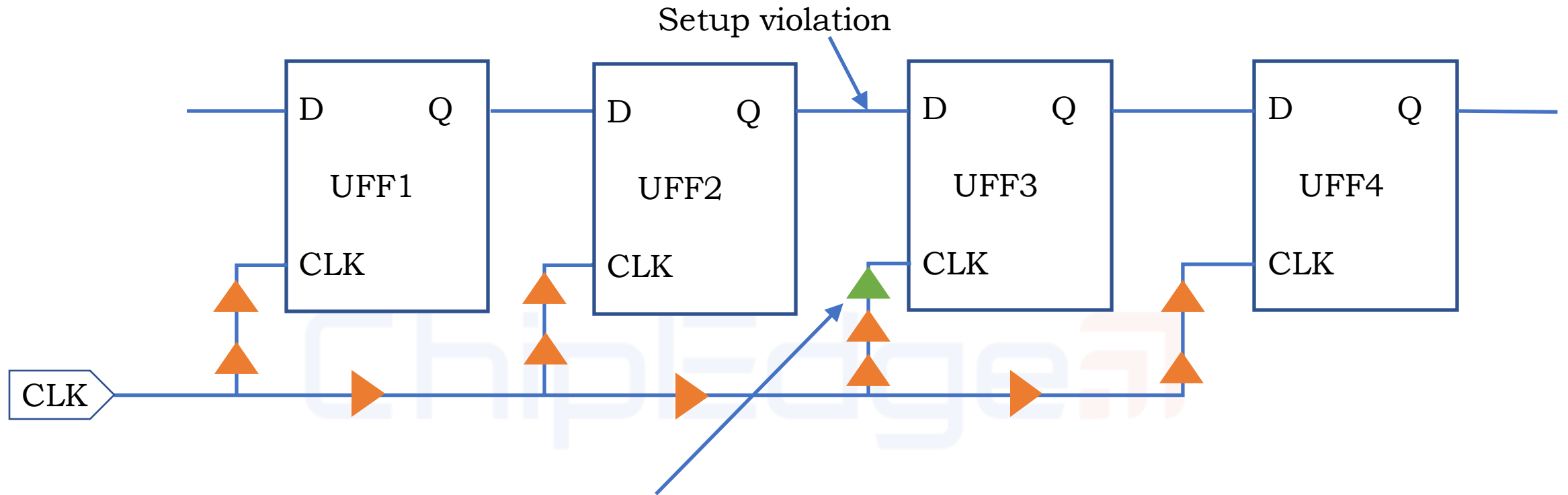




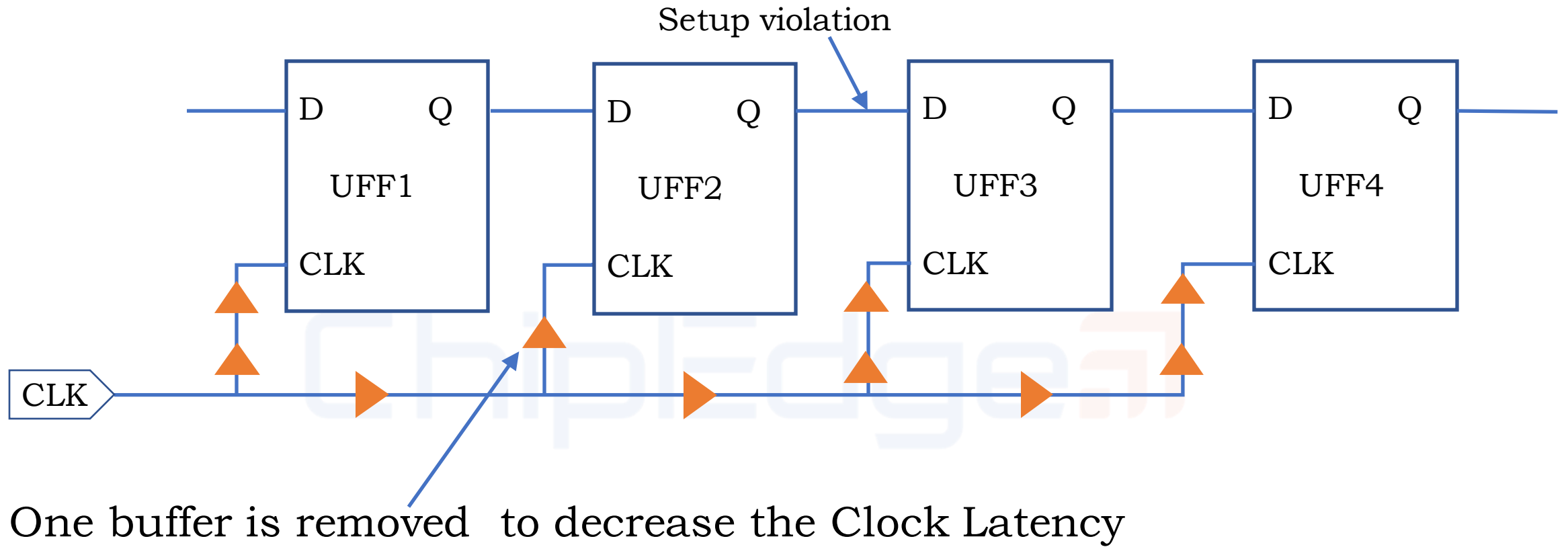


- Setup Violation can be fixed:
 - ✓ By increasing the capture flop clock latency (Clock push)
 - ✓ By decreasing the launch flop clock latency (Clock pull)

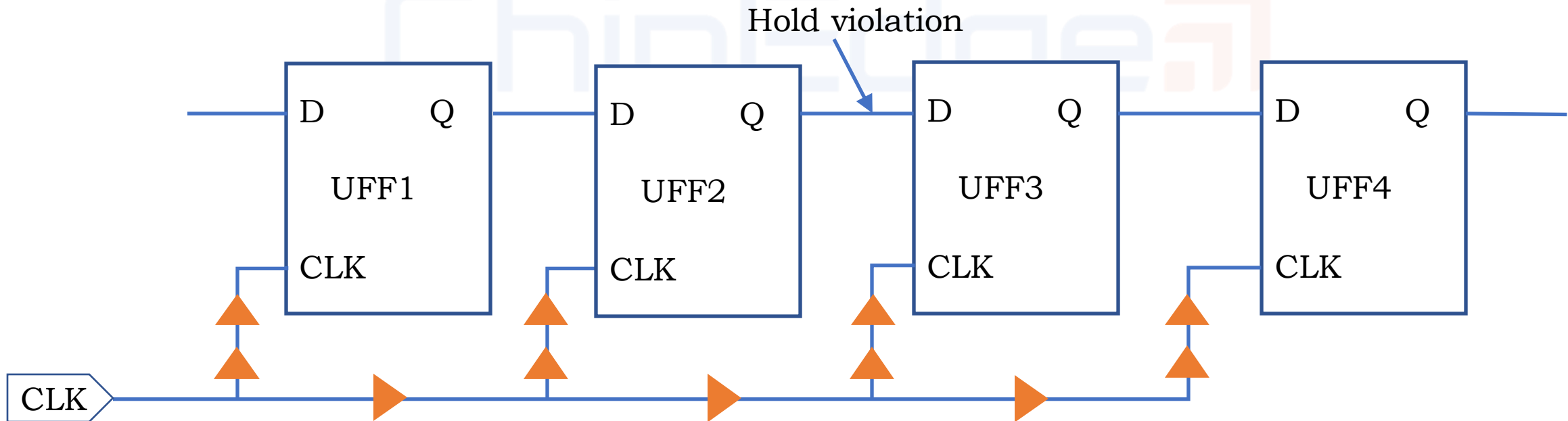


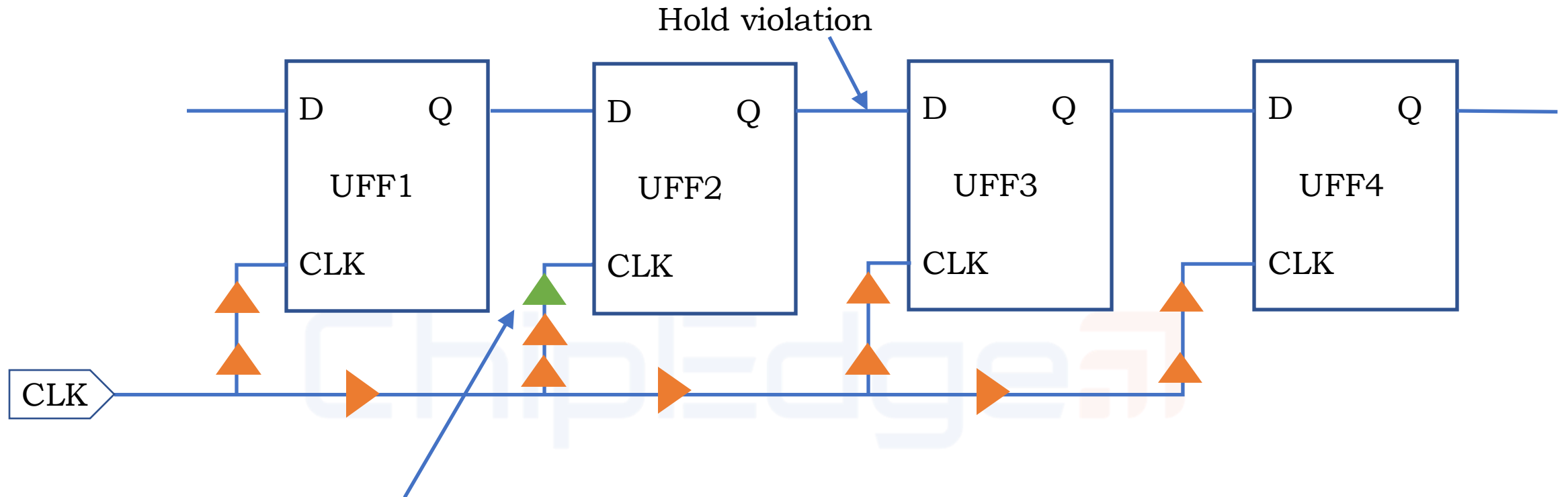


One buffer is added to increase the clock Latency

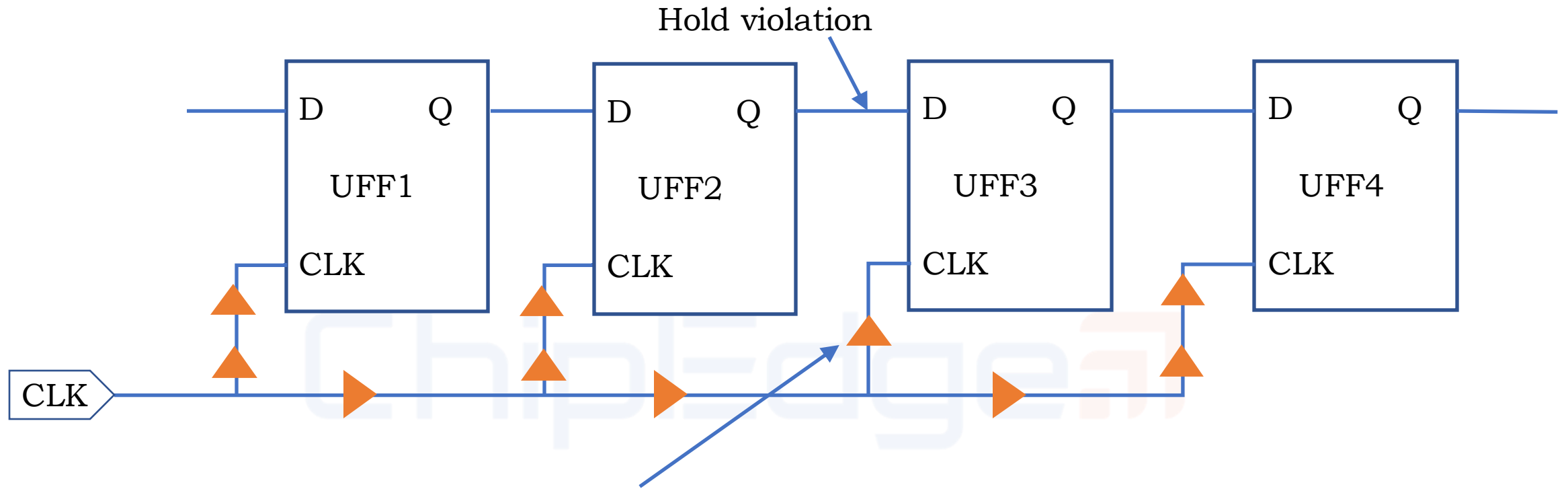


- Hold Violation can be fixed:
 - By increasing the Launch flop clock latency (Clock push)
 - By decreasing the Capture flop clock latency (Clock pull)





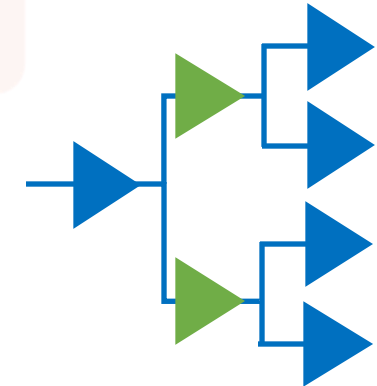
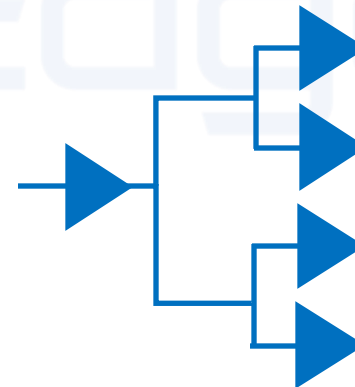
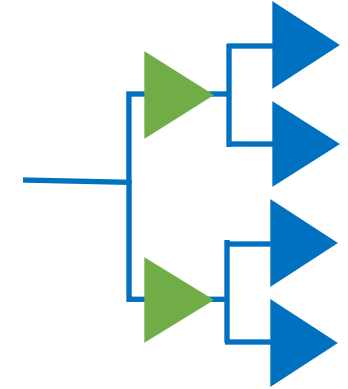
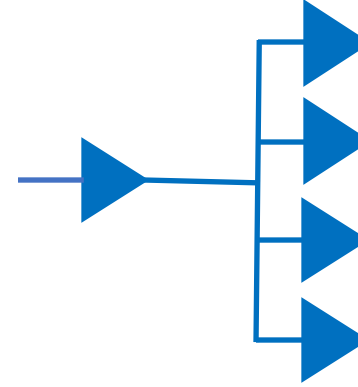
One buffer is added to increase the clock Latency



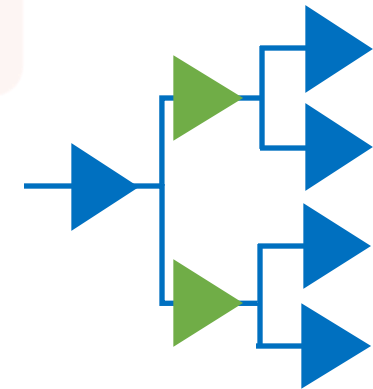
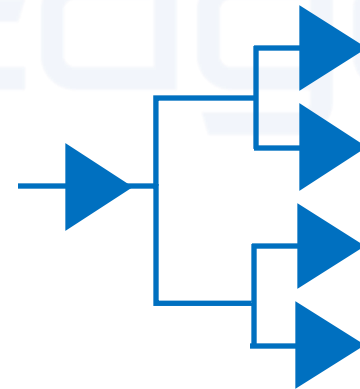
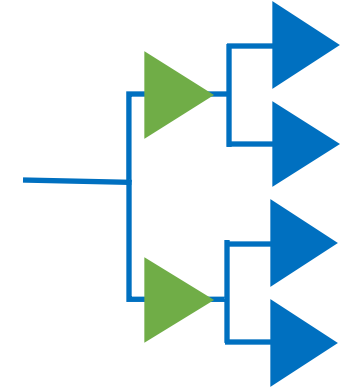
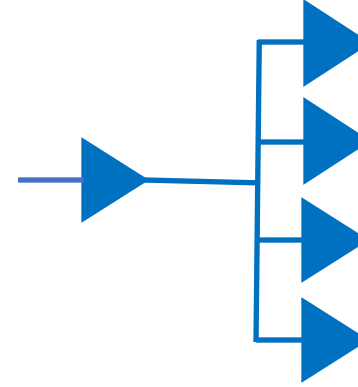
One buffer is removed to decrease the clock Latency

DRV Optimization

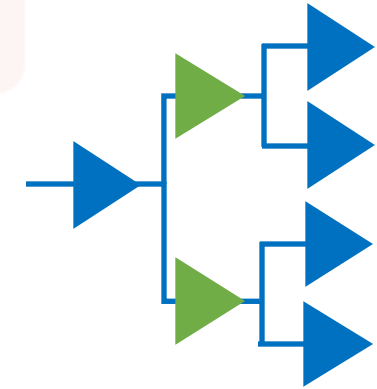
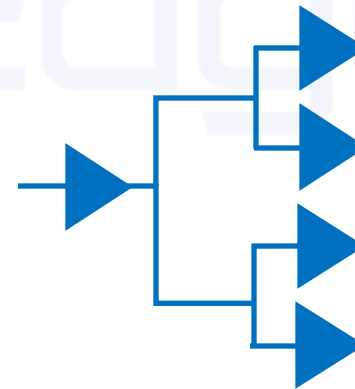
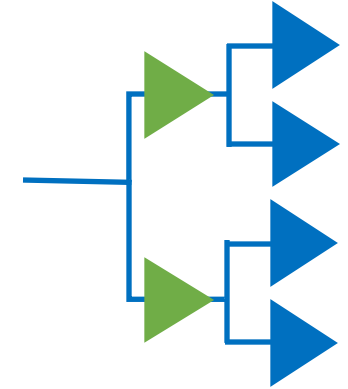
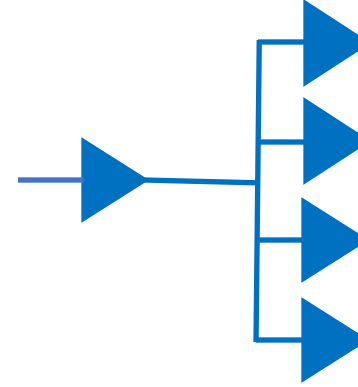
- Max Tran Violations
 - ✓ V_t swaps from high V_t to Low V_t
 - ✓ Upsizing of driving cell
 - ✓ Driver cloning
 - ✓ Fanout splitting
 - ✓ Buffering of nets



- Max Cap Violations
 - ✓ Driver cloning
 - ✓ Fanout splitting
 - ✓ Buffering of nets



- Max Fanout Violations
 - ✓ Driver cloning
 - ✓ Fanout splitting



- High fanout will lead to high Max Cap and Max Tran violations

Introduction to Optimization 

Timing Optimization 

Power Optimization

This section gives the detailed explanation of Leakage Recovery and Dynamic power optimization.

Area Optimization 

- Leakage Recovery
 - ✓ Swapping low V_t cells with high V_t cells
 - ✓ Swapping small channel cells with long channel cells
 - ✓ Paths with enough setup margin
- Tool can perform leakage opt at every stage if power opt options are specified
- Low Power design techniques like power gating can be used to reduce leakage power

- Dynamic Power Optimization
 - Clock gating
 - Multi voltage techniques
 - Power recovery options
 - ✓ Down-sizing of cells in paths with enough setup margin
 - ✓ Removing buffers from paths with positive hold slack

- Tool performs the following for power optimization at various stages
 - ✓ Performs power aware placement during placement stage
 - ✓ Tool identifies nets with high switching activity and tries to shorten their length
 - ✓ Restructures clock gates during clock tree synthesis
- Tool can achieve better power optimization if switching activity is annotated to the design
 - ✓ By reading SAIF file
 - ✓ By setting switching activity

Introduction to Optimization 

Timing Optimization 

Power Optimization 

Area Optimization

This section gives the detailed explanation of Area Recovery

- Area recovery
 - ✓ To create space to ease out congestion
 - ✓ To create space so that cells can be added for timing fixes
- Tool options can be used at every stage so tool can
 - ✓ Remove cells from non-critical paths
 - ✓ Area recovery will also help to reduce power.

- Timing Optimization
- DRV Optimization
- Setup Optimization
- Hold Optimization
- Power Optimization
- Leakage Optimization
- Dynamic Optimization
- Area Optimization

Module 13

Process Variations

- In this module you will learn process variation overview.
- The detailed explanation of Global Variation which includes PVT Variation, RC Corners, Operating Conditions and MCMM Timing Analysis are addressed.
- In addition to these On Chip Variation (OCV), Advanced On Chip Variation (AOCV) and Parametric On Chip Variation (POCV) are explained in detail.

- Introduction to Process Variation
- Global Variation
 - ✓ PVT Corners
 - ✓ MCMM
- Local Variation
 - ✓ On Chip Variation
 - ✓ Advanced On Chip Variation
 - ✓ Parametric On Chip Variation

Introduction to Process Variation

This section gives the introduction to Process Variation

Global Variation



Local Variation



ChipEdge

- All transistors / std cells are not exactly the same during manufacturing
- The device parameters like width, length, thickness, doping concentration etc.. can vary from transistor to transistor, die to die, wafer to wafer and lot to lot.
- Global variation - Variations between different chips.
- Local variation – Variation within a chip (On Chip Variation).
- Timing needs to be closed taking into account all these variations.

- These variations are modelled for timing calculations.
 - Global Variation – Process corners (Slow, Fast, Typical).
 - Local Variation-Derating factors
 - ✓ OCV
 - ✓ AOCV / SBOCV
 - ✓ POCV

Introduction to Process Variation

Global Variation

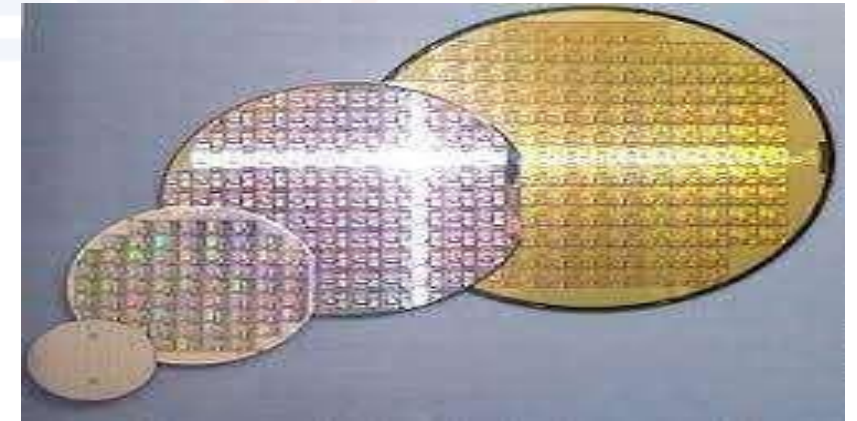
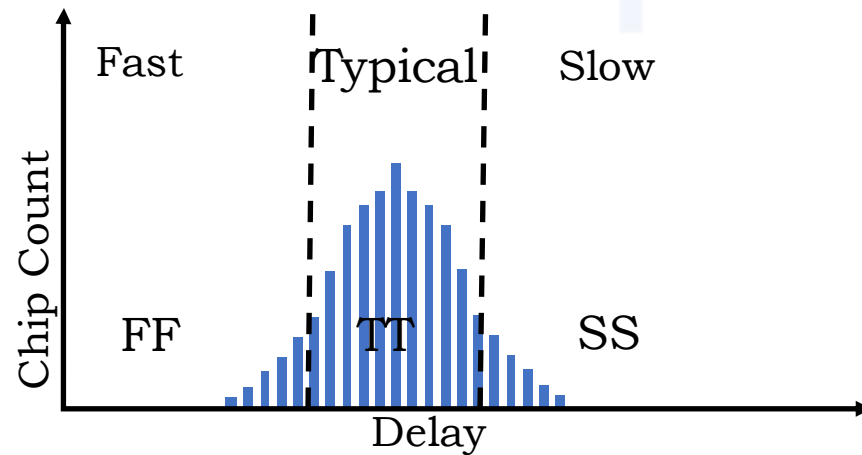
- ✓ PVT Corners
- ✓ MCMM

Local Variation

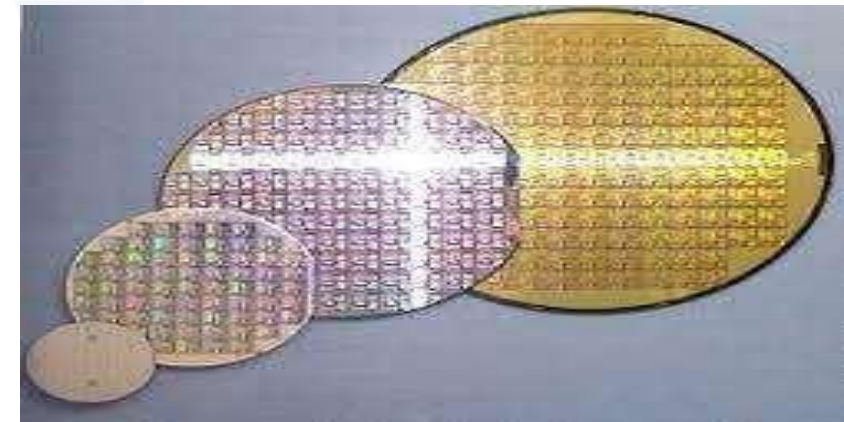
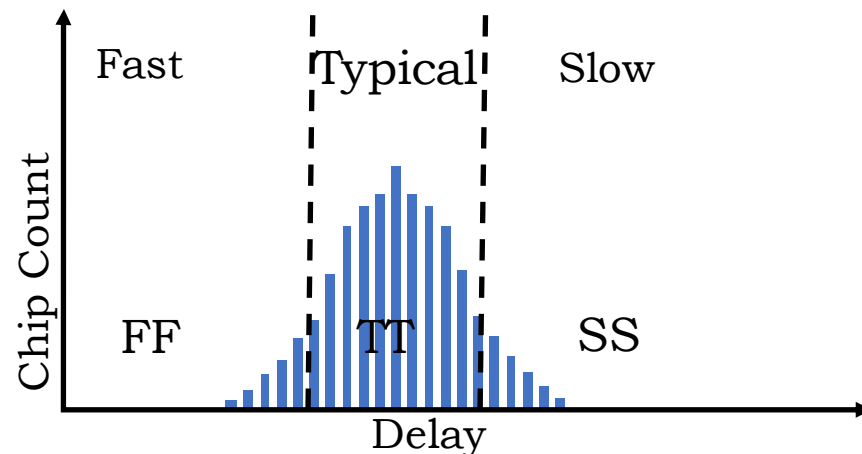
This section gives the detailed explanation of Global Variation which includes PVT Variation, RC Corners, Operating Conditions and MCMM Timing Analysis



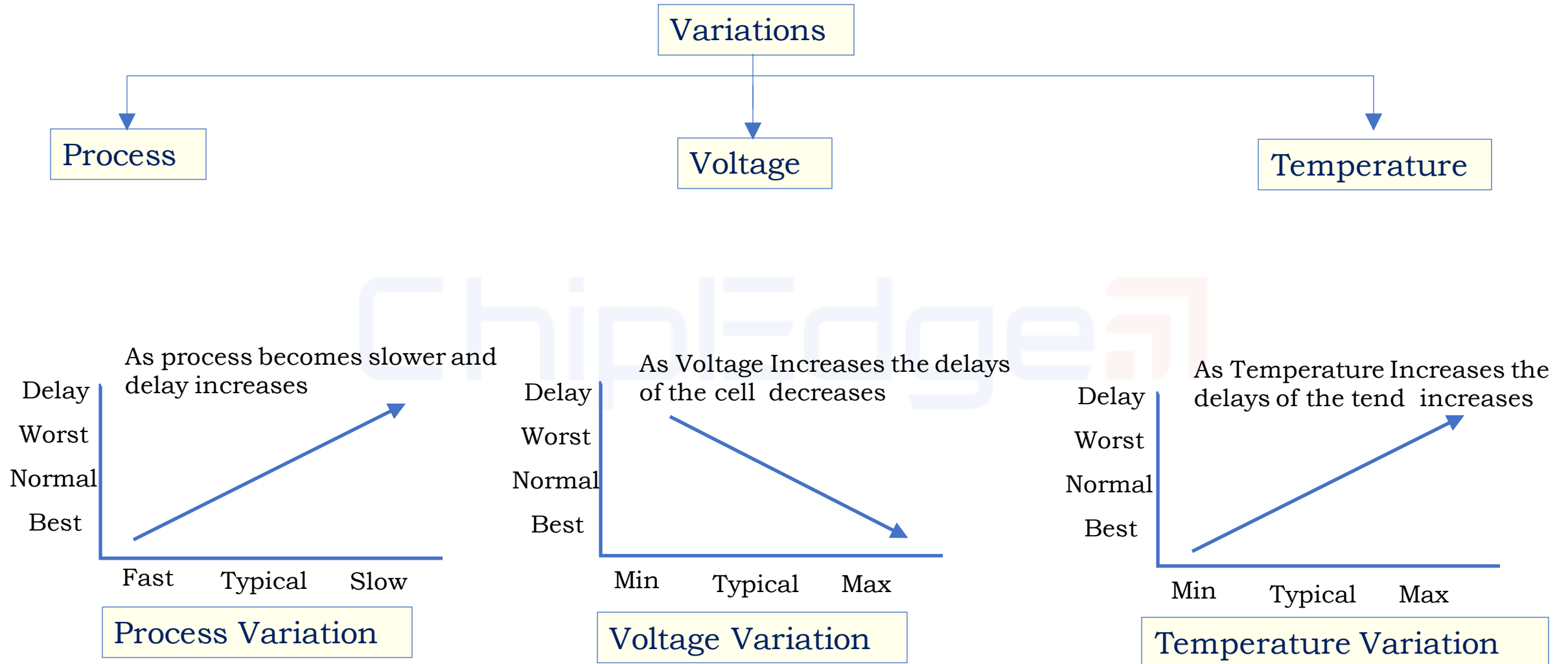
- Global Variation is quantified by timing libs.
- A particular wafer can be quantified by Foundry as belonging to SS or TT or FF corners and all dies inside that wafer will mostly be within that corner.



- A WAT (Wafer Acceptance test) is done by Foundry to determine the corner of the Wafer and then they bin the wafers in SS, TT, FF categories
- As a designer, it's our duty to make sure when we design the chip, we close timing in all the corners so that all DIES work on silicon whatever corner they end up.



- Apart from the Process Variations coming from manufacturing, Voltage and Temperature could also vary from chip to chip
- Voltage Variation could arise from supply voltage, voltage regulator, IR drop etc.
- Temperature Variation could be because of environmental temperature and power dissipation in the chip, or other heating factors on the platform.
- As a designer, it's our job to design such that the chips are working irrespective of these PVT Variations.
- This is taken care by creating .libs with multiple PVT and then the Timing analysis at multiple PVT corners.

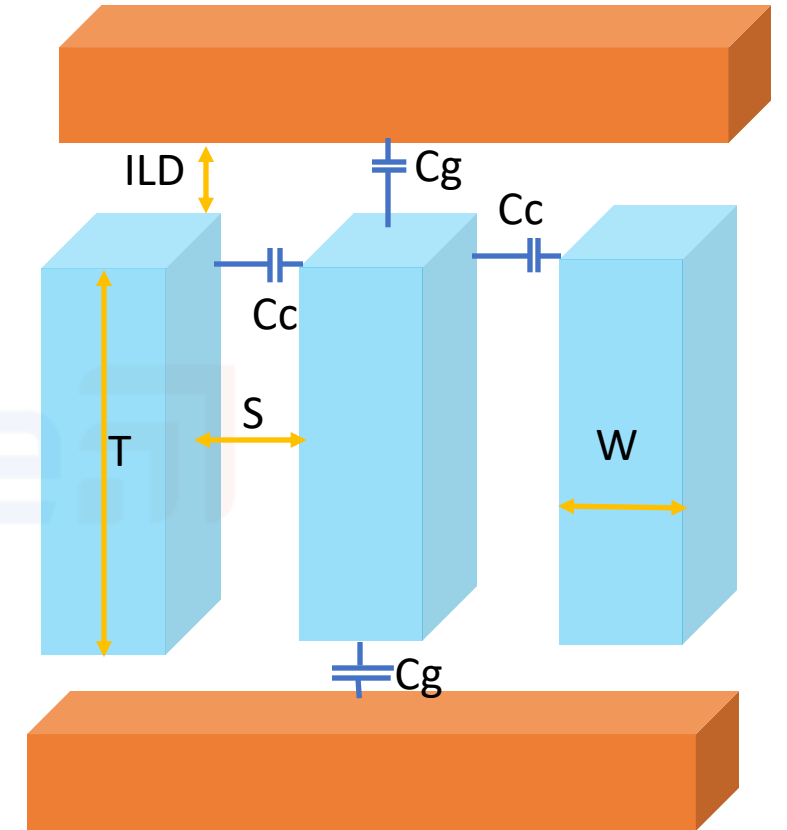
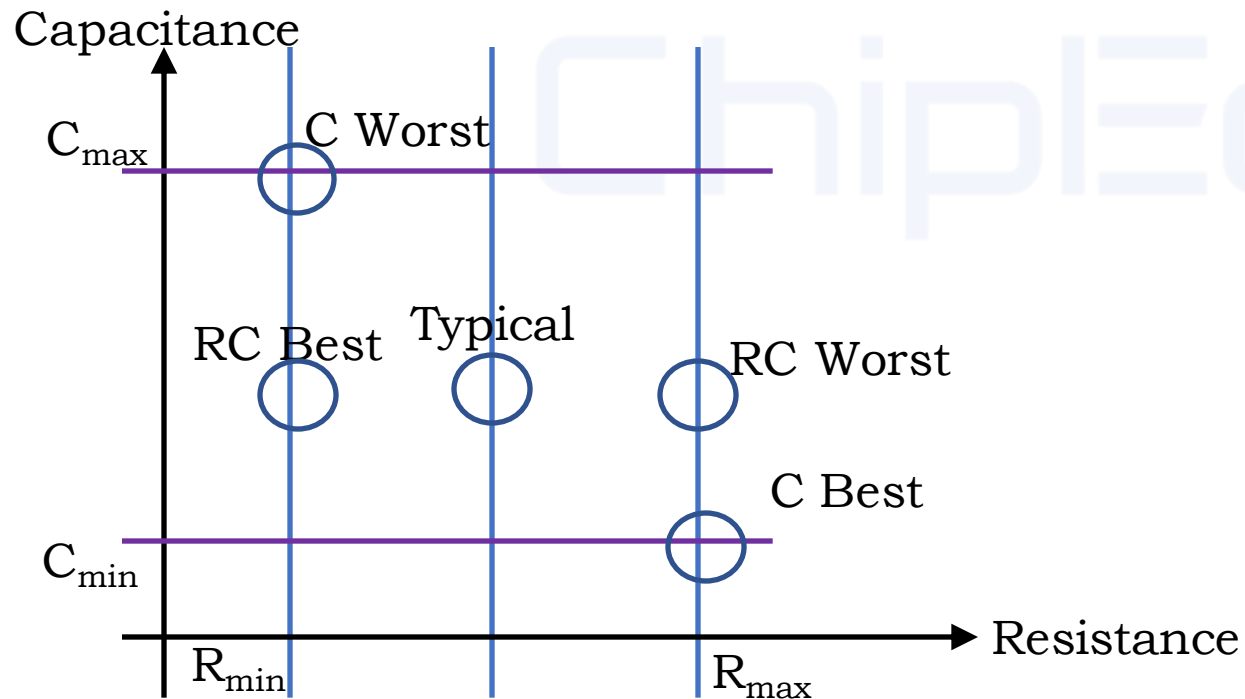


- A corner is defined as a set of libraries characterized for process, voltage, and temperature variations.
- Characterizes the physical environment for Timing Analysis.
- An extreme point in the PVT or RC space, where cell and net delays have extreme values.
- A particular one cell library and RC-model specified for STA run.

Item	Ranges
Process	SS, TT, FF
Voltage	Vnom-10%, Vnom, Vnom+10%
Temperature	-40C, 25C, 125C

- The variations in process parameters during manufacturing impact not only to devices but also to Routing (Metal layers).
- Processes such as Etching, Photo Lithography etc, will lead to variations in Metal shape parameters like width, thickness, Inter layer dielectric thickness(ILD) etc.
- These variations result into R and C variations of the metals which in turn lead to variation in net delays.
- These variations are modelled as different parasitic RC corners ($C_{\max}$, $C_{\min}$, $RC_{\max}$, $RC_{\min}$).

RC Corners	Width (W)	Thickness (T)	ILD thickness
$C_{\max}$	Max	Max	Min
$C_{\min}$	Min	Min	Max
$RC_{\max}$	Min	Min	Min
$RC_{\min}$	Max	Max	Max



Components (Std-cells, Memories, IPs, Macros, IOs)

Item	Ranges
Process	SS, TT, FF
Voltage	$V_{nom}-10\%$, V_{nom} , $V_{nom}+10\%$
Temperature	-40C, 25C, 125C

Wire/Interconnect

Item	Ranges
Process	RC_{max} , C_{max} , Typical, C_{min} , RC_{min}
Temperature	-40C, 25C, 125C

Chip Operating Conditions

Cell			Wire		Process Timing Corner	
P	V	T	P	T		
SS	Vdd _{min}	-40C	RC _{max}	-40C	SS_Vdd _{min_m40C_Cmax_m40C} SS_Vdd _{min_125C_Cmax_125C} SS_Vdd _{min_125C_RCmax_125C}	Primary Max-delay(setup corners)
TT	Vdd _{nom}	25C	C _{max}	25C		
FF	Vdd _{max}		Typ	25C	FF_Vdd _{max_m40C_Cmin_m40C} FF_Vdd _{max_m40C_RCmin_m40C} FF_Vdd _{max_125C_Cmin_125C} FF_Vdd _{max_125C_RCmin_125C}	Primary Min-delay(hold corners)
		125C	C _{min}	125C		
			RC _{min}			

- Most chip designs have multiple modes such as functional modes, test mode, sleep mode etc.
- A Mode is defined as an operational setting of the chip.
- A Mode is defined by a set of clocks, timing constraints.
- Mode is linked to a unique set of timing constraints.
- Mode data is found in sdc file.
- Mode can be associated with a set of corners to include only real combinations.
- As a designer we need to make sure that the chip is working fine in all the modes.
- So the Static Timing Analysis (STA) is done in multiple modes.

- MCMM denotes Multi-Corner Multi-Mode timing analysis.
- For MCMM timing analysis, Multiple scenarios are defined as a combination of modes and corners.
- To make sure chip works in multiple modes - multimode STA.
- To make sure chip works across PVT Variations - multicorner STA.
- Multi-corner multi-mode (MCMM) Timing analysis is to make sure the chip functions as expected in all modes irrespective of possible PVT Variations.

- Scenario=modes + **PVT Corner +RC Corner** ← Corner

- Func_SS_Vdd_{min}_m40C_C_{max}_m40C

- Test_SS_Vdd_{min}_m40C_C_{max}_m40C

- Func_FF_Vdd_{max}_m40C_C_{min}_m40C

- Test_FF_Vdd_{max}_m40C_C_{min}_m40C

SS_Vdd_{min}_m40C_C_{max}_m40C
SS_Vdd_{min}_125C_C_{max}_125C
SS_Vdd_{min}_125C_RC_{max}_125C

} Primary Max-
delay(setup
corners)

FF_Vdd_{max}_m40C_C_{min}_m40C
FF_Vdd_{max}_m40C_RC_{min}_m40C
FF_Vdd_{max}_125C_C_{min}_125C
FF_Vdd_{max}_125C_RC_{min}_125C

} Primary Min-
delay(hold
corners)

Introduction to Process Variation



Global Variation



Local Variation

✓ On Chip Variation



This section gives the introduction to Local Variation.

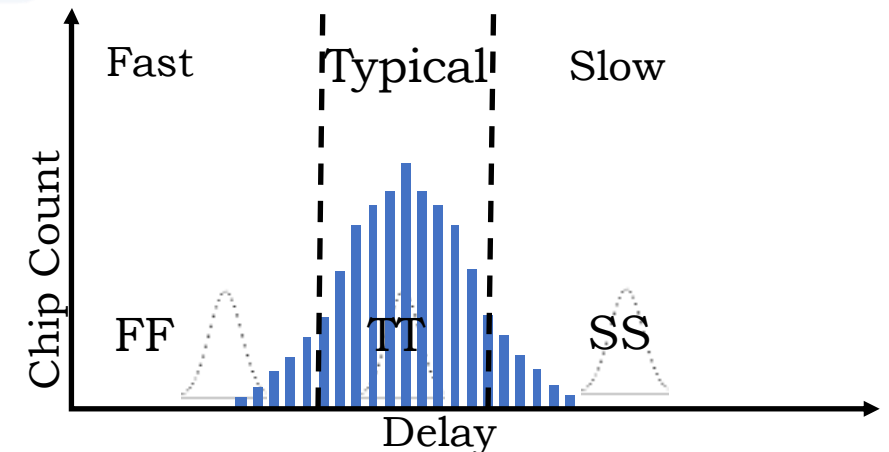
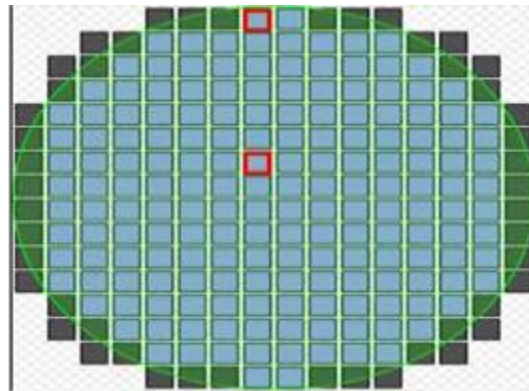
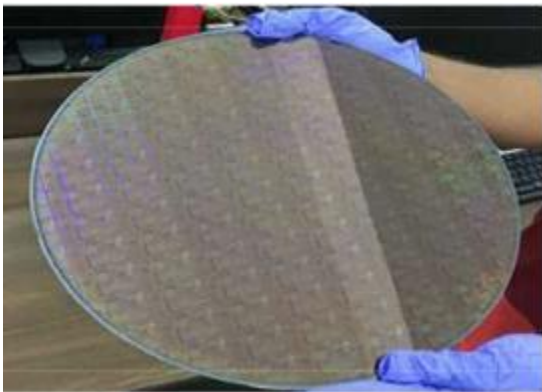
✓ Advanced On Chip Variation



✓ Parametric On Chip Variation



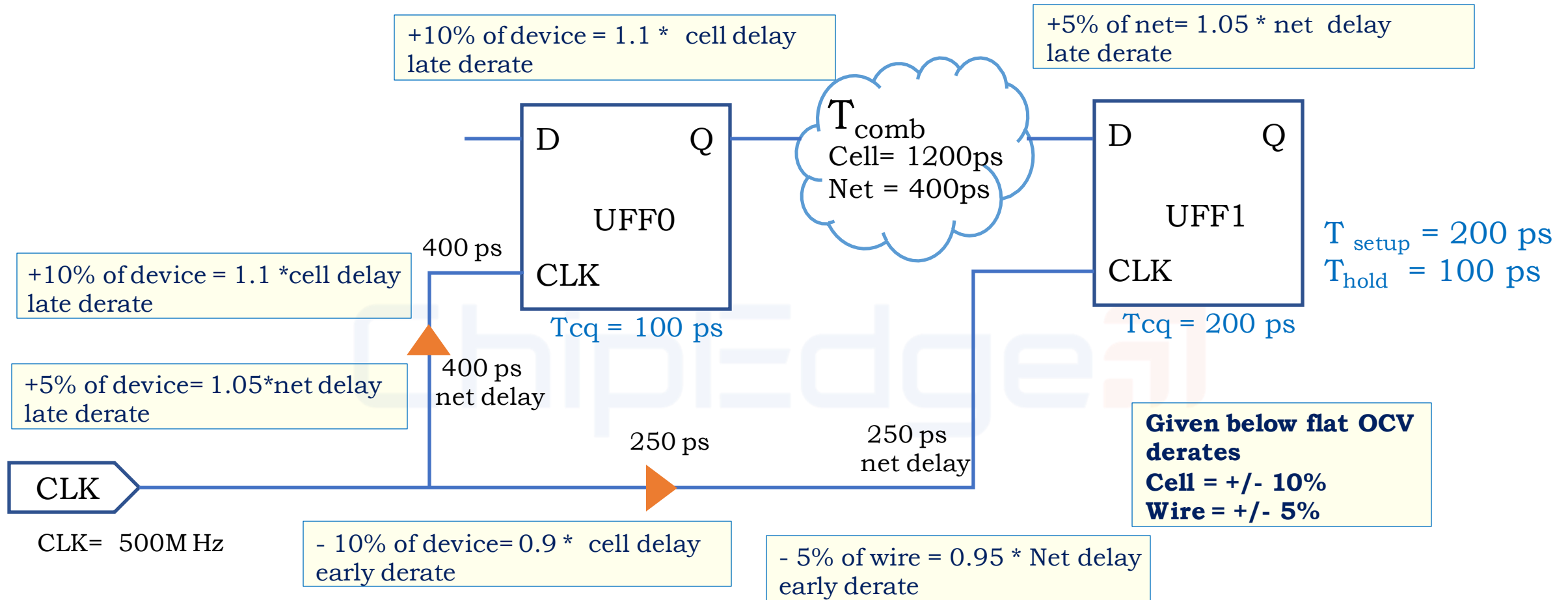
- Inside a wafer, there are hundreds of dies and there is a variation in each die.
- There are variations in the characteristics of transistors even inside a single IC along with the die and lots.
- There could be variations in P,V,T.
- These are local variations and are minor as compared to the global variations.



- On Chip Variation could lead to post-silicon failure if it is not taken care while designing the ASIC.
- To take care of OCV we need to add some pessimism in the timing of standard cells.
- We basically apply $\pm x\%$ of additional delay to all the standard cells. Which is called OCV derate.
- We generally get process only derates from Foundry. Voltage and Temperature derates are either provided separately by IP Vendor.

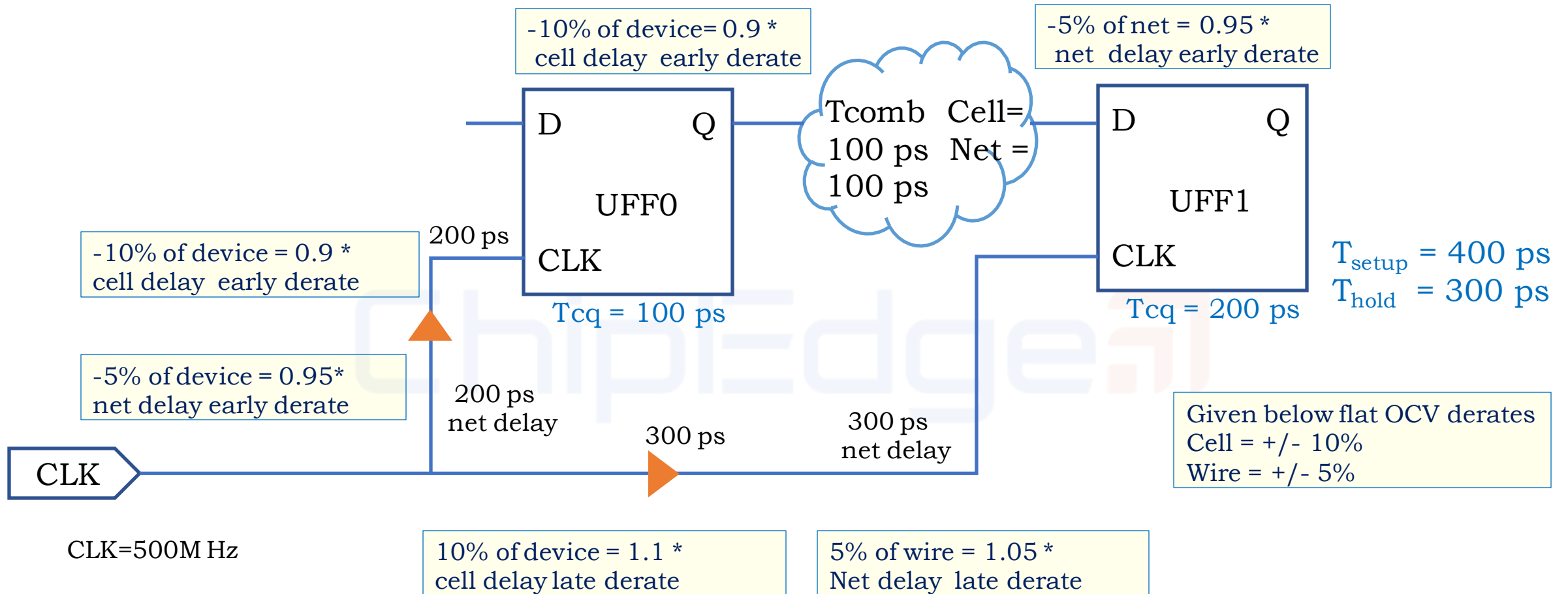
- User has to apply the derates into timing analysis in such a way that the worst-case impacts of OCV on Timing delays are taken care.
- Examples of setting OCV derates
 - ✓ `set_timing_derate -cell_delay -rise -data -early 0.92`
`set_timing_derate -cell_delay -rise -data -late 1.10`
 - ✓ `set_timing_derate -cell_delay -rise -data -early 0.92`
`set_timing_derate -cell_delay -rise -data -late 1.10`

OCV in Setup Timing Path



$$\begin{aligned}
 \text{Setup Slack} &= \text{Required time} - \text{Arrival time} \\
 &= [T_{\text{clk}} + T_{\text{skew}}] - [T_{\text{cq1}} + T_{\text{comb}} + T_{\text{setup2}}] \\
 &= ??
 \end{aligned}$$

OCV in Hold Timing Path



$$\begin{aligned}
 \text{Hold Slack} &= \text{Arrival time} - \text{Required time} \\
 &= [T_{cq1} + T_{comb}] - [T_{hold2} + T_{skew}] \\
 &= ??
 \end{aligned}$$

- The concepts of OCV fixed derate was modelled in technology node above 45nm. It was good for such higher technology node.
- But in lower technology nodes (<45nm) and especially high-frequency design, the timing closure became very difficult due to high pessimism of fixed derate.
- So AOCV is proposed which has
 - ✓ Stage based derates (based on path depth)
 - ✓ Distance based derates

- AOCV is also called SBOCV because they are stage based and the AOCV file has a table for each cell in the library. AOCV files are provided by the IP vendor.
- As number of stages increase, due to averaging effect, the derate numbers decrease and thus some pessimism is removed compared to flat OCV derates.

- Example of a SBOCV derate file

```
object_type : lib_cell
```

```
delay_type : cell
```

```
derate_type : late
```

```
object_spec : 10nmLib/BUF_X4
```

```
depth : 1 2 3 4 5 6 7 8 9 10
```

```
table: 1.183 1.145 1.122 1.109 1.0901 1.0801 1.0736
```

```
1.0650 1.0601 1.055
```


- To address few issues AOCV method has, POCV is proposed at sub 20nm.
- POCV derates can be provided via a POCV coefficient file or an LVF file(liberty variation format which has the POCV variations embedded into the timing library file).

POCV
Each arc has variation models
Variation dependent on input tran and output load
Output transition variations also modeled
Inaccuracy related to half-cycle paths and Min pulse width addressed by correctly calculating CPPR/CRPR
Reduces mis-correlation b/w GBA and PBA approaches

```
ocv_sigma_cell_rise ("pocv_template_4x4")
{ sigma_type : "late";
index_1 ("0.01, 0.04, 0.12, 0.80");
index_2 ("0.01, 0.02, 0.03, 0.10");
values ("σ11, σ12, σ13, σ14", \ "σ21, σ22, σ23, σ24", \ "σ31, σ32, σ33
, σ34", \ "σ41, σ42, σ43, σ44", ); }
```

Note: OCV/AOCV – delay numbers multiplication with variation value

POCV – dependent for each cell $\text{input_trans} / \text{output_cap}$ (Delay=Nominal Delay + 3σ)

- Process Variation
- Global Variation
- PVT Variation
- Modes and Corners
- Local Variation
- On Chip Variation
- Advanced On Chip Variation
- Parametric On Chip Variation

Module 14

Signoff-1

- In this module, you will learn about Extraction, DRV Fixes, Timing Fixes, Engineering Change Order (ECO), and PrimeTime commands .
- In addition to these, Logical Equivalence check(LEC) is discussed.



- Introduction to Signoff
- Extraction
 - ✓ DPT Impact
 - ✓ SPEF
- Timing Analysis
- Timing Violations & Fixes
- PrimeTime commands
- ECO
- LEC

Introduction to Signoff

This section explains different sign off checks to be run on the Post route database before taping out.

Extraction



Timing Analysis



Timing Violations & Fixes



PrimeTime commands

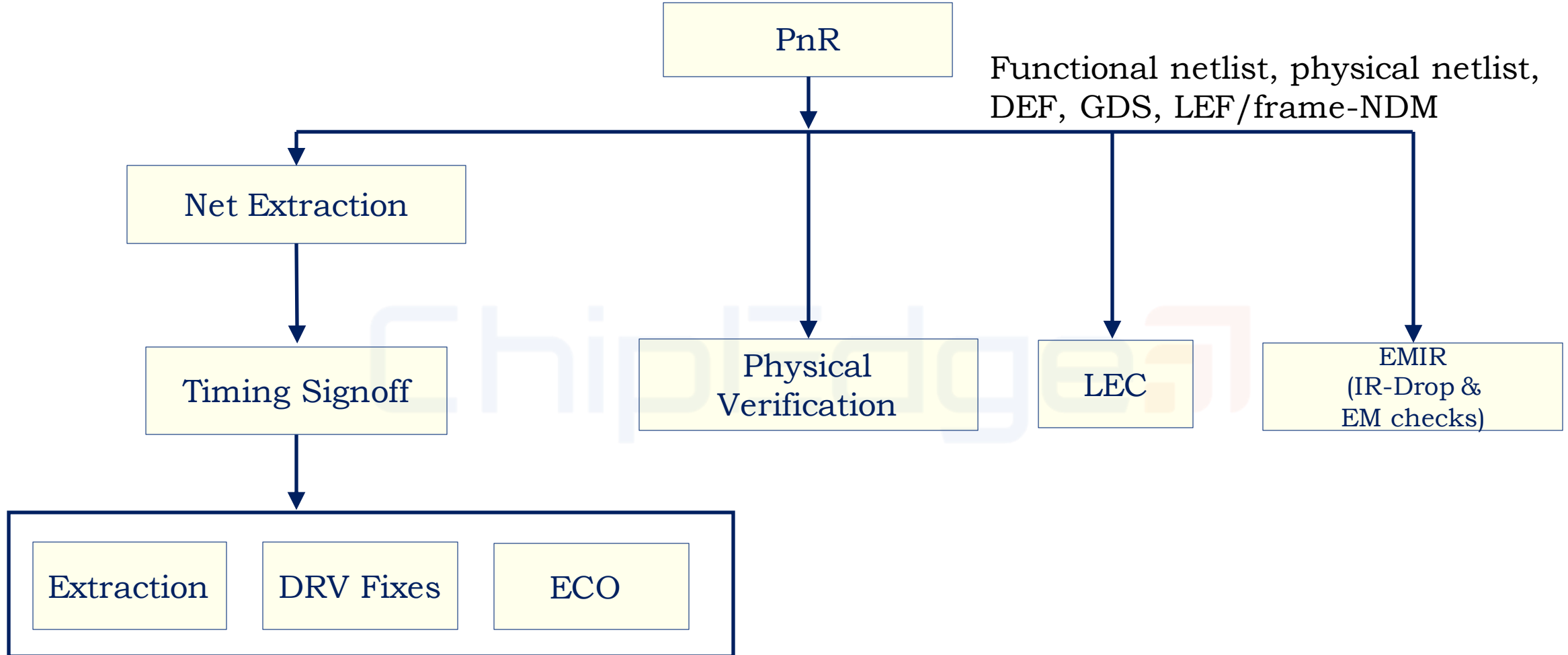


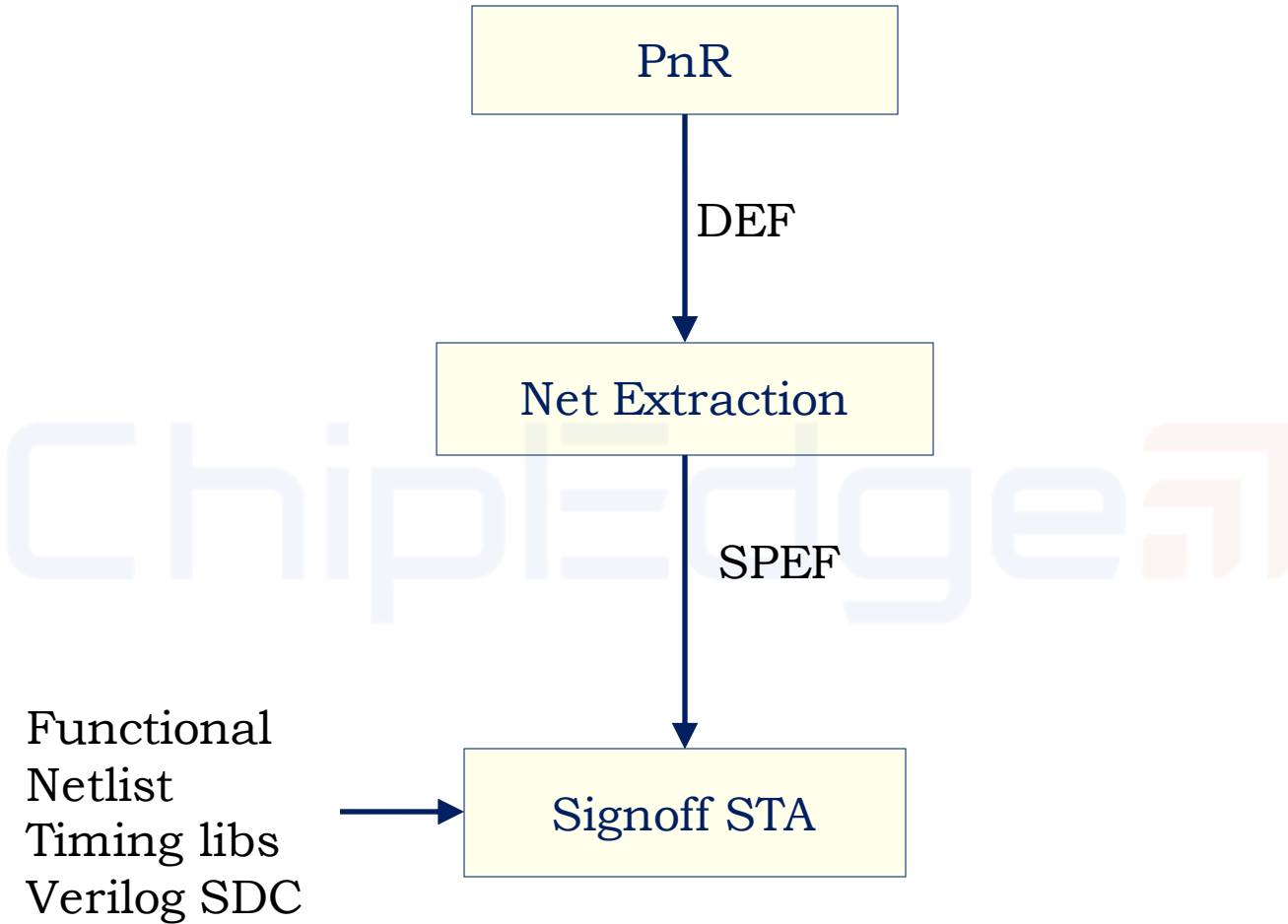
ECO



LEC







Introduction to Signoff



Extraction

This section explains Extraction flow, Extraction in DPT layers and SPEF File.

Timing Analysis



Timing Violations & Fixes



Primetime commands

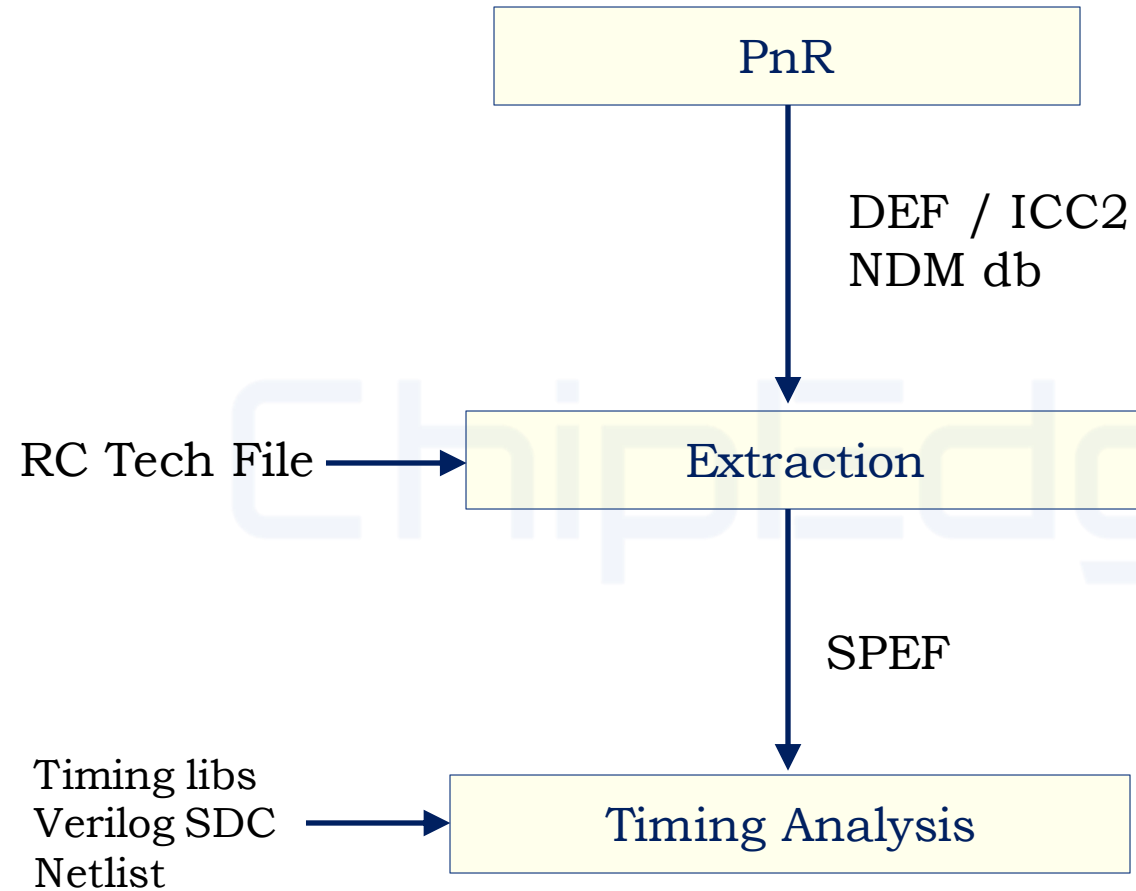


ECO

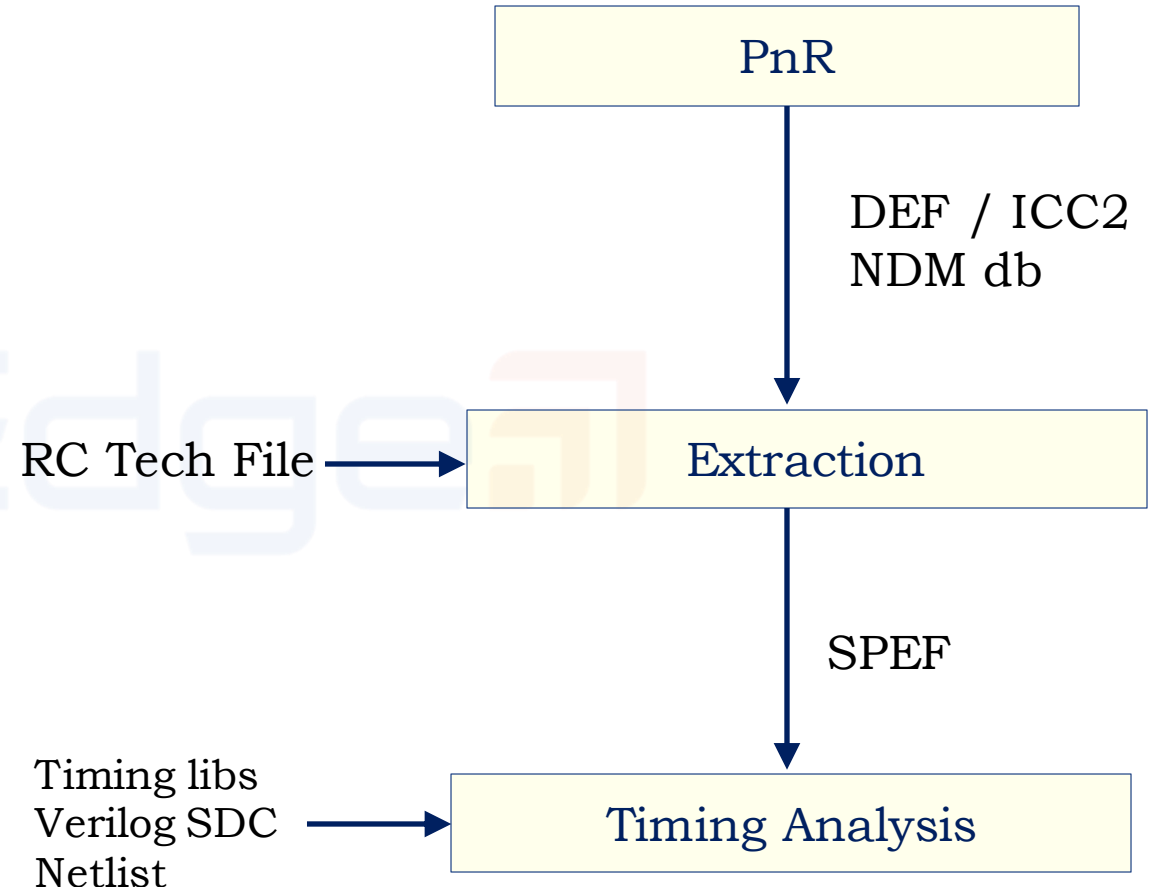


LEC



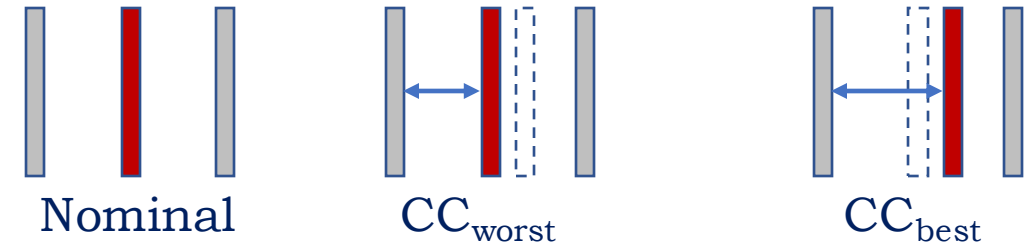


- Extract wire R and C values.
- SPEF is the output from Extraction.
- SPEF Contains R and C information of all the nets in the design.
- SPEF is the output of Extraction and input to Timing analysis.
- STA tool uses SPEF file to calculate net delays.

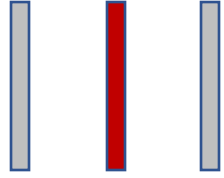
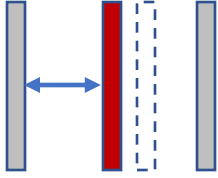
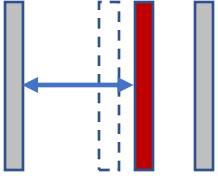


- Extraction of nets only (metal lines).
- Std Cell delays already available (.lib).
- Extraction is generally done for 4 RC corners .
 - ✓ C_{best}
 - ✓ C_{worst}
 - ✓ RC_{best}
 - ✓ RC_{worst}

- With lower nodes, metal width reduces, and thickness of metal lines increases – so coupling Cap becomes a critical part of extraction.
- Mask mis-alignment error can cause increase or decrease in Coupling Cap can result in corners like
 - ✓ CC_{worst}
 - ✓ CC_{best}



- ✓ $C_{\text{worst-CC}_{\text{worst}}}$
- ✓ $RC_{\text{worst-CC}_{\text{worst}}}$
- ✓ $C_{\text{best-CC}_{\text{best}}}$
- ✓ $RC_{\text{best-CC}_{\text{best}}}$

Metal Profiles	 Nominal	 CC_{worst}	 CC_{best}
1	C_{worst}	$C_{\text{worst-CC}_{\text{worst}}}$	
2	C_{best}		$C_{\text{best-CC}_{\text{best}}}$
3	RC_{worst}	$RC_{\text{worst-CC}_{\text{worst}}}$	
4	RC_{best}		$RC_{\text{best-CC}_{\text{best}}}$
5	C_{typical}	$C_{\text{typical-CC}_{\text{worst}}}$	$C_{\text{typical-CC}_{\text{best}}}$

- Standard Parasitic Exchange Format.
- SPEF is Text file, which can be opened and checked for different RC values for different nets in the design.

```
*SPEF "IEEE 1481-1999"
*DESIGN "mux4"
*DATE "Mon Aug 16 14:50:46 2010"
*VENDOR Simucad Design Automation, Inc.
*PROGRAM "HIPEX"
*VERSION "hipex 3.2.15.R 64-bit (Wed Jan 21 15:23:41 PS 2009
*DESIGN_FLOW "MISSING NETS"
*DIVIDER /
*DELIMITER:
*BUSHIT <>
*T_UNIT 1 NS
*C_UNIT 1 PF
*R UNIT 1 Ohm
*L UNIT 1 HENRY

*POWER_NETS VDD
*GROUND_NETS VSS
*D_NET A<3> 0.358485
*CONN
*I IO:A B

*CAP
1 IO:A 0.104173
2 A<3>:15 0.0300278
3 A<3>:16 0.101697
4 A<3>:17 0.0300278
5 A<3>:10 0.0625319
6 A<3>:19 0.0300278
```


Introduction to Signoff



Extraction



Timing Analysis

This section explains the Timing Analysis Flow.

Timing Violations & Fixes



PrimeTime commands

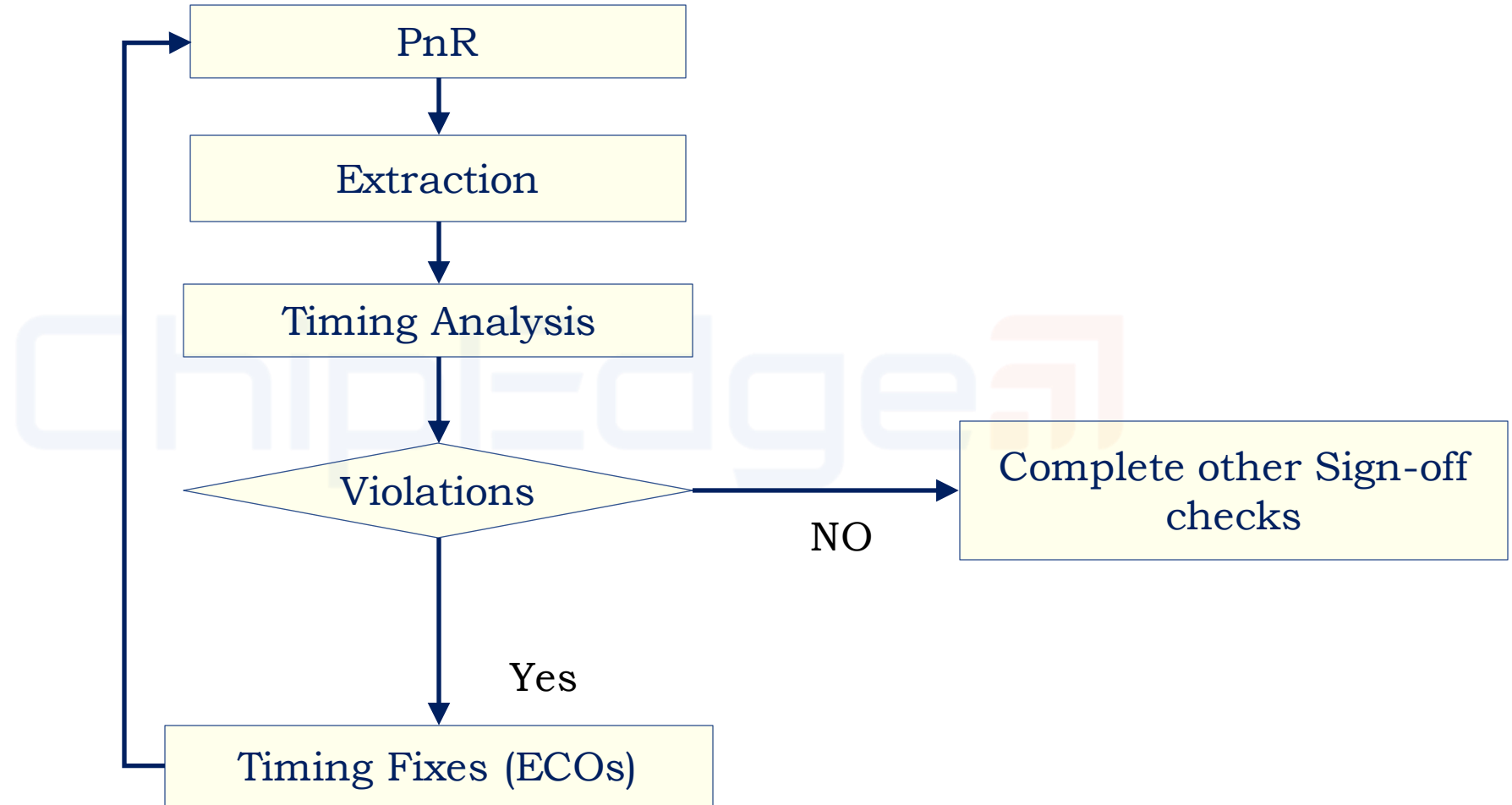


ECO



LEC





Introduction to Signoff



Extraction



Timing Analysis



Timing Violations & Fixes

This section explains the Timing Analysis Flow and fixes.

Primetime commands



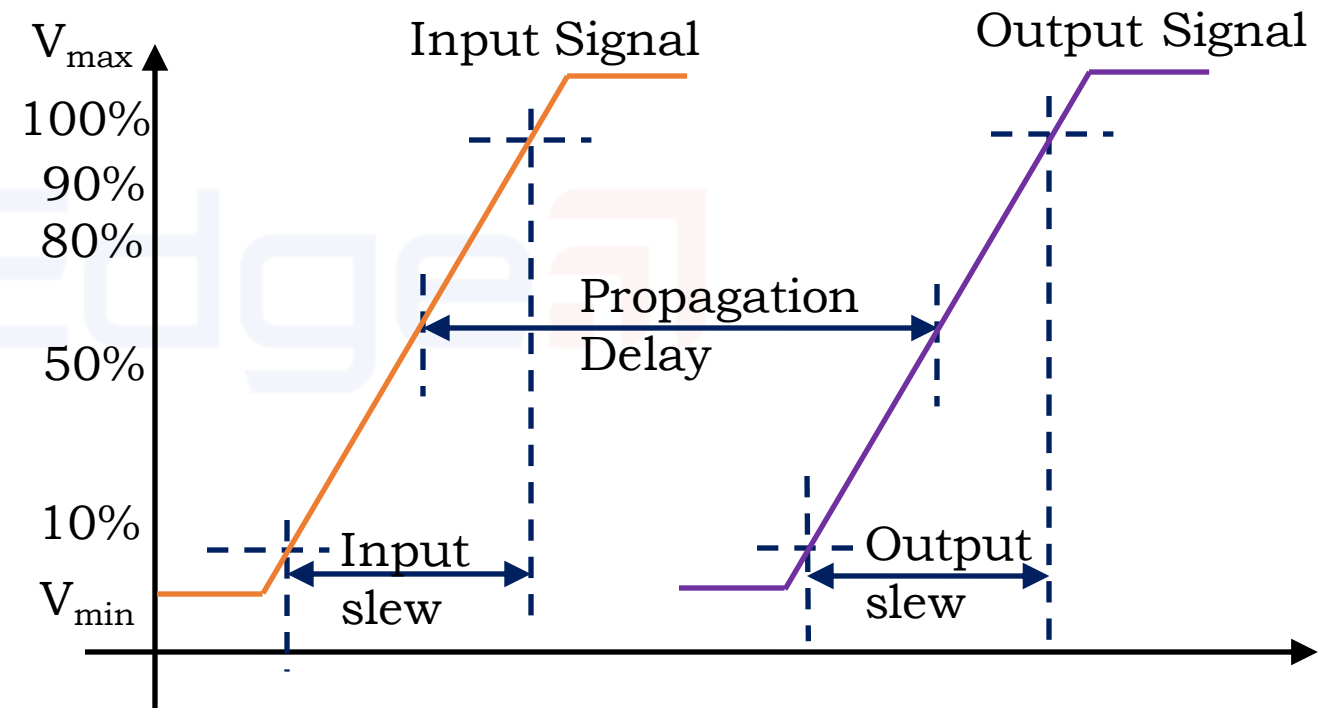
ECO



LEC

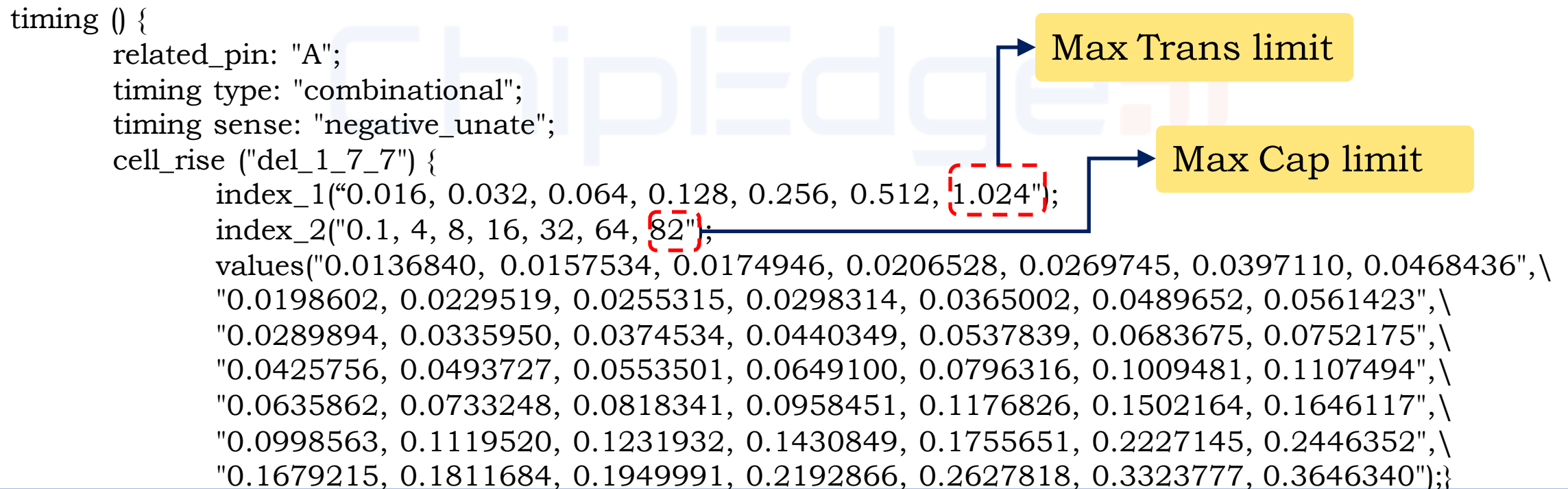


- Max_Transition Violation: If the transition time on any pin is more than maximum transition limit, it is reported as Max_Transition violation.
- Max_Capacitance Violation: The capacitance load on any pin is more than maximum capacitance target it is reported as Max_Capacitance violation



- Max trans and max cap limit is defined in SDC
- If not defined in SDC, the characterization limits in .lib is taken as max trans and max cap limits.

```
timing () {  
  related_pin: "A";  
  timing type: "combinational";  
  timing sense: "negative_unate";  
  cell_rise ("del_1_7_7") {  
    index_1("0.016, 0.032, 0.064, 0.128, 0.256, 0.512, 1.024");  
    index_2("0.1, 4, 8, 16, 32, 64, 82");  
    values("0.0136840, 0.0157534, 0.0174946, 0.0206528, 0.0269745, 0.0397110, 0.0468436",\  
          "0.0198602, 0.0229519, 0.0255315, 0.0298314, 0.0365002, 0.0489652, 0.0561423",\  
          "0.0289894, 0.0335950, 0.0374534, 0.0440349, 0.0537839, 0.0683675, 0.0752175",\  
          "0.0425756, 0.0493727, 0.0553501, 0.0649100, 0.0796316, 0.1009481, 0.1107494",\  
          "0.0635862, 0.0733248, 0.0818341, 0.0958451, 0.1176826, 0.1502164, 0.1646117",\  
          "0.0998563, 0.1119520, 0.1231932, 0.1430849, 0.1755651, 0.2227145, 0.2446352",\  
          "0.1679215, 0.1811684, 0.1949991, 0.2192866, 0.2627818, 0.3323777, 0.3646340");  
  }
```



- A DRC Violation can translate into a huge delay on data path or clock path.
- This huge delay can impact setup/hold depending on the data path or clock path where this delay exits.
- Fixing the DRC violations will be the first step to improve the timing.
- Get all cell delays to normal values for proper PrimeTime Timing analysis.
- After DRC fix, Setup and Hold timing violations are fixed.

- DRV fixes (Max trans / Max Cap)
 - ✓ Upsizing the driver, downsizing the sink, buffer insertion , fanout split.
- Setup and Hold fixes
 - ✓ V_T swap, changing Drive Strength, Buffer insertion.
 - ✓ Clock manipulation.

Introduction to Signoff



Extraction



Timing Analysis



Timing Violations & Fixes



PrimeTime commands

This section explains various PrimeTime commands for DRC fixing

ECO



LEC



- `fix_eco_drc` command in PT can be used
 - ✓ Fixing violation on a specific cell instance or all cell instances .
 - ✓ The `option-type` is mandatory which specifies `max_capacitance/max_transition/max_fanout/noise`.
 - ✓ Fixes DRC even if timing is violated.
- Setup/Hold timing fixes
- `fix_eco_timing` command in PT can be used
 - ✓ Fixing violation on a specific cell instance or all cell instances.
 - ✓ The option `-setup` or `-hold` specifies the timing violation to work on.
 - ✓ Fixes hold only if the setup timing and DRCs are not violated.

Introduction to Signoff



Extraction



Timing Analysis



Timing Violations & Fixes



PrimeTime commands



ECO

This section explains about ECO, Timing ECO file generation and ECO File Manual Creation.

LEC



`write_changes` command in PT can be used to

- Dump the ECO file...
- This ECO file contains all the DRC and Timing fixes done by the Tool saved in the form of Tcl commands.
- The generated ECO file is sourced in PnR post route database to physically implement the timing fix ECO.

Engineering Change Order

- Any change to the structure of a digital netlist.
- Function Induced Change (Functional ECO).
 - ✓ Functional ECOs are generally used to fix any bugs detected or addition of a new feature in the logic.
 - ✓ Cell logic / Net connectivity changed.

- Timing or DRV induced Change (Timing ECO)
 - ✓ Timing and DRC fixes induce changes to the netlist.
 - ✓ Incremental PnR changes may be ideal for changing timing convergence.
 - ✓ Incremental changes to PnR are driven by STA ECOs.
 - ✓ Functionality is unaffected in Timing ECOs.
- Other types of ECOs could be to improve Power or Area.

- Timing fixes take few iterations.
- Last few fixes typically need manual fixes, as we don't want tool to disturb the db too much.

- For manual fixes, these are the typical commands used in PT to create Timing ECO

- | | |
|------------------------------|-------------------------------|
| • <code>remove_cell</code> | • <code>connect_net</code> |
| • <code>Size_cell</code> | • <code>disconnect_net</code> |
| • <code>Create_cell</code> | • <code>rename_net</code> |
| • <code>remove_buffer</code> | • <code>remove_net</code> |
| • <code>create_net</code> | • <code>write_changes</code> |

Introduction to Signoff



Extraction



Timing Analysis



Timing Violations & Fixes



PrimeTime commands



ECO



LEC

This section explains about LEC check, Formal Verification and LEC Flow.

- LEC check ensures that the synthesis process or PnR Flow or manual design changes did not introduce any functional changes to the design.
- Main goal is to ensure final PnR netlist has same functionality as original RTL from where we started.
- LEC check uses a formal verification comparison engine to prove the equivalence of two given designs and presents any differences for follow-on detailed analysis.
- A Formal verification tool detects unexpected differences that might have been introduced into a design during PnR flow.
- Formality from Synopsys and Conformal from Cadence are the major tools to check LEC.

- LEC is used throughout the PnR design flow to make sure that the integrity of the design descriptions are still logically equivalent as they go through different representations.
- Inputs can be two netlists or two RTLs or one RTL and one netlist.
- One input is taken as golden, and another input is taken as implemented design.
- Implemented design is compared against the golden design/reference design and checked for functional equivalence.

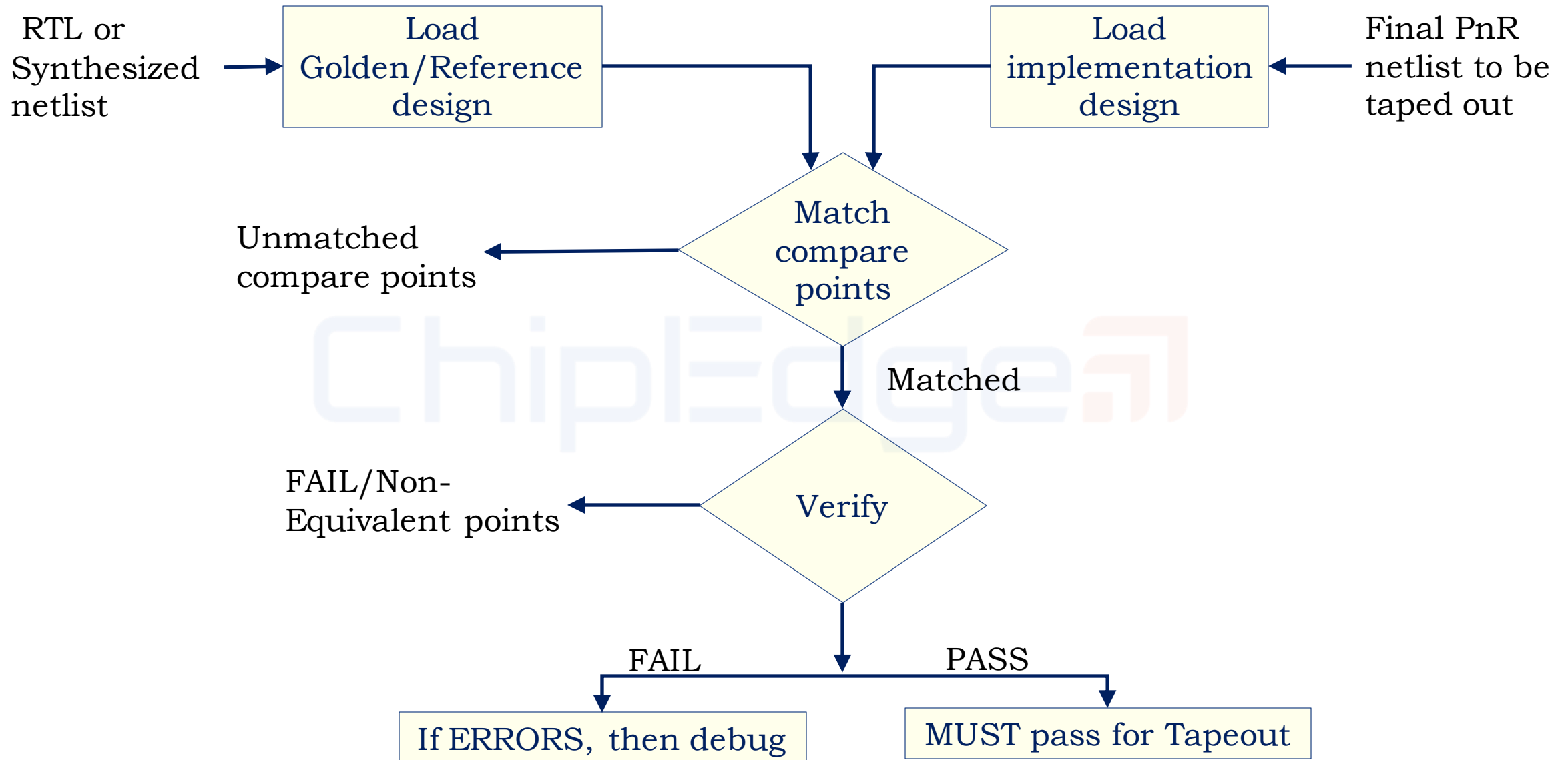
When to run LEC?

- After-synthesis
- After post-route
- After timing fixes
- After any design change ECOs
- On final tape-out database

- LEC is a 2-step process : Match and Verify.
- Match : Prior to design verification, Formality tries to match each primary output, sequential element, black box input pin, and qualified net in the implementation design with comparable design object in the reference design.
- If the Match step fails, it reports out the unmatched compare points.
- If the Match step is successful, proceeds to second step i.e verify.

Verify : Verification is the primary function of equivalence checking. By default, Formality checks for design verification consistency when you verify a design. The term design equivalence refers to the verification test objective.

LEC tool verifies a compare point by comparing the logic cone from a compare point in the implementation design against a logic cone for a matching compare point from the reference design.



- Introduction to Signoff
- Extraction
- DPT Impact
- SPEF
- Timing Analysis
- Timing Violations & Fixes
- PrimeTime commands
- ECO
- LEC

Module 14

Signoff-2

- In this module you will learn Signoff Checks Overview.
- Various physical only cells like Filler cells and Decap cells are discussed.
- The detailed explanation of PV Checks which include DRC Checks, Density, Antenna Checks, LVS, ERC Checks are discussed.
- In addition to these, IR Drop and ESD protection in IC are explained in detail.

- Introduction to Signoff
 - ✓ LVS Checks
- Filler Cells
 - ✓ ERC
- Decap Cells
 - ✓ DFM/DFY
- Dummy Fill
 - ✓ LPC/PM/MAS Checks
 - ✓ PERC Checks
- PV Checks
 - IR Drop / EM
 - ESD
 - ✓ DRC Checks
 - ✓ Antenna Checks

Introduction to Signoff

This section explains overview of signoff checks.

Filler Cells



Decap Cells



Dummy Fill



PV Checks



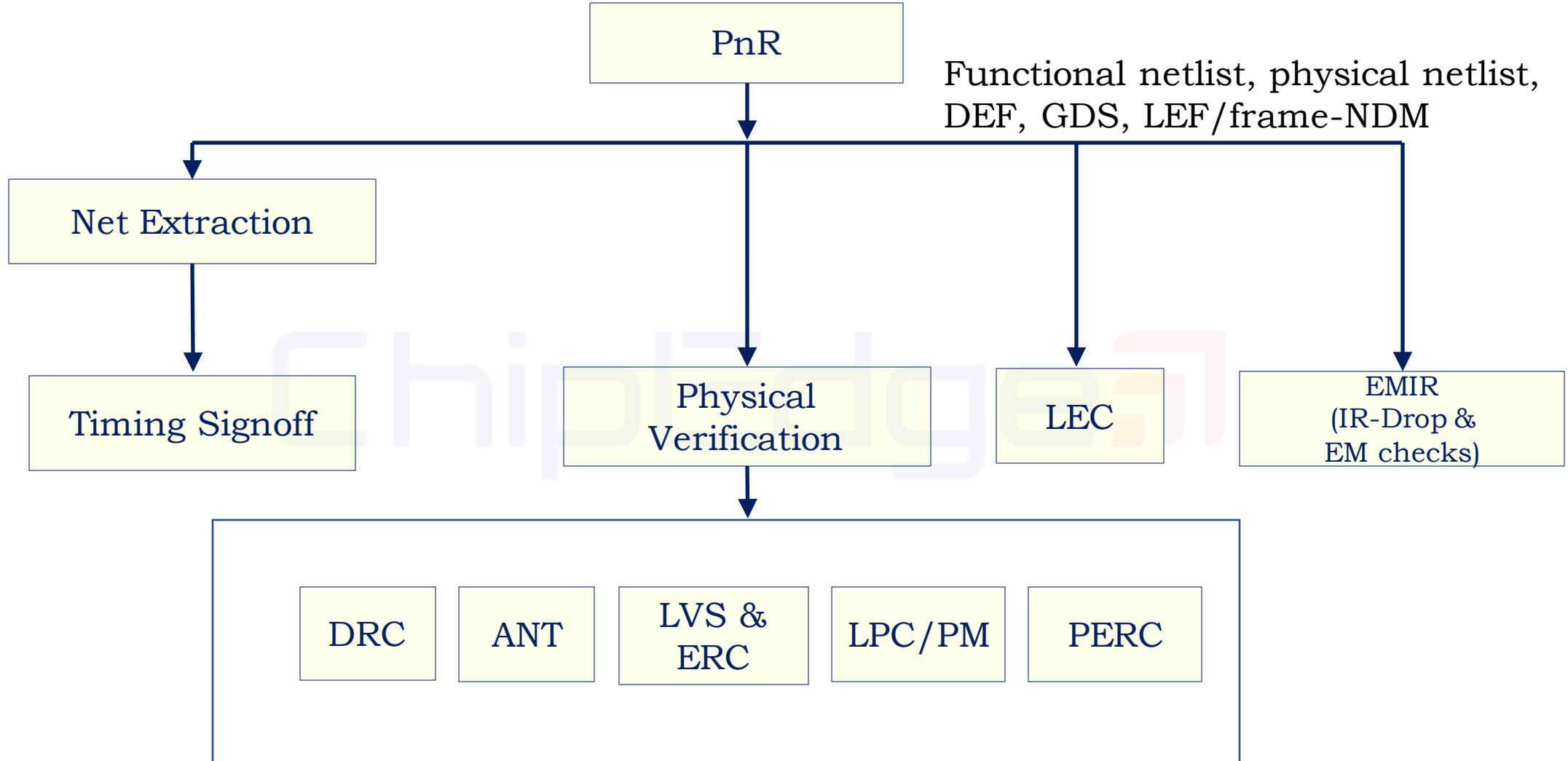
IR Drop / EM



ESD



ChipEdge



Introduction to Signoff



Filler Cells

This section explains about filler cells and Fill flow.

Decap Cells



Dummy Fill



PV Checks

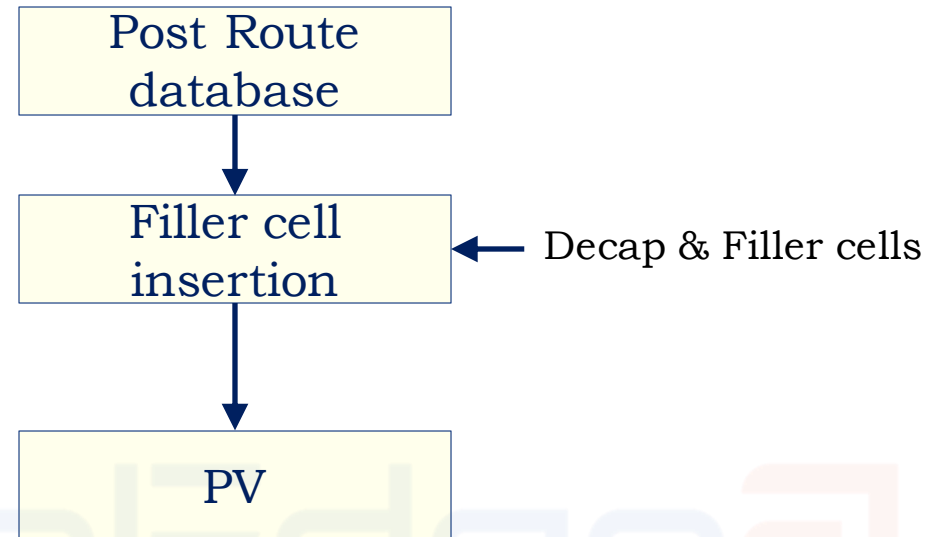


IR Drop / EM



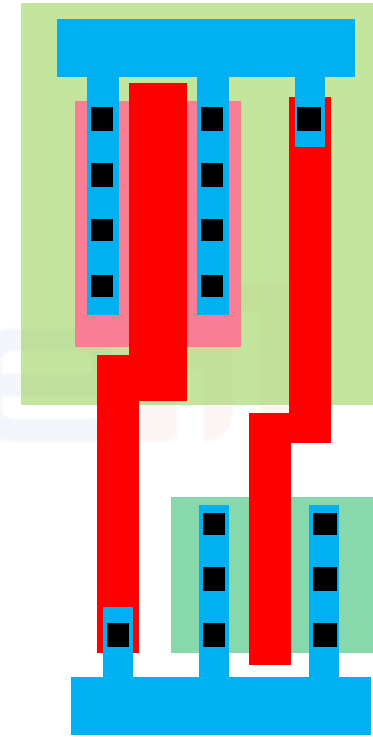
ESD





- In the post route database, there would be empty spaces left between std cells.
- All these empty spaces are filled by adding Filler cells or Decap cells.
- This helps in N-well continuity and meeting base layer density limit.

- Filler cells primarily are non-logical cells (they do not have any functionality) used to fill the gap between standard cells.
- Filler cells establish the N-well continuity in the standard cell rows.
- Reduce the DRC Violations created by the base (N-Well, P-Plus & N-Plus) layers.
- The use of Filler Cells is they reduce the DRC Violations created by the base (N-Well, P-Plus & N-Plus) layers and prevent latch up problem.



Introduction to Signoff



Filler Cells



Decap Cells

This section explains about the decap cells and benefits of decap cells

Dummy Fill



PV Checks



IR Drop / EM



ESD



- Decap cells are std cells which function as capacitors between V_{dd} And ground Rails.
- Decap cells are distributed in all the regions of the layout.
- Decap cells supply current locally to support the instant current requirements in the power delivery network.
- Benefits of Decap cells
 - ✓ Satisfies the benefits of Filler cells (N-well continuity + density requirements).
 - ✓ Helps in reducing Dynamic IR drop.

Introduction to Signoff



Filler Cells



Decap Cells



Dummy Fill

This section explains about Dummy fill flow and extraction with dummy metal fill flow

PV Checks

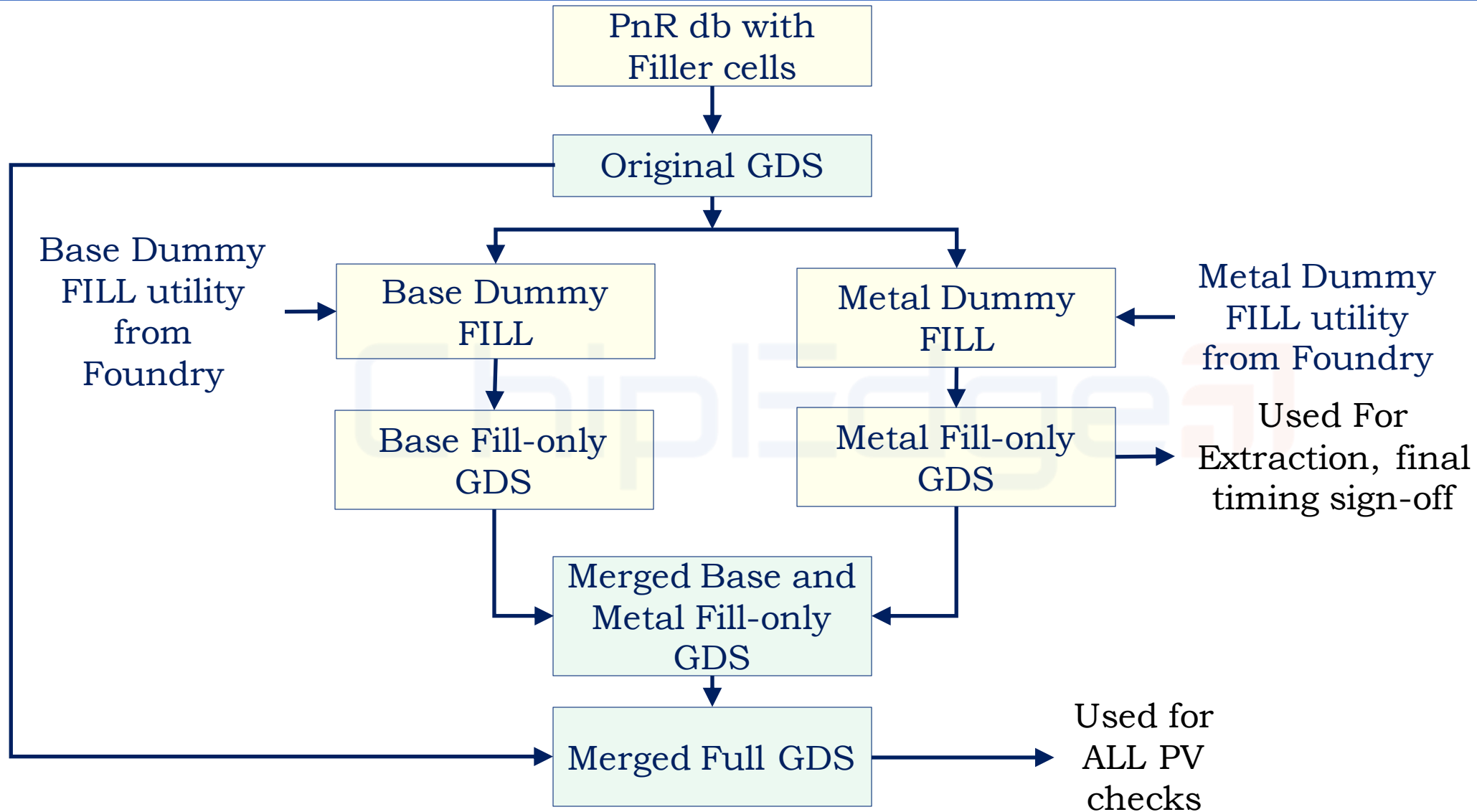


IR Drop / EM



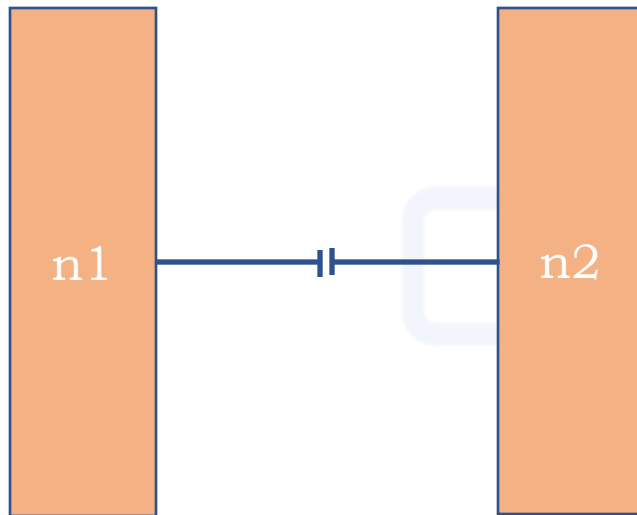
ESD



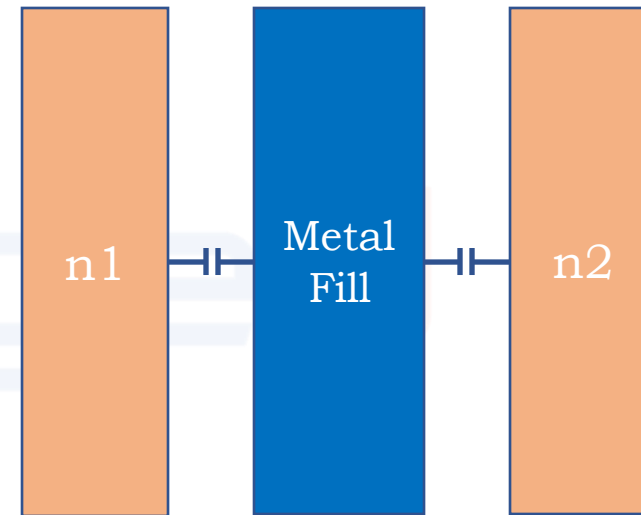


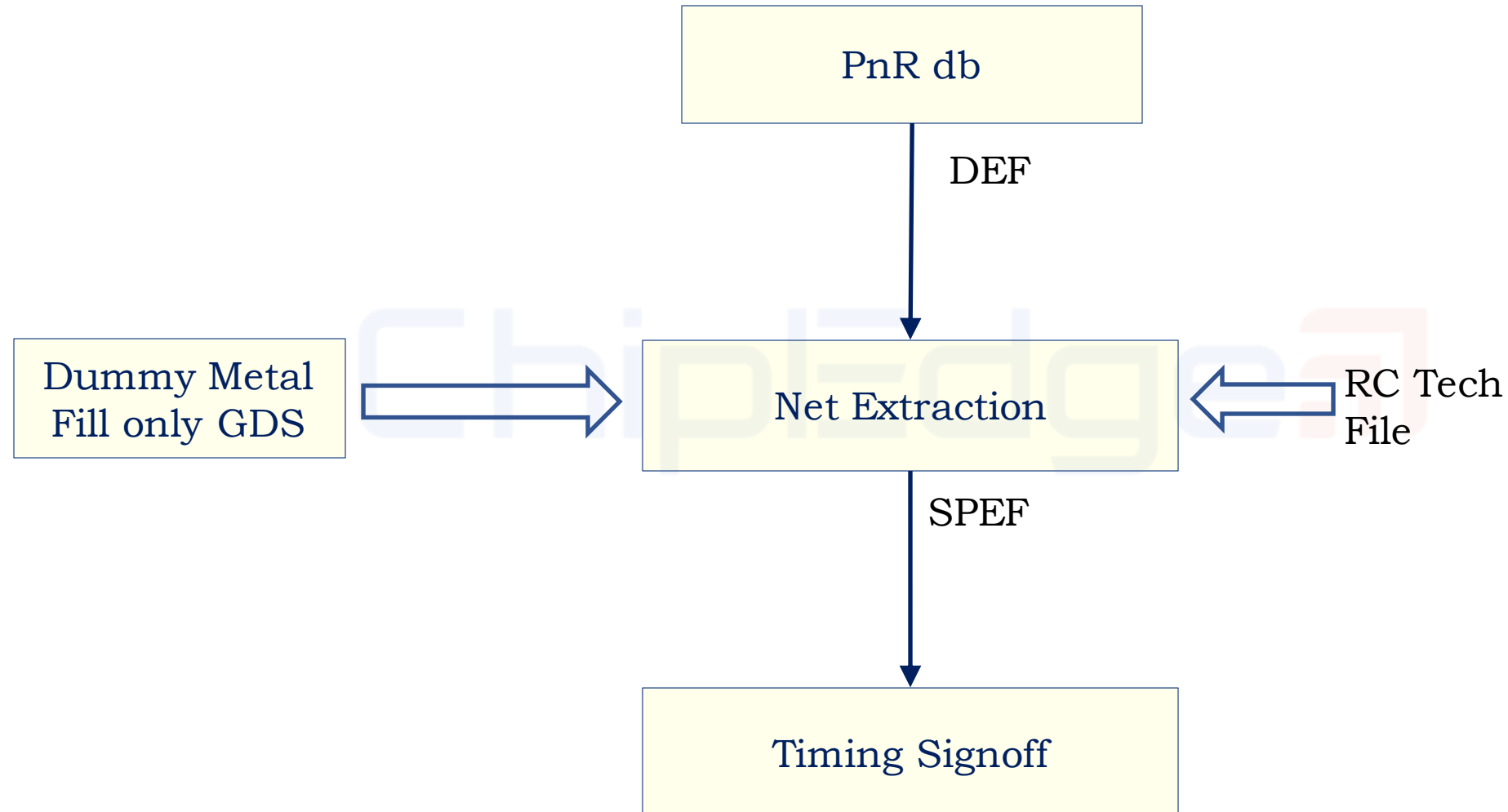
- In deep-submicron VLSI manufacturing, dummy fills are widely applied to reduce topographic variations and improve layout pattern uniformity.
- Dummy fill is the term given to the metal shapes (usually rectangles) placed in an IC design for manufacturing purposes, rather than to contribute to a logical function.
- Extracted R&C values of the nets will change after Metal FILL is done.
- That is why we need to redo extraction for final timing sign-off.
- Dummy Metal-FILL only GDS is supplied to the Extraction tool for it to consider the impact of FILL shapes on wire parasitic and thereby timing.

Before Dummy fill addition



After Dummy fill addition





Introduction to Signoff



Filler Cells



Decap Cells



Dummy Fill



PV Checks

This section explains various PV Checks. Which include DRC Checks, Antenna Checks , LVS Checks, ERC, DFM/DFY, LPC/PM/MAS Checks and PERC Checks

IR Drop / EM



ESD



- PV Checks are needed to ensure manufacturability of the design in the foundry.
- Different kind of PV checks: DRC, Density, LVS and ERC, Antenna, etc.
- Rule decks for all the PV checks are received from the foundry.
- Tools used for PV checks: ICV (from Synopsys) or Calibre (from Mentor Graphics).

- Design Rule Check (DRC) verifies whether a specific design meets the constraints imposed by the process technology to be used for its manufacturing.
- DRC Checking is an essential part of the physical design flow and ensures the design meets manufacturing requirements and will not result in a chip failure.
- The Process Technology rules are provided by process engineers and/or fabrication facility.

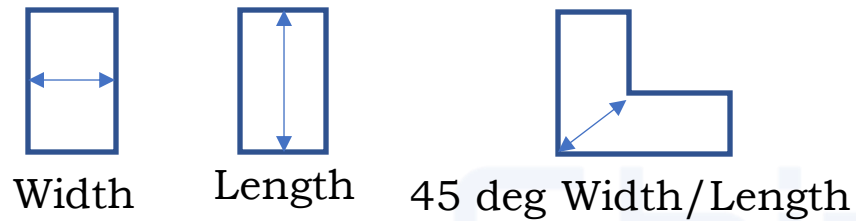
- DRC Rule Decks are the files provided by foundry which contains all the design rules to be passed for reliable manufacturing.
- DRC Rules defined in rule deck are much more extensive than the Tech file used in the PnR flow to build the database.
- Each Process Technology will have its own set of rules. The number of DRC rules and complexity of rules increases as the manufacturing technology shrinks at advanced nodes.

Here are some basic and common types of DRC rules

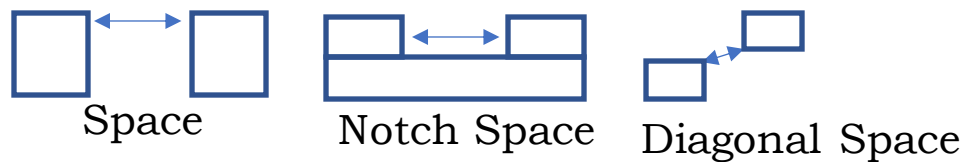
- Minimum width.
- Minimum spacing.
- Minimum area.
- Misaligned via wire.
- Special notch spacing.
- End of line spacing.

- Basic types of DRC rules

Distance of interior facing edge for a single layer



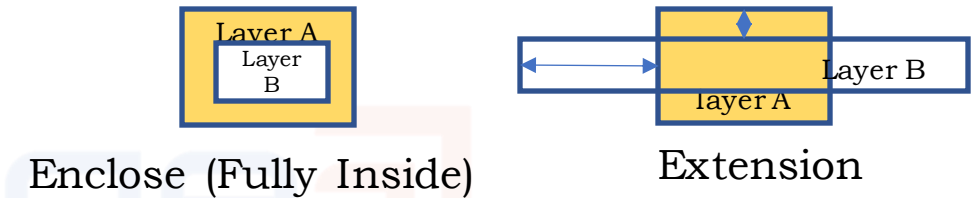
Distance of exterior facing edge of two layer



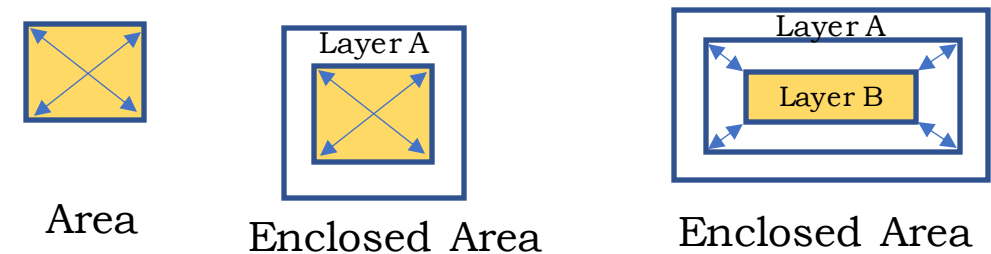
Distance of interior facing edge of two layer



Distance between inside edge to outside edge



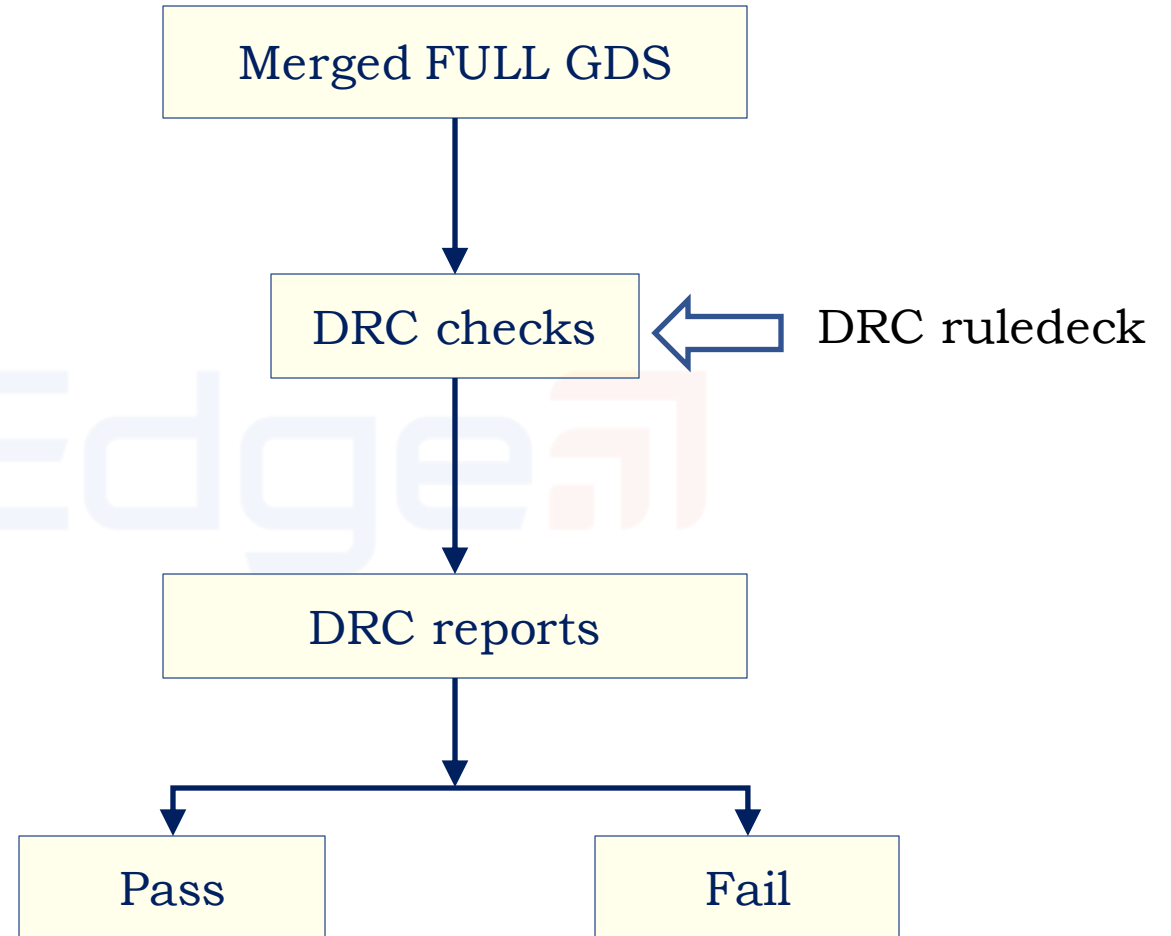
Area and enclosed Area



- Base Layer checks
 - ✓ NWELL rules
 - ✓ Poly rules
 - ✓ Base layer density rules
 - ✓ V_T design rules
 - ✓ Latch-Up rules
- Metal Layer checks
 - ✓ Metal rules
 - ✓ VIA rules
 - ✓ Metal and Via layer density rules
 - ✓ Bump rules

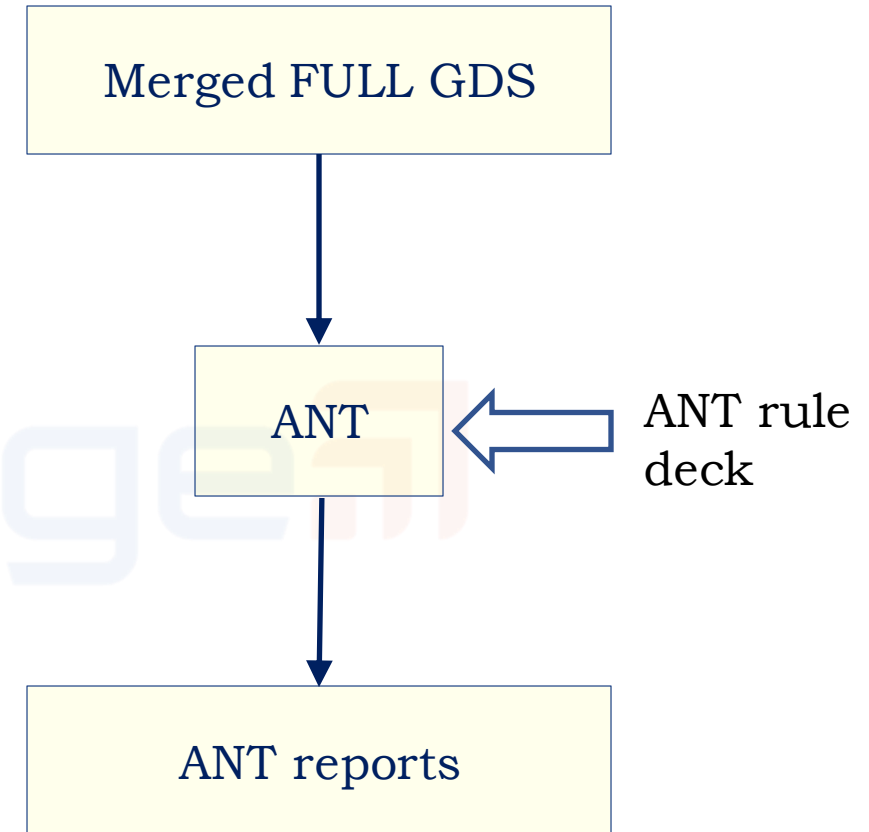
How is it checked?

- Merged GDS (after dummy fill) supplied.
- DRC rule decks received from Foundry which have all the checks.
- Pass-Tapeout/other signoff checks.
- Fail – errors need to be analysed and fixed.



- Density check is performed to check the even density through out the chip to ensure the mechanical sturdiness of the chip to achieve planarity during Chemical Mechanical Processing (CMP).
- Reduce manufacturing variation due to CMP process.
- The density rules are defined for metal layers as well as base layers .
- Types: Minimum density, Maximum density.
- Both global density and local density needs to be checked.
- Typically, Density checks are also part of the DRC rule deck from foundry.

- Metal and VIA Antenna Checks.
- Antenna rules apply separately to thin and thick oxide.
- Antenna Ratio (Design) = $\text{Metal Area} / (\text{Gate Area} + \text{Diff Area})$.

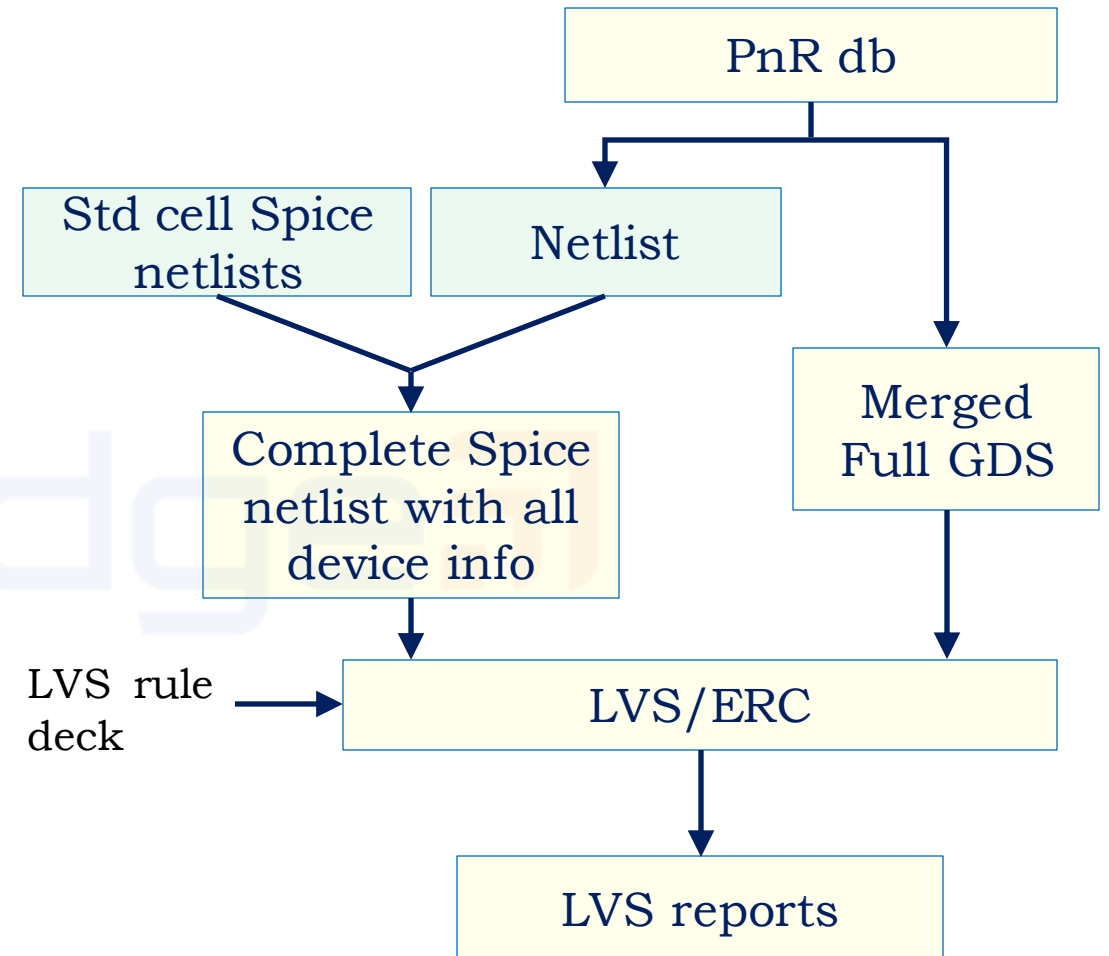


- The Antenna effect is formally plasma induced gate oxide damage, is an effect that can potentially damage the Gate of the MOSFET during Fabrication process.
- In design, this is represented by Antenna Ratio (metal to gate-area ratio), which if higher than a limit, leads to antenna violation.
- $\text{Antenna Ratio (Design)} = \text{Metal Area} / (\text{Gate Area} + \text{Diff Area})$.
- $\text{Antenna Ratio (Design)} \leq \text{Antenna Ratio Limit}$ (Pass).
- $\text{Antenna Ratio (Design)} > \text{Antenna Ratio Limit}$ (Violation).

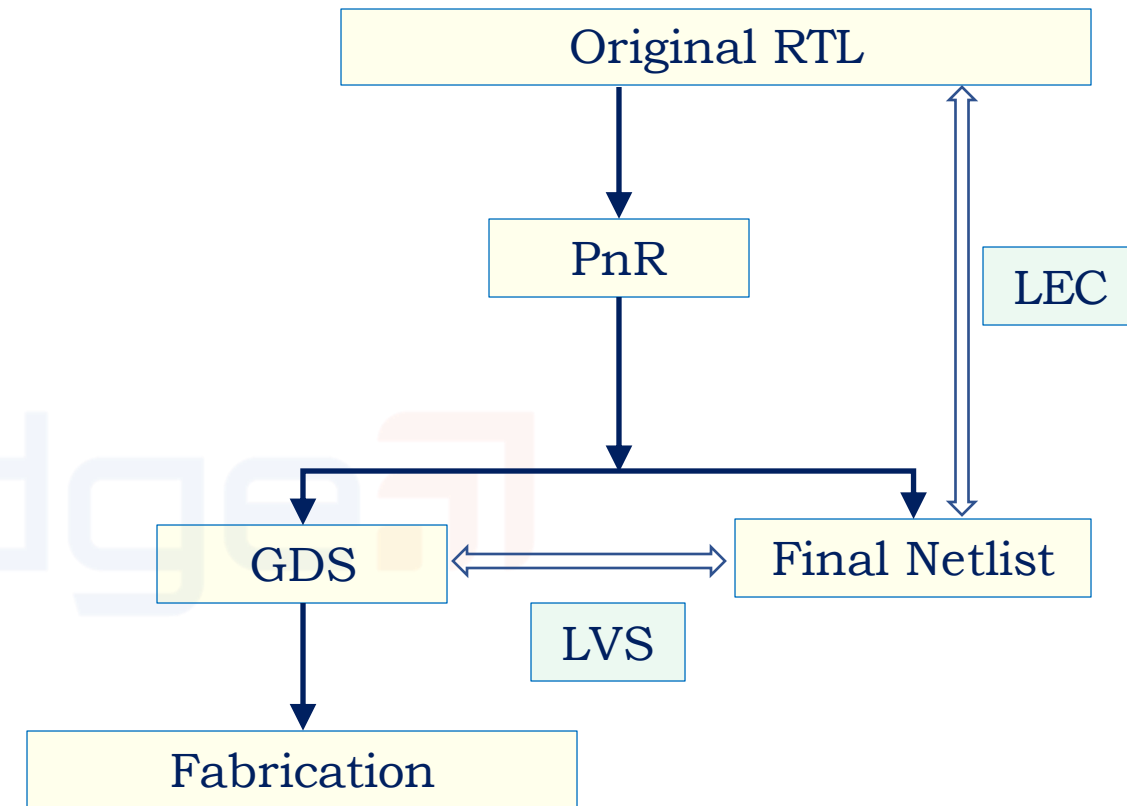
- Layout vs. Schematic check
- To confirm that Physical database has the same functionality as the PnR logical netlist
- Create a netlist out of Physical database, based on connectivity and compare against the logical netlist from the PnR database.

- Layout Versus Schematic (LVS) checking compares the extracted netlist from the layout GDS to the original schematic netlist to determine if they match.
- The comparison check is considered clean if all the devices and nets of the schematic match the devices and the nets of the layout.
- Optionally, the device properties can also be compared to determine if they match within a certain tolerance.

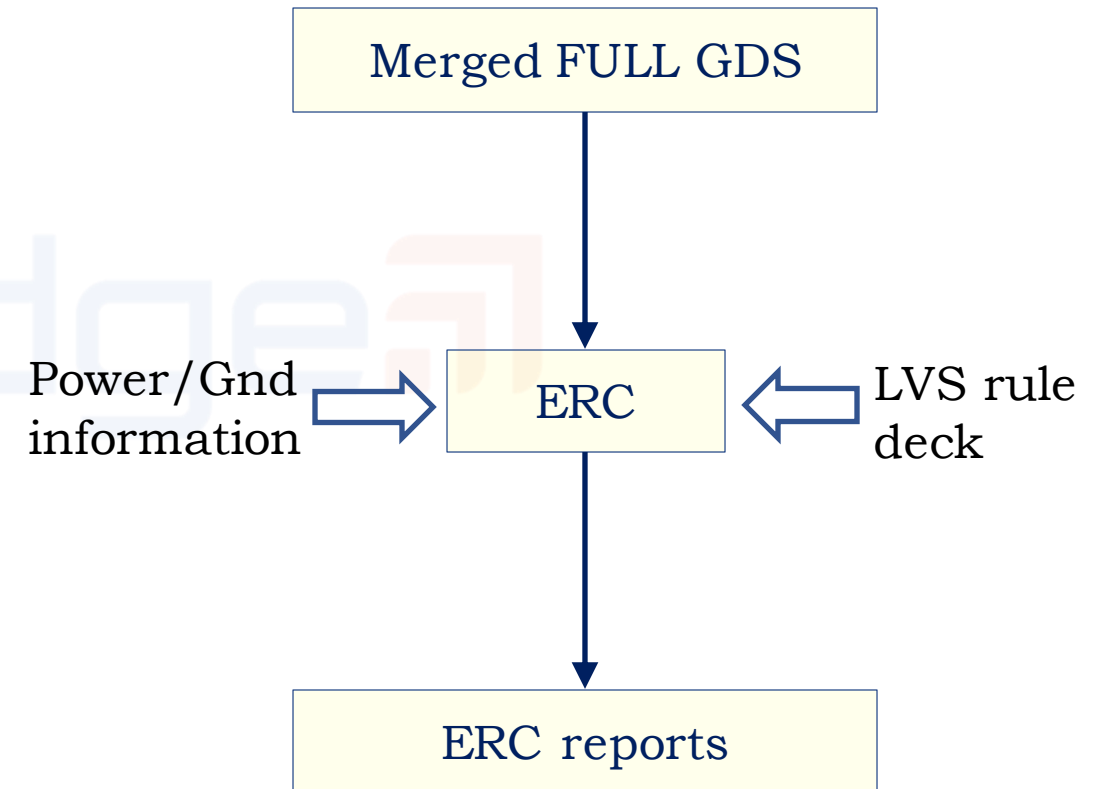
- Layout vs. Schematic.
- Internally tool converts merged GDS to Spice.
- Comparison between 2 spice netlists.
- Common LVS issues
 - ✓ Net Shorts
 - ✓ Net Opens
 - ✓ Device mismatches
 - ✓ Net mismatches
 - ✓ property mismatches
 - ✓ Missing well connections



- GDS → goes for manufacturing.
- LEC Checks the final netlist is equivalent to original RTL.
- LVS Checks the GDS equivalent to final Netlist.
- LEC + LVS confirm that at the GDS going to Fab is equivalent to the original RTL design.



- ERC is a methodology used to check the robustness of a design both at schematic and layout levels against various electrical design rules.
- These ERC rules are often design-specific and developed based on knowledge from previous projects or in anticipation of potential new failures.



- Floating gates.
- Wrong transistor connections (Source and Drain connected together for instance).
- Floating interconnect, Metal, Poly.
- Shorted Drain & Source of a MOS .
- No substrate or Well contact.
- Different contacts of substrate / well are connected to different nets.
- Distance of MOS to next substrate / well contact too large (Latchup rule).

Well and substrate bias checks Soft connect checks: checks related to

- ✓ high resistance layers
- ✓ Guard ring checks
- ✓ Power and ground checks
- ✓ Floating gate checks
- ✓ Floating well/substrate checks

- PERC is a method for checking reliability issues of IC designs that cannot be checked with DRC or LVS.
- PERC checks majorly for ESD and Latch Up related issues in the design.
- Foundry provides a rule deck with a set of rules which user has to check against the design.
- PERC majorly has the following categories of check
 - ✓ Topology checks : ESD device related checks
 - ✓ LDL checks : Latch-Up related checks (mostly external latch-up)
 - ✓ CD checks: Current density related checks on ESD paths
 - ✓ P2P checks : Point to Point resistance checks on ESD paths

- DFM Stands For Design For Manufacturability.
- DFM Checks Are Over And Above DRC Checks.
- Not Following The Mandatory DFM Rules Can Lead To Defect Induced Yield Loss.
- Few Of The DFM Checks Are
 - ✓ LPC (Litho Physical Checks)
 - ✓ PM (Pattern Matching)
 - ✓ MAS (Manufacturability Analysis Scoring) etc.
- DFY Stands For Design For Yield
 - ✓ Multi-cut Via Insertion
 - ✓ All Corner Case Design
 - ✓ Dummy Fill

- LPC stands for Litho Physical Checks. Patterns that are difficult to manufacture using lithography are found using this check and user has to fix them.
- PM (Pattern Matching) is efficient and fast to create and detect known-process hotspots on new designs regardless of the type of pattern configuration. The PM library is based on real hotspots by accumulating wafer data.
- MAS stands for Manufacturability Analysis Scoring. MAS is a relative yield measurement based on the DFM recommended rules and failure rate.

Introduction to Signoff



Filler Cells



Decap Cells



Dummy Fill



PV Checks



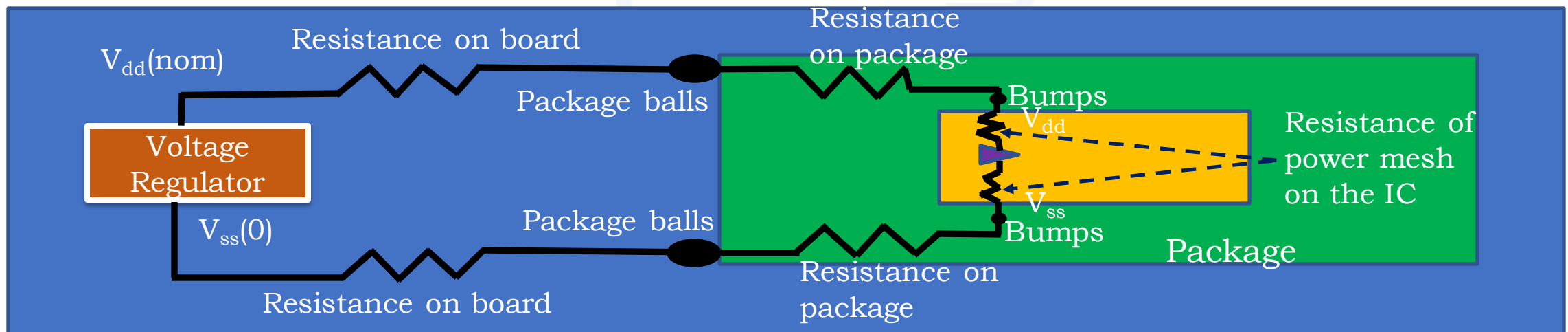
IR Drop / EM

This section explains IR Drop, power delivery to the standard cells and Electro-Migration

ESD



- IR Drop is the voltage drop in the metal wires constituting the power grid before it reaches the power/gnd pins of standard cells.
- General IR drop budgets : 5% for Voltage Regulator fluctuations, 2% for resistances from Voltage Regulator to the IC bumps and then have 3% for drop from the IC bumps to the std cells (on the power grid)



Static IR drop

- General targets have been 2-3%.
- Uses average switching factor to derive current values and the IR drop numbers.
- Can provide vectors using VCD or SAIF file to the tool to derive average switching factors.

Dynamic IR drop

- General targets have been 10 – 15%.
- Uses switching either from VCD or SAIF or switching factor numbers provided by user to derive peak current numbers.
- Tool then uses the peak current numbers to derive dynamic IR drop violations in the design.

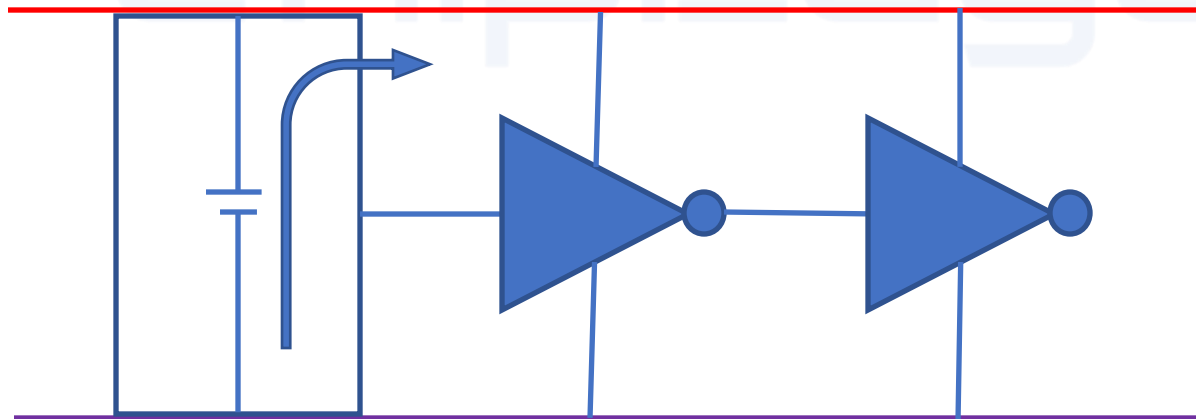
Static IR drop: Fix options

- Debug for any missing straps, vias. Add them incrementally if missing.
- Might have to add parallel stripes/straps to reduce resistance.
- Might have to add additional vias.
- Might have to move cells if high switching/drive cells are placed closer.

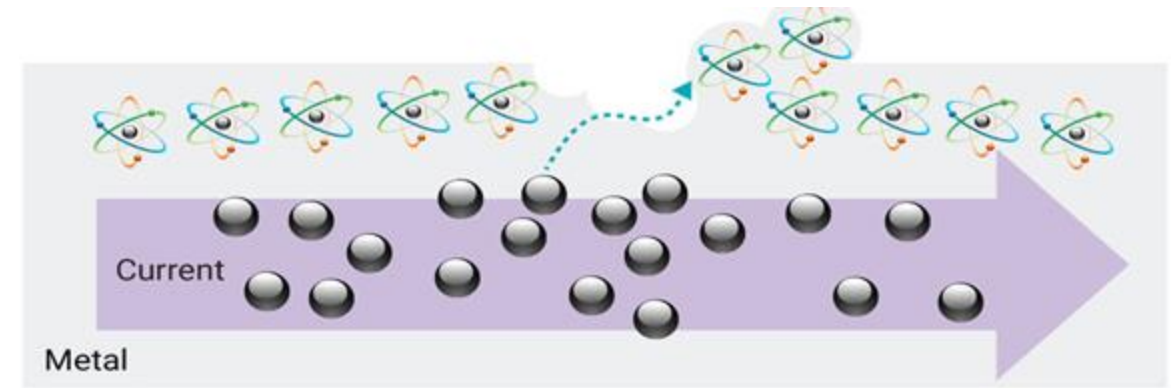
Dynamic IR drop: Fix options

- Adding parallel metals, vias etc can work.
- Check if high drive, power consuming cells are placed closer.
- Adding decaps.

- Can supply current locally during high transient IR drop conditions.
- As a better design methodology these days IC designers add them in a grid fashion and preplace them before placement.
- For clock buffers/inverters better to pad with decap cells.



- Electro-Migration is the movement of atoms based on the flow of high current through a material.
- If the current density is high enough, the heat dissipated will break atoms from the structure and move them.
- Will create both 'vacancies' and 'deposits' leading to Shorts, Opens.



Depletion of atoms (voids) result in an open



Depositions of atoms (Hillocks) result in a short



- Higher current density in a metal may lead to EM.
- For a Technology node, Maximum current density (EM Limit) for Each metal layer is provided by Foundry.
- In Power planning width of the metal layer is decided based on the EM Limit.
- Increasing the metal width will help in avoiding EM Violation.

Electro-Migration (EM):

- Power EM: EM Violation on power nets
- Signal EM: EM Violation on signal nets
- Adding parallel stripes/vias
- Increase width of wires, or buffer them
- Reducing the drive strength of driver

Introduction to Signoff



Filler Cells



Decap Cells



Dummy Fill



PV Checks



IR Drop / EM

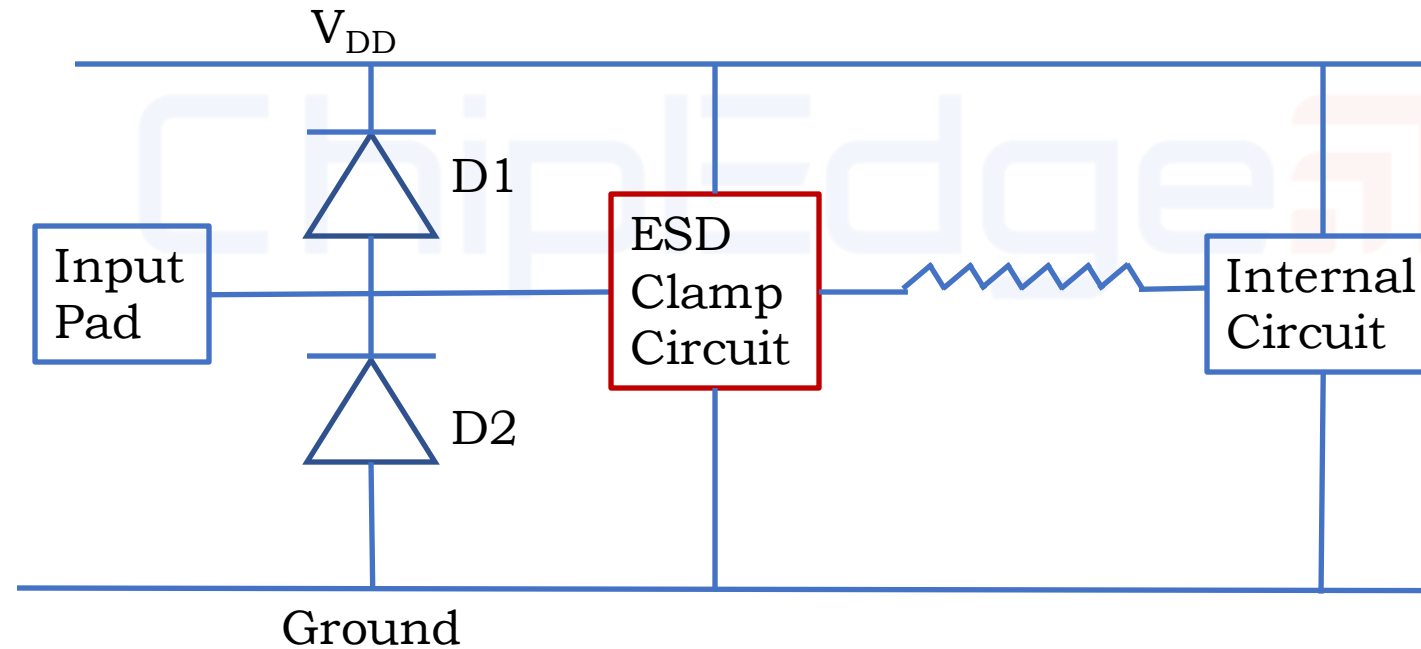


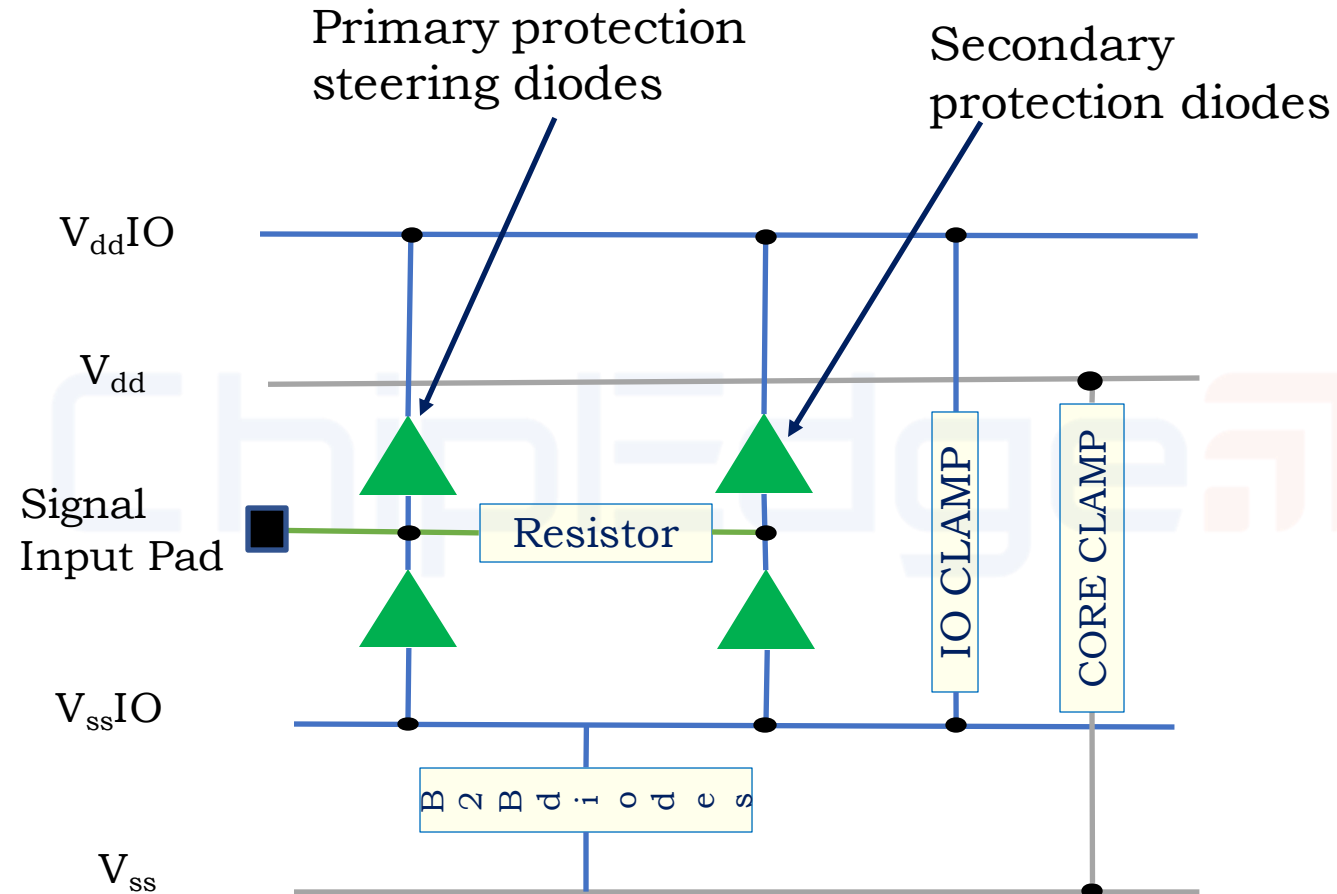
ESD

This section explains about ESD and ESD protection in IC

- ESD is sudden flow of static electricity between two electrically charged objects for a very short duration of time.
- ESD occurs when two bodies at different potentials come in direct contact or if there is high electrostatic field between two objects that are in close proximity.
- During manufacturing it is inevitable that the IC will suffer various kinds of ESD damages.
- Different environments, wafer during CMOS process, package, testing, human handling will generate different kinds of ESDs.

- ESD protection circuits are designed and placed in the IC to protect the active/functional circuitry in the design against any ESD strikes.





- Introduction to Signoff
- Filler Cells
- Decap Cells
- Dummy Fill
- Extraction
- PV Checks
- DRC Checks
- Antenna Checks
- LVS Checks
- ERC
- DFM/DFY
- LPC/PM/MAS Checks
- PERC Checks
- IR Drop / EM
- ESD

Module 16

Advanced Timing Concepts

- In this module, you will learn about timing window and how the cross talk, OCV and CRPR effects on setup and hold timing on various paths.
- Brief overview of various Timing Exceptions like Multi Cycle Path, False Path, Half Cycle Path and Multiple Clock Domain is given in this module.
- You will understand about the concepts of GBA vs PBA, Asynchronous Path and Lock Up Latch.
- Brief overview of various Delay Models which includes cell delay models and interconnect delay models is given in this module.

- Cross Talk Effect On Timing
 - ✓ Cross Talk On Data Path
 - ✓ Cross Talk On Launch Clock Path
 - ✓ Cross Talk On Capture Clock Path
 - ✓ Cross Talk On Common Clock Path
- OCV Effect On Timing
 - ✓ OCV On Data Path
 - ✓ OCV On Launch Clock Path
 - ✓ OCV On Capture Clock Path
 - ✓ OCV On Common Clock Path
- CRPR

- Timing Exception
 - ✓ Multi Cycle Path
 - ✓ False Path
 - ✓ Half Cycle Path
 - ✓ Multiple Clock Domain
 - ✓ STA with Latch: Time Borrowing
 - ✓ Clock Gating Checks
- GBA vs PBA
- Asynchronous Path
- Lock Up Latch
- Delay Models
 - ✓ Cell Delay Models
 - ✓ Interconnect Delay Models

Cross Talk Effect On Timing

This section explains about timing window and how the cross talk effects on setup and hold timing on various paths.

OCV Effect On Timing



CRPR



Timing Exception



GBA vs PBA



Asynchronous Path

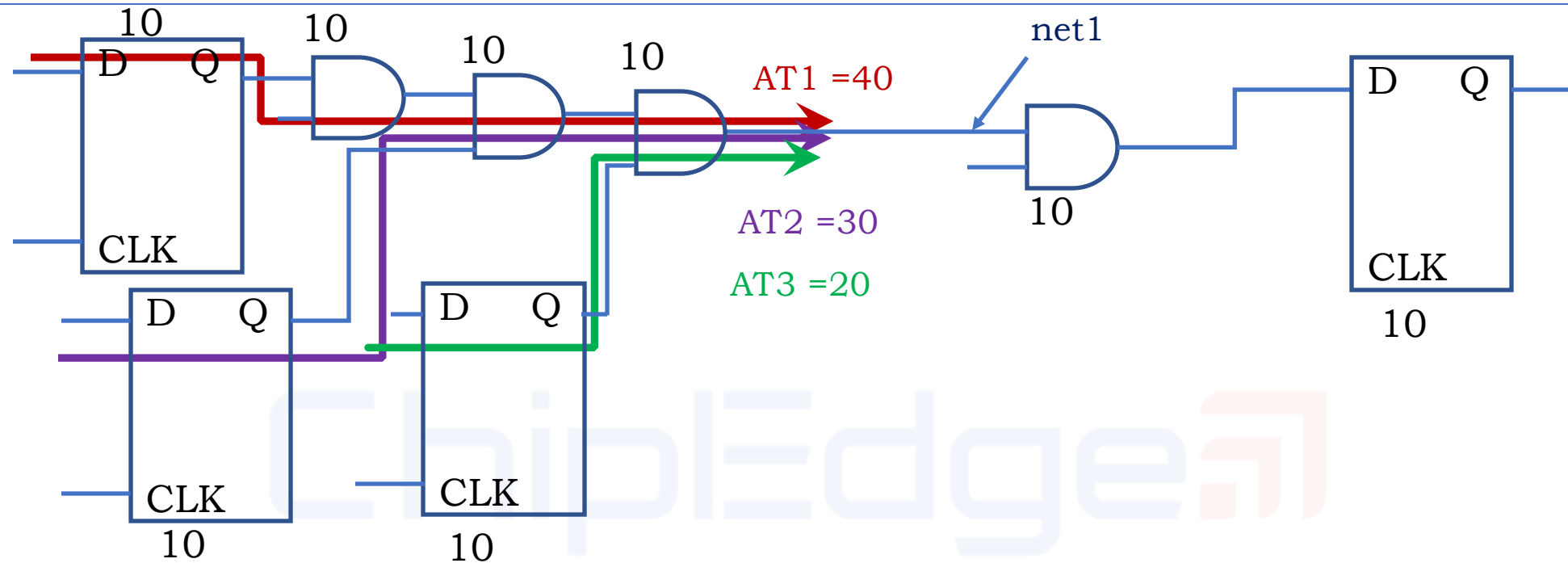


Lock Up Latch

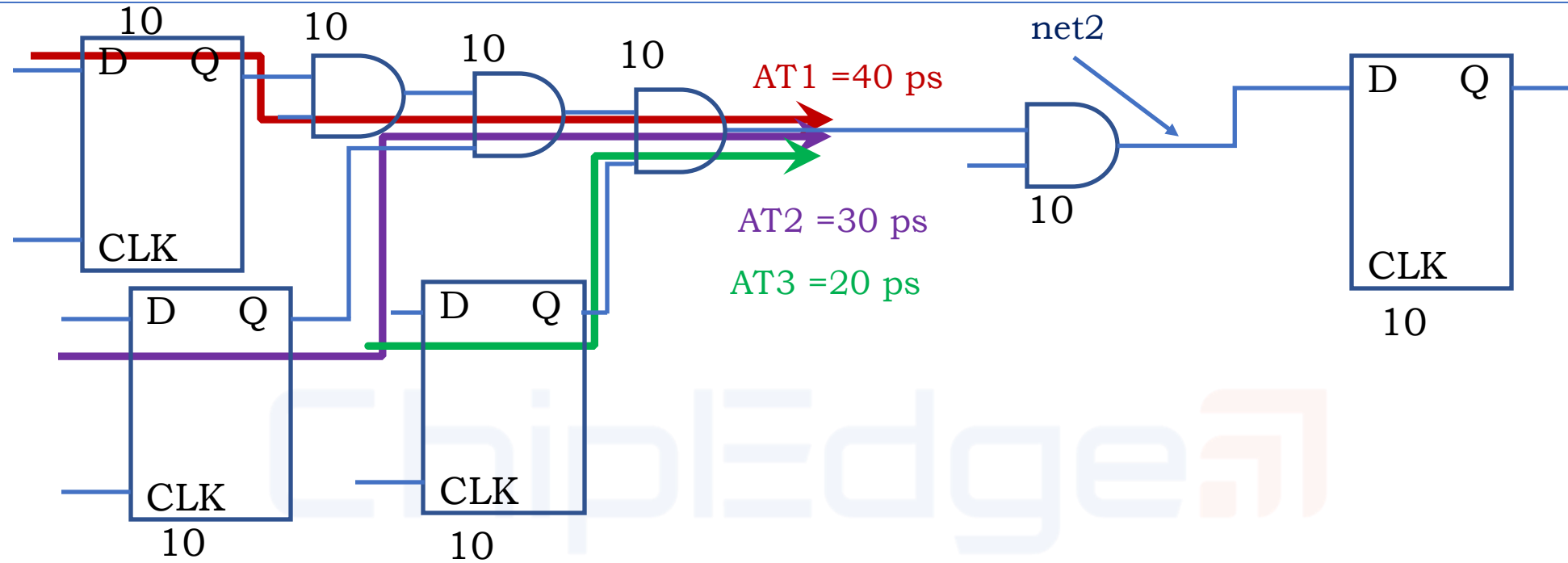


Delay Models

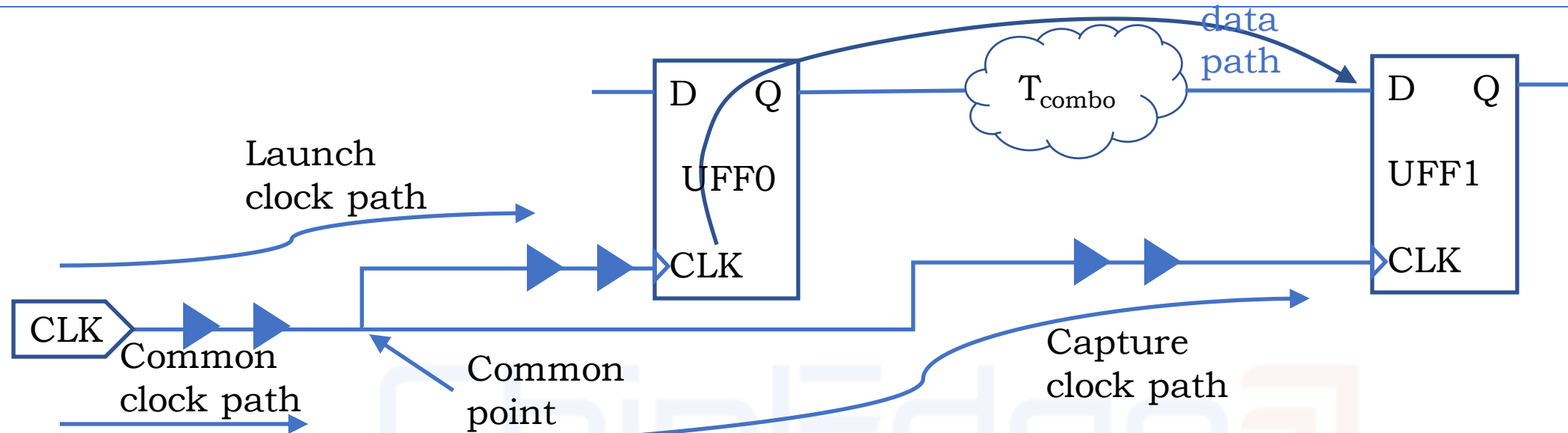




- Minimum Arrival Time at net1 = AT3 = 20ps
- Maximum Arrival Time at net1 = AT1 = 40ps
- Timing Window = [Minimum Arrival Time - Maximum Arrival Time]
= [20 - 40ps]



- Minimum Arrival Time at net2 = 30 ps
- Maximum Arrival Time at net2 = 50 ps
- Timing Window = [Minimum Arrival Time - Maximum Arrival Time]
= [30 – 50 ps]



Worst case impact of crosstalk on Setup and Hold

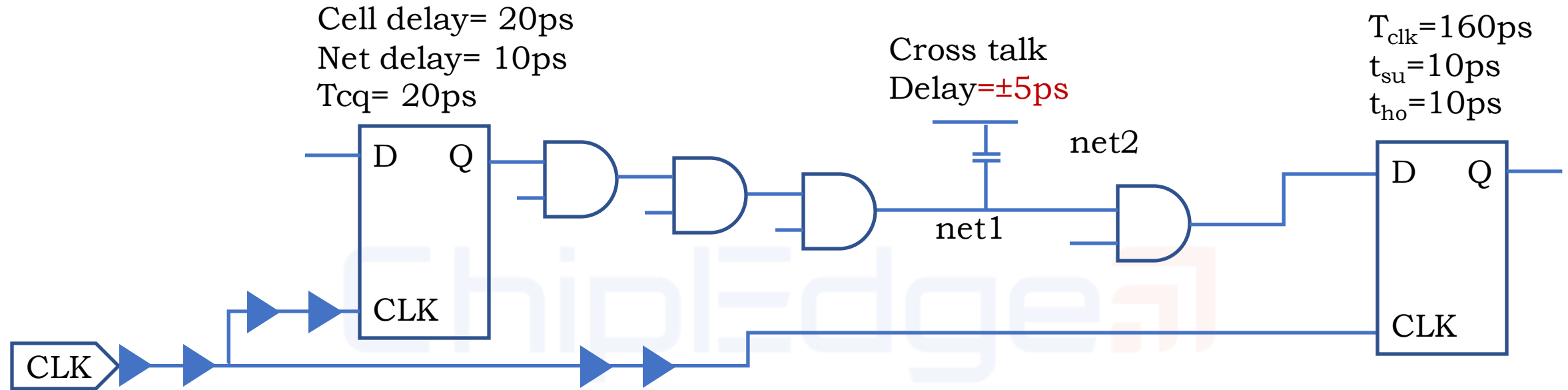
Setup Analysis:

- Launch clock path sees positive crosstalk delay so that data is launched late.
- Data path sees positive crosstalk delay so that it takes longer for data to reach destination (D pin in capture FF).

- Capture clock path sees negative crosstalk delay so that data is captured by capture FF early.

Hold Analysis:

- Launch clock (not including common path) sees negative crosstalk delay so that data is launched early.
- Data path sees negative cross talk delay so that it reaches destination early (D pin in capture FF).
- Capture clock (not including common path) sees positive crosstalk delay so that data is captured by capture FF late.
- *Note: crosstalk can improve setup/hold numbers also (other way)*



Without Crosstalk :

$$T_{lan} = (20+10) + (20+10) + (20+10) + (20+10) = 120$$

$$T_{cq} = 20$$

$$T_{comb} = 10 + (20+10) + (20+10) + (20+10) + (20+1) = 130$$

$$T_{cap} = (20+10) + (20+10) + (20+10) + (20+10) = 120$$

With Crosstalk :

$$T_{lan} = (20+10) + (20+10) + (20+10) + (20+10) = 120$$

$$T_{cq} = 20$$

$$T_{comb} = 10 + (20+10) + (20+10) + (20+10+5) + 20 + 10 = 135$$

$$T_{cap} = (20+10) + (20+10) + (20+10) + (20+10) = 120$$

- $AT \text{ (without CT)} = T_{lan} + T_{cq} + T_{comb}$

$$AT = 120 + 20 + 130 = 270$$

- $RAT = T_{cap} + T_{clk} - t_{su}$

$$RAT = 120 + 160 - 10 = 270$$

- Setup slack check = $RAT - AT$

$$\text{Setup Slack} = 270 - 270 = \mathbf{0}$$

- $AT \text{ (CT)} = T_{lan} + T_{cq} + T_{comb}$

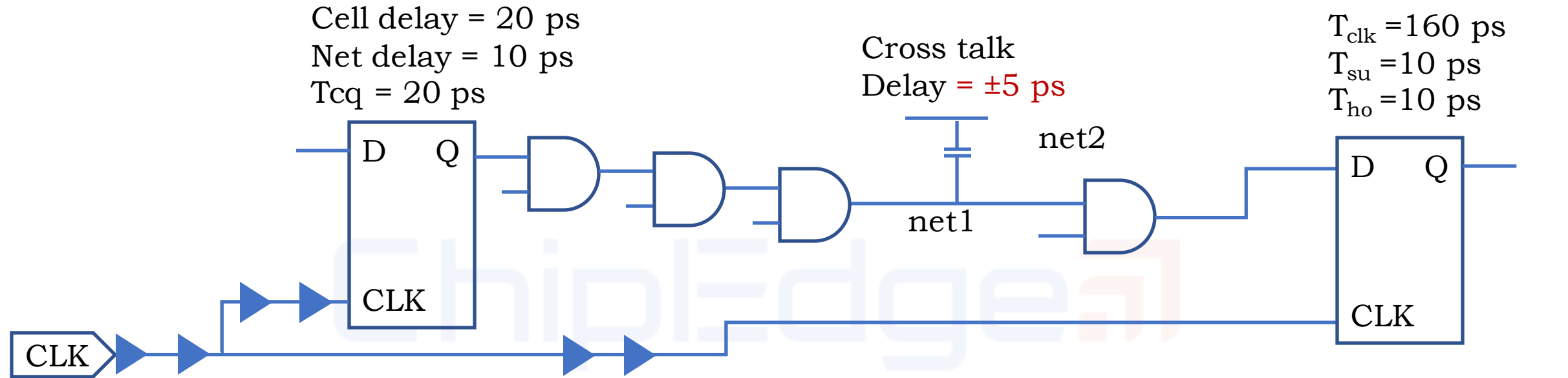
$$AT = 120 + 20 + \mathbf{135} = 275$$

- $RAT = T_{cap} + T_{clk} - t_{su}$

$$RAT = 120 + 160 - 10 = 270$$

- Setup slack check = $RAT - AT$

$$\text{Setup Slack} = 270 - 275 = \mathbf{-5}$$



Without Crosstalk :

$$T_{lan} = (20+10)+(20+10)+(20+10)+(20+10) = 120$$

$$T_{cq} = 20$$

$$T_{comb} = 10+(20+10)+(20+10)+(20+10)+(20+1) = 130$$

$$T_{cap} = (20+10)+(20+10)+(20+10)+(20+10) = 120$$

With Crosstalk :

$$T_{lan} = (20+10)+(20+10)+(20+10)+(20+10) = 120$$

$$T_{cq} = 20$$

$$T_{comb} = 10+(20+10)+(20+10)+(20+10-5)+20+10 = 125$$

$$T_{cap} = (20+10)+(20+10)+(20+10)+(20+10) = 120$$

- $AT \text{ (without CT)} = T_{\text{lan}} + T_{\text{cq}} + T_{\text{comb}}$

$$AT = 120 + 20 + 130 = 270$$

- $RAT = T_{\text{cap}} + t_{\text{ho}}$

$$RAT = 120 + 10 = 130$$

- Hold slack check = $AT - RAT$

$$\text{Hold Slack} = 270 - 130 = 140$$

- $AT \text{ (CT)} = T_{\text{lan}} + T_{\text{cq}} + T_{\text{comb}}$

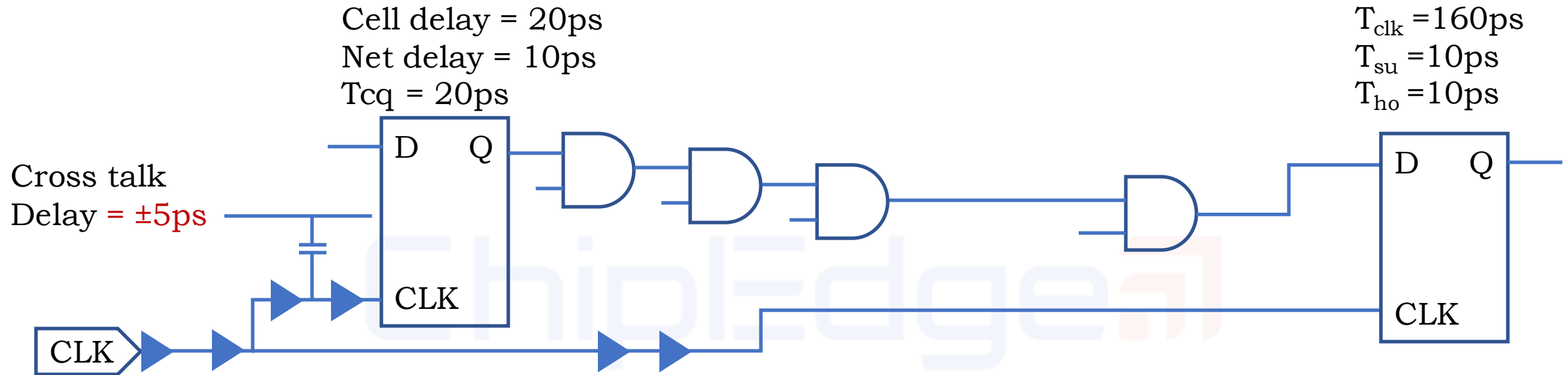
$$AT = 120 + 20 + 125 = 265$$

- $RAT = T_{\text{cap}} + t_{\text{ho}}$

$$RAT = 120 + 10 = 130$$

- Hold slack check = $AT - RAT$

$$\text{Hold Slack} = 265 - 130 = 135$$



Without Crosstalk :

$$T_{lan} = (20+10)+(20+10)+(20+10)+(20+10) = 120$$

$$T_{cq} = 20$$

$$T_{comb} = 10+(20+10)+(20+10)+(20+10)+(20+1) = 130$$

$$T_{cap} = (20+10)+(20+10)+(20+10)+(20+10) = 120$$

With Crosstalk :

$$T_{lan} = (20+10)+(20+10)+(20+10+5)+(20+10) = 125$$

$$T_{cq} = 20$$

$$T_{comb} = 10+(20+10)+(20+10)+(20+10)+20+10 = 130$$

$$T_{cap} = (20+10)+(20+10)+(20+10)+(20+10) = 120$$

- $AT \text{ (without CT)} = T_{\text{lan}} + T_{\text{cq}} + T_{\text{comb}}$

$$AT = 120 + 20 + 130 = 270$$

- $RAT = T_{\text{cap}} + T_{\text{clk}} - t_{\text{su}}$

$$RAT = 120 + 160 - 10 = 270$$

- Setup slack check = $RAT - AT$

$$\text{Setup Slack} = 270 - 270 = \mathbf{0}$$

- $AT \text{ (CT)} = T_{\text{lan}} + T_{\text{cq}} + T_{\text{comb}}$

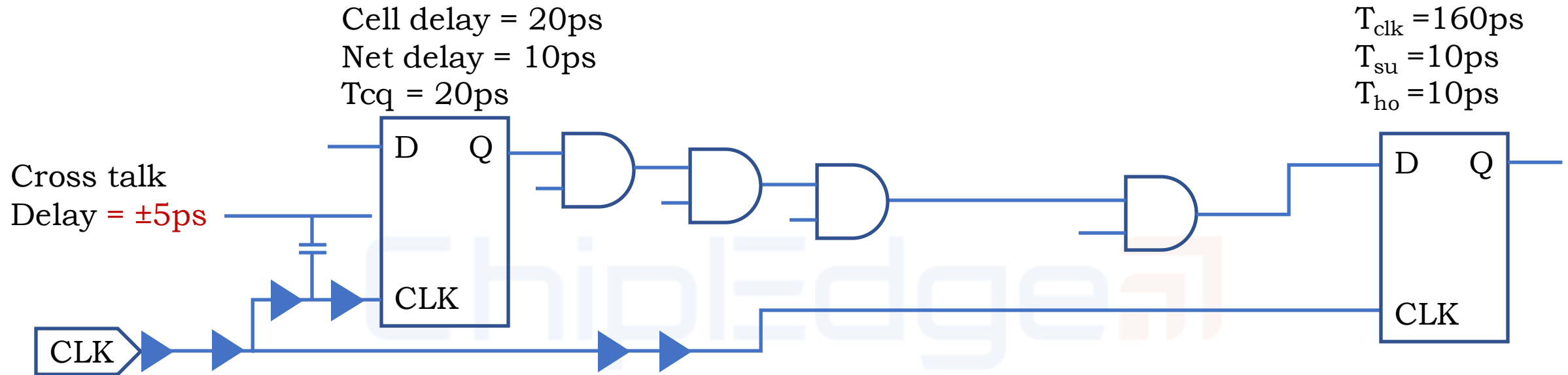
$$AT = 125 + 20 + 130 = 275$$

- $RAT = T_{\text{cap}} + T_{\text{clk}} - t_{\text{su}}$

$$RAT = 120 + 160 - 10 = 270$$

- Setup slack check = $RAT - AT$

$$\text{Setup Slack} = 270 - 275 = \mathbf{-5}$$



Without Crosstalk :

$$T_{lan} = (20+10)+(20+10)+(20+10)+(20+10) = 120$$

$$T_{cq} = 20$$

$$T_{comb} = 10+(20+10)+(20+10)+(20+10)+(20+1) = 130$$

$$T_{cap} = (20+10)+(20+10)+(20+10)+(20+10) = 120$$

With Crosstalk :

$$T_{lan} = (20+10)+(20+10)+(20+10-5)+(20+10) = 115$$

$$T_{cq} = 20$$

$$T_{comb} = 10+(20+10)+(20+10)+(20+10)+20+10 = 130$$

$$T_{cap} = (20+10)+(20+10)+(20+10)+(20+10) = 120$$

- $AT \text{ (without CT)} = T_{\text{lan}} + T_{\text{cq}} + T_{\text{comb}}$

$$AT = 120 + 20 + 130 = 270$$

- $RAT = T_{\text{cap}} + t_{\text{ho}}$

$$RAT = 120 + 10 = 130$$

- Hold slack check = $AT - RAT$

$$\text{Hold Slack} = 270 - 130 = \mathbf{140}$$

- $AT \text{ (CT)} = T_{\text{lan}} + T_{\text{cq}} + T_{\text{comb}}$

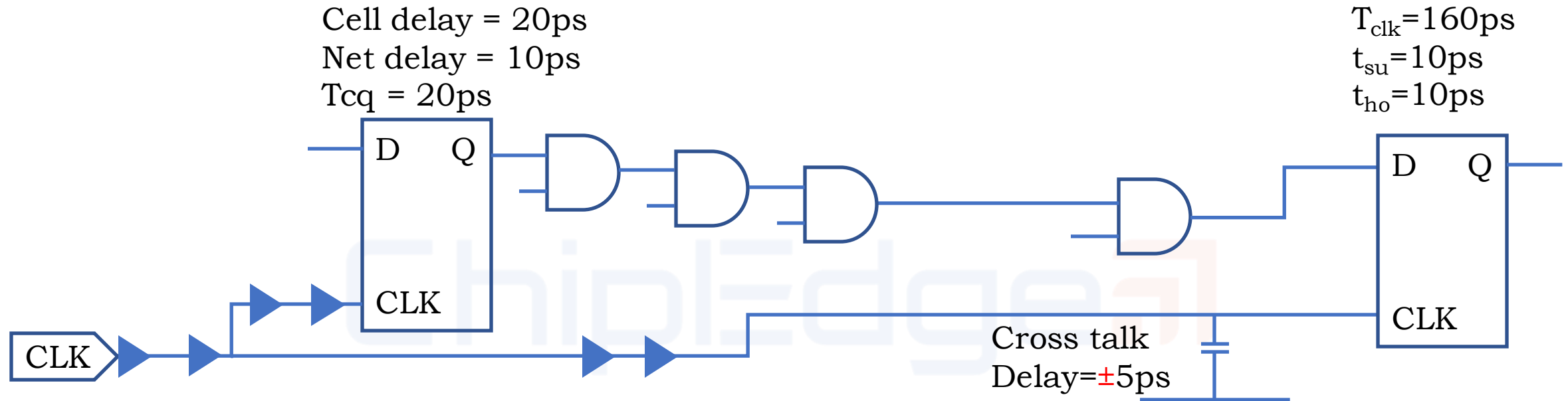
$$AT = 115 + 20 + 130 = 265$$

- $RAT = T_{\text{cap}} + t_{\text{ho}}$

$$RAT = 120 + 10 = 130$$

- Hold slack check = $AT - RAT$

$$\text{Hold Slack} = 265 - 130 = \mathbf{135}$$



Without Crosstalk :

$$T_{lan} = (20+10) + (20+10) + (20+10) + (20+10) = 120$$

$$T_{cq} = 20$$

$$T_{comb} = 10 + (20+10) + (20+10) + (20+10) + (20+1) = 130$$

$$T_{cap} = (20+10) + (20+10) + (20+10) + (20+10) = 120$$

With Crosstalk :

T_{lan} and T_{cq} are same, no change

$$T_{comb} = 10 + (20+10) + (20+10) + (20+10) + 20 + 10 = 130$$

$$T_{cap} = (20+10) + (20+10) + (20+10) + (20-5+10) = 115$$

- $AT \text{ (without CT)} = T_{lan} + T_{cq} + T_{comb}$

$$AT = 120 + 20 + 130 = 270$$

- $RAT = T_{cap} + T_{clk} - t_{su}$

$$RAT = 120 + 160 - 10 = 270$$

- Setup slack check = $RAT - AT$

$$\text{Setup Slack} = 270 - 270 = \mathbf{0}$$

- $AT \text{ (CT)} = T_{lan} + T_{cq} + T_{comb}$

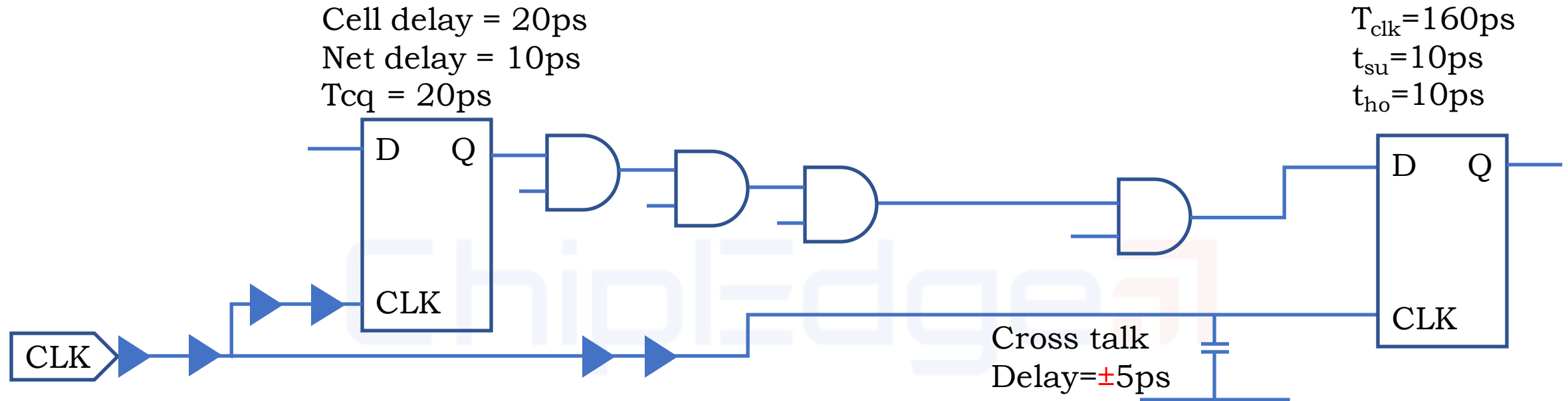
$$AT = 120 + 20 + 130 = 270$$

- $RAT = T_{clk} + T_{cap} - t_{su}$

$$RAT = 160 + \mathbf{115} - 10 = 265$$

- Setup slack check = $RAT - AT$

$$\text{Setup Slack} = 265 - 270 = \mathbf{-5}$$



Without Crosstalk :

$$T_{lan} = (20+10)+(20+10)+(20+10)+(20+10) = 120$$

$$T_{cq} = 20$$

$$T_{comb} = 10+(20+10)+(20+10)+(20+10)+(20+1) = 130$$

$$T_{cap} = (20+10)+(20+10)+(20+10)+(20+10) = 120$$

With Crosstalk :

T_{lan} and T_{cq} are same, no change

$$T_{comb} = 10+(20+10)+(20+10)+(20+10)+20+10 = 130$$

$$T_{cap} = (20+10)+(20+10)+(20+10)+(20+5+10) = 115$$

- $AT(\text{without CT}) = T_{\text{lan}} + T_{\text{cq}} + T_{\text{comb}}$

$$AT = 120 + 20 + 130 = 270$$

- $RAT = T_{\text{cap}} + t_{\text{ho}}$

$$RAT = 120 + 10 = 130$$

- Hold slack check = $AT - RAT$

$$\text{Hold Slack} = 270 - 130 = 140 \text{ ps}$$

- $AT(\text{CT}) = T_{\text{lan}} + T_{\text{cq}} + T_{\text{comb}}$

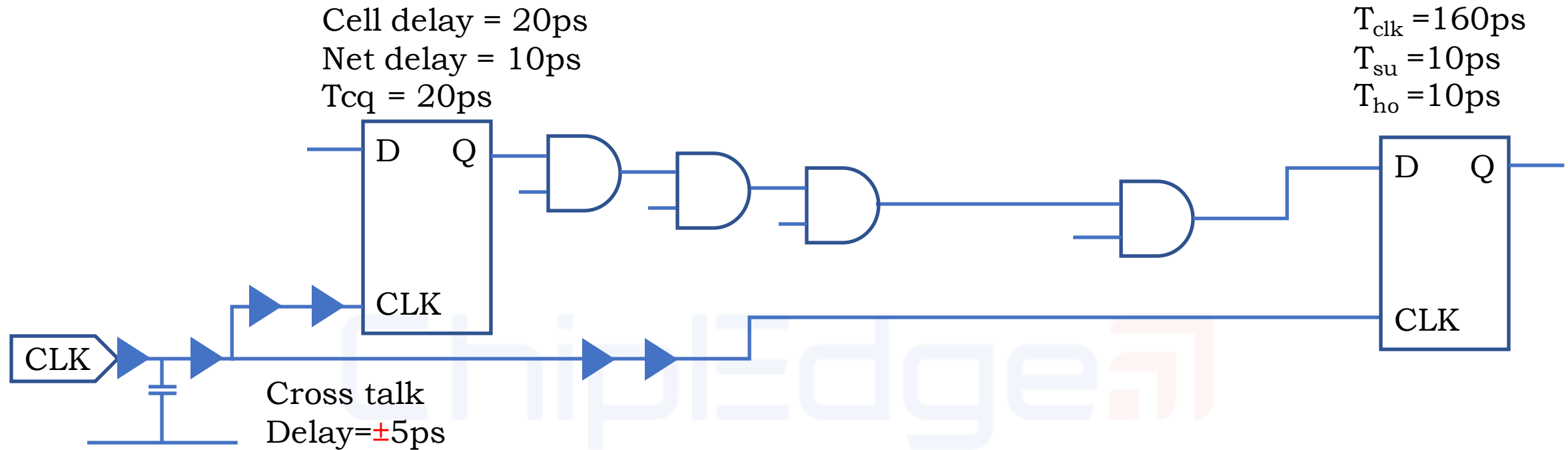
$$AT = 120 + 20 + 130 = 270$$

- $RAT = T_{\text{cap}} + t_{\text{ho}}$

$$RAT = 125 + 10 = 135$$

- Hold slack check = $AT - RAT$

$$\text{Hold Slack} = 270 - 135 = \mathbf{135}$$



Without Crosstalk :

$$T_{lan} = (20+10)+(20+10)+(20+10)+(20+10) = 120$$

$$T_{cq} = 20$$

$$T_{comb} = 10+(20+10)+(20+10)+(20+10)+(20+10) = 130$$

$$T_{cap} = (20+10)+(20+10)+(20+10)+(20+10) = 120$$

With Crosstalk :

$$T_{lan} = (20+10+5)+(20+10)+(20+10)+(20+10) = 120$$

$$T_{cq} = 20$$

$$T_{comb} = 10+(20+10)+(20+10)+(20+10)+20+10=130$$

$$T_{cap} = (20+10-5)+(20+10)+(20+10)+(20+10)=115$$

- $AT \text{ (without CT)} = T_{lan} + T_{cq} + T_{comb}$

$$AT = 120 + 20 + 130 = 270$$

- $RAT = T_{cap} + T_{clk} - t_{su}$

$$RAT = 120 + 160 - 10 = 270$$

- Setup slack check = $RAT - AT$

$$\text{Setup Slack} = 270 - 270 = \mathbf{0}$$

- $AT \text{ (CT)} = T_{lan} + T_{cq} + T_{comb}$

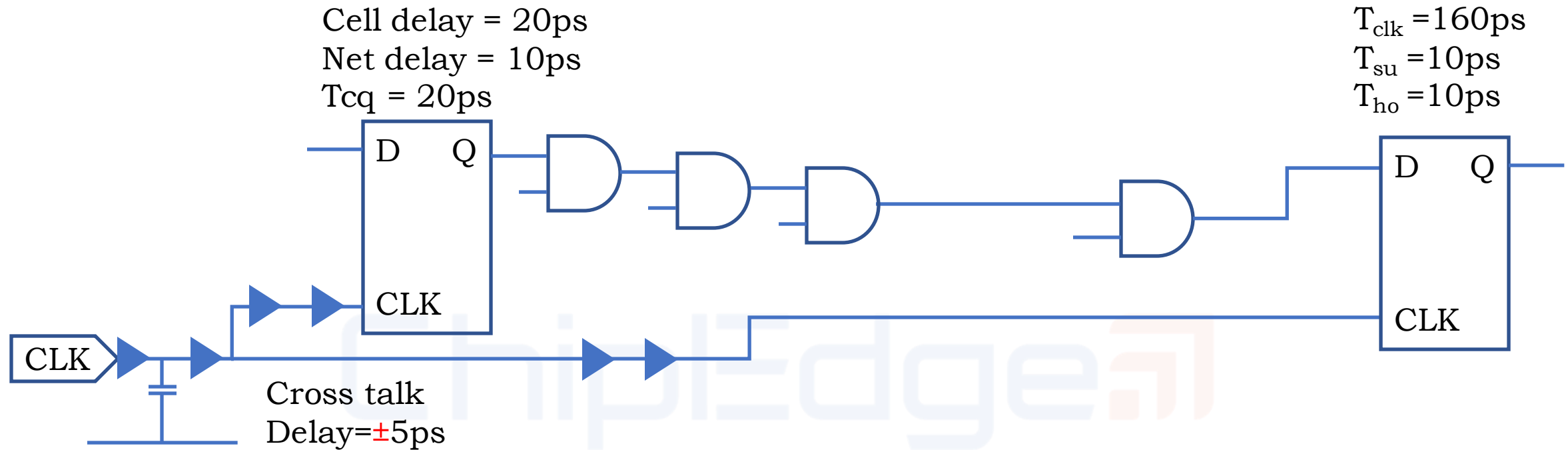
$$AT = 125 + 20 + 130 = 275$$

- $RAT = T_{clk} + T_{cap} - t_{su}$

$$RAT = 160 + 115 - 10 = 265$$

- Setup slack check = $RAT - AT$

$$\text{Setup Slack} = 265 - 275 = \mathbf{-10}$$



Without Crosstalk :

$$T_{lan} = (20+10)+(20+10)+(20+10)+(20+10) = 120$$

$$T_{cq} = 20ps$$

$$T_{comb} = 10+(20+10)+(20+10)+(20+10)+(20+1) = 130$$

$$T_{cap} = (20+10)+(20+10)+(20+10)+(20+10) = 120$$

With Crosstalk :

$$T_{lan} = (20+10+5)+(20+10)+(20+10)+(20+10) = 120$$

$$T_{cq} = 20ps$$

$$T_{comb} = 10+(20+10)+(20+10)+(20+10)+20+10 = 130$$

$$T_{cap} = (20+10+5)+(20+10)+(20+10)+(20+10) = 120$$

- $AT \text{ (without CT)} = T_{lan} + T_{cq} + T_{comb}$

$$AT = 120 + 20 + 130 = 270$$

- $RAT = T_{cap} + t_{ho}$

$$RAT = 120 + 10 = 130$$

- Hold slack check = $AT - RAT$

$$\text{Hold Slack} = 270 - 130 = \mathbf{140}$$

- $AT \text{ (CT)} = T_{lan} + T_{cq} + T_{comb}$

$$AT = 125 + 20 + 130 = 275$$

- $RAT = T_{cap} + t_{ho}$

$$RAT = 125 + 10 = 135$$

- Hold slack check = $AT - RAT$

$$\text{Hold Slack} = 275 - 135 = \mathbf{140}$$

Timing effects on	Without Crosstalk		With Crosstalk	
	Setup	Hold	Setup	Hold
Cross Talk On Data Path	0	140	-5	135
Cross Talk On Launch Clock Path	0	140	-5	135
Cross Talk On Capture Clock Path	0	140	-5	135
Cross Talk On Common Clock Path	0	140	-10	140

- Cross talk degrades both setup and hold slack.
- Cross talk on common clock path has double impact on setup and no impact on hold.

Cross Talk Effect On Timing 

OCV Effect On Timing

This section explains about OCV Derate, OCV effect on setup and hold timing and structural skew VS OCV skew.

CRPR



Timing Exception



GBA vs PBA



Asynchronous Path



Lock Up Latch

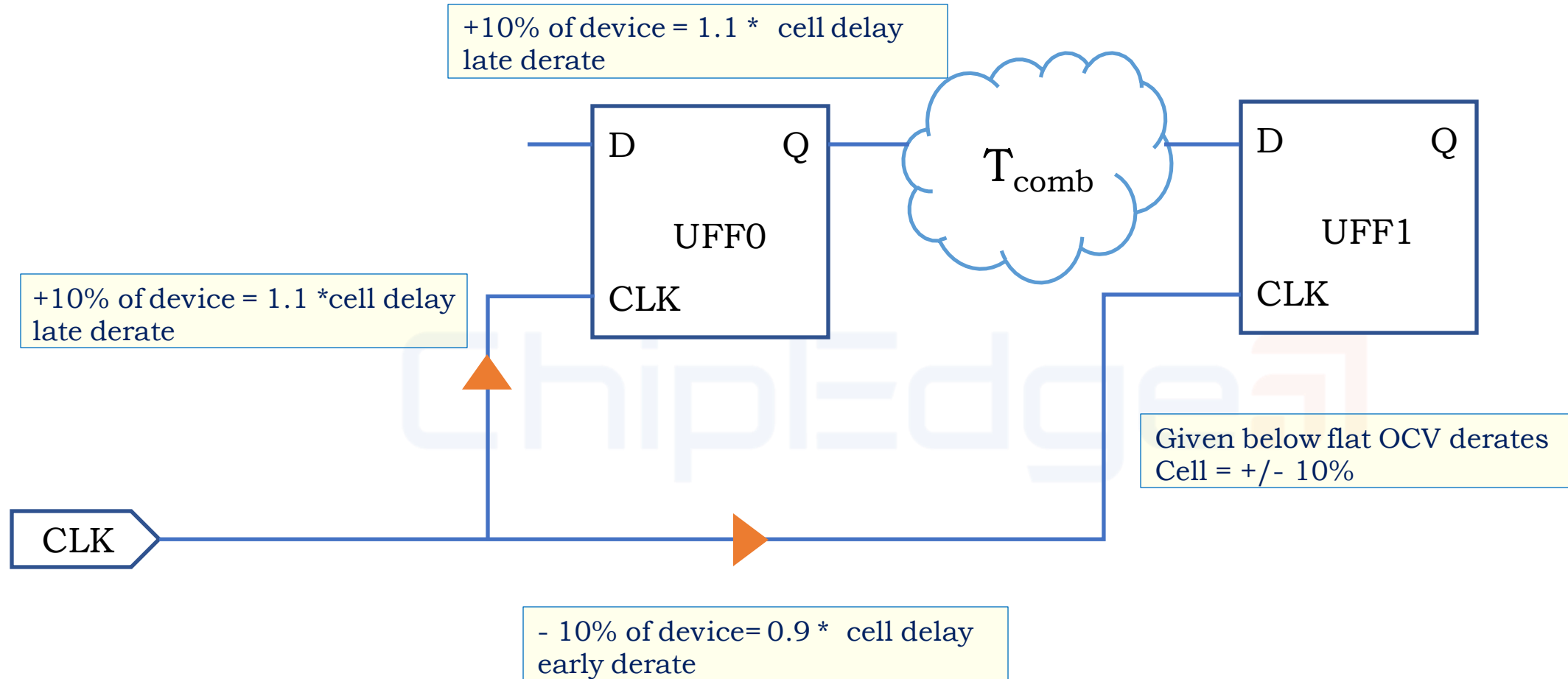


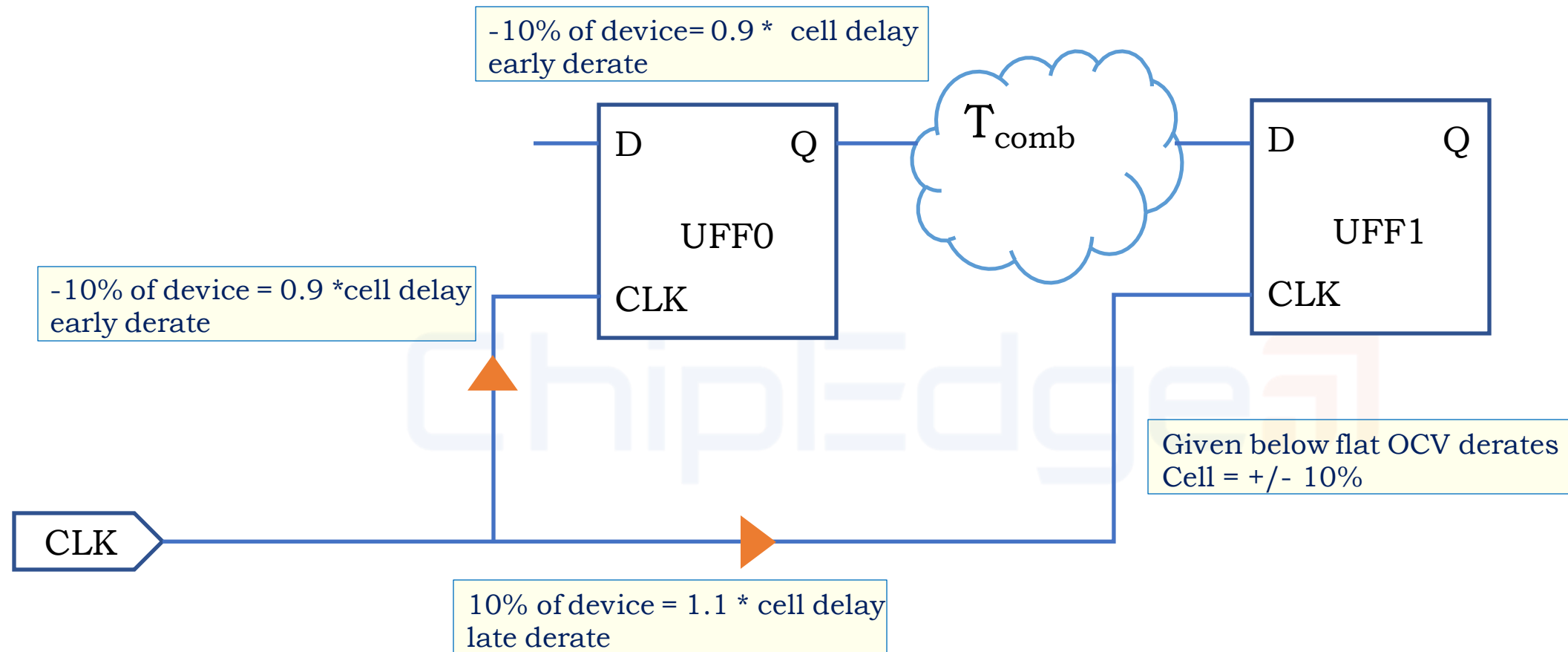
Delay Models

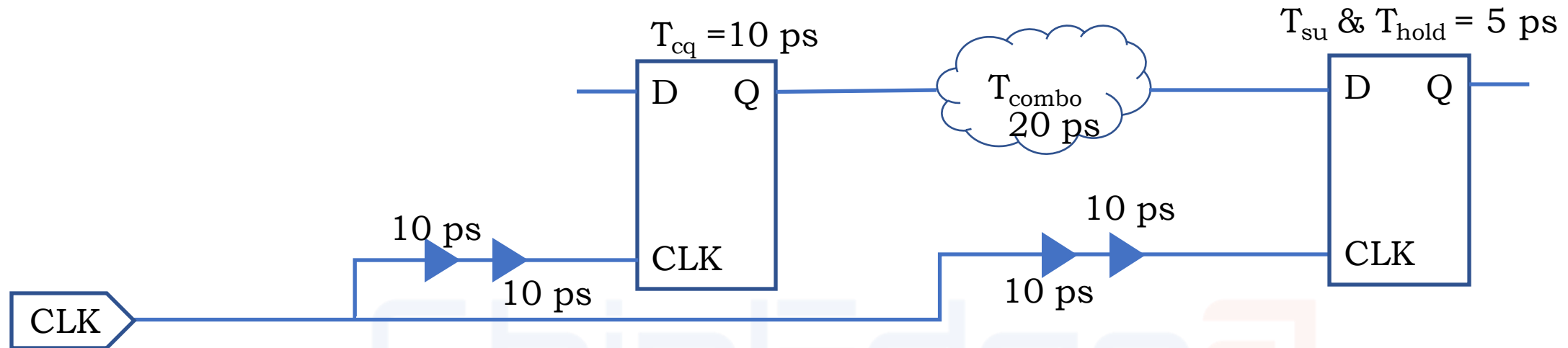


- On Chip Variation could lead to post-silicon failure if it is not taken care while designing the ASIC.
- To take care of OCV we need to add some pessimism in the timing of standard cells.
- We basically apply $\pm x\%$ of additional delay to all the standard cells. Which is called OCV derate.
- User has to apply the derates into timing analysis in such a way that the worst case impacts of OCV on Timing delays are taken care.

- Setup Analysis: To calculate the setup analysis, late derate along the launch path and early derate along the capture path must be considered.
- Hold Analysis: Early derate along the launch path and late derate along the capture path must be considered during the calculation of hold analysis.







- $T_{clk} = 35 \text{ ps}$
- $T_{cq} = 10 \text{ ps}$
- $T_{su} \& T_{hold} = 5 \text{ ps}$
- Late Derate = 1.1
- Early Derate = 0.9

Without OCV:-

$$\begin{aligned} AT &= T_{lan} + T_{cq} + T_{combo} \\ &= 20 + 10 + 20 \\ &= 50 \text{ps} \end{aligned}$$

$$\begin{aligned} RAT &= T_{cap} + T_{clk} - t_{su} \\ &= 20 + 35 - 5 \\ &= 50 \text{ps} \end{aligned}$$

$$\begin{aligned} \text{Setup slack} &= RT - AT \\ &= 50 - 50 \\ &= 0 \text{ ps} \end{aligned}$$

With OCV:-

$$\begin{aligned} AT &= T_{lan} * \text{Late Derate} + T_{cq} * \text{Late Derate} \\ &\quad + T_{combo} * \text{Late Derate} \\ &= 20 * 1.1 + 10 * 1.1 + 20 * 1.1 \\ &= 55 \text{ ps} \end{aligned}$$

$$\begin{aligned} RAT &= T_{cap} * \text{Early Derate} + T_{clk} - t_{su} \\ &= 20 * 0.9 + 35 - 5 \\ &= 48 \text{ ps} \end{aligned}$$

$$\begin{aligned} \text{Setup slack} &= RT - AT \\ &= 48 - 55 \\ &= -7 \text{ ps (negative slack)} \end{aligned}$$

Without OCV:-

$$\begin{aligned} AT &= T_{\text{lan}} + T_{\text{cq}} + T_{\text{combo}} \\ &= 20 + 10 + 20 \\ &= 50 \text{ ps} \end{aligned}$$

$$\begin{aligned} RAT &= T_{\text{cap}} + t_{\text{hold}} \\ &= 20 + 5 \\ &= 25 \text{ ps} \end{aligned}$$

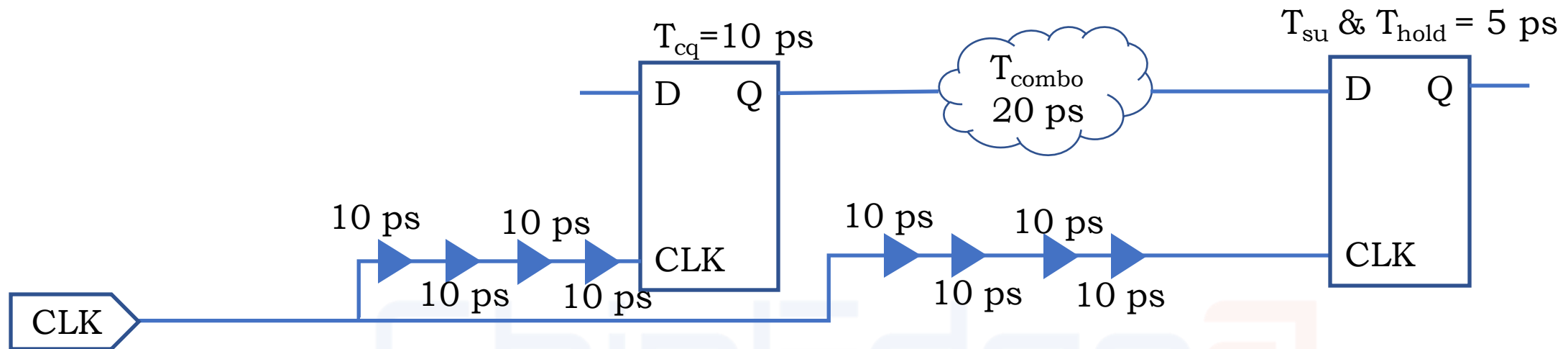
$$\begin{aligned} \text{Hold slack} &= AT - RT \\ &= 50 - 25 \\ &= 25 \text{ ps (positive slack)} \end{aligned}$$

With OCV:-

$$\begin{aligned} AT &= T_{\text{lan}} * \text{Early Derate} + T_{\text{cq}} * \text{Early Derate} \\ &\quad + T_{\text{combo}} * \text{Early Derate} \\ &= 20 * 0.9 + 10 * 0.9 + 20 * 0.9 \\ &= 45 \text{ ps} \end{aligned}$$

$$\begin{aligned} RAT &= T_{\text{cap}} * \text{Late Derate} + t_{\text{hold}} \\ &= 20 * 1.1 + 5 \\ &= 27 \text{ ps} \end{aligned}$$

$$\begin{aligned} \text{Hold slack (OCV)} &= AT - RT \\ &= 45 - 27 \\ &= 18 \text{ ps (positive slack)} \end{aligned}$$



- $T_{\text{clk}} = 35 \text{ ps}$
- $T_{\text{cq}} = 10 \text{ ps}$
- $T_{\text{su}} \& T_{\text{hold}} = 5 \text{ ps}$
- Late Derate 1.1
- Early Derate = 0.9

Without OCV:-

$$\begin{aligned}AT &= T_{\text{lan}} + T_{\text{cq}} + T_{\text{combo}} \\&= 40 + 10 + 20 \\&= 70 \text{ ps}\end{aligned}$$

$$\begin{aligned}\text{RAT} &= T_{\text{capture}} + T_{\text{clk}} - t_{\text{su}} \\&= 40 + 35 - 5 \\&= 70 \text{ ps}\end{aligned}$$

$$\begin{aligned}\text{Setup slack} &= \text{RT} - \text{AT} \\&= 70 - 70 \\&= 0 \text{ ps}\end{aligned}$$

With OCV:-

$$\begin{aligned}AT &= T_{\text{lan}} * \text{Late Derate} + T_{\text{cq}} * \text{Late Derate} + T_{\text{combo}} * \text{Late Derate} \\&= 40 * 1.1 + 10 * 1.1 + 20 * 1.1 \\&= 77 \text{ ps}\end{aligned}$$

$$\begin{aligned}\text{RAT} &= T_{\text{cap}} * \text{Early Derate} + T_{\text{clk}} - t_{\text{su}} \\&= 40 * 0.9 + 35 - 5 \\&= 66 \text{ ps}\end{aligned}$$

$$\begin{aligned}\text{Setup slack} &= \text{RT} - \text{AT} \\&= 66 - 77 \\&= -11 \text{ ps (negative slack)}\end{aligned}$$

Without OCV:-

$$\begin{aligned} AT &= T_{\text{lan}} + T_{\text{cq}} + T_{\text{combo}} \\ &= 40 + 10 + 20 \\ &= 70 \text{ ps} \end{aligned}$$

$$\begin{aligned} RAT &= T_{\text{cap}} + t_{\text{hold}} \\ &= 40 + 5 \\ &= 45 \text{ ps} \end{aligned}$$

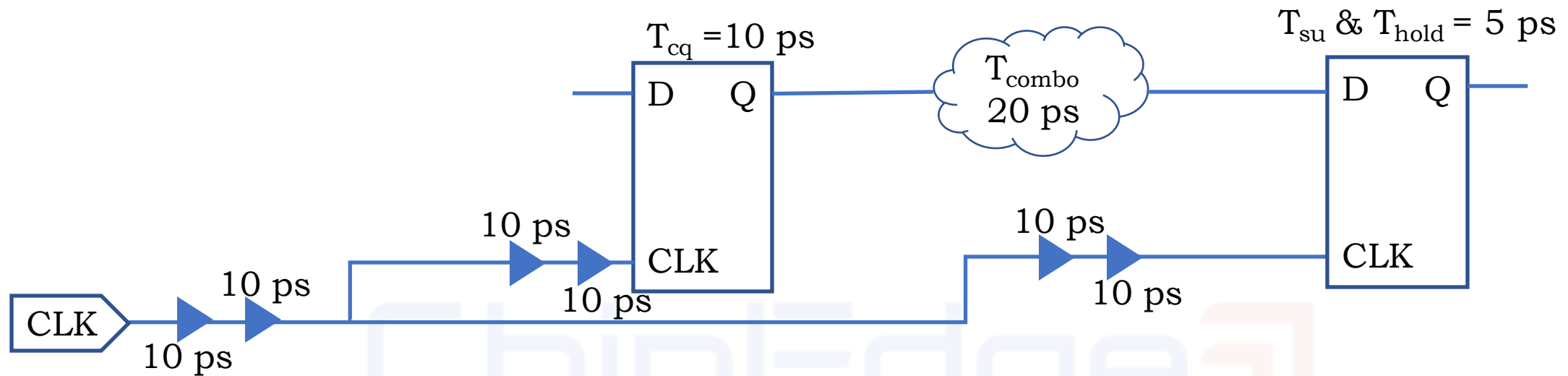
$$\begin{aligned} \text{Hold slack} &= AT - RT \\ &= 70 - 45 \\ &= 25 \text{ ps (positive slack)} \end{aligned}$$

With OCV:-

$$\begin{aligned} AT &= T_{\text{lan}} * \text{Early Derate} + T_{\text{cq}} * \text{Early Derate} \\ &\quad + T_{\text{combo}} * \text{Early Derate} \\ &= 20 * 0.9 + 10 * 0.9 + 20 * 0.9 \\ &= 63 \text{ ps} \end{aligned}$$

$$\begin{aligned} RAT &= T_{\text{cap}} * \text{Late Derate} + t_{\text{hold}} \\ &= 20 * 1.1 + 5 \\ &= 49 \text{ ps} \end{aligned}$$

$$\begin{aligned} \text{Hold slack (OCV)} &= AT - RT \\ &= 63 - 49 \\ &= 14 \text{ ps (positive slack)} \end{aligned}$$



- $T_{clk} = 35 \text{ ps}$
- $T_{cq} = 10 \text{ ps}$
- $T_{su} \& T_{hold} = 5 \text{ ps}$
- Late Derate 1.1
- Early Derate = 0.9

Without OCV:-

$$\begin{aligned}AT &= T_{\text{lan}} + T_{\text{cq}} + T_{\text{combo}} \\&= 40 + 10 + 20 \\&= 70 \text{ ps}\end{aligned}$$

$$\begin{aligned}\text{RAT} &= T_{\text{capture}} + T_{\text{clk}} - t_{\text{su}} \\&= 40 + 35 - 5 \\&= 70 \text{ ps}\end{aligned}$$

$$\begin{aligned}\text{Setup slack} &= \text{RT} - \text{AT} \\&= 70 - 70 \\&= 0 \text{ ps}\end{aligned}$$

With OCV:-

$$\begin{aligned}AT &= T_{\text{lan}} * \text{Late Derate} + T_{\text{cq}} * \text{Late Derate} \\&\quad + T_{\text{combo}} * \text{Late Derate} \\&= 40 * 1.1 + 10 * 1.1 + 20 * 1.1 \\&= 77 \text{ ps}\end{aligned}$$

$$\begin{aligned}\text{RAT} &= T_{\text{cap}} * \text{Early Derate} + T_{\text{clk}} - t_{\text{su}} \\&= 40 * 0.9 + 35 - 5 \\&= 66 \text{ ps}\end{aligned}$$

$$\begin{aligned}\text{Setup slack} &= \text{RT} - \text{AT} \\&= 66 - 77 \\&= -11 \text{ ps (negative slack)}\end{aligned}$$

Without OCV:-

$$\begin{aligned} AT &= T_{lan} + T_{cq} + T_{combo} \\ &= 40 + 10 + 20 \\ &= 70 \text{ ps} \end{aligned}$$

$$\begin{aligned} RAT &= T_{cap} + t_{hold} \\ &= 40 + 5 \\ &= 45 \text{ ps} \end{aligned}$$

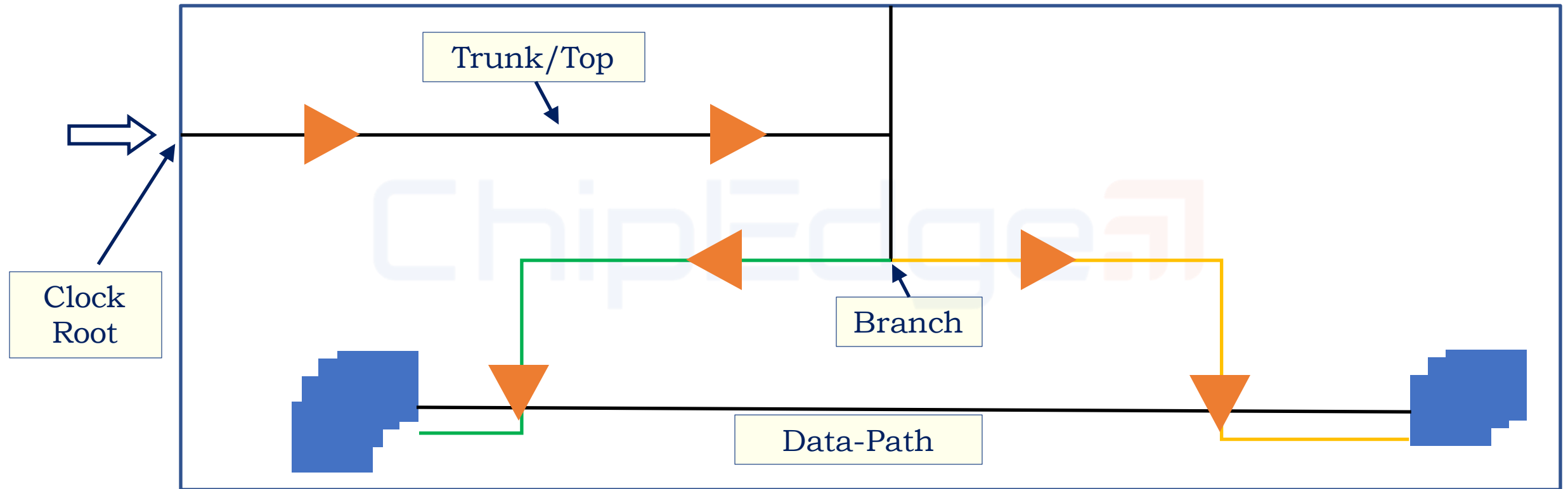
$$\begin{aligned} \text{Hold slack} &= AT - RT \\ &= 70 - 45 \\ &= 25 \text{ ps (positive slack)} \end{aligned}$$

With OCV:-

$$\begin{aligned} AT &= T_{lan} * \text{Early Derate} + T_{cq} * \text{Early Derate} \\ &\quad + T_{combo} * \text{Early Derate} \\ &= 20 * 0.9 + 10 * 0.9 + 20 * 0.9 \\ &= 63 \text{ ps} \end{aligned}$$

$$\begin{aligned} RAT &= T_{cap} * \text{Late Derate} + t_{hold} \\ &= 20 * 1.1 + 5 \\ &= 49 \text{ ps} \end{aligned}$$

$$\begin{aligned} \text{Hold slack (OCV)} &= AT - RT \\ &= 63 - 49 \\ &= 14 \text{ ps (positive slack)} \end{aligned}$$



- Impact of OCV derates on Clock skew

Structural Skew

Launch Latency = 900 ps &

Capture Latency = 1000 ps

Skew = Launch Latency – Capture
Latency

$$= 900 - 1000 = -100 \text{ ps}$$

OCV Skew (assuming 10 % derate
on the divergent path)

Launch latency = 900 ps & Capture
Latency = 1000 ps

Skew = Launch Latency (-10%) –
Capture Latency (+10%)

$$= 810 - 1100 = -290 \text{ ps}$$

- Higher latencies, the impact is more [even if initial skew is same]

Cross Talk Effect On Timing 

OCV Effect On Timing 

CRPR

This section explains about the CRPR effect on setup and hold timing on various paths.

Timing Exception 

GBA vs PBA 

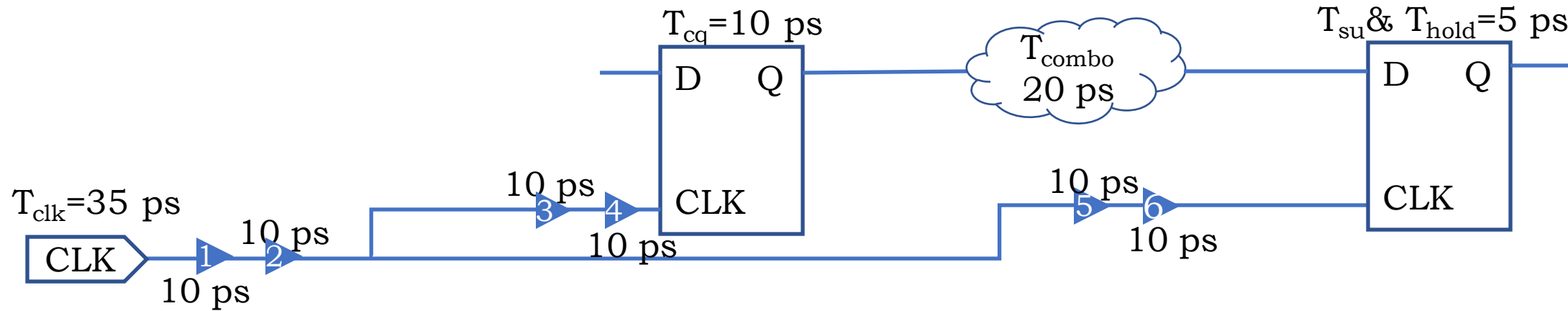
Asynchronous Path 

Lock Up Latch 

Delay Models 

- CRPR Stand for Clock Reconvergence Pessimism Removal.
- Clock cells in the common path cannot have early/late derates at the same time, hence we try to give back the extra pessimism to have an accurate Timing Analysis.
- Same Clock Path may be a Launch Path for one Data Path and can be a Capture Path for another Data Path, while doing OCV derating, same path may get both Min./ Max. delay.

- But a path can have either as a Maximum delay or a Minimum delay (or anything in between) but never both delays at the same time.
- STA tools will have techniques to remove artificially introduced pessimism between the Launch Clock Path and the Capture Clock Path.
- $CRPR = \text{Max common path delay} - \text{min common path delay}$.



Cell Name	Max Delay	Min Delay
Buf1	11	9
Buf2	11	9
Buf3	11	9
Buf4	11	9
Buf5	11	9
Buf6	11	9
Tcomb	22	18
Tcq	11	9

Common path max delay = $11 + 11 = 22$ ps

Common path Min delay = $9 + 9 = 18$ ps

CRPR = $22 - 18 = 4$ ps

Setup slack:-

- Arrival Time = $(T_{lan} + T_{cq} + T_{comb}) = (11 + 11 + 11 + 11) + 10 + 20 = 101 \text{ ps}$
- Required Arrival Time = $(T_{cap} + T_{clk} - t_{su}) = (9 + 9 + 9 + 9) + 35 - 5 = 66 \text{ ps}$
- Setup slack = $RAT - AT = 66 \text{ ps} - 101 \text{ ps} = -35 \text{ ps}$

Hold slack:-

- Arrival time = $(T_{lan} + T_{cq} + T_{comb}) = (9 + 9 + 9 + 9 + 9) + 18 = 63 \text{ ps}$
- Required Arrival time = $(T_{cap} + t_{hold}) = (11 + 11 + 11 + 11) + 5 = 49 \text{ ps}$
- Hold time = $AT - RAT = 63 \text{ ps} - 49 \text{ ps} = 14 \text{ ps}$

Setup slack:-

- Arrival Time = $(T_{lan} + T_{cq} + T_{comb}) = (11 + 11 + 11 + 11) + 10 * 1.1 + 20 * 1.1 = 77 \text{ ps}$
- Required Arrival Time = $(T_{cap} + T_{clk} - t_{su} + \text{CRPR}) = (9 + 9 + 9 + 9) + 35 - 5 + 4 = 70 \text{ ps}$
- Setup slack = $RAT - AT = 70 \text{ ps} - 77 \text{ ps} = -7 \text{ ps}$

Hold slack:-

- Arrival time = $(T_{lan} + T_{cq} + T_{comb}) = (9 + 9 + 9 + 9 + 9) + 18 = 63 \text{ ps}$
- Required Arrival time = $(T_{cap} + t_{hold} - \text{CRPR}) = (11 + 11 + 11 + 11) + 5 - 4 = 45 \text{ ps}$
- Hold time = $AT - RAT = 63 \text{ ps} - 45 \text{ ps} = 18 \text{ ps}$

	Latency	Without OCV			With OCV		
		Setup	Hold	Skew	Setup	Hold	Skew
Low Latency	20	0	25	0	-7	18	-4
High Latency	40	0	25	0	-11	14	-8
High Latency with Common clock path(CRPR)	40	0	25	0	-7	18	-8

- OCV effect is more if the clock latency is high.
i.e setup and hold slacks degrade more for high latency paths due to OCV.
- To be more specific ocv effect is more if non common clock path latency is high.

The CTS goal should be:

1. Achieve low Clock latency
2. Minimize non common clock path

Cross Talk Effect On Timing 

OCV Effect On Timing 

CRPR 

Timing Exception

This section explains about various timing exceptions which include Multi Cycle Path, False Path, Half cycle paths etc..

GBA vs PBA 

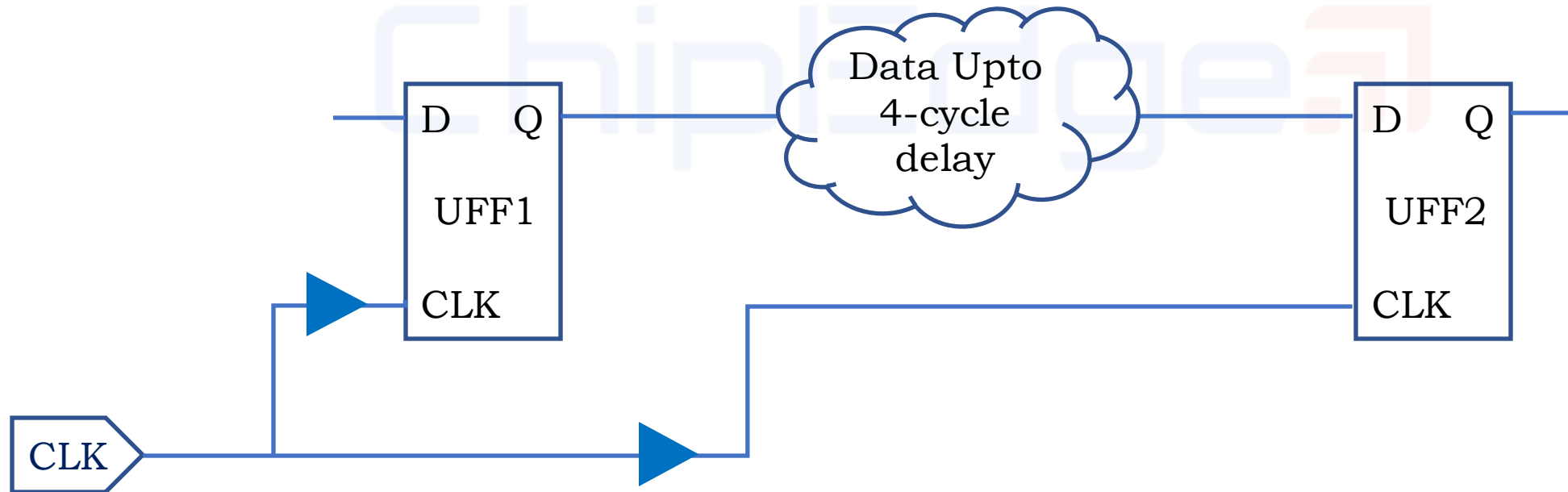
Asynchronous Path 

Lock Up Latch 

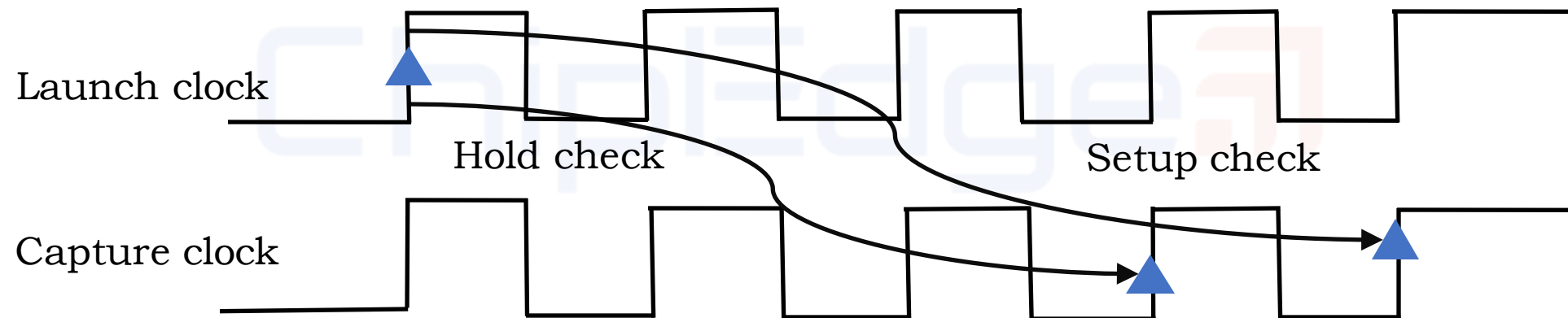
Delay Models 

- By default, STA tool assumes that data launched at a path startpoint is captured at the path endpoint by the very next occurrence of a clock edge at the endpoint.
- If certain paths are not intended by design to operate according to the default setup and hold behavior assumed by the STA tool, you need to specify those paths as timing exceptions. Otherwise, the tool might incorrectly report those paths as having timing violations.
- The STA tools lets you specify the following types of exceptions:
 - ✓ False path – A path that is never sensitized due to the logic configuration, expected data sequence, or operating mode.
 - ✓ Multicycle path – A path designed to take more than one clock cycle from launch to capture.

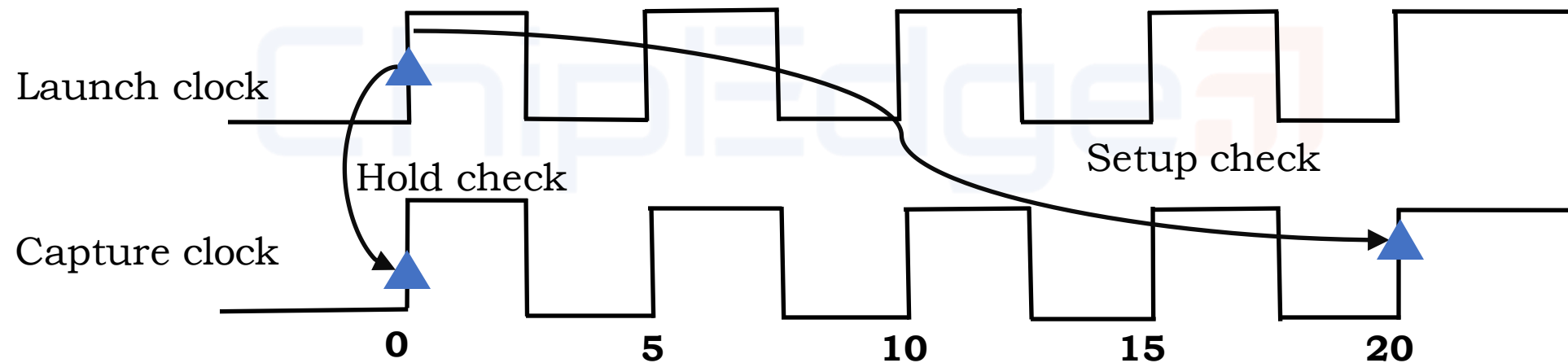
The combinational data path between two flipflops can take more than one clock cycle to propagate through the logic. In such cases, the combinational path is declared as Multicycle path.



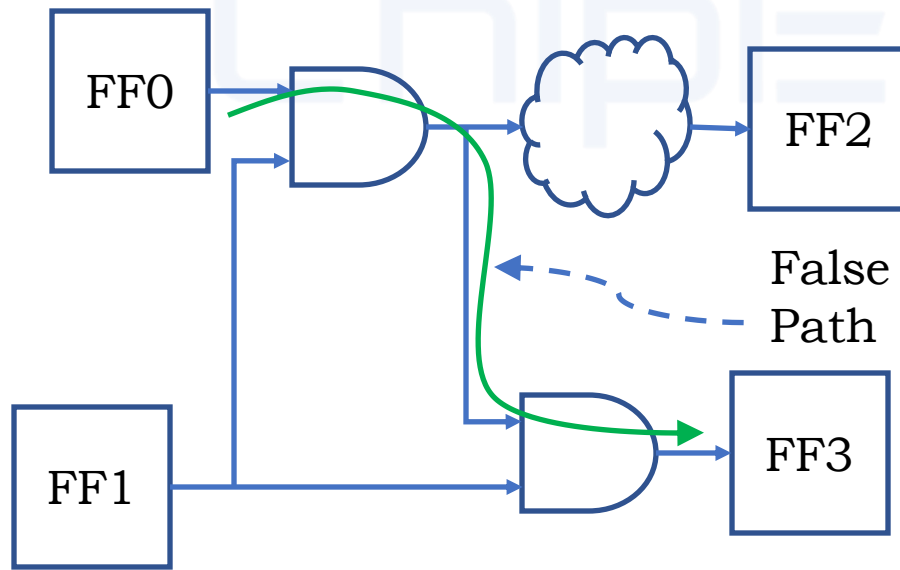
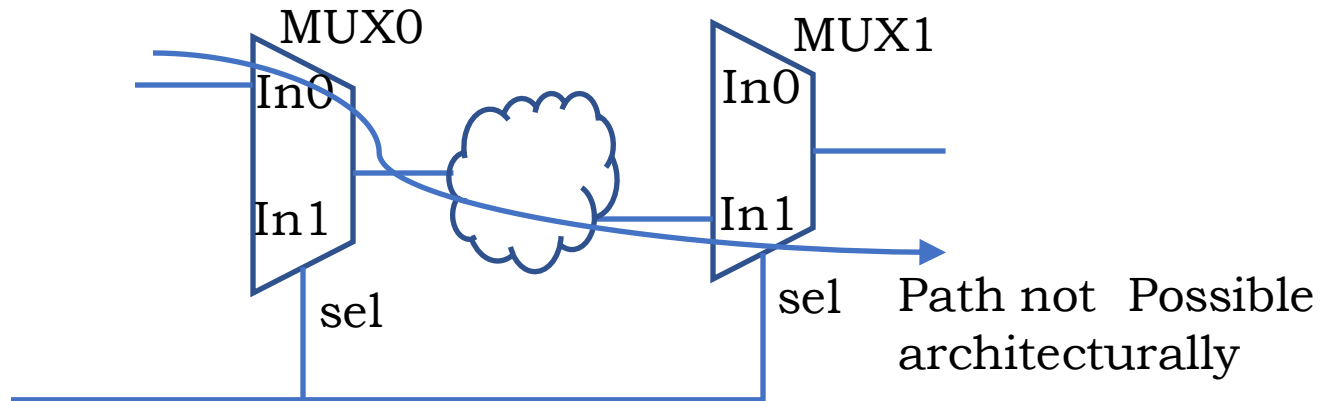
- `set_multicycle_path 4 -setup -from [get_cells UFF1] -to [get_cells UFF2]`



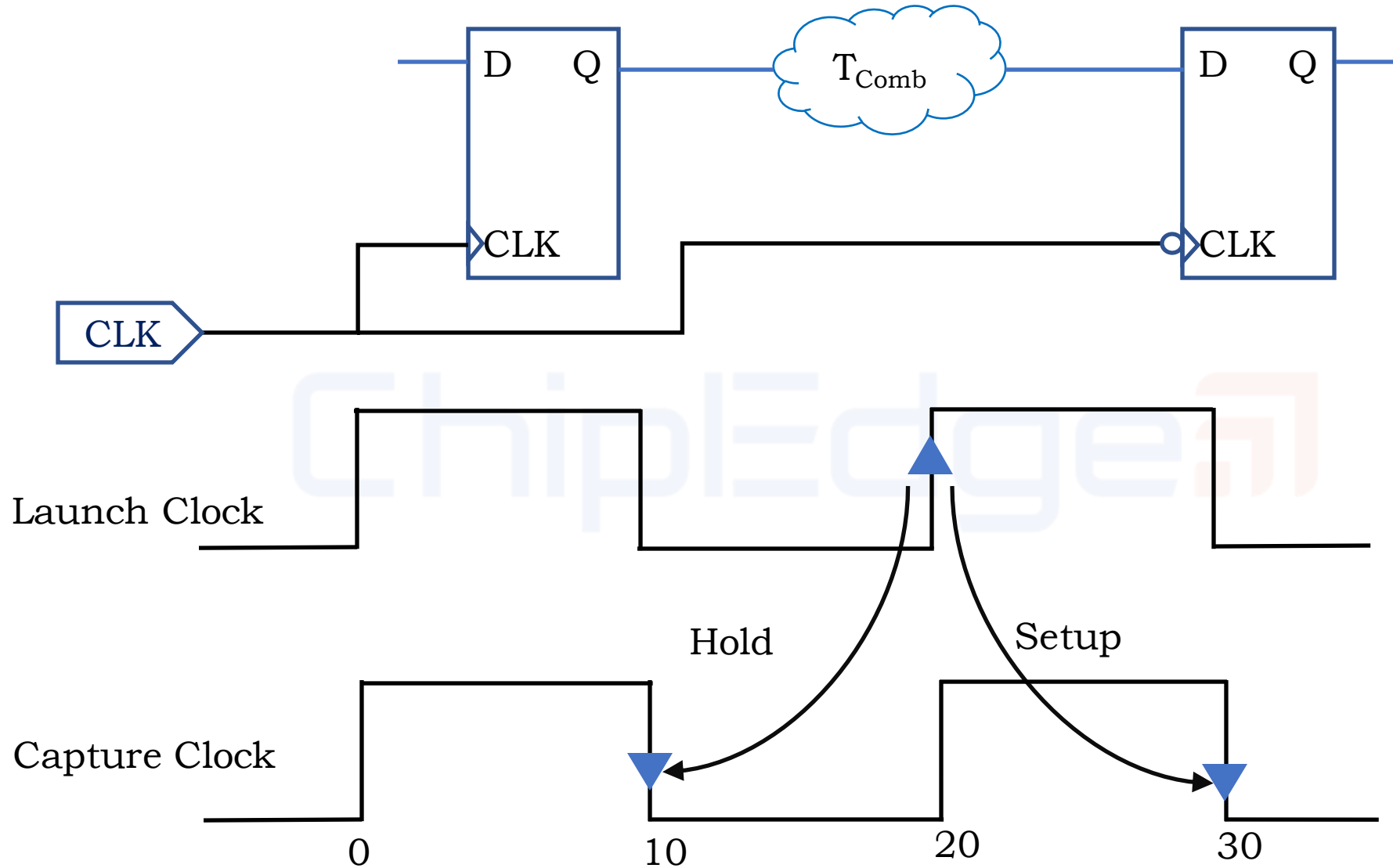
- `set_multicycle_path 3 -hold -from [get_cells UFF1] -to [get_cells UFF2]`



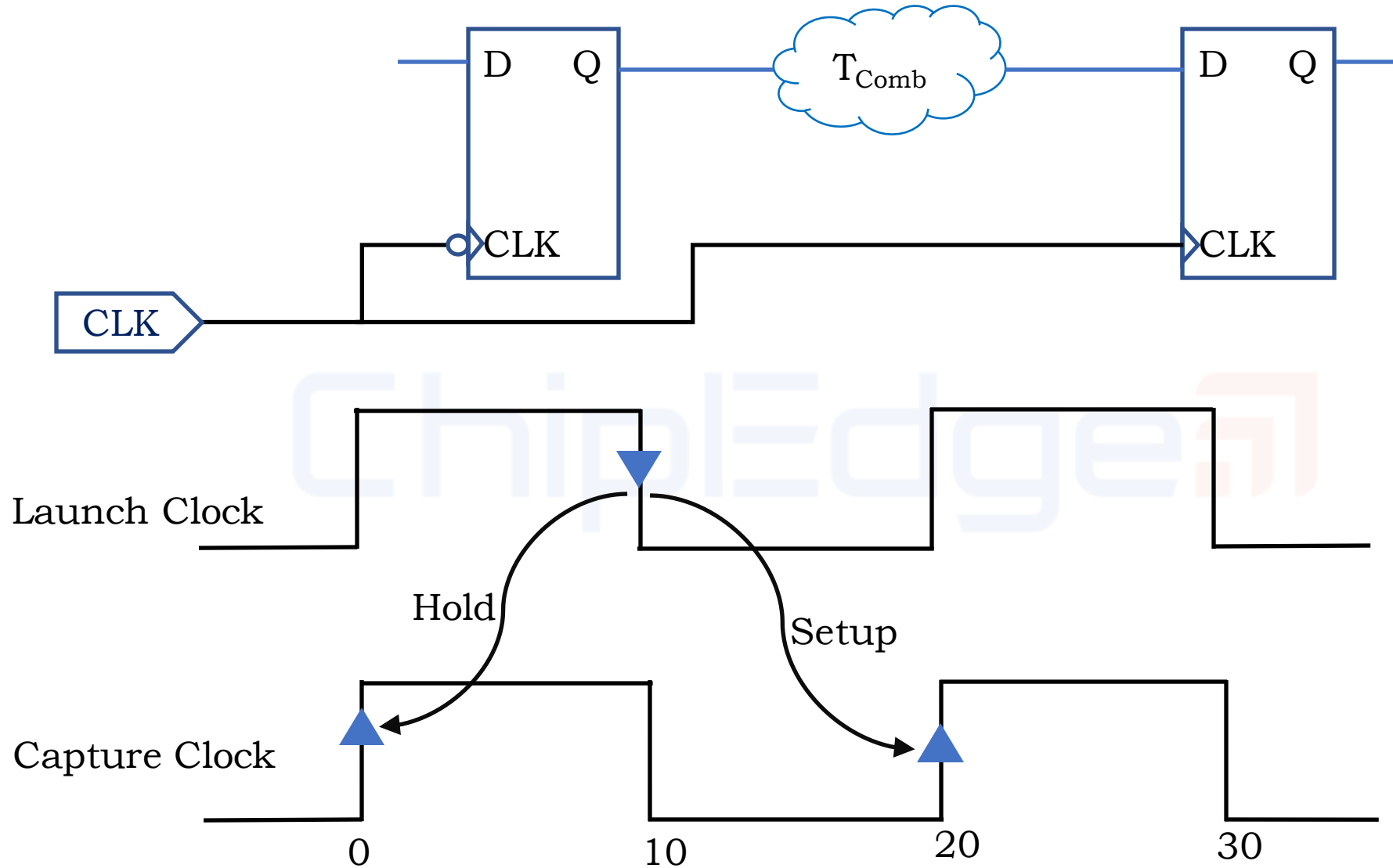
- False Paths are those timing paths in design where changes in source registers are not expected to get captured by the destination register within a particular time interval.
- Physically exist in the design but are Logically/ Functionally inactive/incorrect path.
- Means no valid data is transferred from Start Point to End Point.
- The goal in STA is to do timing analysis on all “true” timing paths, so these paths are excluded from timing analysis.

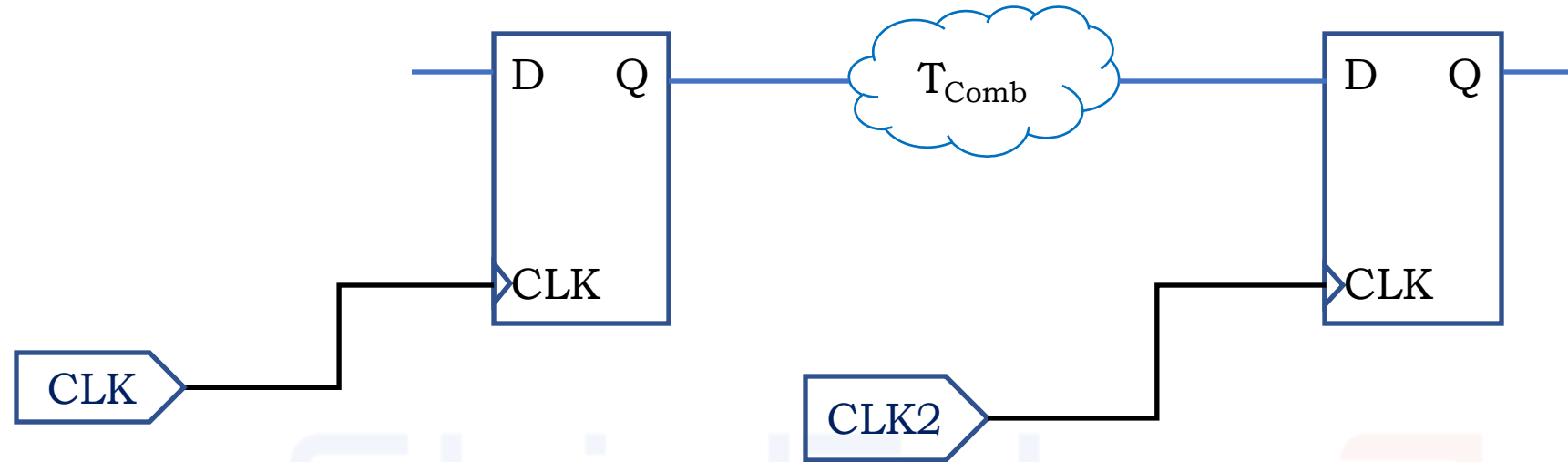


Half Cycle Path-Rise to Fall path

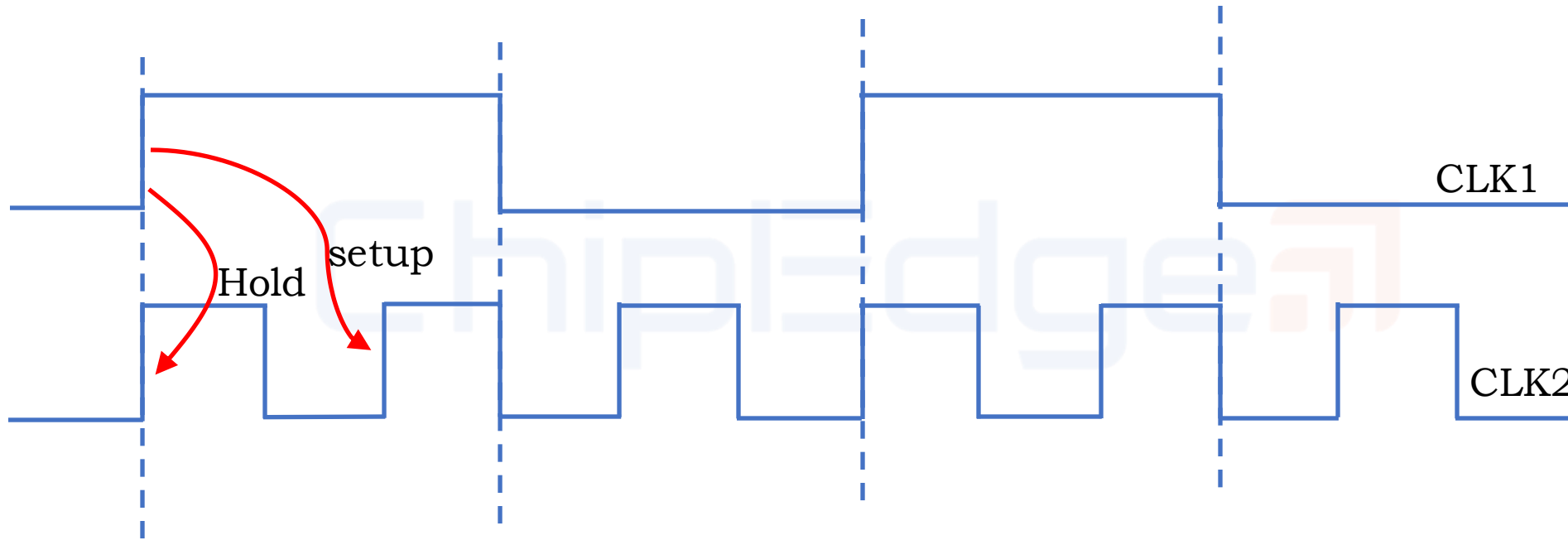


Half Cycle Path-Fall to Rise path





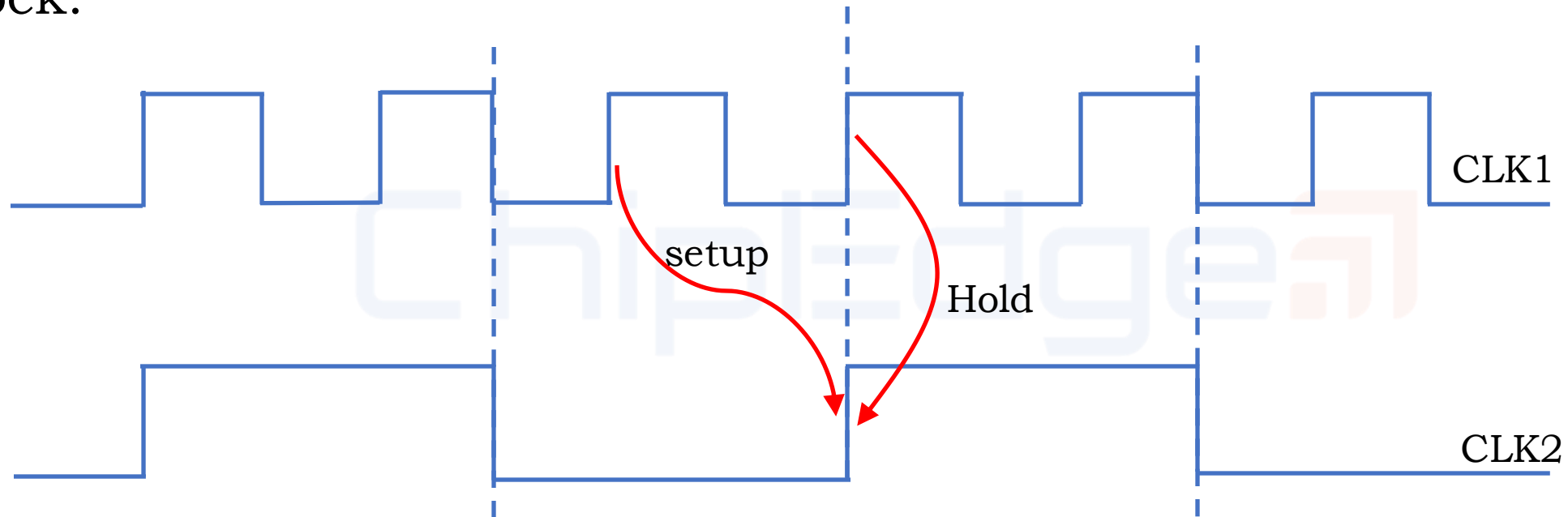
CLK1(Launch Clock) is slow clock and CLK2(Capture Clock) is fast clock.



Setup Check: Next positive edge of Capture clock

Hold Check : Same edge of capture clock

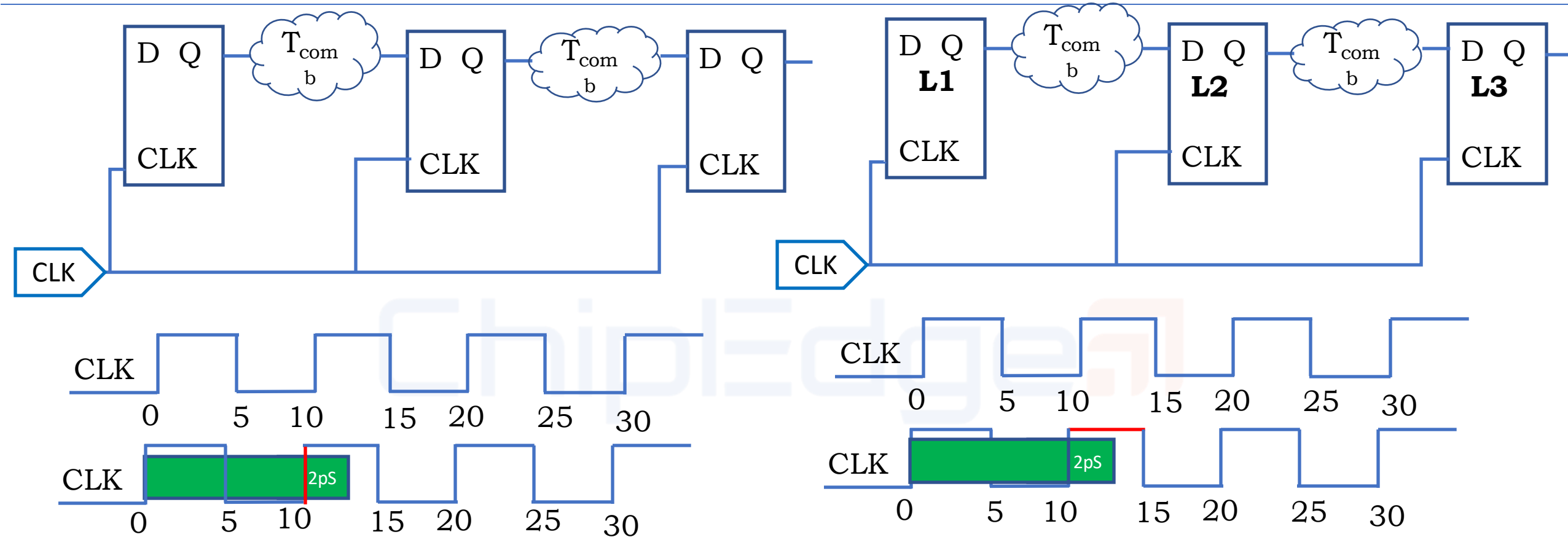
CLK1(Launch Clock) is fast clock and the CLK2(Capture Clock) is slow clock.



- One of the major differences between edge-triggered flip-flops and latches is that the former change states at the clock edges, whereas the latter change states if the clock pin is enabled.
- As a result, the delay of a combinational logic path of a design using edge-triggered flip-flops cannot be longer than the clock period except for those specified as false paths and multiple-cycle paths.

- The performance of a circuit is therefore limited by the longest path of a design. Latch based designs allows the use of a longer combinational path as long as it can be compensated by shorter path delays in the subsequent logic stages.
- The technique of borrowing time from the shorter paths of the subsequent logic stages is called time borrowing or cycle stealing.
- Latch-based designs typically use two-phase, non-overlapping clocks to control successive registers in a data path.

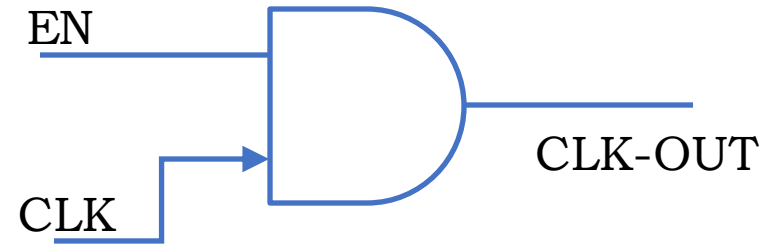
STA with Latch: Time Borrowing



Setup violation occur because path delay is more than clock period and here, we are using flip flop.

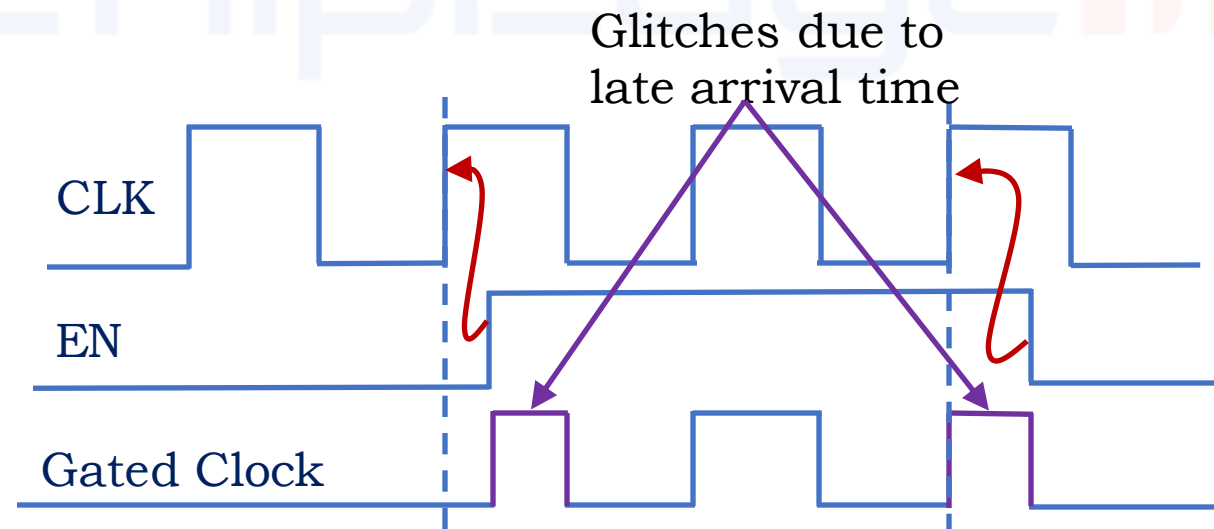
Setup violation will not occur because as the latch is level sensitive the data will be captured properly even if it arrives at 12 ps (Infact the data can arrive upto 15 ps).

- The latch has a property, where a path ending at a latch can borrow time from the next path in the pipeline such that the overall time of two paths remains the same.
- STA uses a time borrowing concept for latch-based designs.
- Any data launched from Flip Flop1 at 0 ns should reach Flip Flop 2 at the following active edge, which is 10 ns.
- If data reaches at flipflop 2 after 10ns, the correct data cannot be captured.
- In case of latches even if the data arrives after 10 ps it will be captured. The only condition is the data should arrive before the next falling edge i.e 15ps.

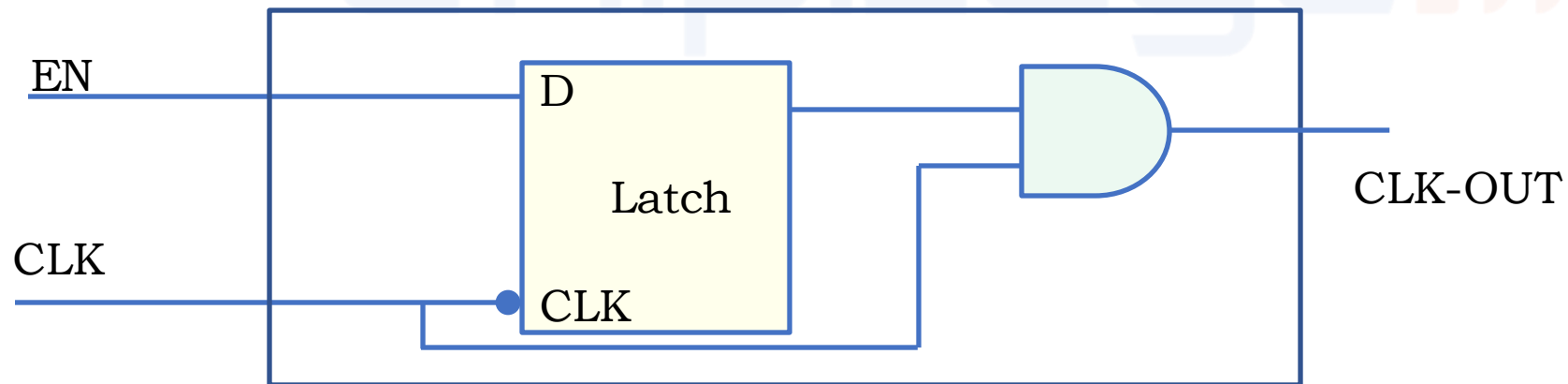


Basic Clock Gating Circuit

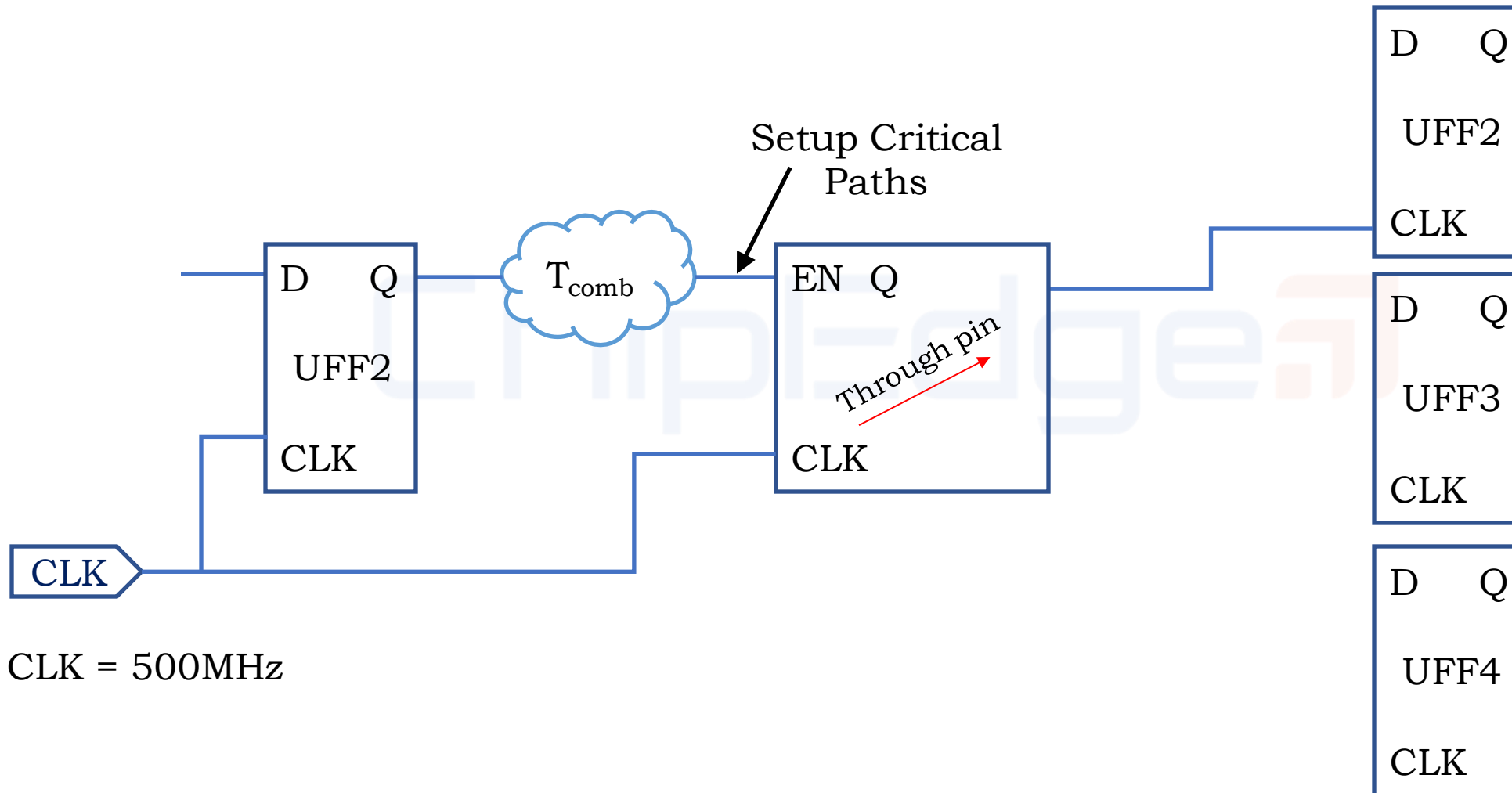
- This circuit might result in a glitch on clock signal:-



- Clock gating checks are used to make sure this kind of glitch does not occur. Part of STA Timing analysis.
- Basically, we need to make sure for AND type clock gating, “EN” must toggle only when clk is in “LOW” state i.e., clk is in controlling state.
- This is achieved by creating an ICG (Integrated Clock Gating cell).



ICG(Integrated Clock Gating) cell



Cross Talk Effect On Timing 

OCV Effect On Timing 

CRPR 

Timing Exception 

GBA vs PBA

This section explains about GBA VS PBA.

Asynchronous Path 

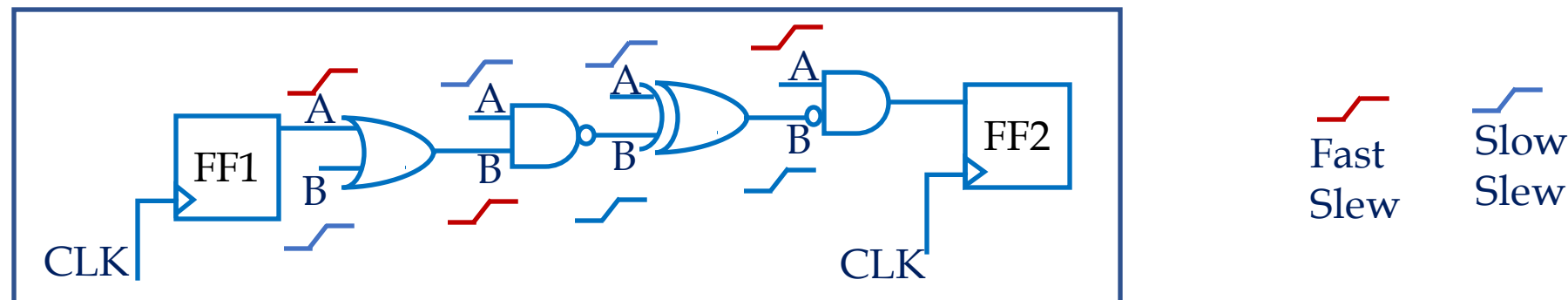
Lock Up Latch 

Delay Models 

- Static timing analysis tools are designed to be pessimistic to ensure detection of all timing violations.
- Graph based Analysis (GBA) for STA is a pessimistic approach but is quicker and is less memory intensive.
- GBA : In GBA, tool considers the worst of the input slews of a cell while calculating its delay and to derive the output slew.
- PBA : In PBA, tool considers the actual slew that appears at the input of a cell that is part of the path and ignores slews from any side paths. This approach removes pessimism but is time consuming and more memory intensive.

GBA : In adjacent path between FF1 and FF2, the tool always considers the worst slew (slow slew for setup analysis, fast for hold) even if that slew does not appear on the inputs part of the timing path in question. No actual slew calculations are done by the tool.

PBA : In adjacent path between FF1 and FF2, the tool recalculates the delays and propagation slews, and it considers the slews at inputs only part of the timing path in question and ignores any slews from side paths.



Path Based Analysis

- Enabled on demand.
- Time consuming method for analysis.
- Accurate approach to analysis.
- For each multi-input gate, the slew of the pin through which the timing path travels is considered for delay through the gate.

Cross Talk Effect On Timing 

OCV Effect On Timing 

CRPR 

Timing Exception 

GBA vs PBA 

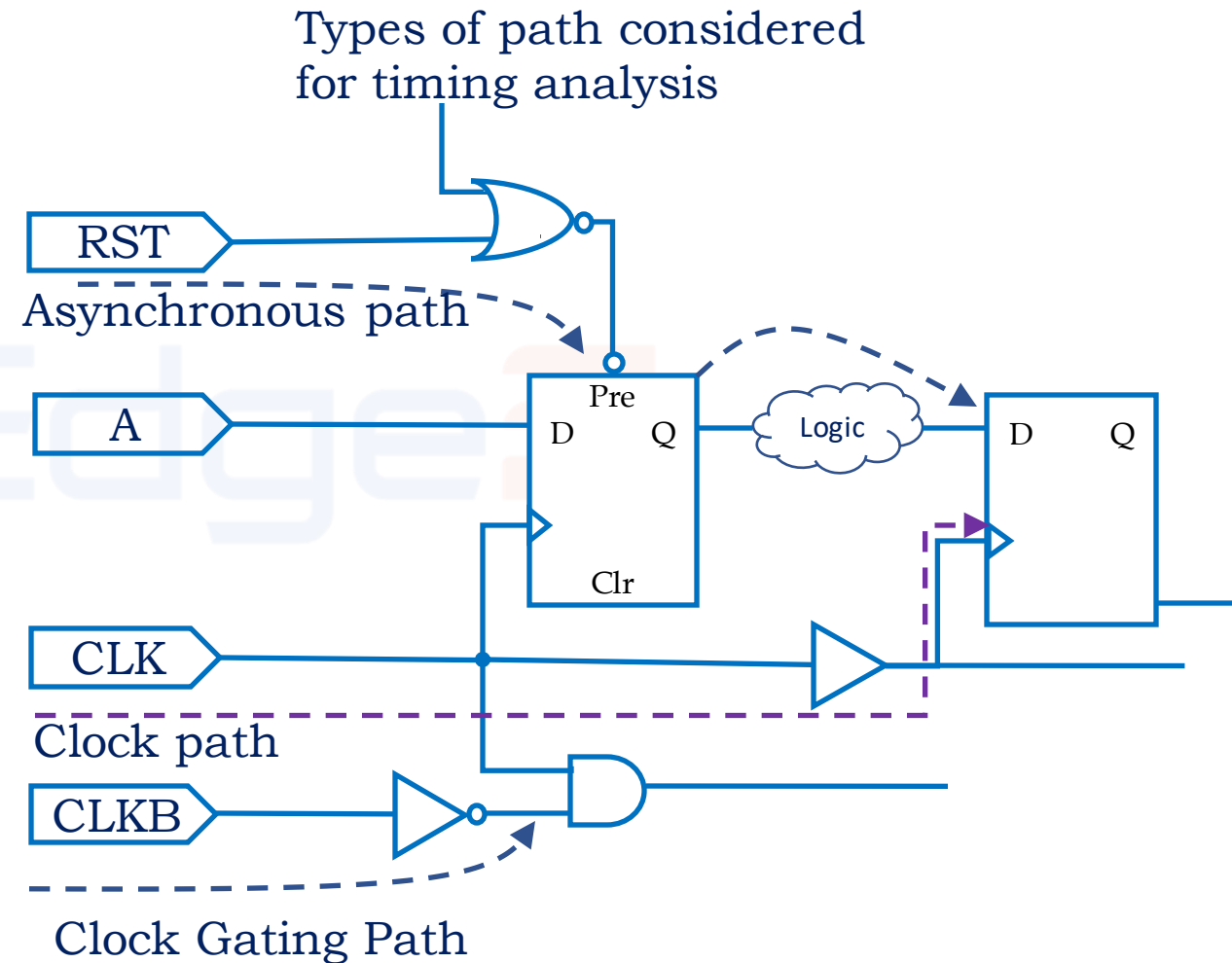
Asynchronous Path

This section explains about Asynchronous path.

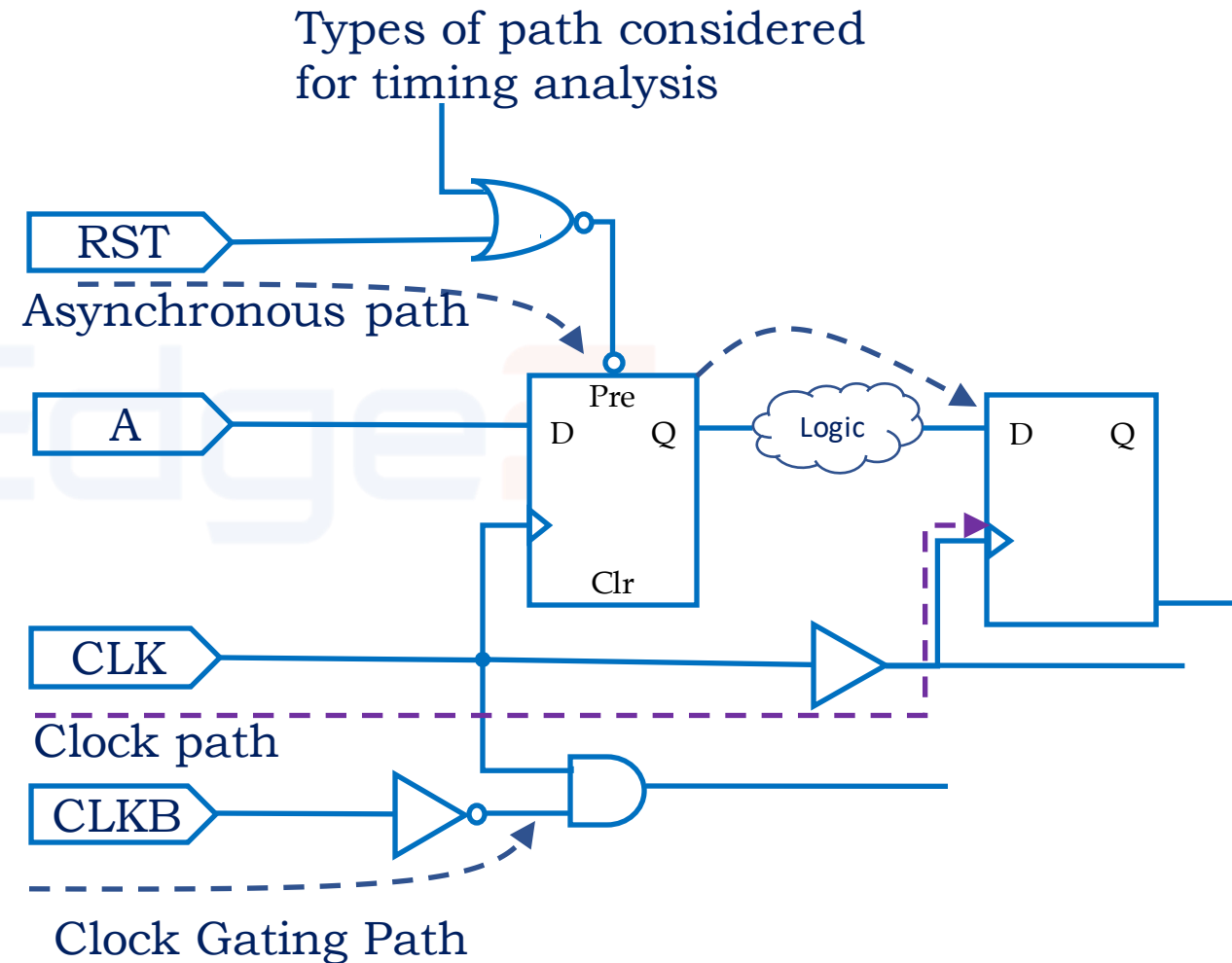
Lock Up Latch 

Delay Models 

- A path from an input port to an asynchronous set or reset pin of a sequential element. See the following figure for understanding clearly.
- As you know that the functionality of set/reset pin is independent from the clock edge. It is level triggered pins and can start functioning at any time of data.



- So in other way we can say that this path is not in synchronous with the rest of the circuit and that's the reason we are saying such type of path an Asynchronous path.



Cross Talk Effect On Timing 

OCV Effect On Timing 

CRPR 

Timing Exception 

GBA vs PBA 

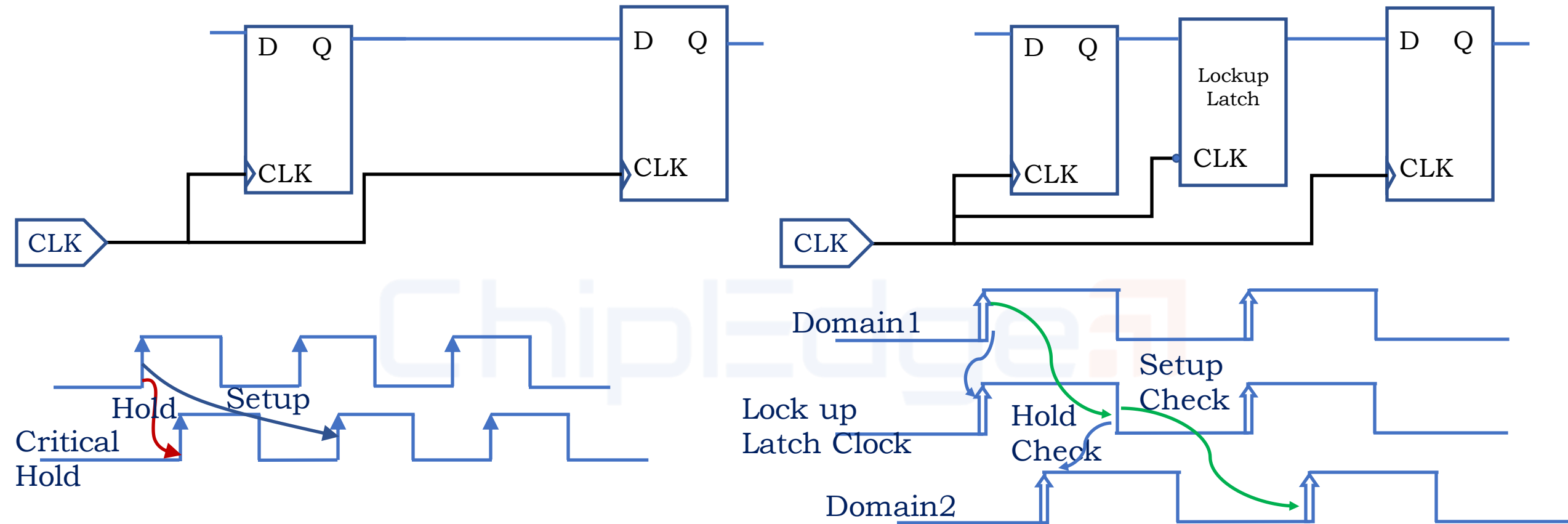
Asynchronous Path 

Lock Up Latch

This section explains about Lock Up Latch

Delay Models





- Lock-up latches are used to meet hold timing without need to add too many buffers in situations where large skew is involved.

- Lock-up latches are necessary to avoid hold violation due to large skew during shift phase of scan-based testing.
- Lock up Latches are also used in places with significant clock skew and meeting hold timing is a challenge due to large uncommon clock path.
- Where launch and capture flops are positive edge triggered, we use negative sensitive latch as our lock-up latch.
- The latch is to be connected to the launch domain clock and balanced with the register in that domain.

Cross Talk Effect On Timing 

OCV Effect On Timing 

CRPR 

Timing Exception 

GBA vs PBA 

Asynchronous Path 

Lock Up Latch 

Delay Models

This section explains about Various Cell Delay models and Interconnect Delay models

- There are two type of delay models
 1. Cell Delay models
 2. Interconnect Delay Models



Various Cell Delay Models –

- Non-Linear Delay Model (NLDM)
- Composite Current Source (CCS) Delay Model



Non-Linear Delay Model	Composite current source Delay model
NLDM driver model characterizes input-to-output delay and output transition times with sensitivity to input transition time, output load and side input states.	CCS driver model is characterized by capturing current waveform flowing into the load capacitor of the cell.
NLDM is less accurate	CCS is more accurate
Run time required is less for NLDM	Needs more Run time for CCS
NLDM doesn't address the effects of Deep sub-micron process and it is less accurate for deep sub-micron process.	CCS address the effects of deep sub-micron processes. It is more accurate for lower technology nodes.
NLDM libraries are smaller in size compared to CCS	CCS libraries are huge in size

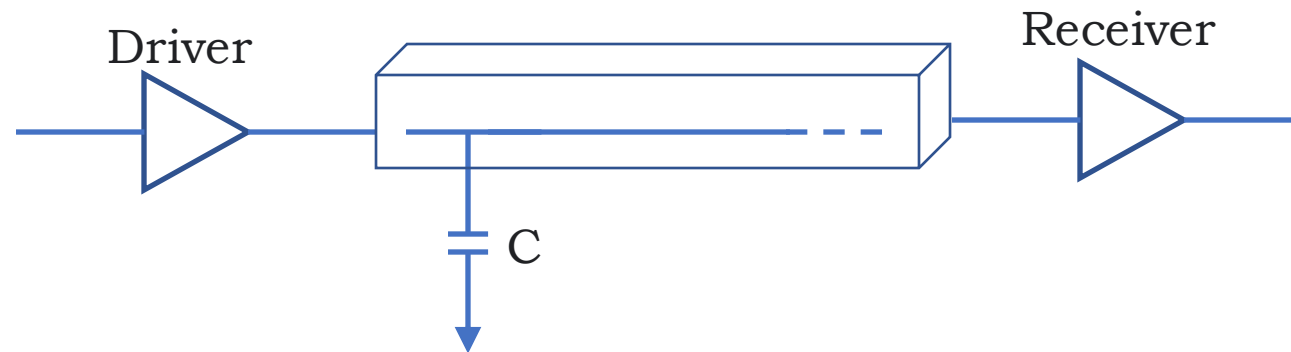
- Wire Load Model
- Lumped Capacitor Model
- Lumped RC Model
- Distributed RC Model
- Pi & T RC Network
- Elmore Delay Model

- Wire load model is statistical based model used to estimate net delay in the pre-layout phase of the design and it varies for different technologies.
- Information about wire load models will be present in the liberty files (.libs). Resistance, Capacitance are taken per unit length of Wire.
- A set of tables is obtained between Net Fan-out vs Area, Net Fan-out vs Resistance and Net Fan-out vs Load shown in below table.

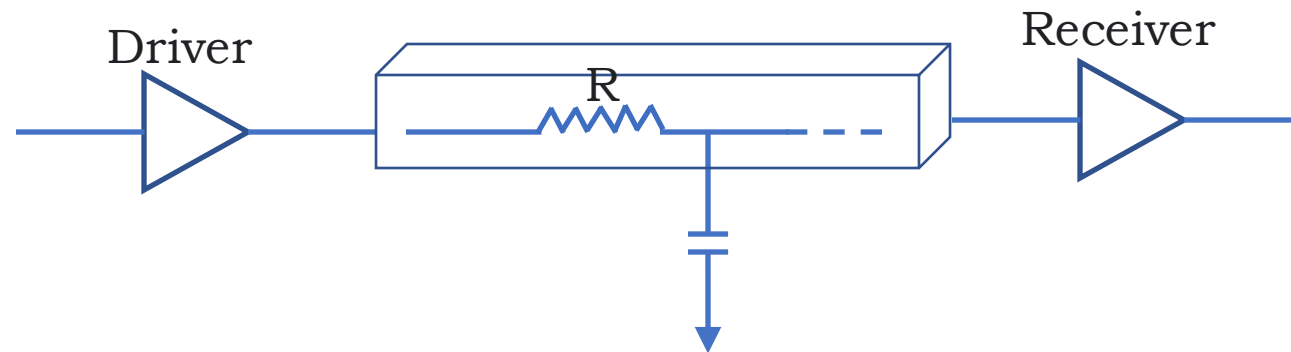
Number of Fanout	Resistance (K_{ohm})	Capacitance(pF)
1	0.00587	0.00434
2	0.0978	0.00624

- Before going for floor planning or layout, wire load models (WLM) can be used to calculate interconnect wiring delays (capacitance (C), resistance (R)), and the area overhead (A) due to interconnect.
- The chip area of the block is typically taken into account when choosing a wire load model. However, the designers can alter or adapt these WLM models in accordance with the needs of the user.

- In Higher Technologies (above 350 nm) Metal Wires were wide and have more cross-sectional area. So, the Resistance factor in higher technologies was considered negligible. Only Capacitance was considered because it was dominating factor in Higher Technologies. In Lumped Capacitor Model, Source Driver considers a single Loading Capacitance that is the sum of Interconnect Capacitance and Load Capacitance at Sink Point.



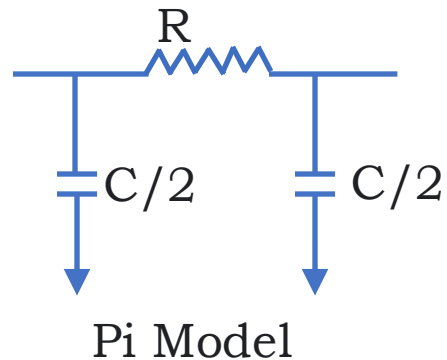
- The reduction in nm Technology causes reduction in the width of Metal Wires Resistance which can no longer be ignored. So it is necessary to model resistance as well.
- Therefore, Lumped RC Model is used in this situation. For each segment of wire, total resistance is combined with total capacitance to form a single resistor in a lumped RC model.



- The distributed RC delay can be modeled by breaking up a wire into one or more segments and using a lumped model for each segment.
- R and C is distributed along the wire.
- Lumped RC model is always pessimistic and Distributed RC model provide better accuracy over lumped model.
- Two types of distributed RC model are
 - 1) Pi Model
 - 2) T Model

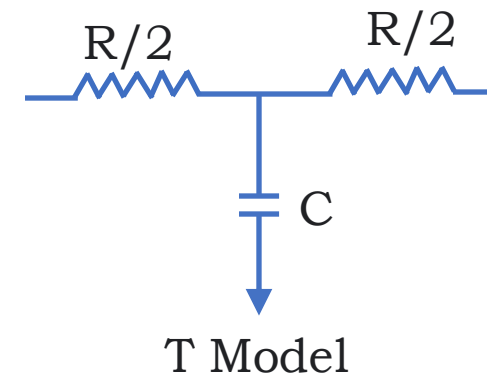
Pi Model :

Total Resistance is present in between the Capacitance and total Capacitance is divided into two parts and connected on either side of Resistance.

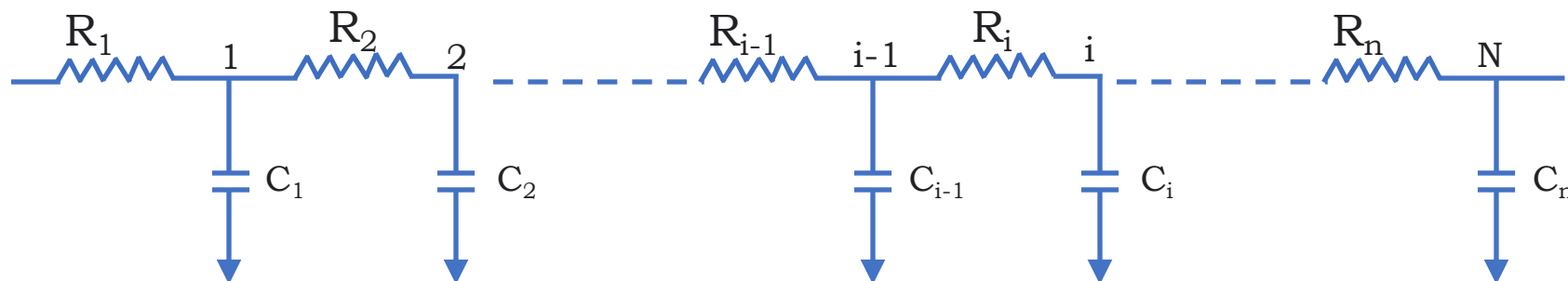


T Model :

Total Resistance is divided into two parts and total Capacitance is considered at the mid-point of the two Resistances.



- The Elmore delay model estimates the delay from a root to one of the leaf nodes as the sum of the resistance in the path to the I_{th} node multiplied by the capacitance present at the end of the branch.
- $T_{d1} = C_1 * R_1$
- $T_{d2} = C_2 * R_2 + C_2 * (R_1 + R_2)$
- $T_{dn} = \sum(i=1, N) C_i (\sum(j=1, i) R_j)$



- Cross Talk Effect On Timing
- Cross Talk On Data Path
- Cross Talk On Launch Clock Path
- Cross Talk On Capture Clock Path
- Cross Talk On Common Clock Path
- OCV Effect On Timing
- OCV On Data Path
- OCV On Launch Clock Path
- OCV On Capture Clock Path
- OCV On Common Clock Path
- CRPR

- Timing Exception
- Multi Cycle Path
- False Path
- Half Cycle Path
- Multiple Clock Domain
- STA with Latch: Time Borrowing
- Clock Gating Checks
- GBA vs PBA
- Asynchronous Path
- Lock Up Latch
- Delay Models
- Cell Delay Models
- Interconnect Delay Models

Module 17

Introduction to Low Power Design

- In this module, you will learn about the overview of low power design.
- Brief overview of various low power methods, low power cells and UPF file are explained in this module.



- Overview
 - ✓ Multi V_{dd}
- Power Component
 - ✓ Multi V_t
 - ✓ Dynamic Power
 - ✓ Leakage Power
 - ✓ Power Gating
- Low Power Methods
 - Low Power cells
 - ✓ Level Shifters
 - ✓ Retention Flops
 - ✓ Clock Gating
 - ✓ Gate level Power Optimization
 - UPF

Overview

This section explains about the overview of low power.

Power Component



Low Power Methods



Low Power cells



UPF



- Technology Scaling, advent of sophisticated processes lead to integration of multi-million of gates/transistors over small die area. It causes increase in power density & total power dissipation.
- Aggressive technology scaling causing exponential increase in leakage current (e.g., at 45nm leakage current is as high as dynamic current)
- Increased power density and power dissipation are critical issues with battery-powered systems, because the overall amount of available power is limited increased power will lead to a lower time of operation and the need for repeated charging.

- The power dissipated by high-performance microprocessors used in server's results in significant heat dissipation, high power consumption, and high cooling solution expenses.



Node	90nm	65nm	45nm
Dynamic Power per cm^2	1X	1.4X	2X
Static Power per cm^2	1X	2.5X	6.5X
Total Power per cm^2	1X	2X	4X

Overview



Power Component

This section explains about dynamic power and leakage power.

Low Power Methods



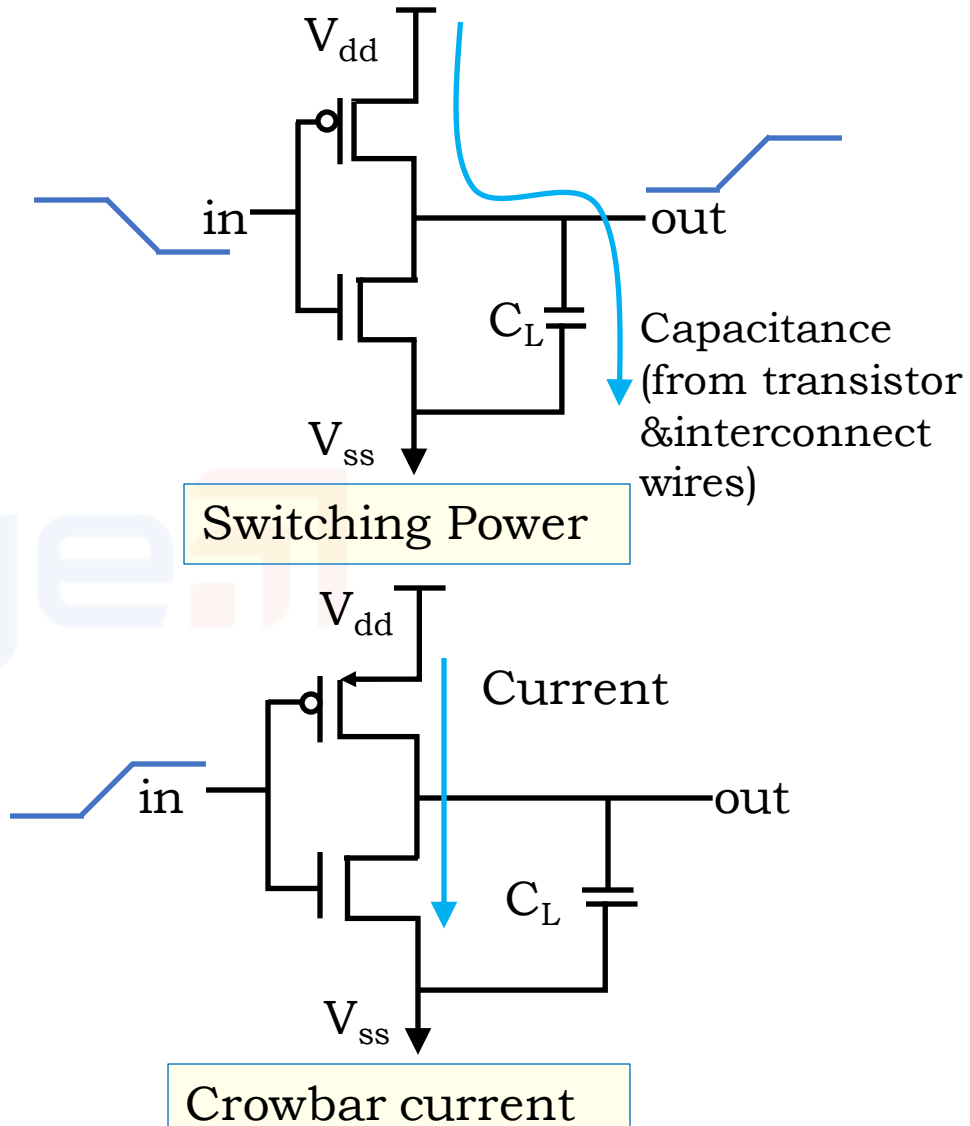
Low Power cells



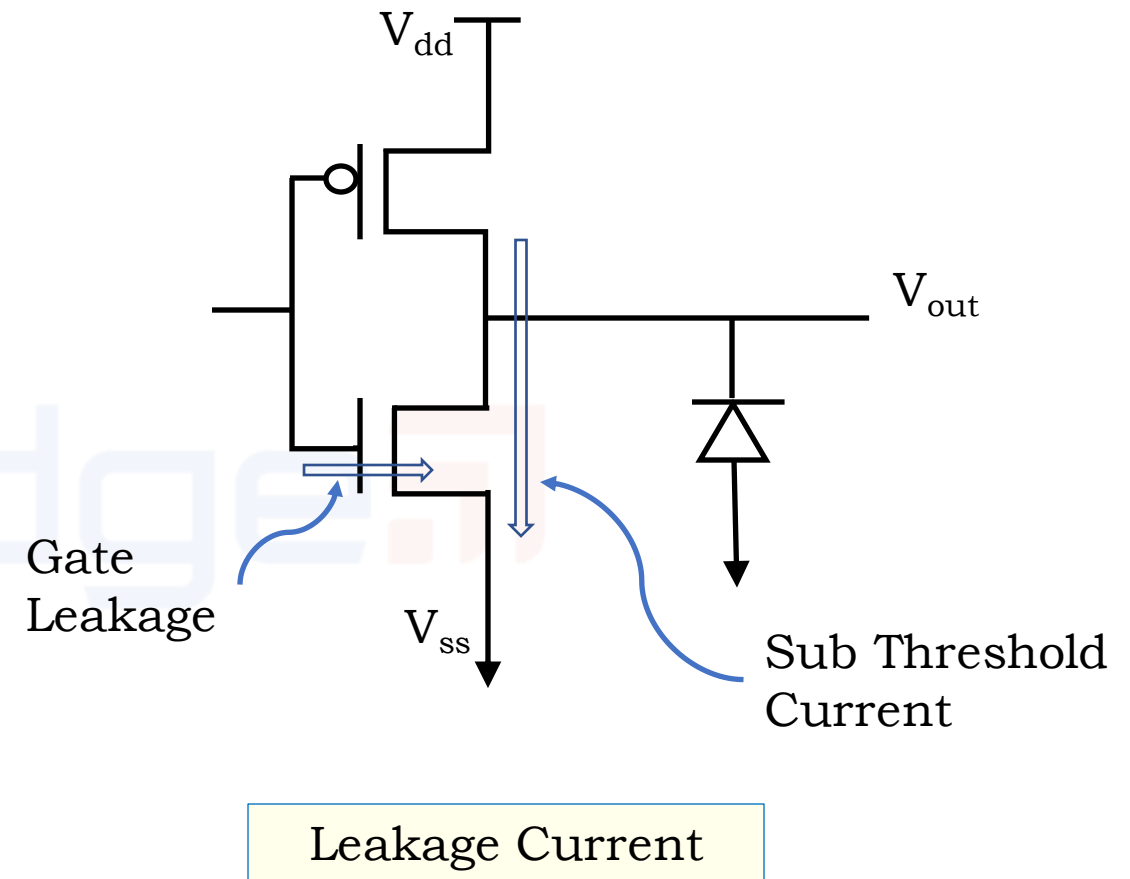
UPF



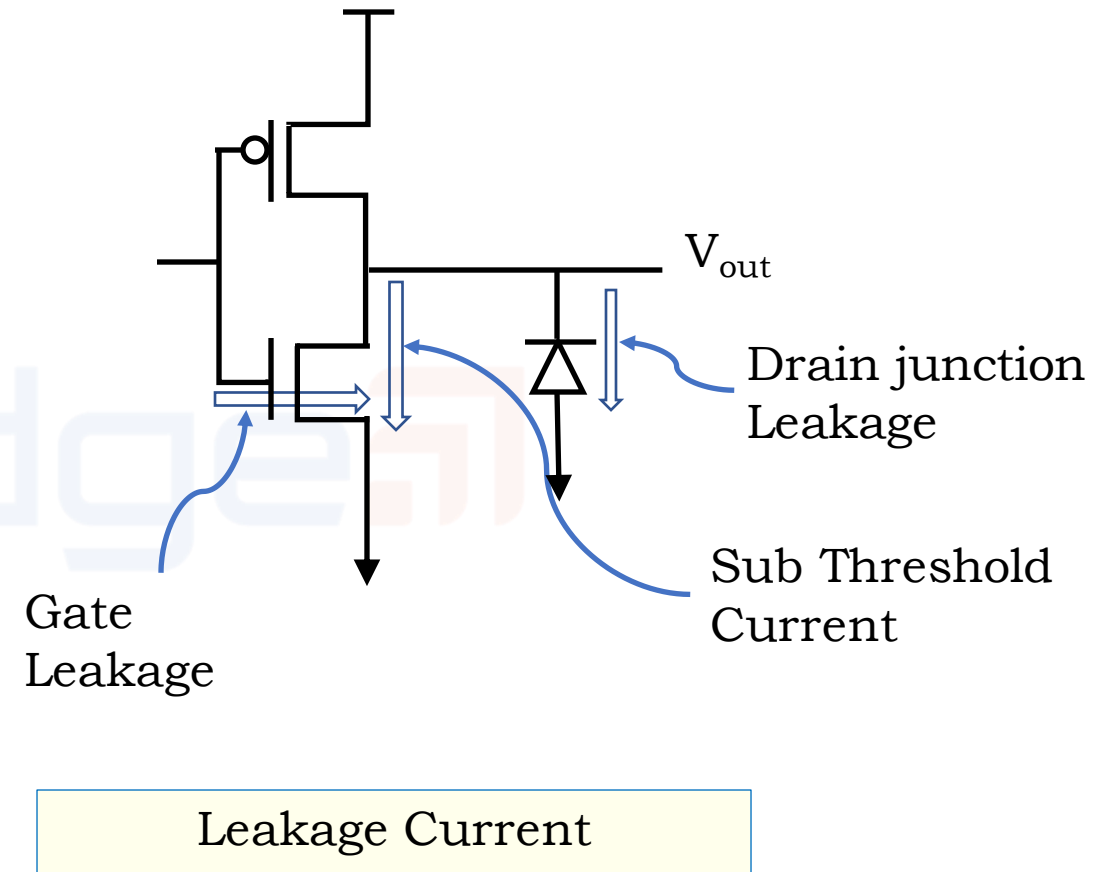
- Power consumed when the device is active i.e switching power
- $P_{\text{dyn}} = C_{\text{eff}} * V_{\text{dd}}^2 * f_{\text{clock}}$
- Internal power (crowbar current) is also a component of dynamic power, which occurs when both NMOS & PMOS transistors are ON, as well as current required to charge internal capacitor



- Power consumed during in Steady state of circuits i.e leakage power.
- There are Four source :-
 - ✓ Subthreshold leakage-current which flows drain to source
 - ✓ Gate leakage-current that flows directly from gate through oxide tunneling and hot carrier effect.



- ✓ Gate induced drain leakage current-Current which flows from the drain to substrate induced by high field effect in the MOSFET caused by high V_{dg} .
- ✓ Reverse bias junction leakage-caused by minority carrier drift and generation of electron/hole pairs in depletion region.



Overview



Power Component



Low Power Methods

This section explains about various low power methods used in the design.

Low Power cells



UPF

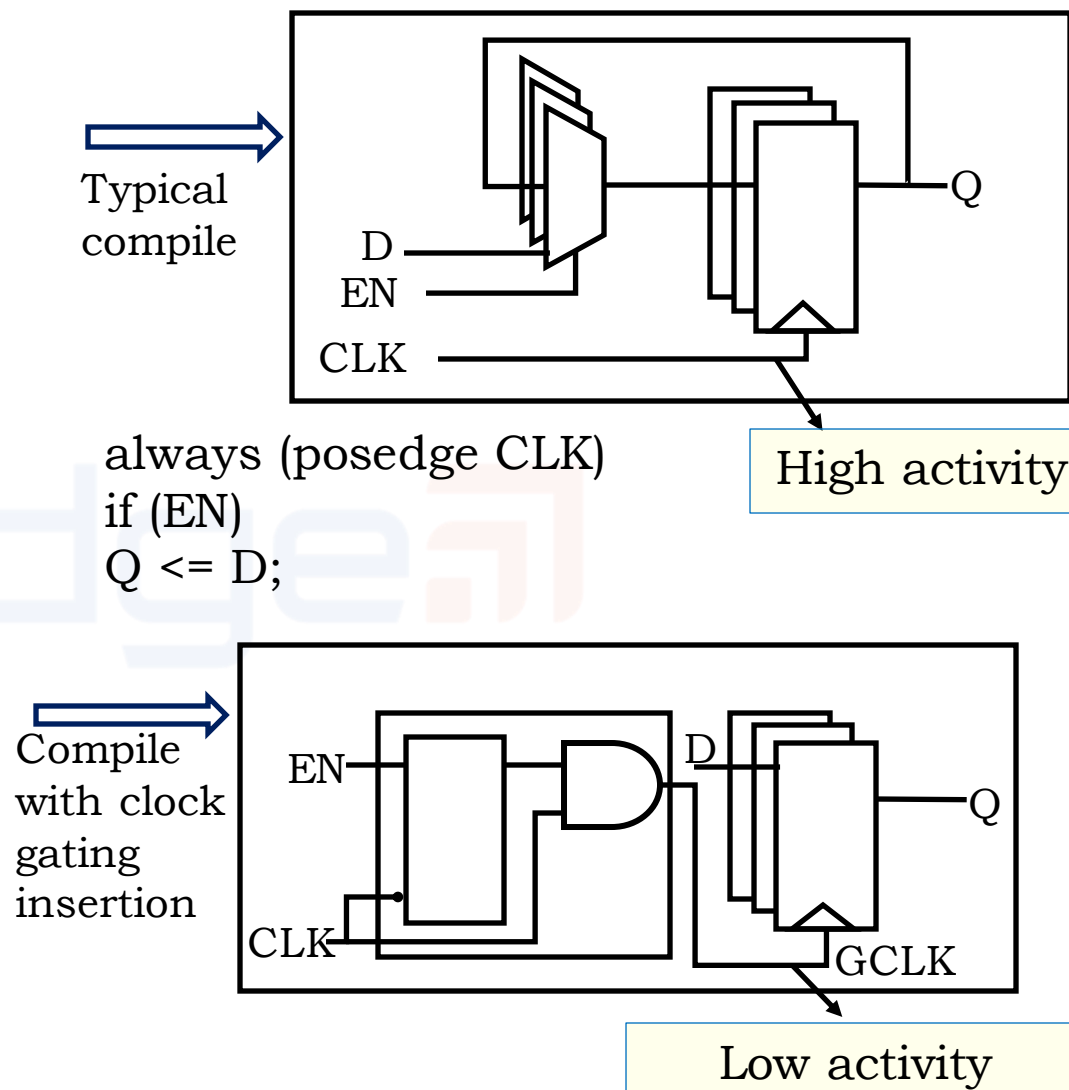


Some of the most commonly used low power methods are :-

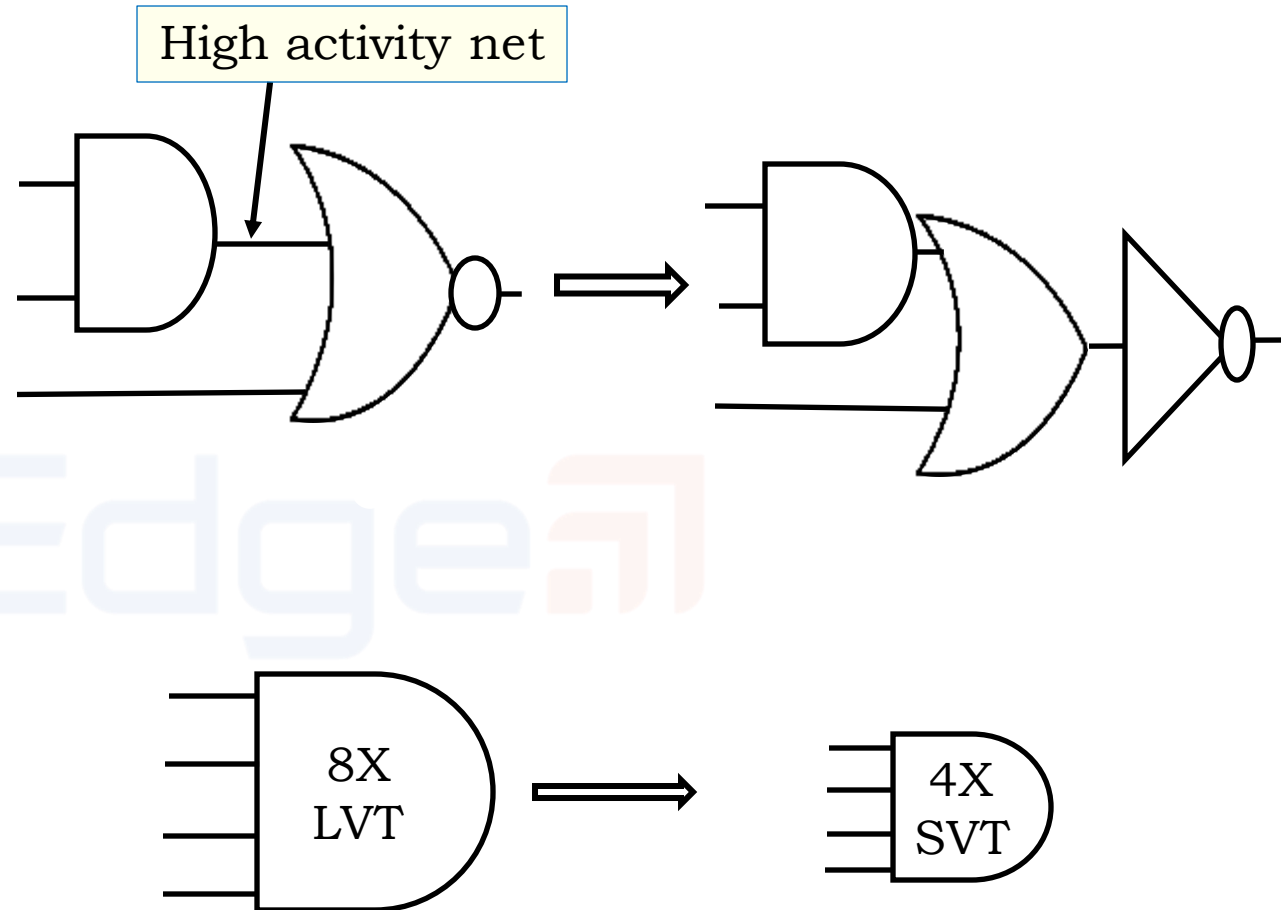
- Clock Gating
- Gate level Power Optimization
- Multi V_{dd}
- Multi V_t
- Power Gating



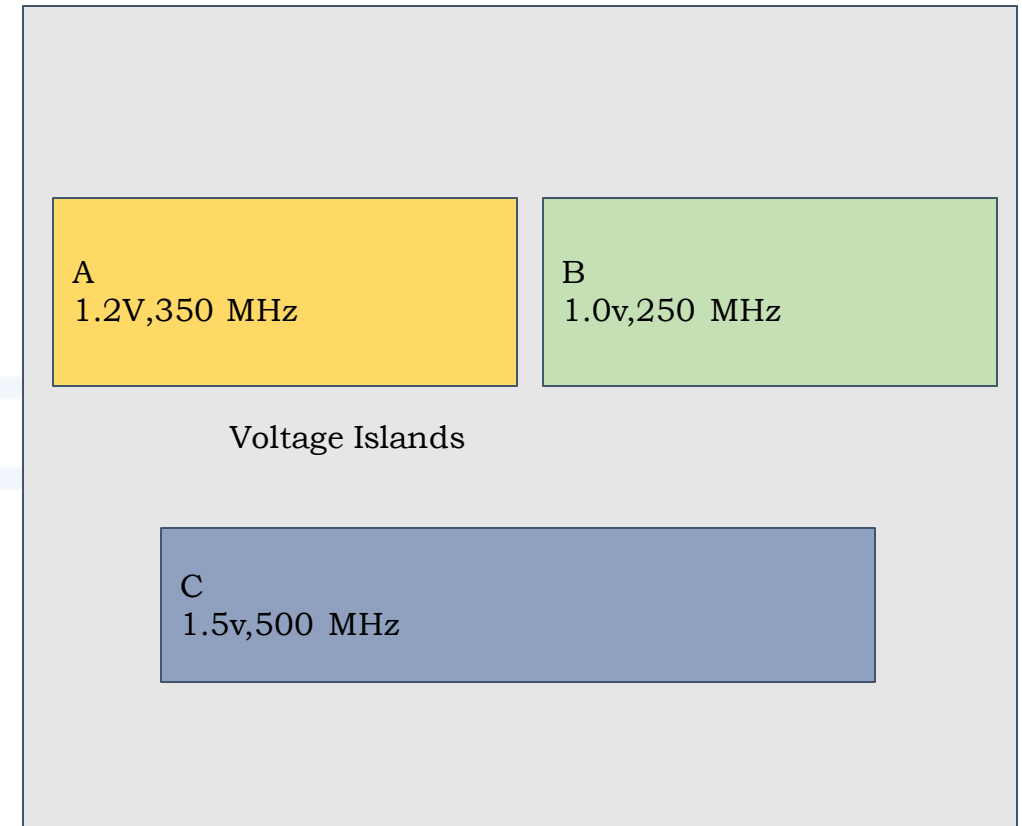
- Clock Network Switching alone contributes significant fraction of dynamic power.
- More than 50 % dynamic power spent in the switching of clock buffers, due to high toggle rate.
- Turning off the clocks – when not required – can have significant saving in the dynamic power.



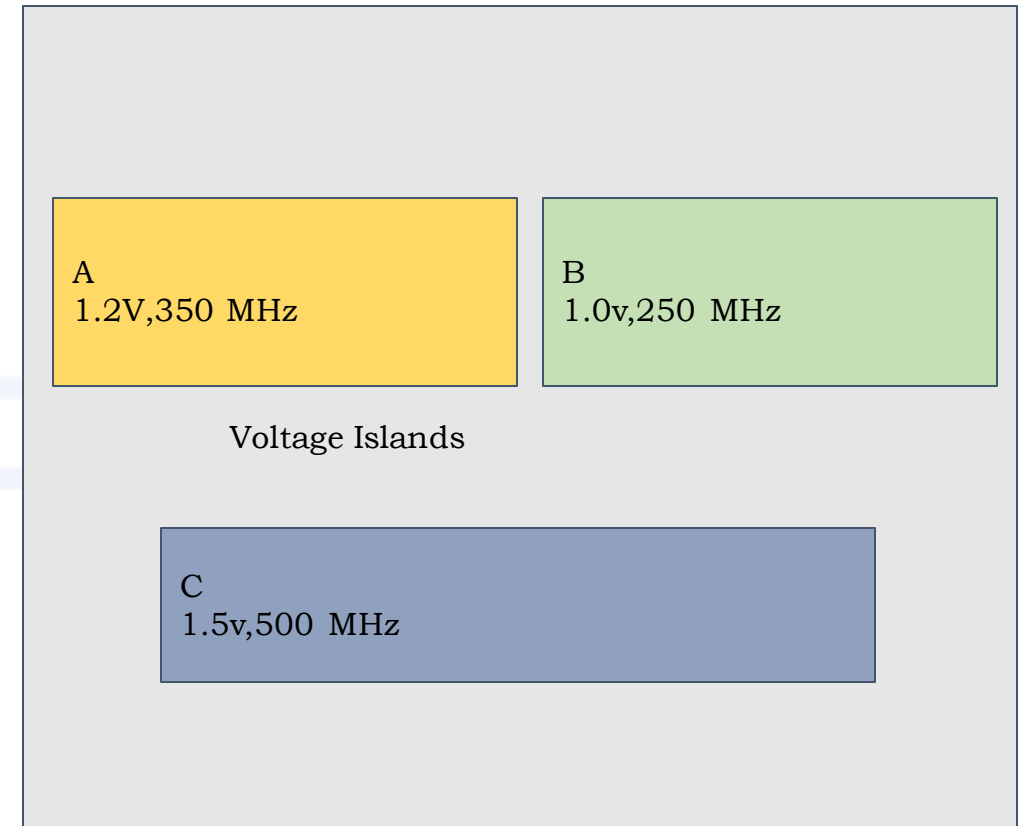
- Logic Restructuring –
 - ✓ AND Gate output has high activity.
 - ✓ Since it's followed by NOR gate, hence it can map the two cells to AND-OR-INVERT gate, so high activity nets become internal to the cell.
- Cell sizing
- Vt Swapping
- Buffer Insertion



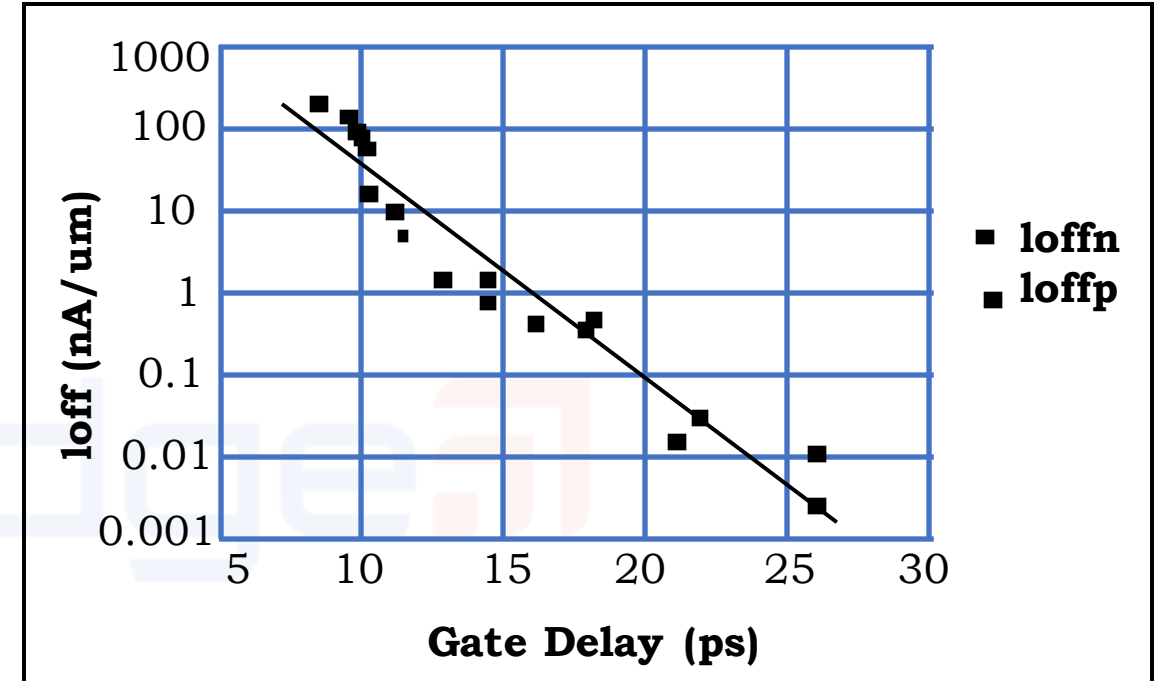
- Multi V_{dd} techniques operate various blocks at various voltages. Operating at a lower voltage reduces power consumption, but at the expense of speed.
- P_{dym} proportional to V_{dd}^2 hence lowering the power supply can result significant saving on the dynamic power .



- Lowering V_{dd} can impact block performance since gate delay increases.
- Out of 3 blocks Block C is required to run at high voltage since its need to operate at higher Frequency.
- Rest of the Blocks can run at lower voltage.

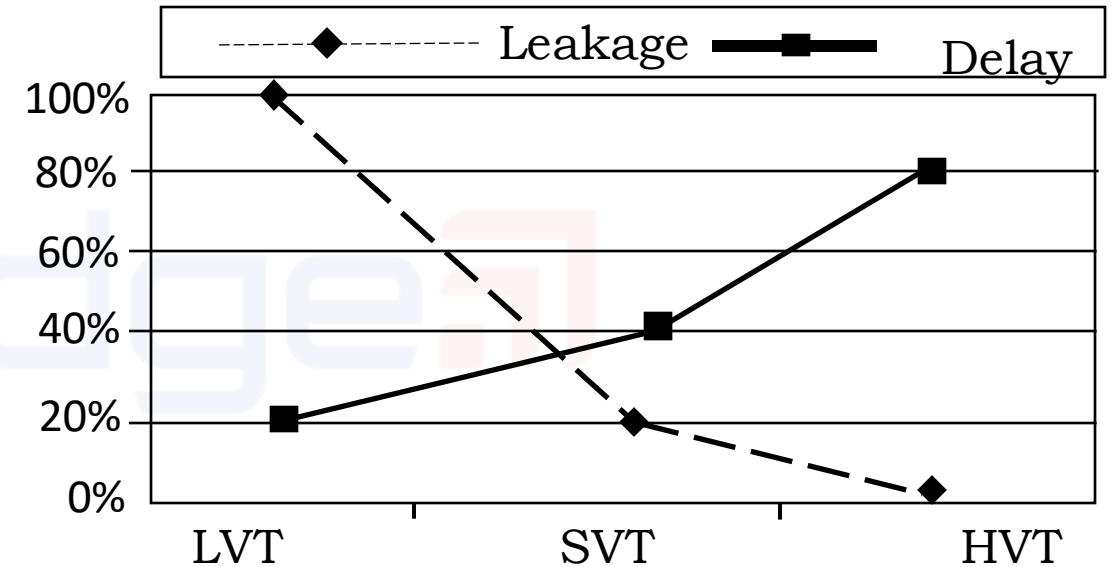


- Sub-threshold leakage depends exponentially on V_t .
- $I_{\text{sub}} \propto e^{\left(\frac{V_{\text{GS}} - V_t}{nV_t}\right)}$.
- Many library offers two(or) three variants Low V_t Standard V_t , High V_t .

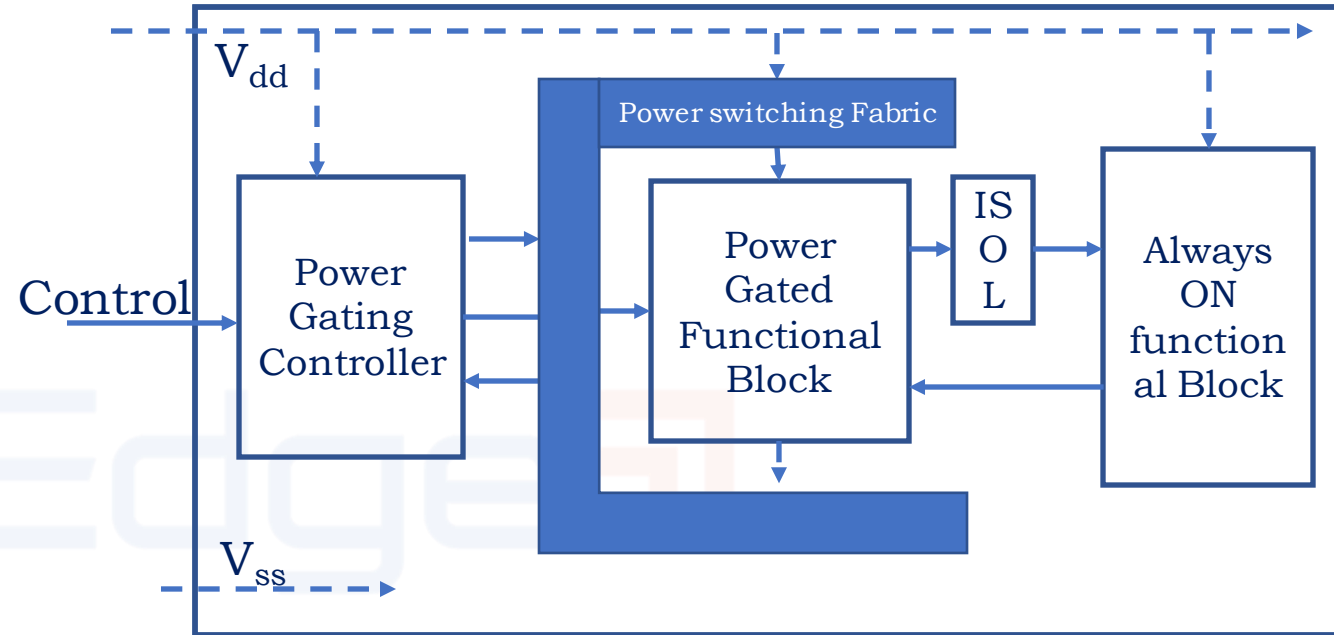


Delay Vs. Leakage at 90nm

- High $V_t \rightarrow$ lower leakage power.
- Quite often multi V_t synthesis is done.
- Power Recovery- In the non-timing critical path, the low V_t cells are swapped with high V_t cells.

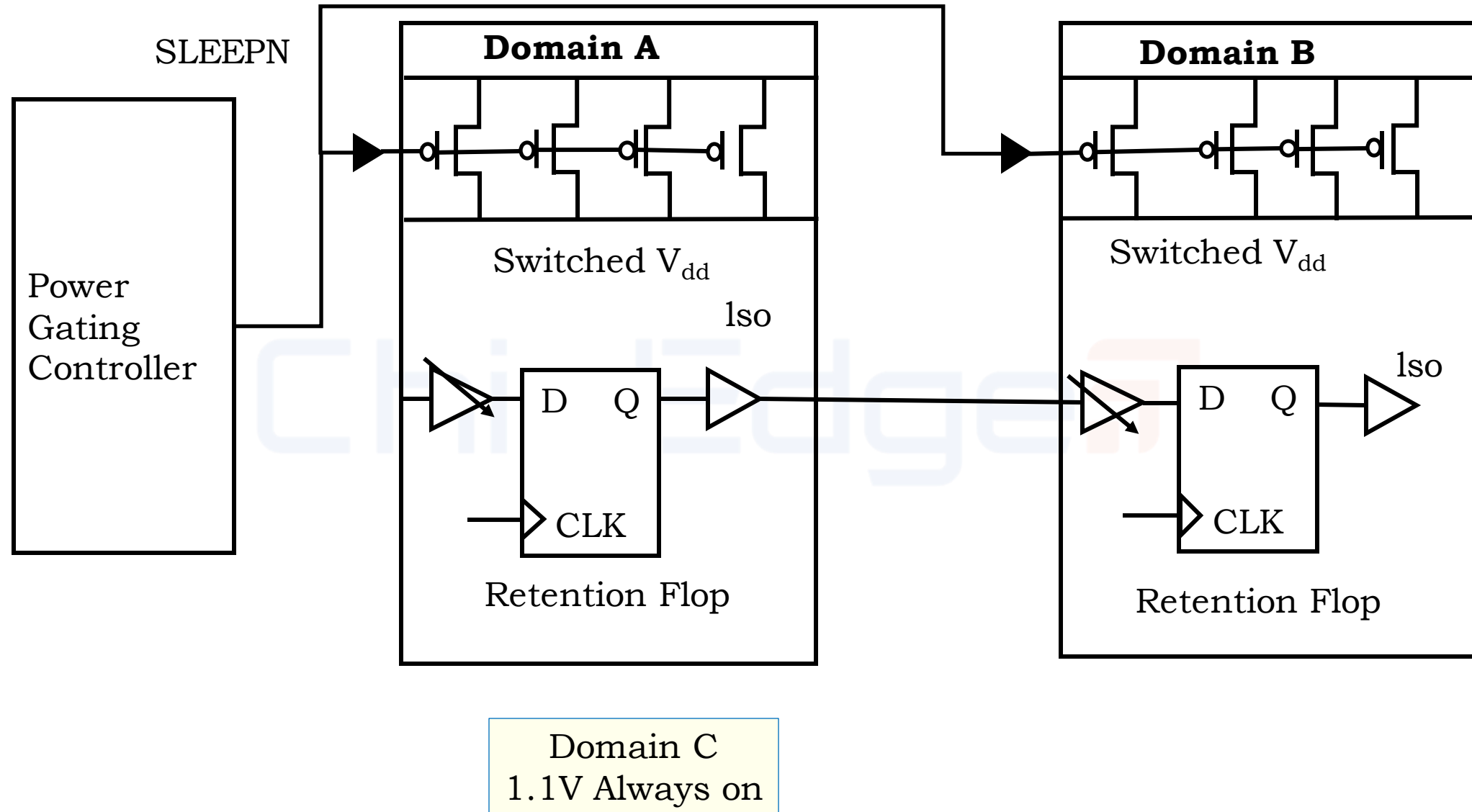


- Two Modes
 - ✓ Active Mode – All the blocks are powered up.
 - ✓ Sleep Mode – Selective blocks are powered down by shutting off the power supply to those blocks.
- Either V_{dd} or V_{SS} can be cut-off to the blocks.
- In the e.g. V_{dd} is switched, V_{SS} is provided directly to the chip.

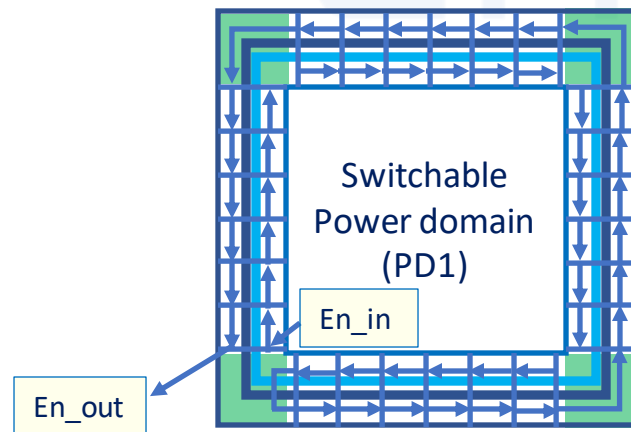


Block Diagram of SoC using power gating

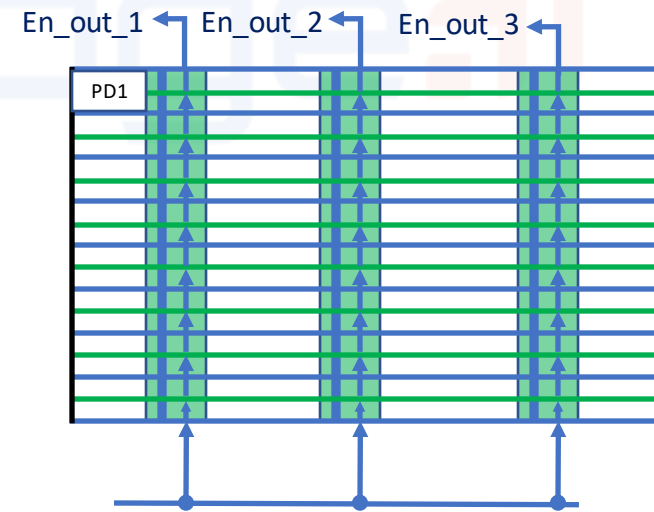
Power Gating Using Header Cells



- The Header/Footer power gating is implemented using special std cells called as power switch cells.
- Power switch cells must be inserted for on-chip power shut-off. Power switch cells can be placed in a ring or a column fashion.

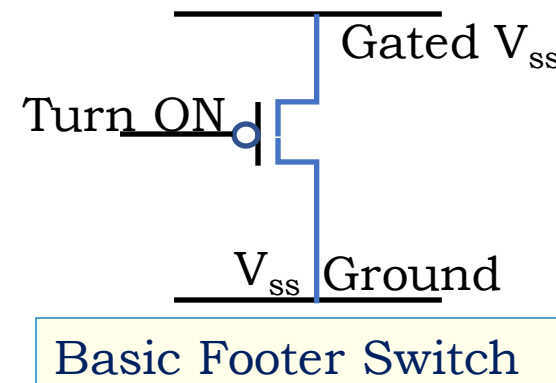
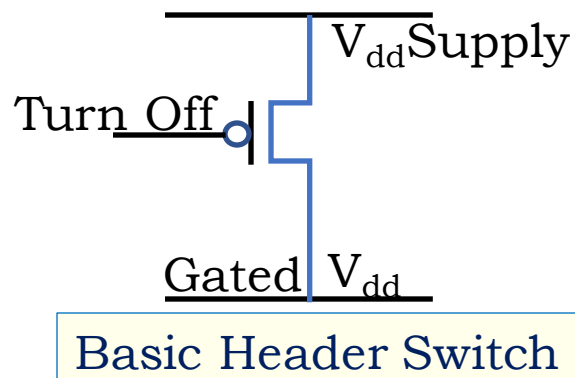


Ring Style Power switch Insertion



Column Style Power switch Insertion

- Whether to switch V_{DD} (with header cells) or to switch V_{SS} (with footer cells) or both.
- Using both may have significant IR drop due to two power switches in series.
- In general power gating is realized by switching V_{DD} since V_{SS} is generally common for chip.



Overview



Power Component



Low Power Methods



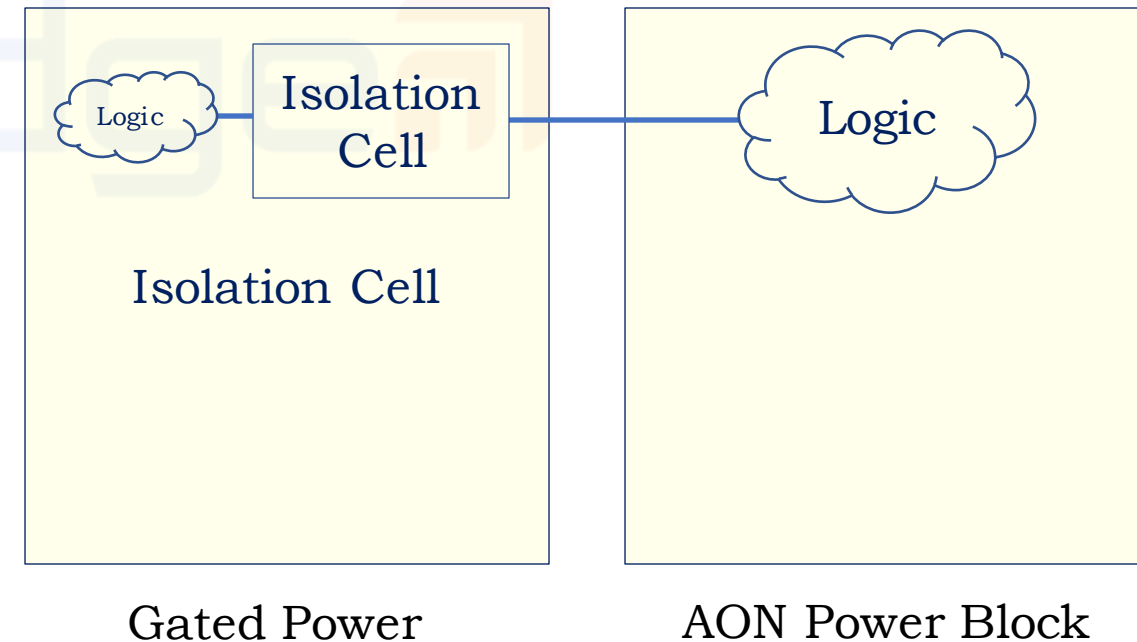
Low Power cells

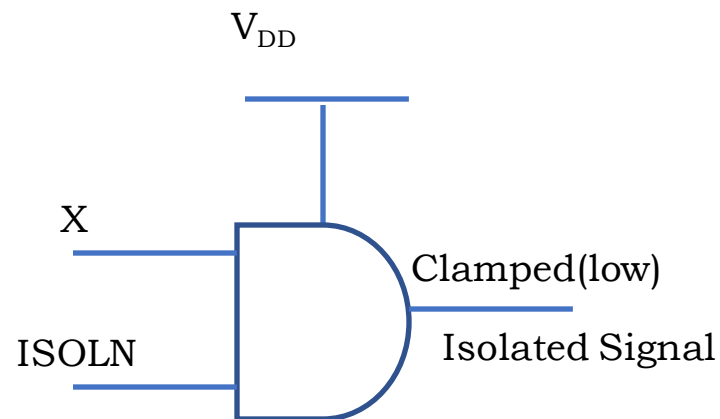
This section explains about various low power cells.

UPF

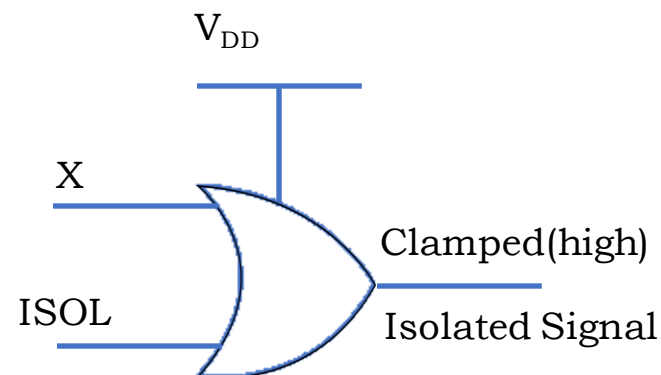


- ISOLATION cells
 - To clamp the output of powered down blocks interfacing with other 'always-on' blocks to a specific, legal value.
 - Two basic type of ISO cells
 - ✓ Clamp the signal to “0” (ISOWLOW)
 - ✓ Clamp the signal to “1” (ISOHIGH)





AND-style ISOLOW

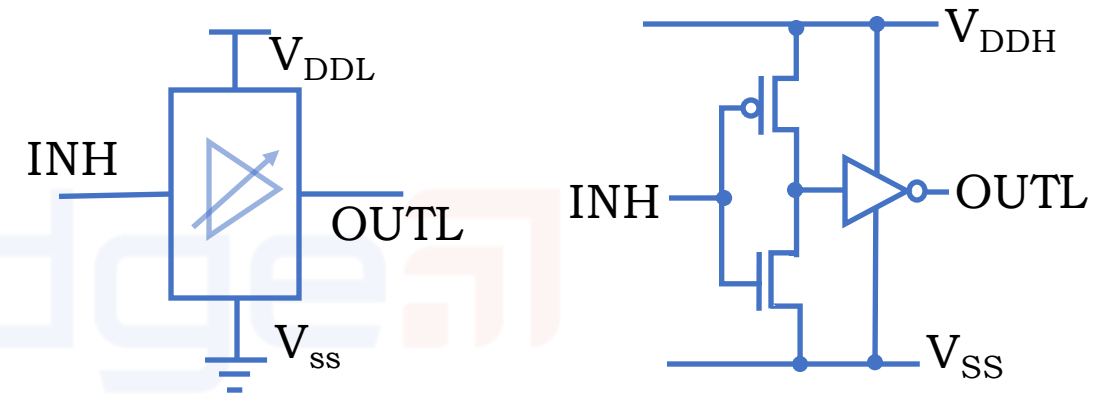


OR-style ISOHIGH

- Output Isolations using normal AND/OR gate.
- It requires that V_{dd} be connected to AON supply.

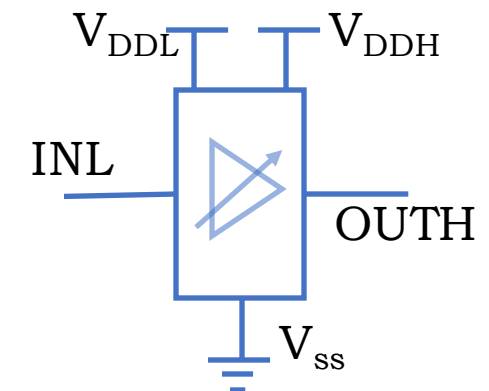
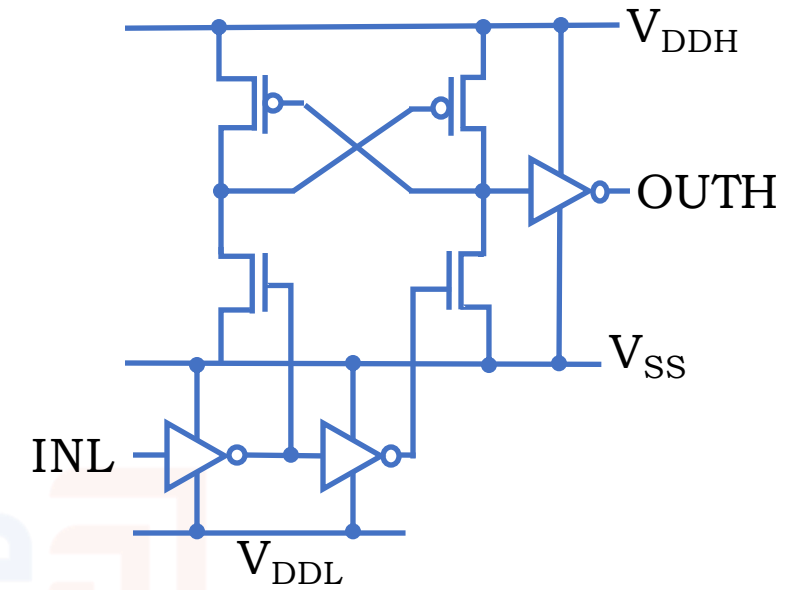
- Signal that crosses the blocks which operates at different voltage levels requires level shifters.
- Signal that crosses two power domain which operates at 1V & 5V clearly requires level shifters in order to match the voltage swing of respective power domain.
- Signal that crosses two power domain which operates at 0.9V & 1.2V also requires level shifters to avoid Data corruption, crowbar current.
- Standard cell libraries are characterized assuming a voltage at V_{DD} Pin and input signal pin of a cell. If the signal is of a different voltage, this can cause incorrect timing analysis.

- For safe timing closure
- It requires single supply source
- As seen in the figure, High-To-Low level shifters introduces only buffer delay.



High-To-Low Level Shifter

- It requires two supply – One corresponding to Low Voltage & another corresponding to High Voltage.
- It introduces significant delays in the timing paths.



- Retention flops cells are special flops with multiple power supply.
- They are typically used as a shadow register to retain its value even if the block in which its residing is shut-down.
- In a nut-shell, "When design blocks are switched off for sleep mode, data in all flip-flops contained within the block will be lost. If the designer desires to retain state, retention flip-flops must be used".
- Example: Sleep / Hibernate mode to wake up (laptop).

Overview



Power Component



Low Power Methods



Low Power cells



UPF

This section explains about the upf file.

- Unified Power Format (UPF) is an industry-wide power format specification to implement low power techniques in a power-aware design flow.
- UPF is intended to ease the job of specifying, simulating, and verifying IC designs that have a number of power states and power islands.
- UPF is designed to reflect the power intent of a design.

UPF file help describe power intent such as:

- Which power rails to be routed to individual blocks.
- When blocks are expected to be powered up or shut down.
- Isolation cell requirements
- Level shifters
- Power switches
- Associated power domain for pins of a block

```
## CREATE POWER DOMAINS
#####
create_power_domain TOP
create_power_domain GPRS -elements GPRS

## TOPLEVEL CONNECTIONS
#####
# VDD
create_supply_port VDD
create_supply_net VDD -domain TOP
connect_supply_net VDD -ports VDD

# VSS
create_supply_port VSS
create_supply_net VSS -domain TOP
create_supply_net VSS -domain GPRS -reuse
connect_supply_net VSS -ports VSS

create_supply_net VDD -domain GPRS -reuse

# VDDG
create_supply_port VDDGS
create_supply_net VDDGS -domain TOP
create_supply_net VDDGS -domain GPRS -reuse
connect_supply_net VDDGS -ports VDDGS

# VDDX

# VDDY

## PRIMARY POWER NETS
#####
set_domain_supply_net TOP -primary_power_net VDD -primary_ground_net VSS
set_domain_supply_net GPRS -primary_power_net VDDGS -primary_ground_net VSS

## GPRS SETUP
#####
# SWITCH

#set_isolation gprs_iso_out \
# -domain GPRS \
```



- Overview
- Power Component
- Dynamic Power
- Leakage Power
- Low Power Methods
- Clock Gating
- Gate level Power Optimization
- Multi V_{dd}
- Multi V_t
- Power Gating
- Low Power cells
- Level Shifters
- Retention Flops
- UPF